
ANALYST TO ARCHITECT · PART 3

Advanced Analytics & Analytics Engineering

Advanced SQL and data modelling, the command line, Python as software, dbt, the computer science behind fast code, experiments, and causal inference.

Contents

Part 3 — Advanced Analytics & Analytics Engineering	1
Chapter 28. Advanced SQL, Performance & Data Modeling	2
28.1 Three practice databases	5
28.2 Recursive CTEs: walking trees of any depth	8
28.3 Advanced window frames	25
28.4 How a database finds rows	32
28.5 Reading query plans with EXPLAIN	34
28.6 Indexes in practice	38
28.7 Normalization, worked properly	45
28.8 The grain discipline	51
28.9 Dimensional modeling: facts, dimensions, and the star schema	53
28.10 Slowly changing dimensions	65
28.11 Denormalization trade-offs	71
28.12 Schema migrations: changing a live database safely	73
28.13 The same SQL in MySQL	80
Chapter 34. The Command Line, Linux & Networking Basics	106
34.0 Where Chapter 26 left you	108
34.1 Setting up for this chapter	109
34.2 Pipes, redirection, and exit codes	111
34.3 Looking inside files, and finding them	113
34.4 Searching and summarizing text	115
34.5 Permissions: who can read, write, and run	119
34.6 Environment variables and PATH	121
34.7 Shell scripts	123
34.8 SSH: working on another machine	127
34.9 Networking in plain English	129
Chapter 29. Python as Software, Not Scripts	146
29.1 The script that works until it doesn't	148
29.2 Functions and modules: one job each	150
29.3 Project structure and configuration	157
29.4 Environments and lockfiles	161
29.5 Classes, when they help	165
29.6 Type hints	172
29.7 Testing with pytest	175
29.8 Errors and logging	184
29.9 Robust API clients	190
29.10 Code review	199
Chapter 32. Analytics Engineering with dbt	213
32.1 What analytics engineering is	215
32.2 Your first project	216
32.3 Models, sources, and ref	221
32.4 Layers: staging, intermediate, marts	225
32.5 Building the star schema as models	227
32.6 Materializations: view, table, incremental, ephemeral	233
32.7 Tests: the part that earns the trust	235
32.8 Documentation and lineage	238
32.9 Snapshots: history dbt keeps for you	239
32.10 Incremental models	240

32.11 Macros: write a rule once	242
32.12 Linting and continuous integration	244
32.13 What dbt is not, and the semantic layer	248
Chapter 33. The Computer Science You Actually Need	260
33.1 Big-O: how work grows	262
33.2 The four structures you'll use every day	265
33.3 Hashing, and why dictionaries are instant	268
33.4 Stacks and queues	269
33.5 Recursion	271
33.6 Trees and graphs	273
33.7 Searching and sorting	276
33.8 Memoization, dynamic programming, and greedy	279
33.9 Memory, and the other costs	282
33.10 What coding interviews actually test	284
Chapter 30. Inference & Experiments	299
30.1 The data, and the question	301
30.2 Standard error and confidence intervals	304
30.3 Comparing two groups: the t-test	308
30.4 Effect size: how big, not just whether	311
30.5 Categorical outcomes: proportions and chi-square	312
30.6 More than two groups: ANOVA	315
30.7 Power and sample size	319
30.8 Designing an experiment	323
30.9 Riverstone's website test, end to end	324
30.10 Four ways to fool yourself, demonstrated	330
30.11 Regression for inference	334
30.12 Writing the result up	339
Chapter 31. Causal Inference Without Experiments	353
31.0 Logs in ten minutes	355
31.1 The data, and the three true answers	357
31.2 Why the two obvious comparisons mislead	360
31.3 Difference-in-differences	361
31.4 The assumption: parallel trends	364
31.5 Synthetic control	368
31.6 Matching and propensity scores	372
31.7 Regression discontinuity: let the rule do the randomizing	379
31.8 Instrumental variables: the idea	383
31.9 How much to trust each answer	384

Part 3 — Advanced Analytics & Analytics Engineering

Part 3 takes the analyst's tools further: advanced SQL and data modelling, the command line, Python as software, analytics engineering with dbt, the computer science behind fast code, experiments, and causal inference.

Read the chapters in this order:

Chapter	Time needed
28. Advanced SQL, Performance & Data Modeling	22–30 hours
34. The Command Line, Linux & Networking Basics	9–12 hours
29. Python as Software, Not Scripts	20–24 hours
32. Analytics Engineering with dbt	18–22 hours
33. The Computer Science You Actually Need	14–18 hours
30. Inference & Experiments	18–22 hours
31. Causal Inference Without Experiments	15–18 hours

In total, allow 116–146 hours, including the exercises and projects.

Chapter 28. Advanced SQL, Performance & Data Modeling

Chapter at a glance

You will learn to: walk org charts and bills of materials of any depth with recursive CTEs · protect a recursive query against loops · choose between `ROWS`, `RANGE`, and `GROUPS` frames, and use `EXCLUDE` and named windows · read a query plan with `EXPLAIN` and `EXPLAIN ANALYZE` · decide which indexes to create (single-column, composite, covering, partial) and which to avoid · recognize queries that can't use an index, and rewrite them · normalize a messy table to first, second, and third normal form · declare and test the grain of any table · design a star schema with facts, dimensions, and a date dimension · build slowly changing dimensions of types 1, 2, and 3 · decide when denormalizing is worth it · change a production database safely with versioned migration scripts.

Before you start: Chapter 12 (joins, keys, `CREATE TABLE`, `ALTER TABLE`, transactions, the fan-out trap) and Chapter 13 (CTEs, views, window functions, the `sales_lines` view). The chapter also leans on Chapter 4 (the median), Chapters 15 and 21 (percentiles in SQL, sections 15.5 and 21.3), Chapter 16 (section 16.3, the star schema you built in Power BI), Chapter 17 (running a Python script), and Chapter 26 (section 26.0, the terminal).

Time needed: 22–30 hours of reading and practice, including the project, spread over four to five weeks.

Tools: PostgreSQL 15 or later (every query was run on PostgreSQL 16) and DBeaver, as in Chapter 12, plus PostgreSQL's command-line client, `psql`, which you installed with the server in section 12.3. MySQL works for most of the chapter; section 28.13 shows what changes.

Practice data: the one-year database `riverstone_2025` with three new companion tables, a large generated database `riverstone_perf` (1.9 million order lines) that you build in section 28.1, and the `riverstone_lab` database you build yourself. Every query was run, and every result shown is the real output. Timings were measured on the author's test machine and are labeled as such.

In this chapter

START	Why this matters · In plain English	3
LEARN	28.1 Three practice databases	5
	28.2 Recursive CTEs: walking trees of any depth	8
	28.3 Advanced window frames	25
	28.4 How a database finds rows	32
	28.5 Reading query plans with EXPLAIN	34
	28.6 Indexes in practice	38
	28.7 Normalization, worked properly	45
	28.8 The grain discipline	51
	28.9 Dimensional modeling: facts, dimensions, and the star schema	53
	28.10 Slowly changing dimensions	65
	28.11 Denormalization trade-offs	71
	28.12 Schema migrations: changing a live database safely	73
	28.13 The same SQL in MySQL	80
APPLY	Common mistakes · In the real world · Project	85
REVIEW	Recap · Key terms · Check yourself	90
PRACTISE	Exercises	91
	Answers	93
NEXT	Where this leads	104

Plan your sittings. This is the longest chapter in Part 3: six topics that are each a chapter elsewhere. Take it in six sittings, and stop at the end of one rather than in the middle:

1. Sections 28.1 (the one-year tables) to 28.3: recursive queries and advanced windows, 5–6 hours.
2. Section 28.1's box "Build riverstone_perf", then sections 28.4 to 28.6: plans and indexes, 5–6 hours.
3. Sections 28.7 and 28.8: normalization and grain, about 3 hours.
4. Sections 28.9 to 28.11: the star schema, slowly changing dimensions, and denormalization, 5–6 hours.
5. Sections 28.12 and 28.13: migrations and MySQL, about 3 hours.
6. The project, 4–6 hours.

Each sitting ends at a line marked **Good place to stop**.

START

Why this matters

The queries in Chapters 12 and 13 answer questions. This chapter is about what happens when those queries become part of how a company runs:

- "Show me everyone who reports to the Head of Production, however many levels down." A plain join handles one level. Real org charts, product structures, and account hierarchies have as many levels as they like.

- *"The dashboard took two seconds last year. Now it takes forty."* The query didn't change. The data grew, and nobody looked at how the database was finding the rows.
- *"Why does the sales report show Sharma Hardware in two cities?"* Somebody stored the same fact in two places, and updated only one of them.
- *"Revenue by segment for last year changed, and nobody touched last year's orders."* A customer was reclassified, and the report quietly rewrote history.
- *"Can you add a column to the orders table this afternoon?"* On a live system, with reports and applications reading that table, the honest answer is: yes, with a reviewed script, in the right order, and a way back.

These are the problems that separate someone who writes SQL from someone who is trusted with a company's data. They are also the topics that turn up in interviews for senior analyst, analytics engineer, and data engineer roles: *"How would you speed up this query?"*, *"What is the grain of this table?"*, *"Explain a type 2 slowly changing dimension."* You'll be able to answer all three with examples you've built yourself.

In plain English

A database is a large library, and your query is a request at the front desk.

Some requests are about **family trees**: *"find this book, then every book that cites it, then every book that cites those."* The librarian can't answer in one trip. She fetches the first shelf, reads it, and goes back for the next, until a trip brings back nothing new. That's a **recursive query**.

For every request, the librarian makes a **plan**. If you ask for one author's books, she can walk every aisle and look at every spine (a **scan**), or she can use the **catalog cards**, sorted by author, which tell her exactly which shelves to visit (an **index**). The catalog makes finding faster, but every new book now means writing a card too, so a library with a catalog for every possible question would spend all day filing cards. You can ask her to **explain her plan** before she sets off.

How the books are organized decides how hard the library is to keep correct. If each book's publisher address is written inside every book, a publisher that moves means correcting thousands of books. So you keep a publisher list and write only the publisher's name in each book. That's **normalization**: every fact in one place.

The reading room upstairs is organized differently. Readers there want to count and compare, not edit, so the librarian keeps a big **ledger of loans** (one line per loan: a **fact**) surrounded by small, friendly lookup lists: dates, readers, books (the **dimensions**). When a reader moves house, the lists keep the old address for old loans and the new address for new ones, so last year's report still describes last year. That's a **slowly changing dimension**.

And when the library wants a new shelf layout, it doesn't move books at random on a busy afternoon. It writes a numbered plan, gets it checked, and carries it out in order, noting each step in a logbook. That's a **migration**.

Keep the library in mind. Recursive queries, plans, indexes, normalization, dimensional models, and migrations are all in this chapter, and each one is a way of keeping a very large library fast and correct.

LEARN

28.1 Three practice databases

This chapter uses three databases. Each section says which one it uses.

The one-year database (`riverstone_2025`) is Chapter 13's: the 24 key accounts, 175 orders in 2025, monthly targets, and leads, with the `sales_lines` view. (It is one slice of Riverstone, not the whole company: the key accounts the Chapter 13 team looks after.) For this chapter you add three tables to it by running `ch28_2025_addons.sql` from the companion files (`ch28_2025_addons_mysql.sql` for MySQL):

Table	One row is	Where it comes from	Used in
<code>staff</code>	one person on Riverstone's payroll, with their manager	the HR system (HRMS)	section 28.2
<code>parts and bom_lines</code>	a part (product, assembly, or bought material); one parent-and-child link in a product's structure	production	section 28.2
<code>customer_changes</code>	one change to a customer's city or segment during 2025	the ERP's audit log	sections 28.9–28.10

The `staff` table is Riverstone's reporting lines, from the managing director, Arvind Kapoor, down to the machine operators at the Taloja and Chakan plants: 35 people, including the key accounts sales team and the three regional sales managers. The five sales people you know from the `employees` table are in it too, with their `employee_id` filled in, so you can connect the two systems. (In the ERP's `employees` table Anita Rao has no manager, because the ERP only knows the sales team. In the HR system she reports to the managing director. Both are true; they're different systems with different scopes.)

The large database (`riverstone_perf`) exists to make performance visible. Indexes don't matter on 175 orders: every query is instant. So the companion script `generate_riverstone_perf.py` builds three years of Riverstone-like sales (2023 to 2025) with the same tables and columns as `riverstone_2025`:

Table	Rows
<code>customers</code>	5,000
<code>products</code>	8
<code>employees</code>	5
<code>orders</code>	714,285
<code>order_items</code>	1,926,847

It uses a fixed seed (28), so your data is identical to the book's. If you have the book's full three-year dataset loaded, every command in sections 28.4 to 28.6 works on that instead; only the numbers change. The data takes about 200 MB in PostgreSQL. Apart from primary keys, the tables start with **no indexes**, because adding them is the lesson. You build it at the start of sitting 2, with the box below; skip it until then.

The lab database (`riverstone_lab`) is a scratch database you create and change freely, as in section 12.13. Sections 28.7, 28.10, and 28.12 build tables in it step by step. If you still have Chapter 12's lab, the first lab step drops and recreates it.

Watch out: timings are measurements, not facts. Every timing in this chapter was measured on the author's test machine: four virtual processors and 16 GB of memory, running PostgreSQL 16 and MySQL 8.0.46 on Ubuntu 24.04, with other work running alongside. Each query was run seven times and the median is reported. Your laptop will give different numbers. What should match is the *plan* the database chooses and the *ratio* between a slow and a fast version. The companion script `ch28_perf.py` reruns every experiment on your machine and writes its plans and timings to `ch28_perf_log.txt`.

Build `riverstone_perf` (about 20 minutes)

This is the chapter's only setup that needs the terminal. You need three things from earlier chapters: a terminal open in a folder (section 26.0), Python (Chapter 17), and your PostgreSQL password (section 12.3).

Step 1. Create the empty database. In DBeaver, on your usual PostgreSQL connection, run:

```
CREATE DATABASE riverstone_perf;
```

Step 2. Generate the data. In the terminal, move into the chapter's companion folder and run the generator (type `python` instead of `python3` on Windows, as in Chapter 17). It takes about half a minute and prints one line:

```
# terminal, in the companion files folder
$ cd ch28
$ python3 generate_riverstone_perf.py
customers 5,000 orders 714,285 order lines 1,926,847 (seed 28)
```

It has written a folder `perf_data` next to the script: five CSV files (about 80 MB) and two load scripts, `load_postgresql.sql` and `load_mysql.sql`.

Step 3. Find `psql`. The load script uses `\copy`, a command only PostgreSQL's command-line client understands. The client was installed with the server in section 12.3 (on Windows, the *Command Line Tools* component). Check that the terminal can find it:

```
# terminal
$ psql --version
psql (PostgreSQL) 16.13 (Ubuntu 16.13-0ubuntu0.24.04.1)
```

Your version line will differ. If the terminal says *command not found* instead:

- **Windows:** type the full path in place of `psql`, in quotes, for example `"C:\Program Files\PostgreSQL\16\bin\psql.exe" --version` (change 16 to your version). Chapter 34, section 34.6, explains `PATH`, the list of folders the terminal searches when you type a short name like `psql`.
- **macOS:** with Postgres.app, use `/Applications/Postgres.app/Contents/Versions/latest/bin/psql`; with Homebrew, `brew install libpq` installs the client.
- **Linux:** `sudo apt install postgresql-client`.

Step 4. Load the data. Still in the `ch28` folder:

```
# terminal
$ psql -U postgres -d riverstone_perf -f perf_data/load_postgresql.sql
Password for user postgres:
psql:perf_data/load_postgresql.sql:2: NOTICE: table "order_items" does not exist, skipping
psql:perf_data/load_postgresql.sql:2: NOTICE: table "orders" does not exist, skipping
psql:perf_data/load_postgresql.sql:2: NOTICE: table "employees" does not exist, skipping
psql:perf_data/load_postgresql.sql:2: NOTICE: table "products" does not exist, skipping
psql:perf_data/load_postgresql.sql:2: NOTICE: table "customers" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
COPY 5000
COPY 8
COPY 5
COPY 714285
COPY 1926847
ANALYZE
```

What each part does:

- `-U postgres` connects as the user `postgres`, the administrator account from section 12.3. `psql` asks for its password; nothing appears on the screen while you type it, which is normal.
- `-d riverstone_perf` chooses the database, the one you created in step 1.
- `-f perf_data/load_postgresql.sql` runs every statement in that file, in order, and prints what each one did.
- The five **NOTICE** lines are harmless: the script starts by dropping the tables in case you run it twice, and on the first run there's nothing to drop. **COPY 714285** means 714,285 rows were read from `orders.csv`. The load takes under a minute; the final **ANALYZE** collects the statistics section 28.5 explains.

Step 5. One setting. Back in DBeaver, run:

```
ALTER DATABASE riverstone_perf SET max_parallel_workers_per_gather = 0;
```

It turns off *parallel query*, which PostgreSQL uses on machines with several processor cores. Parallel plans are correct but longer to read, and they make timings jump about. The book's plans were all made with this setting. It applies to connections opened *after* you run it, so disconnect and reconnect

DBeaver's `riverstone_perf` connection. You can reset it later with `ALTER DATABASE riverstone_perf RESET max_parallel_workers_per_gather;`

Step 6. Check it. Make a DBeaver connection to `riverstone_perf` (as in section 12.3, with *Database* set to `riverstone_perf`) and run:

```
SELECT COUNT(*) FROM order_items;
```

```
count
-----
1926847
(1 row)
```

If you see 1,926,847, you're ready for section 28.4.

No terminal access? DBeaver can do the load instead. Run the five `CREATE TABLE` statements from `load_postgresql.sql` in DBeaver, then right-click each table → *Import Data* → *CSV*, and load the files in this order, so every foreign key finds its row: `customers.csv`, `products.csv`, `employees.csv`, `orders.csv`, `order_items.csv`. The files have no header row, so untick *Header*. It's slower, but the result is the same.

Using MySQL? The same steps, with MySQL's client. The server must allow file loading once: as the root user, run `SET GLOBAL local_infile = 1;` in DBeaver. Then, in the `ch28` folder, `mysql -u root -p --local-infile=1 < perf_data/load_mysql.sql`. Here `-u root` is the user, `-p` makes the client ask for the password, `--local-infile=1` lets the client send files from your computer, and `<` feeds the script to the client as its input. The script creates the database itself. Section 28.13 uses it.

28.2 Recursive CTEs: walking trees of any depth

A **recursive CTE** is a CTE that refers to itself. It is how SQL walks data where rows point at other rows of the same table: org charts, product structures, account hierarchies. It can also build a list, such as every month of a year, when no table holds one. This section starts with the smallest recursive query there is, then climbs to Riverstone's org chart and its bills of materials.

Your first recursive query: counting to five

The plan: start with the number 1, and keep adding 1 until you reach 5. In the one-year database (any database works; this query reads no tables):

```
WITH RECURSIVE numbers AS (  
    SELECT 1 AS n          -- anchor: runs once  
    UNION ALL  
    SELECT n + 1          -- recursive part: runs again and again  
    FROM numbers  
    WHERE n < 5           -- stop rule  
)  
SELECT n FROM numbers;
```

```
n  
---  
1  
2  
3  
4  
5  
(5 rows)
```

How it works, line by line:

- **WITH RECURSIVE numbers AS (...)** is a CTE, as in section 13.2, with one new word. **RECURSIVE** tells the database that a CTE in this **WITH** list may refer to itself. It is written once, straight after **WITH**, and covers the whole list, even when some of the CTEs in it aren't recursive (you'll see that in "Guarding against loops").
- **SELECT 1 AS n** is the **anchor**. It runs once and produces the starting row. Its column names and types fix the CTE's columns: there is one column, **n**, and it holds whole numbers.
- **UNION ALL** glues each pass's rows onto the result.
- **SELECT n + 1 FROM numbers WHERE n < 5** is the **recursive part**. Inside it, **numbers** does not mean the whole result so far. It means only the rows the **previous pass** produced (the database's name for them is the *working table*). So each pass reads one row and makes the next number.
- **The stop rule:** when a pass produces no rows, the query ends. **WHERE n < 5** is what makes that happen.
- The final **SELECT n FROM numbers** reads everything the passes produced, like any CTE.

Trace it pass by pass:

Pass	Reads (the previous pass's rows)	Produces
anchor	nothing	1
1	1	2
2	2	3
3	3	4
4	4	5
5	5	nothing, because 5 < 5 is false: stop

What happens if you change it? Predict before you run it: what does the query do without `WHERE n < 5`? Nothing ever stops the passes. In PostgreSQL the query keeps going until you cancel it (DBeaver's *Cancel* button, next to the results) or the numbers grow too large for their type. MySQL protects you. It stops any recursive CTE that runs more than 1,000 passes, a limit held in the setting `cte_max_recursion_depth`:

```
WITH RECURSIVE numbers AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1 FROM numbers  
)  
SELECT COUNT(*) FROM numbers;
```

```
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increasing @@cte_max_recursion_depth to a  
larger value.
```

For a legitimate walk longer than 1,000 steps, raise the limit for your session first: `SET SESSION cte_max_recursion_depth = 10000;`. PostgreSQL has no such limit, which is why "Guarding against loops", later in this section, matters there.

UNION or UNION ALL? Plain `UNION` (section 12.12) would remove duplicate rows on every pass. Use `UNION ALL` unless you have a reason not to: it's faster, and it keeps repeats that are real. You'll see one in the bill of materials below, where steel tube legitimately appears twice.

A list of months when there's no calendar table

Chapter 13's Pattern 5 filled in the missing months with a **date spine**, and used the calendar table `calendar_months` for it. Most companies have one; when yours doesn't, a recursive CTE builds the same list. Change the counter to count months: start at 1 January 2025 and add one month per pass until December.

```
WITH RECURSIVE months AS (  
    SELECT DATE '2025-01-01' AS month  
    UNION ALL  
    SELECT month + INTERVAL '1 month'  
    FROM months  
    WHERE month < DATE '2025-12-01'  
)  
SELECT month FROM months;
```

```
ERROR: recursive query "months" column 1 has type date in non-recursive term but type timestamp without time  
zone overall
```

A real error, and a useful one. The anchor's DATE '2025-01-01' is a date, but month + INTERVAL '1 month' gives a timestamp (a date with a time of day, as generate_series did in Pattern 5). The anchor fixes the column's type, so the recursive part must produce the same type. CAST(... AS DATE) (section 12.8) turns each new month back into a date:

```
WITH RECURSIVE months AS (
  SELECT DATE '2025-01-01' AS month
  UNION ALL
  SELECT CAST(month + INTERVAL '1 month' AS DATE)
  FROM months
  WHERE month < DATE '2025-12-01'
)
SELECT COUNT(*) AS months, MIN(month) AS first_month, MAX(month) AS last_month
FROM months;
```

```
months | first_month | last_month
-----+-----+-----
      12 | 2025-01-01 | 2025-12-01
(1 row)
```

Twelve months, the same rows as calendar_months. Pass 11 produces 1 December; pass 12 reads it, finds month < '2025-12-01' false, and produces nothing. To use it, put this months step where Pattern 5 had calendar_months. In MySQL only the step changes spelling (month + INTERVAL 1 MONTH, and no CAST is needed, because MySQL keeps a date plus a month as a date). Here is Pattern 5's Garden Chair query in MySQL with that spine:

```
WITH RECURSIVE months AS (
  SELECT DATE '2025-01-01' AS month          -- the first month
  UNION ALL
  SELECT month + INTERVAL 1 MONTH           -- add one month...
  FROM months
  WHERE month < DATE '2025-12-01'          -- ...until December
),
furniture AS (
  SELECT CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE) AS month,
         SUM(net_revenue) AS revenue
  FROM sales_lines
  WHERE category = 'Furniture'
  GROUP BY CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE)
),
filled AS (
  SELECT m.month,
         COALESCE(f.revenue, 0) AS revenue,
         LAG(COALESCE(f.revenue, 0)) OVER (ORDER BY m.month) AS previous_month
  FROM months AS m
  LEFT JOIN furniture AS f ON f.month = m.month
)
SELECT month, ROUND(revenue, 0) AS revenue, ROUND(previous_month, 0) AS previous_month
FROM filled
WHERE month BETWEEN '2025-02-01' AND '2025-06-01'
ORDER BY month;
```

month	revenue	previous_month
2025-02-01	0	0
2025-03-01	16388	0
2025-04-01	0	16388
2025-05-01	17250	0
2025-06-01	0	17250

The same five rows as Pattern 5. Only the first CTE is recursive, and `WITH RECURSIVE` is still written once, at the start of the list. `DATE_FORMAT(order_date, '%Y-%m-01')` is Chapter 13's MySQL spelling of "the first of the month" (section 13.9). A monthly spine needs 11 passes, far below MySQL's limit; a *daily* spine for three years needs 1,095 and would stop with error 3636 unless you raise `cte_max_recursion_depth` first. For dates you'll use again and again, a permanent calendar table is still the better answer.

Counting and listing months are warm-ups. The real power of recursion is walking **hierarchies**.

The problem

"List everyone at Riverstone with their level in the organization."

Each `staff` row stores only its own manager:

```
SELECT staff_id, staff_name, job_title, manager_id
FROM staff
WHERE staff_name IN ('Arvind Kapoor', 'Harpreet Sethi', 'Ramesh Patil', 'Ajay Kumar', 'Gopal Sahu')
ORDER BY staff_id;
```

staff_id	staff_name	job_title	manager_id
100	Arvind Kapoor	Managing Director	
130	Harpreet Sethi	Head of Production	100
131	Ramesh Patil	Plant Manager, Taloja	130
133	Ajay Kumar	Shift Supervisor	131
136	Gopal Sahu	Machine Operator	133

(5 rows)

A self-join (section 12.10) gets you one level: each person and their manager. Two self-joins get you two levels. But the org chart is five levels deep today and may be six next year, and you can't write a query with a join for every level that might exist. You need a query that keeps going until it runs out of people.

The org chart, level by level

The same shape as the counter. The anchor is the one person with no manager. The recursive part finds everyone whose manager was found on the previous pass, and adds 1 to the level, where the counter added 1 to `n`.

Here is the query:

```
WITH RECURSIVE org AS (  
  -- anchor: the person with no manager  
  SELECT staff_id, staff_name, job_title, manager_id,  
         1 AS level  
  FROM staff  
  WHERE manager_id IS NULL  
  UNION ALL  
  -- recursive part: everyone whose manager is already in org  
  SELECT s.staff_id, s.staff_name, s.job_title, s.manager_id,  
         o.level + 1  
  FROM staff AS s  
  JOIN org AS o ON s.manager_id = o.staff_id  
)  
SELECT level, staff_name, job_title  
FROM org  
ORDER BY level, staff_id;
```

level	staff_name	job_title
1	Arvind Kapoor	Managing Director
2	Anita Rao	Sales Head
2	Suresh Menon	Finance Manager
2	Harpreet Sethi	Head of Production
2	Joseph D'Souza	Purchasing Manager
2	Mahesh Yadav	Warehouse & Dispatch Manager
2	Lakshmi Reddy	HR Manager
2	Zoya Mirza	Marketing Manager
2	Tenzin Dorji	Customer Support Lead
3	Vikram Singh	Sales Manager, Key Accounts
3	Farah Khan	Sales Executive
3	Meera Iyer	Sales Coordinator
3	Arjun Nair	Regional Sales Manager, South
3	Pooja Desai	Regional Sales Manager, West
3	Sandeep Gill	Regional Sales Manager, North
3	Priya Nambiar	Accounts Executive
3	Ramesh Patil	Plant Manager, Taloja
3	Kiran Bhosale	Plant Manager, Chakan
3	Mohan Das	Dispatch Supervisor
4	Neha Kulkarni	Sales Executive
4	Rahul Mehta	Sales Executive
4	Ajay Kumar	Shift Supervisor
4	Farhan Ali	Quality Inspector
4	Swati Joshi	Shift Supervisor
4	Divya Krishnan	Sales Executive
4	Vivek Chandran	Sales Executive
4	Sneha Pillai	Sales Executive
4	Nisha Bhatt	Sales Executive
4	Aditya Verma	Sales Executive
4	Rohit Kamat	Sales Executive
4	Karan Ahuja	Sales Executive
4	Ritu Bansal	Sales Executive
5	Gopal Sahu	Machine Operator
5	Sunita Pawar	Machine Operator
5	Farid Shaikh	Machine Operator

(35 rows)

How it works:

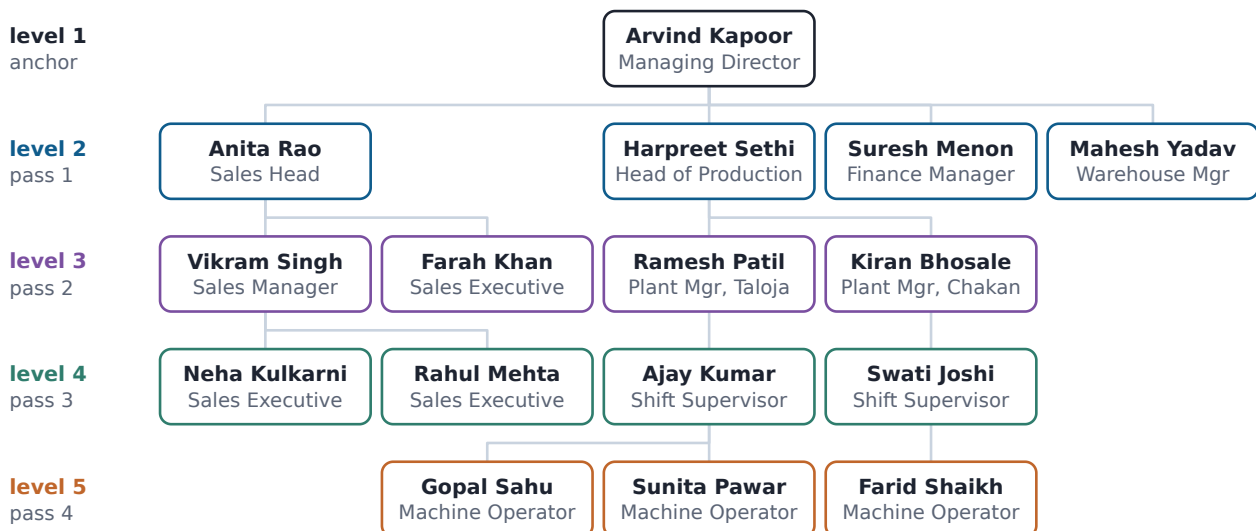
- A recursive CTE has two queries joined by `UNION ALL`. The first is the **anchor**: it runs once and produces the starting rows. Here that's Arvind Kapoor, the only person with no manager.
- The second is the **recursive part**. It refers to `org`, the CTE's own name. On each pass it sees only the rows the *previous* pass produced, and joins `staff` to them: "who reports to someone we found last time?"
- `o.level + 1` carries information down the tree. Each person's level is their manager's level plus one.
- The recursion stops by itself when a pass finds no new rows. After the machine operators, nobody reports to anybody new, so pass 5 returns nothing and the query ends.
- The final `SELECT` reads the whole accumulated result like any other CTE.

The passes, counted from the output:

Pass	Reads	Finds	Who they are
anchor	nothing	1	the managing director (level 1)
1	1	8	the eight department heads (level 2)
2	8	10	the ten people at level 3: six managers, and four people who manage no one
3	10	13	level 4: supervisors, an inspector, and the sales executives
4	13	3	the three machine operators (level 5)
5	3	0	nobody reports to an operator: stop

$1 + 8 + 10 + 13 + 3 = 35$ people.

staff: each row points to its manager (manager_id)



Pass 5 reads the three operators, finds nobody who reports to them, and the query stops.

Figure 28.1 – Each pass of the recursive part adds one level. The query stops when a pass finds nobody new. The chart shows 16 of the 35 people; the query returns all of them.

Check a number by hand: Farah Khan reports directly to Anita Rao, who reports to Arvind Kapoor. Level 1, 2, 3. ✓ And the row count, 35, matches `SELECT COUNT(*) FROM staff`, so nobody was lost. That check matters more than it looks: a person whose `manager_id` points to someone who doesn't exist would silently disappear from a recursive walk. Exercise 4 asks you to find such orphans.

Walking up instead of down

"Who is in Gopal Sahu's chain of command?" Reverse the join: start from one person and repeatedly fetch *their manager*.

```
WITH RECURSIVE chain AS (  
    SELECT staff_id, staff_name, manager_id, 0 AS steps_up  
    FROM staff  
    WHERE staff_name = 'Gopal Sahu'  
    UNION ALL  
    SELECT m.staff_id, m.staff_name, m.manager_id, c.steps_up + 1  
    FROM staff AS m  
    JOIN chain AS c ON m.staff_id = c.manager_id  
)  
SELECT steps_up, staff_name  
FROM chain  
ORDER BY steps_up;
```

steps_up	staff_name
0	Gopal Sahu
1	Ajay Kumar
2	Ramesh Patil
3	Harpreet Sethi
4	Arvind Kapoor

(5 rows)

The only change is the join direction: `m.staff_id = c.manager_id` ("the person whose id is my manager's id") instead of `s.manager_id = o.staff_id`. An approval workflow uses exactly this walk. Riverstone's discount rules (Chapter 3) send large discounts up the chain until they reach someone with enough authority.

Counting everyone below each manager

"How many people does each manager have under them, directly or indirectly?" The trick is to pair every manager with every person below them.

Start each person as "under themselves", then walk down:

```
WITH RECURSIVE under AS (  
    SELECT staff_id AS manager_id, staff_id AS member_id  
    FROM staff  
    UNION ALL  
    SELECT u.manager_id, s.staff_id  
    FROM under AS u  
    JOIN staff AS s ON s.manager_id = u.member_id  
)  
SELECT m.staff_name,  
       m.job_title,  
       COUNT(*) - 1 AS people_below  
FROM under AS u  
JOIN staff AS m ON m.staff_id = u.manager_id  
GROUP BY m.staff_id, m.staff_name, m.job_title  
HAVING COUNT(*) > 1  
ORDER BY people_below DESC, m.staff_id;
```

staff_name	job_title	people_below
Arvind Kapoor	Managing Director	34
Anita Rao	Sales Head	16
Harpreet Sethi	Head of Production	8
Ramesh Patil	Plant Manager, Talaja	4
Arjun Nair	Regional Sales Manager, South	3
Pooja Desai	Regional Sales Manager, West	3
Vikram Singh	Sales Manager, Key Accounts	2
Sandeep Gill	Regional Sales Manager, North	2
Kiran Bhosale	Plant Manager, Chakan	2
Ajay Kumar	Shift Supervisor	2
Suresh Menon	Finance Manager	1
Swati Joshi	Shift Supervisor	1
Mahesh Yadav	Warehouse & Dispatch Manager	1

(13 rows)

How it works:

- The anchor has one row per person: (`manager_id = me`, `member_id = me`). That's 35 rows.
- Each pass takes every pair and adds the member's direct reports as new members of the same manager. Arvind's pairs grow to include everyone; Ajay Kumar's grow to include his two operators.
- `COUNT(*) - 1` removes the "under myself" row. `HAVING COUNT(*) > 1` keeps only people who manage somebody.
- Reconcile: the managing director has 34 people below him, which is everyone except himself (35 - 1). ✓ Harpreet Sethi's 8 is the two plant managers, two supervisors, one inspector, and three operators.

Combining a hierarchy with facts

Now join the tree to numbers. "What did Anita Rao's key accounts team sell in 2025?" Anita's branch of the tree now includes the regional sales managers and their executives, but the one-year database holds only the key accounts, so the query keeps the five people the ERP knows by their `employee_id`.

```

WITH RECURSIVE sales_team AS (
    SELECT staff_id, staff_name, employee_id
    FROM staff
    WHERE staff_name = 'Anita Rao'
    UNION ALL
    SELECT s.staff_id, s.staff_name, s.employee_id
    FROM staff AS s
    JOIN sales_team AS t ON s.manager_id = t.staff_id
)
SELECT t.staff_name,
       COUNT(DISTINCT sl.order_id) AS orders,
       ROUND(SUM(sl.net_revenue)) AS net_revenue
FROM sales_team AS t
LEFT JOIN sales_lines AS sl ON sl.sales_rep_id = t.employee_id
WHERE t.employee_id IS NOT NULL      -- the ERP knows only the key accounts team
GROUP BY t.staff_name
ORDER BY net_revenue DESC NULLS LAST;

```

staff_name	orders	net_revenue
Rahul Mehta	53	1494001
Farah Khan	64	1489073
Neha Kulkarni	46	1147895
Anita Rao	0	
Vikram Singh	0	

(5 rows)

The three executives' revenue adds up to ₹41,30,969. Chapter 13's total for 2025 is ₹43,35,471, so ₹2,04,502 belongs to orders with no sales rep recorded (Chapter 13's data-quality check, Pattern 10, found 11 orders with no rep; 10 of them are not cancelled, and those hold the ₹2,04,502). The team walk is right; the gap is a data problem, and it's exactly the kind of gap you should explain before anyone else finds it. Meera Iyer isn't in the result at all: she has no `employee_id`, because a coordinator doesn't take orders. Vikram Singh has no orders recorded under his own name in 2025; his team's orders are credited to the executives who took them.

Watch out: a recursive CTE's column types must match. You met this with the list of months. It bites again in the next example: if the anchor returns `quantity` as `NUMERIC(8,3)` and the recursive part multiplies it (giving plain `NUMERIC`), PostgreSQL stops with `ERROR: recursive query "explode" column 3 has type numeric(8,3) in non-recursive term but type numeric overall`. The fix is to cast the anchor's column to the wider type, `CAST(b.quantity AS NUMERIC)`. MySQL has the opposite problem: it sizes text columns from the anchor, so a path string that grows on each pass fails with a `Data too long` error unless you `CAST` it to a long enough `CHAR` in the anchor (section 28.13).

Bills of materials

Riverstone's production team keeps a **bill of materials** (BOM): the list of parts that make each product, where some parts are themselves made of parts. The `bom_lines` table stores one link per row: "one unit of *parent* needs *quantity* units of *child*". The `parts` table says what each part is:

```
SELECT part_type, COUNT(*) AS parts
FROM parts
GROUP BY part_type
ORDER BY part_type;
```

part_type	parts
Assembly	9
Material	6
Product	5

(3 rows)

Five products, nine assemblies made in-house (a box body, a lid, a steel frame), and six materials bought from the suppliers you met in Chapter 12's lab.

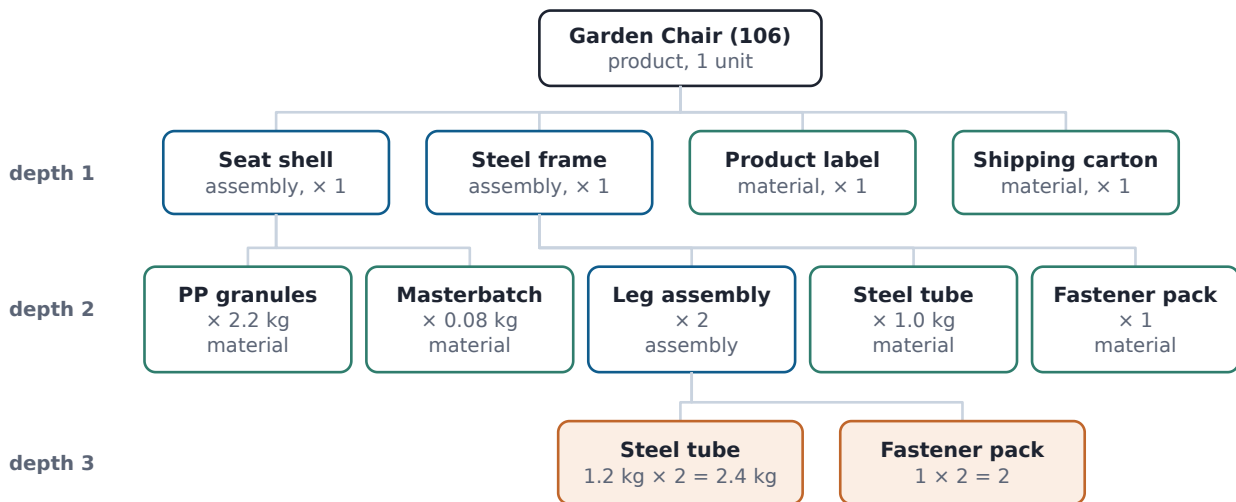
"What does it take to make one Garden Chair?" This is called **exploding** the BOM. It's the org-chart walk with one addition: quantities multiply on the way down.

```
WITH RECURSIVE explode AS (
  SELECT b.parent_part_id AS product_id,
         b.child_part_id,
         CAST(b.quantity AS NUMERIC) AS qty_per_product,
         1 AS depth
  FROM bom_lines AS b
  WHERE b.parent_part_id = 106          -- 106 = Garden Chair
  UNION ALL
  SELECT e.product_id,
         e.child_part_id,
         e.qty_per_product * b.quantity,
         e.depth + 1
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT e.depth,
       p.part_name,
       p.part_type,
       CAST(e.qty_per_product AS NUMERIC(8,3)) AS qty_per_chair,
       p.unit
FROM explode AS e
JOIN parts AS p ON p.part_id = e.child_part_id
ORDER BY e.depth, p.part_id;
```

depth	part_name	part_type	qty_per_chair	unit
1	Seat shell	Assembly	1.000	each
1	Steel frame	Assembly	1.000	each
1	Product label	Material	1.000	each
1	Shipping carton	Material	1.000	each
2	Leg assembly	Assembly	2.000	each
2	Polypropylene granules	Material	2.200	kg
2	Color masterbatch	Material	0.080	kg
2	Steel tube	Material	1.000	kg
2	Fastener pack	Material	1.000	each
3	Steel tube	Material	2.400	kg
3	Fastener pack	Material	2.000	each

(11 rows)

Bill of materials: quantities multiply as you go down



Depth-3 boxes (shaded) are found on the second pass of the recursive part: each child's quantity is multiplied by its parent's running quantity. Steel tube appears twice: 1.0 kg + 2.4 kg = 3.4 kg per chair.

Material cost of one chair

PP granules 2.2 kg × ₹110 = ₹242.00
 Masterbatch 0.08 kg × ₹260 = ₹20.80
 Steel tube 3.4 kg × ₹95 = ₹323.00

Fasteners 3 × ₹14 = ₹42.00
 Label ₹2 + carton ₹18 = ₹20.00
Total ₹647.80

Figure 28.2 – Quantities multiply down the tree: 2 leg assemblies × 1.2 kg each = 2.4 kg of steel tube.

How it works:

- The anchor fetches the chair's direct children (depth 1).
- The recursive part fetches each child's own children and multiplies: the frame needs 2 leg assemblies, each leg assembly needs 1.2 kg of steel tube, so through the legs one chair needs 2 × 1.2 = 2.4 kg. That's the steel tube row at depth 3, the shaded box in Figure 28.2.
- Steel tube appears twice (1.0 kg directly in the frame, 2.4 kg in the legs). That's correct: it's used in two places. To buy steel, you add them: 3.4 kg per chair.
- Assemblies appear in the list, but you don't buy them. For purchasing, keep only `part_type = 'Material'`.

Rolling up material cost

"What are the materials in each product worth, compared with its standard cost?" Explode every product at once, keep the materials, multiply by their cost, and add up per product:

```
WITH RECURSIVE explode AS (  
  SELECT b.parent_part_id AS product_id,  
         b.child_part_id,  
         CAST(b.quantity AS NUMERIC) AS qty_per_product  
  FROM bom_lines AS b  
  WHERE b.parent_part_id IN (SELECT part_id FROM parts WHERE part_type = 'Product')  
  UNION ALL  
  SELECT e.product_id,  
         b.child_part_id,  
         e.qty_per_product * b.quantity  
  FROM explode AS e  
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id  
)  
SELECT p.part_id    AS product_id,  
       p.part_name  AS product_name,  
       CAST(SUM(e.qty_per_product * m.material_cost) AS NUMERIC(10,2)) AS material_cost,  
       pr.unit_cost AS standard_cost  
FROM explode AS e  
JOIN parts    AS m ON m.part_id = e.child_part_id AND m.part_type = 'Material'  
JOIN parts    AS p ON p.part_id = e.product_id  
JOIN products AS pr ON pr.product_id = p.part_id  
GROUP BY p.part_id, p.part_name, pr.unit_cost  
ORDER BY p.part_id;
```

product_id	product_name	material_cost	standard_cost
101	Storage Box 10L	110.30	300.00
102	Storage Box 25L	199.30	540.00
104	Food Container Set	59.42	430.00
106	Garden Chair	647.80	850.00
108	Stackable Bin	55.40	190.00

(5 rows)

The last join works because Riverstone numbered its parts to match its products: part 106 is product 106, the Garden Chair, so `pr.product_id = p.part_id` fetches the product's standard cost from Chapter 12's `products` table. Products 103, 105, and 107 are missing from the result because the `parts` table holds the bills of materials for only five of the eight products.

Check the chair by hand, using the explosion above: polypropylene 2.2 kg × ₹110 = ₹242.00, masterbatch 0.08 kg × ₹260 = ₹20.80, steel tube 3.4 kg × ₹95 = ₹323.00, fasteners 3 × ₹14 = ₹42.00, label ₹2.00, carton ₹18.00. Total ₹647.80. ✓

What to tell the production head. Materials are three-quarters of the Garden Chair's ₹850 standard cost (₹647.80, or 76.2%), and half of that (₹323.00, 49.9%) is steel tube. A 10% rise in the steel price would add ₹32.30 to every chair, while the same rise in polypropylene would add ₹24.20. For the Food Container Set, materials are only ₹59.42 of ₹430: the cost is in molding time, not plastic. Those are two different conversations with two different suppliers. (Standard cost also includes labor, machine time, and overheads, which the BOM doesn't hold.)

Where is a part used?

The reverse question matters when a supplier has a problem: "Kaveri Steel Works can't deliver steel tube this month. Which products are affected, and how much tube does each unit need?" Walk up from the part:

```
WITH RECURSIVE uses AS (  
    SELECT b.parent_part_id, CAST(b.quantity AS NUMERIC) AS qty  
    FROM bom_lines AS b  
    WHERE b.child_part_id = 305          -- 305 = Steel tube  
    UNION ALL  
    SELECT b.parent_part_id, u.qty * b.quantity  
    FROM uses AS u  
    JOIN bom_lines AS b ON b.child_part_id = u.parent_part_id  
)  
SELECT p.part_name,  
       CAST(SUM(u.qty) AS NUMERIC(8,3)) AS kg_steel_tube_per_unit  
FROM uses AS u  
JOIN parts AS p ON p.part_id = u.parent_part_id  
WHERE p.part_type = 'Product'  
GROUP BY p.part_name;
```

part_name	kg_steel_tube_per_unit
Garden Chair	3.400

(1 row)

Only the Garden Chair, at 3.4 kg per unit, matching the explosion. ✓ This is called a **where-used** query.

Guarding against loops

A tree has no loops: nobody is their own manager's manager. But data entered by people can contain one. Suppose, by mistake, person 1 reports to 3, 3 reports to 2, and 2 reports to 1. To experiment safely, build those three rows as a tiny table inside the query, with **VALUES**:

```
WITH bad_staff (staff_id, manager_id) AS (  
    VALUES (1, 3), (2, 1), (3, 2)  
)  
SELECT * FROM bad_staff;
```

staff_id	manager_id
1	3
2	1
3	2

(3 rows)

How it works: `VALUES (1, 3), (2, 1), (3, 2)` builds a three-row table from typed-in values, exactly as in section 14.3. There you named its columns with `AS v(row_no, branch, line)`; here the names go in brackets after the CTE's name, `bad_staff (staff_id, manager_id)`. It's the quickest way to make test data that no real table holds.

Now walk up the chain from person 1, as in "Walking up instead of down". The loop means the walk would go round forever, so `LIMIT` stops the output after seven rows:

```
WITH RECURSIVE bad_staff (staff_id, manager_id) AS (  
  VALUES (1, 3), (2, 1), (3, 2)  
)  
,  
chain AS (  
  SELECT staff_id, manager_id, 1 AS steps  
  FROM bad_staff  
  WHERE staff_id = 1  
  UNION ALL  
  SELECT b.staff_id, b.manager_id, c.steps + 1  
  FROM bad_staff AS b  
  JOIN chain AS c ON b.staff_id = c.manager_id  
)  
SELECT * FROM chain LIMIT 7;
```

staff_id	manager_id	steps
1	3	1
3	2	2
2	1	3
1	3	4
3	2	5
2	1	6
1	3	7

(7 rows)

`bad_staff` isn't recursive, but `WITH RECURSIVE` covers the whole list, so `chain` can refer to itself. Without the `LIMIT`, the query never ends in PostgreSQL (until it runs out of memory or someone cancels it). There are three defenses.

Defense 1: a depth limit. Add a condition on the level to the recursive part, so the passes stop at a set depth:

```
WITH RECURSIVE bad_staff (staff_id, manager_id) AS (  
  VALUES (1, 3), (2, 1), (3, 2)  
)  
,  
chain AS (  
  SELECT staff_id, manager_id, 1 AS steps  
  FROM bad_staff  
  WHERE staff_id = 1  
  UNION ALL  
  SELECT b.staff_id, b.manager_id, c.steps + 1  
  FROM bad_staff AS b  
  JOIN chain AS c ON b.staff_id = c.manager_id  
  WHERE c.steps < 7          -- defense 1: a depth limit  
)  
SELECT * FROM chain;
```

staff_id	manager_id	steps
1	3	1
3	2	2
2	1	3
1	3	4
3	2	5
2	1	6
1	3	7

(7 rows)

The same seven rows, but now the query stops on its own: the pass that reads the row with `steps = 7` finds `7 < 7` false and produces nothing. It's simple and works in every database. The catch: it also stops a *legitimate* tree that's deeper than the limit, and it doesn't tell you there was a loop. Set it well above the real depth (Riverstone's org chart has five levels; 20 is a safe limit).

Defense 2: remember the path. Carry the ids visited so far in a text column, and refuse to visit an id that's already in it:

```
WITH RECURSIVE bad_staff (staff_id, manager_id) AS (
    VALUES (1, 3), (2, 1), (3, 2)
),
chain AS (
    SELECT staff_id, manager_id,
           CAST(staff_id AS VARCHAR(200)) AS path
    FROM bad_staff
    WHERE staff_id = 1
    UNION ALL
    SELECT b.staff_id, b.manager_id,
           CAST(c.path || '/' || b.staff_id AS VARCHAR(200))
    FROM bad_staff AS b
    JOIN chain AS c ON b.staff_id = c.manager_id
    WHERE '/' || c.path || '/' NOT LIKE '%/' || b.staff_id || '/' -- defense 2
)
SELECT * FROM chain;
```

staff_id	manager_id	path
1	3	1
3	2	1/3
2	1	1/3/2

(3 rows)

How it works:

- The anchor starts the path with the first id, '1'. Each pass adds / and the new id with ||, PostgreSQL's operator for joining text (section 12.8), so the path grows 1, 1/3, 1/3/2.
- The `CAST(... AS VARCHAR(200))` in both parts keeps the column's type the same on every pass (the rule you met with the list of months).
- The `WHERE` wraps the path in slashes (/1/3/2/) and asks, with `NOT LIKE` (section 12.6), whether /1/ is already in it. It is, so the fourth row is never made and the walk stops after three rows, each person once.

The same idea works in MySQL (with `CONCAT` instead of `||`); section 28.13 uses it.

Defense 3: PostgreSQL's CYCLE clause (PostgreSQL 14 and later) does the path check for you:

```
WITH RECURSIVE bad_staff (staff_id, manager_id) AS (  
    VALUES (1, 3), (2, 1), (3, 2)  
)  
,  
chain AS (  
    SELECT staff_id, manager_id  
    FROM bad_staff  
    WHERE staff_id = 1  
    UNION ALL  
    SELECT b.staff_id, b.manager_id  
    FROM bad_staff AS b  
    JOIN chain AS c ON b.staff_id = c.manager_id  
)  
CYCLE staff_id SET is_loop USING visited  
SELECT staff_id, manager_id, is_loop, visited  
FROM chain;
```

staff_id	manager_id	is_loop	visited
1	3	f	{(1)}
3	2	f	{(1),(3)}
2	1	f	{(1),(3),(2)}
1	3	t	{(1),(3),(2),(1)}

(4 rows)

How it works: `CYCLE staff_id` watches that column. `SET is_loop` names a new true/false column, and `USING visited` names the column that holds the path. When a row's `staff_id` is already in its path, it's marked `is_loop = t` and not followed further. PostgreSQL prints the path as a list in curly braces, `{(1),(3)}`; each id sits in its own round brackets because `CYCLE` can watch several columns at once, and then each entry would hold several values. Filter `WHERE is_loop` to list the loops so someone can fix the data.

Ordering a tree: SEARCH DEPTH FIRST

The org chart query sorted people by level: everyone at level 2, then everyone at level 3. An indented org chart is drawn differently: each manager, followed straight away by their whole team.

PostgreSQL's `SEARCH` clause (PostgreSQL 14 and later) produces that order. Here's Anita Rao's branch:

```
WITH RECURSIVE org AS (  
    SELECT staff_id, staff_name, 1 AS level  
    FROM staff  
    WHERE staff_name = 'Anita Rao'  
    UNION ALL  
    SELECT s.staff_id, s.staff_name, o.level + 1  
    FROM staff AS s  
    JOIN org AS o ON s.manager_id = o.staff_id  
)  
SEARCH DEPTH FIRST BY staff_id SET visit_order  
SELECT REPEAT('    ', level - 1) || staff_name AS team  
FROM org  
ORDER BY visit_order;
```

```

      team
-----
Anita Rao
  Vikram Singh
    Neha Kulkarni
    Rahul Mehta
  Farah Khan
  Meera Iyer
  Arjun Nair
    Divya Krishnan
    Vivek Chandran
    Sneha Pillai
  Pooja Desai
    Nisha Bhatt
    Aditya Verma
    Rohit Kamat
  Sandeep Gill
    Karan Ahuja
    Ritu Bansal
(17 rows)

```

How it works:

- **SEARCH DEPTH FIRST BY staff_id** goes down each branch to the bottom before moving to the next branch ("depth first"), taking people with the same manager in `staff_id` order.
- **SET visit_order** names a new column that records that order. You don't show it; you sort by it with `ORDER BY visit_order`.
- **SEARCH BREADTH FIRST BY staff_id** would sort level by level instead, like the org chart query above.
- **REPEAT(' ', level - 1)** repeats four spaces once per level below the top, so each team is indented under its manager; `||` joins the spaces to the name.

Anita has 16 people below her plus herself: 17 rows. ✓ MySQL has neither `SEARCH` nor `CYCLE`; section 28.13 builds the same order with a `sort path`, which works in both databases.

28.3 Advanced window frames

Section 13.6 introduced frames and the trap of the default `RANGE` frame. This section completes the picture: the three frame types, frames measured in days, excluding rows, reusing window definitions, and the percentile functions.

ROWS, RANGE, and GROUPS

A frame answers "*which rows around me count?*", and there are three ways to measure "around":

Frame type	Counts in units of	"1 PRECEDING" means
ROWS	physical rows	the one row before me
GROUPS	groups of rows with the same <code>ORDER BY</code> value	every row in the previous distinct value (a <i>peer group</i>)
RANGE	the <code>ORDER BY</code> value itself	every row whose value is within the offset of mine; on a date column the offset must be an interval, such as <code>INTERVAL '1 day'</code>

The difference only shows when values repeat or have gaps. On 16, 17, and 20 February 2025, Riverstone received two orders each. Take the three frame types one at a time, starting with `ROWS`:

```
SELECT o.order_date,
       o.order_id,
       COUNT(*) OVER (ORDER BY o.order_date, o.order_id
                      ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_in_frame
FROM orders AS o
WHERE o.order_date BETWEEN DATE '2025-02-14' AND DATE '2025-02-21'
ORDER BY o.order_date, o.order_id;
```

order_date	order_id	rows_in_frame
2025-02-16	10011	1
2025-02-16	10012	2
2025-02-17	10013	2
2025-02-17	10014	2
2025-02-20	10015	2
2025-02-20	10016	2

(6 rows)

Reading it: `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` counts the row before and this row: always 2 after the first row, whatever the dates. The window orders by `order_date, order_id`, so the order of two same-day orders is fixed; without `order_id`, which of them counts as "first" would be up to the database. The first row has no row before it, because **windows are calculated after WHERE**: the frame can't see rows the `WHERE` removed. (14 and 15 February had no orders anyway.)

Now add a `GROUPS` column beside it:

```
SELECT o.order_date,
       o.order_id,
       COUNT(*) OVER (ORDER BY o.order_date, o.order_id
                      ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_in_frame,
       COUNT(*) OVER (ORDER BY o.order_date
                      GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_in_2_dates
FROM orders AS o
WHERE o.order_date BETWEEN DATE '2025-02-14' AND DATE '2025-02-21'
ORDER BY o.order_date, o.order_id;
```

order_date	order_id	rows_in_frame	rows_in_2_dates
2025-02-16	10011	1	2
2025-02-16	10012	2	2
2025-02-17	10013	2	4
2025-02-17	10014	2	4
2025-02-20	10015	2	4
2025-02-20	10016	2	4

(6 rows)

Reading it: rows with the same `ORDER BY` value are **peers**, and each set of peers is a **peer group**: here, one date's orders. This window orders by date alone, so that it has peers to count. `GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW` counts this date's peer group plus the previous one, the previous *date that has orders*. On 20 February that's 20 Feb and 17 Feb: 4 orders, even though 17 Feb is three days earlier.

Finally, `RANGE` with an interval. Three windows with three frames make a long `SELECT`, so name them with the `WINDOW` clause from section 13.6, where you named one window `w`. Each `OVER` then refers to a window by name:

```
SELECT o.order_date,
       o.order_id,
       COUNT(*) OVER w_rows AS rows_in_frame,
       COUNT(*) OVER w_groups AS rows_in_2_dates,
       COUNT(*) OVER w_range AS rows_in_2_days
FROM orders AS o
WHERE o.order_date BETWEEN DATE '2025-02-14' AND DATE '2025-02-21'
WINDOW w_rows AS (ORDER BY o.order_date, o.order_id ROWS BETWEEN 1 PRECEDING AND CURRENT ROW),
       w_groups AS (ORDER BY o.order_date GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW),
       w_range AS (ORDER BY o.order_date RANGE BETWEEN INTERVAL '1 day' PRECEDING AND CURRENT ROW)
ORDER BY o.order_date, o.order_id;
```

order_date	order_id	rows_in_frame	rows_in_2_dates	rows_in_2_days
2025-02-16	10011	1	2	2
2025-02-16	10012	2	2	2
2025-02-17	10013	2	4	4
2025-02-17	10014	2	4	4
2025-02-20	10015	2	4	2
2025-02-20	10016	2	4	2

(6 rows)

Reading it:

- The first two columns are unchanged: naming a window changes nothing about what it calculates. Several windows go in one `WINDOW` clause, separated by commas.
- **`RANGE BETWEEN INTERVAL '1 day' PRECEDING AND CURRENT ROW`** counts orders from yesterday and today, measured on the calendar. On 20 February, 19 February had no orders, so only today's 2 count. On 17 February, 16 February is exactly one day earlier: 4.
- On a date column, the offset must be an interval. Write `RANGE BETWEEN 1 PRECEDING ...` and PostgreSQL refuses: *RANGE with offset PRECEDING/FOLLOWING is not supported for column type date and offset type integer.*

Use `ROWS` for "the last N records", `GROUPS` for "the last N distinct dates that had activity", and `RANGE` with an interval for "the last N calendar days". You can also extend a named window:

`OVER (w ROWS BETWEEN ...)` adds a frame to a window that defines only the partition and order. You'll use that in "FIRST_VALUE, LAST_VALUE, and the default frame" below.

Calendar windows with RANGE and intervals

"What was revenue over the last seven days, on each day we took orders?" Riverstone doesn't take orders every day, so "the last seven rows" is not "the last seven days":

```
WITH daily AS (  
  SELECT order_date, SUM(net_revenue) AS revenue  
  FROM sales_lines  
  WHERE order_date BETWEEN DATE '2025-11-20' AND DATE '2025-12-10'  
  GROUP BY order_date  
)  
SELECT order_date,  
  ROUND(revenue) AS revenue,  
  ROUND(SUM(revenue) OVER (ORDER BY order_date  
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS last_3_rows,  
  ROUND(SUM(revenue) OVER (ORDER BY order_date  
    RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)) AS last_7_days  
FROM daily  
ORDER BY order_date;
```

order_date	revenue	last_3_rows	last_7_days
2025-11-22	86615	86615	86615
2025-11-23	2731	89346	89346
2025-11-25	10213	99559	99559
2025-11-26	10060	23004	109619
2025-11-28	17200	37473	126819
2025-12-01	71862	99122	109334
2025-12-04	9809	98870	98870
2025-12-05	13705	95375	95375
2025-12-08	65818	89331	89331
2025-12-10	7698	87221	97029

(10 rows)

Hand-check 26 November. The last three rows are 23, 25, and 26 November: ₹2,731 + ₹10,213 + ₹10,060 = ₹23,004. ✓ The last seven days run from 20 to 26 November, which includes the big ₹86,615 order on the 22nd: ₹86,615 + ₹2,731 + ₹10,213 + ₹10,060 = ₹1,09,619. ✓ On 1 December, the seven days (25 November to 1 December) include 25, 26, and 28 November and 1 December: ₹1,09,334. ✓

This is Chapter 13's date-spine problem (Pattern 5) solved without a spine: `RANGE` with an interval measures by the calendar, so missing days take care of themselves. Because the frame begins before the query's first date, the first few rows here only see part of their window, which is why 22 November shows a single order. In a real report, start the `WHERE` six days earlier and filter the output afterwards.

Dialect note. PostgreSQL writes `INTERVAL '6 days'`; MySQL writes `INTERVAL 6 DAY`. MySQL supports `RANGE` with intervals but not `GROUPS` (section 28.13). SQL Server supports neither interval ranges nor `GROUPS`; there, use a date spine.

Excluding the current row

"Was this month unusual compared with the other months of its quarter?" Comparing a month with an average that includes itself dilutes the comparison.

EXCLUDE CURRENT ROW leaves the month itself out:

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month,  
           SUM(net_revenue) AS revenue  
    FROM sales_lines  
    GROUP BY 1  
)  
SELECT month,  
       ROUND(revenue) AS revenue,  
       ROUND(AVG(revenue) OVER (  
           PARTITION BY DATE_TRUNC('quarter', month)  
           ORDER BY month  
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
           EXCLUDE CURRENT ROW)) AS avg_other_months_in_quarter  
FROM monthly  
WHERE month >= DATE '2025-07-01'  
ORDER BY month;
```

month	revenue	avg_other_months_in_quarter
2025-07-01	232692	443798
2025-08-01	329282	395504
2025-09-01	558315	280987
2025-10-01	681071	536616
2025-11-01	633408	560447
2025-12-01	439824	657239

(6 rows)

September's ₹5,58,315 against the other two months of Q3 (₹2,32,692 and ₹3,29,282, averaging ₹2,80,987) shows the festive season starting in September, not October. (Check: $(2,32,692 + 3,29,282) \div 2 = 2,80,987$. ✓) `DATE_TRUNC('quarter', month)` works like the 'month' version from Chapter 13: it returns the first day of the quarter, so each quarter's three months form one partition. `EXCLUDE` has four forms: `CURRENT ROW`, `GROUP` (the current row and its peers), `TIES` (the peers but not the current row), and `NO OTHERS` (the default). It is PostgreSQL only; MySQL has no `EXCLUDE` (section 28.13).

FIRST_VALUE, LAST_VALUE, and the default frame

Chapter 13's mistakes table warned that `LAST_VALUE` with the default frame returns the current row. Here it is on Patel Kitchenware's orders, with the fix beside it:

```
WITH order_values AS (  
    SELECT o.customer_id, o.order_id, o.order_date, SUM(sl.net_revenue) AS order_value  
    FROM orders AS o  
    JOIN sales_lines AS sl ON sl.order_id = o.order_id  
    WHERE o.customer_id = 2 -- 2 = Patel Kitchenware  
    GROUP BY o.customer_id, o.order_id, o.order_date  
)  
SELECT order_date,  
       ROUND(order_value) AS order_value,  
       ROUND(FIRST_VALUE(order_value) OVER w) AS first_order,  
       ROUND(LAST_VALUE(order_value) OVER w) AS last_value_default,  
       ROUND(LAST_VALUE(order_value) OVER (w ROWS BETWEEN UNBOUNDED PRECEDING  
                                           AND UNBOUNDED FOLLOWING)) AS latest_order  
FROM order_values  
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date, order_id)  
ORDER BY order_date;
```

order_date	order_value	first_order	last_value_default	latest_order
2025-01-02	2900	2900	2900	39870
2025-03-28	35138	2900	35138	39870
2025-04-11	19825	2900	19825	39870
2025-05-04	23650	2900	23650	39870
2025-06-10	14155	2900	14155	39870
2025-08-14	38610	2900	38610	39870
2025-11-09	39870	2900	39870	39870

(7 rows)

With an `ORDER BY` and no frame, the frame ends at the current row, so "the last value in the frame" is always the current row: `last_value_default` copies `order_value`. Extending the frame to `UNBOUNDED FOLLOWING` gives the real latest order, ₹39,870 on 9 November. `FIRST_VALUE` works with the default frame because the frame always starts at the first row. `NTH_VALUE(order_value, 2)` gives the second order, with the same frame caution.

Percentiles and quartiles

"What's a typical order worth?" Chapter 4 explained why the median is often more honest than the mean, Chapter 15 (section 15.5) used `PERCENTILE_CONT` for medians and quartiles, and Chapter 21 (section 21.3) added `PERCENTILE_DISC`. This is a recap, with a name for the family. Functions written with `WITHIN GROUP (ORDER BY ...)` are called **ordered-set aggregates**: aggregates that need their input sorted first. Here are the mean, the median, and the 90th percentile of the year's order values:

```
WITH order_values AS (
  SELECT order_id, SUM(net_revenue) AS order_value
  FROM sales_lines
  GROUP BY order_id
)
SELECT COUNT(*)           AS orders,
       ROUND(AVG(order_value)) AS mean_value,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)::NUMERIC) AS median_value,
       ROUND(PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY order_value)::NUMERIC) AS p90_value
FROM order_values;
```

orders	mean_value	median_value	p90_value
173	25061	21375	48334

(1 row)

There are 173 orders here, not the 175 of section 28.1, because `sales_lines` leaves out the two cancelled ones. The mean (₹25,061) is above the median (₹21,375), because a few large orders pull it up. One order in ten is worth more than ₹48,334. As in section 21.3, `::NUMERIC` converts the percentile before rounding: `PERCENTILE_CONT` returns a type (double precision) that `ROUND` accepts only without a number of decimals, and the cast keeps this query working if you later ask for `ROUND(..., 2)`.

The difference between the two percentile functions, on four numbers you can check by eye:

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x) AS cont_median,
       PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY x) AS disc_median
FROM (VALUES (10), (20), (30), (40)) AS v(x);
```

cont_median	disc_median
25	20

(1 row)

`PERCENTILE_CONT` interpolates between the two middle values, 20 and 30, and returns 25, a value that isn't in the data. `PERCENTILE_DISC` returns an actual value: the first one at or past the halfway mark, 20. These are aggregates, not window functions: in PostgreSQL they can't take `OVER`.

To put every order into a band instead, use the window function `NTILE(n)`, which Chapter 27 introduced (section 27.4). As a reminder, it splits the ordered rows into n groups as equal in size as possible and numbers them 1 to n :

```
WITH order_values AS (
  SELECT order_id, SUM(net_revenue) AS order_value
  FROM sales_lines
  GROUP BY order_id
),
banded AS (
  SELECT order_value, NTILE(4) OVER (ORDER BY order_value) AS quartile
  FROM order_values
)
SELECT quartile,
       COUNT(*) AS orders,
       ROUND(MIN(order_value)) AS smallest,
       ROUND(MAX(order_value)) AS largest,
       ROUND(SUM(order_value)) AS revenue
FROM banded
GROUP BY quartile
ORDER BY quartile;
```

quartile	orders	smallest	largest	revenue
1	44	1378	12355	307338
2	43	12398	21375	716217
3	43	21650	35138	1196060
4	43	35300	100278	2115857

(4 rows)

173 orders don't divide by 4, so the first band gets the extra order ($44 + 43 + 43 + 43 = 173$ ✓). The revenue column tells the story: the largest quarter of orders brings ₹21,15,857, almost half of the year's ₹43,35,471 (48.8%), and the smallest quarter brings 7.1%. The bands add up to the year's total: $3,07,338 + 7,16,217 + 11,96,060 + 21,15,857 = 43,35,472$, one rupee more than the total because each band was rounded separately. ✓

Watch out: `NTILE` splits by count, not by value. If many orders have the same value, `NTILE` may put identical values into different bands, because it only cares about getting equal-sized groups. For value-based bands, compute the cut points with `PERCENTILE_CONT` and use `CASE`.

Good place to stop.

28.4 How a database finds rows

To speed up a query, you need a picture of what the database does with it. The picture is simpler than you might expect.

Tables live in pages

PostgreSQL stores a table as a file divided into **pages** (also called blocks) of 8 KB each. Rows are packed into pages in roughly the order they were inserted, not in any useful order. `riverstone_perf`'s `orders` table is 41 MB, which is about 5,200 pages; `order_items` is about 101 MB. Reading pages from disk (or even from memory) is most of the work a query does, so the question behind every performance problem is: **how many pages did the database have to read to find the rows it needed?**

Scanning

With no other help, the only way to find customer 2718's orders is a **sequential scan** (in PostgreSQL's plans, `Seq Scan`): read every page and check every row. For 714,285 orders, that's every one of them, to keep 40. It is slow when you want a few rows, but it isn't always wrong. To total *all* the year's revenue, reading everything is exactly the job, and a scan does it in the most efficient order.

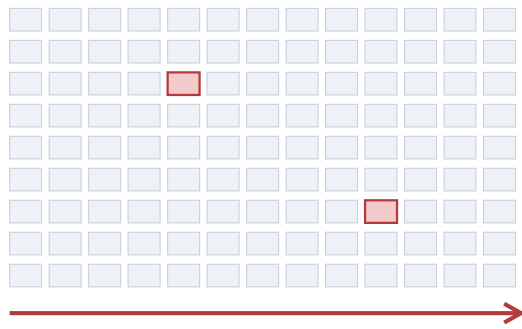
Indexes

An **index** is a separate structure that stores one column's values (or several columns') in sorted order, each with a pointer to where its row lives in the table. It works like the index at the back of a printed book: to find "grain", you don't read every page, you look it up and go straight to the right page.

The standard kind is the **B-tree** (balanced tree). Its top page, the root, divides the key range into a few hundred slices and points to a page for each; those pages divide their slices again, until the bottom **leaf** pages list individual values and row pointers. Because each page fans out into hundreds of children, even millions of values need only three or four levels. Finding customer 2718 means reading the root, a page or two below it, a leaf, and then the 40 table pages holding those rows.

Finding customer 2718's 40 orders among 714,285

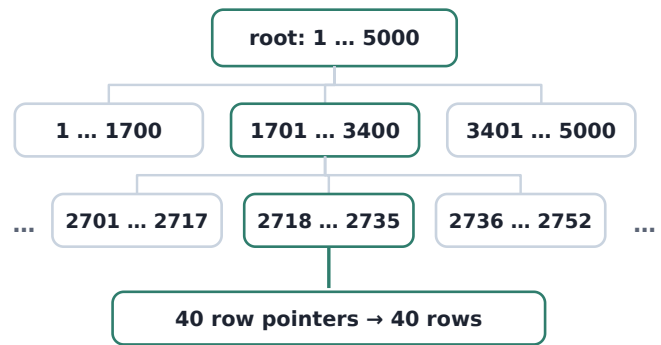
Sequential scan: read every row



714,285 rows checked, 714,245 thrown away

median 38.77 ms

B-tree index: follow the sorted keys



(leaf pages under 1701 ... 3400; one branch shown)
a few index pages, then 40 table pages

median 0.18 ms

PostgreSQL 16 on the author's test machine, riverstone_perf. Your times will differ; the ratio (about 200 to 1) is what matters.

Figure 28.3 – The same question answered two ways. On the test machine the index made it about 200 times faster.

What indexes cost

Indexes aren't free, which is why databases don't index every column automatically:

- **Space.** The index on `orders.customer_id` takes 4,960 kB; the primary key index on `order_items` takes 41 MB, almost half the size of the table itself.
- **Slower writes.** Every `INSERT`, every `DELETE`, and every `UPDATE` of an indexed column must update each index too. Section 28.6 measures this.
- **Choices for the planner.** More indexes mean more plans to consider, and occasionally a poor choice.

A **primary key** always has an index, because the database needs one to enforce uniqueness. So does a `UNIQUE` constraint. Everything else is your decision.

Watch out: PostgreSQL does not index foreign keys for you. Declaring `order_items.order_id REFERENCES orders(order_id)` creates a rule, not an index. Joins on that column, and every delete from `orders` (which must check for lines that still point at the order), then scan the whole `order_items` table. MySQL's InnoDB engine *does* create an index for each foreign key automatically. Section 28.13 shows the difference on real data.

28.5 Reading query plans with EXPLAIN

Chapter 12 (section 12.11) promised you'd see what the database actually does with a query. From here to the end of section 28.6, work in `riverstone_perf`, the database you built in section 28.1. Put **EXPLAIN** in front of any query, and PostgreSQL shows its **query plan**, the steps it has chosen, without running the query:

```
EXPLAIN
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 2718;
```

```
          QUERY PLAN
-----
Seq Scan on orders  (cost=0.00..14149.56 rows=97 width=17)
  Filter: (customer_id = 2718)
(2 rows)
```

(Customer 2718 is a typical customer, with 40 orders. There is no index on `customer_id` yet. The plans in sections 28.5 and 28.6 are copied from the recorded runs in `ch28_perf_log.txt`, because timings change on every run; the costs and row counts on your machine should match or come very close.)

Reading it:

- **Seq Scan on orders**: the step, a sequential scan of the whole table.
- **Filter: (customer_id = 2718)**: the condition checked on each row as it's read.
- **cost=0.00..14149.56**: the planner's *estimate* of effort, in arbitrary units (roughly, the cost of reading one page in order is 1). The first number is the cost before the first row can be returned; the second is the cost to return all rows. Costs let the planner compare plans. They are not milliseconds.
- **rows=97**: the estimated number of rows returned. The planner guesses from **statistics** it keeps about each column (how many distinct values, which are most common), refreshed by **ANALYZE**. The real answer is 40, so the estimate is in the right neighborhood.
- **width=17**: the estimated average size of each row in bytes.

EXPLAIN ANALYZE: run it and measure

EXPLAIN ANALYZE runs the query for real and adds what actually happened. Options go in brackets after **EXPLAIN**; the option **COSTS OFF** hides the estimates to keep lines short:

```
EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 2718;
```

QUERY PLAN

```
-----  
Seq Scan on orders (actual time=1.051..47.426 rows=40 loops=1)  
  Filter: (customer_id = 2718)  
  Rows Removed by Filter: 714245  
Planning Time: 0.260 ms  
Execution Time: 47.479 ms  
(5 rows)
```

- **actual time=1.051..47.426**: milliseconds until the first row, and until the last row.
- **rows=40 loops=1**: rows actually returned, and how many times this step ran.
- **Rows Removed by Filter: 714245**: the tell-tale sign. The database read 714,285 rows to keep 40. When you see a filter throwing away almost everything, an index on the filtered column is the first thing to consider.
- **Planning Time**: the time spent choosing the plan, before anything ran.
- **Execution Time**: the whole query, 47.5 ms on this run. Across seven runs the median was 38.77 ms.

Watch out: EXPLAIN ANALYZE really runs the statement. `EXPLAIN ANALYZE DELETE FROM orders ...` deletes the rows. To measure a change safely, wrap it: `BEGIN; EXPLAIN ANALYZE ...; ROLLBACK; .`

Now create an index and ask again:

```
CREATE INDEX idx_orders_customer_id ON orders (customer_id);  
  
EXPLAIN (ANALYZE, COSTS OFF)  
SELECT order_id, order_date, status  
FROM orders  
WHERE customer_id = 2718;
```

QUERY PLAN

```
-----  
Bitmap Heap Scan on orders (actual time=0.023..0.132 rows=40 loops=1)  
  Recheck Cond: (customer_id = 2718)  
  Heap Blocks: exact=40  
    -> Bitmap Index Scan on idx_orders_customer_id (actual time=0.010..0.011 rows=40 loops=1)  
        Index Cond: (customer_id = 2718)  
Planning Time: 0.303 ms  
Execution Time: 0.179 ms  
(7 rows)
```

Building the index took about a quarter of a second. The query's median fell from 38.77 ms to 0.18 ms.

Read plans from the most indented line outwards: the innermost step runs first and feeds the one above it.

- **Bitmap Index Scan** reads the index and collects the locations of the 40 matching rows. **Index Cond** is the condition the index answered.
- **Bitmap Heap Scan** then visits those locations in page order. (The *heap* is PostgreSQL's name for the table's own pages.) **Heap Blocks: exact=40** says it read 40 table pages: this customer's orders are scattered, one per page.

- **Recheck Cond** is the condition the heap scan would check again on each row if memory ran short. Then the bitmap remembers only *which pages* to read, not which rows (PostgreSQL calls that *lossy*), and every row on those pages must be rechecked. `exact=40` means every location was exact, so no rechecking was needed.

The planner chooses, and it can say no

PostgreSQL has several ways to use an index, and chooses by estimated cost:

Plan node	What it does	Typical when
Seq Scan	reads every page	most rows are needed, or no useful index
Index Scan	walks the index, fetching each row as it goes	few rows, or rows wanted in index order
Bitmap Index Scan + Bitmap Heap Scan	collects all matching locations first, then reads pages in order	a moderate number of rows
Index Only Scan	answers from the index alone, never touching the table	every column needed is in the index

Try the customer with the most orders, customer 4534, who has 23,526 of them:

```
EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 4534;
```

QUERY PLAN

```
-----
Bitmap Heap Scan on orders (actual time=1.851..14.610 rows=23526 loops=1)
  Recheck Cond: (customer_id = 4534)
  Heap Blocks: exact=5170
    -> Bitmap Index Scan on idx_orders_customer_id (actual time=1.123..1.124 rows=23526 loops=1)
        Index Cond: (customer_id = 4534)
Planning Time: 0.319 ms
Execution Time: 15.571 ms
(7 rows)
```

The same index is used, but `Heap Blocks: exact=5170` says it read 5,170 of the table's 5,221 pages, because this customer has an order on almost every page. The median was 15.26 ms. And for a condition that matches almost everything, the planner ignores the index entirely, even when one exists:

```
EXPLAIN
SELECT order_id FROM orders WHERE status = 'Delivered';
```

QUERY PLAN

```
-----
Seq Scan on orders (cost=0.00..14149.56 rows=688214 width=4)
  Filter: ((status)::text = 'Delivered'::text)
(2 rows)
```

(PostgreSQL prints the column and the text it's compared with as type `text`, with `::text`; that's how it displays the comparison, not something you wrote.) 689,050 of the 714,285 orders are delivered (96.5%). Jumping from an index to a table page 689,050 times would be slower than reading the table from start to end, and the planner knows it from its statistics. **An index helps when a condition is selective**, meaning it keeps a small fraction of rows.

Joins in a plan

Plans for joins have one more thing to read: the **join method**. Here's "customer 2718's total revenue", which joins orders to their lines. `order_items.order_id` has no index yet:

```
EXPLAIN (ANALYZE, COSTS OFF)
SELECT SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.customer_id = 2718;
```

Figure 28.4 shows its plan, with the steps numbered in the order they run.

Reading EXPLAIN ANALYZE from the inside out

customer 2718's revenue, before `order_items.order_id` had an index (loops=1 and the Hash's memory line trimmed)

```
5 Aggregate (actual time=285.475..285.485 rows=1)
4  -> Hash Join (actual time=5.181..285.266 rows=108)
    Hash Cond: (oi.order_id = o.order_id)
3  -> Seq Scan on order_items oi (actual time=0.027..142.289 rows=1926847)
2  -> Hash (actual time=0.151..0.158 rows=40)
    -> Bitmap Heap Scan on orders o (actual time=0.025..0.143 rows=40)
1  -> Bitmap Index Scan on idx_orders_customer_id (actual time=0.014..0.015 rows=40)
Execution Time: 285.549 ms
```

- 1 Start at the most indented step: the index finds customer 2718's 40 orders in 0.015 ms.
- 2 The table rows are fetched, and their 40 order ids go into a small in-memory hash table.
- 3 The expensive step: all 1,926,847 order lines are read, taking 142 ms of the total.
- 4 The Hash Join checks each line against the hash table and keeps 108: another 143 ms.
- 5 The top node is the final result. Its last time, 285 ms, is the whole query.

Times are milliseconds, shown as first row..last row. This is one run; the median of seven was 262.47 ms.

Figure 28.4 – Read the plan from the inside out, and look for the step where the time jumps. (One run; the median of seven was 262.47 ms.)

PostgreSQL uses three join methods:

- **Nested loop:** for each row on one side, look up matches on the other. Excellent when one side is small *and* the other side has an index on the join column. Disastrous when the other side must be scanned each time.
- **Hash join:** load the smaller side into an in-memory hash table, then scan the larger side once, probing the table. Good for large, unindexed joins. That's what Figure 28.4 shows: the planner had no index to look lines up by `order_id`, so it scanned all 1.9 million lines to find 108.
- **Merge join:** sort both sides on the join key, then walk them together. Good when both sides are already sorted, for example by an index.

You don't choose the method; the planner does. Your job is to give it the indexes that make the cheap methods possible, and then check that it used them.

Try it. Run `EXPLAIN SELECT * FROM sales_lines; in riverstone_2025`. A view is replaced by its query before planning, so the plan shows the joins inside `sales_lines`. With tables this small you'll mostly see sequential scans and a hash join, and that's the right choice: reading a few pages whole is cheaper than using an index.

28.6 Indexes in practice

This section runs six experiments on `riverstone_perf`. Each one starts from a question, measures it, changes one thing, and measures again. Run them in order: each experiment keeps the indexes the earlier ones created, as a real database would. The table at the end collects the results.

1. Index the foreign keys you join on

The query from Figure 28.4, unchanged, run after one new index:

```
CREATE INDEX idx_order_items_order_id ON order_items (order_id);

EXPLAIN (ANALYZE, COSTS OFF)
SELECT SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.customer_id = 2718;
```

```

                                QUERY PLAN
-----
Aggregate (actual time=0.541..0.541 rows=1 loops=1)
  -> Nested Loop (actual time=0.050..0.468 rows=108 loops=1)
    -> Bitmap Heap Scan on orders o (actual time=0.027..0.132 rows=40 loops=1)
        Recheck Cond: (customer_id = 2718)
        Heap Blocks: exact=40
        -> Bitmap Index Scan on idx_orders_customer_id (actual time=0.017..0.017 rows=40 loops=1)
            Index Cond: (customer_id = 2718)
    -> Index Scan using idx_order_items_order_id on order_items oi (actual time=0.007..0.008 rows=3
        loops=40)
        Index Cond: (order_id = o.order_id)
Planning Time: 0.798 ms
Execution Time: 0.601 ms
(11 rows)
```

The median fell from 262.47 ms to 0.64 ms. The join method changed from a hash join to a **nested loop**: for each of the 40 orders (`loops=40`), look up its lines in the new index, about 3 lines each (40×3 is close to the 108 lines found; `rows=3` is a rounded average).

The same index serves a simpler question that a screen in an order-entry system asks thousands of times a day, "show me one order's lines":

```
EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_item_id, product_id, quantity
FROM order_items
WHERE order_id = 500000;
```

QUERY PLAN

```
-----  
Index Scan using idx_order_items_order_id on order_items (actual time=0.023..0.023 rows=1 loops=1)  
  Index Cond: (order_id = 500000)  
Planning Time: 0.283 ms  
Execution Time: 0.053 ms  
(4 rows)
```

Before the index, the same query was a Seq Scan that removed 1,926,846 rows to keep 1, with a median of 109.78 ms. With it, the median is 0.06 ms.

The rule: in PostgreSQL, index every foreign key column that you join on or delete through. It's the single most common missing index.

2. Don't wrap the indexed column in a function

"How many orders came in December 2025?" First give order_date an index:

```
CREATE INDEX idx_orders_order_date ON orders (order_date);
```

Now two ways to write the filter, each with its plan:

```
-- version A: extract parts of the date  
EXPLAIN (ANALYZE, COSTS OFF)  
SELECT COUNT(*) FROM orders  
WHERE EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 12;  
  
-- version B: a date range  
EXPLAIN (ANALYZE, COSTS OFF)  
SELECT COUNT(*) FROM orders  
WHERE order_date >= DATE '2025-12-01' AND order_date < DATE '2026-01-01';
```

Without EXPLAIN, both return 25,078. Their plans are very different. Version A's plan:

QUERY PLAN

```
-----  
Aggregate (actual time=131.698..131.701 rows=1 loops=1)  
-> Seq Scan on orders (actual time=123.024..130.388 rows=25078 loops=1)  
  Filter: ((EXTRACT(year FROM order_date) = '2025'::numeric) AND (EXTRACT(month FROM order_date) =  
    '12'::numeric))  
  Rows Removed by Filter: 689207  
Planning Time: 0.432 ms  
Execution Time: 131.749 ms  
(6 rows)
```

Version B's plan:

QUERY PLAN

```
-----  
Aggregate (actual time=2.797..2.797 rows=1 loops=1)  
-> Index Only Scan using idx_orders_order_date on orders (actual time=0.021..1.672 rows=25078  
  loops=1)  
  Index Cond: ((order_date >= '2025-12-01'::date) AND (order_date < '2026-01-01'::date))  
  Heap Fetches: 0  
Planning Time: 0.383 ms  
Execution Time: 2.834 ms  
(6 rows)
```

- **Version A** scans the whole table (median 131.33 ms), although the index exists.
- **Version B** uses the index (median 2.95 ms), about 45 times faster. It's an **Index Only Scan**: `COUNT(*)` needs no column values, so the index alone answers it. **Heap Fetches: 0** means no table page was visited; experiment 4 explains when that's possible.

The index stores dates sorted, so it can find "from 1 December up to (but not including) 1 January" directly. It does not store "the month part of each date", so it can't help with `EXTRACT(MONTH ...) = 12`. A condition the database can answer from an index is called **sargable** (from *search argument*).

The same trap appears in many disguises. Each pair below returns the same rows, but only the second form can use a plain index on the column:

Not sargable (the condition after <code>WHERE</code>)	Sargable
<code>EXTRACT(YEAR FROM order_date) = 2025</code>	<code>order_date >= '2025-01-01' AND order_date < '2026-01-01'</code>
<code>order_date::text LIKE '2025-12%'</code>	the date range above
<code>UPPER(city) = 'PUNE'</code>	store city in one agreed case, then <code>city = 'Pune'</code>
<code>quantity * unit_price > 50000</code>	often unavoidable; see <i>expression indexes</i> below
<code>customer_name LIKE '%Hardware'</code>	<code>LIKE 'Sharma%'</code> can use an index; a leading % can't

The half-open range (`>=` the start, `<` the next start) is also the correct form for timestamps, which `BETWEEN ... AND '2025-12-31'` gets wrong by missing everything after midnight on the 31st (section 12.6's warning).

If a function in the filter is truly needed, you can index the expression itself, an **expression index**. In PostgreSQL: `CREATE INDEX idx_customers_upper_city ON customers (UPPER(city));`. MySQL (8.0.13 and later) needs an extra pair of brackets around the expression: `CREATE INDEX idx_customers_upper_city ON customers ((UPPER(city)))`. Either way, the query must then use exactly the same expression.

3. Composite indexes: order matters

"Customer 4534's orders since 1 January 2025." This filters on two columns. With the indexes you have so far (one on `customer_id`, one on `order_date`), here's the plan:

```
EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_date, status
FROM orders
WHERE customer_id = 4534 AND order_date >= DATE '2025-01-01';
```


QUERY PLAN

```
-----
Bitmap Heap Scan on orders (actual time=10.308..16.677 rows=9301 loops=1)
  Recheck Cond: (customer_id = 4534)
  Filter: (order_date >= '2025-01-01'::date)
  Rows Removed by Filter: 14225
  Heap Blocks: exact=5170
  -> Bitmap Index Scan on idx_orders_customer_id (actual time=1.513..1.513 rows=23526 loops=1)
        Index Cond: (customer_id = 4534)
Planning Time: 0.400 ms
Execution Time: 17.093 ms
(9 rows)
```

The index finds all 23,526 of the customer's orders; the table pages are then read and each order's date checked, and 14,225 are thrown away (23,526 – 9,301). A **composite index** stores several columns sorted together, like a phone book sorted by surname and then first name:

```
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date);
```

The same EXPLAIN now gives:

QUERY PLAN

```
-----
Bitmap Heap Scan on orders (actual time=0.786..7.325 rows=9301 loops=1)
  Recheck Cond: ((customer_id = 4534) AND (order_date >= '2025-01-01'::date))
  Heap Blocks: exact=2047
  -> Bitmap Index Scan on idx_orders_customer_date (actual time=0.516..0.517 rows=9301 loops=1)
        Index Cond: ((customer_id = 4534) AND (order_date >= '2025-01-01'::date))
Planning Time: 0.399 ms
Execution Time: 7.776 ms
(7 rows)
```

Both conditions are now in the Index Cond, there's no Filter line, and only the 9,301 matching orders are fetched, from 2,047 pages instead of 5,170. The median fell from 16.15 ms to 7.78 ms. (For a typical customer such as 2718, with 40 orders, the saving is tiny: the index lets you skip 28 unwanted rows.)

The column order decides which questions an index can answer. An index on (customer_id, order_date) is sorted by customer first:

- ✓ WHERE customer_id = 4534 (the first column alone).
- ✓ WHERE customer_id = 4534 AND order_date >= '2025-01-01' (equality on the first, range on the second).
- ✓ WHERE customer_id = 4534 ORDER BY order_date DESC LIMIT 5 (the rows are already in date order within the customer).
- ✗ WHERE order_date >= '2025-12-01' on its own. The dates are sorted *within each customer*, not overall, like trying to find everyone named Priya in a phone book sorted by surname.

The usual guideline: put columns you test with = first, and a column you test with a range (>=, <, BETWEEN) last.

Watch out: extra indexes are often redundant. Once `(customer_id, order_date)` exists, a separate index on `customer_id` alone is almost never needed, because the composite index answers the same lookups. Dropping the redundant one saves space and speeds up writes.

4. Covering indexes: never touch the table

The query in experiment 3 still read 2,047 table pages, only to fetch `status`. If every column a query needs is inside the index, the database can skip the table altogether. PostgreSQL lets you carry extra columns in an index without sorting by them, using `INCLUDE`. The new index answers everything the plain composite one did, so it replaces it:

```
DROP INDEX idx_orders_customer_date;

CREATE INDEX idx_orders_customer_date_incl
  ON orders (customer_id, order_date) INCLUDE (status);
VACUUM ANALYZE orders;

EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_date, status
FROM orders
WHERE customer_id = 4534 AND order_date >= DATE '2025-01-01';
```

QUERY PLAN

```
-----
Index Only Scan using idx_orders_customer_date_incl on orders (actual time=0.032..1.029 rows=9301
 loops=1)
  Index Cond: ((customer_id = 4534) AND (order_date >= '2025-01-01'::date))
  Heap Fetches: 0
Planning Time: 0.363 ms
Execution Time: 1.374 ms
(5 rows)
```

Index Only Scan with **Heap Fetches: 0**: all 9,301 rows came from the index. The median fell from 7.78 ms (composite index, reading 2,047 table pages) to 1.68 ms. The `VACUUM` matters: PostgreSQL can only skip a table page if it knows every row on it is visible to everyone, and `VACUUM` records that. On a busy table, autovacuum does this in the background.

A covering index is a strong tool for one important, frequent query. It's a poor default, because every included column makes the index bigger.

5. Partial indexes: index only the rows you ask about

The warehouse screen asks one question all day: "Which orders are still pending, oldest first?" Only 3,930 of 714,285 orders are pending. Without help, the plan scans all 714,285 orders, removes 710,355 of them, and sorts the rest: a median of 44.12 ms. An index on `status` would mostly store the word *Delivered*. A **partial index** stores only the rows that match its `WHERE`:

```
CREATE INDEX idx_orders_pending ON orders (order_date) WHERE status = 'Pending';

EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_id, customer_id, order_date
FROM orders
WHERE status = 'Pending'
ORDER BY order_date;
```

QUERY PLAN

```
-----
Index Scan using idx_orders_pending on orders (actual time=0.015..0.596 rows=3930 loops=1)
Planning Time: 0.404 ms
Execution Time: 0.746 ms
(3 rows)
```

The median fell from 44.12 ms to 0.75 ms, and the plan has no sort step because the index already stores the pending orders in date order. There's no `Index Cond` or `Filter` line either: every entry in a partial index already satisfies `status = 'Pending'`, so the planner has nothing left to check. The query's `WHERE` must imply the index's `WHERE`, or the index can't be used. The index is 48 kB, against 5,056 kB for the index on all order dates. As orders move from Pending to Shipped, they drop out of the index automatically. MySQL has no partial indexes (section 28.13).

6. Measure what indexes cost on writes

"How much slower do writes get?" Copy the first 500,000 order lines into two empty tables: one with no indexes at all, and one with four. `CREATE TABLE ... (LIKE order_items)` makes an empty table with the same columns; `INCLUDING INDEXES` also copies `order_items`' two indexes (its primary key and `idx_order_items_order_id` from experiment 1), and two more are added:

```
CREATE TABLE oi_copy_0 (LIKE order_items);
CREATE TABLE oi_copy_4 (LIKE order_items INCLUDING INDEXES);
CREATE INDEX ON oi_copy_4 (product_id);
CREATE INDEX ON oi_copy_4 (order_id, product_id);

INSERT INTO oi_copy_0 SELECT * FROM order_items WHERE order_item_id <= 500000;
INSERT INTO oi_copy_4 SELECT * FROM order_items WHERE order_item_id <= 500000;
```

CREATE INDEX ON with no name lets PostgreSQL choose one. Time each INSERT (DBeaver shows the time under the results; `ch28_perf.py` repeats the whole thing three times). The medians of three rounds:

Table	Median time to insert 500,000 rows
oi_copy_0, no indexes	416 ms
oi_copy_4, four indexes (primary key, order_id, product_id, (order_id, product_id))	2,301 ms

About five and a half times slower. Each index is a sorted structure that must be updated for every row. That's why data loads often **drop indexes, load, and recreate them**, and why "add an index for every column someone might filter on" is bad advice for a table that receives thousands of writes a minute. Drop the copies when you're done: `DROP TABLE oi_copy_0, oi_copy_4;`

The experiments in one table

Question (riverstone_perf)	Before	After	Change made
One customer's orders (40 rows)	38.77 ms	0.18 ms	index on orders.customer_id
One customer's revenue (join)	262.47 ms	0.64 ms	index on order_items.order_id
One order's lines	109.78 ms	0.06 ms	same index
December's order count	131.33 ms	2.95 ms	rewrote EXTRACT as a date range
Big customer's 2025 orders	16.15 ms	7.78 ms	composite index (customer_id, order_date)
Big customer's 2025 orders	7.78 ms	1.68 ms	covering index with INCLUDE (status)
Pending orders, oldest first	44.12 ms	0.75 ms	partial index
Insert 500,000 lines	416 ms	2,301 ms	four indexes (the cost)

A checklist for a slow query

1. **Confirm it's the query.** Time it in DBeaver, which shows the execution time under the results (in `psql`, type `\timing on` first). A slow dashboard can be slow for reasons outside the database.
2. **Run EXPLAIN (ANALYZE, BUFFERS)** on a copy of production-sized data. `BUFFERS` adds how many pages each step read (see the example below the list).
3. **Find the expensive step:** where the time jumps, where `Rows Removed by Filter` is huge, where estimated and actual rows differ by ten times or more.
4. **Estimates far off?** Run `ANALYZE table_name` so the statistics are current, then plan again.
5. **Scan with a selective filter?** Check the filter is sargable; then consider an index (composite if several columns).
6. **Big join without an index on the join column?** Index the foreign key.
7. **Still slow because the question itself reads millions of rows?** No index will fix that. Pre-aggregate (section 28.11).
8. **Measure again**, and write down the before and after. Then check that writes to the table haven't become a problem.

Here is what `BUFFERS` adds, on section 28.5's first query as it ran before `idx_orders_customer_id` existed (recorded output):

```
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF)
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 2718;
```

QUERY PLAN

```
-----
Seq Scan on orders (actual time=0.763..41.560 rows=40 loops=1)
  Filter: (customer_id = 2718)
  Rows Removed by Filter: 714245
  Buffers: shared hit=2421 read=2800
Planning:
  Buffers: shared hit=69
Planning Time: 0.255 ms
Execution Time: 41.611 ms
(8 rows)
```

Buffers: shared hit=2421 read=2800 counts pages: 2,421 were already in PostgreSQL's memory (**hit**), and 2,800 had to be fetched from outside it, from the operating system or the disk (**read**). Together, 5,221: every page of the table. After the index, the same query touched 43 pages (**shared hit=43**). The **Planning:** lines count the pages read while choosing the plan. Run the query twice and the **read** number usually drops, because the first run left the pages in memory; that's one reason to take the median of several runs.

Real-life example: the index that took down checkout. A common story: an analyst notices a slow report and adds an index on a busy production table in the middle of the day. In PostgreSQL, a plain `CREATE INDEX` blocks inserts and updates to the table until it finishes, so on a very large table, orders stop being saved for minutes. `CREATE INDEX CONCURRENTLY` builds the index without blocking writes (slower, and it can't run inside a transaction). Changes to production tables go through a reviewed migration (section 28.12), not a query window.

Good place to stop.

28.7 Normalization, worked properly

Chapter 12 (section 12.1) gave you the intuition: **store each fact once, and link to it**. Normalization turns that intuition into rules you can check, called **normal forms**. Each form removes one kind of problem. You'll take one messy table all the way through the first three, in the lab database.

The starting point: an order sheet

Before Riverstone's order entry was cleaned up, someone kept a tracking sheet of January's orders like this. It's a table in the lab, built from real orders in the one-year database. Start the lab afresh. Run these two lines from your `riverstone_2025` connection, not from a lab connection, because PostgreSQL refuses to drop the database you're connected to:

```
DROP DATABASE IF EXISTS riverstone_lab;  
CREATE DATABASE riverstone_lab;
```

If PostgreSQL answers *database "riverstone_lab" is being accessed by other users*, DBeaver still has Chapter 12's lab connection open: right-click that connection → *Disconnect*, and run the two lines again. (PostgreSQL 13 and later also accept `DROP DATABASE IF EXISTS riverstone_lab WITH (FORCE);`, which closes other connections first.)

Then connect to the new database: in DBeaver, edit your lab connection (or create one) so its *Database* is `riverstone_lab`, as in section 12.13, and open an SQL editor on it. (In `psql`, the same switch is `\c riverstone_lab`.) Now build the order sheet:

```
CREATE TABLE order_sheet (  
  order_id    INTEGER,  
  order_date  DATE,  
  customer    VARCHAR(100),  
  city        VARCHAR(50),  
  sales_rep   VARCHAR(100),  
  products    VARCHAR(300)  
);  
  
INSERT INTO order_sheet VALUES  
(10001, '2025-01-02', 'Patel Kitchenware', 'Ahmedabad', 'Neha Kulkarni', '108 Stackable Bin x10 @290'),  
(10002, '2025-01-05', 'Green Leaf Hotels', 'Pune', 'Farah Khan', '107 Lunch Box Set x55 @380'),  
(10005, '2025-01-20', 'Green Leaf Hotels', 'Pune', 'Farah Khan', '101 Storage Box 10L x15 @430 less  
5%'),  
(10006, '2025-01-21', 'Coastal Foods', 'Chennai', 'Rahul Mehta', '102 Storage Box 25L x25 @750 less  
5%'),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', '101 Storage Box 10L x45 @430; 102  
Storage Box 25L x20 @750; 108 Stackable Bin x20 @290'),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', '101 Storage Box 10L x30 @430; 102  
Storage Box 25L x25 @750; 107 Lunch Box Set x25 @380');
```

It's readable, and for a person it works. For a database it's a dead end. You can count the orders that mention a product:

```
SELECT COUNT(*) AS storage_box_10l_orders  
FROM order_sheet  
WHERE products LIKE '%Storage Box 10L%';
```

```
storage_box_10l_orders  
-----  
3  
  
(1 row)
```

But you can't add up how many Storage Boxes were ordered, or the value of the orders, without taking the text apart. And the `LIKE` is fragile: an order typed as "Storage box 10 L" would be missed.

First normal form (1NF): one value per cell

A table is in **first normal form** when:

1. every cell holds a **single value** (no lists, no "x45 @430" packed into text), and
2. every row is **unique**, identified by a key.

The `products` column breaks rule 1: it holds a list of order lines, each with four facts (product, quantity, price, discount). The fix is one row per order line, with each fact in its own column:

```
CREATE TABLE order_lines_1nf (  
  order_id      INTEGER,  
  order_date    DATE,  
  customer      VARCHAR(100),  
  city          VARCHAR(50),  
  sales_rep     VARCHAR(100),  
  product_id    INTEGER,  
  product_name  VARCHAR(100),  
  quantity      INTEGER,  
  unit_price    NUMERIC(10,2),  
  discount_pct  NUMERIC(5,2),  
  PRIMARY KEY (order_id, product_id)  
);  
  
INSERT INTO order_lines_1nf VALUES  
(10001, '2025-01-02', 'Patel Kitchenware', 'Ahmedabad', 'Neha Kulkarni', 108, 'Stackable Bin', 10, 290, 0),  
(10002, '2025-01-05', 'Green Leaf Hotels', 'Pune', 'Farah Khan', 107, 'Lunch Box Set', 55, 380, 0),  
(10005, '2025-01-20', 'Green Leaf Hotels', 'Pune', 'Farah Khan', 101, 'Storage Box 10L', 15, 430, 5),  
(10006, '2025-01-21', 'Coastal Foods', 'Chennai', 'Rahul Mehta', 102, 'Storage Box 25L', 25, 750, 5),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 101, 'Storage Box 10L', 45, 430, 0),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 102, 'Storage Box 25L', 20, 750, 0),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 108, 'Stackable Bin', 20, 290, 0),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 101, 'Storage Box 10L', 30, 430, 0),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 102, 'Storage Box 25L', 25, 750, 0),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 107, 'Lunch Box Set', 25, 380, 0);
```

The key is (`order_id`, `product_id`): one order can have many products, and a product appears once per order. Now arithmetic works:

```
SELECT COUNT(DISTINCT order_id) AS storage_box_10l_orders,  
       SUM(quantity)           AS units  
FROM order_lines_1nf  
WHERE product_id = 101;
```

```
storage_box_10l_orders | units  
-----+-----  
3 | 90  
(1 row)
```

(15 + 45 + 30 = 90 units. ✓) Most tables you meet in databases are already in 1NF. You'll meet data that breaks it (sometimes called 0NF) mainly in spreadsheets, form exports, and JSON.

The anomalies 1NF still allows

Look at what `order_lines_1nf` repeats: Sharma Hardware's city is stored on six rows, and "Storage Box 10L" is spelled out three times. Section 12.1 named the three problems this causes. Here's the first one, happening:

```
UPDATE order_lines_1nf
SET city = 'Thane'
WHERE order_id = 10008 AND product_id = 101;

SELECT customer, city, COUNT(*) AS lines
FROM order_lines_1nf
WHERE customer = 'Sharma Hardware'
GROUP BY customer, city;
```

customer	city	lines
Sharma Hardware	Mumbai	5
Sharma Hardware	Thane	1

(2 rows)

An **update anomaly**: someone corrected the city on one row, and now the same customer is in two cities. Nothing in the table's design prevents it. The other two are the **insert anomaly** (you can't record a new product until someone orders it, because the product has nowhere to live without an order line) and the **delete anomaly** (delete Patel Kitchenware's only order, and you lose the fact that they're in Ahmedabad).

Second normal form (2NF): every column depends on the whole key

To find the cause, ask of each column: **what does it depend on?** A column **depends on** another (a **functional dependency**, written $A \rightarrow B$) when knowing A tells you B.

- `order_date`, `customer`, `city`, `sales_rep` depend on `order_id` alone. Order 10007 has one date, whichever product you look at.
- `product_name` depends on `product_id` alone.
- `quantity`, `unit_price`, `discount_pct` depend on **both**: they describe one product within one order.

A table is in **second normal form** when it's in 1NF and **no column depends on only part of a composite key**. `order_lines_1nf` has **partial dependencies** (on `order_id` alone, and on `product_id` alone), so it isn't. The fix is to give each group of facts a table keyed by what it depends on:

```
UPDATE order_lines_1nf SET city = 'Mumbai' WHERE order_id = 10008;

CREATE TABLE orders_2nf AS
SELECT DISTINCT order_id, order_date, customer, city, sales_rep
FROM order_lines_1nf;

CREATE TABLE products_2nf AS
SELECT DISTINCT product_id, product_name
FROM order_lines_1nf;

CREATE TABLE order_lines_2nf AS
SELECT order_id, product_id, quantity, unit_price, discount_pct
FROM order_lines_1nf;

SELECT 'orders' AS table_name, COUNT(*) AS row_count FROM orders_2nf
UNION ALL SELECT 'products', COUNT(*) FROM products_2nf
UNION ALL SELECT 'order_lines', COUNT(*) FROM order_lines_2nf;
```

table_name	row_count
orders	6
products	4
order_lines	10

(3 rows)

(The first line undoes the bad update, so the split starts from clean data. `CREATE TABLE ... AS SELECT` copies the data but not keys; in a real design you'd add `PRIMARY KEY` and `REFERENCES` constraints, as Chapter 12 did.)

Six orders, four products, ten order lines. A product's name is now stored once, and a product can exist before anyone orders it.

Watch out: `unit_price` is not a partial dependency here. It's tempting to move `unit_price` to the products table, because every product has a price. But the price *charged on an order* is a fact about that order line: next year's list price will change, and last year's orders must keep what the customer actually paid. The list price belongs in `products`; the charged price belongs in `order_lines`. Riverstone's real tables do exactly this. Normalization is about what the data *means*, which the column names alone won't tell you.

Third normal form (3NF): no column depends on another non-key column

`orders_2nf` still stores `city` on every order. City depends on `order_id`, but only *through* the customer: `order_id` → customer → city. That's a **transitive dependency**. A table is in **third normal form** when it's in 2NF and **no non-key column depends on another non-key column**. A common way to remember 2NF and 3NF together: every non-key column must depend on *the key, the whole key, and nothing but the key*.

Move the customer's facts into their own table, with a **surrogate key** (an id with no business meaning, created only to identify the row), and keep only the id on the order:

```
CREATE TABLE customers_3nf (  
    customer_id INTEGER PRIMARY KEY,  
    customer VARCHAR(100) NOT NULL UNIQUE,  
    city VARCHAR(50)  
);  
  
INSERT INTO customers_3nf  
SELECT ROW_NUMBER() OVER (ORDER BY customer), customer, city  
FROM (SELECT DISTINCT customer, city FROM orders_2nf) AS c;  
  
CREATE TABLE orders_3nf AS  
SELECT o.order_id, o.order_date, c.customer_id, o.sales_rep  
FROM orders_2nf AS o  
JOIN customers_3nf AS c ON c.customer = o.customer;  
  
SELECT * FROM customers_3nf ORDER BY customer_id;
```

customer_id	customer	city
1	Coastal Foods	Chennai
2	Green Leaf Hotels	Pune
3	Patel Kitchenware	Ahmedabad
4	Sharma Hardware	Mumbai

(4 rows)

`ROW_NUMBER() OVER (ORDER BY customer)` (section 13.4) numbers the four customers 1 to 4 in name order, and the numbers become their surrogate keys. It's used here rather than an automatic identity column (section 12.13) because the rows already exist and you choose the order they're numbered in; section 28.9 uses identity columns for tables that are filled as they go. Now Sharma Hardware's city is stored once. The update anomaly from earlier can't happen, because there's only one row to update. (`sales_rep` has the same problem as `city`, since a rep's details depend on the rep, and exercise 9 asks you to fix it.)

Proving nothing was lost

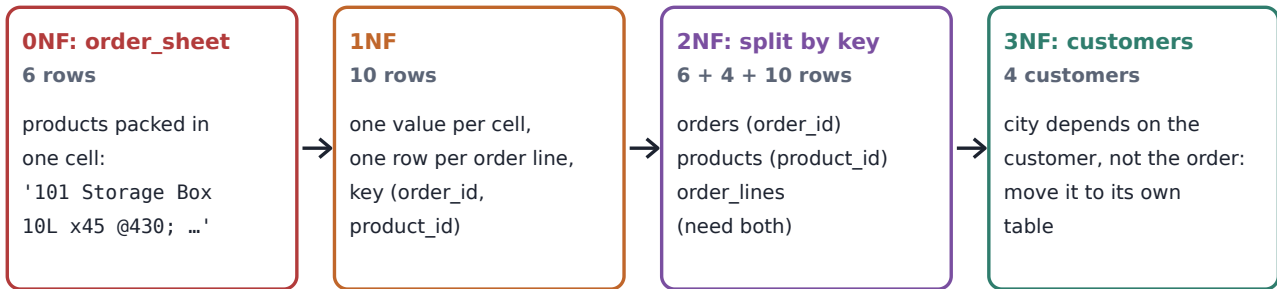
Normalization splits tables without losing information: joining them back must reproduce the original. Reconcile the revenue:

```
SELECT 'flat 1NF table' AS source,  
    ROUND(SUM(quantity * unit_price * (1 - discount_pct / 100)), 2) AS net_revenue  
FROM order_lines_1nf  
UNION ALL  
SELECT '3NF tables joined',  
    ROUND(SUM(l.quantity * l.unit_price * (1 - l.discount_pct / 100)), 2)  
FROM order_lines_2nf AS l  
JOIN orders_3nf AS o ON o.order_id = l.order_id  
JOIN customers_3nf AS c ON c.customer_id = o.customer_id  
JOIN products_2nf AS p ON p.product_id = l.product_id;
```

source	net_revenue
flat 1NF table	129040.00
3NF tables joined (2 rows)	129040.00

Both ₹1,29,040. ✓ Check one line by hand: order 10005 is $15 \times ₹430$ less 5% = ₹6,127.50.

Normalization, one rule at a time (the same six orders)



0NF → 1NF: you can filter and add up. 1NF → 2NF: a product's name is stored once.

2NF → 3NF: a customer's city is stored once, so it can't disagree with itself.

At every step the net revenue still totals ₹1,29,040.

Figure 28.5 — Each normal form removes one kind of repetition. The total never changes.

Beyond 3NF, and when to stop

There are stricter forms. **Boyce–Codd normal form (BCNF)** tightens 3NF for tables with several overlapping candidate keys (a **candidate key** is any column, or set of columns, that could serve as the primary key), and fourth and fifth normal forms deal with multi-valued facts. They're worth knowing by name for interviews; in business databases, a design in 3NF is almost always in BCNF too.

3NF is the standard target for **transactional systems** (the ERP, the CRM), where data is written constantly and must stay consistent. Reporting systems deliberately bend the rules, for speed and simplicity. That's sections 28.9 and 28.11.

Normal form	The rule	Problem it removes	Riverstone fix
1NF	one value per cell; rows unique	can't filter, count, or add	one row per order line
2NF	no column depends on part of the key	product names repeated; can't add a product without an order	products table
3NF	no column depends on a non-key column	customer's city repeated and able to disagree	customers table

28.8 The grain discipline

Chapter 12 (section 12.10) named the idea: the **grain** of a table is what one row represents. Chapter 12 used it to explain fan-out, and promised a whole discipline. Here it is, in four habits.

Habit 1: state the grain in one sentence, before anything else

Every table, view, and query result should have a grain you can say out loud:

Table	Grain	Key that proves it
orders	one order	order_id
order_items	one product line within one order	order_item_id, and (order_id, product_id) should also be unique
sales_lines (view)	one non-cancelled order line	order_id, product_id
sales_targets	one calendar month	target_month
lead_stage_history	one lead entering one stage	(lead_id, stage)
staff	one person	staff_id

If you can't write the sentence, you don't understand the table yet, and any total you compute from it is a guess.

Habit 2: test the grain, don't assume it

A grain is a claim about the data, and data breaks claims. The test is a `GROUP BY` on the columns that should be unique. Back in `riverstone_2025`:

```
SELECT order_id, product_id, COUNT(*) AS copies
FROM order_items
GROUP BY order_id, product_id
HAVING COUNT(*) > 1;
```

```
order_id | product_id | copies
-----+-----+-----
(0 rows)
```

No rows: each product appears at most once per order, so the grain holds. Chapter 13's Pattern 3 found the opposite in `leads`, where 43 rows held 30 real leads. Chapter 47 turns tests like this into automated checks that run every day, and Chapter 32 writes them as dbt tests.

Habit 3: never join tables of different grains and then add

Here's the trap in its most common business form. "Show January's order lines next to January's target." The target's grain is one month; the lines' grain is one order line. Join them and sum the target:

```
SELECT ROUND(SUM(t.target_revenue)) AS target_summed_after_join
FROM sales_lines AS sl
JOIN sales_targets AS t
  ON t.target_month = DATE_TRUNC('month', sl.order_date)
WHERE sl.order_date < DATE '2025-02-01';
```

```
target_summed_after_join
-----
4500000
(1 row)
```

January's target is ₹3,00,000, not ₹45,00,000. The join copied the one monthly target onto each of January's 15 order lines, and SUM added all 15 copies ($15 \times ₹3,00,000$). The query ran without error and returned a believable-looking number. This is the fan-out trap from section 12.10 in a new form, and it's the reason for habit 4.

Habit 4: bring both sides to the same grain, then join

Aggregate the finer table up to the coarser grain first, then join one row to one row:

```
WITH monthly_sales AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month,  
           SUM(net_revenue) AS revenue  
    FROM sales_lines  
    GROUP BY 1  
)  
SELECT m.month,  
       ROUND(m.revenue) AS revenue,  
       ROUND(t.target_revenue) AS target,  
       ROUND(100 * m.revenue / t.target_revenue, 1) AS pct_of_target  
FROM monthly_sales AS m  
JOIN sales_targets AS t ON t.target_month = m.month  
WHERE m.month < DATE '2025-04-01'  
ORDER BY m.month;
```

month	revenue	target	pct_of_target
2025-01-01	202640	300000	67.5
2025-02-01	253664	300000	84.6
2025-03-01	278008	320000	86.9

(3 rows)

January: ₹2,02,640 of ₹3,00,000 is 67.5%. ✓ Both sides now have the grain "one month", so the join can't multiply anything.

Interview extra point. When an interviewer gives you two tables and a question, say the grain of each table before you write a join: "orders is one row per order, order_items is one row per line, so if I join them I'll aggregate the lines first." It shows the habit that prevents the most common wrong answer, and it's the kind of move Part 8's SQL bank (Chapter 71) scores as an extra point.

Good place to stop.

28.9 Dimensional modeling: facts, dimensions, and the star schema

Chapter 16 (section 16.3) built a star schema inside Power BI: a fact table, dimensions around it, and a date table. This section builds one in SQL, from the design method up, and adds what Power BI kept out of sight: surrogate keys, history (slowly changing dimensions), and loading. The first few ideas are a recap.

Normalized tables are built for *writing*: each fact once, so updates are safe. Analysts mostly *read*, and reading a normalized database means long chains of joins, repeated in every report and often subtly wrong. **Dimensional modeling**, popularized by Ralph Kimball, organizes the same data for reading. It's how most data warehouses, Power BI models (Chapter 16), and dbt marts (Chapter 32) are laid out.

Facts and dimensions

Every business question has two kinds of words in it. "What was **net revenue** by **customer segment** and **quarter**?"

- **Facts** (or **measures**) are the numbers you add up: net revenue, quantity, cost. They come from business events: an order line, a payment, a delivery.
- **Dimensions** are the words you slice by: segment, quarter, product category, sales rep. They describe the *who*, *what*, *where*, and *when* of each event.

A **fact table** holds one row per event at a declared grain, with the measures and a key for each dimension. A **dimension table** holds one row per member of the dimension (one per customer, one per day), with every descriptive column a report might need. Arrange the dimensions around the fact table and draw lines to it, and the diagram looks like a star: a **star schema**.

The four design steps

Kimball's method is four questions, asked in order:

1. **Choose the business process.** Not "sales data" but a specific event: *taking orders*.
2. **Declare the grain.** One row per non-cancelled order line. This is section 28.8's habit, and it comes before choosing any columns.
3. **Identify the dimensions.** What describes one order line? When (date), who bought (customer), what (product), who sold (sales rep).
4. **Identify the facts.** What numbers belong to one order line? Quantity, net revenue, product cost.

The answers give Riverstone's first star schema. You'll build it in a schema called `dw` (for *data warehouse*) inside `riverstone_2025`, next to the source tables, as section 12.13's schemas lesson suggested.

Riverstone's 2025 sales as a star schema (schema dw)

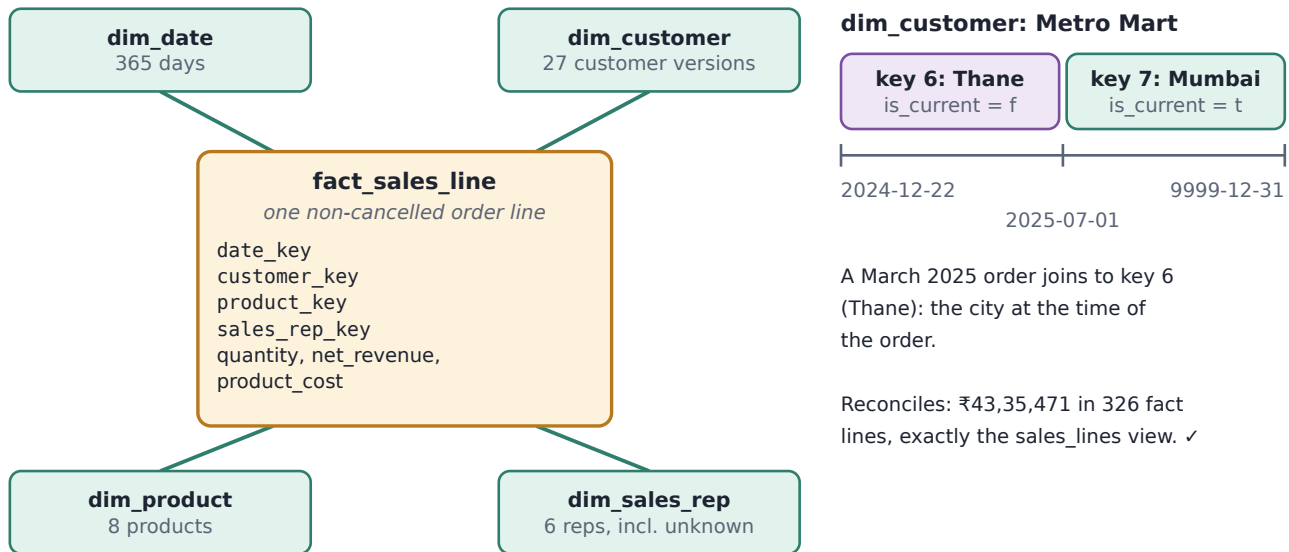


Figure 28.6 — The fact table holds the events; each dimension describes one side of them. Metro Mart has two customer rows because it moved.

The date dimension

The most useful dimension, and the one people most often skip, is a **date dimension**: one row per calendar day, with every way the business describes a date already worked out.

```
DROP SCHEMA IF EXISTS dw CASCADE;
CREATE SCHEMA dw;

CREATE TABLE dw.dim_date (
    date_key      INTEGER PRIMARY KEY,
    full_date     DATE NOT NULL UNIQUE,
    day_name      VARCHAR(9) NOT NULL,
    month_start   DATE NOT NULL,
    month_name    VARCHAR(9) NOT NULL,
    quarter_label CHAR(7) NOT NULL,
    is_weekend    BOOLEAN NOT NULL
);

INSERT INTO dw.dim_date
SELECT TO_CHAR(d, 'YYYYMMDD')::INTEGER,
       d::date,
       TRIM(TO_CHAR(d, 'Day')),
       DATE_TRUNC('month', d)::date,
       TRIM(TO_CHAR(d, 'Month')),
       TO_CHAR(d, 'YYYY-"Q"Q'),
       EXTRACT(ISODOW FROM d) IN (6, 7)
FROM generate_series(DATE '2025-01-01', DATE '2025-12-31', INTERVAL '1 day') AS d;
```

How it works:

- **DROP SCHEMA IF EXISTS dw CASCADE** drops the schema *and everything in it*: CASCADE means "and every table inside". That makes the build safe to rerun, and it's safe here only because dw holds nothing but this chapter's build. Never write CASCADE against a schema you didn't create.

- **generate_series** (Chapter 13, Pattern 5) produces every day of 2025, including days with no orders, so a report on "orders by weekday" or "days without sales" has every day to count. It returns timestamps, so `d::date` turns each one into a plain date.
- **date_key** is a whole number like 20250315: `TO_CHAR(d, 'YYYYMMDD')` writes the date as the text '20250315', and `::INTEGER` turns that text into a number. It's readable, sorts correctly, and joins fast. Using a meaningful number for dates is a common exception to the rule that surrogate keys carry no meaning.
- `TO_CHAR(d, 'Day')` gives the weekday name, padded with spaces to nine characters ('Saturday '), so **TRIM** removes the padding; 'Month' works the same way.
- `'YYYY-"Q"Q'`: inside a `TO_CHAR` format, text in double quotes is printed as it is, so "Q" prints the letter Q, and the plain Q is the quarter number: 2025-Q1.
- **EXTRACT(ISODOW FROM d)** numbers the weekdays Monday = 1 to Sunday = 7, so `IN (6, 7)` is true on Saturdays and Sundays.

Each descriptive column is computed once here instead of in every report. Real date dimensions add fiscal year (April to March, for most Indian companies), holiday flags, and week numbers. When finance decides that the festive season starts in October, you change one table, not fifty reports. Check three rows:

```
SELECT *
FROM dw.dim_date
WHERE full_date IN ('2025-01-01', '2025-03-15', '2025-12-31')
ORDER BY full_date;
```

date_key	full_date	day_name	month_start	month_name	quarter_label	is_weekend
20250101	2025-01-01	Wednesday	2025-01-01	January	2025-Q1	f
20250315	2025-03-15	Saturday	2025-03-01	March	2025-Q1	t
20251231	2025-12-31	Wednesday	2025-12-01	December	2025-Q4	f

(3 rows)

Product and sales rep dimensions

```
CREATE TABLE dw.dim_product (
    product_key  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id   INTEGER NOT NULL UNIQUE,
    product_name VARCHAR(100) NOT NULL,
    category     VARCHAR(30) NOT NULL
);
INSERT INTO dw.dim_product (product_id, product_name, category)
SELECT product_id, product_name, category FROM products ORDER BY product_id;

CREATE TABLE dw.dim_sales_rep (
    sales_rep_key  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id    INTEGER UNIQUE,
    rep_name       VARCHAR(100) NOT NULL,
    job_title      VARCHAR(50) NOT NULL
);
INSERT INTO dw.dim_sales_rep (employee_id, rep_name, job_title)
VALUES (NULL, 'No rep recorded', 'Unknown');
INSERT INTO dw.dim_sales_rep (employee_id, rep_name, job_title)
SELECT employee_id, employee_name, job_title FROM employees ORDER BY employee_id;
```

Each dimension gets its own **surrogate key** (`product_key`, `sales_rep_key`), separate from the source system's id. `INTEGER GENERATED ALWAYS AS IDENTITY` is section 12.13's automatic numbering: the database numbers the rows 1, 2, 3... as they arrive, which is why neither `INSERT` lists the key column. (You'll often see `SERIAL` in older PostgreSQL code; it's the older shorthand for the same thing.) Section 28.10 shows why a separate key is essential for customers. The sales rep dimension also has a row for "No rep recorded", inserted first so it gets key 1, because 11 orders have no rep (10 of them not cancelled); the fact table can then always point to a real dimension row, and reports show an honest "No rep recorded" instead of silently dropping those orders in an inner join.

```
SELECT * FROM dw.dim_sales_rep ORDER BY sales_rep_key;
```

sales_rep_key	employee_id	rep_name	job_title
1		No rep recorded	Unknown
2	1	Anita Rao	Sales Head
3	2	Vikram Singh	Sales Manager
4	3	Neha Kulkarni	Sales Executive
5	4	Rahul Mehta	Sales Executive
6	5	Farah Khan	Sales Executive

(6 rows)

The customer dimension

Customers are harder, because they change: in 2025 Patel Kitchenware was reclassified from Wholesale to Retail, Metro Mart moved from Thane to Mumbai, and Harbour Traders moved from Retail to Wholesale. The ERP's `customers` table only holds the values as they are now; the history is in `customer_changes`:

```
SELECT change_id, customer_id, changed_on, column_name, old_value, new_value
FROM customer_changes
ORDER BY changed_on;
```

change_id	customer_id	changed_on	column_name	old_value	new_value
1	2	2025-04-01	segment	Wholesale	Retail
2	5	2025-07-01	city	Thane	Mumbai
3	11	2025-09-01	segment	Retail	Wholesale

(3 rows)

The dimension keeps a separate row for each version of each customer. Section 28.10 explains the design (it's a *type 2 slowly changing dimension*); for now, read it as "one row per customer per period in which their details stayed the same". First the table:

```
CREATE TABLE dw.dim_customer (  
  customer_key  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  customer_id   INTEGER NOT NULL,  
  customer_name VARCHAR(100) NOT NULL,  
  city          VARCHAR(50),  
  segment      VARCHAR(30) NOT NULL,  
  valid_from    DATE NOT NULL,  
  valid_to      DATE NOT NULL,  
  is_current    BOOLEAN NOT NULL  
);
```

Filling it takes one long query. Build it in four steps, each run on its own and checked on the three customers who changed (2, 5, and 11), before putting it into the table.

Step 1: when could each version start? On the signup date, and on every change date:

```
WITH version_starts AS (  
  SELECT customer_id, signup_date AS valid_from FROM customers  
  UNION  
  SELECT customer_id, changed_on FROM customer_changes  
)  
SELECT customer_id, valid_from  
FROM version_starts  
WHERE customer_id IN (2, 5, 11)  
ORDER BY customer_id, valid_from;
```

```
customer_id | valid_from  
-----+-----  
          2 | 2024-12-15  
          2 | 2025-04-01  
          5 | 2024-12-22  
          5 | 2025-07-01  
         11 | 2025-04-09  
         11 | 2025-09-01  
(6 rows)
```

Each changed customer has two start dates: signup and the change. `UNION` (not `UNION ALL`) removes duplicates, so if a customer changed on the day they signed up, the two identical start dates would become one, not two versions of one day.

Step 2: when does each version end? On the day the next one starts:

```
WITH version_starts AS (
  SELECT customer_id, signup_date AS valid_from FROM customers
  UNION
  SELECT customer_id, changed_on FROM customer_changes
),
versions AS (
  SELECT customer_id,
         valid_from,
         LEAD(valid_from, 1, DATE '9999-12-31')
           OVER (PARTITION BY customer_id ORDER BY valid_from) AS valid_to
  FROM version_starts
)
SELECT customer_id, valid_from, valid_to
FROM versions
WHERE customer_id IN (2, 5, 11)
ORDER BY customer_id, valid_from;
```

customer_id	valid_from	valid_to
2	2024-12-15	2025-04-01
2	2025-04-01	9999-12-31
5	2024-12-22	2025-07-01
5	2025-07-01	9999-12-31
11	2025-04-09	2025-09-01
11	2025-09-01	9999-12-31

(6 rows)

LEAD (section 13.5) looks at the next row of the customer's own versions. Section 13.5 used it with one argument; here it has three. The second, 1, says how many rows ahead to look. The third, DATE '9999-12-31', is the value to use when there is no next row: the latest version gets this far-future date, which means "still current".

Step 3: which city was in effect? For each version, find the latest city change on or before its start. Try it by hand for Metro Mart (customer 5), whose city changed from Thane to Mumbai on 1 July 2025:

```
SELECT
  (SELECT ch.new_value FROM customer_changes AS ch
   WHERE ch.customer_id = 5 AND ch.column_name = 'city'
   AND ch.changed_on <= DATE '2025-07-01'
   ORDER BY ch.changed_on DESC LIMIT 1) AS city_on_2025_07_01,
  (SELECT ch.new_value FROM customer_changes AS ch
   WHERE ch.customer_id = 5 AND ch.column_name = 'city'
   AND ch.changed_on <= DATE '2024-12-22'
   ORDER BY ch.changed_on DESC LIMIT 1) AS city_on_2024_12_22,
  (SELECT ch.old_value FROM customer_changes AS ch
   WHERE ch.customer_id = 5 AND ch.column_name = 'city'
   ORDER BY ch.changed_on LIMIT 1) AS city_before_first_change;
```

city_on_2025_07_01	city_on_2024_12_22	city_before_first_change
Mumbai		Thane

(1 row)

How it works: each column is a scalar subquery (section 12.12), a query that returns one value.

- `ORDER BY ch.changed_on DESC LIMIT 1` keeps the latest change, and `ch.changed_on <= ...` keeps only changes on or before the date. Together they mean "the latest change on or before this date". For the version starting 1 July 2025, that's the move itself, so its `new_value` is Mumbai.
- For the version starting 22 December 2024 there was no change yet, so the subquery finds nothing and returns NULL.
- For that case, the third subquery takes the *old* value of the customer's first change (`ORDER BY ch.changed_on, ascending`): the city before anything changed, Thane.
- A customer who never changed has no rows in `customer_changes` at all; then the city in `customers` is right.

Step 4: put the pieces together. `COALESCE` (section 12.7) takes the first of those three answers that isn't NULL. The same three-way fallback finds the segment:

```
WITH version_starts AS (
    SELECT customer_id, signup_date AS valid_from FROM customers
    UNION
    SELECT customer_id, changed_on FROM customer_changes
),
versions AS (
    SELECT customer_id,
           valid_from,
           LEAD(valid_from, 1, DATE '9999-12-31')
             OVER (PARTITION BY customer_id ORDER BY valid_from) AS valid_to
    FROM version_starts
)
SELECT v.customer_id,
       c.customer_name,
       COALESCE(
           (SELECT ch.new_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'city'
              AND ch.changed_on <= v.valid_from
            ORDER BY ch.changed_on DESC LIMIT 1),
           (SELECT ch.old_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'city'
            ORDER BY ch.changed_on LIMIT 1),
           c.city)
       AS city,
       COALESCE(
           (SELECT ch.new_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'segment'
              AND ch.changed_on <= v.valid_from
            ORDER BY ch.changed_on DESC LIMIT 1),
           (SELECT ch.old_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'segment'
            ORDER BY ch.changed_on LIMIT 1),
           c.segment)
       AS segment,
       v.valid_from,
       v.valid_to,
       v.valid_to = DATE '9999-12-31'
       AS is_current
FROM versions AS v
JOIN customers AS c ON c.customer_id = v.customer_id
WHERE v.customer_id IN (2, 5, 11)
ORDER BY v.customer_id, v.valid_from;
```

customer_id	customer_name	city	segment	valid_from	valid_to	is_current
2	Patel Kitchenware	Ahmedabad	Wholesale	2024-12-15	2025-04-01	f
2	Patel Kitchenware	Ahmedabad	Retail	2025-04-01	9999-12-31	t
5	Metro Mart	Thane	Retail	2024-12-22	2025-07-01	f
5	Metro Mart	Mumbai	Retail	2025-07-01	9999-12-31	t
11	Harbour Traders	Kochi	Retail	2025-04-09	2025-09-01	f
11	Harbour Traders	Kochi	Wholesale	2025-09-01	9999-12-31	t

(6 rows)

Patel Kitchenware is Wholesale until 1 April 2025 and Retail from then on; Metro Mart is in Thane until 1 July. The last column is new: **v.valid_to = DATE '9999-12-31' AS is_current**. A comparison is itself a value, true or false, so it can be a column: true for the version that hasn't ended.

Step 5: load the table. `INSERT INTO ... (columns)` can take any query, including one that starts with `WITH`. Drop the test filter and insert every customer's versions:

```
INSERT INTO dw.dim_customer
(customer_id, customer_name, city, segment, valid_from, valid_to, is_current)
WITH version_starts AS (
  SELECT customer_id, signup_date AS valid_from FROM customers
  UNION
  SELECT customer_id, changed_on FROM customer_changes
),
versions AS (
  SELECT customer_id,
         valid_from,
         LEAD(valid_from, 1, DATE '9999-12-31')
           OVER (PARTITION BY customer_id ORDER BY valid_from) AS valid_to
  FROM version_starts
)
SELECT v.customer_id,
       c.customer_name,
       COALESCE(
         (SELECT ch.new_value FROM customer_changes AS ch
          WHERE ch.customer_id = v.customer_id AND ch.column_name = 'city'
            AND ch.changed_on <= v.valid_from
          ORDER BY ch.changed_on DESC LIMIT 1),
         (SELECT ch.old_value FROM customer_changes AS ch
          WHERE ch.customer_id = v.customer_id AND ch.column_name = 'city'
            ORDER BY ch.changed_on LIMIT 1),
         c.city) AS city,
       COALESCE(
         (SELECT ch.new_value FROM customer_changes AS ch
          WHERE ch.customer_id = v.customer_id AND ch.column_name = 'segment'
            AND ch.changed_on <= v.valid_from
          ORDER BY ch.changed_on DESC LIMIT 1),
         (SELECT ch.old_value FROM customer_changes AS ch
          WHERE ch.customer_id = v.customer_id AND ch.column_name = 'segment'
            ORDER BY ch.changed_on LIMIT 1),
         c.segment) AS segment,
       v.valid_from,
       v.valid_to,
       v.valid_to = DATE '9999-12-31' AS is_current
FROM versions AS v
JOIN customers AS c ON c.customer_id = v.customer_id
ORDER BY v.customer_id, v.valid_from;
```

`customer_key` isn't listed, so the identity column numbers the rows in the order they arrive, customer by customer and oldest version first.

The fact table

```
CREATE TABLE dw.fact_sales_line (  
    order_item_id  INTEGER PRIMARY KEY,  
    order_id       INTEGER NOT NULL,  
    date_key       INTEGER NOT NULL REFERENCES dw.dim_date (date_key),  
    customer_key   INTEGER NOT NULL REFERENCES dw.dim_customer (customer_key),  
    product_key    INTEGER NOT NULL REFERENCES dw.dim_product (product_key),  
    sales_rep_key  INTEGER NOT NULL REFERENCES dw.dim_sales_rep (sales_rep_key),  
    quantity       INTEGER NOT NULL,  
    net_revenue    NUMERIC(12,2) NOT NULL,  
    product_cost   NUMERIC(12,2) NOT NULL  
);  
  
INSERT INTO dw.fact_sales_line  
SELECT oi.order_item_id,  
       o.order_id,  
       TO_CHAR(o.order_date, 'YYYYMMDD')::INTEGER,  
       dc.customer_key,  
       dp.product_key,  
       dr.sales_rep_key,  
       oi.quantity,  
       oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100),  
       oi.quantity * p.unit_cost  
FROM orders AS o  
JOIN order_items AS oi ON oi.order_id = o.order_id  
JOIN products   AS p  ON p.product_id = oi.product_id  
JOIN dw.dim_customer AS dc  
    ON dc.customer_id = o.customer_id  
    AND o.order_date >= dc.valid_from AND o.order_date < dc.valid_to  
JOIN dw.dim_product AS dp ON dp.product_id = oi.product_id  
JOIN dw.dim_sales_rep AS dr  
    ON dr.employee_id IS NOT DISTINCT FROM o.sales_rep_id  
WHERE o.status <> 'Cancelled';
```

The critical line is the join to `dw.dim_customer`: `o.order_date >= dc.valid_from AND o.order_date < dc.valid_to`. Each order line gets the key of the customer version **in effect on the order date**. Once that key is stored, the fact row never needs to look at the date range again. `IS NOT DISTINCT FROM` is a null-safe equals: unlike `=`, it treats two NULLs as equal, so orders with no rep match the "No rep recorded" row.

Check the build

A warehouse you haven't reconciled is a warehouse nobody should trust. Three checks:

```
SELECT 'dim_date' AS table_name, COUNT(*) AS row_count FROM dw.dim_date  
UNION ALL SELECT 'dim_customer',    COUNT(*) FROM dw.dim_customer  
UNION ALL SELECT 'dim_product',     COUNT(*) FROM dw.dim_product  
UNION ALL SELECT 'dim_sales_rep',   COUNT(*) FROM dw.dim_sales_rep  
UNION ALL SELECT 'fact_sales_line', COUNT(*) FROM dw.fact_sales_line;
```

table_name	row_count
dim_date	365
dim_customer	27
dim_product	8
dim_sales_rep	6
fact_sales_line	326
(5 rows)	

365 days. 27 customer rows: 24 customers, plus one extra version for each of the three who changed.

✓

```
SELECT customer_key, customer_name, city, segment, valid_from, valid_to, is_current
FROM dw.dim_customer
WHERE customer_id IN (2, 5, 11)
ORDER BY customer_id, valid_from;
```

customer_key	customer_name	city	segment	valid_from	valid_to	is_current
2	Patel Kitchenware	Ahmedabad	Wholesale	2024-12-15	2025-04-01	f
3	Patel Kitchenware	Ahmedabad	Retail	2025-04-01	9999-12-31	t
6	Metro Mart	Thane	Retail	2024-12-22	2025-07-01	f
7	Metro Mart	Mumbai	Retail	2025-07-01	9999-12-31	t
13	Harbour Traders	Kochi	Retail	2025-04-09	2025-09-01	f
14	Harbour Traders	Kochi	Wholesale	2025-09-01	9999-12-31	t

(6 rows)

Each changed customer has one closed version and one current version, and each version ends on the day the next begins. ✓

```
SELECT (SELECT ROUND(SUM(net_revenue)) FROM sales_lines) AS view_total,
       (SELECT ROUND(SUM(net_revenue)) FROM dw.fact_sales_line) AS fact_total,
       (SELECT COUNT(*) FROM sales_lines) AS view_lines,
       (SELECT COUNT(*) FROM dw.fact_sales_line) AS fact_lines;
```

view_total	fact_total	view_lines	fact_lines
4335471	4335471	326	326

(1 row)

₹43,35,471 and 326 lines in both. ✓ If any order line had failed to find a customer version (a gap between one version's end and the next one's start, for instance), the inner join would have dropped it and the fact total would be lower. This one query catches that.

Querying the star

Every report now has the same simple shape: the fact table, joined to the dimensions you need, grouped by their columns.

```
SELECT d.quarter_label,
       c.segment,
       ROUND(SUM(f.net_revenue)) AS net_revenue
FROM dw.fact_sales_line AS f
JOIN dw.dim_date AS d ON d.date_key = f.date_key
JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
GROUP BY d.quarter_label, c.segment
ORDER BY d.quarter_label, c.segment;
```

quarter_label	segment	net_revenue
2025-Q1	Hospitality	149040
2025-Q1	Retail	356290
2025-Q1	Wholesale	228982
2025-Q2	Hospitality	252170
2025-Q2	Retail	271031
2025-Q2	Wholesale	203367
2025-Q3	Hospitality	307278
2025-Q3	Retail	384294
2025-Q3	Wholesale	428718
2025-Q4	Hospitality	435551
2025-Q4	Retail	562000
2025-Q4	Wholesale	756751

(12 rows)

No `DATE_TRUNC`, no `CASE`, no joins through orders to reach customers. The quarters add up to the year: Q4 alone is ₹17,54,302 (4,35,551 + 5,62,000 + 7,56,751), 40.5% of the total, which matches Chapter 13's festive-season finding.

Star or snowflake?

In a star schema, each dimension is one wide table, even if that repeats values: `category` is stored on every product row. A **snowflake schema** normalizes the dimensions too (`dim_product` points to a `dim_category` table), so the diagram branches like a snowflake.

	Star	Snowflake
Dimension tables	one wide table per dimension	split into sub-dimensions
Joins in a report	one per dimension	more
Repetition in dimensions	yes, deliberately	less
Easier for BI tools and business users	yes	no
Typical use	most warehouses and Power BI models	very large dimensions with many shared levels

Dimension tables are small (Riverstone's biggest has 27 rows; a large company's customer dimension might have a few million), so the repetition costs little, and the simpler joins help everyone who writes reports. **Default to a star.** Power BI's documentation recommends star schemas for its models, which is why Chapter 16's model looked like this one.

Other kinds of fact tables

`fact_sales_line` is a **transaction fact table**: one row per event, added once and never changed. Two other kinds cover questions it can't answer well:

- A **periodic snapshot** records the state of something at regular intervals: stock on hand at each warehouse at the end of each day, or receivables outstanding at each month-end. You can't add snapshots across time (adding daily stock levels makes no sense), but you can average them or take the latest.
- An **accumulating snapshot** has one row per process instance, updated as it moves through fixed milestones: one row per order with the dates it was placed, invoiced, shipped, delivered, and paid. It's the natural home for Chapter 3's order-to-cash durations.

And one more design term you'll hear: **conformed dimensions** are dimensions shared by several fact tables with the same keys and meanings. If the sales facts and the payments facts both use the same `dim_customer` and `dim_date`, a report can put revenue and collections side by side and trust that "Retail, 2025-Q3" means the same thing in both.

28.10 Slowly changing dimensions

Customers move, get reclassified, change their names. Products move between categories. Sales reps change teams. Dimensions change slowly (compared with facts, which arrive every minute), and how you handle those changes decides whether last year's reports still say what they said last year. The standard catalog of techniques is called **slowly changing dimension (SCD) types**. Three matter in practice.

To see each one clearly, go back to the lab and use the example from section 12.1: suppose Sharma Hardware moves from Mumbai to Thane, effective 1 April 2026. A nightly extract from the ERP (`customer_extract`) brings the new city, along with a customer the lab's small dimension doesn't hold yet, Coastal Foods. (The ids are the `customer_ids` from `riverstone_2025`: Sharma Hardware 1, Green Leaf Hotels 3, Coastal Foods 4. They are not the surrogate keys `customers_3nf` gave in section 28.7.)

```
CREATE TABLE dim_customer_t1 (  
    customer_id INTEGER PRIMARY KEY,  
    customer    VARCHAR(100) NOT NULL,  
    city        VARCHAR(50),  
    segment     VARCHAR(30)  
);  
  
INSERT INTO dim_customer_t1 VALUES  
(1, 'Sharma Hardware', 'Mumbai', 'Retail'),  
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality');  
  
CREATE TABLE customer_extract (  
    customer_id INTEGER PRIMARY KEY,  
    customer    VARCHAR(100) NOT NULL,  
    city        VARCHAR(50),  
    segment     VARCHAR(30)  
);  
  
INSERT INTO customer_extract VALUES  
(1, 'Sharma Hardware', 'Thane', 'Retail'),  
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality'),
```



```
(4, 'Coastal Foods', 'Chennai', 'Wholesale');
```

Type 1: overwrite

Type 1 replaces the old value with the new one. No history is kept. It's right for **corrections** (a misspelled name, a wrong PIN code) and for attributes nobody reports history on.

The tidiest way to apply an extract is **MERGE** (PostgreSQL 15 and later), which updates rows that match and inserts rows that don't, in one statement:

```
MERGE INTO dim_customer_t1 AS d
USING customer_extract AS x
ON d.customer_id = x.customer_id
WHEN MATCHED AND (d.city IS DISTINCT FROM x.city OR d.segment IS DISTINCT FROM x.segment) THEN
    UPDATE SET city = x.city, segment = x.segment
WHEN NOT MATCHED THEN
    INSERT (customer_id, customer, city, segment)
    VALUES (x.customer_id, x.customer, x.city, x.segment);

SELECT * FROM dim_customer_t1 ORDER BY customer_id;
```

customer_id	customer	city	segment
1	Sharma Hardware	Thane	Retail
3	Green Leaf Hotels	Pune	Hospitality
4	Coastal Foods	Chennai	Wholesale

(3 rows)

How it works: USING names the incoming data and ON says how to match it. WHEN MATCHED AND ... updates only rows that really changed (IS DISTINCT FROM treats two NULLs as equal, so unchanged NULLs don't count as changes). WHEN NOT MATCHED inserts new customers. Sharma Hardware is now in Thane, and **every past sale of Sharma Hardware now reports as Thane**, including sales made in Mumbai in 2025. For a correction, that's exactly right. For a real move, it rewrites history.

Type 2: add a new row

Type 2 keeps history by closing the current row and inserting a new one. Each row is a *version*, with its own surrogate key and a validity period. This is what `dw.dim_customer` did in section 28.9.

```
CREATE TABLE dim_customer_t2 (
    customer_key INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id  INTEGER NOT NULL,
    customer     VARCHAR(100) NOT NULL,
    city         VARCHAR(50),
    segment      VARCHAR(30),
    valid_from   DATE NOT NULL,
    valid_to     DATE NOT NULL DEFAULT '9999-12-31',
    is_current   BOOLEAN NOT NULL DEFAULT TRUE
);

INSERT INTO dim_customer_t2 (customer_id, customer, city, segment, valid_from) VALUES
(1, 'Sharma Hardware', 'Mumbai', 'Retail', '2024-11-04'),
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality', '2024-09-08');

CREATE UNIQUE INDEX one_current_row ON dim_customer_t2 (customer_id) WHERE is_current;
```

The columns that make it type 2:

- **customer_key**: the surrogate key. `customer_id` (the **natural key** or **business key** from the ERP) is no longer unique, because one customer can have several versions. Facts store `customer_key`, which points to exactly one version.
- **valid_from** and **valid_to**: the period the version was true. A half-open period (from `valid_from`, up to but not including `valid_to`) means consecutive versions never overlap and never leave a gap.
- **is_current**: a convenience flag for "the latest version".
- The **partial unique index** (section 28.6) makes the database enforce the rule that each customer has at most one current row.

Applying the extract is two steps, in one transaction so nobody ever sees a customer with no current row:

```
BEGIN;

-- step 1: close the current version of customers whose details changed
UPDATE dim_customer_t2 AS d
SET valid_to = DATE '2026-04-01',
    is_current = FALSE
FROM customer_extract AS x
WHERE d.customer_id = x.customer_id
    AND d.is_current
    AND (d.city IS DISTINCT FROM x.city OR d.segment IS DISTINCT FROM x.segment);

-- step 2: insert a new current version for every customer without one
INSERT INTO dim_customer_t2 (customer_id, customer, city, segment, valid_from)
SELECT x.customer_id, x.customer, x.city, x.segment, DATE '2026-04-01'
FROM customer_extract AS x
LEFT JOIN dim_customer_t2 AS d
    ON d.customer_id = x.customer_id AND d.is_current
WHERE d.customer_id IS NULL;

COMMIT;

SELECT customer_key, customer_id, customer, city, valid_from, valid_to, is_current
FROM dim_customer_t2
ORDER BY customer_id, valid_from;
```

customer_key	customer_id	customer	city	valid_from	valid_to	is_current
1	1	Sharma Hardware	Mumbai	2024-11-04	2026-04-01	f
3	1	Sharma Hardware	Thane	2026-04-01	9999-12-31	t
2	3	Green Leaf Hotels	Pune	2024-09-08	9999-12-31	t
4	4	Coastal Foods	Chennai	2026-04-01	9999-12-31	t

(4 rows)

How it works: step 1 closes Sharma Hardware's Mumbai version. Its `valid_to` is set to 1 April 2026, the day the new version starts: with half-open periods, `valid_to` is the first day the version is *no longer* true, so the Mumbai version covers everything up to and including 31 March. `UPDATE ... FROM` (section 12.13's dialect table) lets the `UPDATE` read a second table, here the extract. Green Leaf Hotels didn't change, so it's left alone. Step 2 then looks for extract rows with no current version: that's Sharma Hardware (whose current version step 1 closed) and Coastal Foods (new to this dimension). Both get a new row. Green Leaf still has its current row, so the `LEFT JOIN ... IS NULL` skips it.

Simplification note. This version compares two columns and hard-codes the dates. A production load compares every tracked column (often by comparing a hash of them), takes the dates from the extract, handles customers deleted from the source, and runs as a scheduled job. Chapter 32 shows dbt's *snapshots*, which implement type 2 this way from a short configuration.

Joining facts to a type 2 dimension

Four orders: two before the move (one of them on 31 March, the last day in Mumbai), and two after.

```
CREATE TABLE fact_sales (
  order_id      INTEGER,
  order_date    DATE,
  customer_id   INTEGER,
  net_revenue   NUMERIC(12,2)
);

INSERT INTO fact_sales VALUES
(20001, '2026-03-18', 1, 42000),
(20002, '2026-03-31', 1, 12500),
(20003, '2026-04-09', 3, 35500),
(20004, '2026-04-10', 3, 18000);

SELECT f.order_id,
       f.order_date,
       d.customer_key,
       d.city AS city_at_order_time,
       f.net_revenue
FROM fact_sales AS f
JOIN dim_customer_t2 AS d
  ON d.customer_id = f.customer_id
 AND f.order_date >= d.valid_from
 AND f.order_date < d.valid_to
ORDER BY f.order_id;
```

order_id	order_date	customer_key	city_at_order_time	net_revenue
20001	2026-03-18	1	Mumbai	42000.00
20002	2026-03-31	1	Mumbai	12500.00
20003	2026-04-09	3	Thane	35500.00
20004	2026-04-10	2	Pune	18000.00

(4 rows)

The March orders belong to Mumbai, including the one on 31 March, the last day of the Mumbai version; the April order belongs to Thane. This is a **point-in-time join**. Had step 1 stored 31 March as `valid_to`, order 20002 would match neither version (31 March is not *before* 31 March), and the inner join would silently drop it. In a warehouse you do it once, when loading facts, and store the `customer_key`, as `dw.fact_sales_line` did.

Now the trap. Join on the business key alone, forgetting the dates:

```
SELECT d.city, SUM(f.net_revenue) AS net_revenue
FROM fact_sales AS f
JOIN dim_customer_t2 AS d ON d.customer_id = f.customer_id
GROUP BY d.city
ORDER BY d.city;
```

city	net_revenue
Mumbai	90000.00
Pune	18000.00
Thane	90000.00

(3 rows)

Total revenue should be ₹1,08,000. This shows ₹1,98,000. Each of Sharma Hardware's three orders matched both versions, so ₹90,000 (₹42,000 + ₹12,500 + ₹35,500) was counted in Mumbai *and* in Thane. It's the fan-out trap again: a type 2 dimension's grain is *one customer version*, not *one customer*, and joining on `customer_id` alone ignores that.

Type 3: add a "previous" column

Type 3 keeps limited history in extra columns: the current value and the one before it.

```
CREATE TABLE dim_customer_t3 (
  customer_id    INTEGER PRIMARY KEY,
  customer       VARCHAR(100) NOT NULL,
  city           VARCHAR(50),
  previous_city  VARCHAR(50),
  city_changed_on DATE
);

INSERT INTO dim_customer_t3 VALUES (1, 'Sharma Hardware', 'Mumbai', NULL, NULL);

UPDATE dim_customer_t3
SET previous_city = city,
    city          = 'Thane',
    city_changed_on = DATE '2026-04-01'
WHERE customer_id = 1;

SELECT * FROM dim_customer_t3;
```

customer_id	customer	city	previous_city	city_changed_on
1	Sharma Hardware	Thane	Mumbai	2026-04-01

(1 row)

In PostgreSQL's `UPDATE`, every right-hand side sees the row's *old* values, so `previous_city = city` copies Mumbai even though `city` is assigned in the same statement.

Type 3 answers one specific question well: "Show this year's sales under both the old and the new territory map." It can't tell you where a customer was two moves ago, and it doesn't split sales by date.

Choosing a type

Type	What happens on a change	History kept	Use it for	Watch out for
1	overwrite	none	corrections; attributes nobody reports history on (phone, contact name)	silently restates past reports
2	close old row, add new row	full	attributes reports are sliced by, where history matters (segment, city, sales territory, product category)	join facts through the surrogate key or with a date range, never on the business key alone
3	move old value to a "previous" column	one step back	a planned, one-time reorganization compared side by side	only one previous value

A single dimension usually mixes types: in Riverstone's customer dimension, a corrected phone number would be type 1, and a segment change type 2.

Why type 2 changes the numbers: a real Riverstone result

Back in `riverstone_2025`: "What did each segment sell in 2025?" There are two honest answers, and the star schema can give both. **As it was** uses the version on each fact row. **As it is** re-labels every sale with each customer's current segment.

```
WITH as_it_was AS (
    SELECT c.segment, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    GROUP BY c.segment
),
as_it_is AS (
    SELECT cur.segment, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    JOIN dw.dim_customer AS cur ON cur.customer_id = c.customer_id AND cur.is_current
    GROUP BY cur.segment
)
SELECT w.segment,
    ROUND(w.revenue) AS revenue_as_it_was,
    ROUND(i.revenue) AS revenue_as_it_is,
    ROUND(i.revenue - w.revenue) AS difference
FROM as_it_was AS w
JOIN as_it_is AS i ON i.segment = w.segment
ORDER BY w.segment;
```

segment	revenue_as_it_was	revenue_as_it_is	difference
Hospitality	1144039	1144039	0
Retail	1573615	1488774	-84842
Wholesale	1617817	1702659	84842

(3 rows)

Reading it. Retail's 2025 revenue is ₹15,73,615 as it was, or ₹14,88,774 if you re-label history with today's segments: ₹84,842 apart. Reconcile the difference by hand from the monthly figures: Patel Kitchenware's orders before 1 April (₹2,900 in January and ₹35,138 in March, ₹38,038) move into Retail,

and Harbour Traders' orders from April to August (₹1,22,880) move out. ₹38,038 – ₹1,22,880 = –₹84,842.
✓ Both columns total ₹43,35,471.

Neither column is wrong. They answer different questions: "How did the Retail team's customers perform at the time?" versus "How much have the customers who are Retail today bought?" A type 1 dimension can only give the second answer, and it would give it without telling anyone. **What to tell Anita:** say which view a report uses, in its title.

28.11 Denormalization trade-offs

Denormalization means deliberately storing data more than once, or storing results that could be calculated, to make reading faster or simpler. A star schema is already denormalized (category repeated on every product row). This section is about the other common form: storing **pre-computed results**.

The problem: a question that reads everything

"Monthly revenue by product category, for the last three years." On `riverstone_perf` that joins 700,070 non-cancelled orders to 1,888,466 lines and groups them into 144 rows. No index helps, because the question needs every row. Median time: 2,626 ms. For one analyst once a month, fine. For a dashboard opened by forty people every morning, it's forty queries of two and a half seconds, each reading 140 MB, for numbers that only change once a day.

A summary table or materialized view

A **materialized view** is a view whose result is stored as a table and refreshed on request. Still in `riverstone_perf`:

```
CREATE MATERIALIZED VIEW monthly_category_revenue AS
SELECT DATE_TRUNC('month', o.order_date)::date AS month,
       p.category,
       SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
WHERE o.status <> 'Cancelled'
GROUP BY 1, 2;
```

How it works: the query is the live one, unchanged. `CREATE MATERIALIZED VIEW ... AS` runs it once and stores its 144 rows. A plain view (section 13.2) stores only the query and runs it every time; a materialized view stores the answer. `GROUP BY 1, 2` groups by the first and second columns of the `SELECT`.

The stored rows don't change when new orders arrive. Tomorrow, and every night after, one statement recomputes them:

```
REFRESH MATERIALIZED VIEW monthly_category_revenue;
```

Once built, it's read like any table:

```
SELECT month, category, ROUND(net_revenue) AS net_revenue
FROM monthly_category_revenue
WHERE month = DATE '2025-12-01'
ORDER BY net_revenue DESC;
```

month	category	net_revenue
2025-12-01	Storage	292916949
2025-12-01	Industrial	286297690
2025-12-01	Furniture	230339308
2025-12-01	Kitchen	226362946

(4 rows)

(The amounts are far larger than Riverstone's real 2025 figures in `riverstone_2025`: the large database was generated to have realistic *volume*, not realistic totals.)

The measured trade-off on the test machine:

Step	Time
Live query (median of 5)	2,626 ms
Build the materialized view	2,321 ms
Read one month (4 rows) from the materialized view (median of 7)	0.10 ms
REFRESH MATERIALIZED VIEW	2,300 ms

One refresh a night costs about two seconds; every read after that is effectively free. The price is **staleness**: until the next refresh, the view doesn't show today's orders. For a monthly trend that's fine. For "orders taken in the last hour", it isn't.

The trade-offs, laid out

Technique	Faster reads because	You pay with	Good for
Star schema (repeated attributes)	fewer joins	more storage; updates to a dimension attribute touch many rows	all reporting
Summary table / materialized view	results already computed	staleness; refresh time; another object to maintain	dashboards, recurring reports on large data
Copying a column onto another table (for example customer segment onto orders)	saves a join	the two copies can disagree (section 28.7's update anomaly)	rarely; only with an automatic process keeping it in sync
Storing derived columns (for example <code>net_revenue</code> on order lines)	no calculation at read time	must be recomputed if inputs change	expensive, stable calculations

Three rules keep denormalization safe:

1. **Normalize first.** The source of truth stays normalized. Denormalized copies are derived from it and can always be rebuilt.
2. **Denormalize for a measured problem,** not in advance. The table above started with a 2.6-second query, not a guess.
3. **Automate the refresh and make staleness visible.** Put "data as of 06:00" on the dashboard. A summary nobody refreshes becomes a wrong number that everyone trusts.

Dialect note. PostgreSQL's `REFRESH MATERIALIZED VIEW CONCURRENTLY` lets people keep reading during a refresh (it needs a unique index on the view). MySQL has no materialized views: create a summary table and refresh it with a scheduled `DELETE` and `INSERT ... SELECT` in a transaction, or an event (section 28.13). Warehouses such as Snowflake and BigQuery have their own materialized views with automatic refresh, and dbt builds summary tables as *models* on a schedule (Chapter 32).

Good place to stop.

28.12 Schema migrations: changing a live database safely

Section 12.13 ended with a checklist for changing a table's structure, and promised the tools teams use to do it every week without breaking things. Here they are.

The problem migrations solve

Riverstone now has the ERP database, a reporting copy, a test copy, and every developer's laptop copy. A developer adds a column on their laptop. Does the test copy have it? Production? Which changes were run on production, in which order, by whom? When the answer lives in people's memories and old emails, databases **drift** apart, and one Monday a report fails in production that worked perfectly in test.

A **migration** is a small script that moves a database's structure from one version to the next. The discipline is:

1. **Every change is a file,** never typed into a query window on a shared database.
2. **Files are numbered,** and always applied in number order.
3. **The database records which migrations it has run,** in a history table.
4. **A migration never changes once applied.** A mistake is fixed by a *new* migration.
5. **Every migration is reviewed** (a pull request, Chapter 26) and run on test before production.

Building the idea by hand

Before using a tool, see that there's no magic. Continue in the lab:

```
CREATE TABLE schema_migrations (  
  version      INTEGER PRIMARY KEY,  
  description  VARCHAR(100) NOT NULL,  
  checksum     CHAR(32) NOT NULL,  
  applied_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Migrations 1 and 2 create a table and add its first rows. Each runs in a transaction together with its history row, so either both happen or neither does. (In a real project, `checksum` is a hash of the migration file's contents; here it's computed from a stand-in string.)

```
BEGIN;  
CREATE TABLE suppliers (  
  supplier_id  INTEGER PRIMARY KEY,  
  supplier_name VARCHAR(100) NOT NULL,  
  city         VARCHAR(50)  NOT NULL  
);  
INSERT INTO schema_migrations (version, description, checksum)  
VALUES (1, 'create suppliers', md5('V1__create_suppliers.sql contents'));  
COMMIT;  
  
BEGIN;  
INSERT INTO suppliers VALUES  
(1, 'Western Polymers', 'Vapi'),  
(2, 'Deccan Cartons',   'Pune'),  
(3, 'Sagar Labels',     'Mumbai');  
INSERT INTO schema_migrations (version, description, checksum)  
VALUES (2, 'seed suppliers', md5('V2__seed_suppliers.sql contents'));  
COMMIT;
```

Migration 3 adds a column. Section 12.13's rule applies: a new `NOT NULL` column on a table with rows needs a `DEFAULT`, or existing rows break the constraint:

```
BEGIN;  
ALTER TABLE suppliers ADD COLUMN payment_terms_days INTEGER NOT NULL DEFAULT 30;  
INSERT INTO schema_migrations (version, description, checksum)  
VALUES (3, 'add payment terms', md5('V3__add_payment_terms.sql contents'));  
COMMIT;  
  
SELECT version, description FROM schema_migrations ORDER BY version;
```

```
version | description  
-----+-----  
      1 | create suppliers  
      2 | seed suppliers  
      3 | add payment terms  
(3 rows)
```

Any copy of the database can now answer "which version are you on?" with `SELECT MAX(version) FROM schema_migrations;`, and a tool can apply only the files with a higher number.

And here's the failure the `DEFAULT` rule prevents, in a migration someone forgot to test against real data:

```
BEGIN;
ALTER TABLE suppliers ADD COLUMN gst_number VARCHAR(15) NOT NULL;
SELECT MAX(version) FROM schema_migrations;
ROLLBACK;
```

```
BEGIN
ERROR:  column "gst_number" of relation "suppliers" contains null values
ERROR:  current transaction is aborted, commands ignored until end of transaction block
ROLLBACK
```

Reading it:

- The `ALTER` fails: three suppliers exist, and a new `NOT NULL` column with no default would leave them holding nothing.
- After an error inside a transaction, PostgreSQL refuses **every** further statement until you end the transaction. Even the harmless `SELECT` fails with *current transaction is aborted*. It's a safety catch: nothing more can happen on top of a half-finished change.
- **ROLLBACK** (section 12.13) ends the transaction and undoes everything since `BEGIN`. Always run it after an error inside `BEGIN`, before anything else. (`COMMIT` would also end it, but after an error PostgreSQL treats it as a rollback anyway.)

Check that nothing was left behind:

```
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'suppliers'
ORDER BY ordinal_position;

SELECT MAX(version) AS current_version FROM schema_migrations;
```

```
column_name
-----
supplier_id
supplier_name
city
payment_terms_days
(4 rows)

current_version
-----
3
(1 row)
```

No `gst_number` column, and the database is still at version 3. Because the migration ran inside a transaction, the error left nothing half-done: no column, no history row. PostgreSQL can roll back structure changes like this; MySQL can't (section 28.13), which is one more reason to test migrations on a copy first.

Tool note: running migrations in DBeaver. Run each migration block with *Execute script* (Alt+X), which runs the statements one after another. If the script stops on an error, the transaction is still open: run `ROLLBACK;` before anything else, or every later statement fails the same way.

Dialect note: the same migration in MySQL. MySQL 8.0 doesn't raise this error. It adds the column and fills the existing rows with the type's implicit default, an empty string for `VARCHAR`, even in strict mode (tested on 8.0.46). The migration "succeeds", and three suppliers now have a GST number of '' that no validation will ever flag. Same script, same intent, a silent data problem in one database and a loud error in the other: test migrations on every database engine you deploy to.

Renaming without breaking anyone: expand, then contract

Suppose the team wants to rename `suppliers.city` to `supplier_city`. `ALTER TABLE ... RENAME COLUMN` takes a moment, but every report, application, and dbt model that reads `city` breaks at that moment. On a system other people depend on, split the change into steps that are each safe on their own. This is the **expand and contract** pattern (also called *parallel change*):

1. **Expand:** add the new column and copy the data. Old readers still work.
2. **Migrate readers:** change each report and application to use the new column, in their own time. (Keep writers filling both columns meanwhile.)
3. **Contract:** once nothing reads the old column, drop it.

```
BEGIN;
ALTER TABLE suppliers ADD COLUMN supplier_city VARCHAR(50);
UPDATE suppliers SET supplier_city = city;
INSERT INTO schema_migrations (version, description, checksum)
VALUES (4, 'expand: add supplier_city', md5('V4 contents'));
COMMIT;

SELECT supplier_name, city, supplier_city FROM suppliers ORDER BY supplier_id;
```

supplier_name	city	supplier_city
Western Polymers	Vapi	Vapi
Deccan Cartons	Pune	Pune
Sagar Labels	Mumbai	Mumbai

(3 rows)

Days or weeks later, after every reader has moved:

```
BEGIN;
ALTER TABLE suppliers ALTER COLUMN supplier_city SET NOT NULL;
ALTER TABLE suppliers DROP COLUMN city;
INSERT INTO schema_migrations (version, description, checksum)
VALUES (5, 'contract: drop old city column', md5('V5 contents'));
COMMIT;

SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'suppliers'
ORDER BY ordinal_position;
```

column_name	data_type	is_nullable
supplier_id	integer	NO
supplier_name	character varying	NO
payment_terms_days	integer	NO
supplier_city	character varying	NO

(4 rows)

Migration tools

Nobody writes those `INSERT INTO schema_migrations` lines by hand in a real project. A **migration tool** reads a folder of numbered files, compares it with the history table, and applies what's missing. The ideas above are all the tools do; they differ in style:

Tool	How migrations are written	Common with
Flyway (Redgate)	plain SQL files named <code>V1__create_suppliers.sql</code> , <code>V2__seed_suppliers.sql</code>	Java and SQL-first teams; widely used with PostgreSQL, MySQL, SQL Server, Oracle
Liquibase	<i>changelogs</i> in SQL, XML, YAML, or JSON, with changesets	enterprises needing detailed change tracking across many databases
Alembic	Python files, usually generated from SQLAlchemy models	Python applications that use SQLAlchemy
Framework tools (Django, Rails, Laravel, Entity Framework)	code in the application's language	teams whose database belongs to one application
dbt	not a migration tool: it rebuilds analytical models from <code>SELECT</code> statements	warehouses (Chapter 32)

Flyway shows the conventions most clearly. Versioned migration files are named with a `V`, a version number, two underscores, and a description, like `V3__add_payment_terms.sql`, and are applied once each, in version order. Flyway records each one in a **schema history table** (`flyway_schema_history`) with a **checksum** of the file, so if someone edits a file after it was applied, `flyway validate` reports the mismatch instead of silently letting environments drift. `flyway info` lists applied and pending migrations. **Repeatable migrations**, named `R__description.sql`, have no version and are re-applied whenever their checksum changes, which suits views. (Check Redgate's documentation for the current version and edition.)

A migration project for Riverstone's reporting database looks like this:

```
reporting-db/
├── migrations/
│   ├── V1__create_suppliers.sql
│   ├── V2__seed_suppliers.sql
│   ├── V3__add_payment_terms.sql
│   ├── V4__expand_add_supplier_city.sql
│   ├── V5__contract_drop_city.sql
│   └── R__view_supplier_summary.sql
└── README.md
```

The companion file `migrate.py` is a runner of about 50 lines that applies a folder like this to PostgreSQL in the same way, with checksums, so you can watch the mechanism work before adopting a real tool. (It applies only `V` files; the `R__` file shows Flyway's naming.) The migration files hold **only the change**. Here's one:

```
# terminal, in the companion files folder
$ cd ch28
$ cat migrations/V3__add_payment_terms.sql
ALTER TABLE suppliers ADD COLUMN payment_terms_days INTEGER NOT NULL DEFAULT 30;
```

(`cat` prints a file, as in section 26.0; in Windows PowerShell, `type` does the same.) There's no `BEGIN`, no `COMMIT`, and no `INSERT INTO schema_migrations`: the runner adds those around each file. Its core is four short pieces. First, find the files and sort them by number:

```
files = []
for f in pathlib.Path(a.folder).glob('V*__*.sql'):
    m = re.fullmatch(r'V(\d+)__(.+)\.sql', f.name)
    if m: files.append((int(m.group(1)), m.group(2).replace('_', ' '), f))
files.sort()
```

`glob('V*__*.sql')` lists the files whose names fit Flyway's pattern (Chapter 17 used `glob` for CSV files). `re.fullmatch` checks the whole name against a regular expression, the pattern language of section 14.2, just as `pandas' .str.fullmatch` did in Chapter 18: `(\d+)` captures the digits and `(.+)` the description, so `V3__add_payment_terms.sql` gives the number 3 and `add payment terms`. Sorting on the number puts `V10` after `V9`, where sorting the names as text would not. Second, read what the database has already run:

```
applied = {int(v): c for v, c in (line.split('|') for line in psql(
    'SELECT version, checksum FROM schema_migrations;').splitlines() if line)}
```

`psql(...)` is a small function in the file that sends SQL to the `psql` client and returns what it prints. The result is a dictionary from version number to checksum. Third, for each file, compare checksums and stop on a mismatch:

```
body = f.read_text()
checksum = hashlib.md5(body.encode()).hexdigest()
if version in applied:
    if applied[version] != checksum:
        sys.exit(f'checksum mismatch: {f.name} was changed after it was applied. Write a new migration
            instead.')
    continue
```

`hashlib.md5(...).hexdigest()` turns the file's contents into a 32-character fingerprint; change one character of the file and the fingerprint changes. An applied file with the same fingerprint is skipped; one with a different fingerprint stops the run (`sys.exit()`). Fourth, run a new file and its history row in one transaction:

```
psql('BEGIN;\n' + body + f"\nINSERT INTO schema_migrations (version, description, checksum) "\n    f"VALUES ({version}, '{description}', '{checksum}');\nCOMMIT;\n")\nprint(f'applied {f.name}'); ran += 1
```

That's exactly what you typed by hand above: `BEGIN`, the change, the history row, `COMMIT`. The `psql` function runs the client with `-v ON_ERROR_STOP=1`, so the first error stops the file, and the transaction is rolled back. The whole file, with comments, is `migrate.py` in the companion folder.

Try it. Create an empty database for it in DBeaver: `CREATE DATABASE riverstone_lab_migrations;`. The runner connects through `psql` (section 28.1's box), which takes the user and password from two **environment variables** (section 26.4): `PGUSER` and `PGPASSWORD`. In the terminal, set them for this session (Git Bash, macOS, and Linux shown; in PowerShell write `$env:PGUSER = "postgres"`), putting your own password in place of the placeholder:

```
# terminal, in companion/ch28\n$ export PGUSER=postgres\n$ export PGPASSWORD='your-postgres-password'
```

They last until you close the terminal, and they never go into a file that could be shared. If `psql` isn't on your `PATH` (section 28.1, step 3), add `--psql` and the full path to it after `migrate.py`. Now run the runner twice, then make a quiet edit to `V3` and run it a third time (type `python` instead of `python3` on Windows):

```
# terminal, in companion/ch28\n$ python3 migrate.py --db riverstone_lab_migrations migrations\napplied V1__create_suppliers.sql\napplied V2__seed_suppliers.sql\napplied V3__add_payment_terms.sql\napplied V4__expand_add_supplier_city.sql\napplied V5__contract_drop_city.sql\ndatabase is at version 5\n$ python3 migrate.py --db riverstone_lab_migrations migrations\nnothing to apply; database is at version 5\n$ echo "-- reviewed" >> migrations/V3__add_payment_terms.sql\n$ cat migrations/V3__add_payment_terms.sql\nALTER TABLE suppliers ADD COLUMN payment_terms_days INTEGER NOT NULL DEFAULT 30;\n-- reviewed\n$ python3 migrate.py --db riverstone_lab_migrations migrations\nchecksum mismatch: V3__add_payment_terms.sql was changed after it was applied. Write a new migration instead.
```

What each command does:

- `--db riverstone_lab_migrations` names the database; `migrations` is the folder of files.
- The **first run** applies all five files in number order and reports the version reached.
- The **second run** is the point of the history table: running migrations is **idempotent** (running it again changes nothing).
- `echo "-- reviewed" >> ...` adds a comment line to the end of V3. `>>` *appends* to a file; a single `>` would replace the whole file. The edit changes nothing about what V3 does, but it changes the file's fingerprint.
- The **third run** is the point of the checksum: a quiet edit to an applied migration would mean this database and every other copy now disagree about what "version 3" means, so the runner refuses to continue.

To tidy up, remove the comment line from V3 in your editor (or copy the file back from the companion download).

Reviewing a migration

Before approving a migration, a reviewer asks:

1. **Does it run on a copy with realistic data?** Constraints that pass on an empty test table fail on real rows, as `gst_number` did.
2. **Does it lock a busy table?** Adding an index without `CONCURRENTLY`, changing a column's type, or adding a `NOT NULL` column can block writes on large tables. Test the duration on production-sized data.
3. **Does it break a reader?** Renames and drops go through expand and contract. Search the reports, views, and dbt models for the column name.
4. **Is data preserved?** A type change (`VARCHAR` to `INTEGER`) or a split column needs a data check before and after: row counts and a total.
5. **Is there a way back?** Either a written reverse migration, or a backup taken immediately before, with the restore tested (section 2.9).
6. **Is it one change?** Small migrations are quick to review and quick to fix. "V7__misc_changes.sql" is a warning sign.

28.13 The same SQL in MySQL

The ideas in this chapter all carry over to MySQL 8.0: recursive CTEs, frames, plans, indexes, normalization, grain, star schemas, and type 2 dimensions built with `UPDATE` and `INSERT`. The spellings below change. The companion file `ch28_queries_mysql.sql` has a tested MySQL version of every query. Everything here was run on MySQL 8.0.46. MySQL 8.0 reached the end of Oracle's standard support in April 2026; the current long-term-support releases are 8.4 and 9.7 (section 12.3), and the book's MySQL outputs have not been rerun on them, so check anything version-sensitive against the MySQL documentation for your release.

Setting up

Load `riverstone_2025_setup_mysql.sql`, then `ch28_2025_addons_mysql.sql`. For the large database, follow section 28.1's box "Build riverstone_perf", using its MySQL note.

What changes

Feature	PostgreSQL	MySQL 8.0
Recursive CTE	<code>WITH RECURSIVE</code>	same; recursion stops after 1,000 passes by default (<code>cte_max_recursion_depth</code> , section 28.2)
Column sizes in a recursive CTE	taken from both parts; types must match	taken from the anchor; <code>CAST</code> growing strings in the anchor
<code>SEARCH / CYCLE</code> clauses	yes (14+)	no; build a path string and check it
<code>GROUPS</code> frames, <code>EXCLUDE</code>	yes	no
<code>RANGE</code> with interval	<code>INTERVAL '6 days'</code>	<code>INTERVAL 6 DAY</code>
<code>PERCENTILE_CONT</code>	yes	no; use <code>NTILE</code> or <code>ROW_NUMBER</code> with <code>COUNT</code>
First of the month	<code>DATE_TRUNC('month', d)</code>	<code>CAST(DATE_FORMAT(d, '%Y-%m-01') AS DATE)</code>
Date as a number	<code>TO_CHAR(d, 'YYYYMMDD')</code>	<code>DATE_FORMAT(d, '%Y%m%d')</code>
A list of dates	<code>generate_series(...)</code>	a recursive CTE (section 28.2) or a calendar table
Type conversion	<code>x::date</code> or <code>CAST(x AS DATE)</code>	<code>CAST(x AS DATE)</code> only
Null-safe equals	<code>a IS NOT DISTINCT FROM b</code>	<code>a <=> b</code>
Null-safe "differs"	<code>a IS DISTINCT FROM b</code>	<code>NOT (a <=> b)</code>
Update from another table	<code>UPDATE t SET ... FROM other WHERE ...</code>	<code>UPDATE t JOIN other ON ... SET ...</code>
Automatic key	<code>INTEGER GENERATED ALWAYS AS IDENTITY</code>	<code>INT AUTO_INCREMENT</code>
A schema for the warehouse	<code>CREATE SCHEMA dw;</code> <code>DROP SCHEMA dw CASCADE</code>	a schema <i>is</i> a database: <code>CREATE DATABASE dw;</code> <code>DROP DATABASE dw</code>
Expression index	<code>ON customers (UPPER(city))</code>	<code>ON customers ((UPPER(city)))</code> , 8.0.13+
Foreign key indexes	not automatic	created automatically by InnoDB
Partial indexes	yes	no (a functional index on a <code>CASE</code> expression can imitate one)
Covering index	<code>INCLUDE (...)</code>	no <code>INCLUDE</code> ; add the columns to the index key
<code>EXPLAIN</code> output	text tree	<code>EXPLAIN FORMAT=TREE</code> ; <code>EXPLAIN ANALYZE</code> (8.0.18+)
<code>MERGE</code>	yes (15+)	no; <code>INSERT ... ON DUPLICATE KEY UPDATE</code> for type 1
Materialized views	yes	no; summary tables
Transactional structure changes	yes: <code>ALTER</code> can be rolled back	no: each <code>ALTER/CREATE</code> commits immediately

Feature	PostgreSQL	MySQL 8.0
Adding a NOT NULL column with no default to a table with rows	error	succeeds, filling rows with '', 0, or a zero date

A tree in MySQL

With no `SEARCH` clause, order an org chart by building a **sort path** of zero-padded ids. The anchor's `CAST(... AS CHAR(200))` sets the column's size; without it, MySQL sizes the column from the anchor's four-character first value, and the query stops with `ERROR 1406 (22001): Data too long for column 'sort_path' at row 1` as soon as the path grows.

```
WITH RECURSIVE org AS (
  SELECT staff_id, staff_name, 1 AS level,
         CAST(LPAD(staff_id, 4, '0') AS CHAR(200)) AS sort_path
  FROM staff
  WHERE manager_id IS NULL
  UNION ALL
  SELECT s.staff_id, s.staff_name, o.level + 1,
         CONCAT(o.sort_path, '/', LPAD(s.staff_id, 4, '0'))
  FROM staff AS s
  JOIN org AS o ON s.manager_id = o.staff_id
)
SELECT CONCAT(REPEAT('  ', level - 1), staff_name) AS org_chart
FROM org
WHERE sort_path LIKE '0100/0110%'
ORDER BY sort_path;
```

```
+-----+
| org_chart |
+-----+
| Anita Rao |
|   Vikram Singh |
|     Neha Kulkarni |
|     Rahul Mehta |
|   Farah Khan |
|   Meera Iyer |
|   Arjun Nair |
|   Divya Krishnan |
|   Vivek Chandran |
|   Sneha Pillai |
| Pooja Desai |
|   Nisha Bhatt |
|   Aditya Verma |
|   Rohit Kamat |
| Sandeep Gill |
|   Karan Ahuja |
|   Ritu Bansal |
+-----+
```

`WHERE sort_path LIKE '0100/0110%'` keeps Anita Rao's branch (staff 110 under 100). The same path string is also the loop check of section 28.2's defense 2: in the recursive part, add `WHERE LOCATE(LPAD(s.staff_id, 4, '0'), o.sort_path) = 0`. `LOCATE(text, path)` returns where text first appears in path, or 0 if it doesn't, so the walk never visits an id twice; on Riverstone's clean data it still returns all 35 people.

MySQL's safety limit for runaway recursion, `cte_max_recursion_depth`, is in section 28.2's first example.

Frames

`RANGE` with an interval works:

```
SELECT order_date,
       COUNT(*) OVER (ORDER BY order_date
                      RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW) AS orders_last_7_days
FROM orders
WHERE order_date BETWEEN '2025-02-14' AND '2025-02-21'
ORDER BY order_date, order_id;
```

```
+-----+-----+
| order_date | orders_last_7_days |
+-----+-----+
| 2025-02-16 |                2 |
| 2025-02-16 |                2 |
| 2025-02-17 |                4 |
| 2025-02-17 |                4 |
| 2025-02-20 |                6 |
| 2025-02-20 |                6 |
+-----+-----+
```

`GROUPS` does not:

```
SELECT order_date,
       COUNT(*) OVER (ORDER BY order_date GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
       orders_in_frame
FROM orders
LIMIT 3;
```

```
ERROR 1235 (42000): This version of MySQL doesn't yet support 'GROUPS'
```

Plans in MySQL

MySQL's tree format reads the same way as PostgreSQL's, inside out. Here are the December queries from section 28.6 on `riverstone_perf` in MySQL, after `CREATE INDEX idx_orders_order_date ON orders (order_date)`; (recorded output, test machine). `EXPLAIN ANALYZE` runs the query and prints the tree:

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 12;
```

```
-> Aggregate: count(0) (cost=143003 rows=1) (actual time=131..131 rows=1 loops=1)
  -> Filter: ((extract(year from orders.order_date) = 2025) and (extract(month from orders.order_date) = 12))
      (cost=71803 rows=712008) (actual time=125..129 rows=25078 loops=1)
        -> Covering index scan on orders using idx_orders_order_date (cost=71803 rows=712008) (actual
            time=0.548..81 rows=714285 loops=1)
```

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM orders
WHERE order_date >= '2025-12-01' AND order_date < '2026-01-01';
```

```
-> Aggregate: count(0) (cost=14488 rows=1) (actual time=6.74..6.74 rows=1 loops=1)
-> Filter: ((orders.order_date >= DATE'2025-12-01') and (orders.order_date < DATE'2026-01-01')) (cost=9672 ↵
rows=48152) (actual time=0.308..5.62 rows=25078 loops=1)
-> Covering index range scan on orders using idx_orders_order_date over ('2025-12-01' <= order_date < ↵
'2026-01-01') (cost=9672 rows=48152) (actual time=0.306..3.19 rows=25078 loops=1)
```

The lesson transfers exactly. The EXTRACT version reads all 714,285 index entries (MySQL is clever enough to scan the small index instead of the table, but it still reads everything), with a median of 136 ms; the range reads only December's 25,078, with a median of 7.09 ms. And MySQL needed no index for the single-customer lookup: its automatic foreign-key index on `customer_id` gave a median of 0.17 ms straight after loading, the same as PostgreSQL's after section 28.5 added one.

Type 1 without MERGE

MySQL's upsert updates a row when the primary key already exists, and inserts it otherwise. To try it, build section 28.10's two starting tables in your MySQL lab. The lab database from Chapter 12 may already exist; IF NOT EXISTS makes the first line safe either way:

```
CREATE DATABASE IF NOT EXISTS riverstone_lab;
```

Then, connected to `riverstone_lab` in MySQL, the same statements as section 28.10:

```
CREATE TABLE dim_customer_t1 (
  customer_id INTEGER PRIMARY KEY,
  customer    VARCHAR(100) NOT NULL,
  city        VARCHAR(50),
  segment     VARCHAR(30)
);

INSERT INTO dim_customer_t1 VALUES
(1, 'Sharma Hardware', 'Mumbai', 'Retail'),
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality');

CREATE TABLE customer_extract (
  customer_id INTEGER PRIMARY KEY,
  customer    VARCHAR(100) NOT NULL,
  city        VARCHAR(50),
  segment     VARCHAR(30)
);

INSERT INTO customer_extract VALUES
(1, 'Sharma Hardware', 'Thane', 'Retail'),
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality'),
(4, 'Coastal Foods', 'Chennai', 'Wholesale');
```

Now the upsert, and the result:

```
INSERT INTO dim_customer_t1 (customer_id, customer, city, segment)
SELECT customer_id, customer, city, segment
FROM customer_extract AS x
ON DUPLICATE KEY UPDATE city = x.city, segment = x.segment;

SELECT * FROM dim_customer_t1 ORDER BY customer_id;
```

customer_id	customer	city	segment
1	Sharma Hardware	Thane	Retail
3	Green Leaf Hotels	Pune	Hospitality
4	Coastal Foods	Chennai	Wholesale

How it works: the `INSERT ... SELECT` tries to insert every extract row. Rows 1 and 3 already exist, so their primary keys clash, and instead of an error, `ON DUPLICATE KEY UPDATE` runs: it sets the city and segment from the extract row (`x.city`). Row 4 is new and is simply inserted. The same result as `MERGE` in section 28.10. Unlike `MERGE`, it has no `WHEN MATCHED AND ...` condition, so every clashing row goes through the update, changed or not.

The type 2 load (close, then insert) works as in section 28.10, except that MySQL's multi-table `UPDATE` is written `UPDATE dim_customer_t2 AS d JOIN customer_extract AS x ON ... SET ...`. Both versions are in `ch28_queries_mysql.sql`.

Watch out: MySQL can't roll back an ALTER TABLE. In MySQL, `CREATE`, `ALTER`, and `DROP` commit immediately, even inside `BEGIN ... COMMIT`. A migration that fails halfway leaves the first half applied. Keep each MySQL migration to one structure change, and test it on a copy.

Good place to stop.

APPLY

Common mistakes

Mistake	Symptom	Fix
Recursive CTE with no loop protection	Query runs until cancelled, or MySQL stops after 1,000 passes	Add a depth limit, a path check, or PostgreSQL's <code>CYCLE</code> clause
Anchor and recursive part with different column types	<i>recursive query ... has type numeric(8,3) in non-recursive term</i> (PostgreSQL); <i>Data too long</i> (MySQL)	CAST the anchor's columns to the wider type
Adding BOM quantities instead of multiplying them down the tree	Material needs far too low for multi-level products	Multiply each child's quantity by its parent's running quantity
Walking a hierarchy with orphan rows	Some people or parts silently missing from the result	Compare the walk's row count with the table; <code>LEFT JOIN</code> to find missing managers

Mistake	Symptom	Fix
LAST_VALUE with the default frame	"Latest" value equals the current row	Frame ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
ROWS frame on data with missing days	"Last 7 days" really covers several weeks	RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW
Tuning on a tiny test database	Every plan is a sequential scan and every query is instant; production is slow	Test plans on production-sized data
Function wrapped around an indexed column	Seq Scan with a huge Rows Removed by Filter, although an index exists	Rewrite as a range, or create an expression index
No index on a PostgreSQL foreign key	Joins and deletes on the child table slow down as it grows	Index foreign key columns you join or delete through
Composite index in the wrong column order	Index exists but isn't used for the query	Equality columns first, range column last
Indexing every column "just in case"	Loads and inserts get slower; indexes bigger than the table	Index for measured, frequent queries; drop redundant indexes
Trusting estimated costs as timings	Surprise when a "cheap" plan is slow	Use EXPLAIN ANALYZE; costs are relative units
EXPLAIN ANALYZE on a DELETE or UPDATE	Rows really changed	Wrap in BEGIN; ... ROLLBACK;
Moving the charged price into the products table while normalizing	Past orders show today's prices	Separate list price (product) from price charged (order line)
Joining tables of different grains, then summing	Totals several times too large, with no error (₹45,00,000 target for January)	Aggregate to a common grain first
valid_to set to the last day of a version, with a half-open join	Facts dated on the last day of the old version vanish from inner joins	Store the next version's start date in valid_to
Joining facts to a type 2 dimension on the business key	Revenue counted once per customer version	Join on the surrogate key, or on the key plus the date range
Using type 1 for attributes reports are sliced by	Last year's segment or territory totals change without new data	Use type 2 and say which view the report shows
Materialized view nobody refreshes	Dashboard numbers stop moving; everyone trusts them	Schedule the refresh and show "data as of"
Editing a migration that was already applied	Databases drift; tools report checksum errors	Never edit; add a new migration
Renaming or dropping a column in one step	Reports and applications break the moment it runs	Expand, migrate readers, then contract
Adding NOT NULL without a default to a table with rows	PostgreSQL: column ... contains null values; MySQL: no error, rows silently get '' or 0	Add a DEFAULT, or add nullable, backfill, then set NOT NULL

In the real world: the dashboard that got slower every month

It's March 2026. Vikram Singh opens the sales dashboard in the Monday review and waits. The customer page, the December comparison, and the three-year trend each take a few seconds, and the whole team watches the spinner. *"This was instant when we launched it,"* he says. *"Can someone look at it before next Monday?"*

Meera Iyer takes it. She's tempted to start adding indexes, but she has seen that go wrong: an index on the busy orders table during working hours would block order entry while it builds. So she works the checklist from section 28.6, on a copy of the reporting database, which (like `riverstone_perf`) holds three years of orders.

First, find where the time goes. She copies the dashboard's four queries out of the report and runs each with `EXPLAIN (ANALYZE, BUFFERS)`. The numbers below are the chapter's measured medians, from queries of the same shape:

Dashboard tile	Median	What the plan shows
Customer page: one customer's revenue	262.47 ms	Seq Scan on order_items: 1.9 million lines read for 108
December orders compared with last year	131.33 ms	Seq Scan on orders with EXTRACT(YEAR ...) in the filter
Pending orders list	44.12 ms	Seq Scan, 710,355 rows removed
Three-year monthly trend by category	2,626 ms	Hash joins over every non-cancelled line

The trend tile is 85.7% of the 3,064 ms total. That's the headline for Vikram: one tile is the problem, and it's a different kind of problem from the other three.

Second, fix each for the right reason.

- *Customer page.* `order_items.order_id` has no index. The ERP vendor's MySQL version had one automatically; the PostgreSQL reporting copy doesn't. One index: 0.64 ms.
- *December comparison.* No index is missing: the filter can't use the one on `order_date`. She rewrites `EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 12` as a date range. 2.95 ms, no database change needed.
- *Pending list.* A partial index on pending orders: 48 kB, 0.75 ms.
- *Trend.* No index can help a question that reads three years of lines. The numbers only change once a day, so a materialized view refreshed at 06:00 turns it into a 144-row table. Refresh: about 2 seconds a night. Reading it: 0.10 ms.

Third, check she hasn't broken anything. The trend from the materialized view matches the live query for every month and category. She measures the cost to writes too: the reporting copy is loaded nightly, and the chapter's test showed four indexes making a large insert about five and a half times slower. One new index and one partial index add seconds to a load that runs once a night, which is acceptable, and she writes that down.

Fourth, ship it properly. She doesn't run anything on the reporting database herself. She writes three migrations, `V14__index_order_items_order_id.sql` and `V15__index_orders_pending.sql` (each with `CREATE INDEX CONCURRENTLY`, each in its own non-transactional migration) and

`V16__monthly_category_revenue.sql`, and a change to the dashboard's December query. The data engineer reviews them against section 28.12's checklist and applies them on Wednesday evening. The refresh joins the nightly job, and the trend tile gets a caption: "Data as of 06:00 today."

What she tells Vikram: *"The dashboard's four tiles took about 3 seconds of database time; they now take under 5 milliseconds. Three fixes were missing or misused indexes. The fourth was the trend chart re-adding three years of orders every time anyone opened it; it now reads a summary refreshed every morning, so it won't show today's orders until tomorrow. Nothing changes for anyone entering orders."*

Notice what made it work. The measurement came first, so she fixed the tile that mattered most. Each fix matched its cause, rather than "add indexes until it's fast". Every change went through a reviewed migration. And the one trade-off, a trend that's a day old, was put in front of the person who owns the decision.

Project: a star schema with history

Goal: a small, reconciled data warehouse for one business process, with a type 2 dimension, that a BI tool can read without knowing anything about the source tables.

Tools you'll need

Versions used for this chapter, checked in September 2026:

- **PostgreSQL 15 or later** (every query was run on PostgreSQL 16 on Ubuntu 24.04; the current release is 18, section 12.3). `CYCLE` and `SEARCH` need 14 or later, `MERGE` needs 15 or later. The `psql` client comes with the server.
- **MySQL** (every MySQL query was run on 8.0.46). MySQL 8.0 reached the end of Oracle's standard support in April 2026; install a current long-term-support release (8.4 or 9.7, section 12.3). `EXPLAIN ANALYZE` needs 8.0.18 or later, and expression indexes need 8.0.13 or later.
- **DBeaver** for running queries. Its *Explain execution plan* button shows plans as a tree; `EXPLAIN (ANALYZE, BUFFERS)` in the editor gives the full text.
- **Python** (Chapter 17's installation; tested on 3.11 and 3.12) for `generate_riverstone_perf.py`, `migrate.py`, and `ch28_perf.py`. All three use only the standard library; `migrate.py` and `ch28_perf.py` call the `psql` client.
- **A plan visualizer (optional)**. Several free tools turn PostgreSQL's `EXPLAIN (ANALYZE, FORMAT JSON)` output into a diagram. Paste plans only from practice data, never from a production database containing customer data.
- **Migration tools**. Flyway (Redgate; the free edition covers versioned SQL migrations), Liquibase, and Alembic for Python projects. Check the current release and edition details on each tool's documentation site before you install. This chapter describes Flyway's conventions from Redgate's documentation; `migrate.py` demonstrates the same mechanism.
- **Companion files for this chapter:** `ch28_2025_addons.sql` and `ch28_2025_addons_mysql.sql` (staff, parts, BOM, customer changes), `generate_riverstone_perf.py` (the large database), `ch28_perf.py` (reruns every timing experiment and writes `ch28_perf_log.txt`), `ch28_star_schema.sql` (the dw

schema from section 28.9), `migrate.py` with its `migrations/` folder, and `ch28_queries_mysql.sql` (every query in MySQL).

Option A: your own work data. Choose a process you report on (orders, tickets, payments) and a dimension that changes over time (customer segment, account owner, region). Work on a copy, never the production database, and remove or mask personal data before you start. Keep the project on your own machine unless your employer has agreed otherwise.

Option B: Riverstone. Use `riverstone_2025` and the tables from this chapter.

Steps:

1. **Write the design on one page:** the business process, the grain of the fact table in one sentence, the dimensions, the facts, and for each dimension attribute whether changes are type 1 or type 2, with a reason.
2. **Build the date dimension** for your period, with at least day name, month, quarter, and fiscal year (April to March).
3. **Build the other dimensions**, each with a surrogate key. Add an "unknown" row wherever the source has missing values.
4. **Build the type 2 dimension** from your change history (Riverstone: `customer_changes`). Check that each business key has exactly one current row and no gaps or overlaps between versions.
5. **Load the fact table** with a point-in-time join to the type 2 dimension.
6. **Reconcile.** Row counts and totals against the source (Riverstone: ₹43,35,471 and 326 lines). Hand-check one fact row end to end.
7. **Answer three business questions** from the star: one by date attribute, one "as it was" versus "as it is" by the changing attribute, one that uses the unknown row.
8. **Make it repeatable.** Put the build into numbered migration files for the structures and one load script for the data, so the whole warehouse can be dropped and rebuilt from nothing.

Deliverables: the one-page design, the SQL files, the reconciliation query and its output, and a half-page note with the three answers.

Stretch goals:

- Add a second fact table at a different grain (monthly targets, or invoices) that shares `dim_date`, and build a report that puts both side by side correctly.
- Write the type 2 load as an *incremental* script: given tomorrow's extract, it closes and inserts only what changed. Run it twice to prove that the second run changes nothing.
- Load the star into Power BI (Chapter 16) and confirm that the model view shows a star.
- Build the same warehouse in MySQL and list every change you had to make.

Recap

- A **recursive CTE** has an **anchor** and a **recursive part** joined by `UNION ALL`; each pass reads only the previous pass's rows, and it repeats until a pass finds nothing new. The anchor fixes the columns' types. It can build lists (a date spine) as well as walk trees. Use it for org charts, **bills of materials** (multiply quantities down the tree), and **where-used** queries. Guard against loops with a depth limit, a path, or `CYCLE` (MySQL stops at 1,000 passes); order a tree with `SEARCH DEPTH FIRST`.
- **Frames**: `ROWS` counts rows, `GROUPS` counts peer groups, `RANGE` measures values (with intervals for calendar windows). `EXCLUDE` removes the current row or its peers; the `WINDOW` clause names a window for reuse. Windows see only the rows `WHERE` kept. `PERCENTILE_CONT` interpolates, `PERCENTILE_DISC` returns a real value; `NTILE` gives equal-count bands.
- Tables live in **pages**. A **sequential scan** reads them all; a **B-tree index** finds rows through sorted keys. `EXPLAIN` shows the plan and estimates; `EXPLAIN ANALYZE` runs it and measures.
- Index **foreign keys** you join on (PostgreSQL doesn't do it for you), keep filters **sargable**, order **composite** indexes equality-first, and use **covering** and **partial** indexes for important, frequent queries. Every index slows writes.
- **Normalization**: **1NF** one value per cell; **2NF** no partial dependencies; **3NF** no transitive dependencies. It removes update, insert, and delete anomalies.
- **Grain**: say it, test it, never join different grains and then sum, and aggregate to a common grain first.
- **Dimensional modeling**: a **fact table** at a declared grain, surrounded by **dimensions** with **surrogate keys**, including a **date dimension**. Prefer a **star** to a **snowflake**. Facts come as transactions, periodic snapshots, or accumulating snapshots.
- **Slowly changing dimensions**: **type 1** overwrites, **type 2** adds a version with validity dates (`valid_to` = the next version's `valid_from`), **type 3** keeps a previous value. Join type 2 dimensions on the surrogate key or with a **point-in-time join**.
- **Denormalize** only for a measured problem, from a normalized source, with an automated refresh and visible staleness. **Materialized views** trade freshness for speed.
- **Migrations** are numbered, reviewed, never-edited scripts recorded in a history table. Change live structures with **expand and contract**.

Key terms

recursive CTE · `RECURSIVE` · anchor · recursive part · working table (previous pass) · `cte_max_recursion_depth` · `UNION ALL` in recursion · hierarchy · bill of materials (BOM) · exploding a BOM · where-used query · `CYCLE` clause · `SEARCH DEPTH FIRST` · frame type · `GROUPS` frame · peer group · `EXCLUDE` · `WINDOW` clause (recap) · ordered-set aggregate · `PERCENTILE_CONT` (recap) · `PERCENTILE_DISC` · `NTILE` (recap) · page · sequential scan · index · B-tree · leaf page · statistics · query plan · `EXPLAIN` · `EXPLAIN ANALYZE` · cost · heap · index scan · bitmap scan · index-only scan · nested loop join · hash join · merge join · selective · sargable · expression index · composite index · covering index · partial index · normal form · first normal form (1NF) · update anomaly · insert anomaly · delete anomaly · functional

dependency · partial dependency · second normal form (2NF) · transitive dependency · third normal form (3NF) · Boyce–Codd normal form (BCNF) · candidate key · surrogate key · natural (business) key · grain (recap) · dimensional modeling · fact · measure · dimension · fact table (recap) · dimension table (recap) · star schema (recap) · snowflake schema · date dimension (recap) · transaction fact table · periodic snapshot · accumulating snapshot · conformed dimension · slowly changing dimension (SCD) · type 1 · type 2 · type 3 · MERGE · point-in-time join · denormalization · materialized view · staleness · migration · drift · schema history table · checksum · repeatable migration · idempotent · expand and contract

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can write a recursive CTE that walks down or up a hierarchy, and explain the anchor, the recursive part, and when it stops.
- ☐ I multiply quantities down a bill of materials and can reconcile a rolled-up cost by hand.
- ☐ I protect recursive queries against loops, and I check that no rows went missing.
- ☐ I choose `ROWS`, `RANGE`, or `GROUPS` deliberately, and use intervals for calendar windows.
- ☐ I can read an `EXPLAIN ANALYZE` plan from the inside out and point to the expensive step.
- ☐ I know why a function on an indexed column prevents index use, and how to rewrite it.
- ☐ I can justify an index (single, composite, covering, or partial) with a before-and-after measurement, and I know what it costs.
- ☐ I can take a table to 3NF and name the dependency each step removes.
- ☐ I state the grain of every table before joining it, and test it with a query.
- ☐ I can design a star schema and load a type 2 dimension with a point-in-time join.
- ☐ I can explain when to denormalize and how to keep a summary honest.
- ☐ I never change a shared database by hand; I write a numbered, reviewed migration, and I use expand and contract for renames.

PRACTISE

Exercises

Use `riverstone_2025` (with this chapter's companion tables and the `dw` schema from section 28.9) unless an exercise says otherwise. Exercises 9 and 15 continue in `riverstone_lab`. Predict the shape of each result before running it.

Warm-up

1. Using a recursive CTE, list everyone who reports to Ramesh Patil, directly or indirectly, with their level *relative to him* (his direct reports are level 1).
2. How many people are at each level of the whole organization? Check that the levels add up to 35.
3. State the grain of `lead_stage_history` in one sentence, and write a query that tests it.
4. Find any staff whose `manager_id` points to a staff member who doesn't exist. (A correct query can return no rows. Explain how you know it would catch an orphan.)

Core

5. Produce the whole company as an indented org chart, each manager followed by their team, using PostgreSQL's `SEARCH DEPTH FIRST BY` clause.
6. For the Storage Box 25L, list each bought material, its total quantity per box, and its cost per box. Check that the costs add up to the material cost in section 28.2.
7. Purchasing plans to make 500 Garden Chairs and 1,000 Storage Box 10L next month. How many kilograms of polypropylene granules are needed?
8. For each day in December 2025 that had orders, show that day's number of orders and the number of orders in the 30 days up to and including that day.
9. (Lab.) In section 28.7, `orders_3nf` still stores the sales rep's name. Create `sales_reps_3nf` (with a surrogate key) and `orders_3nf_v2` that stores only the rep's key, then show that orders per rep are unchanged.
10. Using the star schema, show 2025 net revenue by customer city **as it was** and **as it is** for the cities where the two differ.

Stretch

11. Build a monthly sales-versus-target report from the star schema: one row per month, with revenue, target, and percentage of target. Explain in one sentence why you aggregated before joining.
12. Show the median order value for each customer segment (as it was) using `PERCENTILE_CONT`, and compare it with the mean.
13. Here's a staff extract with a data-entry error. Use `CYCLE` to find the loop, and list the ids that are in it: (10, NULL), (11, 10), (12, 14), (13, 12), (14, 13), (15, 11), as
(`staff_id`, `manager_id`).
14. For each month of 2025, show revenue and the average revenue of the **other** months in the same half-year (January–June or July–December).
15. (Lab.) Continue section 28.10's type 2 dimension. A new extract on 1 May 2026 changes Green Leaf Hotels' segment from Hospitality to Retail, and nothing else. Apply it with the same two steps, then show every version of every customer.

Think about it (no SQL needed)

16. A query plan shows Seq Scan on orders ... Filter: (status = 'Delivered') ... Rows Removed by Filter: 25235, on a table where 96% of orders are delivered. A colleague wants to add an index

on status. What do you tell them?

17. Rewrite each condition so it can use a plain index on its column, or explain why it can't be done:
(a) WHERE DATE(created_at) = '2025-12-15' on a timestamp column; (b) WHERE order_date + INTERVAL '30 days' > DATE '2025-12-31'; (c) WHERE LOWER(email) = 'rakesh@sharmahardware.example'; (d) WHERE customer_name LIKE '%Traders'.
18. Vikram asks for the customer's segment to be copied onto every row of orders "so reports don't need a join". Give two risks and a better alternative.
19. For each attribute, choose SCD type 1, 2, or 3, with a reason: (a) customer phone number; (b) customer's sales territory, used for commission reports; (c) product category, after a one-off reorganization that management wants to compare side by side for a year; (d) a misspelled customer name.
20. Review this migration for a 700,000-row, busy orders table and list what's wrong: ALTER TABLE orders RENAME COLUMN status TO order_status; ALTER TABLE orders ADD COLUMN channel VARCHAR(20) NOT NULL; CREATE INDEX idx_orders_channel ON orders (channel);

MySQL track (optional)

21. (MySQL.) Write exercise 2 (people per level) in MySQL.
22. (MySQL.) Write the where-used query for the fastener pack (part 306) in MySQL: which products use it, and how many packs per unit?
23. (MySQL.) Build a **daily** date spine for December 2025 with a recursive CTE. Use it to show how many days the month had, how many of those days had no orders, and the month's total revenue. Check the total against Chapter 13 (section 13.6).

Answers

1. Start the anchor at Ramesh Patil's direct reports, not at Ramesh himself:

```
WITH RECURSIVE team AS (  
  SELECT staff_id, staff_name, job_title, 1 AS level  
  FROM staff  
  WHERE manager_id = (SELECT staff_id FROM staff WHERE staff_name = 'Ramesh Patil')  
  UNION ALL  
  SELECT s.staff_id, s.staff_name, s.job_title, t.level + 1  
  FROM staff AS s  
  JOIN team AS t ON s.manager_id = t.staff_id  
)  
SELECT level, staff_name, job_title  
FROM team  
ORDER BY level, staff_id;
```

level	staff_name	job_title
1	Ajay Kumar	Shift Supervisor
1	Farhan Ali	Quality Inspector
2	Gopal Sahu	Machine Operator
2	Sunita Pawar	Machine Operator

(4 rows)

Four people, matching Ramesh Patil's `people_below` of 4 in section 28.2. ✓ A common mistake is to anchor on Ramesh himself and forget to exclude him, which returns five rows.

2.

```
WITH RECURSIVE org AS (  
  SELECT staff_id, 1 AS level FROM staff WHERE manager_id IS NULL  
  UNION ALL  
  SELECT s.staff_id, o.level + 1  
  FROM staff AS s  
  JOIN org AS o ON s.manager_id = o.staff_id  
)  
SELECT level, COUNT(*) AS people  
FROM org  
GROUP BY level  
ORDER BY level;
```

level	people
1	1
2	8
3	10
4	13
5	3

(5 rows)

$1 + 8 + 10 + 13 + 3 = 35$. ✓

3. Grain: *one lead entering one stage*. If that's true, (`lead_id`, `stage`) is unique:

```
SELECT lead_id, stage, COUNT(*) AS copies  
FROM lead_stage_history  
GROUP BY lead_id, stage  
HAVING COUNT(*) > 1;
```

lead_id	stage	copies
---------	-------	--------

(0 rows)

No rows: the grain holds. (The table's primary key already enforces it, but the test still belongs in a data-quality check, because keys are often missing from warehouse copies.)

4.

```
SELECT s.staff_id, s.staff_name, s.manager_id  
FROM staff AS s  
LEFT JOIN staff AS m ON m.staff_id = s.manager_id  
WHERE s.manager_id IS NOT NULL  
  AND m.staff_id IS NULL;
```

staff_id	staff_name	manager_id
----------	------------	------------

(0 rows)

No orphans. The query would catch one because the `LEFT JOIN` keeps every person, and a `manager_id` with no matching staff row leaves `m.staff_id` NULL. The `manager_id IS NOT NULL` condition stops the managing director, who legitimately has no manager, from being reported. (The foreign key on `manager_id` prevents orphans in this table; copies loaded into spreadsheets or warehouses often lack it.)

5.

```
WITH RECURSIVE org AS (
  SELECT staff_id, staff_name, manager_id, 1 AS level
  FROM staff
  WHERE manager_id IS NULL
  UNION ALL
  SELECT s.staff_id, s.staff_name, s.manager_id, o.level + 1
  FROM staff AS s
  JOIN org AS o ON s.manager_id = o.staff_id
)
SEARCH DEPTH FIRST BY staff_id SET visit_order
SELECT repeat('  ', level - 1) || staff_name AS org_chart
FROM org
ORDER BY visit_order;
```

```
      org_chart
-----
Arvind Kapoor
  Anita Rao
    Vikram Singh
      Neha Kulkarni
      Rahul Mehta
    Farah Khan
    Meera Iyer
    Arjun Nair
      Divya Krishnan
      Vivek Chandran
      Sneha Pillai
    Pooja Desai
      Nisha Bhatt
      Aditya Verma
      Rohit Kamat
    Sandeep Gill
      Karan Ahuja
      Ritu Bansal
    Suresh Menon
      Priya Nambiar
    Harpreet Sethi
      Ramesh Patil
        Ajay Kumar
          Gopal Sahu
          Sunita Pawar
        Farhan Ali
    Kiran Bhosale
      Swati Joshi
        Farid Shaikh
    Joseph D'Souza
    Mahesh Yadav
      Mohan Das
    Lakshmi Reddy
    Zoya Mirza
    Tenzin Dorji
(35 rows)
```

SEARCH DEPTH FIRST BY staff_id adds a hidden column, here named visit_order, that sorts each person immediately after their manager, with siblings in staff_id order. SEARCH BREADTH FIRST would sort level by level instead, like section 28.2's org chart query.

6.

```
WITH RECURSIVE explode AS (
  SELECT b.child_part_id, CAST(b.quantity AS NUMERIC) AS qty
  FROM bom_lines AS b
  WHERE b.parent_part_id = 102
  UNION ALL
  SELECT b.child_part_id, e.qty * b.quantity
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT p.part_name,
       CAST(SUM(e.qty) AS NUMERIC(8,3)) AS qty_per_box,
       p.unit,
       CAST(SUM(e.qty) * p.material_cost AS NUMERIC(10,2)) AS cost_per_box
FROM explode AS e
JOIN parts AS p ON p.part_id = e.child_part_id
WHERE p.part_type = 'Material'
GROUP BY p.part_id, p.part_name, p.unit, p.material_cost
ORDER BY p.part_id;
```

part_name	qty_per_box	unit	cost_per_box
Polypropylene granules	1.500	kg	165.00
Color masterbatch	0.055	kg	14.30
Product label	1.000	each	2.00
Shipping carton	1.000	each	18.00

(4 rows)

₹165.00 + ₹14.30 + ₹2.00 + ₹18.00 = ₹199.30, matching section 28.2. ✓ The granules are 1.1 kg in the body plus 0.4 kg in the lid, so group by part before multiplying by cost.

7.

```
WITH RECURSIVE plan (product_id, units) AS (
  VALUES (106, 500), (101, 1000)
),
explode AS (
  SELECT b.child_part_id, CAST(b.quantity * p.units AS NUMERIC) AS qty
  FROM plan AS p
  JOIN bom_lines AS b ON b.parent_part_id = p.product_id
  UNION ALL
  SELECT b.child_part_id, e.qty * b.quantity
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT CAST(SUM(qty) AS NUMERIC(10,1)) AS kg_polypropylene
FROM explode
WHERE child_part_id = 301;
```

```
kg_polypropylene
-----
1850.0
(1 row)
```


Hand-check: a chair needs 2.2 kg ($500 \times 2.2 = 1,100$ kg); a 10-litre box needs $0.55 + 0.20 = 0.75$ kg ($1,000 \times 0.75 = 750$ kg). Total 1,850 kg. ✓ Putting the production plan into a `VALUES` list at the start makes the anchor multiply by planned units, so the rest of the query is unchanged. (The first CTE isn't recursive; `WITH RECURSIVE` is written once for the whole list.)

8. Count orders per day first (the grain is one day), then use a `RANGE` frame. Starting the daily counts 29 days before December makes the first December rows see a full 30 days:

```
WITH daily AS (  
    SELECT order_date, COUNT(DISTINCT order_id) AS orders  
    FROM sales_lines  
    WHERE order_date >= DATE '2025-11-02'  
    GROUP BY order_date  
) ,  
rolling AS (  
    SELECT order_date,  
           orders,  
           SUM(orders) OVER (ORDER BY order_date  
                           RANGE BETWEEN INTERVAL '29 days' PRECEDING AND CURRENT ROW) AS  
                           orders_last_30_days  
    FROM daily  
)  
SELECT *  
FROM rolling  
WHERE order_date >= DATE '2025-12-01'  
ORDER BY order_date;
```

order_date	orders	orders_last_30_days
2025-12-01	2	23
2025-12-04	1	23
2025-12-05	1	23
2025-12-08	1	23
2025-12-10	2	22
2025-12-12	1	20
2025-12-13	1	21
2025-12-14	1	22
2025-12-16	3	22
2025-12-17	1	23
2025-12-18	1	23
2025-12-19	1	23
2025-12-21	1	24
2025-12-23	1	22

(14 rows)

`INTERVAL '29 days' PRECEDING AND CURRENT ROW` covers 30 days including today. The common wrong answer is `ROWS BETWEEN 29 PRECEDING AND CURRENT ROW`, which covers 30 *order days*, more than a month in this data.

9.

```
CREATE TABLE sales_reps_3nf (  
    sales_rep_id INTEGER PRIMARY KEY,  
    sales_rep    VARCHAR(100) NOT NULL UNIQUE  
);  
  
INSERT INTO sales_reps_3nf  
SELECT ROW_NUMBER() OVER (ORDER BY sales_rep), sales_rep  
FROM (SELECT DISTINCT sales_rep FROM orders_3nf) AS r;
```



```

CREATE TABLE orders_3nf_v2 AS
SELECT o.order_id, o.order_date, o.customer_id, r.sales_rep_id
FROM orders_3nf AS o
JOIN sales_reps_3nf AS r ON r.sales_rep = o.sales_rep;

SELECT 'before' AS version, sales_rep, COUNT(*) AS orders
FROM orders_3nf
GROUP BY sales_rep
UNION ALL
SELECT 'after', r.sales_rep, COUNT(*)
FROM orders_3nf_v2 AS o
JOIN sales_reps_3nf AS r ON r.sales_rep_id = o.sales_rep_id
GROUP BY r.sales_rep
ORDER BY sales_rep, version DESC;

```

version	sales_rep	orders
before	Farah Khan	2
after	Farah Khan	2
before	Neha Kulkarni	3
after	Neha Kulkarni	3
before	Rahul Mehta	1
after	Rahul Mehta	1

(6 rows)

Same counts before and after (2 + 1 + 3 = 6 orders). ✓ The rep's name now lives in one row, so a corrected spelling fixes every order at once.

10.

```

WITH was AS (
    SELECT c.city, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    GROUP BY c.city
),
is_now AS (
    SELECT cur.city, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    JOIN dw.dim_customer AS cur ON cur.customer_id = c.customer_id AND cur.is_current
    GROUP BY cur.city
)
SELECT COALESCE(w.city, i.city) AS city,
       ROUND(w.revenue) AS revenue_as_it_was,
       ROUND(i.revenue) AS revenue_as_it_is
FROM was AS w
FULL JOIN is_now AS i ON i.city = w.city
WHERE w.revenue IS DISTINCT FROM i.revenue
ORDER BY city;

```

city	revenue_as_it_was	revenue_as_it_is
Mumbai	928760	1080800
Thane	152040	

(2 rows)

Only Metro Mart moved city, so only Mumbai and Thane differ: Metro Mart's orders from January to June 2025 count under Thane as they were. The `FULL JOIN` matters: Thane has no revenue "as it is" (no current customer is in Thane), and an inner join would drop it.

11.

```
WITH monthly AS (  
    SELECT d.month_start, SUM(f.net_revenue) AS revenue  
    FROM dw.fact_sales_line AS f  
    JOIN dw.dim_date AS d ON d.date_key = f.date_key  
    GROUP BY d.month_start  
)  
SELECT m.month_start,  
       ROUND(m.revenue) AS revenue,  
       ROUND(t.target_revenue) AS target,  
       ROUND(100 * m.revenue / t.target_revenue, 1) AS pct_of_target  
FROM monthly AS m  
JOIN sales_targets AS t ON t.target_month = m.month_start  
ORDER BY m.month_start;
```

month_start	revenue	target	pct_of_target
2025-01-01	202640	300000	67.5
2025-02-01	253664	300000	84.6
2025-03-01	278008	320000	86.9
2025-04-01	210282	320000	65.7
2025-05-01	329359	320000	102.9
2025-06-01	186928	300000	62.3
2025-07-01	232692	280000	83.1
2025-08-01	329282	320000	102.9
2025-09-01	558315	380000	146.9
2025-10-01	681071	520000	131.0
2025-11-01	633408	500000	126.7
2025-12-01	439824	380000	115.7

(12 rows)

The fact table's grain is one order line and the target's grain is one month; aggregating the facts to months first makes the join one-to-one, so the target is counted once (section 28.8). January's 67.5% matches section 28.8. ✓

12.

```
WITH order_values AS (  
    SELECT c.segment, f.order_id, SUM(f.net_revenue) AS order_value  
    FROM dw.fact_sales_line AS f  
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key  
    GROUP BY c.segment, f.order_id  
)  
SELECT segment,  
       COUNT(*) AS orders,  
       ROUND(AVG(order_value)) AS mean_value,  
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)::NUMERIC) AS median_value  
FROM order_values  
GROUP BY segment  
ORDER BY segment;
```

segment	orders	mean_value	median_value
Hospitality	59	19390	17589
Retail	62	25381	22649
Wholesale	52	31112	29199

(3 rows)

The order counts add up to 173. ✓ In every segment the mean is above the median, so each segment has a few large orders pulling its average up. `PERCENTILE_CONT` is an aggregate, so it works with `GROUP BY` like `AVG`.

13.

```
WITH RECURSIVE extract_rows (staff_id, manager_id) AS (  
    VALUES (10, NULL), (11, 10), (12, 14), (13, 12), (14, 13), (15, 11)  
)  
,  
walk AS (  
    SELECT staff_id, manager_id, staff_id AS started_at  
    FROM extract_rows  
    UNION ALL  
    SELECT e.staff_id, e.manager_id, w.started_at  
    FROM extract_rows AS e  
    JOIN walk AS w ON e.staff_id = w.manager_id  
)  
CYCLE staff_id SET is_loop USING visited  
SELECT DISTINCT started_at AS staff_in_loop  
FROM walk  
WHERE is_loop AND staff_id = started_at  
ORDER BY started_at;
```

```
staff_in_loop  
-----  
          12  
          13  
          14  
(3 rows)
```

The walk starts at every person and goes up the chain. People 10, 11, and 15 reach the top (10 has no manager). Starting from 12, 13, or 14, the walk comes back to its starting person, so those three form the loop (12 → 14 → 13 → 12). Filtering on `staff_id = started_at` keeps only people who return to themselves; someone *below* a loop would reach it without returning to their own id.

14.

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month,  
           SUM(net_revenue) AS revenue  
    FROM sales_lines  
    GROUP BY 1  
)  
SELECT month,  
       ROUND(revenue) AS revenue,  
       ROUND(AVG(revenue) OVER (  
           PARTITION BY EXTRACT(MONTH FROM month) <= 6  
           ORDER BY month  
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
           EXCLUDE CURRENT ROW)) AS avg_other_months_in_half  
FROM monthly  
ORDER BY month;
```

month	revenue	avg_other_months_in_half
2025-01-01	202640	251648
2025-02-01	253664	241443
2025-03-01	278008	236574
2025-04-01	210282	250120
2025-05-01	329359	226304
2025-06-01	186928	254790
2025-07-01	232692	528380
2025-08-01	329282	509062
2025-09-01	558315	463255
2025-10-01	681071	438704
2025-11-01	633408	448237
2025-12-01	439824	486954

(12 rows)

Partitioning by the true/false expression `EXTRACT(MONTH FROM month) <= 6` makes two halves. Each month is compared with the five others in its half.

15.

```
DELETE FROM customer_extract;
INSERT INTO customer_extract VALUES
(1, 'Sharma Hardware', 'Thane', 'Retail'),
(3, 'Green Leaf Hotels', 'Pune', 'Retail'),
(4, 'Coastal Foods', 'Chennai', 'Wholesale');

BEGIN;

UPDATE dim_customer_t2 AS d
SET valid_to = DATE '2026-05-01',
    is_current = FALSE
FROM customer_extract AS x
WHERE d.customer_id = x.customer_id
    AND d.is_current
    AND (d.city IS DISTINCT FROM x.city OR d.segment IS DISTINCT FROM x.segment);

INSERT INTO dim_customer_t2 (customer_id, customer, city, segment, valid_from)
SELECT x.customer_id, x.customer, x.city, x.segment, DATE '2026-05-01'
FROM customer_extract AS x
LEFT JOIN dim_customer_t2 AS d
    ON d.customer_id = x.customer_id AND d.is_current
WHERE d.customer_id IS NULL;

COMMIT;

SELECT customer_key, customer, city, segment, valid_from, valid_to, is_current
FROM dim_customer_t2
ORDER BY customer_id, valid_from;
```

customer_key	customer	city	segment	valid_from	valid_to	is_current
1	Sharma Hardware	Mumbai	Retail	2024-11-04	2026-04-01	f
3	Sharma Hardware	Thane	Retail	2026-04-01	9999-12-31	t
2	Green Leaf Hotels	Pune	Hospitality	2024-09-08	2026-05-01	f
5	Green Leaf Hotels	Pune	Retail	2026-05-01	9999-12-31	t
4	Coastal Foods	Chennai	Wholesale	2026-04-01	9999-12-31	t

(5 rows)

Only Green Leaf Hotels gets a new version. Sharma Hardware and Coastal Foods are unchanged, so step 1 leaves their current rows open and step 2 finds they already have one. Running the same two steps a second time would change nothing, which is how you check a load is idempotent. (`valid_to` stores the first day the version is no longer true, which is the next version's `valid_from`, as in `dw.dim_customer`. With half-open joins (`>= valid_from AND < valid_to`) there is then no gap and no overlap. If you store the last day a version was true instead (an inclusive convention, 2026-04-30), you must join with `<= valid_to`. Mixing the two drops or doubles rows at every change.)

16. Don't add it. The filter keeps 96% of rows, so an index on `status` would be slower than scanning: the planner would ignore it, as it did for 'Delivered' in section 28.5, and every insert and status change would pay to maintain it. If a query really needs the rare statuses (pending or cancelled), a **partial index** on those rows is the right tool. And if the delivered query is slow, the fix is elsewhere: a summary table, or not reading every delivered order.

17. (a) `WHERE created_at >= '2025-12-15' AND created_at < '2025-12-16'`. The half-open range also includes times after midnight, which `BETWEEN '2025-12-15' AND '2025-12-15'` would miss. (b) Move the arithmetic to the constant side: `WHERE order_date > DATE '2025-12-31' - INTERVAL '30 days'`, which is `order_date > '2025-12-01'`. (c) Store emails in one case when they're saved and compare directly, or create an expression index on `LOWER(email)` and keep the query as written. (d) A plain B-tree index can't find text by its *ending*. Options: a reversed-text expression index, a trigram index (PostgreSQL's `pg_trgm` extension), or a separate `business_type` column. The common wrong answer is "add an index on `customer_name`", which the planner would not use for a leading %.

18. Risks: (1) the copies disagree when a customer's segment changes and not every order is updated, an update anomaly (section 28.7); (2) it's unclear whether an order should show the segment at the time of the order or today's, and the copy silently picks one. Better alternatives: a star schema with a type 2 customer dimension (both views available, section 28.10), or a view that joins orders to customers so reports don't write the join themselves. If the join is truly too slow, a materialized view refreshed on a schedule keeps the copy derived and rebuildable.

19. (a) Type 1. Nobody reports sales by old phone numbers; the new one replaces it. (b) Type 2. Commission is calculated on who owned the customer when the sale happened, so history must be kept. (c) Type 3. A `previous_category` column lets reports show this year's sales under both the old and new categories side by side, which is exactly the request. (If they later wanted the full history, type 2 would be needed.) (d) Type 1. It's a correction: the misspelled name was never true, so there's no history to keep.

20. Five problems. (1) The rename breaks every report, view, and application that reads `status`, at the moment it runs: use `expand` and `contract`. (2) `channel VARCHAR(20) NOT NULL` with no default fails on a table with rows (*column "channel" ... contains null values*): add a default, or add it nullable, backfill, then set `NOT NULL`. (3) A plain `CREATE INDEX` blocks writes on a busy table while it builds: use `CREATE INDEX CONCURRENTLY`, in its own migration outside a transaction. (4) Three unrelated changes in one migration: split them. (5) An index on `channel` before anyone has measured a query that needs it: add it only for a known query, preferably with a before-and-after plan.

21.

```
WITH RECURSIVE org AS (  
    SELECT staff_id, 1 AS level FROM staff WHERE manager_id IS NULL  
    UNION ALL  
    SELECT s.staff_id, o.level + 1  
    FROM staff AS s  
    JOIN org AS o ON s.manager_id = o.staff_id  
)  
SELECT level, COUNT(*) AS people  
FROM org  
GROUP BY level  
ORDER BY level;
```

level	people
1	1
2	8
3	10
4	13
5	3

Identical to PostgreSQL's result in exercise 2. No `CAST` is needed, because `level` is a number and doesn't grow in size.

22.

```
WITH RECURSIVE uses AS (  
    SELECT b.parent_part_id, CAST(b.quantity AS DECIMAL(12,3)) AS qty  
    FROM bom_lines AS b  
    WHERE b.child_part_id = 306  
    UNION ALL  
    SELECT b.parent_part_id, CAST(u.qty * b.quantity AS DECIMAL(12,3))  
    FROM uses AS u  
    JOIN bom_lines AS b ON b.child_part_id = u.parent_part_id  
)  
SELECT p.part_name, SUM(u.qty) AS packs_per_unit  
FROM uses AS u  
JOIN parts AS p ON p.part_id = u.parent_part_id  
WHERE p.part_type = 'Product'  
GROUP BY p.part_name;
```

part_name	packs_per_unit
Garden Chair	3.000

Three packs per Garden Chair: one in the frame and one in each of the two leg assemblies (Figure 28.2). ✓ In MySQL, the `CAST` in both parts keeps the column's type fixed, so the multiplied quantities aren't cut to the anchor's precision.

23. A recursive CTE produces the 31 days; a left join keeps the days with no sales:

```
WITH RECURSIVE days AS (  
    SELECT DATE '2025-12-01' AS day  
    UNION ALL  
    SELECT day + INTERVAL 1 DAY  
    FROM days  
    WHERE day < DATE '2025-12-31'  
) ,  
daily_sales AS (  
    SELECT order_date, SUM(net_revenue) AS revenue  
    FROM sales_lines  
    WHERE order_date >= '2025-12-01' AND order_date < '2026-01-01'  
    GROUP BY order_date  
)  
SELECT COUNT(*) AS days_in_month,  
       SUM(CASE WHEN s.order_date IS NULL THEN 1 ELSE 0 END) AS days_without_orders,  
       ROUND(SUM(COALESCE(s.revenue, 0)), 0) AS month_revenue  
FROM days AS d  
LEFT JOIN daily_sales AS s ON s.order_date = d.day;
```

```
+-----+-----+-----+  
| days_in_month | days_without_orders | month_revenue |  
+-----+-----+-----+  
|          31 |          17 |    439824 |  
+-----+-----+-----+
```

The revenue matches December in section 13.6 (₹4,39,824). ✓ Orders came in on only 14 of December's 31 days, which is normal for a B2B supplier whose customers order every few weeks, and a reason daily charts for this kind of business need a date spine: without one, the 17 empty days would simply vanish. `SUM(CASE WHEN s.order_date IS NULL THEN 1 ELSE 0 END)` counts the unmatched days. A 31-row spine needs 30 passes, well within MySQL's 1,000-pass limit, so no setting needs changing.

NEXT

Where this leads

- **Chapter 34, The Command Line, Linux & Networking Basics**, comes next in Part 3's reading order. It takes the terminal you used for `riverstone_perf` and `migrate.py` further: PATH, shell scripts, servers, and the network between you and a database.
- **Chapter 29, Python as Software, Not Scripts**, turns scripts like `migrate.py` and `generate_riverstone_perf.py` into tested, packaged code.
- **Chapter 32, Analytics Engineering with dbt**, rebuilds this chapter's star schema as dbt models with tests, documentation, and snapshots for type 2 history, replacing hand-written load scripts.
- **Chapter 47, Data Quality, Observability & Contracts**, turns grain tests and reconciliations into automated daily checks.
- **Chapter 49, Storage, Warehouses & Lakehouses**, takes the same questions of speed and cost to cloud warehouses, partitioning, and columnar storage.
- **Chapter 62, Data Architecture Patterns**, compares warehouse modeling approaches at the architect's level.

- **Chapter 71, SQL Question Bank**, has interview questions on recursive CTEs, indexes, query plans, normalization, grain, and slowly changing dimensions; **Chapter 77** covers data modeling in data engineering and system design interviews.

Chapter 34. The Command Line, Linux & Networking Basics

Chapter at a glance

You will learn to: combine small tools with pipes and redirection to answer questions in seconds · search text with `grep`, and count and summarize it with `wc`, `sort`, `uniq`, `cut`, and `awk` · find files by name, age, and size · read and set file permissions · use environment variables and understand `PATH` · write a shell script that takes arguments, fails safely, and returns a meaningful exit code · connect to a server with SSH keys and copy files to it · explain IP addresses, DNS, ports, and HTTP, and test a web server with `curl`.

Before you start: Chapter 26, section 26.0 (the terminal: moving around, making and copying files, reading a command, `&&`, environment variables, and `echo $?`) and Chapter 2 (files, formats, and what an API is). Chapter 20's scheduled jobs are useful background, and Chapter 14's regular expressions help with `grep -E`.

Time needed: 9–12 hours over two weeks, in three sittings (sections 34.0–34.4; 34.5–34.7; 34.8–34.9 and the project), most of it typing commands rather than reading.

Tools: a terminal running `bash`. On Windows, **WSL** (the Windows Subsystem for Linux), which section 34.1 installs; Git Bash from Chapter 26 is enough for sections 34.0 to 34.4 and 34.7, but not for permissions (34.5), `sudo`, `ss`, or the SSH server. On macOS, the Terminal app with `bash` typed first (section 34.1). On Linux, any terminal. Windows and macOS differences are flagged where they matter.

Practice data: the companion folder `ch34/`, which builds a `practice/` folder from Riverstone's 2025 sales: 61 daily export files for November and December 2025, reference lists, and the daily summary job's log, next to the whole year's sales lines in one file (`sales_lines_2025.csv`). Every command in this chapter was run, and every output shown is the real output.

In this chapter

START	Why this matters · In plain English	107
LEARN	34.0 Where Chapter 26 left you	108
	34.1 Setting up for this chapter	109
	34.2 Pipes, redirection, and exit codes	111
	34.3 Looking inside files, and finding them	113
	34.4 Searching and summarizing text	115
	34.5 Permissions: who can read, write, and run	119
	34.6 Environment variables and PATH	121
	34.7 Shell scripts	123
	34.8 SSH: working on another machine	127
	34.9 Networking in plain English	129
APPLY	Common mistakes · In the real world · Project	132
REVIEW	Recap · Key terms · Check yourself	138
PRACTISE	Exercises	139
	Answers	140
NEXT	Where this leads	145

START

Why this matters

You've been typing commands since Chapter 17: `python` and `pip` there, and `git` in Chapter 26. The rest of the book adds more: `uv` and `pytest` (Chapter 29) are commands, and so is `dbt` (Chapter 32). Scheduled jobs (Chapter 20) are commands the machine runs at 3 a.m. with nobody watching. Cloud servers and containers (Chapter 52) have no desktop at all: a terminal is the only way in. Chapter 26 gave you enough terminal to use Git. This chapter turns "I can type the command I was given" into "I can find my way around a machine I've never seen".

There's a second reason, and it shows up sooner. A surprising share of daily data work is faster in the terminal than anywhere else:

- "How many rows are in these 61 files, and which ones are empty?" One command.
- "The pipeline failed last night. What did it say?" `grep ERROR` on the log, in the time it takes a log viewer to load.
- "This 2 GB CSV crashes Excel and takes five minutes to load in pandas. I only need the rows for Pune." `grep Pune` finishes before pandas has started.
- "Which of these 400 files contains the customer id we're arguing about?" `grep -rl`.

You don't need to memorize the manual. You need about thirty commands, a clear picture of how they fit together, and the confidence to look up the rest.

In plain English

The terminal is a kitchen counter with a lot of small, sharp knives.

A graphical program is a food processor: one machine, a lid, a few buttons, and whatever it does is what you get. The terminal's tools each do one small thing, and you connect them. One tool finds lines; one counts them; one sorts them; one adds up a column. Alone they're almost too simple to be worth having. Connected with **pipes**, they answer questions that no single program on your machine can answer.

The counter has a few other fixtures:

- **A current folder.** Every command runs somewhere. Half of all early mistakes are "I was in the wrong place".
- **Three channels.** Every command takes **input**, produces **output**, and has a separate channel for **errors**, so you can keep the results and read the complaints separately.
- **A verdict.** Every command ends with a number: 0 means it worked. That number is how scheduled jobs know whether to raise an alarm.
- **Labels on the drawers.** **Environment variables** are named notes the shell hands to every program it starts: where to find things, which database to use, your API key.
- **A telephone line to other kitchens.** **SSH** gives you the same counter on a machine in another building, or another country.

LEARN

34.0 Where Chapter 26 left you

Chapter 26, section 26.0, taught the terminal you needed for Git. This chapter starts from there. Here is that section on one screen; every line should look familiar.

Command or key	What it does	Example
<code>pwd</code>	print the folder you're in	<code>pwd</code>
<code>ls</code> , <code>ls -a</code>	list what's here; <code>-a</code> includes hidden names that start with a dot	<code>ls -a</code>
<code>cd</code>	move into a folder; <code>..</code> is the folder above; <code>~</code> is your home folder	<code>cd sql</code> , <code>cd ..</code> , <code>cd ~</code>
<code>mkdir -p</code>	make a folder, with any missing folders on the way	<code>mkdir -p sql/reports</code>
<code>touch</code>	make an empty file	<code>touch notes.txt</code>
<code>cp</code> , <code>mv</code>	copy; move or rename	<code>cp .env.example .env</code>
<code>rm</code>	delete, with no recycle bin and no undo	<code>rm notes.txt</code>
<code>cat</code> , <code>less</code>	print a file; page through a long one (<code>q</code> quits)	<code>cat .gitignore</code>
<code>echo "..."</code> > file	write text into a file, replacing what was there	<code>echo "scratch" > notes.txt</code>
<code>Tab</code> , <code>↑</code> , <code>Ctrl+C</code>	complete a name; bring back the last command; stop a command	
<code>&&</code>	run the next command only if this one worked	<code>cd sql && ls</code>
<code>export NAME=value</code>	set an environment variable for this terminal	<code>export REPORT_MONTH=2026-01</code>

Command or key	What it does	Example
<code>echo \$?</code>	the exit code of the last command: 0 means it worked	<code>echo \$?</code>

If any of these feels unfamiliar, redo the drill at the end of section 26.0 before going on: the rest of this chapter builds on every line.

Three more keys, now that you'll be typing more. `history` lists the commands you've typed, numbered. `Ctrl+R` searches them: press it, type a few letters of an old command, and press Enter to run it again. `Ctrl+D` means "no more input"; in a terminal on its own, it closes the shell, like typing `exit`.

One way of printing commands is new here. A long command doesn't fit on one line of this book, so it's split with a **backslash** at the very end of a line:

```
$ curl --silent --output /dev/null \
  --write-out "%{http_code}\n" \
  http://127.0.0.1:8034/health
```

A backslash as the last character of a line means "the command continues on the next line". You can type it exactly as printed, pressing Enter after each backslash, or type it all on one line without the backslashes; bash runs the same command either way.

What's new in this chapter:

- **Pipes and redirection:** connecting small tools so the output of one becomes the input of the next (section 34.2).
- **Text tools:** `head`, `tail`, `find`, `grep`, `wc`, `sort`, `uniq`, `cut`, and `awk`, which count, search, and add up files without opening them (sections 34.3 and 34.4).
- **Linux itself:** permissions, `sudo`, and `PATH`, which decide who can do what and which program a name runs (sections 34.5 and 34.6).
- **Shell scripts:** saving commands in a file that takes arguments and fails safely (section 34.7).
- **Other machines:** SSH, and the four networking ideas behind every connection (sections 34.8 and 34.9).

34.1 Setting up for this chapter

Chapter 26's terminal was enough for Git. This chapter needs a real Linux shell, because half of it is about how Linux itself works.

Your computer	What to use for this chapter	Why
Linux	your usual terminal (<code>Ctrl+Alt+T</code> on most desktops)	it already runs bash
macOS	the Terminal app, then type <code>bash</code> and press Enter	Terminal starts zsh , which differs from <code>bash</code> in a few places this chapter uses. macOS includes an older <code>bash</code> , 3.2, which runs every script here

Your computer	What to use for this chapter	Why
Windows	WSL , the Windows Subsystem for Linux (steps below)	gives you a real Ubuntu Linux inside Windows
Windows, for now	Git Bash, from Chapter 26	fine for sections 34.0 to 34.4 and 34.7; not for permissions (34.5), <code>sudo</code> , <code>ss</code> , or the SSH server (34.8)

The **shell** is the program that reads what you type and runs it. Bash and zsh are both shells; this chapter uses bash, the one on almost every Linux server.

Windows: install WSL

WSL runs a real Linux, Ubuntu by default, next to Windows. Microsoft's instructions ("How to install Linux on Windows with WSL", Microsoft Learn) come down to five steps. They need administrator rights on the computer, and a restart:

1. Open **PowerShell as administrator**: press the Windows key, type *PowerShell*, right-click it, and choose *Run as administrator*.
2. Type `wsl --install` and press Enter. It turns on the Windows features WSL needs and downloads Ubuntu.
3. **Restart** the computer when it finishes.
4. Open **Ubuntu** from the Start menu. The first start takes a minute, then asks you to create a **UNIX username and password**. They are for Linux only and don't need to match your Windows ones. Remember the password: `sudo` asks for it (section 34.5), and nothing on the screen moves while you type it, which is normal.
5. Bring Ubuntu's software list up to date: `sudo apt update`, then your new password.

Two habits make WSL behave. **Work inside your Linux home folder** (`cd ~`), not in your Windows folders: Windows drives appear under `/mnt/c/...`, and there permissions don't behave as Linux expects and everything runs slower (Microsoft recommends the same, in "Working across Windows and Linux file systems"). And **open VS Code from Ubuntu**: with VS Code's **WSL** extension installed, typing `code .` in the Ubuntu terminal opens the current Linux folder in VS Code.

Build the practice folder

Copy the companion folder `ch34` into your home folder, go into the copy, and run its generator. On macOS and Linux, the companion files are where Chapter 17 put them:

```
# terminal, in your home folder
$ cp -r ~/analyst-to-architect/companion/ch34 ~/ch34

$ cd ~/ch34

$ python3 make_ch34_data.py
practice folder ready: 326 order lines, 23 customers, 61 daily export files
```

`cp -r` copies a folder and everything inside it (`-r` for *recursive*). In WSL, your Windows files are under `/mnt/c/Users/`, so the first command becomes something like `cp -r /mnt/c/Users/Meera/analyst-to-architect/companion/ch34 ~/ch34` (with your Windows user name). The generator is plain Python, so `python3` from Ubuntu or macOS runs it without a virtual environment; on a fresh WSL, `sudo apt install python3` first if `python3 --version` says it isn't there.

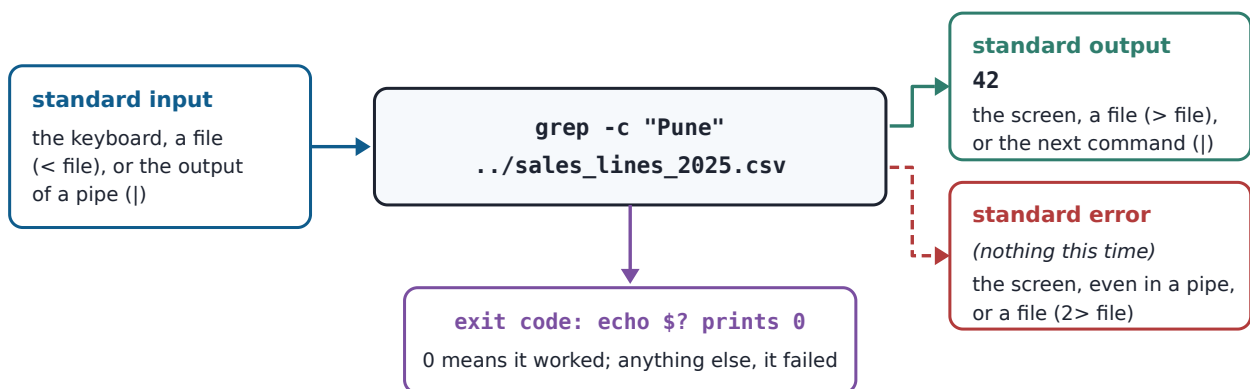
Throughout this chapter, `$` marks a line you type and everything after it is the output, as in Chapter 26. Your prompt shows your own name and folder.

34.2 Pipes, redirection, and exit codes

This is the idea that makes everything else worth learning.

Every command has three channels: **standard input** (where it reads from), **standard output** (where results go), and **standard error** (where complaints go). By default input comes from your keyboard, and both output channels go to the screen. And every command, when it finishes, leaves an **exit code**: 0 for success, anything else for failure.

One command: one channel in, two channels out, and an exit code



Output and errors are separate channels, so a pipe or a file can take one and leave the other on the screen.

Figure 34.1 — Every command reads one channel, writes two, and leaves an exit code behind.

You can point the channels somewhere else:

Symbol	Meaning
<code>command > file</code>	send output to a file, replacing it (section 26.0)
<code>command >> file</code>	append output to the end of a file
<code>command 2> errors.txt</code>	send only errors to a file
<code>command > out.txt 2>&1</code>	send output and errors to the same file
<code>command < file</code>	read input from a file
<code>command1 command2</code>	send command1's output straight into command2: a pipe

The first pipe, and the question from the start of the chapter: how many daily files are there?

```
# terminal, in ~/ch34
$ cd practice

$ ls exports | wc -l
61
```

`ls exports` lists the folder's files, one per line; the `|` sends that list into `wc -l`, which counts lines (word count, `-l` for lines) and prints only the count. Neither program knows the other exists; the shell connects them.

```
# terminal, in ~/ch34/practice
$ head -1 exports/orders_2025-12-16.csv > /tmp/header.csv

$ cat /tmp/header.csv
order_id,order_date,customer_id,customer_name,city,segment,product_id,product_name,category,quantity,net_revenue ↵
,sales_rep

$ ls missing_folder 2> /tmp/errors.txt

$ cat /tmp/errors.txt
ls: cannot access 'missing_folder': No such file or directory
```

`head -1` prints a file's first line, and `>` saved it in `/tmp`, the system's folder for throwaway files. Notice the third command printed nothing: the error went to the file instead of the screen. That's why a scheduled job writes `>> job.log 2>&1`: everything, results and complaints, ends up in one file with nothing lost. **Order matters** in that last one: redirect the output first, then `2>&1`, which means "send errors wherever output is going now". Written the other way round, `2>&1 > job.log`, the errors still go to the screen.

tee does both at once: it shows its input on the screen *and* writes it to a file. `-a` appends instead of replacing. Section 34.7 uses it to keep a log of a script's runs.

Exit codes

The shell keeps the last exit code in `$?` , which section 26.0 printed with `echo $?`. **test** checks a condition and says the answer only through its exit code:

```
# terminal, in ~/ch34/practice
$ test -f exports/orders_2025-12-16.csv; echo $?
0

$ test -f exports/orders_2026-01-05.csv; echo $?
1
```

`test -f` asks "is this an existing file?" and prints nothing; the answer is the exit code. The file for 16 December exists (0); the one for 5 January 2026 hasn't been published (1). The `;` runs the second command whatever happened to the first. A scheduler reads exactly this number to decide whether a job failed.

Exit codes also chain commands:

- `command1 && command2` runs the second **only if** the first succeeded (section 26.0).
- `command1 || command2` runs the second **only if** the first failed.
- `command1 ; command2` runs both regardless.


```
# terminal, in ~/ch34/practice
$ test -f exports/orders_2025-12-16.csv && echo "file is there"
file is there

$ test -f exports/orders_2026-01-05.csv || echo "not published yet"
not published yet
```

34.3 Looking inside files, and finding them

`ls` has a long form that you'll need for permissions in section 34.5:

```
# terminal, in ~/ch34/practice
$ ls
archive
exports
incoming
logs
reference

$ ls -l reference
total 8
-rw-r--r-- 1 meera meera 877 Jan  5 2026 customers.csv
-rw-r--r-- 1 meera meera 259 Jan  5 2026 products.csv
```

- `ls -l` is the long form: permissions, owner, group, size in bytes, when it last changed, and the name. Section 34.5 reads it column by column. `ls -lh` prints human-readable sizes (4.0K, 12M).
- Options combine: `ls -lha` is `-l`, `-h`, and `-a` together. Most commands also accept long forms (`ls --all`), which are clearer in scripts.
- `cd -` goes back to the folder you were in before the last `cd`, which saves typing when you're switching between two places.

Names are **case-sensitive** on Linux: `Exports` and `exports` are different folders. That's one of the most common surprises when a script written on a Mac or Windows machine runs on a Linux server for the first time.

Looking inside files without opening an editor

```
# terminal, in ~/ch34/practice
$ head -2 exports/orders_2025-12-16.csv
order_id,order_date,customer_id,customer_name,city,segment,product_id,product_name,category,quantity,net_revenue ↵
,sales_rep
10168,2025-12-16,5,Metro Mart,Mumbai,Retail,101,Storage Box 10L,Storage,35,15050.00,Rahul Mehta

$ tail -2 exports/orders_2025-12-16.csv
10170,2025-12-16,21,Kitchen Kraft,Bengaluru,Retail,101,Storage Box 10L,Storage,35,15050.00,Rahul Mehta
10170,2025-12-16,21,Kitchen Kraft,Bengaluru,Retail,104,Food Container Set,Kitchen,35,21700.00,Rahul Mehta

$ wc -l exports/orders_2025-12-16.csv
8 exports/orders_2025-12-16.csv
```

- `head` shows the first lines (10 by default), `tail` the last. `tail -f somefile.log` follows a file as it grows, which is how you watch a job while it runs; `Ctrl+C` stops following.

- `wc -l` with a file name prints the count and the name. Eight lines is seven order lines plus the header row.
- `less` (section 26.0) can search: inside it, type `/Pune` and Enter to jump to the next match, `n` for the one after, and `q` to quit. Use `less` for anything longer than a screen.

Wildcards

The shell expands patterns *before* the command runs:

```
# terminal, in ~/ch34/practice
$ ls exports/orders_2025-12-0*.csv
exports/orders_2025-12-01.csv
exports/orders_2025-12-02.csv
exports/orders_2025-12-03.csv
exports/orders_2025-12-04.csv
exports/orders_2025-12-05.csv
exports/orders_2025-12-06.csv
exports/orders_2025-12-07.csv
exports/orders_2025-12-08.csv
exports/orders_2025-12-09.csv

$ ls exports/orders_2025-12-1?.csv | wc -l
10
```

`*` matches any number of characters, `?` matches exactly one, and `[12]` matches either character listed. This is called **globbing**, and it's the shell's work, not the command's: `ls` only ever sees the list of names the shell found.

Finding files

`find` searches a folder tree by name, age, size, or type:

```
# terminal, in ~/ch34/practice
$ find . -name "orders_2025-12-2*.csv" | sort | head -3
./exports/orders_2025-12-20.csv
./exports/orders_2025-12-21.csv
./exports/orders_2025-12-22.csv

$ find . -type f -size +8k
./logs/report.log
```

- `find .` starts at `.`, the current folder, and looks in every folder below it.
- `-name "..."` matches the file name against a pattern. The quotes matter: they stop the shell expanding the `*` itself, so `find` gets the pattern.
- `-type f` keeps only files (`-type d`, only folders), and `-size +8k` keeps those bigger than 8 kilobytes.
- `find` lists files in whatever order it meets them, so `| sort` puts them in order, and `head -3` keeps the first three.

`find . -type f -mtime -1` lists files changed in the last day, which is how you answer "did the export actually run last night?". `find . -name "*.tmp" -delete` cleans up, and is worth running once without `-delete` first, to see what it would delete.

34.4 Searching and summarizing text

Six small tools cover most data questions. The practice files hold one row per order line, with the same columns you know from the `sales_lines` view: order id, date, customer, city, segment, product, category, quantity, net revenue, and sales rep.

grep: find lines

```
# terminal, in ~/ch34/practice
$ grep -c "Pune" ../sales_lines_2025.csv
42

$ grep "Food Container Set" exports/orders_2025-12-16.csv
10169,2025-12-16,18,Evergreen Mart,Indore,Retail,104,Food Container Set,Kitchen,30,18600.00,Rahul Mehta
10170,2025-12-16,21,Kitchen Kraft,Bengaluru,Retail,104,Food Container Set,Kitchen,35,21700.00,Rahul Mehta

$ grep -i "sharma" ../sales_lines_2025.csv | head -2
10007,2025-01-24,1,Sharma Hardware,Mumbai,Retail,101,Storage Box 10L,Storage,45,19350.00,Neha Kulkarni
10007,2025-01-24,1,Sharma Hardware,Mumbai,Retail,102,Storage Box 25L,Storage,20,15000.00,Neha Kulkarni

$ grep -rl "Metro Mart" exports | sort | head -3
exports/orders_2025-11-26.csv
exports/orders_2025-12-16.csv
```

`grep` prints every line that contains the text you give it. `../sales_lines_2025.csv` is the year's file, one folder up (`..`).

Option	Meaning
<code>-c</code>	count matching lines instead of printing them
<code>-i</code>	ignore case
<code>-v</code>	show lines that don't match
<code>-r</code>	search a whole folder tree
<code>-l</code>	print only the names of files that contain a match
<code>-n</code>	show line numbers
<code>-w</code>	match whole words only
<code>-q</code>	quiet: print nothing, only set the exit code
<code>-E</code>	use extended regular expressions (Chapter 14's patterns)

The last `grep` found "Metro Mart" in two files, so `head -3` had only two to show.

`grep` also answers through its exit code, like `test` in section 34.2: 0 when it found something, 1 when it didn't. With `-q` it prints nothing at all, which is what a script wants when it only needs the yes or no:

```
# terminal, in ~/ch34/practice
$ grep -q "Mumbai" reference/customers.csv; echo $?
0

$ grep -q "Kathmandu" reference/customers.csv; echo $?
1
```

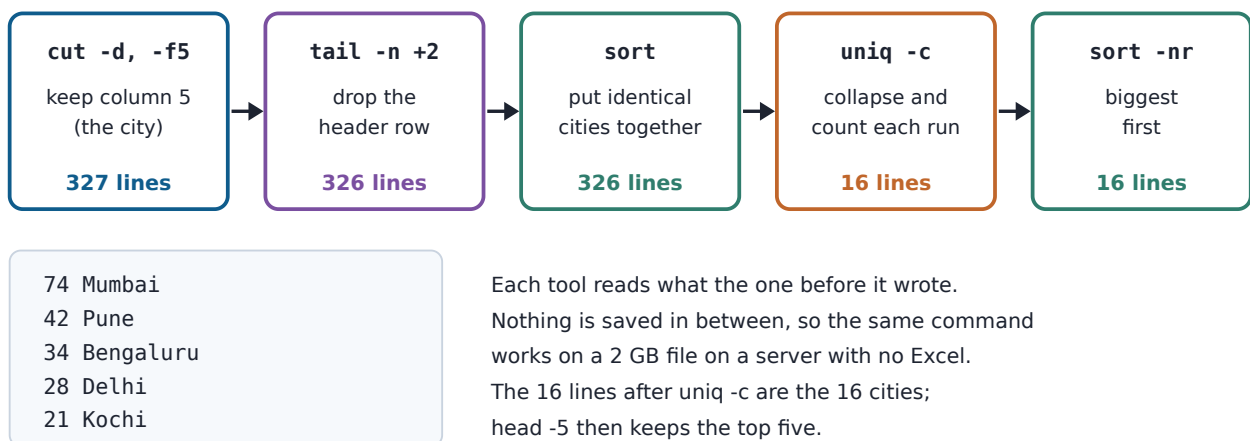
wc, sort, uniq, cut: count and group

```
# terminal, in ~/ch34/practice
$ cut -d, -f5 ../sales_lines_2025.csv | tail -n +2 | sort | uniq -c | sort -nr | head -5
 74 Mumbai
 42 Pune
 34 Bengaluru
 28 Delhi
 21 Kochi
```

How it works, one stage at a time:

1. **cut -d, -f5** takes field 5 of each line, using a comma as the delimiter: the city.
2. **tail -n +2** starts from line 2, dropping the header.
3. **sort** puts identical cities next to each other, because...
4. **uniq -c** collapses *adjacent* identical lines and counts them. Forgetting the **sort** before **uniq** is the classic mistake: it silently gives wrong counts.
5. **sort -nr** sorts numerically (**-n**), largest first (**-r**).
6. **head -5** keeps the top five.

One question, five small tools: order lines by city



Forgetting sort before uniq -c is the classic bug: uniq only collapses lines that are already next to each other.

Figure 34.2 – A pipeline is five small programs, each reading what the one before it wrote.

Mumbai leads with 74 order lines, and Pune's 42 matches the **grep -c "Pune"** count above. ✓ That is Chapter 13's "group by city and count" written with five tiny programs instead of one query, and it works on a file that no database has ever seen. How many cities are there? Swap the last three stages for a count of the distinct values:

```
# terminal, in ~/ch34/practice
$ cut -d, -f5 ../sales_lines_2025.csv | tail -n +2 | sort -u | wc -l
16
```

sort -u sorts and keeps one copy of each line (*unique*), so **wc -l** counts cities, not order lines.

Watch out: sort sorts text unless you tell it otherwise. Without `-n`, 10 comes before 9, because it compares character by character. And sort order depends on your language settings, which is why, if your counts come out in a different order from the book's, `LC_ALL=C sort` gives the plain byte order that everyone's machine agrees on.

awk: arithmetic on columns

`awk` reads a file line by line, splits each line into fields, and runs your rules on each one. It's a small language of its own; four steps cover nearly everything an analyst needs. Start by printing one column of one day:

```
# terminal, in ~/ch34/practice
$ awk -F, '{ print $11 }' exports/orders_2025-12-16.csv
net_revenue
15050.00
2300.00
10212.50
18600.00
6887.50
15050.00
21700.00
```

- `-F`, sets the field separator to a comma. Fields are then `$1`, `$2`, ..., and `$0` is the whole line. Field 11 is `net_revenue`.
- The program is inside single quotes, so the shell passes it to `awk` untouched. `{ print $11 }` is an **action**, and with nothing in front of it, it runs on every line, including the header, which is why `net_revenue` is printed first.

Now add the day up, skipping the header:

```
# terminal, in ~/ch34/practice
$ awk -F, 'NR > 1 { total += $11 } END { print total }' exports/orders_2025-12-16.csv
89800
```

- `NR > 1` is a **condition** in front of the action: `NR` is the line number (the *number of the record*), so the action runs only from line 2 on.
- `total += $11` adds field 11 to a running total. A variable in `awk` starts at 0 without being declared.
- `END { ... }` runs once, after the last line: here it prints the total. ₹89,800 is the day's revenue.

Now the whole month, 31 files at once, printed with two decimals:

```
# terminal, in ~/ch34/practice
$ awk -F, 'NR > 1 { total += $11 } END { printf "%.2f\n", total }' exports/orders_2025-12-*.csv
439823.50
```

- `exports/orders_2025-12-*.csv` is a wildcard, so `awk` reads all 31 December files one after another, as if they were one long file.
- `printf "%.2f\n", total` prints with a format, like Python's `f"{total:.2f}"` in Chapter 17: `%.2f` means a number with two decimals, and `\n` starts a new line (plain `print` adds one for you; `printf`

doesn't).

The total is right, but only by luck. Group the same month by category and the luck runs out:

```
# terminal, in ~/ch34/practice
$ awk -F, 'NR > 1 { revenue[$9] += $11 } \
  END { for (c in revenue) printf "%-12s %12.2f\n", c, revenue[c] }' \
  exports/orders_2025-12-*.csv | sort -k2 -nr
Storage          220363.50
Kitchen          142180.00
Industrial        77280.00
category          0.00
```

- `revenue[$9] += $11` uses an **associative array**, awk's version of a Python dictionary: one running total per category (field 9), built in one pass.
- `for (c in revenue)` loops over the array's keys, in no particular order, hence the `sort` at the end.
- `%-12s` prints text left-aligned in 12 characters, and `%12.2f` a number with two decimals, right-aligned in 12, so the columns line up.
- `sort -k2 -nr` sorts on the 2nd whitespace-separated field (`-k2`, the revenue), numerically, largest first.

The first three lines are right; the fourth is a bug worth seeing: an extra line, `category 0.00`. Each of the 30 files after the first starts with a header row, and `NR > 1` skipped only the first one, because **NR counts lines across all the files**. So each later header's column 9, the word `category`, became a key, and its revenue, the word `net_revenue`, counted as 0. That's also why the monthly total above was right: awk treats text as 0 in arithmetic, which is exactly the kind of silent nonsense to watch for. **FNR** restarts at 1 in each file, so `FNR > 1` skips all 31 headers:

```
# terminal, in ~/ch34/practice
$ awk -F, 'FNR > 1 { revenue[$9] += $11 } \
  END { for (c in revenue) printf "%-12s %12.2f\n", c, revenue[c] }' \
  exports/orders_2025-12-*.csv | sort -k2 -nr
Storage          220363.50
Kitchen          142180.00
Industrial        77280.00
```

Both numbers reconcile with what you already know. December's total, ₹4,39,823.50, is the December 2025 figure of Chapters 10, 11, and 17, and the three categories add up to it: ₹2,20,363.50 + ₹1,42,180.00 + ₹77,280.00 = ₹4,39,823.50. ✓

Watch out: these tools don't understand CSV, only text. `cut -d,` and `awk -F,` split on every comma, including one inside `"Sharma Hardware, Mumbai"`. The practice files have no commas inside fields, so this works, and much real data is the same. When fields may contain commas, quotes, or line breaks, use a real CSV tool: `csvkit` and `xsv/qsv` are command-line tools that parse CSV properly, and `pandas` (Chapter 18) is right there. Use the shell for a quick look and a quick count; use a parser for anything you'll publish.

Reading logs

This is where `grep` earns its keep. The practice folder has the log of Riverstone's daily summary job for November and December: every morning at 07:00 it summarizes the day before, the job that section 34.7's `daily_summary.sh` imitates. Each log line has a date, a time, a **level** (INFO for normal progress, WARNING for something worth a look, ERROR for a failure), the job's name, and a message.

```
# terminal, in ~/ch34/practice
$ wc -l logs/report.log
187 logs/report.log

$ awk '{ print $3 }' logs/report.log | sort | uniq -c | sort -nr
    149 INFO
     37 WARNING
      1 ERROR

$ grep ERROR logs/report.log
2025-11-09 07:00:04 ERROR daily_summary: database connection refused

$ grep -A1 "connection refused" logs/report.log
2025-11-09 07:00:04 ERROR daily_summary: database connection refused
2025-11-09 07:00:04 INFO daily_summary: exit code 1

$ grep WARNING logs/report.log | head -3
2025-11-01 07:00:04 WARNING daily_summary: no sales lines for 2025-11-01; wrote an empty summary
2025-11-02 07:00:04 WARNING daily_summary: no sales lines for 2025-11-02; wrote an empty summary
2025-11-03 07:00:04 WARNING daily_summary: no sales lines for 2025-11-03; wrote an empty summary
```

- `awk '{ print $3 }'` with no `-F` splits on spaces, so field 3 is the level, and the pipeline from before counts each one.
- One **ERROR** in 61 days, on 9 November: the database refused the connection and the job stopped with exit code 1. `-A1` shows one line **after** each match (`-B` before, `-C` both), which is how you see what happened next.
- The **WARNING** lines are mostly quiet days: the job ran, found no orders, and wrote an empty summary, which is correct and needs no action. A few are a slow database that answered on the second try.

Finding that in a log viewer takes minutes; here it takes one line, and the same command works on a server over SSH where there is no log viewer at all.

34.5 Permissions: who can read, write, and run

Every file has an **owner**, a **group**, and nine permission bits. `ls -l` shows them:

```
# terminal, in ~/ch34/practice
$ ls -l reference/customers.csv
-rw-r--r-- 1 meera meera 877 Jan  5 2026 reference/customers.csv
```

Read the first column in four pieces: `-` (a regular file; `d` would be a folder, `l` a link), then three sets of three, for **user** (the owner), **group**, and **others** (everyone else). Each set is `r` read, `w` write, `x` execute, or `-` for "not allowed". So `rw-r--r--` means the owner can read and write, and everyone else can only

read. The 1 after the permissions is the number of links to the file: ignore it. Then come the owner (meera), the group (a group of one, also called meera), the size in bytes, the date, and the name.

Reading ls -l

```
-rw-r--r-- 1 meera meera 877 Jan 5 2026 customers.csv
```

others (everyone else): r-- read only

group: r-- read only

user (the owner): rw- read and write

type: - a file, d a folder, l a link

Each set of three is read (r), write (w), execute (x), or - for "not allowed".
 On a folder, x means "can go into it".
 Then: links, owner, group, size, date, name.

Each set is also a digit: read 4 + write 2 + execute 1.

644	rw- r-- r--	owner writes, everyone reads	data files, documents
755	rwX r-x r-x	owner writes, everyone reads and runs	scripts, folders
600	rw- --- ---	owner only	SSH keys, .env files
700	rwX --- ---	owner only, and can go in	~/ .ssh

chmod +x script.sh adds the execute bit; chmod 600 key removes everyone else's access.

Figure 34.3 – Nine bits, three audiences, and four numbers worth memorizing.

Each set is also a digit, adding read 4, write 2, and execute 1:

Number	Means	Common use
644	owner read and write; everyone else read	data files, documents
755	owner everything; everyone else read and run	scripts, folders
600	owner read and write; nobody else anything	SSH private keys, .env files with passwords
700	owner everything; nobody else anything	private folders such as ~/ .ssh

On a folder, the bits mean something slightly different: **r** lists the names inside, **w** creates and deletes entries, and **x** lets you go into it or reach anything within it. A folder with no **x** is a locked cabinet: you can't get at the files even if their own permissions would allow it.

Watch a script go from unrunnable to runnable. First, write a two-line script into a file:

```
# terminal, in ~/ch34/practice
$ printf '#!/usr/bin/env bash\n echo "the export ran"\n' > run_export.sh

$ cat run_export.sh
#!/usr/bin/env bash
echo "the export ran"
```


printf is like **echo**, but it understands `\n` as "new line", so one command writes both lines. The first line, `#!/usr/bin/env bash`, tells the system to run the file with **bash** (section 34.7 explains it); the second is the script's only command. Now try to run it:

```
# terminal, in ~/ch34/practice
$ ls -l run_export.sh
-rw-rw-r-- 1 meera meera 42 Sep 29 02:34 run_export.sh

$ ./run_export.sh
bash: ./run_export.sh: Permission denied

$ chmod +x run_export.sh

$ ls -l run_export.sh
-rwxrwxr-x 1 meera meera 42 Sep 29 02:34 run_export.sh

$ ./run_export.sh
the export ran
```

`./run_export.sh` means "the file `run_export.sh` in this folder": the shell doesn't look in the current folder for programs unless you say so (section 34.6 explains why). The first attempt fails because the file has no `x`; after `chmod +x`, every `-` in an `x` position becomes `x`, and the script runs. Your group bits may show `r--` instead of `rw-`: new files get their starting permissions from a setting called the **umask**, which differs between systems. Your dates will be today's.

chmod (*change mode*) sets permissions, either by adding and removing (`chmod +x`, `chmod g-w`, `chmod o-rwx`) or by number (`chmod 755 run_export.sh`). **chown** changes the owner, and usually needs administrator rights.

sudo, and when to be careful

sudo runs one command as the **superuser** (**root**), the account that can do anything: `sudo apt install rsync`. Use it for installing software and editing system files, never out of habit. If a command fails with *Permission denied*, the first question is "should I be allowed to do this?", not "how do I force it?". On a shared server, the answer is often that the file belongs to a service account and your script should be writing somewhere else.

Watch out: SSH refuses keys that other people can read. If your private key is 644, **ssh** prints `WARNING: UNPROTECTED PRIVATE KEY FILE!` and refuses to use it. `chmod 600 ~/.ssh/id_ed25519` fixes it. The same logic applies to a `.env` file holding a database password (Chapter 26, section 26.4): `chmod 600`, and keep it out of Git.

34.6 Environment variables and PATH

An **environment variable** is a named value the shell hands to every program it starts. Chapter 26 used them to keep a database password out of the code (section 26.4); here's what's actually happening.


```
# terminal, in ~/ch34/practice
$ export RIVERSTONE_OUTPUT_DIR=/tmp/reports

$ echo "$RIVERSTONE_OUTPUT_DIR"
/tmp/reports

$ python3 -c 'import os; print(os.environ["RIVERSTONE_OUTPUT_DIR"])'
/tmp/reports

$ env | grep RIVERSTONE
RIVERSTONE_OUTPUT_DIR=/tmp/reports
```

`python3 -c '...'` runs a one-line Python program given in quotes, a quick way to show that a program started from this shell sees the variable.

- **export NAME=value** sets a variable and passes it to programs the shell starts. Without **export**, it stays in the shell itself.
- No spaces around **=**. **NAME = value** runs a command called **NAME**.
- **\$NAME**, or better **"\$NAME"** with quotes, reads it. The quotes matter: an unquoted value containing spaces is split into several arguments.
- **env** lists everything currently set; **unset NAME** removes one.
- Variables set this way last until you close the terminal. To make them permanent, put the **export** lines in `~/.bashrc` (bash) or `~/.zshrc` (zsh), which the shell reads when it starts. On a server, the scheduler or service manager supplies them instead.

PATH

PATH is the list of folders the shell searches, in order, when you type a command name:

```
# terminal, in ~/ch34/practice
$ echo "$PATH" | tr ':' '\n' | head -4
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin

$ which python3 grep
/usr/local/bin/python3
/usr/bin/grep

$ type cd
cd is a shell builtin
```

PATH is one long line with the folders separated by colons; `tr ':' '\n'` translates every colon into a new line, so each folder gets a line of its own. Yours will list different folders.

`which` shows which file would run. This is the answer to the two most common "but it works in my terminal" mysteries: a virtual environment puts its own folder at the front of **PATH** (that's the heart of what `activate` does; it also changes your prompt), and a scheduled job runs with a much shorter **PATH** than your interactive shell, which is why `cron` jobs should call `/usr/bin/python3` or a full path to the environment rather than trusting `python3` to be found.

type explains what a name is: a file, a **built-in** command of the shell itself (like `cd`), an **alias**, or a function.

Watch out: never put secrets in your shell history. Typing `export DB_PASSWORD=hunter2` writes the password into `~/.bash_history`, which is readable by anyone with your account and is often backed up. Keep secrets in a `.env` file with `600` permissions, or in a **secrets manager** (a service that stores passwords and hands them only to the programs allowed to have them), and load them with `set -a; . ./env; set +a` (section 26.0: `set -a` exports everything the file defines) or a library such as `python-dotenv` (Chapter 18).

34.7 Shell scripts

A **shell script** is a file of commands. Anything you can type, you can save and run again, which is how a sequence of clever one-liners becomes something a colleague or a scheduler can use.

The companion folder has `daily_summary.sh`, one folder above `practice/`. Read it before running it; this is the whole file:

```
#!/usr/bin/env bash
# Analyst to Architect · Chapter 34 · summarize one day's export.
# Usage: ./daily_summary.sh 2025-12-16      Exit codes: 0 done, 2 no export for that day.
set -euo pipefail

DAY="${1:?usage: $0 YYYY-MM-DD}"
FILE="exports/orders_${DAY}.csv"

if [[ ! -f "$FILE" ]]; then
    echo "no export for $DAY" >&2
    exit 2
fi

ORDER_LINES=$(( $(wc -l < "$FILE") - 1 ))
REVENUE=$(awk -F, 'FNR > 1 { total += $11 } END { printf "%.2f", total + 0 }' "$FILE")
CUSTOMERS=$(awk -F, 'FNR > 1 { seen[$4] = 1 } END { print length(seen) }' "$FILE")

echo "$DAY: order lines $ORDER_LINES, customers $CUSTOMERS, revenue $REVENUE"
```

```
# terminal, in ~/ch34/practice
$ cp ../daily_summary.sh .

$ chmod +x daily_summary.sh

$ ./daily_summary.sh 2025-12-16
2025-12-16: order lines 7, customers 3, revenue 89800.00

$ ./daily_summary.sh 2025-12-25
2025-12-25: order lines 0, customers 0, revenue 0.00

$ ./daily_summary.sh 2026-01-05; echo "exit code $?"
no export for 2026-01-05
exit code 2

$ ./daily_summary.sh; echo "exit code $?"
./daily_summary.sh: line 6: 1: usage: ./daily_summary.sh YYYY-MM-DD
exit code 1
```

How it works:

- **`#!/usr/bin/env bash`**, the **shebang**, is the first line of every script: it tells the system which program runs the file. `env bash` finds bash wherever it's installed. It must be the very first line, before any comment.
- **Lines starting with `#`** after it are comments, as in Python.
- **`set -euo pipefail`** belongs at the top of every script you write:
 - `-e` stop at the first command that fails, instead of carrying on with bad data,
 - `-u` treat an unset variable as an error, so a typo doesn't silently become an empty string (see the Watch out below),
 - `-o pipefail` make a pipeline fail if **any** stage fails, not just the last one.
- **`"${1:?message}"`** takes the first argument, or exits with that message if it wasn't given. `$1`, `$2`, ... are the arguments, `$0` is the script's name, and `"$@"` is all of them.
- **`$( ... )` is **command substitution****: run the command and use its output as text. `$(( ... ))` is arithmetic.
- **`[[ ! -f "$FILE" ]]` is a test**: *is this not a regular file?* Others: `-d` folder, `-s` non-empty file, `-z` empty string, `=` and `!=` for strings, `-eq` and `-lt` for numbers.
- **`>&2`** sends the message to standard error, so it doesn't pollute the script's real output when someone pipes it.
- **`exit 2`** returns a code that says *which* problem occurred: 0 success, 1 general failure, and your own numbers for the cases a caller might want to treat differently.
- **Always quote your variables**: `"$FILE"`, not `$FILE`. Without quotes, a path containing a space becomes two arguments, which is the single most common bug in shell scripts.
- **Name your variables so they can't collide with the system's**. ALL-CAPS names such as `PATH`, `HOME`, `USER`, and `LINES` (bash's own record of the terminal's height) already mean something, which is why this script says `ORDER_LINES`, not `LINES`. A prefix or lowercase names keep you safe.
- The last line prints `customers 1` rather than `1 customers`: a label before the number never needs a plural.

Watch out: `rm -rf` with a variable. Never run `rm -rf` with a variable that might be empty. If `$DIR` is unset, `rm -rf $DIR/` becomes `rm -rf /`, a command that has destroyed more than one production system. `set -u` turns the unset variable into an error before `rm` runs; `"${DIR:?}"` (as in DAY above) does the same for one variable. Before any `rm` with a pattern, run `ls` with the same pattern and read what it lists.

Loops

```
# terminal, in ~/ch34/practice
$ for day in 2025-12-15 2025-12-16 2025-12-17; do ./daily_summary.sh "$day"; done
2025-12-15: order lines 0, customers 0, revenue 0.00
2025-12-16: order lines 7, customers 3, revenue 89800.00
2025-12-17: order lines 2, customers 1, revenue 37628.00
```

`for day in ...; do ...; done` runs the commands between `do` and `done` once for each word in the list, with `$day` set to that word. Loop over a wildcard instead of a list, and you get one run per file:

```
# terminal, in ~/ch34/practice
$ for file in exports/orders_2025-12-2*.csv; do \
    lines=$(( $(wc -l < "$file") - 1 )); \
    [[ $lines -eq 0 ]] && echo "$file is empty"; \
done
exports/orders_2025-12-20.csv is empty
exports/orders_2025-12-22.csv is empty
exports/orders_2025-12-24.csv is empty
exports/orders_2025-12-25.csv is empty
exports/orders_2025-12-26.csv is empty
exports/orders_2025-12-27.csv is empty
exports/orders_2025-12-28.csv is empty
exports/orders_2025-12-29.csv is empty
```

A for loop over a wildcard is the workhorse of file processing: one command per file, in a predictable order. Inside it, `[[ $lines -eq 0 ]] && echo ...` prints only when the test passes, the `&&` of section 34.2.

When the list is in a file, one item per line, `while read` reads it a line at a time:

```
# terminal, in ~/ch34/practice
$ printf '2025-12-16\n2025-12-17\n2025-12-18\n' > /tmp/days.txt

$ while read -r day; do ./daily_summary.sh "$day"; done < /tmp/days.txt
2025-12-16: order lines 7, customers 3, revenue 89800.00
2025-12-17: order lines 2, customers 1, revenue 37628.00
2025-12-18: order lines 2, customers 1, revenue 23850.00
```

`read -r day` reads one line into the variable `day` (`-r` keeps any backslashes as they are), and `< /tmp/days.txt` feeds the file to the loop's standard input. The loop stops when there are no lines left.

To keep a record of what a script said while still watching it, send its output through **tee -a** (section 34.2):

```
# terminal, in ~/ch34/practice
$ ./daily_summary.sh 2025-12-16 | tee -a run.log
2025-12-16: order lines 7, customers 3, revenue 89800.00

$ cat run.log
2025-12-16: order lines 7, customers 3, revenue 89800.00
```

Three more tools for the project

The project at the end of this chapter needs three commands you haven't met. A **checksum** is a long number calculated from every byte of a file: change one byte, and it changes completely. **sha256sum** prints one:

```
# terminal, in ~/ch34/practice
$ sha256sum exports/orders_2025-12-16.csv
861504c535d42b08c99d34cb273d3d191758487454e3d7b6bc5687afee18e65d  exports/orders_2025-12-16.csv
```

A sender publishes the checksum next to the file; the receiver calculates it again and compares, which proves the download arrived intact. **gzip** compresses a file, and **zcat** reads a compressed one without unpacking it:

```
# terminal, in ~/ch34/practice
$ gzip -k ../sales_lines_2025.csv

$ ls -l ../sales_lines_2025.csv ../sales_lines_2025.csv.gz
-rw-r--r-- 1 meera meera 34350 Sep 29 02:34 ../sales_lines_2025.csv
-rw-r--r-- 1 meera meera  3992 Sep 29 02:34 ../sales_lines_2025.csv.gz

$ zcat ../sales_lines_2025.csv.gz | head -2
order_id,order_date,customer_id,customer_name,city,segment,product_id,product_name,category,quantity,net_revenue ↵
,sales_rep
10001,2025-01-02,2,Patel Kitchenware,Ahmedabad,Retail,108,Stackable Bin,Storage,10,2900.00,Neha Kulkarni

$ rm ../sales_lines_2025.csv.gz
```

gzip -k keeps the original (**-k**) and writes **sales_lines_2025.csv.gz** beside it; without **-k**, the original is replaced by the compressed file. And **date** prints the date and time in any format you ask for: **date +%F** prints today's date as YYYY-MM-DD (%F), and **date '+%F %T'** adds the time (%T), which is how the project stamps its log lines.

Watch out: shell scripts stop being the right tool sooner than you think. They're ideal for gluing commands together: fetch a file, check it, move it, call a program. Once you need data structures, CSV parsing, arithmetic with decimals, error handling with retries, or tests, write Python (Chapters 17 and 18; Chapter 29 turns a script into a proper tool). A 300-line bash script that nobody dares to change is a common and avoidable mess.

34.8 SSH: working on another machine

SSH (secure shell) gives you a terminal on a remote machine over an encrypted connection. It's how you reach the report server, a cloud database host, or a container.

To practise, you need a machine that accepts SSH connections, an **SSH server**. A real one belongs to your company's platform team; for learning, your own computer can play the part. None of the three systems runs an SSH server until you turn one on.

Set up a practice server (10 minutes)

- **Ubuntu, and WSL on Windows:** install and start the server, `sudo apt install openssh-server` then `sudo service ssh start`.
- **macOS:** System Settings → General → Sharing, and turn on **Remote Login**.

Then connect to your own machine, by the name `localhost` (section 34.9), and come straight back:

```
# terminal, anywhere
$ ssh localhost
$ exit
```

The first time, `ssh` shows the server's key fingerprint and asks *Are you sure you want to continue connecting (yes/no/[fingerprint])?*. Type `yes`. The fingerprint is the server's identity card: `ssh` remembers it in `~/.ssh/known_hosts`, and if it ever changes it warns you loudly, because a different fingerprint can mean a different machine pretending to be yours. Then it asks for your login password, gives you a prompt on "the server", and `exit` brings you home.

Keys, not passwords

An **SSH key pair** is two matching files: a **private key** (`~/.ssh/id_ed25519`) that never leaves your machine, and a **public key** (`~/.ssh/id_ed25519.pub`) that you give to the server. The server uses the public key to check that you hold the private one. Nothing secret crosses the network, and there's no password to guess or reuse.

```
# terminal, on your own machine (run once)
$ ssh-keygen -t ed25519 -C "meera@laptop"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/meera/.ssh/id_ed25519):
Created directory '/home/meera/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/meera/.ssh/id_ed25519
```

- `-t ed25519` chooses the key type, a modern and short one; `-C "meera@laptop"` is a comment stored with the public key, so you can tell your keys apart later.
- Press Enter at the first question to keep the usual file name. The last lines, a fingerprint and a small picture made of symbols, follow; you can ignore them.
- Choose a **passphrase**: it encrypts the private key, so a stolen laptop isn't a stolen server. As with `sudo`, nothing appears while you type it. `ssh-agent` remembers it for the session so you type it once a day.

Now give the server a short name. Create the file `~/.ssh/config` in your editor (`nano ~/.ssh/config`, or `code ~/.ssh/config`) with these four lines, using your own user name, and make it private:

```
Host reportserver
  HostName 127.0.0.1
  User meera
  IdentityFile ~/.ssh/id_ed25519
```

```
# terminal, on your own machine
$ chmod 600 ~/.ssh/config

$ ssh-copy-id reportserver
```

- `~/.ssh/config` gives a short name to a host, its address, the user, and which key to use, so `ssh reportserver` replaces `ssh -i ~/.ssh/id_ed25519 meera@127.0.0.1`. `127.0.0.1` is this machine (section 34.9); on a real job the platform team gives you the server's address, for example `10.4.2.17`, and your user name there.
- `ssh-copy-id reportserver` copies your public key to the server, asking for your login password this one time. From then on, the key does the work.

Connecting and running commands

```
# terminal, in ~/ch34/practice
$ ssh reportserver "hostname; ls /home/meera/ch34/practice/exports | wc -l"
vm
61
```

- `ssh host "command"` runs one command on the server and returns, which is exactly what a scheduled job or a deployment script needs. Here the server's name is `vm` (yours prints your computer's name), and it sees the same 61 files, because "the server" is your own machine. Without a command, you get an interactive shell; `exit` (or `Ctrl+D`) comes home.

Copying files

```
# terminal, in ~/ch34/practice
$ scp exports/orders_2025-12-16.csv reportserver:/tmp/

$ ssh reportserver "wc -l /tmp/orders_2025-12-16.csv"
8 /tmp/orders_2025-12-16.csv
```

`scp` copies one file or folder, like `cp` with a machine name and a colon in front of the path. The other direction works the same way: `scp reportserver:/var/log/report.log .` brings a file back.

For a whole folder, **rsync** is better: it copies only what differs, which matters when the folder is large or the connection is slow. It works between two folders on one machine exactly as it does between machines, so try it locally first. The trailing slash matters: `exports/` means "the contents of `exports`", while `exports` without it means "the folder itself", which would land as `exports_copy/exports`. And always do a **dry run** first:

```
# terminal, in ~/ch34/practice
$ rsync -anv --delete exports/ /tmp/exports_copy/ | head -4
sending incremental file list
created directory /tmp/exports_copy
./
orders_2025-11-01.csv

$ rsync -a --delete exports/ /tmp/exports_copy/

$ ls /tmp/exports_copy | wc -l
61
```

- **-a** (*archive*) copies folders inside folders and keeps permissions and timestamps.
- **-n** is the dry run: list what *would* happen, and do nothing. **-v** (*verbose*) prints each file, which is what makes the dry run useful. `head -4` shows only the start of the list of 61 files.
- **--delete** makes the destination match the source exactly, deleting anything in it that the source doesn't have. That's why the dry run comes first, every time: with the wrong folder or a missing slash, it deletes the wrong things.

To copy to the server, only the destination changes: `rsync -anv --delete exports/ reportserver:/tmp/exports_copy/`, then the same without `-n` and `-v`.

Watch out: tunnels and ports. `ssh -L 5433:localhost:5432 reportserver` forwards your local port 5433 to the server's PostgreSQL. DBEaver on your laptop then connects to `localhost:5433` and reaches a database that isn't exposed to the internet at all. It's a common and legitimate pattern, and it's also something many companies audit, so use it with your platform team's blessing.

34.9 Networking in plain English

Four ideas cover most of what a data person needs.

IP addresses: which machine

An **IP address** identifies a machine on a network, like `142.250.183.4` (IPv4) or a longer IPv6 address. Some ranges are **private**, used inside company networks and never routed on the public internet: `10.x.x.x`, `172.16–31.x.x`, and `192.168.x.x`. And `127.0.0.1`, called **localhost**, always means *this machine*: this chapter's practice server listens there, which is why nothing it does leaves your computer.

DNS: names into addresses

Typing an address is unworkable, so the **Domain Name System** translates names into addresses. Your machine asks a DNS server "what is the address of `api.example.com`?" and caches the answer for a while.

```
# terminal, in ~/ch34/practice
$ getent hosts localhost
127.0.0.1      localhost
```

`getent hosts` asks the system to look a name up the way every program does, and prints the address it gets. (macOS has no `getent`: `dscacheutil -q host -a name localhost` does the same.)

`dig api.example.com` or `nslookup api.example.com` does a full DNS lookup and prints the result; macOS has both, and on Ubuntu you install them first with `sudo apt install bind9-dnsutils`. DNS is the reason a database connection can fail with "could not translate host name": the name, not the database, is the problem. It's also how a company points `warehouse.internal` at a new server without changing a single connection string.

Ports: which program on that machine

One machine runs many services, so each listens on a numbered **port**. Some numbers are conventions: 22 SSH, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 3306 MySQL, 8080 and 8000 for test servers.

```
# terminal, in ~/ch34/practice
$ python3 ../daily_file_server.py > /tmp/server.log 2>&1 &

$ sleep 1; cat /tmp/server.log

$ ss -ltn | grep 8034
LISTEN 0      5            127.0.0.1:8034      0.0.0.0:*
```

The first command starts the companion's small web server, `daily_file_server.py`, which serves the practice exports the way a partner's file server would. `&` at the end runs it in the **background** and gives you the prompt back; its messages go to `/tmp/server.log`, and `sleep 1` waits a second for it to start. `jobs` lists background jobs, `fg` brings one forward, and `kill %1` stops the first one.

`ss -ltn` lists listening TCP sockets (`-l` listening, `-t` TCP, `-n` numbers, not names). On macOS, which has no `ss`, `lsof -iTCP -sTCP:LISTEN -n -P` answers the same question. (Older Linux systems have `netstat -ltn`.) An address of `127.0.0.1:8034` means only this machine can connect; `0.0.0.0:8034` would mean anyone who can reach the machine. When something "can't connect", this command answers the first question: is anything listening at all?

HTTP: asking for something

HTTP is the language of web requests: a **method** (GET to read, POST to send), a path, headers, and a body. The answer has a **status code**, a three-digit number: 200 means OK, 404 means not found, 401 means you must say who you are, 500 and above mean the server failed. It usually comes with a body. `curl` speaks HTTP from the command line. The practice server from the ports example is still running in the background, so ask it for things.

```
# terminal, in ~/ch34/practice
$ curl --silent http://127.0.0.1:8034/health
ok

$ curl --silent --output /dev/null --write-out "%{http_code}\n" \
  http://127.0.0.1:8034/exports/orders_2025-12-16.csv
200

$ curl --silent --output /dev/null --write-out "%{http_code}\n" \
  http://127.0.0.1:8034/exports/orders_2026-01-05.csv
404

$ curl --silent --head http://127.0.0.1:8034/exports/orders_2025-12-16.csv \
  | grep -i "^content-"
Content-Type: text/csv
Content-Length: 826

$ curl --silent http://127.0.0.1:8034/exports/orders_2025-12-16.csv | wc -l
8

$ kill %1
```

What happens when curl asks for a file

```
curl --silent http://127.0.0.1:8034/exports/orders_2025-12-16.csv
```

http	the protocol: HTTP, or https for an encrypted connection
127.0.0.1	which machine: localhost, this one
8034	which program on it: the port it listens on
/exports/orders_...	what to ask for: the path

1. Name → address

A name like api.example.com is looked up in DNS. 127.0.0.1 is already an address.

2. Connect to the port

The program listening on port 8034 accepts. ss -ltn shows what is listening.

3. Send the request

GET /exports/orders_2025-12-16.csv, plus headers such as an API key.

4. Read the answer

A status code (200 OK, 404 not found, 500 and up: the server failed), headers, the body.

Figure 34.4 – What a URL contains, and what happens after you press Enter.

- `--silent` hides the progress meter; `--output file` saves the body; `--write-out "%{http_code}"` prints just the status.
- `/dev/null` is the system's bin: output sent there disappears. `--output /dev/null` throws the body away, because only the status code is wanted.
- `--head` asks for the headers only, which tells you the type and size before downloading 2 GB. `grep -i "^content-"` keeps the lines that start with `content-` in any case: `^` means "at the start of the line", as in Chapter 14's regular expressions. (The other headers, including the date, are filtered out so the output is the same every time.)

- `--fail` makes curl return a non-zero exit code on a 4xx or 5xx status, which is what a script needs. `--max-time 30` sets a timeout. Both are in this chapter's project.
- For an API that needs a key: `curl -H "X-API-Key: demo-key" ...`, and `-d '{"json": "body"}' -H "Content-Type: application/json"` for a POST.

The 404 means the file for 5 January 2026 hasn't been published, and no amount of retrying will change that today; a 500-range code, by contrast, is worth retrying later. Chapter 29 writes a Python client that makes exactly this distinction for an API.

APPLY

Common mistakes

Mistake	Symptom	Fix
Running a command in the wrong folder	<i>No such file or directory</i> for a file you can see in Finder or Explorer	<code>pwd</code> first; use Tab completion so wrong paths can't be typed (section 26.0)
Forgetting that Linux is case-sensitive	A script that worked on your laptop fails on the server	Match the case exactly; keep file names lowercase
<code>rm</code> with a wildcard, or <code>rm -rf \$DIR/</code>	Files gone, with no recycle bin	Run <code>ls</code> with the same pattern first; <code>set -u</code> in scripts so an empty variable is an error
Unquoted variables in scripts	A path with a space becomes two arguments and half a file name disappears	Always "\$VAR"
<code>uniq</code> without <code>sort</code>	Counts too low, silently	<code>sort uniq -c</code>
<code>sort</code> on numbers without <code>-n</code>	9 comes after 10	<code>sort -n</code> , or <code>sort -k2 -nr</code> for a numeric column
<code>cut -d,</code> or <code>awk -F,</code> on real CSV	Wrong columns when a field contains a comma or a quote	Check the data; use <code>csvkit</code> , <code>qsv</code> , or <code>pandas</code> for anything published
<code>NR > 1</code> with several files in <code>awk</code>	Every file's header counted after the first	<code>FNR > 1</code>
Writing output over the input file (<code>sort f.csv > f.csv</code>)	An empty file	Write to a new name, then <code>mv</code>
Scheduled job "runs" but does nothing	Works by hand, not in cron	The job's <code>PATH</code> and environment are minimal: use full paths, set variables in the script
Missing shebang or execute bit	<i>Permission denied</i> , or the file opens as text	<code>#!/usr/bin/env bash</code> on line 1, and <code>chmod +x</code>
<code>rsync --delete</code> without a dry run, or with a missing <code>/</code>	Files deleted on the destination, or copied one folder too deep	<code>rsync -avn --delete</code> first; <code>exports/</code> means the folder's contents
Working under <code>/mnt/c/...</code> in WSL	Permissions that don't change, and slow commands	Work in your Linux home folder (<code>cd ~</code>)
No <code>set -euo pipefail</code>	The script carries on after a failure and writes a wrong file	Put it at the top of every script

Mistake	Symptom	Fix
Private key readable by others	<i>WARNING: UNPROTECTED PRIVATE KEY FILE!</i> and SSH refuses	<code>chmod 600 ~/.ssh/id_ed25519</code>
<code>curl</code> without <code>--fail</code> in a script	A 404 or 500 page saved as if it were data	<code>--fail --show-error --max-time N</code> , then check the exit code
Secrets typed on the command line	Passwords in <code>~/.bash_history</code> and in process lists	Use a <code>.env</code> file with <code>600</code> , or a secrets manager
Growing a shell script past a page or two	Nobody dares change it; no tests	Move to Python as soon as there's real logic (Chapter 29 shows how to make it a proper tool)
Assuming the shell parsed your pattern	<code>find . -name *.csv</code> fails oddly	Quote patterns for <code>find</code> : <code>find . -name "*.csv"</code>

In the real world: the disk that filled up at 2 a.m.

At 07:40 on a Tuesday, Meera's phone shows an alert from the report server, the small Linux machine that runs Riverstone's scheduled jobs: the monthly report job exited with code 1. The log line is one she hasn't seen before: *ERROR riverstone_report: could not write reports/riverstone_monthly_2026-01.xlsx: No space left on device.*

The report server has no desktop. Everything below happens over SSH, and takes four minutes.

```
# terminal, on Meera's laptop
$ ssh reportserver
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1        40G   40G    0 100% /

$ du -h --max-depth=1 /var/riverstone | sort -h | tail -4
1.2G    /var/riverstone/exports
2.1G    /var/riverstone/reports
34G     /var/riverstone/incoming
38G     /var/riverstone

$ ls -lh /var/riverstone/incoming | tail -3
-rw-r--r-- 1 reportjob reportjob 412M Feb  1 02:00 orders_2026-01-30.csv
-rw-r--r-- 1 reportjob reportjob 412M Feb  2 02:00 orders_2026-01-31.csv
-rw-r--r-- 1 reportjob reportjob 412M Feb  3 02:00 orders_2026-02-01.csv

$ ls /var/riverstone/incoming | wc -l
84
```

Command	What it answers
<code>df -h /</code>	how full is the disk? (<i>disk free</i> ; <code>-h</code> for human-readable sizes, <code>/</code> for the disk the system runs from)
<code>du -h --max-depth=1 /var/riverstone</code>	how big is each folder directly inside this one? (<i>disk usage</i>)
<code>sort -h</code>	sort human-readable sizes correctly, so <code>34G</code> comes after <code>2.1G</code>
<code>ls -lh ... tail -3</code>	the newest files and their sizes (the names sort by date)

Eighty-four daily files of 412 MB each, one for every day since the download job was switched on, on 10 November. These aren't the small practice exports: each is the ERP's full daily extract, every order line with its history plus audit columns. The download step ran every night; nothing ever moved the files out or deleted them. The archive folder the original script was supposed to use is empty, because that line was never written.

She clears just enough to let the report run, checking before deleting:

```
# terminal, on the report server
$ find /var/riverstone/incoming -name "orders_2025-*.csv" | wc -l
52
$ find /var/riverstone/incoming -name "orders_2025-*.csv" -print0 \
  | xargs -0 du -ch | tail -1
21G      total
$ find /var/riverstone/incoming -name "orders_2025-*.csv" -delete
$ df -h / | tail -1
/dev/vda1      40G   20G   21G  48% /
```

- The first `find` counts the 2025 files before touching anything: 52, from 10 November to 31 December.
- `-print0 | xargs -0 du -ch` hands the file names to `du` safely, even if a name contained a space: `-print0` ends each name with an invisible zero byte instead of a new line, and `xargs -0` reads them that way and runs `du` on them. `du -ch` adds a grand **total** line (`-c`), which `tail -1` keeps.
- Only then does `-delete` run, with exactly the same pattern. `df` confirms the disk is at 48%.

The report runs by hand and goes out at 08:20. Then she fixes the cause, in the project below: a script that downloads the day's file, checks it, compresses it into the archive, and deletes the raw copy, with a monthly clean-up of archives older than a year. She adds one more line to the job's crontab: a weekly check that emails her when the disk is more than 80% full.

What she tells the data engineer: "The job only ever added files. Downloads now get checked, compressed into the archive, and removed from `incoming`; compressed CSV is many times smaller (usually 5 to 10 times with `gzip`, and our practice sales file shrinks about 8 times). There's a disk check on Mondays. And we should give `incoming` its own volume, so a runaway download can't take the whole server down with it."

Three things made this fixable in minutes rather than a day: the failure was loud and had a reason (an exit code and an ERROR line with the cause), the machine was reachable by SSH with a key, and `df`, `du`, `find`, and `ls` answer "what is filling the disk?" faster than any console.

Project: a daily file that looks after itself

Goal: a script that fetches one day's export, proves it arrived intact, archives it, logs what it did, and tells its caller what happened.

Tools you'll need

These are the versions used for this chapter, checked in September 2026.

- **bash 5.2** on **Ubuntu 24.04**. **macOS**: Terminal runs `zsh`, so type `bash` first (macOS includes `bash 3.2`, which runs every script here). A few Linux tools differ on a Mac: use `lsof -iTCP -sTCP:LISTEN -n -P` for `ss -ltn`, `dscacheutil -q host -a name localhost` for `getent hosts`, and `date -v-1d` for `date -d yesterday` (or `brew install coreutils` and use `gdate`).
- **Windows**: **WSL 2** with Ubuntu, installed with `wsl --install` from an administrator PowerShell (section 34.1). Git Bash is enough for the text tools, not for permissions, `ss`, or the SSH server; PowerShell is a different language.
- **curl 8.5**, **OpenSSH 9.6**, **rsync 3.2**, **iproute2** (for `ss`), all standard on Ubuntu.
- **Python 3** (any version Chapter 17's rule allows) for the practice data generator and the local file server; neither needs a virtual environment.
- **Companion files** in `ch34/`: `make_ch34_data.py` (builds `practice/` from `sales_lines_2025.csv`), `daily_file_server.py` (the local file server), `daily_summary.sh` and `fetch_daily.sh` (the scripts), and `weekly_summary.sh` (exercise 10's answer).
- **Worth adding later**: `jq` for JSON on the command line, `ripgrep (rg)` as a faster `grep`, `fzf` for searching history, `tmux` for sessions that survive a dropped connection, and `qsv` or `csvkit` for proper CSV parsing.

Option A: your own data. A file your team downloads, exports, or receives every day. Work in a folder you own, and keep credentials in environment variables, not in the script.

Option B: Riverstone. Use the companion `daily_file_server.py` as the partner's server, and write the script yourself before comparing with `fetch_daily.sh`.

What it must do:

1. Take the day as an argument, and reject anything that isn't `YYYY-MM-DD` with exit code 1.
2. Download `exports/orders_<day>.csv`, with a timeout, failing on a 404 rather than saving the error page. Exit 2 when the file isn't published yet.
3. Download the matching `.sha256` file and verify the download with `sha256sum --check`. Exit 3 on a mismatch, leaving the file for inspection.
4. Report what arrived: the number of order lines and the day's revenue (`wc -l` and `awk`).
5. Compress the file into `archive/` and remove the raw copy from `incoming/`.
6. Log every step, with a timestamp, to both the screen and `logs/fetch_daily.log`.
7. Exit 0 only when all of that worked.

Start the server in one terminal, then run the script in another:

```
# terminal, in ~/ch34/practice
$ cp ../fetch_daily.sh . && chmod +x fetch_daily.sh

$ python3 ../daily_file_server.py > /tmp/server.log 2>&1 &

$ sleep 1; ./fetch_daily.sh 2025-12-16 | cut -d' ' -f3-
INFO downloading orders_2025-12-16.csv from http://127.0.0.1:8034
INFO orders_2025-12-16.csv passed its checksum: 7 order lines, revenue 89800.00
INFO archived archive/orders_2025-12-16.csv.gz

$ ./fetch_daily.sh 2026-01-05 | cut -d' ' -f3-; echo "exit code ${PIPESTATUS[0]}"
curl: (22) The requested URL returned error: 404
```

```

INFO downloading orders_2026-01-05.csv from http://127.0.0.1:8034
ERROR could not download orders_2026-01-05.csv (not published yet, or the server is unreachable)
exit code 2

$ ls archive
orders_2025-12-16.csv.gz

$ ls incoming

$ zcat archive/orders_2025-12-16.csv.gz \
  | awk -F, 'FNR > 1 { total += $11 } END { printf "%.2f\n", total }'
89800.00

$ kill %1

```

(`cut -d' ' -f3-` drops the date and time from each log line so the output is the same every time. `${PIPESTATUS[0]}` is the exit code of the first command in the pipeline, not of `cut`; it's a bash feature, one more reason this chapter runs bash on macOS. The `curl:` line comes first because `curl` writes it to standard error, which doesn't go through `cut`, so it appears on the screen as soon as `curl` writes it.)

The compressed archive holds the same ₹89,800.00 the summary reported, so nothing was lost on the way in. ✓

The reference script

Here is `fetch_daily.sh`, the companion's answer. Compare it with yours once yours runs.

```

#!/usr/bin/env bash
# Analyst to Architect · Chapter 34 · the project: download one day's export, check it, archive it.
# Usage: ./fetch_daily.sh 2025-12-16 [base_url]
# Exit codes: 0 done, 1 bad arguments, 2 file not published yet, 3 checksum mismatch.
# Tested on: bash 5.2.21, curl 8.5, Ubuntu 24.04.
set -euo pipefail

DAY="${1:-}"
BASE_URL="${2:-http://127.0.0.1:8034}"
INCOMING="incoming"
ARCHIVE="archive"
LOG="logs/fetch_daily.log"

log() { printf '%s %s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$1" "$2" | tee -a "$LOG"; }

if [[ ! "$DAY" =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2}$ ]]; then
    echo "usage: $0 YYYY-MM-DD [base_url]" >&2
    exit 1
fi

mkdir -p "$INCOMING" "$ARCHIVE" "$(dirname "$LOG")"
NAME="orders_${DAY}.csv"
log INFO "downloading $NAME from $BASE_URL"

if ! curl --silent --show-error --fail --max-time 30 \
    --output "$INCOMING/$NAME" "$BASE_URL/exports/$NAME"; then
    log ERROR "could not download $NAME (not published yet, or the server is unreachable)"
    rm -f "$INCOMING/$NAME"
    exit 2
fi

curl --silent --show-error --fail --max-time 30 \
    --output "$INCOMING/$NAME.sha256" "$BASE_URL/exports/$NAME.sha256"

```



```

if ! (cd "$INCOMING" && sha256sum --check --status "$NAME.sha256"); then
    log ERROR "checksum mismatch for $NAME; leaving it in $INCOMING for inspection"
    exit 3
fi

ROWS=$(( $(wc -l < "$INCOMING/$NAME") - 1 ))
REVENUE=$(awk -F, 'NR > 1 { total += $11 } END { printf "%.2f", total + 0 }' "$INCOMING/$NAME")
log INFO "$NAME passed its checksum: $ROWS order lines, revenue $REVENUE"

gzip --force "$INCOMING/$NAME"
mv "$INCOMING/$NAME.gz" "$ARCHIVE/"
rm -f "$INCOMING/$NAME.sha256"
log INFO "archived $ARCHIVE/$NAME.gz"

```

How it works:

1. **Arguments.** `DAY="${1:-}"` takes the first argument, or an empty string if there is none (`:` gives a default instead of stopping, so the script can print its own usage line). The second argument, the server's address, defaults to the practice server.
2. **log()** is a **function**, a named group of commands, like a Python function. It prints the date and time (date '+%Y-%m-%d %H:%M:%S'), the level, and the message, and `tee -a` writes the same line to the screen and to the log file.
3. **The date check.** `[[ ! "$DAY" =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2}$ ]]` is true when the argument does not match the pattern four digits, dash, two digits, dash, two digits (`==~` compares with a regular expression, as in Chapter 14). Then the script prints its usage to standard error and exits 1.
4. **mkdir -p** makes the three folders if they're missing, so the first run on a new machine works.
5. **The download.** `curl --silent --show-error --fail --max-time 30 --output ...` saves the file, shows an error if there is one, returns a non-zero exit code on a 404 or 500 instead of saving the error page, and gives up after 30 seconds. `if ! curl ...; then` runs the block when curl fails: log, delete any half-written file, exit 2.
6. **The checksum.** A second curl fetches `orders_<day>.csv.sha256`, and `sha256sum --check --status` recomputes the checksum and compares it, saying nothing and answering only with its exit code. It runs inside `( cd "$INCOMING" && ... )`: the brackets run the commands in a separate copy of the shell, so the `cd` doesn't change the script's own folder. A mismatch logs an error and exits 3, leaving the file for inspection.
7. **The report.** `wc -l` and `awk` count the order lines and add up the revenue, as in section 34.4.
8. **The archive.** `gzip --force` compresses the file in place (`--force` overwrites an old copy), `mv` moves the `.gz` into `archive/`, and `rm -f` removes the checksum file (`-f`: no complaint if it's already gone).
9. **The exit code.** A script that reaches its last line ends with the exit code of its last command, 0 here. With `set -euo pipefail`, any unexpected failure on the way stops the script with a non-zero code.

Deliverables: the script, a run of each outcome (success, not published, bad checksum), and the log file.

Stretch goals:

- Add `--force` to re-fetch a day that's already archived, and refuse without it.
- Add a weekly clean-up that deletes archives older than 365 days (`find archive -name "*.gz" -mtime +365 -delete`), after running it once without `-delete`.
- Schedule it with `cron`: `crontab -e`, then `15 7 * * * cd /var/riverstone && ./fetch_daily.sh "$(date -d yesterday +%F)" >> logs/cron.log 2>&1`. Remember that `cron`'s `PATH` is minimal (section 34.6), and that `%` must be escaped in a `crontab`.
- Make the script fetch a range of days and keep going after a failure, reporting a summary at the end.
- Rewrite it in Python with `requests` (Chapter 18) and compare: which parts got simpler, which got longer? Chapter 29 shows how to turn that into a tool with tests.

REVIEW

Recap

- Recap of section 26.0: `pwd`, `ls`, `cd`, `cp`, `mv`, `rm`, and friends. New here: `head`, `tail -f`, searching in `less`, `find`, and wildcards. Linux is case-sensitive, and **globbing** (`*`, `?`) is done by the shell before the command runs.
- Every command has **standard input**, **standard output**, **standard error**, and an **exit code**. `>`, `>>`, `2>`, `<`, and `|` connect them; `&&` and `||` chain on success and failure.
- **grep** finds lines; **wc**, **sort**, **uniq -c**, and **cut** count and group; **awk** does arithmetic on columns (`FNR > 1` skips each file's header). These tools see text, not CSV.
- **Permissions** are user, group, and others, each with read, write, and execute: 644 data, 755 scripts, 600 secrets. `chmod` changes them.
- **Environment variables** pass settings to programs; `PATH` decides which program a name runs; scheduled jobs get a minimal environment.
- **Shell scripts** start with a **shebang** and set `-euo pipefail`, quote every variable, take arguments, and exit with a meaningful code. Move to Python when logic grows.
- **SSH** uses a key pair; `~/.ssh/config` names hosts; `scp` and `rsync` copy files; keys must be 600.
- **Networking**: an **IP address** is a machine, **DNS** turns names into addresses, a **port** is a program on that machine, and **HTTP** is the request and its **status code**. `ss -ltn` shows what's listening; `curl` makes the request.

Key terms

shell · bash · zsh · WSL · globbing · standard input · standard output · standard error · redirection · pipe · tee · exit code · test · grep · wc · sort · uniq · cut · awk · NR and FNR · field separator · associative array · printf · head · tail · less · find · chmod · permission bits · owner · group · others · sudo · superuser · environment variable · export · PATH · shell built-in · shebang · set -euo pipefail · command substitution · function · quoting · for loop · argument (\$1, \$@) · background job · SSH · SSH server · host key fingerprint · known_hosts · key pair · public key · private key · passphrase · ~/.ssh/config · scp · rsync · dry run · port forwarding · IP address · localhost · private address range · DNS · port · HTTP · method · status code · curl · ss · cron · checksum · sha256sum · gzip

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can find my way around a strange Linux server with `pwd`, `ls`, `cd`, and `find`.
- ☐ I can answer "how many rows, which files, which values" with a pipeline of small tools.
- ☐ I know where output and errors go, and how to send them where I want.
- ☐ I check exit codes, and my scripts return meaningful ones.
- ☐ I can search a log for errors and see the lines around them.
- ☐ I can read `ls -l` permissions and fix them with `chmod`.
- ☐ I know what `PATH` is, and why a cron job can't find a command my terminal finds.
- ☐ I can write a script that takes arguments, uses `set -euo pipefail`, quotes its variables, and fails safely.
- ☐ I can connect to a server with an SSH key and copy files both ways.
- ☐ I can explain IP, DNS, port, and HTTP status codes to a colleague, and test an API with `curl`.

PRACTISE

Exercises

Work in `companion/ch34/practice`. Predict each answer before you run it.

Warm-up

1. How many daily export files are there for November 2025, and how many lines do they hold in total (including headers)?
2. Which export file is the largest, and how many order lines does it hold?
3. Show the first two order lines of 26 November 2025 without opening an editor.
4. How many lines of `../sales_lines_2025.csv` mention Bengaluru, and how many mention `unassigned`?

Core

5. List the ten customers with the most order lines in 2025, with their counts, using `cut`, `sort`, and `uniq`.
6. Total 2025 revenue from `../sales_lines_2025.csv` with `awk`, and check it against the ₹43,35,471 Chapter 13 reports.
7. Revenue by segment for December 2025, largest first.
8. Which days in December have no orders? Answer it twice: once with a `for` loop, once with `find` and `-size`.
9. Save the report job's `WARNING` lines to `/tmp/warnings.txt` and count them, without the count including the file name.
10. Write `weekly_summary.sh`, which takes a Monday's date and prints one line per day of that week using `daily_summary.sh`, then a total.

Stretch

11. Use `awk` to find the three order lines with the highest revenue in 2025, printing date, customer, product, and revenue.
12. Count the `curl` status codes you get for the first five days of January 2026 from the local file server, using a loop.
13. `scp` November's exports to `reportserver:/tmp/nov/`, then verify on the server that the line counts match, in a single `ssh` command.
14. A colleague's script runs `cd $DIR && rm -rf *`. Explain, precisely, what happens when `DIR` is unset, and rewrite the two lines safely.

Think about it (no commands needed)

15. Your Python script works in your terminal, with your virtual environment active, and fails in cron with `ModuleNotFoundError: No module named 'pandas'`. What's happening, and give two fixes.
16. When is `grep` on a CSV the right tool, and when is it a trap? Give one example of each from this chapter's data.

Answers

1.

```
# terminal, in ~/ch34/practice
$ ls exports/orders_2025-11-*.csv | wc -l
30

$ cat exports/orders_2025-11-*.csv | wc -l
65
```

30 files, 30 headers plus 35 order lines. `wc -l exports/orders_2025-11-*.csv | tail -1` gives the same total with a `total` label, and also shows each file.

2.

```
# terminal, in ~/ch34/practice
$ ls -S exports | head -2
orders_2025-11-16.csv
orders_2025-12-16.csv

$ wc -l exports/orders_2025-11-16.csv
8 exports/orders_2025-11-16.csv
```

`ls -S` sorts by size, largest first. The file for 16 November is the biggest of the two months. (`du -h exports/* | sort -h | tail -2` is the more general form, since it works on folders too.)

3.

```
# terminal, in ~/ch34/practice
$ head -3 exports/orders_2025-11-26.csv | tail -2
10155,2025-11-26,3,Green Leaf Hotels,Pune,Hospitality,107,Lunch Box Set,Kitchen,10,3610.00,Farah
    Khan
10156,2025-11-26,5,Metro Mart,Mumbai,Retail,101,Storage Box 10L,Storage,15,6450.00,Rahul Mehta
```

`head -3` takes the header and the first two order lines; `tail -2` drops the header.

4.

```
# terminal, in ~/ch34/practice
$ grep -c Bengaluru ../sales_lines_2025.csv
34

$ grep -c unassigned ../sales_lines_2025.csv
16
```

34 lines mention Bengaluru, and 16 order lines have no sales rep recorded, which is Chapter 13's data-quality finding (11 orders, 10 of them not cancelled) seen at the line level.

5.

```
# terminal, in ~/ch34/practice
$ cut -d, -f4 ../sales_lines_2025.csv | tail -n +2 | sort | uniq -c | sort -nr | head -10
    32 Sharma Hardware
    32 Green Leaf Hotels
    29 Metro Mart
    25 Northgate Distributors
    25 Fresh Bowl Kitchens
    21 Harbour Traders
    21 Coastal Foods
    20 Deccan Packaging
    16 Spice Route Restaurants
    16 Evergreen Mart
```

Field 4 is the customer name. The same warning as before: this works because no customer name contains a comma.

6.

```
# terminal, in ~/ch34/practice
$ awk -F, 'FNR > 1 { total += $11 } END { printf "%.2f\n", total }' ../sales_lines_2025.csv
4335471.00
```

₹43,35,471.00, matching Chapter 13's total for 2025 to the rupee. ✓

7.

```
# terminal, in ~/ch34/practice
$ awk -F, 'FNR > 1 { revenue[$6] += $11 } \
    END { for (s in revenue) printf "%-12s %12.2f\n", s, revenue[s] }' \
    exports/orders_2025-12-*.csv | sort -k2 -nr
Wholesale      205238.50
Hospitality    131080.00
Retail         103505.00
```

The three segments add up to December's ₹4,39,823.50. ✓

8.

```
# terminal, in ~/ch34/practice
$ for f in exports/orders_2025-12-*.csv; do \
    [[ $(wc -l < "$f") -eq 1 ]] && basename "$f"; \
    done | head -5
orders_2025-12-02.csv
orders_2025-12-03.csv
orders_2025-12-06.csv
orders_2025-12-07.csv
orders_2025-12-09.csv

$ find exports -name "orders_2025-12-*.csv" -size -125c | sort | wc -l
17
```

In the loop, `basename` strips the folder from a path, leaving the file name. A file with only a header is 1 line and 123 bytes (122 characters and the line ending), so `-size -125c` (smaller than 125 bytes) finds them, and no file with an order in it is that small. The loop is clearer; the `find` version is shorter and depends on knowing the header's exact length, which is the kind of assumption that breaks when a column is added.

9.

```
# terminal, in ~/ch34/practice
$ grep WARNING logs/report.log > /tmp/warnings.txt

$ wc -l < /tmp/warnings.txt
37
```

`wc -l < file` reads from standard input, so `wc` never learns the file's name and prints only the number. `wc -l /tmp/warnings.txt` would print `37 /tmp/warnings.txt`, which is awkward inside `$( ... )`.

10.

```
#!/usr/bin/env bash
# weekly_summary.sh 2025-12-15
set -euo pipefail

MONDAY="${1:?usage: $0 YYYY-MM-DD (a Monday)}"
TOTAL=0

for offset in 0 1 2 3 4 5 6; do
    DAY=$(date -d "$MONDAY + $offset days" +%F)      # macOS: date -j -v+"$offset"d -f %Y-%m-%d "$MONDAY"
    LINE=$(./daily_summary.sh "$DAY")
    echo "$LINE"
    REVENUE=${LINE##*revenue }
    TOTAL=$(awk -v a="$TOTAL" -v b="$REVENUE" 'BEGIN { printf "%.2f", a + b }')
done

echo "week of $MONDAY: revenue $TOTAL"
```

```
# terminal, in ~/ch34/practice
$ cp ../weekly_summary.sh . && chmod +x weekly_summary.sh

$ ./weekly_summary.sh 2025-12-15
2025-12-15: order lines 0, customers 0, revenue 0.00
2025-12-16: order lines 7, customers 3, revenue 89800.00
2025-12-17: order lines 2, customers 1, revenue 37628.00
2025-12-18: order lines 2, customers 1, revenue 23850.00
2025-12-19: order lines 1, customers 1, revenue 5750.00
2025-12-20: order lines 0, customers 0, revenue 0.00
2025-12-21: order lines 1, customers 1, revenue 2755.00
week of 2025-12-15: revenue 159783.00
```

`date -d "$MONDAY + $offset days" +%F` works out each day of the week (`-d` gives `date` a date to use instead of today; on macOS, the `date -j -v` form in the comment does the same). `${LINE##*revenue }` strips everything up to the last "revenue ", one of the shell's parameter expansions, leaving the number. `awk -v a="$TOTAL"` hands a shell variable to `awk` as the `awk` variable `a`, and `BEGIN { ... }` runs before any input is read, so `awk` works as a calculator. The total uses `awk` because `bash`'s arithmetic is whole numbers only, which is a good sign that anything more complicated belongs in Python.

11.

```
# terminal, in ~/ch34/practice
$ awk -F, 'FNR > 1 { printf "%12.2f %s %-20s %s\n", $11, $2, $4, $8 }' \
  ../sales_lines_2025.csv | sort -nr | head -3
56700.00  2025-09-13  Deccan Packaging      Industrial Crate
51520.00  2025-12-08  Northgate Distributors Industrial Crate
51520.00  2025-10-13  Harbour Traders      Industrial Crate
```

Printing the sort key first and sorting numerically is the usual shell pattern for a top-N. The largest single order line is ₹56,700: 45 Industrial Crates to Deccan Packaging on 13 September. (Chapter 13's biggest *order*, ₹86,615 on 22 November, is several lines added together, which is the grain difference from Chapter 28 showing up in a shell pipeline.)

12.

```
# terminal, in ~/ch34/practice
$ python3 ../daily_file_server.py > /tmp/server.log 2>&1 &

$ sleep 1; for d in 01 02 03 04 05; do \
    curl --silent --output /dev/null --write-out "%{http_code}\n" \
    "http://127.0.0.1:8034/exports/orders_2026-01-$d.csv"; \
done | sort | uniq -c
      5 404

$ kill %1
```

All five are 404: the server only has November and December 2025. A script that didn't check the status would have saved five files containing the words "not found".

13.

```
# terminal, in ~/ch34/practice
$ ssh reportserver "mkdir -p /tmp/nov"

$ scp exports/orders_2025-11-*.csv reportserver:/tmp/nov/

$ ssh reportserver "cat /tmp/nov/*.csv | wc -l"
65

$ cat exports/orders_2025-11-*.csv | wc -l
65
```

Both 65, so nothing was lost. For many files or a slow connection, `rsync -a exports/orders_2025-11-*.csv reportserver:/tmp/nov/` is the better tool, and the same command with `-anv` shows what it would do first.

14. With `DIR` unset, `cd $DIR` expands to plain `cd`, which succeeds and moves to the home folder. The `&&` then runs `rm -rf *` there, deleting everything the user owns. Two changes make it safe:

```
set -euo pipefail                # -u makes an unset variable an error
cd "${DIR:?DIR is not set}" && rm -rf -- ./*
```

`"${DIR:?message}"` stops with a message if `DIR` is empty or unset, the quotes handle spaces, and `--` stops a file named like an option from being read as one. Better still, delete by pattern (`rm -f ./*.tmp`) rather than everything.

15. Cron runs with a minimal environment: a short `PATH` (often just `/usr/bin:/bin`), no `~/.bashrc`, no activated virtual environment, and a different current folder. So `python3` in the crontab is the system's Python, which doesn't have the packages you installed into `.venv`. Two fixes: call the environment's Python by its full path (`/home/meera/analyst-to-architect/.venv/bin/python report.py`, as Chapter 20's crontab does), or set `PATH=` at the top of the crontab or the script so the environment's `bin` folder comes first. A third, often best: have cron run a small wrapper script that sets the environment explicitly, so the job doesn't depend on anyone's login setup.

16. `grep` is right for a quick look and a quick count on data you trust: *"how many of these 61 files mention Metro Mart, and which ones"* took one command and no parsing. It's a trap when the answer has to be correct in a published number and the data can contain commas, quotes, or newlines inside fields, or when the match could appear in the wrong column: `grep -c Storage ../sales_lines_2025.csv` counts lines where *any* field contains "Storage", including a product name and a category, which is not the same as "order lines in the Storage category". For that, use `awk -F, '$9 == "Storage"'`, or `pandas`.

NEXT

Where this leads

- **Chapter 29, Python as Software, Not Scripts**, takes over when a shell script grows logic: it runs with the same exit codes, environment variables, and HTTP ideas you've just used.
- **Chapter 32, Analytics Engineering with dbt**, is a command-line tool end to end.
- **Chapter 46, Pipelines & Orchestration**, and **Chapter 49, Storage, Warehouses & Lakehouses**, run on machines you reach only this way.
- **Chapter 52, The Cloud, Containers & Infrastructure as Code**, builds on IP addresses, ports, and SSH keys for virtual machines and networks.
- **Chapter 77, Data Engineering & Data System Design Bank**, asks about permissions, exit codes, and debugging a failed job.
- **Looking back:** Chapter 20's `cron` lines and Chapter 26's Git commands are commands you now understand end to end.

Chapter 29. Python as Software, Not Scripts

Chapter at a glance

You will learn to: recognize the signs that a script has outgrown being a script · split a script into small functions and modules with one job each · lay out a project with `src/`, `tests/`, and `pyproject.toml` · keep passwords and settings out of code · create reproducible environments with `venv`, pinned requirements, and a lockfile managed by `uv` · write small classes and data classes, and know when not to · add type hints and let `mypy` find bugs before users do · write fast, focused tests with `pytest`, fixtures, and parametrized cases, and run them on every push · raise meaningful exceptions and log instead of printing · build an API client that handles pagination, timeouts, retries, and rate limits · review code, and have your code reviewed, with a checklist.

Before you start: Chapter 17 (functions, files, errors, packages), Chapter 18 (pandas and the monthly report), Chapter 34 (the terminal, environment variables, exit codes), Chapter 12 (connecting to PostgreSQL), Chapter 2 (what an API is), and Chapter 26 (Git, pull requests, and the automated check of section 26.8).

Time needed: 20–24 hours of reading and practice, over three weeks, in four sittings marked "Stop here": sections 29.1–29.3 (about 5 hours), 29.4–29.6 (5 hours), 29.7–29.8 (6 hours), and 29.9–29.10 with the project (6–8 hours). Classes, type hints, a lockfile, and a test framework are all new here, so type the examples rather than reading them.

Tools: Python 3.14 (Chapter 17), `uv` (section 29.4), VS Code with its Python and Jupyter extensions, and the `riverstone_2025` database from Chapter 13.

Practice data: the companion folder `ch29/`: the starting script `start/monthly_report.py`, the finished package `riverstone-report/`, and a local mock CRM API, `mock_crm_api.py`, with 43 leads. Every command was run, and every output shown is real.

In this chapter

START	Why this matters · In plain English	147
LEARN	29.1 The script that works until it doesn't	148
	29.2 Functions and modules: one job each	150
	29.3 Project structure and configuration	157
	29.4 Environments and lockfiles	161
	29.5 Classes, when they help	165
	29.6 Type hints	172
	29.7 Testing with <code>pytest</code>	175
	29.8 Errors and logging	184
	29.9 Robust API clients	190
	29.10 Code review	199
APPLY	Common mistakes · In the real world · Project	201
REVIEW	Recap · Key terms · Check yourself	204
PRACTISE	Exercises	206
	Answers	207
NEXT	Where this leads	212

Why this matters

Riverstone's monthly report still runs from Imran's original script, the one that Chapter 18's version was written to replace. It pulls December's sales, adds them up, and writes a workbook. Then this happens:

- *"Can Priya run it while you're on leave?"* It needs your password, your folder, and the exact versions of pandas on your laptop.
- *"The January report says revenue was zero."* The script found no data, wrote a report full of zeros anyway, printed `done`, and overwrote December's file.
- *"Can we also pull leads from the CRM?"* The CRM's API returns 10 leads per page, sometimes fails, and blocks you if you ask too often.
- *"Finance changed the revenue rule. Did anything break?"* Nobody knows until someone reads every number.

If you finished Chapter 18, your own script already fixes part of this: its password is in `.env`, its SQL uses parameters, it logs, and it returns an exit code. This chapter goes the rest of the way.

None of these are problems with Python. They're problems with treating code that a business depends on as a one-off script. This chapter turns the report into **software**: code that someone else can install, run, understand, change, and trust. The techniques are the everyday habits of data engineers and analytics engineers, and the questions interviewers use to tell them apart from people who have only written notebooks: *"How do you structure a Python project?"*, *"How would you test this function?"*, *"What happens when the API returns a 429?"*

In plain English

A script is a recipe scribbled on the back of an envelope. Software is a restaurant kitchen.

The envelope works when you cook alone. It says "a handful of salt" because you know your hand. It lists the steps in one long paragraph. If the shop is out of an ingredient, you improvise. Nobody else could cook from it.

A restaurant kitchen is built so that any trained cook can produce the same dish every night:

- The work is divided into **stations**: one prepares vegetables, one grills, one plates. Each has one job. In code, these are **functions** and **modules**.
- Ingredients come from **named suppliers with fixed specifications**, so tonight's dish tastes like last week's. That's a **lockfile** pinning exact package versions.
- The **head chef tastes every station's output** before it leaves the kitchen, the same way every time. Those are **tests**.
- Recipes state **exactly what goes in and comes out** ("200 g, diced"), not "some onions". That's **type hints**.
- When something goes wrong, the kitchen doesn't quietly serve a bad plate. It **stops and says why**, and someone writes it in the **log book**. Those are **exceptions** and **logging**.

- Deliveries are late sometimes. The kitchen **calls again after a sensible wait**, not every ten seconds, and doesn't give up on the first try. That's a **robust API client**.
- A new recipe isn't added to the menu until **another chef has read it**. That's **code review**.

LEARN

29.1 The script that works until it doesn't

Work the way Chapter 18 did: copy the folder `companion/ch29` into `work`, so you have `work/ch29` to practise in. Here is the starting point, `start/monthly_report.py`. It's the monthly report as Imran left it, and it runs:

```
# monthly_report.py - Riverstone's monthly sales report, as a first working script.
# Analyst to Architect · Chapter 29 · the "before" version: it works, and it has every problem section 29.1 lists.
# How: python3 monthly_report.py (needs riverstone_2025 in PostgreSQL, pandas, openpyxl, SQLAlchemy, psycopg)
# Tested on: Python 3.14.7, pandas 3.0.6, openpyxl 3.1.5, SQLAlchemy 2.1.1, psycopg 3.3.6, PostgreSQL 16.
# Riverstone Supplies is fictional; every name and number is invented.
import pandas as pd
from sqlalchemy import create_engine

conn = create_engine("postgresql+psycopg://postgres:riverstone123@localhost/riverstone_2025")
month = "2025-12"

df = pd.read_sql("SELECT sl.*, c.customer_name, p.category AS cat FROM sales_lines sl "
                "JOIN customers c ON c.customer_id = sl.customer_id "
                "JOIN products p ON p.product_id = sl.product_id", conn)
df["order_date"] = pd.to_datetime(df["order_date"])
df = df[df["order_date"].dt.strftime("%Y-%m") == month]

total = df["net_revenue"].astype(float).sum()
orders = df["order_id"].nunique()
print("Revenue:", total)
print("Orders:", orders)

try:
    t = pd.read_sql("SELECT target_revenue FROM sales_targets WHERE target_month = '" + month + "-01'", conn)
    target = float(t.iloc[0, 0])
except:
    target = 0

cats = df.groupby("cat")["net_revenue"].sum().astype(float).reset_index()
cats["share"] = cats["net_revenue"] / total
cats = cats.sort_values("net_revenue", ascending=False)

top = df.groupby("customer_name")
      ["net_revenue"].sum().astype(float).sort_values(ascending=False).head(5).reset_index()

with pd.ExcelWriter("Monthly_Report_Dec_FINAL.xlsx") as w:
    pd.DataFrame({"metric": ["Revenue", "Orders", "Target", "% of target"],
                  "value": [total, orders, target, total / target * 100]}).to_excel(w, sheet_name="Summary",
                                          index=False)
    cats.to_excel(w, sheet_name="Categories", index=False)
    top.to_excel(w, sheet_name="Top customers", index=False)
print("done")
```

The password in its `create_engine` line is Imran's. To run it, change `riverstone123` to your own PostgreSQL password, which is already the first of its problems. Then run it from a terminal in `work/ch29/start`, with the book's `.venv` active (Chapter 17, section 17.0):

```
# terminal, in work/ch29/start
$ python monthly_report.py
Revenue: 439823.5
Orders: 18
done
```

December's numbers are right: ₹4,39,823.50 (Chapter 13 showed it rounded, as ₹4,39,824) from 18 orders. Now it's the first week of February, and someone changes `month` to `"2026-01"` to produce January's report. Riverstone's `riverstone_2025` database ends on 31 December 2025, so there is no January data, exactly as there would be if the nightly load had failed:

```
# terminal, in work/ch29/start
$ python monthly_report.py
Revenue: 0.0
Orders: 0
/home/meera/ch29/start/monthly_report.py:37: RuntimeWarning: invalid value encountered in scalar divide
  "value": [total, orders, target, total / target * 100])).to_excel(w, sheet_name="Summary",
      index=False)
done
```

It printed `done`. The exit code was 0, so a scheduler would record success. And because the file name is fixed, it replaced `Monthly_Report_Dec_FINAL.xlsx`, the good December report, with a January report showing zero revenue and a percentage of target that isn't a number. (The warning names the script's full path, which on your computer is your own `work/ch29/start` folder.)

Here's every problem in those 40 lines, and the section that fixes it:

Problem in the script	Why it hurts	Fixed in
Everything runs at the top level, in one file	Can't test a calculation without a database, can't reuse a piece	29.2, 29.3
Password and connection details in the code	Anyone with the file has the password; it ends up in Git	29.3
Month and file name typed into the code	Every run means editing code; files overwrite each other	29.3, 29.5
No record of which package versions it needs	"Works on my machine"; breaks after an upgrade (it already broke once: pandas 3 requires <code>sheet_name=</code> by name)	29.4
No tests	Nobody knows whether a change broke a number	29.7
<code>except:</code> catches everything and sets <code>target = 0</code>	Hides the real cause, and produces a wrong percentage	29.8
<code>print</code> instead of logging; exit code 0 on failure	Scheduled runs look successful when they aren't	29.8
SQL built by gluing strings (<code>"... = '" + month + "-01'"</code>)	Breaks on unexpected input, and is how SQL injection happens	29.2
Loads every order line of the year, then filters in pandas	Fine for 326 lines, slow for 2 million (Chapter 28)	29.2

Problem in the script	Why it hurts	Fixed in
Joins products although the <code>sales_lines</code> view already has <code>category</code>	Two category columns (<code>category</code> and <code>cat</code>); a reader can't tell which is right	29.2

Simplification note. Scripts aren't bad. A script you run once, or a notebook for exploring data, doesn't need a package, tests, and a lockfile. The signal to change is when **other people depend on the output, it runs on a schedule, or it will be changed by someone other than its author**. Section 29.10 comes back to this judgment.

29.2 Functions and modules: one job each

The first step is always the same: pull each job into a **function** with a clear name, inputs, and a return value. A function that only calculates from its inputs, without reading files, databases, clocks, or global variables, and without changing anything outside itself, is called a **pure function**. Pure functions are the easiest code in the world to test, because the same input always gives the same output.

The report has four jobs, and they separate cleanly:

1. **Configure:** which month, which database, where to write.
2. **Read:** fetch the month's sales lines and target from the database.
3. **Transform:** summarize, break down by category, rank customers. *Pure*.
4. **Write:** produce the formatted workbook.

Start with the transform job, in a notebook, one function per cell. Create `ch29.ipynb` in `work/ch29` and choose the book's `.venv` kernel, as in Chapter 18.

Five lines you can add up in your head

A pure function doesn't need the database, so try each one on a DataFrame small enough to check by hand:

```
import pandas as pd

lines = pd.DataFrame({
    "order_id":    [1, 1, 2, 3, 3],
    "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart",
                    "Green Leaf Hotels", "Green Leaf Hotels"],
    "category":    ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
    "net_revenue": [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
})
print(lines)
```

	order_id	customer_name	category	net_revenue
0	1	Sharma Hardware	Storage	10000.0
1	1	Sharma Hardware	Kitchen	2500.0
2	2	Metro Mart	Storage	6000.0
3	3	Green Leaf Hotels	Kitchen	1500.0
4	3	Green Leaf Hotels	Furniture	5000.0

Five order lines from three orders: Sharma Hardware's order 1 has two lines (₹10,000 + ₹2,500), and so does Green Leaf Hotels' order 3. The columns are the four the report needs from the `sales_lines` view.

The summary, as a function

Before you run the next cell, write down the revenue, the number of orders, and the percentage of a ₹20,000 target you expect.

```
def summarize(lines, target):
    revenue = round(float(lines["net_revenue"].sum()), 2)
    return {
        "revenue": revenue,
        "orders": int(lines["order_id"].nunique()),
        "customers": int(lines["customer_name"].nunique()),
        "pct_of_target": round(100 * revenue / target, 1),
    }

print(summarize(lines, 20000.0))
```

```
{'revenue': 25000.0, 'orders': 3, 'customers': 3, 'pct_of_target': 125.0}
```

How it works:

- **def summarize(lines, target):** takes the month's lines and its target, and **returns** a dictionary (Chapter 17, section 17.7) instead of printing, so other code can use the result.
- **nunique()** counts **orders**, not lines: five lines, three orders. That's Chapter 12's grain lesson, now written once in a function everyone uses.
- **float(...)** and **int(...)** turn pandas' own number types into plain Python numbers, so the dictionary prints as 25000.0 rather than `np.float64(25000.0)`.
- ₹25,000 ÷ ₹20,000 = 125.0%. ✓

Categories and customers

The category table uses the named aggregation of Chapter 18, section 18.6:

```
def revenue_by_category(lines):
    result = (lines.groupby("category", as_index=False)
              .agg(net_revenue=("net_revenue", "sum"))
              .sort_values(["net_revenue", "category"], ascending=[False, True],
                           ignore_index=True))
    total = result["net_revenue"].sum()
    result["share_pct"] = (100 * result["net_revenue"] / total).round(1)
    return result

print(revenue_by_category(lines))
```

	category	net_revenue	share_pct
0	Storage	16000.0	64.0
1	Furniture	5000.0	20.0
2	Kitchen	4000.0	16.0

How it works:

- `.agg(net_revenue=("net_revenue", "sum"))` is a named aggregation: the new column's name on the left, and on the right the column to use and what to do with it. `as_index=False` keeps `category` as an ordinary column.
- `.sort_values(["net_revenue", "category"], ascending=[False, True], ...)` sorts by two columns: revenue, largest first, and then by name, A to Z. The second key makes the order stable when two categories tie, the same reason Chapter 13 added tie-breakers to `ROW_NUMBER`.
- `ignore_index=True` numbers the sorted rows 0, 1, 2 afresh, as it did for `concat` in Chapter 18.
- The shares add to 100.0% (64.0 + 20.0 + 16.0). ✓

The customer ranking is the same pattern, cut to the top `n`:

```
def top_customers(lines, n=5):
    totals = (lines.groupby("customer_name", as_index=False)
               .agg(net_revenue=("net_revenue", "sum")))
    ranked = totals.sort_values(["net_revenue", "customer_name"], ascending=[False, True],
                                ignore_index=True)
    return ranked.head(n)

print(top_customers(lines, n=2))
```

	customer_name	net_revenue
0	Sharma Hardware	12500.0
1	Green Leaf Hotels	6500.0

`n=5` in the `def` line is a **default**: leave `n` out and you get the top five. Sharma Hardware's two lines add to ₹12,500, so it comes first.

An empty month must stop, not print zeros

What does `summarize` do with January, when there's no data? `lines.iloc[0:0]` makes the test case: `iloc` picks rows by position (Chapter 18), and `0:0` is the empty range, so it's a DataFrame with the same columns and no rows.

```
print(summarize(lines.iloc[0:0], 20000.0))
```

```
{'revenue': 0.0, 'orders': 0, 'customers': 0, 'pct_of_target': 0.0}
```

That's the script's bug, reproduced in one line: no data, and a summary that looks like a real month with zero sales. The fix is a function that checks the lines first and **raises** an exception (Chapter 17, section 17.10) when they can't make a correct report:

```
REQUIRED_COLUMNS = {"order_id", "customer_name", "category", "net_revenue"}

def check_lines(lines):
    missing = REQUIRED_COLUMNS - set(lines.columns)
    if missing:
        raise ValueError(f"sales lines are missing columns: {sorted(missing)}")
    if lines.empty:
        raise ValueError("no sales lines for this month")

check_lines(lines)
print("five lines: OK")
check_lines(lines.iloc[0:0])
```

```
five lines: OK
ValueError: no sales lines for this month
```

How it works:

- **REQUIRED_COLUMNS - set(lines.columns)** is set difference (Chapter 17, section 17.7): the names that are required but not present. An empty set is false, so **if missing:** only fires when something is absent.
- **sorted(missing)** lists the missing names in a fixed order, so the message reads the same every time.
- **lines.empty** is True when the DataFrame has no rows.
- **raise ValueError(...)** stops with a message instead of carrying on. The last line of the cell raised it, so the notebook shows the error under the cell.

From cells to a module

Each job goes in its own **module**, a `.py` file you can `import`. The four cells above become the first version of `transform.py`, with one change: every function calls `check_lines` before it calculates.

```
"""Pure calculations: DataFrames in, results out. No database, no files, so easy to test."""

REQUIRED_COLUMNS = {"order_id", "customer_name", "category", "net_revenue"}

def check_lines(lines):
    missing = REQUIRED_COLUMNS - set(lines.columns)
    if missing:
        raise ValueError(f"sales lines are missing columns: {sorted(missing)}")
    if lines.empty:
        raise ValueError("no sales lines for this month")

def summarize(lines, target):
    check_lines(lines)
    revenue = round(float(lines["net_revenue"].sum()), 2)
    return {
```

```

        "revenue": revenue,
        "orders": int(lines["order_id"].nunique()),
        "customers": int(lines["customer_name"].nunique()),
        "pct_of_target": round(100 * revenue / target, 1),
    }

def revenue_by_category(lines):
    check_lines(lines)
    result = (lines.groupby("category", as_index=False)
              .agg(net_revenue=("net_revenue", "sum"))
              .sort_values(["net_revenue", "category"], ascending=[False, True],
                           ignore_index=True))
    total = result["net_revenue"].sum()
    result["share_pct"] = (100 * result["net_revenue"] / total).round(1)
    return result

def top_customers(lines, n=5):
    if n < 1:
        raise ValueError("n must be at least 1")
    check_lines(lines)
    totals = (lines.groupby("customer_name", as_index=False)
              .agg(net_revenue=("net_revenue", "sum")))
    ranked = totals.sort_values(["net_revenue", "customer_name"], ascending=[False, True],
                                ignore_index=True)
    return ranked.head(n)

```

- The text in triple quotes on the first line is the module's **docstring**: what the file is for, the first thing a colleague reads.
- **if n < 1: raise ValueError(...)** refuses a nonsense request (the top zero customers) instead of returning an empty table.
- The file doesn't import pandas: it only calls methods on the DataFrames it's given.

This is the first of three versions. Section 29.5 turns the dictionary into a small class, section 29.6 adds type hints, and section 29.8 gives the empty month its own kind of error. The finished file is `riverstone-report/src/riverstone_report/transform.py`.

The read job

The read module is the only place that knows SQL. It fixes three problems from the script: it asks the database for **only the month's rows**; it passes the dates as **query parameters** (`:month_start`, Chapter 18, section 18.13), so values are never glued into the SQL text; and it takes `category` from the `sales_lines` view, without the extra join.

Connect the notebook first. The finished package keeps its settings in a `.env` file (Chapter 18, section 18.13) inside `riverstone-report`: copy `riverstone-report/.env.example` to `riverstone-report/.env` and put your own PostgreSQL password in place of `YOUR_PASSWORD`. The file has two lines:

```

RIVERSTONE_DATABASE_URL=postgresql+psycopg://postgres:YOUR_PASSWORD@localhost/riverstone_2025
RIVERSTONE_OUTPUT_DIR=reports

```

```
import os
from datetime import date

from dotenv import load_dotenv
from sqlalchemy import create_engine, text

load_dotenv("riverstone-report/.env")
engine = create_engine(os.environ["RIVERSTONE_DATABASE_URL"])
print(engine.dialect.name, engine.url.database)
```

```
postgresql riverstone_2025
```

- `load_dotenv("riverstone-report/.env")` reads that file into the environment. Given a path, it reads exactly that file instead of searching for one.
- This package uses its own variable, `RIVERSTONE_DATABASE_URL`, because it reads `riverstone_2025`, not the `riverstone_full` that Chapter 18's `RIVERSTONE_DB` points at.
- `engine.url.database` is the database name from the URL. Printing it, rather than the whole URL, confirms the connection without showing the password.

Now the two read functions, in one cell:

```
SALES_LINES_SQL = text("""
    SELECT sl.order_id, sl.order_date, c.customer_name, sl.category, sl.net_revenue
    FROM sales_lines AS sl
    JOIN customers AS c ON c.customer_id = sl.customer_id
    WHERE sl.order_date >= :month_start AND sl.order_date < :next_month
""")

TARGET_SQL = text("SELECT target_revenue FROM sales_targets WHERE target_month = :month_start")

def fetch_sales_lines(engine, month_start, next_month):
    """One row per non-cancelled order line in the month, with net_revenue as a float."""
    with engine.connect() as conn:
        lines = pd.read_sql(SALES_LINES_SQL, conn,
                           params={"month_start": month_start, "next_month": next_month})
    return lines.astype({"net_revenue": "float64"})

def fetch_target(engine, month_start):
    """The month's revenue target, or None if no target was set."""
    with engine.connect() as conn:
        value = conn.execute(TARGET_SQL, {"month_start": month_start}).scalar_one_or_none()
    return None if value is None else float(value)

december = fetch_sales_lines(engine, date(2025, 12, 1), date(2026, 1, 1))
print(december.shape, round(december["net_revenue"].sum(), 2))
print(fetch_target(engine, date(2025, 12, 1)))
print(fetch_target(engine, date(2026, 1, 1)))
```

```
(35, 5) 439823.5
380000.0
None
```

How it works:

- `text("""...""")` and the `:month_start` and `:next_month` slots are Chapter 18's parameters. The month is a range, `>=` the first day and `<` the first day of the next month, so every date and time inside the month counts and nothing outside it does.
- `with engine.connect() as conn:` opens one connection and closes it when the block ends, even if the query fails.
- `conn.execute(TARGET_SQL, {...})` runs a query without pandas, and `.scalar_one_or_none()` returns the single value it found, or `None` if there was no row. January 2026 has no target row, so the last line prints `None`, not 0.
- `.astype({"net_revenue": "float64"})`: PostgreSQL's `NUMERIC` arrives in Python as `Decimal` values, which don't mix with ordinary numbers in every calculation; `float64` makes the column ordinary decimal numbers.
- December: 35 lines, ₹4,39,823.50, and a target of ₹3,80,000. ✓

The real month goes through the same pure functions as the five made-up lines:

```
print(summarize(december, fetch_target(engine, date(2025, 12, 1))))
```

```
{'revenue': 439823.5, 'orders': 18, 'customers': 15, 'pct_of_target': 115.7}
```

18 orders from 15 customers, 115.7% of target. The two read functions become `db.py`, the only module that imports SQLAlchemy.

Watch out: never build SQL by adding strings. `"WHERE target_month = '" + month + "-01'"` works until `month` contains a quote, or until someone passes `2025-12' OR '1'='1`. That's **SQL injection**, one of the most common security holes in software. Parameters (`:month_start` with SQLAlchemy's `text`, `%s` with `psycopg` directly) send the value separately from the SQL, so it's always treated as a value.

`if __name__ == "__main__"`

Chapter 17, section 17.12, used this line to separate a script's command from its functions. It matters more in a package. When Python runs a file directly, it sets the special variable `__name__` to `"__main__"`; when the same file is imported, `__name__` is the module's name. So code under `if __name__ == "__main__":` runs only when the file is executed, not when a test imports it. The script had all its code at the top level, so importing it to test one calculation would have connected to the database and written a file. Every module in the package defines functions and nothing else at the top level; only `cli.py` has a `__main__` block.

29.3 Project structure and configuration

The layout

The finished package is in `work/ch29/riverstone-report`. List every file in it with Chapter 34's `find`, piped into `sort` so the list is in a fixed order:

```
# terminal, in work/ch29/riverstone-report
$ find . -type f | sort
./env
./env.example
./gitignore
./python-version
./README.md
./pyproject.toml
./src/riverstone_report/__init__.py
./src/riverstone_report/cli.py
./src/riverstone_report/config.py
./src/riverstone_report/crm_client.py
./src/riverstone_report/db.py
./src/riverstone_report/errors.py
./src/riverstone_report/excel.py
./src/riverstone_report/headline.py
./src/riverstone_report/transform.py
./tests/conftest.py
./tests/test_config.py
./tests/test_crm_client.py
./tests/test_reconcile.py
./tests/test_transform.py
./uv.lock
```

`.env` is there because you made it in section 29.2; everything else came with the companion files.

From one script to a package

Before: `monthly_report.py` (40 lines)

✗ connect with a password in the code

✗ hard-coded month and file name

load every order line, filter in pandas

calculate, print

✗ bare except: `target = 0`

✗ write `Monthly_Report_Dec_FINAL.xlsx`

`print('done')`

✗ a problem this chapter fixes

Everything in one file, run top to bottom.



After: `riverstone-report/`

`pyproject.toml`

what it needs; tools; the command

`uv.lock`

exact versions of all 58 packages

`src/riverstone_report/`

`config.py`

settings, checked

`db.py`

the only module with SQL

`transform.py`

pure calculations

`excel.py`

the formatted workbook

`crm_client.py`

retries, pages, rate limits

`cli.py`

`riverstone-report --month`

`errors.py`

`ReportError` and its kinds

`tests/`

23 tests

Each file has one job, so each can be read, tested, and changed alone.

Figure 29.1 – The same report, split so that each file has one job.

- A **package** is a folder of modules that Python can import as one unit. `src/riverstone_report/` is the package. Its `__init__.py` marks the folder as a package and says what it offers.

- The **src/ layout** puts the package one level down, so tests import the *installed* package rather than whatever happens to be in the current folder. It catches a whole class of "passes on my machine" mistakes, and it's what `uv init --package` creates.
- **tests/** sits beside **src/**, not inside it, so tests aren't shipped with the package.
- **pyproject.toml** is the standard file describing a Python project: its name, version, dependencies, entry points, and settings for tools like `pytest` and `mypy`.
- **.gitignore** keeps the virtual environment, caches, `.env`, and generated reports out of version control. **.env.example** is the same file as `.env` with the password replaced by a placeholder, so a colleague knows which settings to supply.

`__init__.py` is three lines:

```
"""Riverstone's monthly sales report, as a tested package (Analyst to Architect, Ch. 29)."""
from riverstone_report.cli import main

__all__ = ["main"]
```

- `from riverstone_report.cli import main` makes `main`, the function that runs the report (section 29.8), available as `riverstone_report.main`.
- `__all__` lists the names the package offers to other code. It's a statement of intent: everything else inside is the package's own business.

pyproject.toml, and TOML in one minute

`pyproject.toml` is written in **TOML**, a format for settings files that is easier to read than JSON. Six rules cover everything in this file.

TOML in one minute.

- `[project]` on its own line starts a **section**; everything below it, up to the next header, belongs to it.
- `name = "riverstone-report"` sets a **key** to a **value**. Text goes in double quotes; numbers and `true/false` don't.
- `["tests"]` in square brackets is a **list**; it may run over several lines, with a comma after each item.
- `{ name = "Meera Iyer", email = "..."} in curly braces is a small group of keys on one line.`
- A dotted header such as `[tool.mypy]` means "the `mypy` part of the `tool` section".
- A line starting with `#` is a comment.

Here is the package's file. `uv init --package` wrote the first version, and `uv add` filled in the dependencies (section 29.4):

```
[project]
name = "riverstone-report"
version = "0.1.0"
description = "Riverstone's monthly sales report: database → tested summary → Excel workbook."
readme = "README.md"
```



```

authors = [
    { name = "Meera Iyer", email = "meera@riverstone.example" }
]
requires-python = ">=3.14"
dependencies = [
    "openpyxl>=3.1.5",
    "pandas>=3.0.6",
    "psycopg[binary]>=3.3.6",
    "requests>=2.34.2",
    "sqlalchemy>=2.1.1",
]

[project.scripts]
riverstone-report = "riverstone_report:main"

[build-system]
requires = ["uv_build>=0.12.19,<0.13.0"]
build-backend = "uv_build"

[dependency-groups]
dev = [
    "ipykernel>=7.3.0",
    "mypy>=2.3.1",
    "pandas-stubs>=3.0.5.260914",
    "pytest>=9.1.1",
    "types-openpyxl>=3.1.5.20260827",
    "types-requests>=2.33.0.20260906",
]

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.mypy]
strict = true
files = ["src"]

```

Section	What it says
[project]	The package itself: name, version, a one-line description, the <code>readme</code> file, the authors, the Python it needs (<code>requires-python = ">=3.14"</code> : 3.14 or later), and its dependencies. <code>>=3.0.6</code> means "this version or newer". <code>psycopg[binary]</code> means <code>psycopg</code> plus its optional ready-built "binary" part, so nothing has to be compiled on your computer
[project.scripts]	A command to create when the package is installed: <code>riverstone-report</code> runs the main function that <code>__init__.py</code> imports from <code>cli.py</code>
[build-system]	How to turn the folder into an installable package. <code>uv</code> fills this in; you don't edit it
[dependency-groups]	Extra packages needed only while developing: the dev group holds the test runner, the type checker and its helpers, and <code>ipykernel</code> for the notebook (section 29.4)
[tool.pytest.ini_options]	Settings for <code>pytest</code> : look for tests in the <code>tests</code> folder
[tool.mypy]	Settings for <code>mypy</code> (section 29.6): check strictly, and check the <code>src</code> folder

`uv init` writes `description = "Add your description here"`; replace it with one sentence that says what the package does, as here. The `authors` line comes from your Git settings (Chapter 26), so yours shows your name.

Configuration: settings belong outside the code

The script had three kinds of setting mixed into its code: the **secret** (the password), the **environment** (which database), and the **run** (which month). They change for different reasons and at different times, so they come from different places:

Setting	Changes when	Comes from
Month to report	every run	a command-line argument: <code>--month 2025-12</code>
Database URL (with password)	per machine or server; must stay secret	an environment variable , <code>RIVERSTONE_DATABASE_URL</code>
Output folder	per machine	an environment variable with a default, <code>RIVERSTONE_OUTPUT_DIR</code>
Number of top customers	rarely; a business decision	a default in code

An **environment variable** is a named value that the operating system passes to every program you start from that terminal, as Chapter 34 showed. Setting one in a bash terminal (macOS, Linux, or Git Bash and WSL on Windows):

```
# terminal
$ export
  RIVERSTONE_DATABASE_URL="postgresql+psycpg://postgres:YOUR_PASSWORD@localhost/riverstone_2025"
```

Typing that in every new terminal is tedious, and it puts the password in your shell's history. So this package keeps its settings in the **.env file** you made in section 29.2, which is listed in `.gitignore`. The package itself never reads `.env`: its code reads only environment variables, which keeps it working unchanged on a server where the settings come from somewhere else. Instead, `uv` loads the file for one command at a time:

```
# terminal, in work/ch29/riverstone-report
$ uv run --env-file .env riverstone-report --month 2025-12
```

`--env-file .env` tells `uv run` to read `.env` into the environment of that one command, and of nothing else. Section 29.4 introduces `uv run`, and section 29.8 shows what this command prints. On servers and schedulers, the platform's **secrets manager** supplies the variables instead. Either way, the rule is the same: **a password never appears in a file that goes into version control**.

PowerShell instead? This chapter's terminal commands are written for bash, as in Chapter 34. In Windows PowerShell three of them are different: set a variable with

```
$env:RIVERSTONE_DATABASE_URL = "..."; activate an environment with .venv\Scripts\Activate.ps1
(with the execution-policy fix of Chapter 17, section 17.0); and print the last exit code with echo
"exit code: $LASTEXITCODE", because PowerShell's $? prints True or False, not the number.
```

Watch out: a password committed once is exposed forever. Deleting it in a later commit doesn't remove it from Git's history, and public repositories are scanned for credentials within minutes. If a secret is ever committed, treat it as leaked: change the password first, then clean up.

Stop here: sitting 1 of 4. You've seen the script fail, rebuilt its calculations as pure functions in a notebook, and met the package's layout, its `pyproject.toml`, and where its settings live.

29.4 Environments and lockfiles

Why "pip install pandas" isn't enough

Every Python project depends on packages, and those packages depend on others. The report needs 5 packages directly, and on Linux they pull in 10 more. When you type `pip install pandas`, you get whatever version is newest *today*, installed into whatever Python you happen to be using. Six months later a colleague installs the "same" requirements and gets pandas 3.1 instead of 3.0, and something behaves differently. The starting script has already been bitten once: written for pandas 2, it stopped running under pandas 3, because `to_excel` now requires the sheet name to be passed by keyword.

Two tools solve two different problems:

- A **virtual environment** is a private folder of packages for one project, with its own copy of the `python` and `pip` commands (Chapter 17, section 17.0). Projects stop interfering with each other.
- A **lockfile** records the **exact version of every package**, including the indirect ones, so every installation is identical.

Recap from Chapter 17: venv and pinned requirements

Python ships with `venv`, and Chapter 17 used it to make the book's environment. It works everywhere, including locked-down company laptops. For a project of its own, the report's environment would be made the same way, in the project folder, with the report's five packages:

```
# terminal, in a new project folder
$ python -m venv .venv
$ source .venv/bin/activate
(.venv) $ python -m pip install pandas openpyxl sqlalchemy "psycopg[binary]" requests
(.venv) $ python -m pip freeze
certifi==2026.7.22
charset-normalizer==3.5.1
et_xmlfile==2.0.0
idna==3.20
numpy==2.5.3
openpyxl==3.1.5
pandas==3.0.6
psycopg==3.3.6
psycopg-binary==3.3.6
python-dateutil==2.9.0.post0
requests==2.34.2
six==1.17.0
SQLAlchemy==2.1.1
typing_extensions==4.16.0
urllib3==2.8.0
(.venv) $ python -m pip freeze > requirements.txt
(.venv) $ deactivate
```

- On Windows in Git Bash, the activate line is `source .venv/Scripts/activate`. Once it's active, the prompt starts with `(.venv)`.

- `pip install` prints a long list of downloads and ends with a line starting `Successfully installed`.
- `pip freeze` lists every package installed, with its exact version: the 5 you asked for and the 10 they need. `> requirements.txt` saves the list.

A colleague then creates and activates their own `.venv` and runs `python -m pip install -r requirements.txt`. That makes `requirements.txt` a simple lockfile. Its weaknesses: it can't tell which packages you asked for and which came along as dependencies, it doesn't record which versions work on other operating systems, and keeping it up to date is manual.

The modern way: uv, pyproject.toml, and uv.lock

`uv` is a fast, free package and project manager from Astral that does the whole job: it creates the virtual environment, resolves versions, writes a cross-platform lockfile, and runs commands inside the environment. It was version 0.12.19 when this chapter was tested (released on 25 September 2026). Install it from the official instructions at `docs.astral.sh/uv` (a one-line installer on each operating system, or `pip install uv`).

Build a package of your own from nothing, beside the finished one, so you can compare them. `mkdir` makes a folder, and `uv init --package` turns the folder you're in into a package:

```
# terminal, in work/ch29
$ uv --version
uv 0.12.19 (x86_64-unknown-linux-gnu)
$ mkdir mine
$ cd mine
$ mkdir riverstone-report
$ cd riverstone-report
$ uv init --package
Initialized project `riverstone-report`
$ ls -a
.
..
.git
.gitignore
.python-version
README.md
pyproject.toml
src
```

- `uv init --package` creates the `src/` layout with a starter `__init__.py`, a `pyproject.toml` named after the folder, a `.gitignore`, an empty `README.md`, and `.python-version`, which records the Python the project uses (3.14; if it isn't installed, `uv` downloads it the first time it needs it).
- It also runs `git init` (Chapter 26), which is the `.git` folder: the project is a repository from the start.

Now add the report's packages, and then the tools that are only needed while developing:

```
# terminal, in work/ch29/mine/riverstone-report
$ uv add pandas openpyxl sqlalchemy "psycopg[binary]" requests
Using CPython 3.14.7
Creating virtual environment at: .venv
Resolved 17 packages in 10ms
...
Installed 16 packages in 28ms
+ certifi==2026.7.22
```

```

+ charset-normalizer==3.5.1
+ et-xmlfile==2.0.0
+ idna==3.20
+ numpy==2.5.3
+ openpyxl==3.1.5
+ pandas==3.0.6
+ psycpg==3.3.6
+ psycpg-binary==3.3.6
+ python-dateutil==2.9.0.post0
+ requests==2.34.2
...
+ six==1.17.0
+ sqlalchemy==2.1.1
+ typing-extensions==4.16.0
+ urllib3==2.8.0
$ uv add --dev pytest mypy pandas-stubs types-requests types-openpyxl ipykernel
Resolved 58 packages in 451ms
...
Installed 38 packages in 80ms
+ ast-serialize==0.11.2
+ asttokens==3.0.2
...

```

(Shortened: the lines left out name your project's own folder, and the second list goes on to all 38 packages.)

- **uv add** adds each package to `pyproject.toml`'s dependencies with a minimum version (pandas>=3.0.6), works out every indirect dependency, writes **uv.lock**, and installs everything into a `.venv` it creates the first time. The 16 installed are the 5 you named, the 10 they need, and your own package.
- **uv add --dev** puts tools that are only needed while developing in the **dev dependency group**: `pytest` (section 29.7), `mypy` and the type information for `pandas`, `requests` and `openpyxl` (section 29.6), and `ipykernel`, which lets a notebook use this environment (below).

`cat pyproject.toml` now shows the same file as section 29.3's, apart from the description you haven't edited yet and the two `[tool]` sections, which you add by hand.

`pyproject.toml` says what the project *needs*; `uv.lock` says exactly what was *installed*. Here's the start of one entry in the lockfile, with the long download addresses and fingerprints cut short:

```

[[package]]
name = "pandas"
version = "3.0.6"
source = { registry = "https://pypi.org/simple" }
dependencies = [
  { name = "numpy" },
  { name = "python-dateutil" },
  { name = "tzdata", marker = "sys_platform == 'emscripten' or sys_platform == 'win32' },
]
sdist = { url = "https://files.pythonhosted.org/packages/.../pandas-3.0.6.tar.gz", hash = "sha256:66b07ef7...", ... }

```

- **[[package]]** in double brackets starts one entry in a *list* of entries: the file has one for every package.
- **dependencies** are the packages `pandas` itself needs. `tzdata` has a **marker**: it's installed only on Windows (and in browsers), which is why the file works on every operating system.

- **hash** is a fingerprint of the exact file. If a download doesn't match it, uv refuses to install it, so nobody can slip in a tampered copy.

Both files go into version control. The payoff is on someone else's machine. Your copy of the finished package, `work/ch29/riverstone-report`, has no `.venv` yet, exactly like a colleague's fresh copy. Set it up from the lockfile:

```
# terminal, in work/ch29/riverstone-report
$ uv sync --locked
Using CPython 3.14.7
Creating virtual environment at: .venv
Resolved 58 packages in 1ms
...
Installed 53 packages in 102ms
+ ast-serialize==0.11.2
+ asttokens==3.0.2
+ certifi==2026.7.22
+ charset-normalizer==3.5.1
+ comm==0.2.3
...
$ uv run python -c "import pandas, openpyxl, requests; print(pandas.__version__, openpyxl.__version__,
requests.__version__)"
3.0.6 3.1.5 2.34.2
```

(Shortened: the full output names the project's folder and lists all 53 packages.)

- **--locked** means "install exactly what the lockfile says, and fail if `pyproject.toml` and `uv.lock` disagree". That's the command for servers and automated test runs.
- **uv run** runs any command inside the project's environment without activating it: `uv run pytest`, `uv run mypy`, `uv run riverstone-report --help`. **python -c "..."** runs the Python code in quotes and exits, which is handy for a one-line check.
- The numbers reconcile. The lockfile has 58 entries: your package, the 15 it needs, the dev tools and everything they need, plus 5 packages used only on Windows or macOS. On Linux, 53 of them are installed.
- **uv sync installs the dev group too**, which is why it installed 53 packages, not 16. On a server that only runs the report, use `uv sync --locked --no-dev`: it installs the 16.

If a system needs a `requirements.txt` (some older deployment tools do), `uv export --no-dev --no-hashes` produces one from the lockfile.

Task	venv + pip	uv
Create environment	<code>python -m venv .venv</code>	automatic on <code>uv sync</code> , <code>uv add</code> , or <code>uv run</code>
Add a package	<code>pip install pandas</code> , then update <code>requirements.txt</code> by hand	<code>uv add pandas</code>
Record exact versions	<code>pip freeze > requirements.txt</code>	<code>uv.lock</code> , updated automatically
Install on another machine	<code>pip install -r requirements.txt</code>	<code>uv sync --locked</code>
Install without dev tools	a separate requirements file, by hand	<code>uv sync --locked --no-dev</code>
Run a tool	activate, then <code>pytest</code>	<code>uv run pytest</code>

Task	venv + pip	uv
Upgrade one package	<code>pip install -U pandas, refreeze</code>	<code>uv lock --upgrade-package pandas,</code> then <code>uv sync</code>

Use the package from your notebook

Your notebook still runs on the book's `.venv`, which doesn't have this package. Switch it: in VS Code, click the kernel name at the top right of `ch29.ipynb`, choose **Select Another Kernel → Python Environments**, and pick the one in `riverstone-report/.venv`. That environment can run a notebook because `ipykernel` is in its dev group.

A new kernel starts empty, so run the cell with the five-line `lines` DataFrame from section 29.2 again. Then import from the package instead of defining the functions in cells:

```
from riverstone_report.transform import check_lines, revenue_by_category
print(revenue_by_category(lines))
```

```

   category  net_revenue  share_pct
0   Storage      16000.0        64.0
1  Furniture       5000.0        20.0
2   Kitchen       4000.0        16.0
```

The same table as in section 29.2, now from `src/riverstone_report/transform.py`. The import works because `uv sync` installed your package into its own environment in **editable** mode: Python reads the files in `src/` where they are, so an edit to a module is seen the next time it's imported. (A notebook imports a module only once, so after editing one, restart the kernel.)

Tool note. **Poetry** and **PDM** are older project managers that do similar jobs with their own lockfiles, and you'll meet them in existing company projects. **conda** manages non-Python dependencies too and is common in scientific computing. The ideas transfer: a project file for what you need, a lockfile for what you got, an isolated environment, and one command to reproduce it.

29.5 Classes, when they help

A **class** bundles data with the functions that work on it. Analysts coming from notebooks either never write one, or suddenly write classes for everything. The useful middle is small, and it starts with knowing what a class is. Chapter 17 met the word only in outputs such as `<class 'str'>`: every value in Python belongs to a class, and this section shows how to write your own.

Classes in twenty minutes

Run each cell in your notebook. A **class** is a template; an **instance** is one object made from it:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

box = Product("Storage Box 10L", 430.0)
print(box.name, box.price)
```

```
Storage Box 10L 430.0
```

- **class Product:** defines the template. By convention, class names start with a capital letter.
- **__init__** (two underscores each side) is the function Python runs when you create an instance. You never call it by name: `Product("Storage Box 10L", 430.0)` calls it for you.
- **self** is the particular object being created or used. Python passes it in as the first argument automatically, which is why `Product(...)` has two arguments and `__init__` has three.
- **self.name = name** stores a value on the object. A value stored this way is an **attribute**, read with a dot: `box.name`.

A function written inside a class is a **method**. It receives the object as `self`, so it can use the object's attributes:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def price_after(self, discount_pct):
        return round(self.price * (1 - discount_pct / 100), 2)

box = Product("Storage Box 10L", 430.0)
print(box.price_after(5))
```

```
408.5
```

`box.price_after(5)` passes `box` as `self` and `5` as `discount_pct`: ₹430 less 5% is ₹408.50. ✓

A line starting with `@` just above a `def` or a `class` is a **decorator**: it hands what follows to another function, which adds some behaviour to it. The first one you need is `@property`, which lets a method be read like an attribute, without brackets:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    @property
    def label(self):
        return f"{self.name} (₹{self.price:,.0f})"

box = Product("Storage Box 10L", 430.0)
print(box.label)
```

```
Storage Box 10L (₹430)
```

`box.label` is calculated each time it's read, from the current name and price, so it can never be out of date.

Writing `__init__` by hand gets tedious. A **data class** writes it for you, from a list of fields:

```
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float

box = Product("Storage Box 10L", 430.0)
print(box)
print(box == Product("Storage Box 10L", 430.0))
```

```
Product(name='Storage Box 10L', price=430.0)
True
```

- **@dataclass** reads the fields listed in the class and writes `__init__`, a readable print (the first line), and equality (the second: two products with the same fields are equal).
- **name: str** and **price: float** are the fields. The part after the colon is a **type hint**, which says what kind of value belongs there; a data class needs one for every field. Section 29.6 explains hints.

Add `frozen=True`, and nothing can change an instance after it's made. This cell fails on purpose:

```
@dataclass(frozen=True)
class Product:
    name: str
    price: float

box = Product("Storage Box 10L", 430.0)
box.price = 450.0
```

```
dataclasses.FrozenInstanceError: cannot assign to field 'price'
```

A frozen object is **immutable**. That's what you want for a settings bundle or a result: nothing can change it halfway through a run by accident.

One class can build on another. `class NoDataError(ReportError):` makes `NoDataError` a kind of `ReportError`, which is itself a kind of `Exception`; this is **inheritance**. It's how you make your own exceptions, and section 29.8 uses exactly these:

```
class ReportError(Exception):
    pass

class NoDataError(ReportError):
    pass

try:
    raise NoDataError("no sales lines for this month")
except ReportError as exc:
    print("caught:", exc)
    print(isinstance(exc, ReportError), isinstance(exc, NoDataError))
```

```
caught: no sales lines for this month
True True
```

- **pass** means "nothing more to add": these classes get everything from `Exception`.
- **except ReportError** caught a `NoDataError`, because a `NoDataError` is a `ReportError`.
isinstance(object, class) asks exactly that question.

Last, a second way to build an object. A method marked **@classmethod** receives the class itself, by convention named **cls**, instead of an instance, and is usually used as an **alternative constructor**: a named way to build the object from something else.

```
@dataclass(frozen=True)
class Product:
    name: str
    price: float

    @classmethod
    def from_text(cls, text):
        name, price = text.split(";")
        return cls(name, float(price))

print(Product.from_text("Garden Chair;1150"))
```

```
Product(name='Garden Chair', price=1150.0)
```

`Product.from_text(...)` is called on the class, not on an instance; `cls(name, float(price))` is the same as `Product(name, float(price))`. That's every piece of class syntax this chapter uses.

Data classes: named bundles of values

Now use them on the report. `summarize` returns a dictionary, and its `pct_of_target` has two problems. It's stored when the summary is made, so it can disagree with `revenue` if someone changes one and not the other. And a month with no target (`None`) or a zero target crashes the division. A frozen data class with a property fixes both:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class MonthSummary:
    revenue: float
    orders: int
    customers: int
    target: float | None

    @property
    def pct_of_target(self):
        if not self.target:
            return None
        return round(100 * self.revenue / self.target, 1)

def summarize(lines, target):
    check_lines(lines)
    return MonthSummary(
        revenue=round(float(lines["net_revenue"].sum()), 2),
        orders=int(lines["order_id"].nunique()),
        customers=int(lines["customer_name"].nunique()),
        target=target,
    )

summary = summarize(lines, 20000.0)
print(summary)
print(summary.pct_of_target)
print(summarize(lines, None).pct_of_target)
```

```
MonthSummary(revenue=25000.0, orders=3, customers=3, target=20000.0)
125.0
None
```

- `target: float | None` says the target is a number, or `None` when no target was set.
- `if not self.target:` is true for both `None` and `0`, so neither reaches the division; the property returns `None`, and the report can say "no target set".
- `MonthSummary(revenue=..., orders=..., ...)` passes each field by name, so the values can't arrive in the wrong order.

`MonthSummary` and this `summarize` replace the dictionary version in `transform.py`. Section 29.6 shows the whole file.

Watch out: don't store what you can calculate. If `MonthSummary` stored `pct_of_target` as a field, someone could change `revenue` and forget to update it. As a property, it's calculated from the current values every time. This is Chapter 28's normalization rule (store each fact once) applied to code.

The configuration as a data class

The report passes the same four settings everywhere. As loose variables, a function call looks like `run("2025-12", url, "reports", 5)`, and nothing stops the arguments arriving in the wrong order. `config.py` bundles them:

```
"""Settings for one report run, read from arguments and environment variables."""
import os
from collections.abc import Mapping
from dataclasses import dataclass
from datetime import date
from pathlib import Path

from riverstone_report.errors import ConfigError

def parse_month(text: str) -> date:
    """Turn '2025-12' into date(2025, 12, 1). Raise ConfigError for anything else."""
    try:
        year, month = (int(part) for part in text.split("-"))
        return date(year, month, 1)
    except ValueError:
        raise ConfigError(f"month must look like YYYY-MM, got {text!r}") from None

@dataclass(frozen=True)
class ReportConfig:
    month: date
    database_url: str
    output_dir: Path = Path("reports")
    top_n: int = 5

    @classmethod
    def from_env(cls, month: str, env: Mapping[str, str] = os.environ) -> ReportConfig:
        url = env.get("RIVERSTONE_DATABASE_URL")
        if not url:
            raise ConfigError("set RIVERSTONE_DATABASE_URL to the database connection URL")
        return cls(
            month=parse_month(month),
            database_url=url,
            output_dir=Path(env.get("RIVERSTONE_OUTPUT_DIR", "reports")),
        )

    @property
    def next_month(self) -> date:
        if self.month.month == 12:
            return date(self.month.year + 1, 1, 1)
        return date(self.month.year, self.month.month + 1, 1)

    @property
    def output_path(self) -> Path:
        return self.output_dir / f"riverstone_monthly_{self.month:%Y-%m}.xlsx"
```

How it works:

- `parse_month` splits "2025-12" at the - and turns each part into a whole number. Anything that doesn't give exactly two numbers making a real date raises a `ValueError` inside the `try`, and the `except` turns every such error into one kind, `ConfigError`, with a clear message. `from None` hides the less helpful internal `ValueError` from the traceback.
- `{text!r}` inside an f-string shows the value the way Python would type it, with quotes: 'Dec 2025', so an empty or space-padded input is visible in the message.
- `output_dir: Path = Path("reports")` and `top_n: int = 5` are fields with defaults, so they can be left out when a `ReportConfig` is made.
- `from_env` is an alternative constructor. The `env` parameter defaults to the real `os.environ`, but a test can pass a plain dictionary, as the next cell does, and no test has to change your real environment variables. `env.get(name)` returns `None` when the name is missing; `env.get(name, "reports")` returns the default instead.
- `next_month` and `output_path` are properties. December correctly rolls into January of the next year. `{self.month:%Y-%m}` formats the date inside the f-string with the date codes of Chapter 17 (2025-12), and `/` between a `Path` and text joins them into a longer path (Chapter 17, section 17.9). The file name always contains the month, so two runs for different months can't overwrite each other.
- The type hints (`text: str, -> date, Mapping[str, str]`) are explained in section 29.6.
-> `ReportConfig` on `from_env` names the class inside its own definition, which Python 3.14 allows.

Try it, one cell at a time. Build a configuration from a dictionary that stands in for the environment:

```
from riverstone_report.config import ReportConfig, parse_month
from riverstone_report.errors import ConfigError

env = {"RIVERSTONE_DATABASE_URL": "postgresql+psycopg://user:secret@db.example/riverstone"}
config = ReportConfig.from_env("2025-12", env=env)
print(config.month, config.next_month, config.output_path)
```

```
2025-12-01 2026-01-01 reports/riverstone_monthly_2025-12.xlsx
```

(On Windows the path prints as `reports\riverstone_monthly_2025-12.xlsx`.) A month in the wrong format:

```
try:
    parse_month("Dec 2025")
except ConfigError as exc:
    print("ConfigError:", exc)
```

```
ConfigError: month must look like YYYY-MM, got 'Dec 2025'
```

And the frozen data class at work. This cell fails on purpose:

```
config.month = parse_month("2026-01")
```

```
dataclasses.FrozenInstanceError: cannot assign to field 'month'
```

Once a run's configuration is built, nothing can change it halfway through.

A class with behavior: the API client

Section 29.9's `CrmClient` is a class for a different reason: it holds **state that several methods share** (the base URL, the API key, an open network session, retry settings) and it's created once and used many times. That's the test: **use a class when data and the functions that use it travel together**. For a calculation, a function is simpler. `transform.py` has no classes except the `MonthSummary` data class, and doesn't need any.

29.6 Type hints

A **type hint** declares what a variable, parameter, or return value is supposed to be: `def parse_month(text: str) -> date:` takes text and returns a date. Python **doesn't enforce** hints when the code runs; they're documentation that tools can check. Their value comes from a **type checker**, a program that reads your code without running it and reports places where the types can't work.

The hints this chapter uses:

Hint	Means
<code>int, float, str, bool, date</code>	that type
<code>list[str], dict[str, int], tuple[float, float]</code>	a container of that type
<code>float None</code>	a float, or None (older code writes <code>Optional[float]</code>)
<code>SomeClass None</code>	an object of that class, or None, such as <code>requests.Session None</code>
<code>pd.DataFrame</code>	a pandas DataFrame (checked with the <code>pandas-stubs</code> package)
<code>Mapping[str, str]</code>	anything dictionary-like that is only read: <code>os.environ</code> or a plain dict
<code>Iterator[X]</code>	something you loop over that hands out one X at a time (a generator, section 29.9)
<code>Callable[[float], None]</code>	a function taking a float and returning nothing (section 29.9)
<code>Any</code>	any type at all: the checker stops checking here
<code>-> None</code>	the function returns nothing

`Mapping`, `Iterator`, and `Callable` are imported from `collections.abc`, and `Any` from `typing`, both in the standard library.

Here's `transform.py` with its data class and its hints, the second of its three versions:

```
"""Pure calculations: DataFrames in, results out. No database, no files, so easy to test."""
from dataclasses import dataclass

import pandas as pd

REQUIRED_COLUMNS = {"order_id", "customer_name", "category", "net_revenue"}
```



```

@dataclass(frozen=True)
class MonthSummary:
    revenue: float
    orders: int
    customers: int
    target: float | None

    @property
    def pct_of_target(self) -> float | None:
        if not self.target:
            return None
        return round(100 * self.revenue / self.target, 1)

def check_lines(lines: pd.DataFrame) -> None:
    missing = REQUIRED_COLUMNS - set(lines.columns)
    if missing:
        raise ValueError(f"sales lines are missing columns: {sorted(missing)}")
    if lines.empty:
        raise ValueError("no sales lines for this month")

def summarize(lines: pd.DataFrame, target: float | None) -> MonthSummary:
    check_lines(lines)
    return MonthSummary(
        revenue=round(float(lines["net_revenue"].sum()), 2),
        orders=int(lines["order_id"].nunique()),
        customers=int(lines["customer_name"].nunique()),
        target=target,
    )

def revenue_by_category(lines: pd.DataFrame) -> pd.DataFrame:
    check_lines(lines)
    result = (lines.groupby("category", as_index=False)
              .agg(net_revenue=("net_revenue", "sum"))
              .sort_values(["net_revenue", "category"], ascending=[False, True],
                           ignore_index=True))
    total = result["net_revenue"].sum()
    result["share_pct"] = (100 * result["net_revenue"] / total).round(1)
    return result

def top_customers(lines: pd.DataFrame, n: int = 5) -> pd.DataFrame:
    if n < 1:
        raise ValueError("n must be at least 1")
    check_lines(lines)
    totals = (lines.groupby("customer_name", as_index=False)
              .agg(net_revenue=("net_revenue", "sum")))
    ranked = totals.sort_values(["net_revenue", "customer_name"], ascending=[False, True],
                                ignore_index=True)
    return ranked.head(n)

```

- **import pandas as pd** is back, because the hints name `pd.DataFrame`.
- **-> None** on `check_lines`: it returns nothing; it either passes quietly or raises.
- **-> MonthSummary** on `summarize`: the dictionary has become the data class.
- **n: int = 5**: a hint and a default together.

Why `.agg(net_revenue=("net_revenue", "sum"))` rather than a shorter spelling: the named aggregation always returns a `DataFrame`, which is what the hints promise, while some shorter pandas spellings return a `Series` in some cases, and the type checker would flag the mismatch.

Here's the bug a type checker is best at catching. Someone adds `headline.py`, a function to write a one-line headline for emails:

```
def headline(summary: MonthSummary) -> str:
    gap = summary.pct_of_target - 100
    return f"Revenue ₹{summary.revenue:,.0f}, {gap:+.1f} points against target"
```

`{gap:+.1f}` shows the number with a sign and one decimal place: `+15.7`. It works for December. Run **mypy**, the most widely used Python type checker, in strict mode (the `[tool.mypy]` settings in `pyproject.toml`):

```
# terminal, in work/ch29/riverstone-report
$ uv run mypy
src/riverstone_report/headline.py:6: error: Unsupported operand types for - ("None" and "int")
  [operator]
src/riverstone_report/headline.py:6: note: Left operand is of type "float | None"
Found 1 error in 1 file (checked 9 source files)
```

`pct_of_target` is `float | None`, because a month can have no target. For such a month the function would crash with a `TypeError`, on the day the email goes out. `mypy` found it without a database, a test, or a run. The fix handles the `None` case explicitly:

```
"""A one-line headline for emails (section 29.6)."""
from riverstone_report.transform import MonthSummary

def headline(summary: MonthSummary) -> str:
    if summary.pct_of_target is None:
        return f"Revenue ₹{summary.revenue:,.0f} (no target set)"
    gap = summary.pct_of_target - 100
    return f"Revenue ₹{summary.revenue:,.0f}, {gap:+.1f} points against target"
```

```
# terminal, in work/ch29/riverstone-report
$ uv run mypy
Success: no issues found in 9 source files
```

After the `is None` check returns, `mypy` knows that `pct_of_target` must be a float on the next line.

Tool note. Your editor uses the same hints: VS Code's Python extension underlines type errors as you type, and completes `summary.` with the right fields. Other checkers include **Pyright** (used by VS Code) and newer, faster ones appearing each year. Hints cost a few characters per function and pay back on every change, so add them to any code that more than one person maintains.

Checkpoint. In `work/ch29/riverstone-report`, `uv run mypy` says `Success`, and `uv run python -c "import riverstone_report"` prints nothing, which means the package imports cleanly.

Stop here: sitting 2 of 4. You can set up a project with uv, reproduce it from its lockfile, write a small class or data class, and let mypy check your hints.

29.7 Testing with pytest

A **test** is code that runs your code with known inputs and checks the result. **pytest** is the standard Python test runner: it finds files named `test_*.py`, runs every function named `test_*`, and reports what failed. Tests turn "I think it still works" into a command anyone can run. Chapter 17's exercise 28 wrote a few `assert` lines in a file; pytest is the same idea, organized.

A first test

In `work/ch29/riverstone-report`, create the file `tests/test_first.py` with one test in it:

```
import pandas as pd

from riverstone_report.transform import summarize

def test_orders_counted():
    lines = pd.DataFrame({
        "order_id":      [1, 1, 2, 3, 3],
        "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart",
                        "Green Leaf Hotels", "Green Leaf Hotels"],
        "category":      ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
        "net_revenue":    [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
    })
    summary = summarize(lines, 20000.0)
    assert summary.orders == 3
```

- A test is an ordinary function whose name starts with `test_`. It builds its own data, calls the code, and checks the answer.
- **assert condition** does nothing when the condition is true, and stops the test with a failure when it's false.
- The file imports `summarize` from the package. That works in a test for the same reason it worked in your notebook: `uv sync` installed the package into `.venv` in editable mode.

Run it, asking for one line per test with `-v` (verbose):

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest tests/test_first.py -v
===== test session starts =====
...
cachedir: .pytest_cache
...
configfile: pyproject.toml
plugins: platformdirs-4.12.1
collecting ... collected 1 item

tests/test_first.py::test_orders_counted PASSED [100%]

===== 1 passed in 0.18s =====
```

(Shortened: the two lines left out show folders on your computer.) Read the header once:

- The first line left out starts with **platform**: your operating system and the versions of Python, pytest, and pluggy (a part of pytest), and, with **-v**, the full path of the Python it ran. The next run shows it without the path.
- **cachedir** is where pytest remembers the last run. The second line left out is **rootdir**, the project folder.
- **configfile**: **pyproject.toml** says pytest read its settings from there; **plugins** lists extras that other installed packages add to pytest, which you can ignore.
- **collected 1 item**: pytest found one test.

Then the result. **tests/test_first.py::test_orders_counted** is the test's **node ID**: the file, two colons, the function. Any node ID can be run on its own: `uv run pytest tests/test_first.py::test_orders_counted`.

A failing test

Before you run it, predict what pytest will print if the test expects the wrong number. Change the last line to `assert summary.orders == 5` and run the file again, without **-v**:

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest tests/test_first.py
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
...
configfile: pyproject.toml
plugins: platformdirs-4.12.1
collected 1 item

tests/test_first.py F [100%]

===== FAILURES =====
_____ test_orders_counted _____

    def test_orders_counted():
        lines = pd.DataFrame({
            "order_id": [1, 1, 2, 3, 3],
            "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart",
                           "Green Leaf Hotels", "Green Leaf Hotels"],
            "category": ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
            "net_revenue": [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
        })
        summary = summarize(lines, 20000.0)
> assert summary.orders == 5
E       assert 3 == 5
E       + where 3 = MonthSummary(revenue=25000.0, orders=3, customers=3, target=20000.0).orders

tests/test_first.py:15: AssertionError
===== short test summary info =====
FAILED tests/test_first.py::test_orders_counted - assert 3 == 5
===== 1 failed in 0.23s =====
```

Read it from the top:

- Without `-v`, each test is one character: **F** for failed (a pass is a dot).
- Under **FAILURES**, pytest prints the failing test's code, with `>` on the line that failed.
- The **E** lines are the explanation: `assert 3 == 5` is the comparison with the real values filled in, and `where 3 = MonthSummary(...).orders` says where the 3 came from.
- **tests/test_first.py:15** is the file and line number.
- The **short test summary** repeats every failure in one line, which is what you read first when there are many.

For a shorter report, add `--tb=short` (the traceback, shortened to the failing line and the **E** lines) and `-q` (quiet: no header). You'll use both below. Change the line back to `== 3` before going on.

Fixtures: shared test data

A second test needs the same five lines. Instead of copying them, move them into a **fixture**: a function marked `@pytest.fixture` that builds test data. A test asks for a fixture by using its name as a parameter, and pytest calls the fixture and passes in the result. Replace `tests/test_first.py` with:

```
import pandas as pd
import pytest

from riverstone_report.transform import summarize

@pytest.fixture
def lines():
    return pd.DataFrame({
        "order_id":      [1, 1, 2, 3, 3],
        "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart",
                        "Green Leaf Hotels", "Green Leaf Hotels"],
        "category":      ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
        "net_revenue":    [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
    })

def test_orders_counted(lines):
    assert summarize(lines, 20000.0).orders == 3

def test_revenue_added_up(lines):
    assert summarize(lines, 20000.0).revenue == 25000.0
```

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest tests/test_first.py -v
...
collecting ... collected 2 items

tests/test_first.py::test_orders_counted PASSED [ 50%]
tests/test_first.py::test_revenue_added_up PASSED [100%]

===== 2 passed in 0.22s =====
```

pytest calls `lines()` afresh for each test, so one test can't spoil the data for the next. A fixture that several test files need goes in `tests/conftest.py`, a file pytest loads automatically; its fixtures are available to every test file without an import. The finished package's `conftest.py` has exactly this `lines` fixture, with a docstring: "Five order lines from three orders: small enough to check every number by hand." And its `tests/test_transform.py` already has these two checks, in a test named `test_summarize_counts_orders_not_lines`. So delete `tests/test_first.py`: it has done its job.

Testing many cases and testing failures

`tests/test_config.py` tests `config.py` with two more tools:

```
from datetime import date
from pathlib import Path

import pytest

from riverstone_report.config import ReportConfig, parse_month
from riverstone_report.errors import ConfigError

@pytest.mark.parametrize("text, expected", [
    ("2025-12", date(2025, 12, 1)),
    ("2026-01", date(2026, 1, 1))])
def test_parse_month(text, expected):
    assert parse_month(text) == expected

@pytest.mark.parametrize("bad", ["Dec 2025", "2025-13", "2025", ""])
def test_parse_month_rejects_bad_input(bad):
    with pytest.raises(ConfigError):
        parse_month(bad)

def test_from_env_needs_database_url():
    with pytest.raises(ConfigError, match="RIVERSTONE_DATABASE_URL"):
        ReportConfig.from_env("2025-12", env={})

def test_december_rolls_into_next_year():
    env = {"RIVERSTONE_DATABASE_URL": "postgresql://example"}
    config = ReportConfig.from_env("2025-12", env=env)
    assert config.next_month == date(2026, 1, 1)
    assert config.output_path == Path("reports/riverstone_monthly_2025-12.xlsx")
```

- `@pytest.mark.parametrize("text, expected", [...])` runs one test function once per item in the list, passing each item's values in as `text` and `expected`. Four bad month formats are four separate test results.
- `with pytest.raises(ConfigError):` passes only if the code inside raises `ConfigError`. Testing that `parse_month("2025-13")` fails *the right way* is as important as testing that "2025-12" works, because the original script's bugs were all on the failure path.
- `match="RIVERSTONE_DATABASE_URL"` also checks that the error message contains that text, so the message stays helpful.
- `env={}` tests a missing environment variable without touching the real environment.

Run the file with `-v` to see every case:

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest tests/test_config.py -v
...
collecting ... collected 8 items

tests/test_config.py::test_parse_month[2025-12-expected0] PASSED [ 12%]
tests/test_config.py::test_parse_month[2026-01-expected1] PASSED [ 25%]
tests/test_config.py::test_parse_month_rejects_bad_input[Dec 2025] PASSED [ 37%]
tests/test_config.py::test_parse_month_rejects_bad_input[2025-13] PASSED [ 50%]
tests/test_config.py::test_parse_month_rejects_bad_input[2025] PASSED [ 62%]
tests/test_config.py::test_parse_month_rejects_bad_input[] PASSED [ 75%]
tests/test_config.py::test_from_env_needs_database_url PASSED [ 87%]
tests/test_config.py::test_december_rolls_into_next_year PASSED [100%]

===== 8 passed in 0.30s =====
```

Each parametrized case gets its own node ID, with the values in square brackets: `[Dec 2025]` is the bad input. A value pytest can't write simply, such as a date, becomes the parameter's name and a number: `expected0`, `expected1`. The empty string shows as `[]`. If one case fails, pytest names the input that failed.

The integration test: reconcile to a known number

Unit tests use made-up data. One test should prove the whole path works against the real database, and the best check is one this book already trusts: December 2025's net revenue of ₹4,39,823.50, 115.7% of its ₹3,80,000 target (Chapter 28's answer to exercise 11). That test is `tests/test_reconcile.py`:

```
"""Integration test: runs only when RIVERSTONE_DATABASE_URL points at riverstone_2025."""
import os

import pytest
from sqlalchemy import create_engine

from riverstone_report import db, transform
from riverstone_report.config import ReportConfig

pytestmark = pytest.mark.skipif(not os.environ.get("RIVERSTONE_DATABASE_URL"),
                                reason="no database configured")

def test_december_2025_matches_the_book():
    config = ReportConfig.from_env("2025-12")
    engine = create_engine(config.database_url)
    lines = db.fetch_sales_lines(engine, config.month, config.next_month)
    summary = transform.summarize(lines, db.fetch_target(engine, config.month))
    assert summary.revenue == 439823.50
    assert summary.pct_of_target == 115.7
    categories = transform.revenue_by_category(lines)
    assert categories["net_revenue"].sum() == pytest.approx(summary.revenue)
```

- `pytestmark = pytest.mark.skipif(condition, reason=...)` applies to every test in the file: if the condition is true (no database URL is set), pytest skips them instead of running them. The fast unit tests still run anywhere, on a colleague's laptop or in the automated check below.

- `pytest.approx(summary.revenue)`: decimal numbers that should be equal can differ in the fifteenth digit after a sum (Chapter 17, section 17.3), so `approx` allows a tiny tolerance. The categories' total must equal the summary's revenue, to within a whisker.

Running the tests

Run the whole suite with `-v`, reading the database settings from `.env`, and the output lists every test by name, which is a fair description of what the package promises:

```
# terminal, in work/ch29/riverstone-report
$ uv run --env-file .env pytest -v
...
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.1
collecting ... collected 23 items

tests/test_config.py::test_parse_month[2025-12-expected0] PASSED [ 4%]
tests/test_config.py::test_parse_month[2026-01-expected1] PASSED [ 8%]
tests/test_config.py::test_parse_month_rejects_bad_input[Dec 2025] PASSED [ 13%]
tests/test_config.py::test_parse_month_rejects_bad_input[2025-13] PASSED [ 17%]
tests/test_config.py::test_parse_month_rejects_bad_input[2025] PASSED [ 21%]
tests/test_config.py::test_parse_month_rejects_bad_input[] PASSED [ 26%]
tests/test_config.py::test_from_env_needs_database_url PASSED [ 30%]
tests/test_config.py::test_december_rolls_into_next_year PASSED [ 34%]
tests/test_crm_client.py::test_follows_pages_until_next_page_is_none PASSED [ 39%]
tests/test_crm_client.py::test_retries_server_errors_then_succeeds PASSED [ 43%]
tests/test_crm_client.py::test_honors_retry_after_on_429 PASSED [ 47%]
tests/test_crm_client.py::test_does_not_retry_a_bad_api_key PASSED [ 52%]
tests/test_crm_client.py::test_gives_up_after_max_attempts PASSED [ 56%]
tests/test_reconcile.py::test_december_2025_matches_the_book PASSED [ 60%]
tests/test_transform.py::test_summarize_counts_orders_not_lines PASSED [ 65%]
tests/test_transform.py::test_no_target_gives_no_percentage PASSED [ 69%]
tests/test_transform.py::test_categories_are_sorted_and_shares_add_up PASSED [ 73%]
tests/test_transform.py::test_top_customers[1-expected0] PASSED [ 78%]
tests/test_transform.py::test_top_customers[2-expected1] PASSED [ 82%]
tests/test_transform.py::test_top_customers_rejects_zero PASSED [ 86%]
tests/test_transform.py::test_empty_month_raises_no_data PASSED [ 91%]
tests/test_transform.py::test_missing_column_is_reported PASSED [ 95%]
tests/test_transform.py::test_zero_target_gives_no_percentage PASSED [100%]

===== 23 passed in 0.36s =====
```

`testpaths: tests` is the setting from `pyproject.toml`: with no file named, `pytest` looked in `tests`. The client tests belong to section 29.9. Now run it the way a colleague without the database would, without `.env` and without `-v`:

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
...
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.1
collected 23 items

tests/test_config.py ..... [ 34%]
tests/test_crm_client.py ..... [ 56%]
tests/test_reconcile.py s [ 60%]
tests/test_transform.py ..... [100%]

===== 22 passed, 1 skipped in 0.26s =====
```

Each dot is a passing test and the `s` is the skipped integration test. 23 tests in under half a second, so there's no reason not to run them after every change.

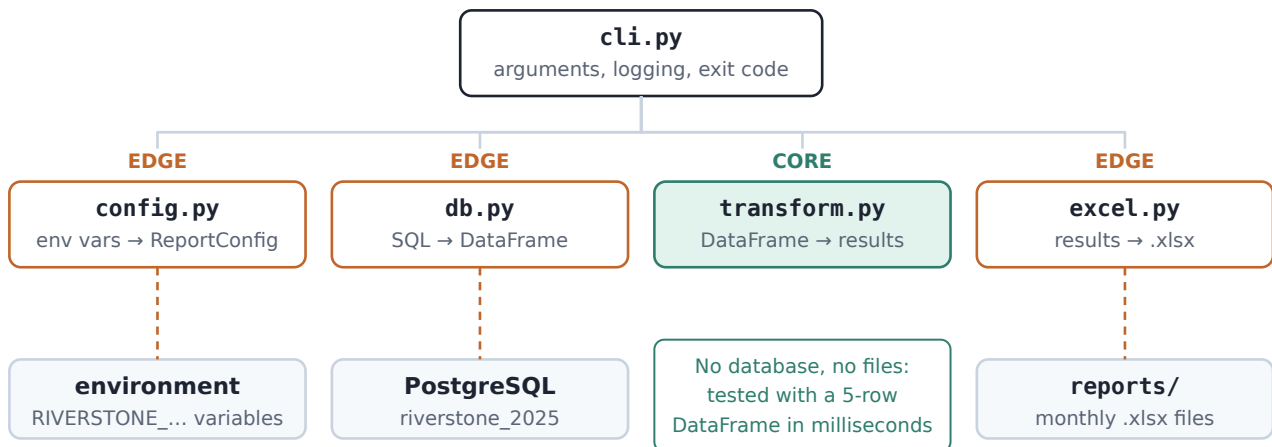
What a failure looks like

Suppose someone "simplifies" `summarize` and writes `orders=len(lines)`, counting lines instead of orders:

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest -q --tb=short tests/test_transform.py
F..... [100%]
===== FAILURES =====
_____ test_summarize_counts_orders_not_lines _____
tests/test_transform.py:11: in test_summarize_counts_orders_not_lines
    assert summary.orders == 3
E   assert 5 == 3
E   + where 5 = MonthSummary(revenue=25000.0, orders=5, customers=3, target=20000.0).orders
===== short test summary info =====
FAILED tests/test_transform.py::test_summarize_counts_orders_not_lines - asse...
1 failed, 8 passed in 0.24s
```

The test's name says what rule was broken, and the output shows the wrong value (5) and where it came from. This is the fan-out mistake from Chapter 12, caught in a quarter of a second instead of in a board meeting.

Keep the calculations pure: input and output at the edges

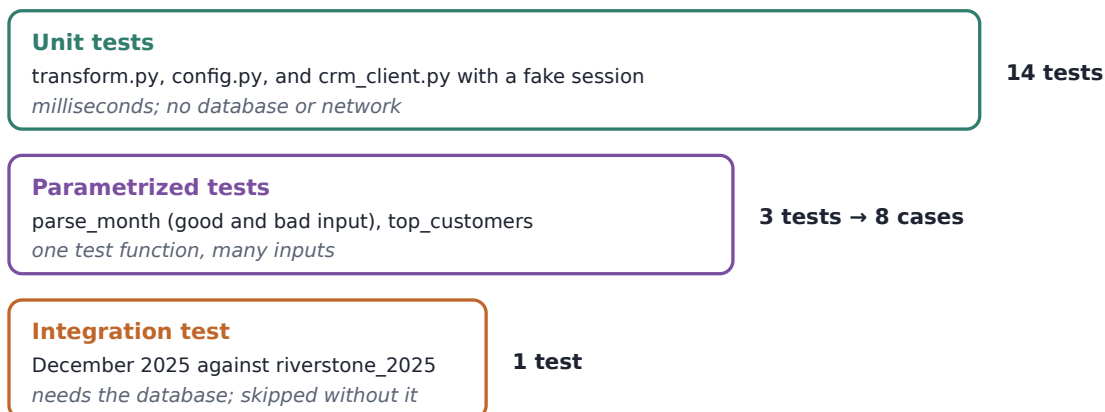


EDGE modules (orange) touch the outside world and get a few integration tests.

The CORE (green) holds the business rules and gets many fast unit tests.

Figure 29.2 — The design that makes testing cheap: business rules in pure functions, input and output at the edges.

What each kind of test is for



The integration test reconciles to a number the book already trusts:

December 2025 net revenue ₹4,39,823.50, 115.7% of target.

Figure 29.3 — Many fast tests on the pure core, a few slow ones on the edges.

What to test

- **Business rules:** every calculation someone would argue about (what counts as an order, how shares round, what happens with no target).
- **Edge cases:** empty input, a single row, ties, the last month of the year, None.
- **Failures:** bad input raises the right exception with a helpful message.
- **One reconciliation** against a known total, run when the database is available.
- **Not** third-party libraries (pandas' `groupby` is tested by pandas), and not every line for its own sake. A number called "coverage" measures which lines tests ran (pytest-cov reports it); treat it as a way to find untested code, not as a target.

Where this leads. Chapter 47 applies the same idea to data itself: automated checks that run against every day's data, not only against your code.

Run the tests on every push

Chapter 26, section 26.8, set up an automated check that runs on every push and every pull request. Now the project has tests, a lockfile, and a type checker, so the check can run all three. The workflow file goes at the top of the repository that holds the package, as `.github/workflows/checks.yml`; for a package in a repository of its own, as `uv init` makes it (section 29.4), that's the package folder:

```
name: checks
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: astral-sh/setup-uv@v10.2.0
      - run: uv sync --locked
      - run: uv run pytest -q
      - run: uv run mypy
```

What changed from Chapter 26's version (the job name, `check`, and the `checkout` line stay the same):

- **uses: `astral-sh/setup-uv@v10.2.0`** installs `uv` on the fresh machine. It replaces `actions/setup-python`: `uv` reads `.python-version` and fetches that Python itself. The action publishes no short `@v10` tag, so the full version is pinned instead of the major version Chapter 26 recommends: a stricter pin, updated the same way. (10.2.0 was the current version on 28 September 2026; check the action's page for newer ones.)
- **run: `uv sync --locked`** replaces `pip install -r requirements.txt`: the machine gets exactly the versions in `uv.lock`, and the check fails if `pyproject.toml` and the lockfile disagree.
- **run: `uv run pytest -q`** runs the tests quietly. The machine has no database and no `.env`, so the integration test shows as skipped, and 22 tests must pass.
- **run: `uv run mypy`** fails the check if a type error comes in. With the `pytest` step, it replaces Chapter 26's `compileall` and `--help` steps.

If any step exits with a non-zero code, the check fails and the pull request shows a red cross, as in Chapter 26. These are the same three commands you can run yourself before pushing:

```
# terminal, in work/ch29/riverstone-report
$ uv run pytest -q
.....S..... [100%]
22 passed, 1 skipped in 0.27s
$ uv run mypy
Success: no issues found in 9 source files
```

Push a change that breaks `summarize`, like the `len(lines)` one above, and the check's `pytest` step fails with the same report as on your machine; the pull request shows a red cross until it's fixed.

29.8 Errors and logging

Fail loudly, with a reason

The script's worst line was `except: target = 0`. It caught *every* possible error (a typo in the SQL, a lost connection, even the user pressing Ctrl+C), replaced it with a plausible-looking wrong value, and carried on. The rules the package follows instead:

1. **Catch only exceptions you can do something about**, by name: `except ConfigError:`, never a bare `except:`.
2. **Don't turn an error into a default value** unless the default is truly correct. A missing target isn't zero; it's `None`, and the report says "no target set".
3. **Define your own exceptions** for failures the business cares about, so callers can handle them as a group.
4. **Stop at the top**: one place (the command-line entry point) catches `ReportError`, logs it, and exits with a non-zero **exit code** (Chapter 17, section 17.12, and Chapter 34). Schedulers, automated checks, and shell scripts read it.

The package's own exceptions are in `errors.py`, built by inheritance exactly as in section 29.5, with a docstring in place of `pass`:

```
"""Exceptions the report can raise. Callers catch ReportError to handle any of them."""

class ReportError(Exception):
    """Base class for every expected failure of the monthly report."""

class ConfigError(ReportError):
    """A setting is missing or invalid."""

class NoDataError(ReportError):
    """The requested month has no sales, so a report would be misleading."""

class CrmApiError(ReportError):
    """The CRM API kept failing after all retries, or refused the request."""
```

Because `NoDataError` and `ConfigError` both inherit from `ReportError`, except `ReportError` handles all of them, while a genuine bug (a `KeyError`, say) is not caught and crashes with a full traceback, which is what you want for a bug.

That gives `transform.py` its third and final version. An empty month is an expected failure, not a bug, so `check_lines` raises `NoDataError` for it instead of `ValueError`; a missing column stays a `ValueError`, because it means the code or the query is wrong. Two lines change:

```
from riverstone_report.errors import NoDataError      # added below "import pandas as pd"

    if lines.empty:
        raise NoDataError("no sales lines for this month")    # was ValueError
```

Try it in the notebook, with the package's `summarize`:

```
from riverstone_report.errors import NoDataError, ReportError
from riverstone_report.transform import summarize

try:
    summarize(lines.iloc[0:0], target=380000.0)
except ReportError as exc:
    print(type(exc).__name__, "is a ReportError:", exc)
    print(isinstance(exc, NoDataError))
```

```
NoDataError is a ReportError: no sales lines for this month
True
```

`lines.iloc[0:0]` is section 29.2's empty month. `type(exc).__name__` is the name of the exception's class, so the message says which kind of `ReportError` arrived.

Logging, one logger per module

Chapter 18 (section 18.15) and Chapter 20 (section 20.11) replaced `print` with **logging**: messages with a **level**, a timestamp, and a logger's name, configured once by `logging.basicConfig`, with `%s` placeholders instead of f-strings. The five levels, with an example of each from the report:

Level	Use for	Example from the report
DEBUG	detail you only want while investigating	the SQL parameters used
INFO	normal progress worth a record	"read 35 sales lines for 2025-12"
WARNING	something odd that was handled	an API call failed and will be retried
ERROR	this run failed	"no sales lines for this month"
CRITICAL	the whole system is in trouble	rarely used in reports

A package adds one habit. Each module asks for its own logger with `logging.getLogger(__name__)`. Inside a module, `__name__` is the module's full name (section 29.2), so the logger in `cli.py` is called `riverstone_report.cli` and the one in `crm_client.py` is `riverstone_report.crm_client`, and every log line says which file wrote it. Only the program's entry point decides where messages go and which levels to show.

The write job

The fourth job, writing the workbook, is `excel.py`. It's Chapter 18's formatting (section 18.12), in one function:

```
"""Writing the formatted workbook."""
from pathlib import Path

import pandas as pd
from openpyxl.styles import Font

from riverstone_report.transform import MonthSummary

RUPEES = '"₹"#,##0'
```

```
def write_report(summary: MonthSummary, categories: pd.DataFrame,
                customers: pd.DataFrame, path: Path) -> Path:
    path.parent.mkdir(parents=True, exist_ok=True)
    headline = pd.DataFrame({
        "metric": ["Net revenue", "Orders", "Customers", "Target", "% of target"],
        "value": [summary.revenue, summary.orders, summary.customers,
                  summary.target, summary.pct_of_target],
    })
    sheets = [("Summary", headline), ("Categories", categories), ("Top customers", customers)]
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        for sheet, frame in sheets:
            frame.to_excel(writer, sheet_name=sheet, index=False)
            ws = writer.sheets[sheet]
            for cell in ws[1]:
                cell.font = Font(bold=True)
            ws.column_dimensions["A"].width = 26
            ws.column_dimensions["B"].width = 16
        for row in (2, 5):
            writer.sheets["Summary"].cell(row=row, column=2).number_format = RUPEES
        for sheet in ("Categories", "Top customers"):
            for cell in writer.sheets[sheet]["B"][1:]:
                cell.number_format = RUPEES
    return path
```

- **path.parent.mkdir(parents=True, exist_ok=True)** makes the output folder, and any missing folders above it, if they aren't there yet.
- **headline** is the Summary sheet as a two-column table, built from the **MonthSummary**; a month with no target leaves the last two cells empty.
- **pd.ExcelWriter(path, engine="openpyxl")** opens one workbook for all three sheets; **writer.sheets[sheet]** is the openpyxl worksheet just written, which is where the formatting goes.
- **ws[1]** is the first row: the headers, made bold. **column_dimensions["A"].width** widens columns A and B.
- **RUPEES = '₹#,##0'** is an Excel number format: a ₹ sign and thousands separators, no decimals. It's applied to the revenue and target cells (rows 2 and 5 of the Summary) and to column B of the other two sheets, skipping the header ([1:]).
- It returns the path it wrote, so the caller can log it.

The entry point

cli.py puts the jobs together and is the one place that catches **ReportError**:

```
"""Command-line entry point: riverstone-report --month 2025-12"""
import argparse
import logging
import sys

from sqlalchemy import create_engine

from riverstone_report import db, excel, transform
from riverstone_report.config import ReportConfig
from riverstone_report.errors import ReportError

log = logging.getLogger(__name__)

def run(config: ReportConfig) -> None:
    engine = create_engine(config.database_url)
```



```

lines = db.fetch_sales_lines(engine, config.month, config.next_month)
log.info("read %d sales lines for %s", len(lines), f"{config.month:%Y-%m}")
summary = transform.summarize(lines, db.fetch_target(engine, config.month))
path = excel.write_report(summary, transform.revenue_by_category(lines),
                        transform.top_customers(lines, config.top_n), config.output_path)
log.info("net revenue %.2f from %d orders; wrote %s",
        summary.revenue, summary.orders, path)

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="riverstone-report",
                                    description="Build Riverstone's monthly sales report.")
    parser.add_argument("--month", required=True, help="the month to report, as YYYY-MM")
    parser.add_argument("--verbose", action="store_true", help="show debug messages")
    args = parser.parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(name)s: %(message)s")
    try:
        run(ReportConfig.from_env(args.month))
    except ReportError as exc:
        log.error("%s", exc)
        return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

How it works:

- **run(config)** is the four jobs in order: connect, read, transform, write. It knows nothing about the command line, so a test or another program can call it with any `ReportConfig`.
- **argparse.ArgumentParser(prog=..., description=...)**, from Chapter 18, section 18.15: `prog` is the command's name and `description` one line about it, both printed by `--help`.
- **add_argument("--month", required=True, ...)**: an option written with two dashes is normally optional; `required=True` makes `argparse` refuse to run without it.
- **add_argument("--verbose", action="store_true", ...)** is a switch with no value, as in Chapter 20: `True` if typed, `False` if not. Here it turns on `DEBUG` messages.
- **parse_args(argv)**: when `argv` is `None`, the default, `argparse` reads the real command line. A list lets a test call `main(["--month", "2025-12"])` without a terminal.
- **format="%(asctime)s %(levelname)s %(name)s: %(message)s"** puts the time, the level, the logger's name, and the message on each line.
- **log.info("read %d sales lines for %s", len(lines), ...)** fills the placeholders from the values after the message. Three placeholders appear in this file:

Placeholder	Shows
%d	a whole number: 35
%s	anything, as text: 2025-12
%.2f	a number with two decimal places: 439823.50

- **except ReportError** is the one catch, at the top: it logs the reason and returns 1. Anything else is a bug, and crashes with its traceback.

- `sys.exit(main())` is the same as Chapter 17's `raise SystemExit(main())`: it ends the program with `main`'s return value as the exit code.

`--help` shows what `argparse` built from those lines:

```
# terminal, in work/ch29/riverstone-report
$ uv run riverstone-report --help
usage: riverstone-report [-h] --month MONTH [--verbose]

Build Riverstone's monthly sales report.

options:
  -h, --help            show this help message and exit
  --month MONTH         the month to report, as YYYY-MM
  --verbose             show debug messages
$ uv run riverstone-report
usage: riverstone-report [-h] --month MONTH [--verbose]
riverstone-report: error: the following arguments are required: --month
$ echo "exit code: $?"
exit code: 2
```

Without `--month`, `argparse` stops before any of the report's code runs, and uses exit code 2, its code for "the command was typed wrongly".

The command now behaves like software a scheduler can trust. Run it for December, January, and a mistyped month, printing the exit code after each:

```
# terminal, in work/ch29/riverstone-report
$ uv run --env-file .env riverstone-report --month 2025-12
2026-09-28 19:11:00,876 INFO riverstone_report.cli: read 35 sales lines for 2025-12
2026-09-28 19:11:00,897 INFO riverstone_report.cli: net revenue 439823.50 from 18 orders; wrote
    reports/riverstone_monthly_2025-12.xlsx
$ echo "exit code: $?"
exit code: 0
$ uv run --env-file .env riverstone-report --month 2026-01
2026-09-28 19:11:01,656 INFO riverstone_report.cli: read 0 sales lines for 2026-01
2026-09-28 19:11:01,657 ERROR riverstone_report.cli: no sales lines for this month
$ echo "exit code: $?"
exit code: 1
$ uv run --env-file .env riverstone-report --month Dec-2025
2026-09-28 19:11:02,298 ERROR riverstone_report.cli: month must look like YYYY-MM, got 'Dec-2025'
$ echo "exit code: $?"
exit code: 1
```

January now fails with a reason and exit code 1, and writes nothing, so December's report is safe. The workbook for December is in `reports`; read it back in the notebook:

```
import pandas as pd

report = pd.read_excel("riverstone-report/reports/riverstone_monthly_2025-12.xlsx",
    sheet_name=None)
for sheet, frame in report.items():
    print(sheet)
    print(frame, end="\n\n")
```

```

Summary
      metric      value
0 Net revenue 439823.5
1      Orders      18.0
2    Customers      15.0
3      Target 380000.0
4 % of target    115.7

Categories
      category  net_revenue  share_pct
0      Storage    220363.5      50.1
1      Kitchen    142180.0      32.3
2 Industrial     77280.0      17.6

Top customers
      customer_name  net_revenue
0 Northgate Distributors    75626.0
1      Prime Wholesale    65521.5
2      Sea Breeze Hotel    60675.0
3      Royal Banquets    38050.0
4      Deccan Packaging    37628.0

```

`sheet_name=None` reads every sheet, as a dictionary from sheet name to DataFrame (Chapter 18 read one sheet at a time). ₹4,39,823.50 from 18 orders and 15 customers, 115.7% of target, and the three categories add up to the total. ✓

To run it every month, schedule it the way Chapter 20, section 20.7, scheduled the Flash. A crontab line for 07:00 on the 2nd of each month, by the server's clock, reporting on the month before:

```

0 7 2 * * cd /home/meera/riverstone-report && /home/meera/.local/bin/uv run --env-file .env riverstone-report -e
-month "$(date -d 'last month' +%Y-%m)" >> cron.log 2>&1

```

- `$(date -d 'last month' +%Y-%m)` is Chapter 34's command substitution: on 2 February it becomes `2026-01`. The `%` signs are written `\%` because cron treats a bare `%` as a line break (Chapter 20).
- `/home/meera/.local/bin/uv` is `uv` by its full path, because cron's `PATH` is short (Chapter 34, section 34.6); which `uv` prints yours.
- When the month has no data, the command exits with 1, and the failure alert of Chapter 20, section 20.11, fires.

Watch out: never log secrets. `log.info("connecting to %s", config.database_url)` writes the database password into a log file that many more people can read than the password itself. Log *which* database (`db.example/riverstone`), not the full connection string, and never log API keys or personal data.

Stop here: sitting 3 of 4. You can write and run tests, read a failure, test the failure paths, and build a command that logs, fails loudly, and reports its result as an exit code.

29.9 Robust API clients

Chapter 18, section 18.14, called an API: it checked status codes, set a timeout, followed pages, and retried 429 and 5xx answers after a growing wait. Real APIs used by scheduled code need four more things:

1. **Pagination that can't lose a page.** An API returns large lists in **pages** (10, 100, or 1,000 items at a time) and tells you how to ask for the next one. A failure on page 3 must stop the run or be retried, never skip the page.
2. **Transient failures beyond status codes.** Servers restart, networks drop, gateways time out. The same request a second later usually works, even when no status code came back at all. These are errors worth **retrying**.
3. **Rate limits.** APIs cap how many requests you may make. Exceed it and you get **HTTP 429 Too Many Requests**, often with a **Retry-After** header saying how many seconds to wait.
4. **Retries you can test**, without waiting for real seconds to pass.

The practice API

Riverstone's CRM (Chapter 3) isn't connected to anything you can reach, so the companion folder has a local stand-in, `mock_crm_api.py`. It serves the 43 leads from `riverstone_2025` at `GET /leads?page=N&page_size=M`, requires the header `X-API-Key: demo-key`, and misbehaves on purpose: the first request for page 2 gets **503 Service Unavailable**, and the first request for page 3 gets **429** with `Retry-After: 1`. It runs on your own machine and sends nothing anywhere.

A response looks like this:

```
{ "page": 5, "items": [{"lead_id": 41, "created_at": "2025-01-06T09:35:00", "company_name": "Crown Hotels", "source": "Referral"}], "next_page": null, "total": 43}
```

(One item shown; page 5 has three.)

Your notebook runs in `work/ch29`, the folder that holds `mock_crm_api.py`, so it can import it.

`start_server(8029)` starts the mock API in the background of the notebook, listening at `http://127.0.0.1:8029` (127.0.0.1 always means "this computer", Chapter 34, section 34.9; 8029 is the port). It returns the server object and its address. `server.shutdown()` stops it and frees the port. If you ever see *Address already in use*, a server from an earlier cell is still running: restart the kernel.

The naive client

Here's how most first API calls are written:

```
import requests
from mock_crm_api import start_server

server, base_url = start_server(8029)

leads, page = [], 1
while page is not None:
    response = requests.get(f"{base_url}/leads", params={"page": page, "page_size": 10},
                            headers={"X-API-Key": "demo-key"})
    data = response.json()
    leads.extend(data["items"])
    page = data["next_page"]
print(len(leads))
```

```
KeyError: 'items'
```

In the notebook the error arrives as a traceback (Chapter 17, section 17.10). Its last lines are:

```
---> 11     leads.extend(data["items"])
      12     page = data["next_page"]
      13 print(len(leads))
```

```
KeyError: 'items'
```

Page 2 returned a 503 whose body is `{"error": "temporarily unavailable"}`, the code never checked the status, and it crashed looking for a key that isn't there. Worse variants don't crash: they skip the page and report 33 leads as if that were all of them. And this call has **no timeout**: `requests` waits forever by default, so if the server stops responding, a scheduled job hangs until someone notices.

The robust client is built in five small steps below, each run against the mock API, and then gathered into one class.

Step 1: one page, checked

Before you run it, predict which of the two `get_page` calls fails.

```
from riverstone_report.errors import CrmApiError

def get_page(session, url, page):
    response = session.get(url, params={"page": page, "page_size": 10}, timeout=(3.05, 10.0))
    if response.status_code != 200:
        raise CrmApiError(f"{url} returned {response.status_code}")
    return response.json()

server.shutdown()
server, base_url = start_server(8029)          # a fresh server, with its planned failures reset
session = requests.Session()
session.headers["X-API-Key"] = "demo-key"

print(len(get_page(session, f"{base_url}/leads", 1)["items"]), "leads on page 1")
get_page(session, f"{base_url}/leads", 2)
```

```
10 leads on page 1
riverstone_report.errors.CrmApiError: http://127.0.0.1:8029/leads returned 503
```

- `requests.Session()` is an object that keeps one network connection open across requests and remembers settings. `session.headers["X-API-Key"] = "demo-key"` stores the key once; every request through the session sends it.
- `params={"page": page, "page_size": 10}` becomes the part of the address after `?`, as in Chapter 18.
- `timeout=(3.05, 10.0)`: give up if connecting takes more than about 3 seconds, or if the server goes silent for 10 once connected. Every network call in production code needs a timeout (Chapter 18).
- `CrmApiError` is the package's own exception (section 29.8), so the caller can catch it with the report's other failures.

Step 2: retry, with try, except, and else

A request can fail in two different ways: the server answers with an error status, or no answer comes at all and `requests` raises an exception (`ConnectionError`, `Timeout`). Handling both neatly needs one more part of `try`, which Chapter 17 mentioned: `else` runs only if nothing in the `try` raised. A toy first:

```
for text in ["430", "four"]:
    try:
        price = float(text)
    except ValueError:
        print(text, "-> not a number")
    else:
        print(text, "-> worked:", price)
```

```
430 -> worked: 430.0
four -> not a number
```

The `else` block is for code that should run only when the risky line succeeded. Keeping it out of the `try` means a mistake inside it isn't mistaken for the failure you meant to catch. Now the retrying version, with a fixed wait for the moment:

```
import time

RETRY_STATUSES = {429, 500, 502, 503, 504}

def get_page_retrying(session, url, page, attempts=4):
    for attempt in range(1, attempts + 1):
        try:
            response = session.get(url, params={"page": page, "page_size": 10},
                                   timeout=(3.05, 10.0))
        except (requests.ConnectionError, requests.Timeout) as exc:
            problem = type(exc).__name__
        else:
            if response.status_code == 200:
                return response.json()
            if response.status_code not in RETRY_STATUSES:
                raise CrmApiError(f"{url} returned {response.status_code}")
            problem = f"HTTP {response.status_code}"
        print(f"attempt {attempt} for page {page} failed ({problem}); waiting 0.5 s")
```

```

        time.sleep(0.5)
        raise CrmApiError(f"page {page} failed after {attempts} attempts")

server.shutdown()
server, base_url = start_server(8029)
data = get_page_retrying(session, f"{base_url}/leads", 2)
print(len(data["items"]), "leads on page", data["page"])

```

```

attempt 1 for page 2 failed (HTTP 503); waiting 0.5 s
10 leads on page 2

```

- **except (requests.ConnectionError, requests.Timeout) as exc:** catches either of two exception types, listed in a tuple. `type(exc).__name__` records which one it was.
- In the **else** branch a response did arrive: 200 returns the data; a status not in **RETRY_STATUSES** (401 bad key, 403 not allowed, 404 wrong address) raises at once, because it will fail identically every time; anything else is noted as the problem and retried.
- After the loop, **raise CrmApiError(...)** gives up with a message. Retrying forever turns a failure into a hang.

Step 3: exponential backoff with jitter

Chapter 18 waited 2, 4, 8, then 16 seconds. The same doubling from a smaller start is **exponential backoff**: 0.5 s after the first failure, then 1 s, then 2 s, so a struggling server gets breathing room. **Jitter** adds a random extra of up to half the wait, so a hundred clients that failed together don't all retry at the same instant. A **seed** makes the waits repeatable, as it made Chapter 21's samples repeatable; here the generator comes from the standard library's `random` module rather than from NumPy:

```

import random

rng = random.Random(29)
for attempt in (1, 2, 3):
    base = 0.5 * 2 ** (attempt - 1)
    print(attempt, base, round(base + rng.uniform(0, base / 2), 3))

```

```

1 0.5 0.637
2 1.0 1.173
3 2.0 2.845

```

- **2 ** (attempt - 1)** is 1, 2, 4 (** is "to the power of"), so `base` is 0.5, 1.0, 2.0.
- **rng.uniform(0, base / 2)** is a random decimal number between 0 and half the base; **random.Random(29)** is a generator started from the seed 29, so it gives the same numbers every run.

What happens if you change the seed? The jitter changes, but every wait stays between the base and one and a half times it:

```

rng = random.Random(30)
print([round(base + rng.uniform(0, base / 2), 3) for base in (0.5, 1.0, 2.0)])

```

```
[0.635, 1.145, 2.03]
```

The list comprehension (Chapter 17, section 17.6) runs the same calculation for each base. `round(..., 3)` dropped the last zero of 2.030.

Step 4: respect Retry-After

A 429 often says exactly how long to wait. When it does, that beats any backoff:

```
def wait_time(attempt, response, rng, backoff=0.5):
    if response is not None and response.status_code == 429:
        retry_after = response.headers.get("Retry-After", "")
        if retry_after.isdigit():
            return float(retry_after)
        base = backoff * 2 ** (attempt - 1)
        return round(base + rng.uniform(0, base / 2), 3)

page_3 = session.get(f"{base_url}/leads", params={"page": 3, "page_size": 10}, timeout=(3.05,
10.0))
print(page_3.status_code, page_3.headers["Retry-After"], wait_time(1, page_3, random.Random(29)))
print(wait_time(1, None, random.Random(29)))
```

```
429 1 1.0
0.637
```

- `response.headers.get("Retry-After", "")` reads the header, or an empty string if there isn't one. `.isdigit()` is True only for a whole number of seconds; the header can also be a date, which this version ignores (exercise 10).
- `response is not None` covers the case where no response arrived at all (a connection error): then there's no header to read, and the backoff decides.
- The first request for page 3 got its planned 429 with `Retry-After: 1`, so the wait is exactly 1 second. With no response, attempt 1 waits 0.637 s, as in step 3.

Step 5: a generator hands out the leads

The caller wants leads, not pages. A **generator** is a function that hands out values one at a time with **yield** instead of returning them all at once; Python pauses it after each value and resumes it when the next one is wanted. A toy:

```
def count_up(n):
    for i in range(n):
        yield i

def two_pages():
    yield from ["lead 1", "lead 2"]
    yield from ["lead 3"]

print(list(count_up(3)))
print(list(two_pages()))
```



```
[0, 1, 2]
['lead 1', 'lead 2', 'lead 3']
```

yield from hands out every item of a list (or of another generator), one by one. `list(...)` collects everything a generator yields; a `for` loop takes the items one at a time. A lead generator yields each page's items and then asks for the next page, so a caller can write `for lead in client.iter_leads():` without knowing pages exist.

The robust client, as a class

The five steps share the same settings: the address, the key, the session, the number of attempts, the backoff, the random generator. That is section 29.5's test for a class: data and the functions that use it travel together. `crm_client.py` gathers the steps into one:

```
"""A careful client for the CRM leads API: pages, timeouts, retries, backoff, rate limits."""
import logging
import random
import time
from collections.abc import Callable, Iterator
from typing import Any

import requests

from riverstone_report.errors import CrmApiError

log = logging.getLogger(__name__)

RETRY_STATUSES = {429, 500, 502, 503, 504}

class CrmClient:
    def __init__(
        self,
        base_url: str,
        api_key: str,
        *,
        session: requests.Session | None = None,
        max_attempts: int = 4,
        backoff_seconds: float = 0.5,
        timeout: tuple[float, float] = (3.05, 10.0),
        sleep: Callable[[float], None] = time.sleep,
        rng: random.Random | None = None,
    ) -> None:
        self.base_url = base_url.rstrip("/")
        self.session = session or requests.Session()
        self.session.headers["X-API-Key"] = api_key
        self.max_attempts = max_attempts
        self.backoff_seconds = backoff_seconds
        self.timeout = timeout
        self.sleep = sleep
        self.rng = rng or random.Random()

    def _wait_time(self, attempt: int, response: requests.Response | None) -> float:
        if response is not None and response.status_code == 429:
            retry_after = response.headers.get("Retry-After", "")
            if retry_after.isdigit():
                return float(retry_after)
            base: float = self.backoff_seconds * 2 ** (attempt - 1)
            return round(base + self.rng.uniform(0, base / 2), 3)
```

```

def get_json(self, path: str, params: dict[str, Any]) -> dict[str, Any]:
    url = f"{self.base_url}{path}"
    for attempt in range(1, self.max_attempts + 1):
        response: requests.Response | None = None
        try:
            response = self.session.get(url, params=params, timeout=self.timeout)
        except (requests.ConnectionError, requests.Timeout) as exc:
            problem = type(exc).__name__
        else:
            if response.status_code == 200:
                data: dict[str, Any] = response.json()
                return data
            if response.status_code not in RETRY_STATUSES:
                raise CrmApiError(
                    f"{url} returned {response.status_code}: {response.text[:100]}"
                )
            problem = f"HTTP {response.status_code}"
        if attempt == self.max_attempts:
            raise CrmApiError(f"{url} failed after {attempt} attempts (last: {problem})")
        wait = self._wait_time(attempt, response)
        log.warning("attempt %d for %s failed (%s); retrying in %.3f s",
                    attempt, params, problem, wait)
        self.sleep(wait)
    raise AssertionError("unreachable")

def iter_leads(self, page_size: int = 10) -> Iterator[dict[str, Any]]:
    page = 1
    while True:
        data = self.get_json("/leads", {"page": page, "page_size": page_size})
        yield from data["items"]
        if data["next_page"] is None:
            return
        page = data["next_page"]

```

How it works, beyond the five steps:

- `*`, in the parameter list means that everything after it must be passed by name: `CrmClient(url, key, max_attempts=6)`, never `CrmClient(url, key, None, 6)`. With six settings, names are the only readable way.
- `session` or `requests.Session()`: or gives the left value unless it's `None` (or empty), and otherwise the right one. So a caller may hand in a session, and a test does; normally the client makes its own. `rng` or `random.Random()` works the same way.
- `base_url.rstrip("/")` removes a trailing `/`, so `http://.../` and `http://...` both work when the path is added.
- `_wait_time` is step 4 as a method. The leading underscore is a convention meaning "internal: for this class's own use, not for callers".
- `get_json` is steps 2 and 4 together, for any path. It logs each retry as a `WARNING` instead of printing, sleeps with `self.sleep`, and on the last attempt raises `CrmApiError` with the last problem. A non-retryable status also includes the first 100 characters of the server's reply (`response.text[:100]`), which usually says why.
- `raise AssertionError("unreachable")` after the loop can never run: the loop always returns or raises. It's there for `mypy`, which checks that every path through a function returns a value of the promised type or raises.
- `iter_leads` is step 5: it yields each page's items and stops when `next_page` is `null` (`None` in Python).
- `sleep` and `rng` are parameters. Normally they're `time.sleep` and a real random generator. A test hands in its own, so a test of four retries runs instantly and checks the exact waits (below).

The same fetch, with the robust client, logging to the screen:

```
import logging
import sys

from riverstone_report.crm_client import CrmClient

logging.basicConfig(stream=sys.stdout, level=logging.INFO,
                    format="%(levelname)s %(name)s: %(message)s", force=True)

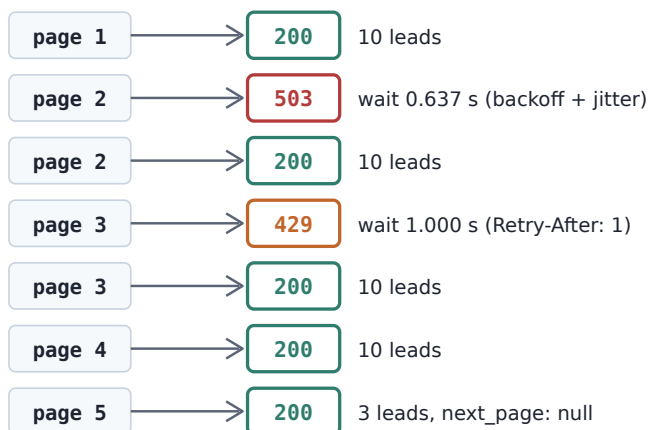
server.shutdown()
server, base_url = start_server(8029)

client = CrmClient(base_url, "demo-key", rng=random.Random(29))
leads = list(client.iter_leads(page_size=10))
print(len(leads), "leads; last:", leads[-1]["company_name"], leads[-1]["created_at"])
```

```
WARNING riverstone_report.crm_client: attempt 1 for {'page': 2, 'page_size': 10} failed (HTTP 503); retrying in 0.637 s
WARNING riverstone_report.crm_client: attempt 1 for {'page': 3, 'page_size': 10} failed (HTTP 429); retrying in 1.000 s
43 leads; last: Daily Fresh Mart 2025-11-25T10:50:00
```

`stream=sys.stdout` and `force=True` are Chapter 20's notebook settings for logging. Both failures were logged, waited out, and retried, and all 43 leads arrived. ✓ The 43 matches Chapter 13's count of lead records (30 real leads plus 13 duplicates, which is Chapter 13's Pattern 3 problem, not the client's). The waits were 0.637 s (0.5 s backoff plus jitter) and exactly 1 s for the 429.

Fetching 43 leads from the mock CRM API: two failures, no lost data



The client's rules

- **Retry 429, 500, 502, 503, 504,** connection errors, timeouts
- **Don't retry 400, 401, 403, 404:** retrying can't fix them
- **Wait $0.5 \text{ s} \times 2^{(\text{attempt} - 1)}$,** plus jitter up to half of that
- **429 with Retry-After:** wait exactly that long
- **Give up after 4 attempts:** raise `CrmApiError`
- **Always set a timeout: (3.05, 10) s**

A real run against the local mock server; the waits are the chapter's output.

Figure 29.4 — Retry what might work next time, wait sensibly, and stop when the API says there are no more pages.

And the failure the client must *not* retry:

```
from riverstone_report.errors import ReportError

bad_client = CrmClient(base_url, "wrong-key", rng=random.Random(29))
try:
    list(bad_client.iter_leads())
except ReportError as exc:
    print(type(exc).__name__ + ":", exc)
server.shutdown()
```

```
CrmApiError: http://127.0.0.1:8029/leads returned 401: {"error": "invalid or missing API key"}
```

No retries, no waiting: a wrong key won't become right in two seconds.

Testing the client without a network

`tests/test_crm_client.py` tests every retry rule in a fraction of a second, with no server at all. It replaces the network with a **test double**: an object that stands in for a real one in a test. This kind, which gives canned answers, is called a **fake**:

```
import random

import pytest
import requests

from riverstone_report.crm_client import CrmClient
from riverstone_report.errors import CrmApiError

class FakeResponse:
    """Just enough of a requests.Response for CrmClient: a status, headers, a body."""
    def __init__(self, status_code, body=None, headers=None):
        self.status_code = status_code
        self._body = body or {}
        self.headers = headers or {}
        self.text = str(self._body)

    def json(self):
        return self._body

class FakeSession:
    """Plays back a list of responses (or exceptions) instead of calling a real server."""
    def __init__(self, replies):
        self.replies = list(replies)
        self.headers = {}
        self.calls = []

    def get(self, url, params, timeout):
        self.calls.append(params)
        reply = self.replies.pop(0)
        if isinstance(reply, Exception):
            raise reply
        return reply

def make_client(replies, sleeps):
    """A CrmClient that talks to a FakeSession and records its waits instead of sleeping."""
    return CrmClient("https://crm.example", "test-key", session=FakeSession(replies),
```

```

        sleep=sleeps.append, rng=random.Random(29))

def test_retries_server_errors_then_succeeds():
    sleeps = []
    client = make_client([FakeResponse(503), requests.ConnectionError(),
                             FakeResponse(200, {"items": [], "next_page": None})], sleeps)
    assert list(client.iter_leads()) == []
    assert sleeps == [0.637, 1.173]

```

- **FakeResponse** has the four things `CrmClient` uses from a real response: `status_code`, `headers`, `text`, and `json()`. **FakeSession** has the one method it uses, `get`, which hands back the next reply in its list with `pop(0)`, or raises it if the reply is an exception. It also records each call's parameters in `calls`.
- `make_client` passes the fake session in, and `sleep=sleeps.append`: instead of sleeping, the client appends each wait to the list `sleeps`. A test of four retries takes no time, and the test can check the waits.
- The test plays back a 503, then a connection error, then success. The client must retry twice and end with no leads; with the seed 29, the waits are exactly 0.637 s and 1.173 s, as in step 3.

The file's other four tests follow the same pattern: pages followed until `next_page` is `None`, `Retry-After: 7` obeyed exactly, a 401 not retried, and four 500s giving up after four attempts.

```

# terminal, in work/ch29/riverstone-report
$ uv run pytest -q tests/test_crm_client.py
.....
5 passed in 0.22s

```

[100%]

Watch out: retries must be safe to repeat. Reading leads twice does no harm. *Creating* an order twice does. A request that can be repeated without changing the outcome is **idempotent** (Chapter 20). GET requests normally are; POST requests that create things normally aren't. Before retrying a write, check whether the API supports an **idempotency key** (a unique id sent with the request so the server ignores duplicates). Chapter 51 covers writing to other systems safely.

Tool note. `urllib3`'s `Retry` class can add retries and backoff to a `requests.Session` in a few lines, and the `tenacity` package provides retry decorators. **httpx** is a newer HTTP client with a similar interface that also supports asynchronous code. Writing the loop by hand once, as here, is how you learn what those tools configure.

29.10 Code review

Code review means another person reads a change before it's merged into the shared code. It catches bugs, but its bigger value is that at least two people understand every part of the system, and that standards spread without anyone writing a manual. On most teams, changes arrive as **pull requests** (Chapter 26): a set of commits with a description, which reviewers comment on line by line.

As the author

1. **Keep changes small:** one purpose per pull request. A reviewer can check 200 lines properly; 2,000 lines get a skim and an approval.
2. **Explain the why** in the description, and how you tested it: *"Counts orders with `nunique()`, not `rows`. Added `test_summarize_counts_orders_not_lines`. Reconciles to ₹4,39,823.50 for December."*
3. **Run the checks first:** tests, type checker, formatter. Don't spend a reviewer's time on what a tool can find.
4. **Point to the risky part:** *"Please look closely at the retry logic."*

As the reviewer

Ask	For example, in this chapter's package
Is it correct for the business rule?	Orders counted with <code>nunique</code> ; cancelled orders excluded (by the <code>sales_lines</code> view)
What happens on the failure path?	Empty month raises <code>NoDataError</code> ; 401 not retried
Could it leak a secret or personal data?	No password in code or logs; <code>.env</code> in <code>.gitignore</code>
Is it tested, and do the tests test the rule?	A test named after the rule, with hand-checkable data
Will the next person understand it?	Names say what things are; comments say why
Does it handle scale?	Query filters by month in SQL, not in pandas
Is it the simplest thing that works?	A dataclass, not a class hierarchy

How to comment. Comment on the code, never the person. Ask rather than instruct when you might be missing context (*"What happens if the target row is missing here?"*), label optional suggestions (*"nit:"* for trivial style points), and say what's good as well. Approve when the change is correct and safe, not when it's exactly how you'd have written it.

Real-life example: the review comment that saved a quarter's numbers. A common catch in analytics teams: a pull request changes a revenue calculation to "include pending orders so the dashboard updates sooner". The code is clean and the tests pass, because the tests were updated too. The reviewer asks one question: *"Has finance agreed to count pending orders as revenue?"* They hadn't. Review isn't only about code; it's the moment someone checks that a business rule changed on purpose.

Formatting and linting

Arguments about spacing and quote styles waste review time, so teams hand them to tools. A **formatter** rewrites code into a consistent style automatically, and a **linter** flags likely mistakes (unused imports, undefined names, a bare `except`). **Ruff** does both and is widely used with `uv` (`uv add --dev ruff`, then `uv run ruff check` and `uv run ruff format`). Run them before review, and in automated checks on every pull request (Chapter 32 does the same for SQL).

Common mistakes

Mistake	Symptom	Fix
All code at the top level of one file	Importing it to test anything connects to the database and writes files	Functions in modules; a <code>__main__</code> block or entry point only in the CLI
Password or API key in the code	It appears in Git history, screenshots, and emails	Environment variables or a secrets manager; <code>.env</code> in <code>.gitignore</code>
Month or file name typed into the code	Every run edits code; reports overwrite each other	Command-line arguments; file names built from the settings
Installing packages into the system Python	Projects break each other; "works on my machine"	One virtual environment per project
No lockfile, or not committing it	A new install gets different versions and behaves differently	Commit <code>uv.lock</code> (or pinned requirements); install with <code>uv sync --locked</code>
SQL built by adding strings	Breaks on quotes; SQL injection	Query parameters
Bare <code>except:</code> or <code>except Exception: pass</code>	Wrong results with no error; Ctrl+C doesn't stop the program	Catch named exceptions you can handle; let bugs crash
Turning a failure into a default value (<code>target = 0</code>)	Plausible-looking wrong numbers	Raise, or use <code>None</code> and handle it visibly
<code>print</code> in scheduled code	No timestamps, no levels, nothing saved; scheduler thinks it succeeded	<code>logging</code> , and a non-zero exit code on failure
Calling <code>logging.basicConfig</code> inside a library module	Other programs' logging settings get overridden	Configure logging once, in the entry point
Tests that need the database for every calculation	Tests are slow, flaky, and skipped	Pure functions tested with tiny DataFrames; one integration test
Test data too large to check by hand	Nobody can tell whether the expected value is right	Five rows you can add up in your head
Tests that only cover the happy path	Failures surprise you in production	<code>pytest.raises</code> for bad input, empty data, and missing settings
No timeout on an HTTP call	A job hangs forever when the server stops responding	Always pass <code>timeout=</code>
Not checking the HTTP status	<code>KeyError</code> , or silently missing pages	Check <code>status_code</code> ; raise or retry
Retrying every error, forever, immediately	Bans for hammering the API; hangs; duplicate writes	Retry only transient errors, with backoff, a limit, and <code>Retry-After</code>
Retrying non-idempotent requests	Duplicate records in the other system	Retry only safe requests, or use idempotency keys
Classes for everything	Code is harder to follow than the script it replaced	Functions for calculations; data classes for bundles; classes for shared state
Ignoring type checker errors about <code>None</code>	Crash on the first month with no target	Handle the <code>None</code> case explicitly

In the real world: the report that said January was zero

It's Tuesday, 3 February 2026. At 9:10 the managing director forwards Riverstone's monthly report to Anita Rao with one line: "Revenue zero in January?"

Meera Iyer runs the report. Since Imran left, it's a copy of his script, scheduled with Windows Task Scheduler to run on the second of each month. She opens the log folder: there isn't one. The scheduler's history says the task **completed successfully**. The shared drive has one file, `Monthly_Report_Dec_FINAL.xlsx`, dated yesterday, showing January with ₹0 revenue and a blank percentage of target.

It takes her an hour to piece together:

1. The load from the ERP to the reporting database had been failing since the last day of December, and nobody was alerted. The reporting copy had no January orders at all.
2. The script found no rows, summed them to zero, and divided zero by a target of zero, because `except: target = 0` had also hidden the fact that January's target row hadn't been entered yet.
3. It printed done and exited with code 0, so the scheduler recorded success.
4. It saved over the file with the fixed name, so December's good report was gone too. (IT restored it from the previous night's backup, as Chapter 2 hoped someone would.)

The data team reloads the database, and Meera reruns the script by hand. January's report goes out at 2 p.m., with the real numbers.

Then she fixes the cause, not the symptom, in three pull requests over the following two weeks, each reviewed by the data engineer:

- **Fail loudly.** The report became the package in this chapter. An empty month raises `NoDataError`, a missing target shows "no target set" instead of 0, and every failure exits with code 1. The scheduler now emails her when the exit code isn't 0.
- **Stop guessing.** Tests for the rules that bit her: orders counted, not lines; empty month; missing target; December rolling into January. And one reconciliation test against a month everyone agrees on.
- **Make it runnable by anyone.** The password moved into an environment variable on the report server. `uv.lock` pins the versions, so the server and her laptop run the same code. Output files are named by month, so no report can overwrite another.

On 2 March, the nightly load fails again. At 07:00 the report runs, logs `ERROR riverstone_report: no sales lines for this month`, exits with code 1, and Meera has an email before she reaches her desk. The MD never sees a wrong number.

What she tells Anita: "The report used to succeed even when it had nothing to report. Now, if the data isn't there, it stops, tells me, and leaves last month's file alone. It also has tests for the rules finance cares about, so when someone changes a calculation, we'll know the same day."

Notice that the fix wasn't clever code. It was ordinary habits: explicit failures, a meaningful exit code, tests for real rules, settings outside the code, and a second pair of eyes.

Project: turn a script into a tested package

Goal: a script you (or your team) depend on becomes a package that someone else can install and run with one command, with tests that protect its business rules.

Tools you'll need

Versions used for this chapter, checked on 28 September 2026:

- **Python 3.14.7**, as installed in Chapter 17. The package's `.python-version` asks for 3.14, and `uv` downloads it if it's missing.
- **uv 0.12.19** (released on 25 September 2026), installed from `docs.astral.sh/uv`. `uv` releases frequently; check the documentation for current installation commands.
- **Packages** (as locked in `uv.lock`): `pandas` 3.0.6, `openpyxl` 3.1.5, `SQLAlchemy` 2.1.1, `psycopg` 3.3.6, `requests` 2.34.2; for development, `pytest` 9.1.1, `mypy` 2.3.1, `ipykernel` 7.3.0, `pandas-stubs`, `types-requests`, `types-openpyxl`.
- **An editor with Python support**, such as VS Code with the Python and Jupyter extensions (runs tests and notebooks, shows type errors as you type).
- **Ruff** (optional) for formatting and linting.
- **Git and a GitHub account** (Chapter 26) for version control, pull requests, and the automated check.
- **Companion files** in `ch29/`: `start/monthly_report.py` (the starting script), `riverstone-report/` (the finished package with `pyproject.toml`, `uv.lock`, `.env.example`, `source`, and 23 tests), and `mock_crm_api.py` and `leads.json` (the practice API).

Option A: your own script. Choose a real script or notebook that produces something people use. Work in a private copy; remove passwords, server names, and personal data before sharing it with anyone, including in a portfolio.

Option B: Riverstone. Start from `start/monthly_report.py` and rebuild the package yourself in `work/ch29/mine/riverstone-report` (section 29.4) without looking at `riverstone-report/`, then compare.

Steps:

1. **List the problems** in the original, using section 29.1's table as a checklist.
2. **Create the project** with `uv init --package`, and add dependencies with `uv add`. Commit `pyproject.toml` and `uv.lock`.
3. **Split the jobs** into modules: configuration, reading, transforming (pure), writing, and a command-line entry point.
4. **Move settings out of the code:** arguments for each run, environment variables for secrets and machine-specific settings.
5. **Write tests first for the rules that matter**, with a fixture of five to ten rows you can check by hand. Include at least three failure cases.
6. **Add type hints** and get `uv run mypy` passing in strict mode.

7. **Replace print with logging**, catch your own exceptions at the entry point, and return an exit code.
8. **Reconcile** one real run against a number you already trust.
9. **Run the checks on every push**: add section 29.7's workflow to the repository, push, and confirm that the pull request shows a green check.
10. **Write a README**: what it does, how to install (`uv sync`), how to run, which environment variables it needs, how to run the tests.
11. **Ask someone to review it** with section 29.10's checklist, and act on the comments.

Deliverables: the repository with its passing check, the test output, the reconciliation, and a half-page note listing each problem from step 1 and how it was fixed.

Stretch goals:

- Add a `--dry-run` flag that runs everything except writing the file.
- Pull the month's new leads from the mock CRM API with `CrmClient` and add a "Leads by source" sheet.
- Add Ruff and a pre-commit hook that runs Ruff and the tests before every commit.

REVIEW

Recap

- **Software** is code others can install, run, understand, change, and trust. The time to write it is when people depend on the output, it runs on a schedule, or someone else will change it.
- Split code into **functions** and **modules** with one job each. Keep business rules in **pure functions**; put databases, files, and networks at the edges.
- Use the **src/ layout**, a **pyproject.toml**, and **environment variables** for secrets and machine settings. Use query **parameters**, never string-built SQL.
- A **virtual environment** isolates a project's packages; a **lockfile** pins every version. **uv** manages both (`uv add`, `uv sync --locked`, `uv run`); `venv` with pinned requirements is the universal fallback.
- A **class** is a template for objects: `__init__` sets an instance's attributes, **methods** receive the object as `self`, and **decorators** such as `@property` and `@classmethod` add behaviour. **Data classes** bundle values with names and types; **properties** calculate instead of storing; `frozen=True` prevents accidental changes.
- **Type hints** document intent, and **mypy** checks them without running code, catching bugs such as unhandled `None`.
- **pytest** runs `test_*` functions; **fixtures** supply fresh test data; **parametrize** tests many inputs; **pytest.raises** tests failures. Many fast unit tests, one reconciliation against a known total, and all of them run on every push.
- Catch **named exceptions**, never bare `except`. Define your own (`ReportError`), handle them at the entry point, and return an **exit code**. Use **logging** with levels, configured once, and never log secrets.

- **Robust API clients** use a session, **timeouts**, **pagination**, and **retries** for transient errors (429, 5xx, connection errors) with **exponential backoff**, **jitter**, **Retry-After**, and a limit. Retry only **idempotent** requests.
- **Code review**: small, explained changes; reviewers check rules, failures, secrets, tests, and clarity. Formatters and linters handle style.

Key terms

software · script · pure function · module · docstring · package · `__name__` · SQL injection · query parameter · `src/` layout · `pyproject.toml` · TOML · entry point · environment variable · `.env` file · secrets manager · virtual environment · `venv` · lockfile · pinned requirements · `uv` · `uv.lock` · dependency group · dev dependency · editable install · class · instance · attribute · method · `self` · decorator · inheritance · class method · data class · immutable · property · alternative constructor · type hint · type checker · `mypy` · test · `pytest` · assertion · node ID · fixture · parametrize · unit test · integration test · coverage · exception · custom exception · exit code · logging · log level · logger · pagination · transient failure · rate limit · HTTP 429 · `Retry-After` · timeout · retry · exponential backoff · jitter · session · keyword-only argument · generator · test double · fake · idempotent · idempotency key · code review · pull request · formatter · linter · Ruff

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can say when a script should become a package, and when it shouldn't.
- ☐ My business calculations are pure functions that I can run on a five-row DataFrame.
- ☐ My projects have a `src/` layout, a `pyproject.toml`, and a committed lockfile.
- ☐ I can recreate a project's exact environment on a new machine with one command.
- ☐ No password or key appears in my code, my Git history, or my logs.
- ☐ I can write a small class with `__init__`, a method, and a property, and I use data classes for bundles of settings and results, and classes only when data and behavior belong together.
- ☐ I add type hints and fix what `mypy` finds, especially `None` cases.
- ☐ I write tests for business rules, edge cases, and failures, using fixtures and parametrize, and they run on every push.
- ☐ My scheduled code logs with levels and exits with a non-zero code on failure.
- ☐ My API clients set timeouts, follow pages, retry only transient errors with backoff, and respect `Retry-After`.
- ☐ I review code with a checklist, and write pull requests that are quick to review.

Exercises

Run code exercises in your `ch29.ipynb` notebook in `work/ch29`, with the `riverstone-report/.venv` kernel (section 29.4), or inside the package with `uv run`. Exercises that need the database say so.

Warm-up

1. Write a pure function `net_revenue(quantity: int, unit_price: float, discount_pct: float) -> float` that returns the net value of one order line, rounded to 2 decimal places. Check it on order 10005's line from Chapter 28: 15 boxes at ₹430 with 5% discount.
2. Predict what each call prints or raises, then run them: `parse_month("2025-02")`, `parse_month("2025-2")`, `parse_month("2025/02")`.
3. Read the starting script's `except:` block. Name three different problems that would all end up as `target = 0`.

Core

4. Add a function `average_order_value(lines: pd.DataFrame) -> float` to your own copy of `transform.py`: net revenue divided by the number of orders, rounded to 2 decimals. What does it return for the five-line test DataFrame? Write its test.
5. Write a parametrized test for `net_revenue` from exercise 1 with three cases, including a 0% and a 100% discount.
6. Change the logging setup so that `INFO` and above go to the screen and `DEBUG` and above also go to a file called `report.log`. Show the code.
7. Use `CrmClient` against the mock API to count leads by `source`, most common first. How many leads came from the website?
8. Extend `ReportConfig.from_env` to read an optional `RIVERSTONE_TOP_N`. It must be a whole number from 1 to 20; anything else raises `ConfigError`. Show what happens with `"25"` and `"five"`.
9. `mypy` reports: `error: Argument 1 to "round" has incompatible type "float | None"; expected "_SupportsRound1[int]" [arg-type]` for `round(summary.pct_of_target)`. Explain the problem in one sentence and fix the line.

Stretch

10. A server sends `Retry-After: Wed, 21 Oct 2026 07:28:00 GMT` (a date instead of a number of seconds). What does `CrmClient._wait_time` do with it? Write a test that proves your answer.
11. Make `write_report` safe against half-written files: write to a temporary file in the same folder, then rename it to the final name. Explain why that matters for a report someone might open while it's being written.
12. The mock API can be started with a different failure plan: `start_server(failures={2: [500, 500, 500, 500]})`. Predict what `CrmClient` with the default settings does, then run it. How many seconds does it spend waiting, roughly?

Think about it (no code needed)

13. A colleague says: "Logging is overkill; my script prints what it's doing and I can see it." When is she right, and when isn't she?
14. Give two situations where turning a script into a package with tests would be a waste of time.
15. You're reviewing a pull request that changes `revenue_by_category` to exclude the Furniture category "because it's being discontinued", with the test updated to match. What do you ask before approving?

Answers

1.

```
def net_revenue(quantity: int, unit_price: float, discount_pct: float) -> float:
    """Net value of one order line, rounded to paise."""
    return round(quantity * unit_price * (1 - discount_pct / 100), 2)

print(net_revenue(15, 430, 5))
```

6127.5

₹6,127.50, the same as the hand check in section 28.7. ✓ It's pure: no database, no global variables, and the same inputs always give the same output.

2.

```
from riverstone_report.config import parse_month
from riverstone_report.errors import ConfigError

for text in ["2025-02", "2025-2", "2025/02"]:
    try:
        print(text, "->", parse_month(text))
    except ConfigError as exc:
        print(text, "-> ConfigError:", exc)
```

```
2025-02 -> 2025-02-01
2025-2 -> 2025-02-01
2025/02 -> ConfigError: month must look like YYYY-MM, got '2025/02'
```

"2025-2" works, because `int("2")` is 2. "2025/02" fails because splitting on `-` gives one part, which can't be unpacked into `year, month`, a `ValueError` that `parse_month` turns into a `ConfigError`. If you want to insist on two-digit months, that's a rule to add *and* a test to write.

3. Any three of: the `sales_targets` table doesn't exist in the database the script connected to; the target row for that month hasn't been entered yet (`iloc[0, 0]` raises `IndexError`); a typo in the SQL; the database connection dropped; the value can't be converted to float; the user pressed Ctrl+C at that moment (a bare `except` even catches `KeyboardInterrupt`). They all need different responses, and the script gave every one of them the same wrong answer.

4.

```
import pandas as pd

def average_order_value(lines: pd.DataFrame) -> float:
    orders = lines["order_id"].nunique()
    if orders == 0:
        raise ValueError("no orders to average")
    return round(float(lines["net_revenue"].sum()) / orders, 2)

lines = pd.DataFrame({
    "order_id":      [1, 1, 2, 3, 3],
    "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart",
        "Green Leaf Hotels", "Green Leaf Hotels"],
    "category":      ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
    "net_revenue":    [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
})
print(average_order_value(lines))
```

8333.33

₹25,000 ÷ 3 orders = ₹8,333.33. The common wrong answer is `lines["net_revenue"].mean()`, which averages *lines* (₹5,000). The tests, in `tests/test_transform.py`, where the `lines` fixture comes from `conftest.py`:

```
import pytest

from riverstone_report.transform import average_order_value

def test_average_order_value_divides_by_orders(lines):
    assert average_order_value(lines) == 8333.33

def test_average_order_value_of_no_orders(lines):
    with pytest.raises(ValueError):
        average_order_value(lines.iloc[0:0])
```

5.

```
import pytest

from riverstone_report.transform import net_revenue

@pytest.mark.parametrize("quantity, unit_price, discount_pct, expected", [
    (15, 430.0, 5.0, 6127.50),
    (20, 450.0, 0.0, 9000.00),
    (10, 380.0, 100.0, 0.00),
])
def test_net_revenue(quantity, unit_price, discount_pct, expected):
    assert net_revenue(quantity, unit_price, discount_pct) == expected
```

Three results in pytest's output, one per row. (This assumes you added `net_revenue` to `transform.py`.) Consider adding a case that should *fail*, such as a discount of 120%, and deciding whether `net_revenue` should raise for it.

6. Configure a handler for each destination, each with its own level, and set the logger's level to the lowest:

```
import logging

formatter = logging.Formatter("%(asctime)s %(levelname)s %(name)s: %(message)s")

console = logging.StreamHandler()
console.setLevel(logging.INFO)
console.setFormatter(formatter)

logfile = logging.FileHandler("report.log", encoding="utf-8")
logfile.setLevel(logging.DEBUG)
logfile.setFormatter(formatter)

root = logging.getLogger()
root.setLevel(logging.DEBUG)
root.addHandler(console)
root.addHandler(logfile)
```

`encoding="utf-8"` lets the file hold ₹, as in Chapter 20. This replaces `logging.basicConfig(...)` in `main`. A common mistake is setting only the handlers' levels: the logger's own level (default `WARNING`) is checked first, so `DEBUG` messages would never reach the file.

7.

```
import random
from collections import Counter

from mock_crm_api import start_server
from riverstone_report.crm_client import CrmClient

server, base_url = start_server(8029, failures={}) # no planned failures for this exercise
client = CrmClient(base_url, "demo-key", rng=random.Random(7))
by_source = Counter(lead["source"] for lead in client.iter_leads(page_size=20))
server.shutdown()
for source, count in by_source.most_common():
    print(f"{source:<18} {count}")
print("total", sum(by_source.values()))
```

```
Website          27
Referral          5
Trade fair        5
Cold call         3
IndiaMART listing 3
total 43
```

27 of the 43 lead records came from the website. The counts include Chapter 13's duplicate lead records, because the API returns what the CRM holds; deduplicate (Chapter 13, Pattern 3) before reporting leads by source to anyone.

8.

```
top_n_text = env.get("RIVERSTONE_TOP_N", "5")
if not top_n_text.isdigit() or not 1 <= int(top_n_text) <= 20:
    raise ConfigError(f"RIVERSTONE_TOP_N must be a whole number from 1 to 20, got {top_n_text!r}")
# then pass top_n=int(top_n_text) to cls(...)
```

With this added to `from_env`, a value of "25" raises `ConfigError: RIVERSTONE_TOP_N must be a whole number from 1 to 20, got '25'`, and "five" raises the same message with 'five'. `isdigit()` is checked first, so `int()` is never called on text that isn't a number.

9. `pct_of_target` is `None` when a month has no target, and `round(None)` fails, so the code must decide what to show in that case: `round(summary.pct_of_target)` if `summary.pct_of_target` is not `None` else "no target". After the `is not None` check, mypy knows the value is a float. `_SupportsRound1` is mypy's name for "anything that can be rounded"; `None` can't.

10. "Wed, 21 Oct 2026 07:28:00 GMT".`isdigit()` is `False`, so `_wait_time` ignores the header and falls back to exponential backoff. The HTTP standard allows `Retry-After` to be either a number of seconds or a date. The test, added to `tests/test_crm_client.py`:

```
def test_retry_after_as_a_date_falls_back_to_backoff():
    sleeps = []
    client = make_client([FakeResponse(429, headers={"Retry-After": "Wed, 21 Oct 2026 07:28:00
        GMT"}),
                          FakeResponse(200, {"items": [], "next_page": None})], sleeps)
    list(client.iter_leads())
    assert len(sleeps) == 1 and 0.5 <= sleeps[0] <= 0.75
```

With `backoff_seconds=0.5`, the first wait is 0.5 s plus jitter of up to 0.25 s. Supporting the date form is a reasonable improvement: parse it with `email.utils.parsedate_to_datetime` and wait until that moment.

11.

```
import os
import tempfile

# inside write_report, replacing the direct write:
fd, temp_name = tempfile.mkstemp(suffix=".xlsx", dir=path.parent)
os.close(fd)
temp_path = Path(temp_name)
try:
    with pd.ExcelWriter(temp_path, engine="openpyxl") as writer:
        ...
        os.replace(temp_path, path)
except BaseException:
    temp_path.unlink(missing_ok=True)
    raise
```

`tempfile.mkstemp(suffix=".xlsx", dir=path.parent)` creates an empty temporary file with that ending, in the report's own folder, and returns an open file number and the file's name; `os.close(fd)` closes it so pandas can write to it; `temp_path.unlink(missing_ok=True)` deletes it, without an error if it's already gone. `except BaseException:` rather than `except Exception:` because the clean-up must also happen when someone presses Ctrl+C, which raises `KeyboardInterrupt`, a `BaseException` but not

an `Exception`; raise on its own then re-raises the same exception, so nothing is hidden. If the program crashes or is stopped halfway, a direct write leaves a damaged workbook under the real name, and anyone opening it (or an email job attaching it) gets a broken file. Writing to a temporary file and then replacing the final file means the real name only ever points to a complete report. `os.replace` works as a single step when both files are in the same folder, which is why the temporary file is created in `path.parent`.

12. Four 500s for page 2 exhaust all four attempts, so the client raises `CrmApiError` after waiting three times: about 0.5, 1, and 2 seconds plus jitter, between 3.5 and 5.25 seconds in total. Page 1's 20 leads are fetched first, but because `iter_leads` is consumed by `list(...)`, the caller gets the exception, not a partial list:

```
import logging
import random
import sys
import time

from mock_crm_api import start_server
from riverstone_report.crm_client import CrmClient
from riverstone_report.errors import ReportError

logging.basicConfig(stream=sys.stdout, level=logging.WARNING,
                    format="%(levelname)s %(name)s: %(message)s", force=True)

server, base_url = start_server(8029, failures={2: [500, 500, 500, 500]})
client = CrmClient(base_url, "demo-key", rng=random.Random(12))
started = time.perf_counter()
try:
    leads = list(client.iter_leads(page_size=20))
except ReportError as exc:
    print(type(exc).__name__ + ":", exc)
waited = time.perf_counter() - started
print("between 3.5 and 5.3 seconds:", 3.5 <= waited <= 5.3)
server.shutdown()
```

```
WARNING riverstone_report.crm_client: attempt 1 for {'page': 2, 'page_size': 20} failed (HTTP 500); retrying in 0.619 s
WARNING riverstone_report.crm_client: attempt 2 for {'page': 2, 'page_size': 20} failed (HTTP 500); retrying in 1.329 s
WARNING riverstone_report.crm_client: attempt 3 for {'page': 2, 'page_size': 20} failed (HTTP 500); retrying in 2.666 s
CrmApiError: http://127.0.0.1:8029/leads failed after 4 attempts (last: HTTP 500)
between 3.5 and 5.3 seconds: True
```

The three warnings show the growing waits; the fourth failure ends the attempts. `CrmApiError` is a `ReportError`, so catching the base class works.

13. She's right for code she runs by hand and watches, once. She isn't when it runs on a schedule or on a server: nobody is watching the screen, prints have no timestamps or levels, nothing is kept after the window closes, and a scheduler can't tell a printed error from success. Logging costs two lines per module, and turns the same messages into a record you can search on the day something goes wrong.

14. Two of: a one-off analysis for a single question that won't be repeated; exploring a new dataset in a notebook to decide whether it's worth anything; a throwaway conversion of a file that will be deleted afterwards; a quick prototype whose purpose is to find out whether an idea works at all. The test is

whether anyone will run, depend on, or change it again. When the answer changes to yes, that's the moment to refactor.

15. Whether the business decision has been made and by whom (Anita? finance?); whether "discontinued" means past Furniture revenue should disappear from historical reports too, or only future sales stop; whether this should be a filter in the report at all, or a product status in the data so every report agrees; and whether the total revenue on the Summary sheet will still reconcile with the categories. Updating a test to match changed code proves only that the code does what it now says, not that it should.

NEXT

Where this leads

- **Chapter 32, Analytics Engineering with dbt**, applies these habits to SQL: version control, tests, review, and automated checks.
- **Chapter 33, The Computer Science You Actually Need**, explains why some Python code is slow and how to measure and fix it.
- **Chapter 30, Inference & Experiments**, uses Python for statistics; its final stretch goal turns the analysis into a tested package.
- **Chapter 46, Pipelines & Orchestration**, and **Chapter 56, MLOps**, build on packages, lockfiles, logging, and retries at production scale.
- **Chapter 72, Python & pandas Question Bank**, has interview questions on project structure, testing, error handling, and API clients.

Chapter 32. Analytics Engineering with dbt

Chapter at a glance

You will learn to: explain what analytics engineering is and where dbt fits · set up a dbt project against a real database, with credentials kept out of the code · turn SQL you already write into models, sources, and `ref` calls that dbt orders for you · lay a project out in staging, intermediate, and marts layers · choose materializations, and know what each one costs · write tests that fail loudly when a business rule breaks, including a reconciliation test · generate documentation and lineage nobody has to maintain by hand · keep type 2 history with snapshots · make a large model incremental and measure what it saves · write a macro so a business rule exists once · lint SQL and run the whole project on every pull request.

Before you start: Chapters 12 and 13 (SQL, the `sales_lines` view and its net-revenue rule), Chapter 28 (star schemas, grain, slowly changing dimensions, and its additions to `riverstone_2025`), Chapter 17 (virtual environments), Chapter 29 (`uv` in section 29.4, settings in environment variables, exit codes), Chapter 26 (Git, pull requests, the automated check and its YAML file), Chapter 34 (the command line, environment variables, running scripts, ports), and Chapter 16 (connecting Power BI to PostgreSQL, used in the project).

Time needed: 18–22 hours, spread over three weeks, in four sittings: sections 32.1–32.5 (setup, sources, staging, marts, the first build: 5–6 hours); sections 32.6–32.9 (materializations, tests, docs, snapshots: 4–5 hours); sections 32.10–32.13 (incremental models, macros, linting, CI: 4–5 hours); and the project (5–6 hours).

Tools: dbt Core 1.12 with the Postgres adapter, PostgreSQL 16, Git, and SQLFluff. Everything runs locally; no dbt Cloud account is needed. This chapter uses PostgreSQL only: dbt reaches MySQL only through a community adapter (`dbt-mysql`, at version 1.7.0 on PyPI in September 2026, well behind dbt Core), so the examples don't use it.

Practice data: the `riverstone_2025` database from Chapter 13, with Chapter 28's additions, plus `riverstone_perf` from Chapter 28 for the incremental example. The finished project is in the companion folder as `ch32/riverstone_dbt`, and the chapter builds it from nothing.

In this chapter

START	Why this matters · In plain English	214
LEARN	32.1 What analytics engineering is	215
	32.2 Your first project	216
	32.3 Models, sources, and ref	221
	32.4 Layers: staging, intermediate, marts	225
	32.5 Building the star schema as models	227
	32.6 Materializations: view, table, incremental, ephemeral	233
	32.7 Tests: the part that earns the trust	235
	32.8 Documentation and lineage	238
	32.9 Snapshots: history dbt keeps for you	239
	32.10 Incremental models	240
	32.11 Macros: write a rule once	242
	32.12 Linting and continuous integration	244
	32.13 What dbt is not, and the semantic layer	248
APPLY	Common mistakes · In the real world · Project	249
REVIEW	Recap · Key terms · Check yourself	252
PRACTISE	Exercises	253
	Answers	255
NEXT	Where this leads	259

START

Why this matters

Chapter 28 built Riverstone a star schema. It works, and it has four problems that every hand-built warehouse has:

- **Nobody knows the order.** The build script has to run the dimensions before the fact table. That order lives in a file somebody wrote once and in the head of whoever wrote it.
- **Nothing checks it.** If tomorrow's load drops half the customers, the fact table shrinks and the dashboard is quietly wrong.
- **Nobody knows what depends on what.** Change `sales_lines` and you find out what breaks when someone complains.
- **There's no history of changes.** Who changed the revenue rule in March, and why?

Chapter 29 solved exactly these problems for Python: structure, tests, review, version control.

Analytics engineering is the same answer for SQL, and **dbt** is the tool most teams use to do it. You write `SELECT` statements; dbt works out the dependency order, builds the tables and views, runs the tests, writes the documentation, and does it all from version control with a command that a schedule can run at 3 a.m.

It is also, right now, the job title that sits between analyst and data engineer, and the skill that most often turns "senior analyst" into "analytics engineer" with the salary that comes with it (Chapter 8). Interviewers ask about it directly: "*what does `ref` do?*", "*what's the difference between a table and an incremental model?*", "*how would you test that a load is complete?*"

In plain English

dbt is a kitchen rota for your SQL.

You have thirty dishes to prepare, and some need others finished first: the sauce before the pasta, the stock before the sauce. Writing the order down by hand works until a new dish arrives, and then somebody serves pasta with no sauce.

dbt asks you to write each dish's recipe on its own card, and to say which other cards it uses, by name. From those names it works out the order itself. Add a dish and the rota updates. Change the stock and dbt knows every dish that needs remaking.

Three more things come with the cards:

- **Tasting notes:** rules each dish must satisfy before it leaves the kitchen (no duplicate order ids, no missing revenue, the total matches the source). If a rule fails, the pass stops and somebody is told.
- **A menu:** documentation and a diagram of what feeds what, generated from the same cards, so it can't drift from reality.
- **A logbook:** the cards live in Git, so every change was reviewed, dated, and attributed.

What dbt does not do is move data into the warehouse, or run itself on a schedule. It only transforms what's already there (Chapter 46 handles the moving and the scheduling).

LEARN

32.1 What analytics engineering is

	Data engineer	Analytics engineer	Data analyst
Works on	pipelines, infrastructure, ingestion	transforming raw tables into trusted models	questions, analysis, dashboards
Main tools	Python, orchestration, cloud	SQL, dbt, Git, the warehouse	SQL, BI tools, spreadsheets
Deliverable	data arrives, reliably	a modelled, tested, documented layer	an answer somebody acts on
Chapter	46, 49	this one	13, 16

The role exists because of one change in how warehouses work. When storage and compute were expensive, you transformed data *before* loading it, because you couldn't afford to keep the raw version: **ETL**, extract-transform-load. Cloud warehouses made storage cheap and compute elastic, so the sensible order became **ELT**: load the raw data first, then transform it inside the warehouse with SQL.

That one change puts transformation where analysts already live, and it's why a tool whose entire job is "run my SQL in the right order, and test it" became a standard.

	ETL	ELT
Transformation happens	in a separate tool, before loading	inside the warehouse, after loading
Raw data kept?	often not	usually, and it's the point (retention and privacy rules may still delete or mask some of it)
Written by	data engineers, often in a GUI or Python	analysts and analytics engineers, in SQL
Fixing a mistake means	reloading from the source	rebuilding models from raw data you still have
dbt's place	not used	the T

32.2 Your first project

Installing dbt

dbt is a Python program, so it goes into a virtual environment (Chapter 17), managed with `uv` as in section 29.4. It needs a folder to live in first. Make one, then turn it into a `uv` project:

```
# terminal, in the folder where you keep your projects
$ mkdir analytics
$ cd analytics
$ uv init --bare
Initialized project `analytics`
```

`uv init --bare` creates only a `pyproject.toml`, the list of what this folder needs. Section 29.4 used `uv init --package`, which also creates a `src/` folder for your own Python code; here the only code will be SQL, so the bare version is enough. Now add dbt and the **adapter** for your database, the piece that lets dbt talk to PostgreSQL:

```
# terminal, in analytics
$ uv add dbt-core dbt-postgres
Using CPython 3.14.7
Creating virtual environment at: .venv
Resolved 61 packages in 46ms
Installed 59 packages in 49ms
+ agate==1.9.1
...
```

(Trimmed: the full output lists all 59 packages, including `dbt-core` 1.12.5 and `dbt-postgres` 1.11.0. The first time, `uv` also downloads them, which takes a minute.)

`uv add` did three things you know from section 29.4: it created the virtual environment in `.venv`, installed the two packages and everything they need, and recorded the exact versions in `uv.lock`. The `dbt` command lives inside `.venv`, so switch the environment on (Chapter 17), and check:

```
# terminal, in analytics
$ source .venv/bin/activate
$ dbt --version
Core:
- installed: 1.12.5
- latest:    1.12.5 - Up to date!

Plugins:
- postgres: 1.11.0 - Up to date!
```

On Windows, in Git Bash the first line is `source .venv/Scripts/activate`, and in PowerShell it's `.venv\Scripts\Activate.ps1`. The environment stays on for this terminal window, including when you `cd` into subfolders; in a new window, activate it again. (The alternative is to put `uv run` in front of every command, `uv run dbt --version`, as Chapter 29 did.)

Adapters exist for Snowflake, BigQuery, Databricks, Redshift, DuckDB, SQL Server, and others; the project you write barely changes between them.

Creating the project

```
# terminal, in analytics
$ dbt init riverstone_dbt --skip-profile-setup
19:04:31 Running with dbt=1.12.5
19:04:31 Your new dbt project "riverstone_dbt" was created!
$ cd riverstone_dbt
$ rm -r models/example
```

(Trimmed: `dbt` also prints the new folder's full path and links to its documentation, and on its very first run it creates a `.dbt` folder in your home folder.)

Every line `dbt` prints starts with the time, so yours will show different times. `dbt init` creates the project folder and its subfolders; `--skip-profile-setup` stops it asking connection questions, because you'll write that file yourself in a moment. `rm -r models/example` deletes the two sample models `dbt` includes (Chapter 26's `rm -r`: a folder and everything inside it, with no undo).

The folders that matter:

Folder or file	Holds
dbt_project.yml	the project's name and settings: where things build, and how
profiles.yml	how to connect to the database (which is why it must not contain a password); you create it below
models/	your SELECT statements, one file per model
snapshots/	definitions for keeping type 2 history (section 32.5)
tests/	custom tests written as SQL (section 32.7)
macros/	reusable SQL snippets (section 32.11)
seeds/	small CSV files you want in the warehouse (a mapping table, say)
target/	everything dbt generates: compiled SQL, run results, docs. Never in Git

dbt init also writes a .gitignore that already lists target/, logs/, dbt_packages/, and .env.

Jinja in two minutes. dbt's files are **templates**. Before dbt uses a SQL or YAML file, it runs it through **Jinja**, a templating language from the Python world, and replaces three kinds of marker:

Syntax	Does	Example
{{ ... }}	works something out and puts the result in its place	{{ ref('stg_orders') }}
{% ... %}	an instruction: if, for, set, macro	{% if is_incremental() %}
{# ... #}	a comment that dbt removes, so it never reaches the database	{# grain: one order line #}

The first one you'll meet is `env_var`, which reads an environment variable (Chapter 29). In `profiles.yml` below, `{{ env_var('RIVERSTONE_HOST', 'localhost') }}` becomes `localhost`, unless you have set `RIVERSTONE_HOST`, in which case it becomes your value. Its two arguments are the variable's name and a default to use when the variable isn't set. `{{ env_var('RIVERSTONE_PASSWORD') }}` has no default on purpose: if you forget to set the password, dbt stops with an error instead of trying an empty one. Everything else in this chapter uses the same three markers.

Connecting, without putting a password in a file

`profiles.yml` tells dbt how to reach the database. dbt looks for it first in the folder you run dbt from, then in a `.dbt` folder in your home folder (dbt's documentation, "About profiles.yml", checked 28 September 2026). This project keeps it in the project folder, beside the code, and reads secrets from environment variables, exactly as Chapter 29's package did. Create a file named `profiles.yml` in the `riverstone_dbt` folder (code `profiles.yml` opens it in VS Code) and type:

```
# profiles.yml - how dbt connects. Keep the password out of this file and out of Git.
riverstone_dbt:
  target: dev
  outputs:
    dev:                                # the one-year database, for everyday work
      type: postgres
```

```

    host: "{{ env_var('RIVERSTONE_HOST', 'localhost') }}"
    port: 5432
    user: "{{ env_var('RIVERSTONE_USER', 'postgres') }}"
    password: "{{ env_var('RIVERSTONE_PASSWORD') }}"
    dbname: riverstone_2025
    schema: dbt_dev
    threads: 4
  large:          # the three-year database, for the incremental example
    type: postgres
    host: "{{ env_var('RIVERSTONE_HOST', 'localhost') }}"
    port: 5432
    user: "{{ env_var('RIVERSTONE_USER', 'postgres') }}"
    password: "{{ env_var('RIVERSTONE_PASSWORD') }}"
    dbname: riverstone_perf
    schema: dbt_large
    threads: 4
  ci:            # the throwaway database a CI run creates (section 32.12)
    type: postgres
    host: "{{ env_var('RIVERSTONE_HOST', 'localhost') }}"
    port: 5432
    user: "{{ env_var('RIVERSTONE_USER', 'postgres') }}"
    password: "{{ env_var('RIVERSTONE_PASSWORD') }}"
    dbname: riverstone_2025
    schema: dbt_ci
    threads: 4

```

In YAML the indentation is the structure (Chapter 26): each level is two spaces further in, and a line belongs to the nearest line above it that is indented less. Line by line:

- **riverstone_dbt:** names this **profile**. It must match the `profile:` line in `dbt_project.yml` (below); that's how the project finds its connection.
- **target:** `dev` is the connection used when you don't ask for another.
- **outputs:** lists the connections, called **targets**. There are three: `dev`, `large`, and `ci`.
- **type:** `postgres` says which adapter to use.
- **host, port, user, password:** where the server is and who logs in. 5432 is PostgreSQL's usual port (Chapter 34's ports).
- **dbname:** which database. `dev` and `ci` use `riverstone_2025`; `large` uses Chapter 28's three-year `riverstone_perf`, which section 32.10 uses to time an incremental build on 1.9 million rows.
- **schema:** `dbt_dev`: the schema dbt builds into, and the start of every schema name it creates (explained under the project file).
- **threads:** `4`: build up to four models at the same time, when the dependency graph allows it (Figure 32.1).

The same models run against any of the three targets. The `ci` target is for section 32.12, where an automated check builds the project in a fresh database.

Now give dbt the password. Chapter 34 warned that typing a password into an `export` command leaves it in your shell history, so keep it in a `.env` file that only you can read. Create `.env` in the `riverstone_dbt` folder containing one line, `export RIVERSTONE_PASSWORD='your-password'` (your PostgreSQL password from Chapter 12), then:

```
# terminal, in riverstone_dbt
$ chmod 600 .env
$ source .env
$ dbt debug
19:05:04 Running with dbt=1.12.5
19:05:04 dbt version: 1.12.5
19:05:04 python version: 3.14.7
19:05:04 adapter type: postgres
19:05:04 adapter version: 1.11.0
19:05:04 Configuration:
19:05:04   profiles.yml file [OK found and valid]
19:05:04   dbt_project.yml file [OK found and valid]
19:05:04 Required dependencies:
19:05:04   - git [OK found]
19:05:04 Registered adapter: postgres=1.11.0
19:05:04   Connection test: [OK connection ok]

19:05:04 All checks passed!
```

(Trimmed: the full output also lists file paths and every connection setting.)

`chmod 600` makes the file readable only by you (Chapter 34), `source .env` runs its `export` line in this shell, and `dbt debug` checks everything dbt needs: the two files, Git, and a real connection. `.env` is already in `.gitignore`, so the password never reaches Git. `profiles.yml` itself can go into Git, because it holds nothing secret; many teams keep it in `~/.dbt/` instead, so that credentials never sit next to code at all. Either way, **if a `profiles.yml` ever contains a password, it goes in `.gitignore`**. Run `source .env` again in each new terminal window.

The project file

`dbt init` wrote a `dbt_project.yml` full of comments. Replace its contents with this:

```
name: 'riverstone_dbt'
version: '1.0.0'
profile: 'riverstone_dbt'

model-paths: ["models"]
seed-paths: ["seeds"]
test-paths: ["tests"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

models:
  riverstone_dbt:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts
```

```
snapshots:
  riverstone_dbt:
    +schema: snapshots
```

Line by line:

- **name** is the project's name; **version** is your own version number for it, in quotes so YAML keeps 1.0.0 as text rather than trying to read a number; **profile** names the connection in `profiles.yml`.
- The five ***-paths** lines say which folder holds each kind of file. They're the defaults, written out so you can see them.
- Under **models:**, `riverstone_dbt:` means "this project's models", and `staging:` and `marts:` are **folder names** inside `models/`: they must match real folders. A word **with** a + in front, like `+materialized`, is a **setting**; a word without one is a folder.
- **+materialized** says what dbt builds from each model: a view in `staging`, a table in `marts` (section 32.6 explains why).
- **+schema** puts the folder's models in their own schema. dbt names it `<target schema>_<custom schema>` (dbt's documentation, "Custom schemas", checked 28 September 2026): with the `dev` target, staging builds into `dbt_dev_staging`, marts into `dbt_dev_marts`, and snapshots into `dbt_dev_snapshots`. Another target has another prefix (`dbt_ci_marts`), so environments never overwrite each other.

Settings apply to a folder and everything under it, and a single model can override them (section 32.5 shows how).

32.3 Models, sources, and ref

A **model** is a file containing one `SELECT` statement. dbt wraps it in `CREATE TABLE AS` or `CREATE VIEW AS`, using the file name as the object name. That's the whole idea: you write the query, dbt handles the object.

The CRM extract

Riverstone's customer details come from its CRM, whose nightly extract holds only today's values. Your database doesn't have that extract yet, so the companion file `ch32/setup_raw_crm.sql` makes a stand-in for it: a `raw_crm` schema holding a copy of the customers table, the way a loading tool would deliver it.

```
-- setup_raw_crm.sql - stands in for the CRM's nightly customer extract (Chapter 32).
-- It copies today's customer details into their own schema, the way a loading tool would.
CREATE SCHEMA raw_crm;

CREATE TABLE raw_crm.customers AS
SELECT customer_id, customer_name, city, segment
FROM public.customers;
```

`CREATE TABLE ... AS SELECT` makes a new table from a query's result. Copy the file into your `analytics` folder and run it once. In DBeaver, open it on a `riverstone_2025` connection and execute it as a script, as in Chapter 12. In the terminal, use PostgreSQL's command-line client, `psql`:

```
# terminal, in riverstone_dbt
$ export PGPASSWORD="$RIVERSTONE_PASSWORD"
$ psql -h localhost -U postgres -d riverstone_2025 -f ../setup_raw_crm.sql
CREATE SCHEMA
SELECT 24
```

- `PGPASSWORD` is the variable `psql` reads its password from. It knows nothing about `RIVERSTONE_PASSWORD`, so the first line copies one into the other; without it, `psql` stops and asks.
- `-h localhost` is the server (this machine), `-U postgres` the user, `-d riverstone_2025` the database, and `-f` the file of SQL to run. (`-c "..."` instead runs one statement typed in quotes; section 32.9 uses it.)
- `SELECT 24` is PostgreSQL saying it copied 24 customers.

On Windows, `psql` lives in `C:\Program Files\PostgreSQL\16\bin`; if Git Bash says `psql: command not found`, add that folder to `PATH` (Chapter 34) or use DBeaver.

Declaring where raw data comes from

Before building anything, tell dbt what the raw tables are. Create `models/staging` and `models/marts` (`mkdir models/staging models/marts`), then write `models/staging/sources.yml`:

```
version: 2

sources:
  - name: erp
    description: Riverstone's ERP tables, loaded into the reporting database every night.
    schema: public
    tables:
      - name: orders
        description: One row per order.
        columns:
          - name: order_id
            data_tests: [unique, not_null]
      - name: order_items
        description: One row per product line within an order.
        columns:
          - name: order_item_id
            data_tests: [unique, not_null]
      - name: products
      - name: employees

  - name: crm
    description: The CRM's nightly customer extract, which holds only current values.
    schema: raw_crm
    tables:
      - name: customers
```

YAML, part 2. Chapter 26 used YAML's basics: `key: value`, indentation, and lists written with `-`. dbt's files add three more patterns:

- **A list whose items are maps.** `- name: orders` starts one item of the `tables:` list, and every line indented under it (`description:`, `columns:`) belongs to that item. The next `- name:` at the same indentation starts the next item.
- **One-line forms.** `[unique, not_null]` is a two-item list written on one line, the same as two lines starting `-`. `{count: 24, period: hour}` is a map written on one line.
- **Quotes keep text as text.** `'1.0.0'` and `"3.12"` stay text; unquoted, YAML might read them as numbers.

If the indentation is wrong, dbt stops and names the file and line. VS Code's YAML extension underlines the mistake as you type.

Line by line:

- **version: 2** is the version of dbt's file format, not of your project; every properties file starts with it.
- **sources:** is a list of source systems. Each has a **name** (`erp`, `crm`), a **description**, the **schema** its tables are in, and a list of **tables**.
- Under a table, **columns:** lists columns you want to say something about, and **data_tests:** lists checks to run on them (section 32.7): here, that `order_id` is unique and `not_null` in the raw data itself.

Declaring sources buys three things: `{{ source('erp', 'orders') }}` instead of a hard-coded name, tests that run against the raw tables, and sources that appear in the lineage graph so everyone can see where the data entered.

Note what isn't there: a `database:` line. Leaving it out lets the same project run against `riverstone_2025` or `riverstone_perf` by switching target. Hard-coding the database is a mistake worth avoiding early, because it fails only when someone tries a second environment.

A first model

`models/staging/stg_orders.sql`:

```
-- One row per order, with cancelled orders flagged.
select
  order_id,
  customer_id,
  sales_rep_id,
  order_date,
  status,
  status <> 'Cancelled' as is_counted
from {{ source('erp', 'orders') }}
```

- `{{ source('erp', 'orders') }}` is replaced by the real table name (`"riverstone_2025"."public"."orders"`): the source named `erp` in `sources.yml`, table `orders`.

- **status <> 'Cancelled' as is_counted** makes a true/false column: the comparison is worked out for each row and stored as a **boolean**. If **status** were ever NULL, the comparison would give NULL, and the fact table's where **o.is_counted** would drop that order silently; the **not_null** test on **status** in section 32.7 guards against that.

Net revenue needs a rule, and the rule should exist in exactly one place. Create `macros/net_revenue.sql` with these four lines now; section 32.11 explains how a macro works:

```
{# The one definition of net revenue in the whole project (Chapter 13's rule). #}
{% macro net_revenue(quantity, unit_price, discount_pct) -%}
    round({{ quantity }} * {{ unit_price }} * (1 - {{ discount_pct }} / 100.0), 2)
{% endmacro %}
```

For now, read it as "wherever a model writes `{{ net_revenue(...) }}`, put this arithmetic". Then `models/staging/stg_order_items.sql`, which uses it:

```
-- One row per order line, with net revenue and cost computed once, here, for everyone.
select
    oi.order_item_id,
    oi.order_id,
    oi.product_id,
    oi.quantity,
    oi.unit_price,
    oi.discount_pct,
    {{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }} as net_revenue,
    oi.quantity * p.unit_cost as product_cost
from {{ source('erp', 'order_items') }} as oi
join {{ source('erp', 'products') }} as p on p.product_id = oi.product_id
```

The join to `products` fetches `unit_cost`, one row per product, so it can't change the number of rows (section 32.4). And `models/staging/stg_products.sql`:

```
-- One row per product. The ERP's unit_price is the list price, so staging gives it that name.
select
    product_id,
    product_name,
    category,
    unit_price as list_price,
    unit_cost
from {{ source('erp', 'products') }}
```

Renaming is one of staging's jobs: `unit_price as list_price` gives the column a clearer business name, and if the ERP ever renames it, this is the one place to change.

ref: the function that makes dbt work

Models refer to each other with `{{ ref('model_name') }}`, never by table name. Riverstone's fact model, in section 32.5, starts:

```
from {{ ref('stg_order_items') }} as oi
join {{ ref('stg_orders') }} as o on o.order_id = oi.order_id
```

ref does two jobs at once:

1. **It resolves to the right name at run time.** In your development target it becomes `"riverstone_2025"."dbt_dev_staging"."stg_orders"`; in the `ci` target, `dbt_ci_staging`. You never write a schema name in a model.
2. **It builds the dependency graph.** Because `fct_sales_line` refs `stg_orders`, dbt knows `stg_orders` must be built first. Nobody maintains a build order.

See it for yourself. `dbt compile` renders the Jinja without running anything; `--select` picks which model:

```
# terminal, in riverstone_dbt
$ dbt compile --select stg_order_items
19:05:33 Running with dbt=1.12.5
19:05:33 Registered adapter: postgres=1.11.0
19:05:33 Unable to do partial parsing because saved manifest not found. Starting full parse.
19:05:34 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any
resources.
There are 2 unused configuration paths:
- models.riverstone_dbt.marts
- snapshots.riverstone_dbt
19:05:34 Found 3 models, 4 data tests, 5 sources, 478 macros
19:05:34
19:05:34 Concurrency: 4 threads (target='dev')
19:05:34
Compiled node 'stg_order_items' is:
-- One row per order line, with net revenue and cost computed once, here, for everyone.
select
  oi.order_item_id,
  oi.order_id,
  oi.product_id,
  oi.quantity,
  oi.unit_price,
  oi.discount_pct,
  round(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100.0), 2) as net_revenue,
  oi.quantity * p.unit_cost as product_cost
from "riverstone_2025"."public"."order_items" as oi
join "riverstone_2025"."public"."products" as p on p.product_id = oi.product_id
```

Read the lines before the SQL too. "Starting full parse" means dbt read every file in the project for the first time. The warning is harmless: `dbt_project.yml` has settings for `marts` and `snapshots`, and neither has any files yet. "Found 3 models, 4 data tests, 5 sources" counts what you've written: three staging models, the four tests in `sources.yml`, and five source tables. Then the compiled model: the macro has become arithmetic and the sources have become table names. **Everything dbt runs is SQL you can read**, and it's all written to `target/compiled/` when anything surprises you.

32.4 Layers: staging, intermediate, marts

dbt projects converge on the same three layers, because each answers a different question.

Layer	One model per	Job	Materialized as
staging	source table	rename, cast, clean, compute row-level values. No joins that change the grain	view
intermediate	a step in a calculation	the awkward middle of a complicated transformation, split so each piece is readable and testable	view or ephemeral
marts	business concept	the tables people query: facts and dimensions, at a declared grain	table

Riverstone's project once sections 32.5 to 32.10 are done:

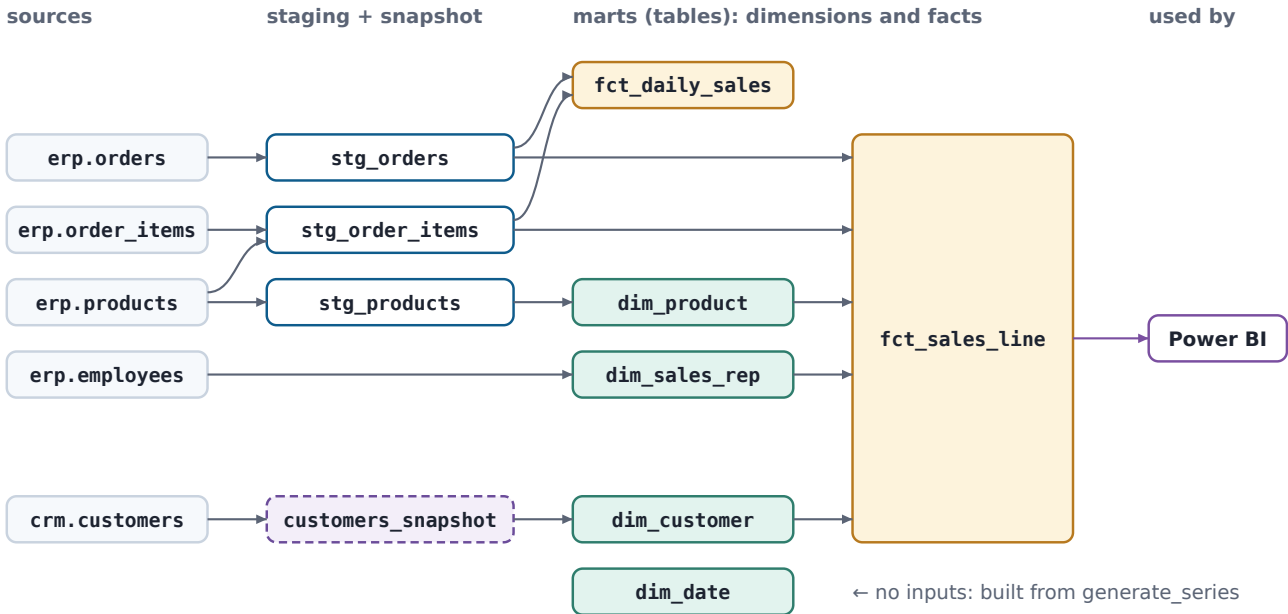
```
models/
├── staging/
│   ├── sources.yml           where raw data comes from, and its tests
│   ├── schema.yml          tests and descriptions for the staging models
│   ├── stg_order_items.sql
│   ├── stg_orders.sql
│   └── stg_products.sql
├── marts/
│   ├── schema.yml          tests and descriptions for the marts
│   ├── dim_customer.sql
│   ├── dim_date.sql
│   ├── dim_product.sql
│   ├── dim_sales_rep.sql
│   ├── fct_daily_sales.sql
│   └── fct_sales_line.sql
├── snapshots/
│   └── customers_snapshot.yml  history of the CRM's customers
├── tests/
│   └── assert_fact_total_matches_source.sql
├── macros/
│   └── net_revenue.sql
```

Naming conventions matter more than they look: `stg_` for staging, `dim_` and `fct_` for marts, `int_` for intermediate. A reader can tell what a model is from its name, and so can a reviewer looking at a pull request.

Rules of thumb that keep projects readable:

- **One source table, one staging model**, and everything downstream refs the staging model, never the source. If the ERP renames a column, exactly one file changes.
- **Don't join in staging** unless the join can't change the grain. `stg_order_items` joins products only to fetch `unit_cost`, one row per product, which can't fan out (Chapter 28's grain discipline, now enforced by habit).
- **Marts are wide and friendly**. Dimensions carry every attribute a report might slice by, spelled out in business language.
- **If a mart model is getting hard to read, split it** into intermediate models. Views cost nothing to keep.

The graph dbt builds from the `ref()`, `source()` and `snapshot` calls



Each arrow points from a model to the model that uses it. Nobody wrote this order down: dbt reads the calls in the SQL, builds the graph, and runs it in dependency order, four models at a time (threads: 4).

Figure 32.1 – Sources on the left, marts on the right; each arrow points from a model to the model that uses it. dbt derives this graph from the `ref`, `source`, and `snapshot` calls in the files. The customer dimension is built from the snapshot of the CRM extract (section 32.5), and the ERP's employees table feeds `dim_sales_rep` directly for now (exercise 5 adds its staging model).

32.5 Building the star schema as models

Chapter 28's dimensions become four short models, and the fact table a fifth. The date dimension is pure SQL with no source at all. `models/marts/dim_date.sql`:

```

{{ config(materialized='table') }}
-- One row per calendar day of 2025 (Chapter 28's date dimension).
with days as (
    select generate_series(date '2025-01-01', date '2025-12-31', interval '1 day')::date as
        full_date
)
select
    to_char(full_date, 'YYYYMMDD')::int as date_key,
    full_date,
    trim(to_char(full_date, 'Day')) as day_name,
    date_trunc('month', full_date)::date as month_start,
    to_char(full_date, 'YYYY-"Q"Q') as quarter_label,
    extract(isodow from full_date) in (6, 7) as is_weekend
from days
  
```

- `{{ config(materialized='table') }}` sets options for this model alone, overriding `dbt_project.yml`. `dim_date` is in `marts`, so it's already a table; the line is here to show the syntax, and exercise 9 changes it to a view.
- `generate_series(...)` makes one row per day from 1 January to 31 December (Chapter 13's date spine); `::date` turns each timestamp into a date.

- `to_char(full_date, 'YYYYMMDD')::int` turns 2025-03-14 into the number 20250314, Chapter 28's date key.
- `to_char(full_date, 'YYYY-"Q"Q')` prints 2025-Q1: text in double quotes is copied as it is, and the second Q is the quarter number.
- `extract(isodow from full_date)` numbers the days Monday 1 to Sunday 7, so 6 and 7 are the weekend.

The product dimension is a rename of a staging model, `models/marts/dim_product.sql`:

```
select
    product_id as product_key,
    product_id,
    product_name,
    category,
    list_price,
    unit_cost
from {{ ref('stg_products') }}
```

It reads `stg_products` through `ref`, so dbt builds the staging view first. `product_id` appears twice: once as `product_key`, the dimension's key in Chapter 28's style, and once as itself, for joining back to the source. `list_price` is the name staging gave `unit_price`.

The sales rep dimension, `models/marts/dim_sales_rep.sql`, adds the "no rep recorded" row that Chapter 28 argued for, so no order is dropped by an inner join:

```
select
    employee_id as sales_rep_key,
    employee_id,
    employee_name as rep_name,
    job_title
from {{ source('erp', 'employees') }}
union all
select -1, null, 'No rep recorded', 'Unknown'
```

The first `select` gives one row per employee, with `employee_id` as the key; `union all` adds a sixth row, key -1, for orders with no rep. This breaks section 32.4's rule on purpose: there is no `stg_employees` yet, so the dimension reads the source directly. Exercise 5 adds the staging model and shows the lineage graph change.

The customer dimension needs history: a snapshot

Chapter 28 built a type 2 customer dimension by reconstructing history from an audit log. Most source systems don't keep one: the CRM holds today's values and overwrites yesterday's. A dbt **snapshot** solves that by recording what a table looked like each time it runs.

`snapshots/customers_snapshot.yml`:

```
snapshots:
- name: customers_snapshot
  relation: source('crm', 'customers')
  config:
    unique_key: customer_id
    strategy: check
    check_cols: [city, segment]
```

- **relation:** is the table to watch: the CRM extract.
- **config:** says how to watch it. **unique_key** identifies the business entity, as in Chapter 28.
strategy: check compares the columns in **check_cols** with the stored version and writes a new row when any of them differ. The alternative, **strategy: timestamp**, uses an **updated_at** column and is cheaper when the source maintains one reliably.
- dbt adds **dbt_valid_from**, **dbt_valid_to**, and **dbt_scd_id** to every row. **dbt_scd_id** is a key for one version of one customer: a long hashed text value rather than Chapter 28's 1, 2, 3, but it does the same job.

(Snapshots have been configured in YAML like this since dbt 1.9; older projects define them in SQL files with a `{% snapshot %}` block. dbt's documentation, "Add snapshots to your DAG", checked 28 September 2026.)

The dimension model then reads the snapshot, `models/marts/dim_customer.sql`:

```
-- A type 2 customer dimension, built from the snapshot dbt maintains (Chapter 28, section 28.10).
-- dbt records history from the first snapshot onwards, so the first version of each customer is
-- opened far enough in the past to cover the orders we already hold.
with versions as (
    select
        dbt_scd_id as customer_key,
        customer_id,
        customer_name,
        city,
        segment,
        dbt_valid_from,
        dbt_valid_to,
        row_number() over (partition by customer_id order by dbt_valid_from) as version_number
    from {{ ref('customers_snapshot') }}
)
select
    customer_key,
    customer_id,
    customer_name,
    city,
    segment,
    case when version_number = 1 then timestamp '1900-01-01' else dbt_valid_from end as valid_from,
    coalesce(dbt_valid_to, timestamp '9999-12-31') as valid_to,
    dbt_valid_to is null as is_current
from versions
```

How it works:

- **ref('customers_snapshot')** reads the snapshot's table, and puts the snapshot before this model in the graph.
- **row_number()** numbers each customer's versions in date order (Chapter 13's window functions).
- The first version's start is moved back to **1900-01-01**. The snapshot's own start date is the day you first run it, long after every 2025 order; without this, no order would find a customer. Section 32.9 comes back to what that costs.
- A current version has no **dbt_valid_to**, so **coalesce** gives it Chapter 28's far-future end date, and **is_current** is true exactly when **dbt_valid_to** is NULL.

The fact table

The fact table, `models/marts/fct_sales_line.sql`, joins the staging models to the dimensions, with the point-in-time join to the customer dimension that Chapter 28 introduced:

```
-- Grain: one non-cancelled order line.
select
    oi.order_item_id,
    oi.order_id,
    o.order_date,
    to_char(o.order_date, 'YYYYMMDD')::int as date_key,
    c.customer_key,
    p.product_key,
    coalesce(r.sales_rep_key, -1) as sales_rep_key,
    oi.quantity,
    oi.net_revenue,
    oi.product_cost
from {{ ref('stg_order_items') }} as oi
join {{ ref('stg_orders') }} as o on o.order_id = oi.order_id
join {{ ref('dim_customer') }} as c
    on c.customer_id = o.customer_id
    and o.order_date >= c.valid_from::date
    and o.order_date < c.valid_to::date
join {{ ref('dim_product') }} as p on p.product_id = oi.product_id
left join {{ ref('dim_sales_rep') }} as r on r.employee_id = o.sales_rep_id
where o.is_counted
```

How it works:

- The grain is one row per `order_item_id`, from `stg_order_items`; every other join adds columns, never rows.
- `to_char(o.order_date, 'YYYYMMDD')::int` makes the `date_key` that matches `dim_date`.
- The join to `dim_customer` picks the customer version in effect on the order date, exactly as in Chapter 28, and stores its `customer_key`.
- `where o.is_counted` keeps only non-cancelled orders, using the flag from `stg_orders`.

It follows Chapter 28's fact table line for line, with two differences. It keeps `order_date` beside `date_key`, which exercise 12 needs. And instead of `IS NOT DISTINCT FROM`, it `left joins` the reps and uses `coalesce(..., -1)` to send an order with no rep to the "No rep recorded" row. This model has no config line: the project's default for marts, a table, is what it needs.

Running it

```
# terminal, in riverstone_dbt
$ dbt build
19:05:53 Running with dbt=1.12.5
19:05:53 Registered adapter: postgres=1.11.0
19:05:54 Found 8 models, 4 data tests, 1 snapshot, 5 sources, 478 macros
19:05:54
19:05:54 Concurrency: 4 threads (target='dev')
19:05:54
19:05:54 2 of 13 START sql table model dbt_dev_marts.dim_sales_rep ..... [RUN]
19:05:54 4 of 13 START snapshot dbt_dev_snapshots.customers_snapshot ..... [RUN]
19:05:54 1 of 13 START sql table model dbt_dev_marts.dim_date ..... [RUN]
19:05:55 3 of 13 START sql view model dbt_dev_staging.stg_products ..... [RUN]
19:05:55 3 of 13 OK created sql view model dbt_dev_staging.stg_products ..... [CREATE VIEW in
0.25s]
19:05:55 2 of 13 OK created sql table model dbt_dev_marts.dim_sales_rep ..... [SELECT 6 in 0.28s]
19:05:55 1 of 13 OK created sql table model dbt_dev_marts.dim_date ..... [SELECT 365 in 0.28s]
```



```

19:05:55 5 of 13 START test source_not_null_erp_order_items_order_item_id ..... [RUN]
19:05:55 6 of 13 START test source_not_null_erp_orders_order_id ..... [RUN]
19:05:55 4 of 13 OK snapshot dbt_dev_snapshots.customers_snapshot ..... [SELECT 24 in 0.28s]
19:05:55 7 of 13 START test source_unique_erp_order_items_order_item_id ..... [RUN]
19:05:55 8 of 13 START test source_unique_erp_orders_order_id ..... [RUN]
19:05:55 6 of 13 PASS source_not_null_erp_orders_order_id ..... [PASS in 0.10s]
19:05:55 9 of 13 START sql table model dbt_dev_marts.dim_product ..... [RUN]
19:05:55 7 of 13 PASS source_unique_erp_order_items_order_item_id ..... [PASS in 0.10s]
19:05:55 5 of 13 PASS source_not_null_erp_order_items_order_item_id ..... [PASS in 0.13s]
19:05:55 10 of 13 START sql table model dbt_dev_marts.dim_customer ..... [RUN]
19:05:55 11 of 13 START sql view model dbt_dev_staging.stg_order_items ..... [RUN]
19:05:55 8 of 13 PASS source_unique_erp_orders_order_id ..... [PASS in 0.07s]
19:05:55 12 of 13 START sql view model dbt_dev_staging.stg_orders ..... [RUN]
19:05:55 9 of 13 OK created sql table model dbt_dev_marts.dim_product ..... [SELECT 8 in 0.08s]
19:05:55 10 of 13 OK created sql table model dbt_dev_marts.dim_customer ..... [SELECT 24 in 0.08s]
19:05:55 11 of 13 OK created sql view model dbt_dev_staging.stg_order_items ..... [CREATE VIEW in
0.08s]
19:05:55 12 of 13 OK created sql view model dbt_dev_staging.stg_orders ..... [CREATE VIEW in
0.07s]
19:05:55 13 of 13 START sql table model dbt_dev_marts.fct_sales_line ..... [RUN]
19:05:55 13 of 13 OK created sql table model dbt_dev_marts.fct_sales_line ..... [SELECT 326 in 0.04s]
19:05:55
19:05:55 Finished running 1 snapshot, 5 table models, 4 data tests, 3 view models in 0 hours 0 minutes and 0.76
seconds (0.76s).
19:05:55
19:05:55 Completed successfully
19:05:55
19:05:55 Done. PASS=13 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=13

```

(The times, and the timings in brackets, will differ on your machine; so may the order of lines that run at the same time.)

How to read it:

- **"Found 8 models, 4 data tests, 1 snapshot, 5 sources"**: three staging models and five marts, the four tests in `sources.yml`, and the snapshot. The 13 **nodes** (models, tests, snapshots) are numbered "1 of 13" to "13 of 13".
- **START** and **OK** lines come in pairs. With four threads, up to four nodes run at once, which is why the numbers arrive out of order. `fct_sales_line` starts last, because everything it refs had to finish first. Nobody told dbt that; it read the `ref` calls.
- The bracket says what the database reported: **SELECT 326** means 326 rows went into the table, and **CREATE VIEW** means a view was made (views hold no rows).
- **PASS=13**: every node succeeded. **ERROR** and **SKIP** count failures and the nodes skipped because of them.

dbt build is the command to learn. It runs models, tests, snapshots, and seeds together, in dependency order, and **stops a branch of the graph when something upstream fails**, so a failed test doesn't let bad data through to the models below it. `dbt run` builds without testing, and `dbt test` tests without building; `build` is what you want on a schedule.

The result is a schema of tables you can query like any other, in DBeaver or `psql`:

```

SELECT COUNT(*) AS fact_rows,
       ROUND(SUM(net_revenue)) AS net_revenue,
       COUNT(DISTINCT customer_key) AS customer_versions
FROM dbt_dev_marts.fct_sales_line;

```

fact_rows	net_revenue	customer_versions
326	4335471	23

(1 row)

326 lines and ₹43,35,471: the same numbers Chapter 28's hand-built star schema produced, which is the point. What changed is that the build order, the tests, and the documentation now come from the code itself. The 23 customer versions are 23 customers with one version each (the 24th, Home Plus, placed no orders in 2025), where Chapter 28's dimension has 27 rows for the same 24 customers.

```
SELECT d.quarter_label,
       c.segment,
       ROUND(SUM(f.net_revenue)) AS net_revenue
FROM dbt_dev_marts.fct_sales_line AS f
JOIN dbt_dev_marts.dim_date AS d ON d.date_key = f.date_key
JOIN dbt_dev_marts.dim_customer AS c ON c.customer_key = f.customer_key
GROUP BY d.quarter_label, c.segment
ORDER BY d.quarter_label, c.segment;
```

quarter_label	segment	net_revenue
2025-Q1	Hospitality	149040
2025-Q1	Retail	394328
2025-Q1	Wholesale	190944
2025-Q2	Hospitality	252170
2025-Q2	Retail	198474
2025-Q2	Wholesale	275925
2025-Q3	Hospitality	307278
2025-Q3	Retail	333973
2025-Q3	Wholesale	479039
2025-Q4	Hospitality	435551
2025-Q4	Retail	562000
2025-Q4	Wholesale	756751

(12 rows)

The twelve cells add up to ₹43,35,473, two rupees more than the total, because each cell was rounded on its own.

The totals match Chapter 28; the history doesn't

Chapter 28 rebuilt three customers' 2025 history from the ERP's audit log: Metro Mart moved from Thane to Mumbai on 1 July, for instance, so its January to June orders belong to Thane. The snapshot has no such log. It starts from today's values, and the dimension stretches each customer's first version back to 1900, so every Metro Mart order shows as Mumbai. This query puts the two builds side by side.

```

SELECT 'Chapter 28 star' AS built_by, c.city, ROUND(SUM(f.net_revenue)) AS net_revenue
FROM dw.fact_sales_line AS f
JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
WHERE c.customer_id = 5
GROUP BY c.city
UNION ALL
SELECT 'dbt project', c.city, ROUND(SUM(f.net_revenue))
FROM dbt_dev_marts.fct_sales_line AS f
JOIN dbt_dev_marts.dim_customer AS c ON c.customer_key = f.customer_key
WHERE c.customer_id = 5
GROUP BY c.city
ORDER BY built_by, city;

```

```

    built_by    | city | net_revenue
-----+-----+-----
Chapter 28 star | Mumbai |      177258
Chapter 28 star | Thane |      152040
dbt project     | Mumbai |      329298
(3 rows)

```

The first half reads Chapter 28's `dw` tables and the second the dbt marts, both for customer 5; `UNION ALL` stacks the two results, and the `built_by` column labels which is which.

Watch out: totals match; breakdowns don't. For Patel Kitchenware, Metro Mart, and Harbour Traders, results by city or segment differ from Chapter 28's. Section 32.9 explains why, and what to do about it.

32.6 Materializations: view, table, incremental, ephemeral

A **materialization** is how dbt turns a model's `SELECT` into something in the database. Four cover almost everything.

Materialization	dbt creates	Build cost	Query cost	Use for
view	a view	almost none	the full query, every time	staging models, small transformations
table	a table, rebuilt each run	full rebuild	fast	marts, anything queried often
incremental	a table, updated with new rows	only the new rows	fast	large fact tables (section 32.10)
ephemeral	nothing: the SQL is pasted into the models that ref it	nothing stored	its SQL runs inside every model that uses it	small intermediate steps you don't want cluttering the warehouse

Four materializations, and what each one costs

materialization	what dbt creates, and what it suits	cost to build, and to query
view	creates: a view; nothing is stored suits: staging models	build: almost free query: the full query, every time
table	creates: a table, rebuilt each run suits: marts	build: a full rebuild query: fast
incremental	creates: a table; later runs add new rows suits: large fact tables	build: new rows only (0.55 s, not 2.19 s) query: fast
ephemeral	creates: nothing; its SQL is pasted in suits: small intermediate steps	build: nothing stored query: runs inside each model that uses it

Set the default per folder in `dbt_project.yml`; override it in one model with `{{ config(materialized='...') }}`.

Figure 32.2 – The choice is a trade between build cost and query cost.

The defaults in `dbt_project.yml` (views for staging, tables for marts) are the usual starting point, and the reasoning is worth keeping:

- **Staging as views** means no storage and no staleness: a view always reflects the raw table underneath. Since marts are built on top and materialized as tables, the view is read once per run, not once per dashboard.
- **Marts as tables** means the expensive joins happen once per run, not once per person who opens a report.

Switch one and see what happens if you change it. Before you run it, predict what the bracket at the end of the line will say. Add `{{ config(materialized='view') }}` as the first line of `dim_product.sql`, then build just that model: `dbt run` builds without testing, and `--select dim_product` picks the one model.

```
# terminal, in riverstone_dbt
$ dbt run --select dim_product
19:06:20 1 of 1 START sql view model dbt_dev_marts.dim_product ..... [RUN]
19:06:20 1 of 1 OK created sql view model dbt_dev_marts.dim_product ..... [CREATE VIEW in
0.08s]
```

Now delete the `config` line and run the same command again:

```
# terminal, in riverstone_dbt
$ dbt run --select dim_product
19:06:23 1 of 1 START sql table model dbt_dev_marts.dim_product ..... [RUN]
19:06:23 1 of 1 OK created sql table model dbt_dev_marts.dim_product ..... [SELECT 8 in
0.13s]
```

(Both trimmed to the lines about the model; the rest is the same header and summary as before.)

The first run replaced the table with a view: `CREATE VIEW`, and no row count, because a view stores none. The second put the table back: `SELECT 8` is PostgreSQL telling you eight rows were written.

Watch out: a table is a snapshot of the moment it was built. Everything in a mart is as fresh as the last dbt build, which is why teams put "data as of" on dashboards (Chapter 28's staleness warning) and why the schedule matters as much as the models. If a number must be live to the second, it doesn't belong in a mart.

32.7 Tests: the part that earns the trust

A dbt **test** is a query that should return **no rows**. If it returns rows, something is wrong, and dbt tells you how many.

Built-in tests

Four generic tests cover most of what goes wrong, and they're declared in YAML beside the models. `models/staging/schema.yml` tests the staging models:

```
version: 2

models:
  - name: stg_orders
    columns:
      - name: order_id
        data_tests: [unique, not_null]
      - name: status
        data_tests: [not_null]
  - name: stg_order_items
    columns:
      - name: order_item_id
        data_tests: [unique, not_null]
      - name: net_revenue
        data_tests: [not_null]
  - name: stg_products
    columns:
      - name: product_id
        data_tests: [unique, not_null]
```

It has the same shape as `sources.yml`, with `models:` in place of `sources:`: a list of models, each with the columns to test. `models/marts/schema.yml` tests the marts, and uses the two tests that take settings:

```
version: 2

models:
  - name: fct_sales_line
    description: "Grain: one non-cancelled order line. The single source for sales reporting."
    columns:
      - name: order_item_id
        data_tests: [unique, not_null]
      - name: customer_key
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_customer')
                field: customer_key
      - name: date_key
```

```

    data_tests:
      - relationships:
          arguments:
            to: ref('dim_date')
            field: date_key
      - name: net_revenue
        data_tests: [not_null]
- name: dim_customer
  description: "Type 2 customer dimension: one row per customer per period their details stayed the
    same."
  columns:
    - name: customer_key
      data_tests: [unique, not_null]
    - name: segment
      data_tests:
        - accepted_values:
            arguments:
              values: ['Retail', 'Hospitality', 'Wholesale']
- name: dim_date
  columns:
    - name: date_key
      data_tests: [unique, not_null]

```

- **unique** and **not_null** are grain and completeness checks: exactly Chapter 28's habit of testing what one row represents.
- **relationships** is a foreign key that the warehouse doesn't enforce: every **customer_key** in the fact table must exist in the dimension. This is the test that catches a dimension load that silently dropped rows. It needs settings, so it's written as a list item that is itself a map: **arguments:** holds them, **to:** names the other model and **field:** its column. (Before dbt 1.10.5, the settings sat directly under the test name without **arguments:**; you'll see both in older projects. dbt's documentation, "data_tests", checked 28 September 2026.)
- **accepted_values** catches new categories appearing without warning, which is how "Retail" quietly becomes "retail " and a dashboard loses a segment. Its one setting, **values**, is the list of allowed values.
- **description** is documentation, used in section 32.8.

More live in packages: **dbt_utils** adds tests for expressions, row counts between two models, recency, and mutually exclusive ranges; **dbt_expectations** ports the Great Expectations catalog (Chapter 47).

A custom test: the reconciliation

Generic tests check shapes. The test that protects the business most directly is the one that checks a number against something outside the model. Chapter 28 reconciled the star schema to the source by hand; here it runs on every build. The test is a file in the **tests/** folder.

```
-- tests/assert_fact_total_matches_source.sql
-- The reconciliation from Chapter 28: the fact table must total what the source says, to the
  paisa.
-- A dbt test passes when it returns no rows.
with fact as (select sum(net_revenue) as total from {{ ref('fct_sales_line') }}),
source_total as (
  select sum({{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }}) as total
  from {{ source('erp', 'order_items') }} as oi
  join {{ source('erp', 'orders') }} as o on o.order_id = oi.order_id
  where o.status <> 'Cancelled'
)
select fact.total as fact_total, source_total.total as source_total
from fact, source_total
where abs(fact.total - source_total.total) > 0.01
```

Any `.sql` file in `tests/` is a **singular test**: it passes when the query returns nothing. This one returns a row only when the fact table and the source disagree by more than a paisa. Its source side reuses the `net_revenue` macro, so it checks the joins and filters (a dropped line, a cancelled order let through), not the revenue formula itself; answer 8 shows how to close that gap with a total from outside the project.

Run `dbt build` again, and the 20 new tests run too: "Found 8 models, 24 data tests" and `PASS=33`.

What a failure looks like

Suppose someone "tidies up" `fct_sales_line` and turns the last line, where `o.is_counted`, into a comment, so cancelled orders creep in. The `+` after the model's name selects it *and everything downstream of it*:

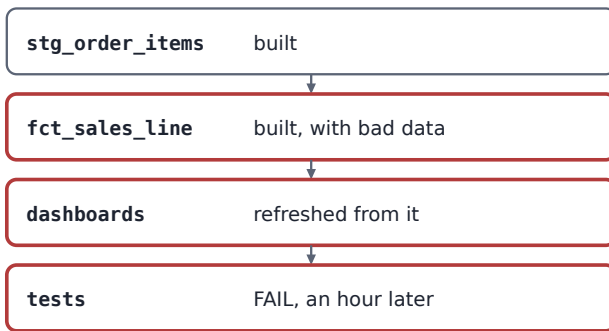
```
# terminal, in riverstone_dbt
$ dbt build --select fct_sales_line+
19:06:44 1 of 8 START sql table model dbt_dev_marts.fct_sales_line ..... [RUN]
19:06:44 1 of 8 OK created sql table model dbt_dev_marts.fct_sales_line ..... [SELECT 330 in
  0.09s]
19:06:44 2 of 8 START test assert_fact_total_matches_source ..... [RUN]
19:06:44 2 of 8 FAIL 1 assert_fact_total_matches_source ..... [FAIL 1 in 0.10s]
19:06:45 Completed with 1 error, 0 partial successes, and 0 warnings:
19:06:45
19:06:45 [ERROR]: in test assert_fact_total_matches_source (tests/assert_fact_total_matches_source.sql)
19:06:45   Got 1 result, configured to fail if != 0
19:06:45
19:06:45   compiled code at target/compiled/riverstone_dbt/tests/assert_fact_total_matches_source.sql
19:06:45
19:06:45 Done. PASS=7 WARN=0 ERROR=1 SKIP=0 NO-OP=0 REUSED=0 TOTAL=8
$ echo "exit code $?"
exit code 1
```

(Trimmed: the header, and the lines for the six tests that passed.)

Three things to notice. `SELECT 330`: the four lines of the two cancelled orders got in, and only the reconciliation noticed; every shape test passed. The build **failed** with exit code 1, so a scheduler sees it and raises an alarm (Chapter 29's exit codes, Chapter 34's `$?`). And the message names the **path to the compiled SQL**, so you can run the failing query yourself and look at the row. Put the line back and the same command ends `PASS=8`.

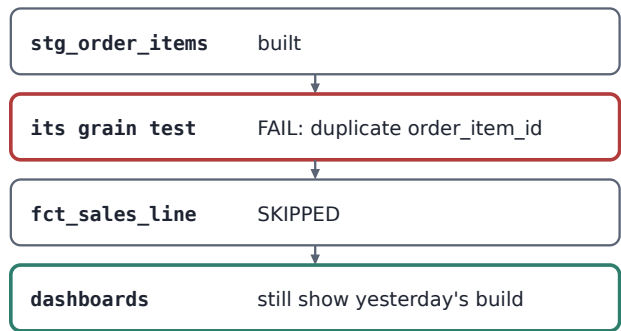
Why dbt build, and not dbt run followed by dbt test

✗ dbt run, then dbt test



✗ Bad numbers reached people before anyone knew.

✓ dbt build



✓ The failure stopped the branch: nothing wrong shipped.

dbt build runs each model and then its tests, in dependency order, and skips everything downstream of a failure. The run ends with a non-zero exit code, so the scheduler raises an alarm (Chapter 29's exit codes).

Figure 32.3 – `dbt build` stops a branch of the graph the moment a test fails, so bad data doesn't reach the models below it.

How much to test

Test	Where	Why
unique + not_null on the grain	every model	the single most common failure
relationships on every key in a fact table	marts	catches incomplete dimension loads
accepted_values on categories reports group by	marts	catches new or misspelled values
a reconciliation to the source	one per mart	catches everything else
freshness on sources (dbt source freshness)	sources	catches a load that didn't run

A useful discipline: **when a number is wrong in production, add the test that would have caught it** before you fix the model. The project's test suite then grows out of real failures rather than good intentions.

32.8 Documentation and lineage

Descriptions live in the same YAML as the tests, next to the thing they describe. That's the only way documentation stays true: if it isn't beside the code, it drifts.

```
# terminal, in riverstone_dbt
$ dbt docs generate
19:07:08 Found 8 models, 24 data tests, 1 snapshot, 5 sources, 478 macros
19:07:08 Building catalog
$ dbt docs serve
Serving docs at 8080
To access from your browser, navigate to: http://localhost:8080

Press Ctrl+C to exit.
```

(Trimmed: the usual header lines, and a last line giving the full path of the `catalog.json` file it wrote into `target/`.)

`dbt docs generate` reads the project and the database's own list of tables and columns, and writes a small website into `target/`. `dbt docs serve` starts a web server on your own machine and opens the site in your browser: `localhost` is this computer and `8080` is the port the server listens on (Chapter 34). The terminal stays busy while it serves; press `Ctrl+C` to stop it.

The generated site gives you, for every model: its description, every column with its description and tests, the compiled SQL, and an interactive **lineage graph** showing what it depends on and what depends on it. The graph isn't drawn by anyone; it's the `ref` calls.

What it's actually for, day to day:

- "Where does this number come from?" Click the column, read the description, open the SQL.
- "If I change this source, what breaks?" Look downstream in the graph.
- "Is this metric the same as that one?" Two models both calling `{{ net_revenue(...) }}` are the same by construction; two different formulas are visible as two different formulas.

Two habits make it worth the small effort: describe a model in **one sentence that states its grain** (Chapter 28), and describe any column whose meaning someone could get wrong. Don't describe `order_id` as "the order id".

32.9 Snapshots: history dbt keeps for you

Section 32.5 set the snapshot up, and its first run stored one version of each of the 24 customers. Its value shows the next time the source changes. `dbt snapshot` runs only the snapshots in the project.

Watch it work. Suppose the CRM reclassifies Evergreen Mart, a Retail customer in Indore, as Wholesale. Make that change in the stand-in extract (in DBeaver, or with `psql`, whose `-c` runs the one statement in quotes), then take a new snapshot:

```
# terminal, in riverstone_dbt
$ psql -h localhost -U postgres -d riverstone_2025 -c "UPDATE raw_crm.customers SET segment = 'Wholesale' WHERE
customer_id = 18"
UPDATE 1
$ dbt snapshot
19:07:29 Found 8 models, 24 data tests, 1 snapshot, 5 sources, 478 macros
19:07:29 1 of 1 START snapshot dbt_dev_snapshots.customers_snapshot ..... [RUN]
19:07:29 1 of 1 OK snapshot dbt_dev_snapshots.customers_snapshot ..... [INSERT 0 1 in 0.24s]
19:07:29 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=1
$ dbt build --select dim_customer+
19:07:32 1 of 12 OK created sql table model dbt_dev_marts.dim_customer ..... [SELECT 25 in 0.09s]
19:07:32 5 of 12 OK created sql table model dbt_dev_marts.fct_sales_line ..... [SELECT 326 in 0.04s]
19:07:32 Done. PASS=12 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=12
```

(Trimmed to the lines that matter; `PGPASSWORD` is still set from section 32.3, or `psql` would ask for the password.)

INSERT 0 1: the snapshot closed Evergreen Mart's old version and added one new row. `dim_customer` now has 25 rows, and the fact table still has 326 lines, because every 2025 order falls in the first version. Evergreen Mart's versions:

```
SELECT customer_id, customer_name, segment,
       valid_from::date AS valid_from, valid_to::date AS valid_to, is_current
FROM dbt_dev_marts.dim_customer
WHERE customer_id = 18
ORDER BY valid_from;
```

customer_id	customer_name	segment	valid_from	valid_to	is_current
18	Evergreen Mart	Retail	1900-01-01	2026-09-28	f
18	Evergreen Mart	Wholesale	2026-09-28	9999-12-31	t

(2 rows)

Two versions, one closed and one current, built by a tool instead of by the careful `UPDATE`-then-`INSERT` transaction of Chapter 28. The change date is the day the snapshot ran, so yours will be the day you run it. That is the snapshot's whole nature: it records *when it noticed* a change, not when the change happened.

Watch out: a snapshot only knows what it has seen. dbt records history from the first time the snapshot runs, not before. Riverstone's first versions therefore start on the day snapshotting began, which would leave every 2025 order without a matching customer version, so `dim_customer` opens each customer's first version in 1900. That is why section 32.5's Metro Mart orders all show as Mumbai: the move from Thane happened in July 2025, long before the first snapshot, and nothing recorded it. Two lessons: **start snapshotting the day you think you might ever want history**, and when a source system does keep an audit log (Chapter 28's `customer_changes`), reconstructing from it gives you history you never captured.

Watch out: snapshots are precious. The snapshot table is the only record of what the source used to say. Never point a snapshot at a different source, never run it against a temporary copy of the data, and back it up like any other irreplaceable table.

32.10 Incremental models

Rebuilding a mart every run is fine until the fact table has millions of rows. An **incremental** model builds fully the first time, and afterwards processes only what's new. Riverstone's daily summary is a good candidate.

```
-- models/marts/fct_daily_sales.sql
{{ config(materialized='incremental', unique_key='order_date', incremental_strategy='delete+insert') }}
-- One row per day: rebuilt only for days that changed since the last run.
select
    o.order_date,
    count(distinct o.order_id) as orders,
    sum(oi.net_revenue) as net_revenue
from {{ ref('stg_orders') }} as o
join {{ ref('stg_order_items') }} as oi on oi.order_id = o.order_id
where o.is_counted
{% if is_incremental() %}
    and o.order_date >= (select coalesce(max(order_date), date '1900-01-01') from {{ this }})
{% endif %}
group by o.order_date
```

How it works:

- **materialized='incremental'** tells dbt to create the table on the first run, and to insert into it afterwards.
- **is_incremental()** is true only when the table already exists and the run isn't a full refresh. The **and** line inside the **{% if %}** limits the work to recent days; on the first run, dbt leaves it out.
- **{{ this }}** is the model's own table, so the filter can ask "what's the latest day I already have?".
- **incremental_strategy='delete+insert'** with a **unique_key** replaces the days it rebuilds instead of duplicating them. That matters because the newest day is usually incomplete: it gets rebuilt tomorrow. Other strategies are **append** (fastest, no deduplication) and **merge** (an upsert, on warehouses that support it).

Add its grain test to `models/marts/schema.yml`, at the end of the `models:` list, and build it with `dbt build --select fct_daily_sales` (the model and its two tests, PASS=3):

```
- name: fct_daily_sales
  columns:
    - name: order_date
      data_tests: [unique, not_null]
```

On 175 orders the saving is invisible. On Chapter 28's three-year database (1,926,847 order lines, 1,096 days) it isn't. Measured on the machine used to test this chapter (four processor cores, 16 GB of memory):

```
# terminal, in riverstone_dbt
$ dbt run --target large --select +fct_daily_sales --full-refresh
19:07:51 1 of 3 OK created sql view model dbt_large_staging.stg_order_items ..... [CREATE VIEW in
0.14s]
19:07:51 2 of 3 OK created sql view model dbt_large_staging.stg_orders ..... [CREATE VIEW in
0.14s]
19:07:53 3 of 3 OK created sql incremental model dbt_large_marts.fct_daily_sales ..... [SELECT 1096 in
2.19s]
19:07:53 Finished running 1 incremental model, 2 view models in 0 hours 0 minutes and 2.50 seconds (2.50s).
$ dbt run --target large --select fct_daily_sales
19:07:57 1 of 1 OK created sql incremental model dbt_large_marts.fct_daily_sales ..... [INSERT 0 1 in
0.55s]
19:07:57 Finished running 1 incremental model in 0 hours 0 minutes and 0.68 seconds (0.68s).
```

(Trimmed to the result lines. Timings vary from run to run and machine to machine; two more runs gave 2.16 and 2.25 seconds for the full build, and 0.50 and 0.51 for the incremental one.)

- `--target large` uses the `large` connection in `profiles.yml`, so the same models build into `riverstone_perf`.
- `+fct_daily_sales`, with the plus in front, selects the model **and everything it depends on**, here its two staging views (a plus after the name, as in section 32.7, selects everything that depends on it).
- `--full-refresh` rebuilds an incremental table from scratch, as if it had never existed.

The full build wrote all 1,096 days (`SELECT 1096`). The second run found the table already there, rebuilt only the latest day, and inserted one row (`INSERT 0 1`). **2.19 seconds against 0.55**: about four times faster here, and the gap grows with the table, because the full build reads every line while the incremental one writes only the newest days. On a cloud warehouse billed by compute, that ratio is the bill.

When to make a model incremental: when a full rebuild is slow enough to hurt, and rows arrive rather than change. **When not to:** small models (the complexity isn't worth it), and models whose history changes (a customer dimension where old rows get corrected). The rule of thumb is to start with a table and convert when the run time bothers you.

Watch out: incremental models drift. Late-arriving data, back-dated corrections, and changed business logic all leave an incremental table holding rows built by yesterday's rules. Defenses: a **lookback window** (`>= max(order_date) - interval '3 days'`) rather than a hard `>=`, a scheduled `--full-refresh` (weekly is common), and a test that compares the model's totals against a full recomputation. If you change the model's logic, refresh it fully; dbt will not do it for you.

32.11 Macros: write a rule once

A **macro** is a reusable piece of SQL written in Jinja. You created Riverstone's only one in section 32.3, `macros/net_revenue.sql`:

```
{# The one definition of net revenue in the whole project (Chapter 13's rule). #}
{% macro net_revenue(quantity, unit_price, discount_pct) -%}
    round({{ quantity }} * {{ unit_price }} * (1 - {{ discount_pct }} / 100.0), 2)
{% endmacro %}
```

How it works:

- `{% macro net_revenue(quantity, unit_price, discount_pct) %} ... {% endmacro %}` defines a macro with a name and three **arguments**, like a Python function (Chapter 17).
- Inside, `{{ quantity }}` is replaced by whatever was passed as `quantity`. A model calls `{{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }}`, with each argument **in quotes**, because the macro works on text: it pastes the text `oi.quantity` into the SQL, where the database then reads it as a column. Without the quotes, Jinja would look for a variable called `oi` and fail.
- The `-` inside `-%}` and `{%-` trims the blank lines and spaces next to the tag, so the compiled SQL shows the arithmetic on one line (section 32.3's compile output) rather than surrounded by empty lines.

- It is Chapter 13's rule, now **rounded to the paisa on each line** with `round(..., 2)`. Chapter 13's `sales_lines` view doesn't round, so a total built from this macro can differ from `sales_lines` by a few paise. The reconciliation test is safe because both of its sides use the macro; a test against `sales_lines` itself would need a tolerance of about a rupee.

The value isn't saving keystrokes; it's that Chapter 13's argument about whose revenue number is right can only have one answer, and changing the rule means changing one file, reviewed in one pull request.

The three markers are the ones from section 32.2's "Jinja in two minutes". A loop earns its keep when a model repeats itself. `models/marts/mart_sales_monthly.sql` pivots revenue by category, without typing each category:

```
-- One row per month, with revenue split by product category.
{% set categories = ['Storage', 'Kitchen', 'Industrial', 'Furniture'] %}

select
    d.month_start,
    {% for category in categories %}
    sum(case when p.category = '{{ category }}' then f.net_revenue else 0 end)
      as revenue_{{ category | lower }}{% if not loop.last %}
    {% endfor %}
from {{ ref('fct_sales_line') }} as f
join {{ ref('dim_product') }} as p on p.product_key = f.product_key
join {{ ref('dim_date') }} as d on d.date_key = f.date_key
group by d.month_start
```

Before you compile it, write down what you expect the lines between `d.month_start`, and `from` to become. Then:

```
# terminal, in riverstone_dbt
$ dbt compile --select mart_sales_monthly
Compiled node 'mart_sales_monthly' is:
-- One row per month, with revenue split by product category.

select
    d.month_start,

    sum(case when p.category = 'Storage' then f.net_revenue else 0 end)
      as revenue_storage,

    sum(case when p.category = 'Kitchen' then f.net_revenue else 0 end)
      as revenue_kitchen,

    sum(case when p.category = 'Industrial' then f.net_revenue else 0 end)
      as revenue_industrial,

    sum(case when p.category = 'Furniture' then f.net_revenue else 0 end)
      as revenue_furniture

from "riverstone_2025"."dbt_dev_marts"."fct_sales_line" as f
join "riverstone_2025"."dbt_dev_marts"."dim_product" as p on p.product_key = f.product_key
join "riverstone_2025"."dbt_dev_marts"."dim_date" as d on d.date_key = f.date_key
group by d.month_start
```


(Trimmed: the header lines.)

Each construct, in one line:

- `{% set categories = [...] %}` stores a list of four names in a Jinja variable.
- `{% for category in categories %} ... {% endfor %}` repeats the lines between them once per name, with `category` holding the current one.
- `'{{ category }}'` puts the name inside SQL quotes: `'Storage'`.
- `{{ category | lower }}` passes the name through the **filter** `lower`, giving `storage`, so the column is `revenue_storage`.
- `{{ ", " if not loop.last }}` writes a comma after every column except the last: `loop.last` is true on the final pass, and `", " if ...` is Jinja's one-line if.
- The empty lines are where the `{% %}` tags stood. They're harmless; the `-` marks from the macro would remove them.

A better version reads the category list from the warehouse itself with `dbt_utils.get_column_values`, so a new category appears in the report the day it appears in the data.

Packages bring other people's macros. A `packages.yml` file in the project folder lists them, and `dbt deps` downloads them into `dbt_packages/`:

```
packages:
- package: dbt-labs/dbt_utils
  version: [">=1.3.0", "<2.0.0"]
```

`version: [">=1.3.0", "<2.0.0"]` means any 1.x release from 1.3.0 on: new fixes arrive, but not a 2.0 that might change how things work. Riverstone's project doesn't need a package yet; exercise 13 adds this one. `dbt_utils` is the one nearly every project uses: `date_spine` (Chapter 13's date spine, as a macro), `star`, `generate_surrogate_key` (called `surrogate_key` before `dbt_utils` 1.0), `pivot`, and a shelf of extra tests. Check the current version on the dbt package hub when you add it.

Watch out: Jinja is a power tool. A model that generates SQL through three nested loops is unreadable, un-lintable, and impossible for the next analyst to debug. The test: could a colleague work out what this produces without running it? If not, write the SQL out.

32.12 Linting and continuous integration

SQLFluff

Chapter 13 promised automatic formatting for teams. **SQLFluff** is the SQL linter and formatter most dbt projects use; it understands dbt's Jinja through a **templater**, a separate package that compiles each model with dbt first, so it lints the SQL rather than choking on `{{ ref(...) }}`. Add both to the environment from the `analytics` folder with `uv add sqlfluff sqlfluff-templater-dbt`, as you added dbt. The configuration goes in a file named `.sqlfluff` in the `riverstone_dbt` folder.


```
# .sqlfluff - settings for the SQL linter
[sqlfluff]
templater = dbt
dialect = postgres
max_line_length = 120

[sqlfluff:templater:dbt]
project_dir = ./
profiles_dir = ./

[sqlfluff:rules:capitalisation.keywords]
capitalisation_policy = lower
```

- **[sqlfluff]** holds the main settings: use the **dbt** templater, read the SQL as PostgreSQL's **dialect**, and allow lines up to 120 characters.
- **[sqlfluff:templater:dbt]** tells the templater where the project and `profiles.yml` are: `./`, this folder.
- **[sqlfluff:rules:capitalisation.keywords]** sets one rule: keywords such as `select` and `join` in lower case, the style this chapter uses.

Before you run it on the fact model, guess how many complaints a linter could have about SQL that works:

```
# terminal, in riverstone_dbt
$ sqlfluff lint models/marts/fct_sales_line.sql
=== [dbt templater] Sorting Nodes...
=== [dbt templater] Compiling dbt project...
=== [dbt templater] Project Compiled.
== [models/marts/fct_sales_line.sql] FAIL
L:   2 | P:   1 | ST06 | Select wildcards then simple targets before calculations
      |       |       | and aggregates. [structure.column_order]
L:  14 | P:   1 | AM05 | Join clauses should be fully qualified. [ambiguous.join]
L:  14 | P:  38 | ST09 | Joins should list the table referenced earlier first.
      |       |       | [structure.join_condition_order]
L:  15 | P:   1 | AM05 | Join clauses should be fully qualified. [ambiguous.join]
L:  16 | P:   1 | LT02 | Expected indent of 4 spaces. [layout.indent]
L:  16 | P:   9 | LT02 | Expected line break and indent of 8 spaces before 'c'.
      |       |       | [layout.indent]
L:  17 | P:   1 | LT02 | Expected indent of 8 spaces. [layout.indent]
L:  18 | P:   1 | LT02 | Expected indent of 8 spaces. [layout.indent]
L:  18 | P:  24 | LT01 | Expected only single space before naked identifier.
      |       |       | Found '  '. [layout.spacing]
L:  19 | P:   1 | AM05 | Join clauses should be fully qualified. [ambiguous.join]
All Finished!
```

Ten. **L:** is the line and **P:** the position in it; then the rule's code and message. The five rules:

- **ST06:** put plain columns before calculated ones (the `select` on line 2 has `to_char(...)` and `coalesce(...)` among plain columns).
- **AM05:** write `inner join`, not a bare `join`, so the kind of join is never left to the reader's memory.
- **ST09:** in a join condition, write the table that appeared earlier first: `oi.order_id = o.order_id`.
- **LT02:** indent in steps of four spaces, with each condition of a multi-line `on` on its own line.
- **LT01:** one space between words; line 18 lined up `<` with two spaces.

sqlfluff fix makes the fixes it can make safely, and says so:

```
# terminal, in riverstone_dbt
$ sqlfluff fix models/marts/fct_sales_line.sql
==== finding fixable violations ====
== [models/marts/fct_sales_line.sql] FIXED
10 fixable linting violations found
$ sqlfluff lint models/marts/fct_sales_line.sql
All Finished!
```

(Trimmed: `fix` lists the same ten violations before "FIXED", and both commands print the templater's three lines.)

All ten were mechanical, and the fixed model runs the same query: the calculated columns moved to the end of the `select`, each `join` became `inner join`, and the join conditions were turned round and re-indented. (The companion project keeps the model as section 32.5 prints it, so you can run the lint yourself.) Two pieces of advice from teams who have adopted SQLFluff: **turn rules off rather than arguing about them** (the point is consistency, not any particular style), and **run fix across the project in one commit** when you adopt it, so the first pull request afterwards isn't an unreadable mess of whitespace changes.

Running everything on every pull request

Chapter 26 introduced continuous integration: automated checks on every change. For a dbt project the checks are obvious, and this workflow runs them. It goes in `.github/workflows/dbt.yml`, as Chapter 26's did:

```
# Runs on every pull request: install, lint, then build and test the whole project against a scratch
  database.
name: dbt
on: [pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: ${ secrets.CI_DATABASE_PASSWORD }
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready --health-interval 10s --health-timeout 5s --health-retries 5
    env:
      RIVERSTONE_HOST: localhost
      RIVERSTONE_USER: postgres
      RIVERSTONE_PASSWORD: ${ secrets.CI_DATABASE_PASSWORD }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install dbt-core dbt-postgres sqlfluff sqlfluff-templater-dbt
      - run: psql -h localhost -U postgres -c "CREATE DATABASE riverstone_2025" && ./load_test_data.sh
      - run: sqlfluff lint models/
      - run: dbt deps
      - run: dbt build --target ci
```

The parts Chapter 26's workflow didn't have:

- **services: postgres:** asks GitHub to start a fresh PostgreSQL 16 for this run, next to the job, and throw it away afterwards. It runs in a **container**, a small self-contained machine (Chapter 52 explains them); you only need to know it's a real, empty PostgreSQL.
- **ports: ["5432:5432"]** makes it reachable at `localhost:5432`, the same address as on your machine.
- **options: >-** passes settings that make the job wait until the database is ready. In YAML, `>-` means "the indented lines below are one long value".
- **{{ secrets.CI_DATABASE_PASSWORD }}** is GitHub's own template syntax, unrelated to dbt's `{{ }}` despite the look. It inserts a **secret** stored in the repository's settings: on GitHub, open the repository, then *Settings* → *Secrets and variables* → *Actions* → *New repository secret*, name it `CI_DATABASE_PASSWORD`, and paste a password. For a throwaway test database, a password written in the file would do no harm; the book uses a secret anyway, because the habit is the point.
- The job's **env:** sets the variables `profiles.yml` reads, and **--target ci** builds with the `ci` connection from section 32.2, into `dbt_ci_...` schemas.
- **pip install** rather than `uv`: the runner starts empty each time, and `pip` is already on it. (`astral-sh/setup-uv` is an alternative that installs `uv` first.)

What it buys. A pull request that breaks a test cannot be merged unnoticed. A model that doesn't compile is caught before it reaches production. And the reviewer's job shrinks to the part a human is good at: *is this the right business rule?*

Two refinements real teams add:

- **Build only what changed**, with dbt's state comparison: `dbt build --select state:modified+ --state ./prod-artifacts`. On a project with hundreds of models, rebuilding everything on every pull request is too slow.
- **Build into a separate schema per pull request**, so reviewers can query the proposed models without touching production.

The finished project

With every section's files in place, one command builds and tests the whole project, exactly as the workflow does:

```
# terminal, in riverstone_dbt
$ dbt build
19:09:09 Running with dbt=1.12.5
19:09:09 Registered adapter: postgres=1.11.0
19:09:10 Found 10 models, 26 data tests, 1 snapshot, 5 sources, 478 macros
19:09:10
19:09:10 Concurrency: 4 threads (target='dev')
19:09:10
19:09:10 3 of 37 START sql view model dbt_dev_staging.stg_products ..... [RUN]
...
19:09:11 37 of 37 OK created sql table model dbt_dev_marts.mart_sales_monthly ..... [SELECT 12 in 0.03s]
19:09:11
19:09:11 Finished running 1 incremental model, 1 snapshot, 6 table models, 26 data tests, 3 view models in 0
hours 0 minutes and 1.22 seconds (1.22s).
19:09:11
19:09:11 Completed successfully
19:09:11
19:09:11 Done. PASS=37 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=37
```

(Trimmed: the full output has a START and a result line for each of the 37 nodes.)

The count checks out: 10 models (3 staging, 7 marts), 1 snapshot, and 26 tests (4 on sources, 8 on staging, 13 on marts, 1 singular) make 37 nodes.

Where dbt runs in production

dbt Core is a command-line tool: something has to run it on a schedule. The options, in rough order of how much they cost you:

Option	What it is
cron on a small server (Chapter 34)	the simplest thing that works
An orchestrator: Airflow, Dagster, Prefect (Chapter 46)	dbt as one task among ingestion and downstream jobs
A container on a cloud scheduler	no server to maintain
dbt Cloud	dbt Labs' hosted product: scheduling, a browser IDE, CI, hosted docs, on a paid plan with a limited free tier

The models, tests, and docs are identical either way. Check current dbt Cloud plans and prices on dbt Labs' site; they change, and this chapter was written against dbt Core 1.12.

32.13 What dbt is not, and the semantic layer

dbt only transforms data that's already in the warehouse. It does **not** extract or load (that's Fivetran, Airbyte, or your own Python, Chapter 46), does **not** schedule itself, is **not** a BI tool, and is **not** a database. A dbt project on a warehouse that nothing loads is a rota for an empty kitchen.

The one boundary that keeps moving is the **semantic layer**: the idea that metric definitions (revenue, active customer, churn) should live in one place that *every* tool asks, rather than being re-implemented in each dashboard. dbt has a semantic layer built on MetricFlow, where you define metrics in YAML alongside models and BI tools query them through an API. Competing approaches exist (Cube, Looker's LookML, and BI tools' own modeling layers), and this area changes fast enough that any specific instruction here would be stale by the time you read it.

The idea, though, is stable, and you already have it: Riverstone's `net_revenue` macro is a one-metric semantic layer. If a metric's definition lives in one file, is tested, and everything downstream refs it, you have most of the benefit. If it lives in six dashboards, no product will save you. Check dbt's documentation for the current state of the semantic layer before adopting it, and Chapter 16 for how BI tools consume a mart.

Common mistakes

Mistake	Symptom	Fix
Hard-coded table names instead of <code>ref</code>	Models build in the wrong order; the lineage graph is wrong	<code>{{ ref(...) }}</code> and <code>{{ source(...) }}</code> , always
database: pinned in <code>sources.yml</code>	Works in dev, fails in every other environment	Let the target decide the database
Credentials in <code>profiles.yml</code> in Git	A warehouse password in the repository forever	Environment variables or a secrets manager
Models that duplicate a business rule	Two dashboards, two revenue numbers	One macro, refed everywhere
Joins in staging that change the grain	Row counts inflate downstream and nobody knows where	Keep staging one-row-per-source-row; join in marts
No tests on the grain	Duplicate keys reach the fact table	<code>unique</code> and <code>not_null</code> on every model's key
Tests only on shapes, never on numbers	Everything passes while the total is wrong	A reconciliation test against the source
<code>dbt run</code> on a schedule instead of <code>dbt build</code>	Bad data ships because tests ran after, or not at all	<code>dbt build</code> , which stops the branch on failure
Ignoring a failing test for a few days	The test gets deleted, and the trust with it	Fix or explicitly change the rule, in a reviewed pull request
Everything materialized as a table	Slow builds, warehouse bills	Views for staging, tables for marts, incremental for the big ones
Incremental models never fully refreshed	Old rows built by old logic	Scheduled <code>--full-refresh</code> , and after any logic change
An incremental filter with no lookback	Late-arriving rows never appear	A few days of overlap plus <code>delete+insert</code> on a unique key
Snapshot pointed at a changing source path	History silently lost	Treat snapshot tables as irreplaceable
Documentation written in a wiki	Stale within a month	Descriptions in the YAML beside the model
Jinja loops generating clever SQL	Nobody can debug it, including you in March	Write the SQL out; save Jinja for repetition that is truly mechanical
No CI	Broken models found in production	Build and test on every pull request
One enormous mart model	Impossible to test or review	Split into intermediate models; views are free

In the real world: the two dashboards that disagreed

Anita's dashboard says December revenue was ₹4,39,824. The finance pack says ₹4,47,000. The gap is ₹7,176 and nobody can explain it, so both numbers stop being used and the argument is settled by whoever speaks last in the meeting.

Meera spends a morning on it. The dashboard reads Chapter 13's `sales_lines` view, which excludes cancelled orders. The finance pack reads a spreadsheet built from a different export, which includes them. Neither is wrong: they are answers to different questions, and nobody wrote either question down.

The fix isn't a better argument, it's a place for the answer to live. Over two weeks she builds the dbt project in this chapter:

- **net_revenue becomes a macro.** One definition, used by staging and by the reconciliation test.
- **fct_sales_line states its grain** in its description: one non-cancelled order line. Cancelled orders keep their own model, so the finance question has an answer too, and the two differ by a number anyone can query.
- **The reconciliation test** compares the fact total to the source on every build, so drift is caught the morning it appears, not in a meeting.
- **The lineage graph** shows the dashboard reading `fct_sales_line`, and `fct_sales_line` reading `stg_order_items`, and that reading the ERP's `order_items`. The chain is on a page.
- **Everything runs in CI** on every pull request, so changing the revenue rule is a reviewed decision with a name and a date attached.

Three months later the ERP team renames a column in `order_items`. The nightly build fails at 03:14 with a clear message, one staging model is changed and reviewed the next morning, and nothing downstream needed touching. Before dbt, that rename would have surfaced as a blank dashboard tile at 9 a.m., traced by hand over a day.

What Meera tells Anita: *"The two numbers were answering different questions, and now both questions have a model with its definition written down. The dashboard will keep matching finance, because a test checks it every night and the build fails if it stops."*

Notice that dbt didn't settle the argument. Writing the definition down did. dbt made writing it down the cheapest option, and kept it honest afterwards.

Project: a tested mart that feeds a dashboard

Goal: a dbt project someone else could clone, build, and trust, ending in a mart a BI tool reads.

Tools you'll need

Versions used for this chapter, checked in September 2026:

- **dbt Core 1.12.5** with **dbt-postgres 1.11.0**, installed with `uv add dbt-core dbt-postgres`. dbt releases often: check `dbt --version` against the documentation, and re-read the release notes before upgrading a project.
- **PostgreSQL 16**, holding `riverstone_2025` and `riverstone_perf`.
- **SQLFluff 4.3.0** with **sqlfluff-templater-dbt** (the templater is a separate package, and forgetting it produces a confusing error).
- **Git** (Chapter 26) and a CI service; the example is GitHub Actions.

- **Adapters** for other warehouses: `dbt-snowflake`, `dbt-bigquery`, `dbt-databricks`, `dbt-redshift`, `dbt-duckdb`, `dbt-sqlserver`. The project changes very little between them; the SQL dialect does.
- **Packages** worth knowing: `dbt_utils` (macros and extra tests), `dbt_expectations` (a larger test catalog), `codegen` (generates boilerplate YAML from your warehouse), `elementary` (observability, Chapter 47).
- **Python 3.14**, which `uv` picked for the `analytics` environment (Chapter 17's recommendation).
- **Companion files** in `ch32/`: the complete `riverstone_dbt` project, including its CI workflow and `.sqlfluff`; `setup_raw_crm.sql` (section 32.3); and `pyproject.toml` with `requirements-pinned.txt`, the exact versions every output in this chapter came from.

Option A: your own warehouse. Pick one report that matters and rebuild its inputs as a dbt project. Work against a development schema, never production, and keep credentials in environment variables.

Option B: Riverstone. Build the project in this chapter from an empty folder, without copying the companion files, then compare.

Steps:

1. **Set it up:** `dbt init`, a `profiles.yml` reading secrets from the environment, `dbt debug` passing, and the whole thing in Git with `target/` and `.env` ignored.
2. **Declare your sources**, with `unique` and `not_null` on their keys, and `dbt source freshness` configured for at least one.
3. **Build staging models**, one per source table, materialized as views.
4. **Build the marts:** at least one fact table with a grain stated in its description, and two dimensions.
5. **Test it:** grain tests on every model, `relationships` on every key in the fact table, `accepted_values` on one category, and one **reconciliation test** against the source.
6. **Break something on purpose** and confirm the build fails and the message tells you where.
7. **Add a macro** for one business rule, and use it in at least two places.
8. **Add a snapshot** for a dimension that changes, and prove it captures a change.
9. **Make one model incremental** and time a full refresh against an incremental run.
10. **Document it:** descriptions for every model and for columns whose meaning isn't obvious. Generate the docs and read your own lineage graph.
11. **Add CI** that lints and builds on every pull request, and open one pull request that fails on purpose.
12. **Point a BI tool at the mart** (Chapter 16) and confirm the numbers match your reconciliation.

Deliverables: the repository, the docs site (a screenshot of the lineage graph is fine), the output of a passing `dbt build` and of a failing one, and a half-page note on which tests you chose and why.

Stretch goals:

- Add `dbt_utils` and replace one hand-written test with `expression_is_true` or `equal_rowcount`.
- Add a second target and prove the same models build against both.
- Convert the fact table to incremental with a three-day lookback, and write a test that compares it with a full recomputation.
- Add `dbt build --select state:modified+` to CI so only changed models rebuild.

REVIEW

Recap

- **Analytics engineering** brings software habits (version control, tests, review, documentation) to SQL transformation. **ELT** put that transformation inside the warehouse, and **dbt** is the tool most teams use for it.
- A **model** is one `SELECT` in a file. **source** declares raw tables; **ref** connects models, resolves names per environment, and **builds the dependency graph** dbt runs in order.
- Projects layer into **staging** (one model per source table, views, grain preserved), optional **intermediate**, and **marts** (facts and dimensions, tables).
- **Materializations**: view, table, incremental, ephemeral. Views for staging, tables for marts, incremental when a rebuild hurts.
- **Tests** return no rows when they pass. `unique`, `not_null`, `relationships`, and `accepted_values` cover shapes; a **singular test** reconciling to the source covers the numbers. **dbt build** runs models and tests together and stops the branch when one fails.
- **Documentation** and the **lineage graph** are generated from the YAML and the `ref` calls, so they can't drift.
- **Snapshots** keep type 2 history from a source that overwrites, but only from the day they start running.
- **Incremental models** process new rows only: 2.19 seconds against 0.55 on 1.9 million lines here. Use a **lookback window**, a **unique key**, and a scheduled **full refresh**.
- **Jinja** turns dbt's files into templates: `{{ }}` for values, `{% %}` for instructions, `{# #}` for comments. **Macros** define a business rule once. **Packages** such as `dbt_utils` bring proven macros and tests.
- **SQLFluff** lints; **CI** builds and tests every pull request; something else (cron, an orchestrator, or dbt Cloud) runs dbt on a schedule.
- dbt transforms; it does not extract, load, schedule, or visualize.

Key terms

analytics engineering · ETL · ELT · dbt Core · dbt Cloud · adapter · profile · target · `dbt_project.yml` · `profiles.yml` · `env_var` · model · source · ref · `source()` · `config()` · Jinja · compiled SQL · DAG (dependency graph) · staging layer · intermediate layer · mart · materialization · view · table · incremental model · ephemeral model · `is_incremental()` · `{{ this }}` · incremental strategy · lookback window · full refresh · generic test · singular test · unique · not_null · relationships · `accepted_values` · source freshness · `dbt build` · `dbt run` · `dbt test` · node selection (+, `state:modified`) · snapshot · `dbt_valid_from` · `dbt_valid_to` · `dbt_scd_id` · check strategy · timestamp strategy · macro · package · `dbt_utils` · docs site · lineage graph · SQLFluff · templater · linting · continuous integration · service container · repository secret · semantic layer · MetricFlow

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can explain ELT, and why transformation moved into the warehouse.
- ☐ I can set up a dbt project whose credentials never appear in the code.
- ☐ Every model of mine uses `ref` and `source`, and I can explain both jobs `ref` does.
- ☐ I lay projects out in staging and marts, and I keep the grain in staging.
- ☐ I choose materializations deliberately and can say what each costs.
- ☐ My models carry grain tests, relationship tests, and at least one reconciliation.
- ☐ I run `dbt build`, not `dbt run`, on a schedule, and I know why.
- ☐ I can read a failing test's output and get to the offending rows.
- ☐ I write descriptions beside the models and can navigate my own lineage graph.
- ☐ I can keep type 2 history with a snapshot, and I know what a snapshot can't recover.
- ☐ I can make a model incremental, with a lookback, and measure what it saves.
- ☐ I write a business rule once, as a macro, and lint and build on every pull request.

PRACTISE

Exercises

Work in the `riverstone_dbt` project, against the `dev` target. Predict each answer before you run it.

Warm-up

1. Run `dbt ls` and `dbt ls --select staging`. What do the outputs tell you about how models are addressed?
2. Compile `fct_sales_line` and read `target/compiled/.../fct_sales_line.sql`. Which parts were written by you, and which by dbt?
3. Run `dbt build --select stg_orders+` and explain, from the output, what the `+` did.
4. Query `dbt_dev_marts.fct_daily_sales`. How many rows does it have, and why that number?

Core

5. Add a `stg_employees` staging model, with `unique` and `not_null` tests on `employee_id`, and change `dim_sales_rep` to ref it instead of the source. Does the lineage graph change?
6. Add an `accepted_values` test on `stg_orders.status`, run it, then add a new status to the source data and watch it fail.
7. Write a singular test asserting that no order line has a negative `net_revenue`, and one asserting that every non-cancelled order in `stg_orders` has at least one line in `fct_sales_line`.
8. Change the `net_revenue` macro to round to zero decimal places. Which models change, and which test catches the consequence?
9. Materialize `dim_date` as a view instead of a table, rebuild, and compare the run output. When would that be the right choice?
10. Add descriptions for every column of `fct_sales_line`, regenerate the docs, and check the column-level lineage.

Stretch

11. Write a model `int_orders_with_totals` as an **ephemeral** materialization, use it in `fct_daily_sales`, and look at the compiled SQL. Where did the model go?
12. Convert `fct_sales_line` to incremental, keyed on `order_item_id`, with a three-day lookback on `order_date`. Then write a test that compares its total against a full recomputation.
13. Add `dbt_utils` to the project and replace the reconciliation test with `dbt_utils.equal_rowcount` plus an expression test. Which version would you rather maintain?
14. Section 32.11's `mart_sales_monthly` pivots revenue by category with a Jinja loop. Write the same model without Jinja, as `mart_sales_monthly_plain`, and compare the two compiled files. Which would you put in the repository?
15. Configure `dbt source freshness` on `erp.orders` with a warn-after of 24 hours and an error-after of 48, and make it fail.

Think about it (no code needed)

16. A colleague wants the dashboard to read `stg_order_items` directly, "because it has everything and it's fresher". Give two reasons to say no, and one case where they'd be right.
17. Your project has 300 models and `dbt build` takes 40 minutes. List four things you'd look at, in order.
18. When would you not use dbt for a transformation?

Answers

1. `dbt ls` lists every node in the project: sources, models, snapshots, and tests, one per line: models as `project.folder.name (riverstone_dbt.staging.stg_orders)`, sources as `source:riverstone_dbt.erp.orders`. `dbt ls --select staging` lists only what's under `models/staging`. That naming is how **node selection** works everywhere: `--select stg_orders` (one model), `stg_orders+` (it and everything downstream), `+fct_sales_line` (it and everything upstream), `tag:nightly`, `source:erp+`, `state:modified+`. Selection is the skill that makes a 300-model project workable.

2. Everything between `select` and the final `from` is yours. dbt replaced `{{ source('erp', 'order_items') }}` with the fully qualified table name, and expanded the `net_revenue` macro into arithmetic. Nothing else was added. The compiled file is your query exactly as the database will run it; when the model is built, dbt wraps it in `create table ... as` and saves that version in `target/run/`, which is why reading it settles most "why is this model doing that?" questions.

3. The `+` selects the model **and everything downstream of it**: `stg_orders`, then the marts that ref it, then the tests on all of them. The output shows the fact models and their tests running after the staging view. Use it before opening a pull request: it proves that a change to one model doesn't break anything below it. `+stg_orders` (the plus in front) would select its upstream instead, which is what you want when a source is misbehaving.

4.

```
SELECT COUNT(*) AS days,
       MIN(order_date) AS first_day,
       MAX(order_date) AS last_day,
       ROUND(SUM(net_revenue)) AS net_revenue
FROM dbt_dev_marts.fct_daily_sales;
```

```
days | first_day | last_day | net_revenue
-----+-----+-----+-----
132 | 2025-01-02 | 2025-12-23 | 4335471
(1 row)
```

One row per day that had at least one non-cancelled order line, which is fewer than 365 because these key accounts don't order every day, and the last order in the source is dated 23 December (Chapter 13's date-spine problem). The total matches `fct_sales_line`, as it must: both come from the same staging models. If you need a row for every calendar day, join to `dim_date` rather than changing this model.

5. The model is short:

```
-- models/staging/stg_employees.sql
select
  employee_id,
  employee_name,
  job_title,
  manager_id
from {{ source('erp', 'employees') }}
```

with tests in `models/staging/schema.yml`, and `dim_sales_rep` changed to `from {{ ref('stg_employees') }}`. The lineage graph gains a node: `erp.employees` → `stg_employees` → `dim_sales_rep` instead of the source feeding the dimension directly. That's the point of the staging layer: one place where the ERP's column names are translated, so a rename upstream is a one-file change.

6.

```
# in models/staging/schema.yml, under stg_orders
- name: status
  data_tests:
    - accepted_values:
      arguments:
        values: ['Delivered', 'Shipped', 'Pending', 'Cancelled']
```

It passes. Now insert an order with status `'Returned'` into the source and run `dbt test --select stg_orders`: the test fails with the count of offending rows and the path to the compiled query, which lists the unexpected value. This is the test that catches a source system quietly adding a category, which otherwise shows up as a dashboard whose segments no longer add to the total.

7.

```
-- tests/assert_no_negative_revenue.sql
select order_item_id, net_revenue
from {{ ref('fct_sales_line') }}
where net_revenue < 0
```

```
-- tests/assert_every_order_has_lines.sql
select o.order_id
from {{ ref('stg_orders') }} as o
left join {{ ref('fct_sales_line') }} as f on f.order_id = o.order_id
where o.is_counted and f.order_id is null
```

The second one is worth thinking about before you write it: it will fail for any non-cancelled order whose lines were dropped by a join in the fact model, which is exactly the bug you want to know about, and it's the kind of test that should have caught Chapter 28's point-in-time join problem.

8. Every model that uses the macro changes: `stg_order_items` (and therefore `fct_sales_line` and `fct_daily_sales`), plus the reconciliation test, which uses the same macro and so moves with it. That last part is a trap worth seeing: a test written with the same macro as the model **cannot catch a change to the macro**, because both sides move together. A reconciliation test is stronger when its other side comes from somewhere the macro doesn't touch, such as a figure finance publishes or a hard-coded expected total for a closed period.

9.

```
# terminal, in riverstone_dbt, after changing 'table' to 'view' in dim_date's config line
$ dbt run --select dim_date
19:19:05 1 of 1 START sql view model dbt_dev_marts.dim_date ..... [RUN]
19:19:05 1 of 1 OK created sql view model dbt_dev_marts.dim_date ..... [CREATE VIEW in
0.10s]
```

(Trimmed: the header and summary lines.)

The run is quicker because nothing is computed or stored, and every query against `dim_date` now runs `generate_series` again. For 365 rows that's free, so a view is a reasonable choice. Make it a table when the dimension is large, when it's joined in every report, or when the warehouse charges for compute rather than storage: then paying once per build beats paying once per query.

10. Descriptions go in `models/marts/schema.yml` under each column. After `dbt docs generate`, the site shows them beside the column, along with the tests attached to it, and the column-level lineage where the adapter supports it. The habit that pays: describe the column's **meaning and unit**, not its name. `net_revenue`: "line value after discount, before tax, in rupees; excludes cancelled orders" is worth writing. "The net revenue" is not.

11. With `{{ config(materialized='ephemeral') }}`, dbt creates nothing in the database. The model's SQL is inlined into every model that refs it, as a CTE at the top of the compiled file. Check `target/compiled/: fct_daily_sales.sql` now starts with `with __dbt__cte__int_orders_with_totals as (...)`, the model's SQL under a name dbt made up. Ephemeral models keep intermediate steps out of the warehouse, which is tidy, but they can't be queried, can't be tested on their own, and get recomputed in every model that uses them. Use them for small steps used once or twice.

12. The model gains a config line at the top and a filter at the bottom. The filter begins with `and`, because the model already has a `where`, and it reads `order_date`, which is why section 32.5 kept that column in the fact table:

```
{{ config(materialized='incremental', unique_key='order_item_id', incremental_strategy='delete+insert') }}
-- ... the select, joins and where o.is_counted, unchanged ...
{% if is_incremental() %}
  and o.order_date >= (select max(order_date) - interval '3 days' from {{ this }})
{% endif %}
```

The existing reconciliation test already compares grand totals, so the new test compares **each day**, which catches drift that a grand total can hide (one day too high, another too low):

```
-- tests/assert_incremental_matches_full.sql
-- Every day's total in the incremental fact must match a full recomputation from the source.
-- Returns one row per day that has drifted.
with incremental_days as (
  select order_date, sum(net_revenue) as total
  from {{ ref('fct_sales_line') }}
  group by order_date
),
full_days as (
  select o.order_date,
    sum({{ net_revenue('oi.quantity', 'oi.unit_price', 'oi.discount_pct') }}) as total
  from {{ source('erp', 'order_items') }} as oi
  join {{ source('erp', 'orders') }} as o on o.order_id = oi.order_id
  where o.status <> 'Cancelled'
  group by o.order_date
)
select f.order_date, i.total as incremental_total, f.total as full_total
from full_days as f
left join incremental_days as i on i.order_date = f.order_date
where i.total is null or abs(i.total - f.total) > 0.01
```


The `left join` and `i.total is null` also catch a day missing from the fact altogether. Run it after a few incremental runs, not just after a full refresh: drift is the failure mode you're testing for. (Tested: after a `--full-refresh`, each incremental run deleted and re-inserted the 4 lines of the last three days, and the test passed every time.)

13. `dbt_utils.equal_rowcount` compares two models' row counts, and `dbt_utils.expression_is_true` checks a condition per row. They're shorter to write and consistent across projects. The hand-written reconciliation is more specific: it compares a *total* against the source, to the *paisa*, which is the thing finance cares about. In practice, keep both: package tests for the mechanical checks, hand-written ones for the business rules that would actually cause an argument.

14. The Jinja version is shorter and adapts when a category is added (more so if the list comes from `dbt_utils.get_column_values`). The written-out version is readable by anyone, lints cleanly, and can be pasted into a query window to debug. For four categories that change once a decade, put the written-out version in the repository. For forty categories that change monthly, use Jinja, and leave a comment saying what it generates.

15. In `sources.yml`:

```
- name: orders
  loaded_at_field: order_date::timestamp
  freshness:
    warn_after: {count: 24, period: hour}
    error_after: {count: 48, period: hour}
```

`dbt source freshness` compares the newest `loaded_at_field` value with now. Against `riverstone_2025`, whose latest order is dated 23 December 2025, it errors immediately (`ERROR STALE`), which is the point of the exercise: **freshness catches the load that didn't run**, which is the single most common cause of a wrong dashboard, and no model test will ever notice it. In production, point it at a genuine load timestamp rather than a business date.

16. Two reasons to say no: staging models aren't guaranteed stable (they're an implementation detail that gets renamed and restructured), and they don't have the marts' joins, tests, or the grain that reports depend on, so two dashboards reading staging will re-implement the same joins differently. The case where they'd be right: an analyst exploring, in a scratch query, to decide what a mart should contain. Exploration reads whatever it likes; production reports read marts.

17. In order: (1) **Which models are slow?** `target/run_results.json` has a duration per node, and a handful of models usually dominate. (2) **Should the slow ones be incremental?** A full rebuild of a large fact table is the usual culprit. (3) **Are there tables that should be views, or views that should be tables?** Both mistakes cost time. (4) **Is the graph too wide or too deep?** Threads help only if the DAG allows parallelism, and a chain of twelve dependent models can't be parallelized at all. After those, look at warehouse-side things: clustering, partitioning, and the warehouse size (Chapter 49).

18. When the transformation isn't SQL and doesn't want to be: machine-learning features that need Python, or parsing awkward files. When the data isn't in the warehouse yet: dbt can't fetch it. When you need row-by-row processing or true real-time results: dbt runs batches. When the project is one

query that one person runs occasionally: a scheduled script is less machinery. And when the team has no version control or review habit, dbt on its own won't create one, though adopting it is a reasonable excuse to start.

NEXT

Where this leads

- **Chapter 16, Business Intelligence with Power BI**, reads the mart this chapter builds.
- **Chapter 28, Advanced SQL, Performance & Data Modeling**, is the modeling this chapter automates; its grain and SCD lessons are the reason the tests here look the way they do.
- **Chapter 46, Pipelines & Orchestration**, gets the data into the warehouse and runs dbt on a schedule alongside everything else.
- **Chapter 47, Data Quality, Observability & Contracts**, goes further than tests: freshness, anomaly detection, contracts, and who gets paged.
- **Chapter 49, Storage, Warehouses & Lakehouses**, is where these models run at scale, and where materialization choices become money.
- **Chapter 62, Data Architecture Patterns**, puts dbt in the wider picture, including when a team shouldn't use it.
- **Chapter 71, SQL Question Bank**, and **Chapter 77** include dbt and analytics-engineering questions.

Chapter 33. The Computer Science You Actually Need

Chapter at a glance

You will learn to: describe how long an algorithm takes in the language everyone uses (Big-O), and see the curves in measured times · choose between a list, a dictionary, a set, and a tuple for a reason · explain why a dictionary lookup is instant, and why the same trick powers a database hash join · use stacks and queues where they fit, including `deque` · write and read recursion without fear, and know when a loop is clearer · walk trees and graphs with depth-first and breadth-first search, and detect cycles · tell linear search from binary search, and know what `sorted()` costs · recognize memoization, dynamic programming, and greedy algorithms when you meet them · think about memory as well as speed · and talk through a coding problem the way interviewers expect.

Before you start: Chapter 17 (Python: lists, dictionaries, sets, loops, functions, CSV files), Chapter 18 (NumPy and pandas, which you'll compare with plain loops, and `lambda`), Chapter 28 (indexes, hash joins, and recursive CTEs, the database version of several ideas here), Chapter 29 (tests and generators), and Chapter 32 (a dbt project as a DAG).

Time needed: 14–18 hours, spread over three weeks, in four sittings: sections 33.1–33.4; recursion, trees and graphs (33.5–33.6); searching, sorting, memoization and dynamic programming (33.7–33.8); sections 33.9–33.10 and the project.

Tools: Python 3.11 or later and its standard library. Nothing else.

Practice data: `generate_ch33_data.py` in the companion folder builds 176,110 order lines and 20,000 invoice lines from Riverstone's logistics partner, which the chapter's project matches. This is a synthetic, scaled-up Riverstone: about 66,700 orders over 23 months, nearly 400 times the orders of the one-year database you've used since Chapter 12, so that the slow version is slow enough to measure. Its amounts are not Riverstone's real revenue. Every timing shown was measured on the author's test machine (an Intel Xeon at 2.1 GHz, Ubuntu 24.04, Python 3.11.15) and is reproduced by `ch33_timings.py`. **Timings vary by machine:** yours will differ, but the shapes (what happens when the data doubles) won't.

In this chapter

START	Why this matters · In plain English	261
LEARN	33.1 Big-O: how work grows	262
	33.2 The four structures you'll use every day	265
	33.3 Hashing, and why dictionaries are instant	268
	33.4 Stacks and queues	269
	33.5 Recursion	271
	33.6 Trees and graphs	273
	33.7 Searching and sorting	276
	33.8 Memoization, dynamic programming, and greedy	279
	33.9 Memory, and the other costs	282
	33.10 What coding interviews actually test	284
APPLY	Common mistakes · In the real world · Project	284
REVIEW	Recap · Key terms · Check yourself	287
PRACTISE	Exercises	289
	Answers	290
NEXT	Where this leads	298

START

Why this matters

You can have a long, well-paid career in data without knowing what a binary tree is. You cannot have one without meeting these three situations:

- **The script that got slower.** It ran in twenty seconds on last year's file. This year's file is ten times the size and it's been going for half an hour. Nothing changed except the data.
- **The interview.** Analytics engineering and data engineering interviews include a coding round, and the questions are all from this chapter's vocabulary: "*can you do this without a nested loop?*", "*what's the complexity of your solution?*"
- **The conversation with engineers.** When a backend engineer says "that endpoint is $O(n^2)$ in the number of orders", you either know what that sentence claims or you nod.

This chapter is not a computer-science course. It's the subset that changes how you write Python next week, with every claim measured rather than asserted. One idea in it, the dictionary index in section 33.2, is worth the whole chapter: it turns the project's two-minute matching job into about 0.06 seconds (0.050 seconds to build the index, 0.007 seconds to match).

In plain English

Complexity is about how the work grows, not how long it takes today.

Imagine finding one person in a hotel:

- You know the room number: walk there. One step, however big the hotel is. That's **constant time**.
- You must knock on every door: a hundred doors, a hundred knocks; a thousand doors, a thousand knocks. **Linear time**.
- The rooms are sorted and you can ask "higher or lower": a thousand rooms takes about ten questions, a million takes about twenty. **Logarithmic time**, and it barely notices when the hotel grows.
- You must introduce every guest to every other guest: a hundred guests is 4,950 introductions; a thousand is half a million. **Quadratic time**, and it's the reason most slow scripts are slow.

The point isn't the maths. It's that the hotel always gets bigger. A method that grows quadratically works fine on today's data, and the day the file doubles it takes four times as long. The way out is usually to build an index first, which is what a dictionary is, and what the database does when it builds a hash table for a join (Chapter 28).

LEARN

33.1 Big-O: how work grows

Big-O describes how an algorithm's work grows with the size of its input, ignoring constants and small terms. It's a shape, not a stopwatch.

Notation	Called	Work when the input doubles	Everyday example
$O(1)$	constant	unchanged	dictionary lookup, appending to a list (on average: the list occasionally grows its storage)
$O(\log n)$	logarithmic	one more step	binary search, index lookup in a database
$O(n)$	linear	doubles	scanning a list, reading a file
$O(n \log n)$	linearithmic	slightly more than doubles	sorting
$O(n^2)$	quadratic	four times	a loop inside a loop over the same data
$O(2^n)$	exponential	squares	trying every combination

The chapter's data

The chapter works on one job, from the project at the end: Riverstone's logistics partner bills Riverstone for every order line it delivered to customers, one invoice line per order line, and each invoice line has to be matched to the order line it bills. First build the two files. In a terminal, in the companion folder:

```
# terminal, in companion/ch33
$ python generate_ch33_data.py
wrote match_data/order_lines.csv: 176,110 order lines
wrote match_data/carrier_invoice_lines.csv: 20,000 invoice lines (19,500 should match, 500 should not)
```

The script writes a folder called `match_data` with two CSV files. Before you read on, predict: of the 20,000 invoice lines, how many will a correct matching script match? The script's own last line tells you what the answer should be, which is exactly what you want from test data.

Now load both files into Python, with Chapter 17's `csv.DictReader`, and look at one row of each:

```
import csv

def load(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

order_lines = load("match_data/order_lines.csv")
invoices = load("match_data/carrier_invoice_lines.csv")
print(f"{len(order_lines):,} order lines, {len(invoices):,} invoice lines")
print(order_lines[0])
print(invoices[0])
```

```
176,110 order lines, 20,000 invoice lines
{'order_id': '100000', 'product_id': '103', 'quantity': '100', 'net_revenue': '38000', 'order_date': '2024-10-10'}
{'invoice_line_id': 'INV012129', 'order_id': '130195', 'product_id': '108', 'amount': '37240.0'}
```

How it works:

- **load(path)** opens a CSV file and returns a list of dictionaries, one per row, keyed by the header. `newline=""` and `encoding="utf-8"` are Chapter 17's settings for reading CSV files safely.
- **An order line** has `order_id`, `product_id`, `quantity`, `net_revenue` and `order_date`. **An invoice line** has its own `invoice_line_id`, the `order_id` and `product_id` of the order line it bills, and the `amount` billed.
- **Every value from `csv.DictReader` is text**, so ids are strings like `'100000'`, and amounts are strings like `'38000'` until you convert them with `float()`.

An invoice line matches an order line when both the `order_id` and the `product_id` agree.

The slow way, measured

Here is the matching job as a careful person first writes it: for each invoice, look through the order lines until you find its match. Time it on the first 250 invoices:

```
import time

started = time.perf_counter()
matched = 0
for invoice in invoices[:250]:
    for line in order_lines:
        if (line["order_id"] == invoice["order_id"]
            and line["product_id"] == invoice["product_id"]):
            matched += 1
            break
elapsed = time.perf_counter() - started
print(f"250 invoices: matched {matched}, took {elapsed:.2f} s")
```

```
250 invoices: matched 242, took 1.52 s
```

How it works:

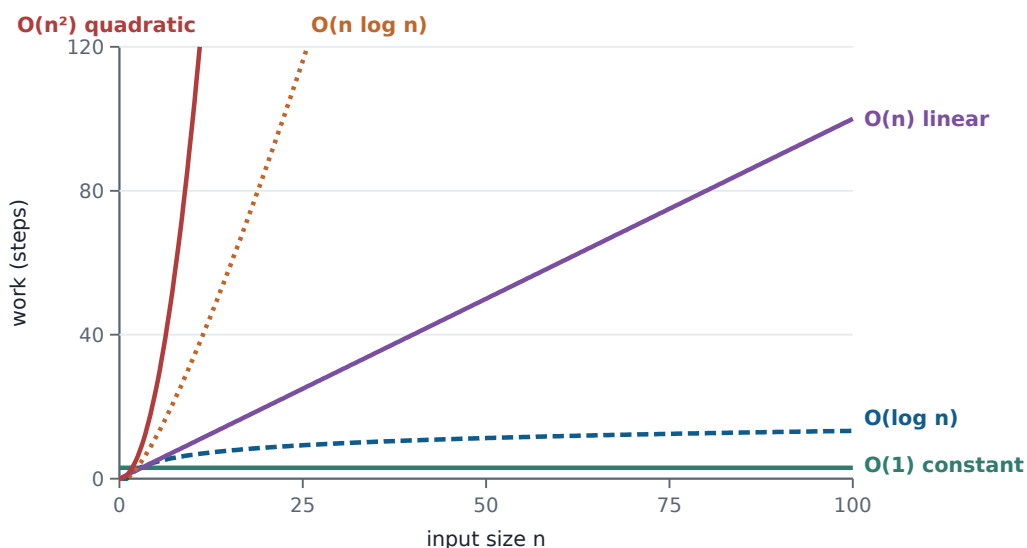
- `time.perf_counter()` is Chapter 18's stopwatch: read it before and after, and subtract.
- **The outer loop** takes one invoice at a time (`invoices[:250]` is the first 250). **The inner loop** walks the order lines from the top.
- **The if** tests both columns; the brackets let the condition continue onto a second line.
- **break** stops the inner loop as soon as the match is found. So a matched invoice costs, on average, a scan of half the order lines, and an **unmatched** invoice costs a scan of all 176,110: the invoices that match nothing are the most expensive ones.

That's up to $250 \times 176,110$, or about 44 million comparisons, for 250 invoices. Now the same cell with 500, 1,000 and 2,000 invoices (the timing script runs each three times and keeps the middle value):

```
# terminal, in companion/ch33 (median of three runs, from ch33_timings.py)
250 invoices x 176,110 order lines:  1.48 s
500 invoices x 176,110 order lines:  3.07 s
1000 invoices x 176,110 order lines:  5.71 s
2000 invoices x 176,110 order lines: 11.62 s
```

Double the invoices, double the time: that's linear *in the number of invoices*, because the order lines stayed the same. The quadratic part appears when both grow together, which is what happens in real life: next year Riverstone has twice the invoices **and** twice the orders, so the job takes four times as long. Extrapolating the last row, the full 20,000-invoice file takes about **two minutes** today ($11.62 \times 10 = 116$ seconds), and almost eight minutes when the business doubles ($116 \times 4 = 464$ seconds).

How the work grows: five shapes on one scale



Doubling n doubles the work of a linear algorithm and quadruples a quadratic one. The curves are drawn to one scale ($O(1)$ as 3 steps, $O(\log n)$ as $2 \times \log_2 n$); the times on the right were measured.

Figure 33.1 – The five shapes (drawn to scale) and the matching job's measured times. Constant and logarithmic barely move; quadratic leaves the chart.

How to read your own code's complexity, without any algebra:

1. **Count the loops over your data.** One loop over n items is $O(n)$. A loop inside a loop over the same n is $O(n^2)$.
2. **A lookup inside a loop is the thing to watch.** `if x in some_list` is itself a loop, so putting it inside a loop makes the whole thing quadratic. `if x in some_set` is not.
3. **Sorting is $O(n \log n)$** , which is close enough to linear that sorting once is almost always worth it.
4. **Constants don't matter to Big-O, but they matter to you.** $O(n)$ that reads from disk can be slower than $O(n^2)$ in memory, for small n . Big-O tells you what happens as n grows; the stopwatch tells you what happens today.

Watch out: Big-O ignores exactly the things that make code fast. Two $O(n)$ algorithms can differ by a factor of fifty. pandas and NumPy are fast not because their complexity is better but because their constants are tiny: the work happens in compiled code over contiguous memory instead of in a Python loop (Chapter 18). Use Big-O to choose the approach, and a measurement to choose between two implementations of it.

33.2 The four structures you'll use every day

Structure	Written	Keeps order?	Lookup by value	Good at	Bad at
list	[1, 2, 3]	yes	$O(n)$	keeping a sequence, appending	finding, removing from the front
dict	{"a": 1}	insertion order	$O(1)$ by key	lookups, grouping, counting	memory; range questions ("everything between two values"); ordering by value
set	{1, 2, 3}	no	$O(1)$	membership, deduplication, difference	anything ordered
tuple	(1, 2)	yes	$O(n)$	fixed records, dictionary keys	changing anything

Measure the difference yourself with `timeit`, the standard library's tool for timing small operations. It runs a piece of code many times and returns the total seconds:

```
import timeit

keys = [(line["order_id"], line["product_id"]) for line in order_lines]
keys_set = set(keys)
wanted = keys[-1] # the last key: the worst case for a list

list_seconds = timeit.timeit(lambda: wanted in keys, number=1000)
set_seconds = timeit.timeit(lambda: wanted in keys_set, number=1000)
print(f"list 'in', 1,000 lookups: {list_seconds:.3f} s")
print(f"set 'in', 1,000 lookups: {set_seconds:.6f} s")
print(f"the set was about {list_seconds / set_seconds:.0f} times faster")
```



```
list 'in', 1,000 lookups: 3.626 s
set 'in', 1,000 lookups: 0.000038 s
the set was about 96,354 times faster
```

How it works:

- **keys** is a list of 176,110 (`order_id`, `product_id`) tuples, and **keys_set** holds the same tuples in a set.
- **timeit.timeit(code, number=1000)** runs `code` 1,000 times and returns the total time. The code is passed as a `lambda` (Chapter 18): a one-line function with no name, so `timeit` can call it again and again. `number=` sets how many runs; more runs give a steadier total.
- **wanted in keys** must compare `wanted` with every element until it finds a match, and `wanted` is the last one. **wanted in keys_set** computes a hash and goes straight there (section 33.3 shows how).

That's not a small improvement, it's four orders of magnitude or more, and it comes from one keystroke: `set(...)` instead of `list(...)`. What happens if you change it? Make `wanted = keys[0]` and run the cell again: the list finds it at once and the gap almost disappears. A list is only slow when it has to look far, and in real data the key you want is rarely at the front.

The dictionary index, which is most of this chapter's value

Any time you find yourself looking something up inside a loop, **build a dictionary first**. Here is the index for the matching job:

```
# the index: one pass over the order lines, keyed by what the invoice knows
index = {(line["order_id"], line["product_id"]): line for line in order_lines}
print(f"index holds {len(index):,} keys")
```

```
index holds 176,110 keys
```

- **A dictionary comprehension** (Chapter 17) builds the whole dictionary in one pass over the order lines.
- **The key is a tuple**, (`order_id`, `product_id`), because that pair is what an invoice line knows. The value is the whole order line.

176,110 keys for 176,110 lines proves that (`order_id`, `product_id`) is unique here. A dictionary silently keeps the **last** value when a key repeats, so if the key count were smaller than the line count, some lines would have been overwritten without a word; then you'd index into a list per key (`defaultdict(list)`), as the grouping section below does. And make both sides the same type: `'100001'` (text) and `100001` (a number) are different keys, so an index built from one file never matches ids read as numbers from another.

Now look each invoice up:

```
matched = sum(1 for invoice in invoices
              if (invoice["order_id"], invoice["product_id"]) in index)
print(f"matched {matched:,,}, unmatched {len(invoices) - matched:,,}")
```

```
matched 19,500, unmatched 500
```

`sum(1 for x in items if condition)` is a **generator expression**: it looks like a list comprehension in round brackets, and it produces its values one at a time without building a list (section 33.9 explains why that saves memory). Adding up a 1 for every invoice that passes the test counts them. The test itself is one dictionary lookup.

One pass to build the index, one lookup per invoice: $O(n + m)$ instead of $O(n \times m)$. Measured by `ch33_timings.py`, that's 0.050 seconds to build and 0.007 seconds to match all 20,000 invoices, against about two minutes for the nested loop.

The same trick appears everywhere once you see it:

Instead of	Do this
<code>for row in rows: if row["id"] in id_list</code>	<code>make id_set = set(id_list) first</code>
<code>for a in table_a: for b in table_b: if a.key == b.key</code>	<code>index table_b by key, then loop table_a once</code>
counting with <code>if key in counts: counts[key] += 1 else: ...</code>	<code>collections.Counter</code> , or <code>defaultdict(int)</code>
looking up a customer's name inside a loop	one dictionary built before the loop

And the database does exactly this: Chapter 28's **hash join** builds a hash table of the smaller table, then scans the larger one once. When you build a dictionary index in Python, you are hand-writing a hash join.

Grouping, counting, and the standard library

Chapter 17's `defaultdict` groups the order lines by product in one pass:

```
from collections import Counter, defaultdict

by_product = defaultdict(list)
for line in order_lines:
    by_product[line["product_id"]].append(float(line["net_revenue"]))

revenue = {product: round(sum(values)) for product, values in by_product.items()}
for product in sorted(revenue, key=revenue.get, reverse=True)[:3]:
    print(f"product {product}: {len(by_product[product]):,} lines, revenue {revenue[product]:,}")
```

```
product 106: 22,084 lines, revenue 563,741,550
product 108: 22,042 lines, revenue 556,953,300
product 101: 22,075 lines, revenue 555,084,450
```

How it works:

- `defaultdict(list)` creates an empty list the first time a product appears, which removes the "is this key there yet?" dance.
- `revenue` adds up each product's list and rounds it.
- `sorted(revenue, key=revenue.get, reverse=True)` sorts the product ids by the value `revenue.get(id)` returns. Passing a function *without* brackets (`revenue.get`, not `revenue.get()`) hands it to `sorted`, which calls it on each item to get the value to sort by. `reverse=True` puts the largest first, and `[:3]` keeps the top three.

Counter, also from Chapter 17, counts in one line:

```
invoiced_products = Counter(invoice["product_id"] for invoice in invoices)
print("most invoiced products:", invoiced_products.most_common(3))
```

```
most invoiced products: [('101', 2607), ('107', 2561), ('104', 2514)]
```

The argument to `Counter` is another generator expression, one product id per invoice line, and `.most_common(3)` returns the three most frequent with their counts. `defaultdict` and `Counter` are both $O(n)$ with small constants, and both are the kind of thing that makes a script shorter *and* faster at once.

33.3 Hashing, and why dictionaries are instant

A **hash function** turns a value into a number. Python's `hash()` does it for any hashable value: numbers, strings, and tuples of those.

```
for value in (100001, 100002, (100001, 101), (100002, 104), (100003, 108), (100004, 105)):
    print(f"{str(value):<15} hash {hash(value):>21} -> bucket {hash(value) % 8}")

print("same value, same hash:", hash((100001, 101)) == hash((100001, 101)))
print("lists are unhashable, which is why they can't be dictionary keys:")
try:
    hash([100001, 101])
except TypeError as exc:
    print(" TypeError:", exc)
```

```
100001      hash      100001 -> bucket 1
100002      hash      100002 -> bucket 2
(100001, 101) hash -5250647048266057633 -> bucket 7
(100002, 104) hash  434502588224997158 -> bucket 6
(100003, 108) hash  9115427311463621200 -> bucket 0
(100004, 105) hash  2954519364476443151 -> bucket 7
same value, same hash: True
lists are unhashable, which is why they can't be dictionary keys:
TypeError: unhashable type: 'list'
```

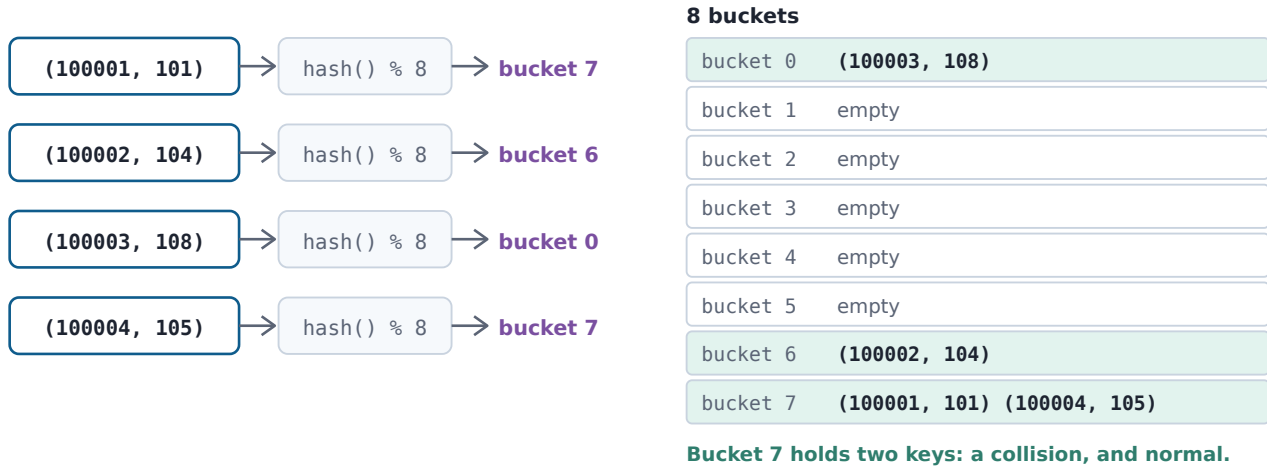
How it works:

- **hash(value)** returns the value's hash. A small whole number hashes to itself; a tuple's hash is computed from its parts.
- **hash(value) % 8** is the remainder after dividing by 8, always 0 to 7: the **bucket** the value would go in if the table had eight buckets.
- **(100001, 101) and (100004, 105) both land in bucket 7**. Two keys in the same bucket is a **collision**, and it's normal.
- **try / except TypeError** (Chapter 17) catches the error that hashing a list raises and prints its message.

(Strings hash differently in every Python process, deliberately, so the numbers above use integers to stay reproducible. The behavior is the same.)

A **hash table** (which is what a Python `dict` and `set` are) keeps an array of buckets. To store a key it hashes it, takes the remainder by the number of buckets, and puts the entry there. To find a key it does the same arithmetic and looks in one bucket. **That's why lookup doesn't get slower as the dictionary grows:** the work is one hash and one bucket, whatever the size.

Why a dictionary lookup does not get slower as it grows



To find a key: hash it, take the remainder, look in one bucket. The same steps whether the dictionary holds ten keys or ten million.

Figure 33.2 – Hash, take the remainder, look in one bucket. The bucket numbers are the real ones from the code above.

This is the textbook "bucket" picture. Python's own `dict`, on a collision, tries another slot instead of stacking entries in one bucket, but the cost is the same: a few steps, not a scan. Two consequences worth carrying:

- **Keys must be immutable**, because a key that changed after being stored would hash to a different bucket and become invisible. That's why tuples work as keys and lists don't, and why the project's index is keyed by `(order_id, product_id)`.
- **Collisions** are normal and handled: Python finds another slot in a step or two. They only hurt if almost everything collides, which is a problem for adversarial input, not for order ids.

This is also what Chapter 28's **hash join** does: build a hash table from the smaller table, scan the larger one, and probe the table for each row. When PostgreSQL's plan says `Hash Join`, it is doing what section 33.2's dictionary index did.

33.4 Stacks and queues

Two structures with one rule each.

- A **stack** is last in, first out. Undo history, the call stack behind recursion (section 33.5), depth-first search.
- A **queue** is first in, first out. Job queues, retry backlogs, breadth-first search.

A Python list is already a fine stack: `append` and `pop` are both $O(1)$. As a queue it is not, because `pop(0)` has to shift every remaining element along:

```
# terminal, in companion/ch33 (from ch33_timings.py, 100,000 items)
list.pop(0)      : 0.805 s
deque.popleft()  : 0.004 s
```

`collections.deque` is a double-ended queue: $O(1)$ at both ends.

```
from collections import deque

# a stack: the last invoice queried is the first one answered
stack = []
for invoice_id in ["INV000001", "INV000002", "INV000003"]:
    stack.append(invoice_id)
print("stack pops:", [stack.pop() for _ in range(3)])

# a queue: invoices are worked in the order they arrived
queue = deque(["INV000001", "INV000002", "INV000003"])
print("queue pops:", [queue.popleft() for _ in range(3)])

# a retry queue, which is where analysts meet this first
# pretend: invoices ending in 0 fail until their third attempt
retries = deque([("INV000010", 1), ("INV000011", 1)])
processed = []
while retries:
    invoice_id, attempt = retries.popleft()
    if attempt < 3 and invoice_id.endswith("0"):
        retries.append((invoice_id, attempt + 1))      # try again later, not immediately
    else:
        processed.append((invoice_id, attempt))
print("processed:", processed)
```

```
stack pops: ['INV000003', 'INV000002', 'INV000001']
queue pops: ['INV000001', 'INV000002', 'INV000003']
processed: [('INV000011', 1), ('INV000010', 3)]
```

How it works:

- `stack.pop()` removes and returns the last item, so the three come back in reverse order.
- `for _ in range(3):` `_` is the conventional name for a loop variable you don't use. The comprehension just calls `pop()` three times.
- `deque(...)` builds a queue from a list, and `popleft()` removes from the front.
- **The retry queue** holds `(invoice_id, attempt)` pairs. A stand-in rule decides what fails: an id ending in "0" fails until its third attempt. A failure goes to the **back** of the queue with its attempt number raised, so `INV000011` is processed first, and `INV000010` succeeds on attempt 3.

That last pattern is a **retry queue**: failures go to the back, so one stubborn item doesn't block the rest. It complements Chapter 29's API client, which waits and retries the same request; a queue is what you use when you have many independent items.

33.5 Recursion

A **recursive** function calls itself on a smaller version of the problem. It needs two things: a **base case** that stops, and a **recursive case** that moves toward it.

Chapter 28 exploded the Garden Chair's bill of materials with a recursive CTE. Here is the same walk in Python, with the bill of materials as a dictionary: each assembly maps to a list of (part, quantity) pairs.

```
parts = {
    # Chapter 28's bill of materials, as a dictionary
    "Garden Chair": [("Seat shell", 1), ("Steel frame", 1),
                     ("Product label", 1), ("Shipping carton", 1)],
    "Seat shell": [("PP granules", 2.2), ("Masterbatch", 0.08)],
    "Steel frame": [("Leg assembly", 2), ("Steel tube", 1.0), ("Fastener pack", 1)],
    "Leg assembly": [("Steel tube", 1.2), ("Fastener pack", 1)],
}

def explode(part, quantity=1.0):
    if part not in parts:
        return {part: quantity}
    totals = {}
    for child, child_quantity in parts[part]:
        for material, amount in explode(child, quantity * child_quantity).items():
            totals[material] = totals.get(material, 0) + amount
    return totals

for material, amount in sorted(explode("Garden Chair").items()):
    print(f"{material:<16} {amount:>6.2f}")
```

Fastener pack	3.00
Masterbatch	0.08
PP granules	2.20
Product label	1.00
Shipping carton	1.00
Steel tube	3.40

How it works:

- **The base case:** a part that isn't in `parts` is a bought material, so `explode` returns it with the quantity needed, and stops.
- **The recursive case:** for each child, `explode` calls **itself** with the child and the multiplied quantity (`quantity * child_quantity`: two leg assemblies each needing 1.2 kg of tube is 2.4 kg). Every call is on a smaller piece of the tree, so the calls always reach materials in the end.
- `totals.get(material, 0) + amount` adds each child's materials into this part's totals, starting from 0 the first time a material appears.

Steel tube comes out at 3.4, matching Chapter 28's SQL exactly, because a recursive CTE and a recursive function are the same idea in two languages. Note that Steel tube and Fastener pack each arrive by two routes, and both must be counted.

What if a part contains itself? A data-entry mistake that makes the leg assembly contain the frame would send `explode` round in circles until Python stopped it. A **path** guard catches it: pass along the parts on the current route, and refuse to go down into one that is already on it.

```

def explode_checked(part, quantity=1.0, path=()):
    if part in path:
        raise ValueError(f"cycle at {part}: {' -> '.join(path + (part,))}")
    if part not in parts:
        return {part: quantity}
    totals = {}
    for child, child_quantity in parts[part]:
        below = explode_checked(child, quantity * child_quantity, path + (part,))
        for material, amount in below.items():
            totals[material] = totals.get(material, 0) + amount
    return totals

parts["Leg assembly"].append(("Steel frame", 1))    # the mistake: the leg contains the frame
try:
    explode_checked("Garden Chair")
except ValueError as exc:
    print("ValueError:", exc)
parts["Leg assembly"].pop()                        # undo the mistake
print(f"fixed again, steel tube: {explode_checked('Garden Chair')['Steel tube']:.2f}")

```

```

ValueError: cycle at Steel frame: Garden Chair -> Steel frame -> Leg assembly -> Steel frame
fixed again, steel tube: 3.40

```

- **path=()** starts as an empty tuple. **path + (part,)** makes a new tuple with this part added: (part,) is a one-item tuple (the comma makes it a tuple).
- **if part in path** fires only when a part appears *on its own route*. Steel tube appearing twice, under two different parents, is fine; Steel frame inside its own leg assembly is not.
- **raise ValueError(...)** stops with a message that names the loop, which is what whoever fixes the data needs.

The call stack. Each call waits for the ones it makes, and Python keeps the waiting calls on a stack (section 33.4): the newest call is on top and finishes first. That stack has a limit (1,000 frames by default), so a walk over a deep structure can end in `RecursionError`. Compare a recursive count with a loop that does the same job:

```

import sys

def depth(n):
    return 0 if n == 0 else 1 + depth(n - 1)

def depth_loop(n):
    count = 0
    while n > 0:
        n, count = n - 1, count + 1
    return count

print("limit:", sys.getrecursionlimit())
try:
    depth(5000)
except RecursionError as exc:
    print("RecursionError:", exc)
print("iterative version:", depth_loop(5000))

```



```
limit: 1000
RecursionError: maximum recursion depth exceeded
iterative version: 5000
```

- **depth(n)** counts down by calling itself: `depth(5000)` needs 5,000 calls waiting at once, more than the limit.
- **depth_loop(n)** does the same count with a `while` loop and two variables, so nothing waits and there is no limit.
- **sys.getrecursionlimit()** reports the limit. You can raise it with `sys.setrecursionlimit`, but a loop is usually the better fix.

Recursion or a loop? Use recursion when the data is itself nested: trees, bills of materials, folders, JSON (Chapter 17), nested comments. Use a loop when the problem is a straight sequence. Any recursion can be rewritten as a loop with an explicit stack, and sometimes should be, which is exactly what exercise 7 asks you to do.

33.6 Trees and graphs

You wrote recursive functions in section 33.5; walking a tree is the same idea.

A **tree** is data where each item has one parent: Riverstone's org chart, its bill of materials, a folder structure, a category hierarchy. A **graph** is the general case, where anything can connect to anything: customers and the orders they share, tables and their dependencies (Chapter 32's DAG is a graph), roads between cities. Each item is a **node**; each link between two nodes (reports to, contains, depends on) is an **edge**.

Chapter 28 walked these in SQL with recursive CTEs. In Python they're a dictionary of lists. Here is a slice of Chapter 28's staff table (the key accounts sales team, and the production branch down to the machine operator Chapter 28 traced), with each manager mapped to their direct reports:

```
reports_to = {
    "Arvind Kapoor": ["Anita Rao", "Harpreet Sethi"],
    "Anita Rao": ["Vikram Singh", "Farah Khan"],
    "Harpreet Sethi": ["Ramesh Patil"],
    "Vikram Singh": ["Neha Kulkarni", "Rahul Mehta"],
    "Ramesh Patil": ["Ajay Kumar"],
    "Ajay Kumar": ["Gopal Sahu"],
}

def depth_first(person, level=0, seen=None):
    seen = seen if seen is not None else set()
    if person in seen:
        return [] # already visited: don't walk it twice
    seen.add(person)
    result = [(level, person)]
    for report in reports_to.get(person, []):
        result += depth_first(report, level + 1, seen)
    return result

for level, person in depth_first("Arvind Kapoor"):
    print("    " * level + person)
```

```

Arvind Kapoor
  Anita Rao
    Vikram Singh
      Neha Kulkarni
      Rahul Mehta
    Farah Khan
  Harpreet Sethi
    Ramesh Patil
      Ajay Kumar
      Gopal Sahu

```

How it works:

- **depth_first** returns a list of (level, person) pairs: this person, then everyone below them, one branch at a time. It calls itself for each direct report with level + 1, exactly as `explode` called itself for each child part.
- **reports_to.get(person, [])** gives a person's direct reports, or an empty list for someone with none: that's the base case.
- **seen=None, then create the set inside.** A default like `seen=set()` would be created once, when the function is defined, and shared by every call, so a second walk would think everyone was already seen. None as the default, and a new set when it's None, gives each walk its own.
- **result += ...** adds the list the inner call returns onto this one.
- **" " * level + person** repeats a four-space string level times, so each level is indented further.

Depth-first goes all the way down one branch before starting the next, which is why the output reads like an indented org chart. Breadth-first goes level by level, which is what you want for "who is two levels below the MD?" and for shortest paths:

```

from collections import deque

def breadth_first(start):
    queue, seen, levels = deque([(start, 0)]), {start}, []
    while queue:
        person, level = queue.popleft()
        levels.append((level, person))
        for report in reports_to.get(person, []):
            if report not in seen:
                seen.add(report)
                queue.append((report, level + 1))
    return levels

walk = breadth_first("Arvind Kapoor")
for level in range(3):
    people = [p for lvl, p in walk if lvl == level]
    print(f"level {level}: {'', '.join(people)}")

```

```

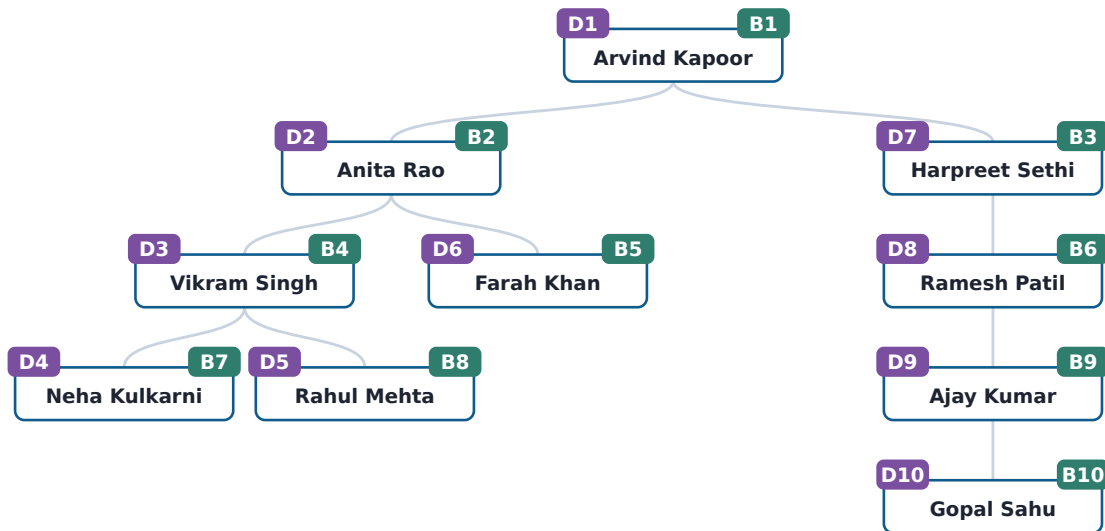
level 0: Arvind Kapoor
level 1: Anita Rao, Harpreet Sethi
level 2: Vikram Singh, Farah Khan, Ramesh Patil

```

- **The queue** starts with the MD at level 0. Each turn takes the person at the front, records them, and puts their reports at the **back**, one level deeper. So everyone at level 1 comes out before anyone at level 2.

- `walk` is computed once, before the loop; the loop then picks each level's people out of it. (Calling `breadth_first` inside the loop would walk the whole tree three times.)
- `', '.join(people)` (Chapter 17) joins the names with commas.

One tree, two walks: depth-first uses a stack, breadth-first a queue



D1 to D10: depth-first (a stack)

down one branch, then back up: the order the indented org chart prints in

B1 to B10: breadth-first (a queue)

level by level: answers "who is two levels below the MD?" and shortest paths

Figure 33.3 – One tree, two orders. The only difference is whether the next node comes off a stack or a queue.

The two are the same algorithm with a different container. Depth-first uses a stack (or recursion, which is a stack); breadth-first uses a queue. That single sentence answers a surprising number of interview questions.

Cycles

Real data has loops in it: an employee whose manager reports to them, a part that contains itself, two dbt models that ref each other. There are two different guards, and it matters which one you use:

- A **seen** set skips anything already visited, anywhere. That's right for reachability ("who is below the MD?"), and it makes each node count once. Both walks above use one.
- A **path** set skips only what's on the current route. That's right for cycle detection, and for bills of materials, where one part can legitimately appear under two parents: a seen set would count Steel tube once and give 1.0 kg, not 3.4. It's the Python version of Chapter 28's `CYCLE` clause, and it's what `explode_checked`'s path did.

A good cycle check uses both: a path set to spot the loop, and a set of nodes already **done** (fully checked, with nothing below them looping), so no part of the graph is walked twice.

```

broken = {"A": ["B"], "B": ["C"], "C": ["A"]}      # A -> B -> C -> A

def has_cycle(graph):
    done = set()      # nodes already proven to lead to no cycle
    on_path = set()   # nodes on the route being walked right now
  
```

```
def visit(node):
    if node in on_path:
        return True          # back to a node on our own route: a cycle
    if node in done:
        return False         # checked before: nothing below it loops
    on_path.add(node)
    for neighbor in graph.get(node, []):
        if visit(neighbor):
            return True
    on_path.remove(node)
    done.add(node)
    return False

    return any(visit(node) for node in graph)

print("cycle in the org chart:", has_cycle(reports_to))
print("cycle in the broken data:", has_cycle(broken))
```

```
cycle in the org chart: False
cycle in the broken data: True
```

How it works:

- **visit is defined inside has_cycle**, a **nested function**. It can read and change `done`, `on_path` and `graph` from the function around it, so they don't have to be passed on every call.
- **on_path** gets a node when the walk goes down into it, and loses it when the walk comes back up. Meeting a node that is still on the path means the walk has come round in a circle.
- **done** gets a node once everything below it has been checked. A second route into that node stops at once, so every node and every edge is handled once: $O(V + E)$, for V nodes and E edges.
- **any(...)** is `True` as soon as one value is `True`, and stops there. Here it tries `visit` from each node in turn.

The remembered `done` set matters. A first version without it, which re-walks every route from every node, is exponential on graphs where routes split and rejoin, and dbt projects are full of those. On a chain of 18 such "diamonds" (no cycle at all), that version took 2.15 seconds on the test machine, and its time doubles with each diamond added; the version above took 0.00002 seconds. dbt runs this kind of check over a project's `refs`, and refuses to run a project whose graph has a loop: Chapter 32's DAG has to stay acyclic, and this is how that is checked.

33.7 Searching and sorting

Linear and binary search

Finding a value in an unsorted list means looking at every element until you find it: **linear search**, $O(n)$. If the list is *sorted*, you can halve the search space with every comparison: **binary search**, $O(\log n)$. A million sorted values take about twenty comparisons.

```
# terminal, in companion/ch33 (from ch33_timings.py, a sorted list of 1,000,000 numbers)
list.index() 200 searches : 0.657 s
bisect       10,000 searches : 0.0070 s
```

Two hundred linear searches took 0.7 seconds; ten thousand binary searches took seven milliseconds. Python's `bisect` module does binary search on any sorted sequence. The everyday use is putting a value into sorted bands, such as Riverstone's loyalty rebate from Chapter 17: 0% as standard, 1% from ₹2,50,000 a year, 2% from ₹4,00,000:

```
from bisect import bisect_right

thresholds = [0, 250_000, 400_000]      # where each band starts, sorted
rates = ["0%", "1%", "2%"]

def rebate_band(annual_value):
    if annual_value < thresholds[0]:
        raise ValueError(f"negative amount {annual_value}: credit notes need their own rule")
    return rates[bisect_right(thresholds, annual_value) - 1]

for value in (120_000, 250_000, 399_999, 400_000, 650_000):
    print(f"{value:>9,}: {rebate_band(value)}")
```

```
120,000: 0%
250,000: 1%
399,999: 1%
400,000: 2%
650,000: 2%
```

How it works:

- **`bisect_right(thresholds, value)`** finds, by binary search, how many thresholds are less than or equal to `value`. For 120,000 that's 1 (only the 0), so `- 1` gives index 0, the first band.
- **A value equal to a threshold goes into the higher band:** 2,50,000 gets 1%, which is "from ₹2,50,000". That's the "right" in `bisect_right`.
- **The guard** at the top refuses a negative amount, for a reason the next cell shows.

Before you run the next cell, predict what band `-500` would get without the guard:

```
try:
    rebate_band(-500)
except ValueError as exc:
    print("ValueError:", exc)
print("without the guard:", rates[bisect_right(thresholds, -500) - 1])
```

```
ValueError: negative amount -500: credit notes need their own rule
without the guard: 2%
```

For `-500`, `bisect_right` returns 0, and index `0 - 1` is `-1`, which Python reads as "the last item": a credit note silently gets the top rebate. Negative indexes are the trap at the low end of any banding, so guard it. Bucketing a value into sorted bands is what a `CASE` statement does in SQL, done in log time instead of by checking every band.

`bisect` also keeps a list sorted as you add to it:

```
from bisect import insert

prices = [290, 430, 750]
insert(prices, 620)          # finds the place by binary search, and inserts there
print("prices:", prices)
```

```
prices: [290, 430, 620, 750]
```

`insert(prices, 620)` finds where 620 belongs by binary search (between 430 and 750) and inserts it there, so `prices` never needs sorting again. The search is $O(\log n)$; the insert still shifts the later items along, as `pop(0)` did in section 33.4.

Sorting

`sorted()` and `list.sort()` use **Timsort**, which is $O(n \log n)$ and unusually fast on data that's already partly ordered, which real data usually is. Two habits are worth having: sorting by a computed value, and sorting by several columns. Start with three order lines:

```
lines = [
    {"order_id": "100002", "product_id": "104", "net_revenue": "18600"},
    {"order_id": "100001", "product_id": "101", "net_revenue": "38000"},
    {"order_id": "100001", "product_id": "108", "net_revenue": "5800"},
]

def revenue_of(row):
    return float(row["net_revenue"])

by_revenue = sorted(lines, key=revenue_of, reverse=True)
print("largest first:", [row["net_revenue"] for row in by_revenue])

same = sorted(lines, key=lambda row: float(row["net_revenue"]), reverse=True)
print("the lambda gives the same:", same == by_revenue)
```

```
largest first: ['38000', '18600', '5800']
the lambda gives the same: True
```

- **key=revenue_of** tells `sorted` to call `revenue_of` on each row and sort by what it returns, the revenue as a number, without changing the rows. (Sorting the text `'5800'` against `'18600'` would put `'5800'` first, because text sorts character by character.)
- **lambda row: float(row["net_revenue"])** is exactly `revenue_of`, written inline without a name (Chapter 18). For a one-line key, most people write the `lambda`.

A **tuple key** sorts by several columns, exactly like SQL's `ORDER BY a, b` (Chapter 12):

```
by_order_then_product = sorted(lines, key=lambda row: (row["order_id"], row["product_id"]))
print("by order, then product:", [(r["order_id"], r["product_id"]) for r in by_order_then_product])
```

```
by order, then product: [('100001', '101'), ('100001', '108'), ('100002', '104')]
```

Tuples compare item by item: the order ids first, and the product ids only when the order ids tie.

Timsort is stable: rows that tie keep the order they already had. So sorting twice, by the secondary key first and then by the primary key, gives the same answer as one tuple key:

```
by_product = sorted(lines, key=lambda row: row["product_id"])
by_order = sorted(by_product, key=lambda row: row["order_id"])
print("two stable sorts:", [(r["order_id"], r["product_id"]) for r in by_order])
print("same as the tuple key:", by_order == by_order_then_product)
```

```
two stable sorts: [('100001', '101'), ('100001', '108'), ('100002', '104')]
same as the tuple key: True
```

The two 100001 lines tie on `order_id` in the second sort, so they stay in the product order the first sort gave them. That's what "stable" buys you.

For "the biggest three of a million", sorting everything to keep three is wasted work. `heapq.nlargest(k, items, key=...)` keeps only the best `k` seen so far in a small structure called a **heap** (a heap keeps its smallest item at the front, and can add or remove one item in log time), so it scans the data once: $O(n \log k)$.

```
import heapq

top_two = heapq.nlargest(2, lines, key=lambda row: float(row["net_revenue"]))
print("largest two without sorting everything:", [row["net_revenue"] for row in top_two])
```

```
largest two without sorting everything: ['38000', '18600']
```

When sorting is the algorithm. Many problems get easy once the data is in order: finding duplicates (equal values sit together), finding the nearest match (binary search), merging two feeds (walk both in step), and grouping (Chapter 12's `GROUP BY` is a sort or a hash under the covers). Paying $O(n \log n)$ once to make everything after it linear is one of the most common good trades in data work.

33.8 Memoization, dynamic programming, and greedy

Three named ideas you'll meet in interviews and, occasionally, in real work.

Memoization: remember what you already computed

The classic demonstration is Fibonacci (each number is the sum of the two before it), where the naive recursion recomputes the same values exponentially often:

```
# terminal, in companion/ch33 (from ch33_timings.py)
fib(25) plain      : 0.009 s
fib(30) plain      : 0.098 s
fib(32) plain      : 0.248 s
fib(32) memoized   : 0.000018 s
fib(300) memoized  : 22232244629420445529739893461909967206666939096499764990979600
```


Each step of five in n multiplies the plain version's time by about eleven: that's exponential growth, $O(2^n)$. Caching turns it linear, and Python has the cache built in:

```
from functools import lru_cache

calls = {"plain": 0, "cached": 0}

def fib(n):
    calls["plain"] += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

@lru_cache(maxsize=None)
def fib_cached(n):
    calls["cached"] += 1
    return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)

fib(25)
fib_cached(25)
print(f"fib(25) plain made {calls['plain']:,} calls")
print(f"fib(25) cached made {calls['cached']:,} calls")
print("same answer:", fib(25) == fib_cached(25))
```

```
fib(25) plain made 242,785 calls
fib(25) cached made 26 calls
same answer: True
```

How it works:

- `calls` is a dictionary of two counters, raised by one on every call, so you can see how much work each version does.
- `@lru_cache(...)` on the line above `def` is a **decorator**: it wraps the function so each call first checks a dictionary of past arguments and their results, and only runs the function for an argument it hasn't seen. `maxsize=None` means never forget; a number such as `maxsize=1024` caps the memory by forgetting the least recently used results.
- `fib_cached.cache_info()` reports the cache's hits and misses, and `fib_cached.cache_clear()` empties it. Python also has `functools.cache`, the same thing with no size limit.

242,785 calls against 26. In analytics work, `@lru_cache` pays off on an expensive pure function called repeatedly with the same arguments: a currency conversion, a geocode, a parsed configuration.

Dynamic programming

Dynamic programming is memoization turned into a table you fill in deliberately: solve the small subproblems once, in order, and build the answer from them. The question it answers is always "the best way to do X given a limit", and it appears in interviews far more often than in dashboards.

Take a small case by hand first: the fewest notes that make ₹60 exactly, with notes of ₹50, ₹20 and ₹10. Fill in a table from the bottom, one amount at a time. For each amount, try every note that fits, and look up the best answer for what's left:

Amount (₹)	0	10	20	30	40	50	60
Fewest notes	0	1	1	2	2	1	2

For ₹60: a ₹10 note leaves ₹50 (best 1), a ₹20 note leaves ₹40 (best 2), a ₹50 note leaves ₹10 (best 1). So $\text{best}[60] = \min(1 + 1, 2 + 1, 1 + 1) = 2$. Every entry is built from entries already in the table, and none is worked out twice. The code does exactly this, for every amount in steps of ₹1:

```
def fewest_notes(amount, notes=(2000, 500, 200, 100, 50, 20, 10)):
    """The fewest notes that make an amount exactly, or None if it can't be made."""
    best = [0] + [None] * amount          # best[v]: fewest notes for v; None = can't be made
    for value in range(1, amount + 1):
        options = [best[value - note] + 1 for note in notes
                    if note <= value and best[value - note] is not None]
        best[value] = min(options) if options else None
    return best[amount]
```

How it works:

- **best** is the table, with one slot for every amount from 0 to `amount`. `best[0] = 0` (no notes make ₹0), and every other slot starts as `None`, meaning "can't be made".
- **For each value**, the comprehension tries every note that fits (`note <= value`) and whose remainder can be made (`best[value - note] is not None`), and counts one more note than the remainder needed.
- **min(options)** keeps the best; if no note works, the slot stays `None`.

Now run it:

```
for amount in (760, 2750, 45, 0):
    answer = fewest_notes(amount)
    if answer is not None:
        print(f"₹{amount:>5}: {answer} notes")
    else:
        print(f"₹{amount:>5}: cannot be made exactly")
```

```
₹ 760: 4 notes
₹ 2750: 4 notes
₹ 45: cannot be made exactly
₹ 0: 0 notes
```

The test is `answer is not None`, not `if answer: 0` counts as false in Python, so `if answer` would call ₹0 impossible. Every amount below the target is solved once and reused, which is what makes this $O(\text{amount} \times \text{notes})$ rather than exponential. (₹2000 notes were withdrawn from circulation in 2023, though they remain legal tender. Take 2000 out of notes and ₹2750 needs 7 notes instead of 4; the method doesn't change.)

Greedy

A **greedy** algorithm takes the best-looking option at each step and never reconsiders. It's simpler and faster, and it's right only when the problem has the right shape.

```
def greedy_notes(amount, notes=(2000, 500, 200, 100, 50, 20, 10)):
    count = 0
    for note in notes:
        count, amount = count + amount // note, amount % note
    return count if amount == 0 else None

odd_notes = (10, 6, 1)

print("Indian notes, ₹760: greedy", greedy_notes(760), "vs best", fewest_notes(760))
print("made-up notes 10/6/1, 12: greedy", greedy_notes(12, odd_notes), "vs best", fewest_notes(12,
    odd_notes))
```

```
Indian notes, ₹760: greedy 4 vs best 4
made-up notes 10/6/1, 12: greedy 3 vs best 2
```

- **amount // note** is how many of this note fit, and **amount % note** is what's left (Chapter 17). The line updates both at once: the count goes up, the amount goes down.
- **Largest note first**, always, and never reconsidered.

With real Indian notes, greedy is optimal, with or without the ₹2000 note (checked for every multiple of ₹10 up to ₹5,000). With notes of 10, 6 and 1, greedy takes 10 + 1 + 1 (three) where 6 + 6 (two) is better. **The lesson isn't about change:** it's that "take the biggest first" is a guess about the shape of the problem, and it needs checking. Greedy algorithms you'll actually meet: assigning leads to the rep with the fewest open ones, packing files into batches, picking the next job by earliest deadline.

33.9 Memory, and the other costs

Speed isn't the only resource. A list of a million rows holds a million objects; a **generator** produces them one at a time and holds one. That's **lazy evaluation**: values are produced only when something asks for them.

```
# terminal, in companion/ch33 (from ch33_timings.py, tracemalloc peak)
list      : 40.4 MB peak, sum 999999000000
generator : 0.0005 MB peak, sum 999999000000
```

Same answer, 80,000 times less memory. Chapter 29's API client used the same idea with `yield from` to hand back leads page by page. Here is a generator function over the order file:

```
import csv

def line_values(path):
    """Yields one value at a time; nothing is held except the current row."""
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            yield float(row["net_revenue"])

total = sum(line_values("match_data/order_lines.csv"))
biggest = max(line_values("match_data/order_lines.csv"))
print(f"total revenue {total:,.0f}, largest line {biggest:,.0f}")

squares_list = [n * n for n in range(5)]      # a list comprehension: builds everything
squares_gen = (n * n for n in range(5))      # a generator expression: builds nothing yet
print("list:", squares_list)
print("generator:", type(squares_gen).__name__, "->", list(squares_gen))
```

```
total revenue 4,421,366,900, largest line 140,000
list: [0, 1, 4, 9, 16]
generator: generator -> [0, 1, 4, 9, 16]
```

How it works:

- **yield** (Chapter 29) hands back one value and pauses the function until the next value is asked for. `sum` and `max` ask for values one at a time, so only one row is ever in memory.
- **The file is read twice**, once for `sum` and once for `max`, because a generator can be read only once.
- `type(squares_gen).__name__` is the name of the object's type, `generator`; `list(squares_gen)` then pulls all its values out.

The trade-offs to keep in mind:

Choice	Costs	Saves
Generator instead of a list	can only be read once; no <code>len()</code>	memory, and time before the first result
A dictionary index	memory for the index	vastly more time
Caching (<code>lru_cache</code>)	memory, and staleness if inputs change	repeated computation
Sorting once	$O(n \log n)$ up front	linear scans afterwards
pandas instead of loops	memory (whole frame) and a dependency	a lot of time, through compiled code

Watch out: the fastest fix is often not an algorithm at all. Before rewriting a loop, ask whether the work should be happening in Python. Chapter 28's lesson applies: if the data is in a database, an indexed query moves the join to a system built for it. If it's in a DataFrame, a vectorized pandas operation runs in compiled code. A dictionary index is the right answer when the data is already in Python and has to stay there.

33.10 What coding interviews actually test

Analytics engineering and data engineering interviews include a coding round, usually 30 to 45 minutes on one or two problems. They are not testing whether you memorized quicksort.

What they're looking for:

They want to see	How to show it
You clarify before coding	ask about size, duplicates, nulls, ordering, and what to return when nothing matches
You pick a sensible structure	"I'll index the orders by id in a dictionary, so the lookup is constant time"
You can state the complexity	"one pass to build the index, one pass over invoices: $O(n + m)$ time, $O(n)$ memory"
You write code that runs	small, named helpers; no clever one-liners
You test it	one normal case, one empty, one duplicate, one no-match
You improve it when prompted	"if the file didn't fit in memory, I'd stream it and index only the keys"

The patterns that come up most, all of which are in this chapter: a hash map to avoid a nested loop; two pointers or a merge over sorted data; a set for deduplication; a counter for frequency; breadth-first or depth-first over a tree or graph; a sort followed by a linear pass.

The sentence that earns marks: after your first working version, say *"this is $O(n^2)$ because of the inner loop; I can make it $O(n)$ with a dictionary"*, then do it. Interviewers care much more about that than about arriving at the optimal answer first.

Chapter 72A's Data Structures & Algorithms bank has the questions themselves (Chapter 72 has the Python ones), and Chapter 69 covers how to talk through a problem under pressure.

APPLY

Common mistakes

Mistake	Symptom	Fix
A lookup inside a loop (<code>x in a_list</code>)	The script slows down as the data grows, with no error	Build a <code>set</code> or <code>dict</code> before the loop
A loop inside a loop over the same data	Fine on a sample, hours on the real file	Index one side, then one pass over the other
<code>list.pop(0)</code> as a queue	Quietly quadratic	<code>collections.deque</code>
Re-reading or re-parsing the same file inside a loop	Disk time dominates everything	Read once into a structure
Recursion with no base case, or on cyclic data	<code>RecursionError</code> , or a hang	A base case, a path check, and a depth limit
Deep recursion on big structures	<code>RecursionError</code> at 1,000 frames	Rewrite with an explicit stack, or raise the limit deliberately

Mistake	Symptom	Fix
Mutable objects as dictionary keys	<code>TypeError: unhashable type</code>	Use tuples
Sorting inside a loop	$O(n^2 \log n)$ by accident	Sort once, before the loop
<code>sorted(...)[:3]</code> on millions of rows	Sorting everything to keep three	<code>heapq.nlargest</code>
Reading a huge file into a list	The process is killed for memory	A generator, or read in chunks
Caching a function whose inputs change	Stale results nobody can explain	Cache pure functions only; clear or bound the cache
Optimizing the wrong line	Hours spent, no change	Measure first: <code>time.perf_counter</code> , <code>timeit</code> , or a profiler
Reaching for an algorithm when the database should do it	A Python join over a million rows	Let SQL join, with the index from Chapter 28
Quoting Big-O as if it were time	"It's $O(n)$, so it's fast"	Big-O is growth; the stopwatch is speed

In the real world: the invoice match that ran all afternoon

Riverstone's accounts team gets one file a month from its logistics partner, which bills Riverstone for every order line it delivered to customers. Somebody has to check each billed line against the order line it bills. Priya Nambiar, an accounts executive, wrote the script two years ago, and it worked: twenty minutes, run over lunch. (The file covers the whole business, not the one-year teaching database's key accounts.)

This month it hasn't finished by four o'clock, and the month-end close needs it.

Meera reads the script. It does exactly what a careful person would write: for each invoice line, loop through the order lines until you find a match. Two years ago that was 3,000 invoices against 40,000 order lines. Today it's 20,000 against 176,000, and the work is the product of the two: **almost seven times the invoices, four and a half times the lines, about thirty times the work.**

She times the current version on a slice, rather than guessing: 2,000 invoices take about 12 seconds, so the full file is about two minutes of matching... and the script had been running for six hours. The nested loop wasn't the whole story. The script also re-read the order file *inside* the loop, so the real cost was 20,000 file reads.

The fix takes twenty minutes:

1. **Read each file once.** That alone takes the job from hours to about two minutes.
2. **Build a dictionary** keyed by `(order_id, product_id)` and look each invoice up: 0.050 seconds to build, 0.007 seconds to match all 20,000.
3. **A set of already-processed invoice ids**, so a re-sent file doesn't double-count.
4. **A generator** for reading, so next year's bigger file doesn't have to fit in memory.

The final run takes about **0.06 seconds** to index and match, inside a script that spends most of its time reading the files. Meera adds Chapter 29's tests (the counts, and one hand-checked match), and a line to the log saying how many matched and how many didn't, because the 500 unmatched invoices are

the actual business output: those are the ones the logistics partner has to explain.

What she tells the accounts manager: *"It wasn't broken, it was written for a smaller file. It now finishes before you've finished reading the email, and it reports the 500 lines that don't match anything, which is the part you actually need."*

Notice the order she worked in: measure, find the real cost, fix the structure, then test. The clever algorithm was a dictionary, which is the first thing a Python programmer learns.

Project: make the matching script fast, and prove it

Goal: take a working but slow script, make it fast by changing structures rather than cleverness, and measure every step.

Tools you'll need

Versions used for this chapter, checked in September 2026:

- **Python 3.11 or later**, standard library only (outputs checked on Python 3.11.15, 3.12 and 3.13): `collections` (`deque`, `Counter`, `defaultdict`), `bisect`, `heapq`, `functools` (`lru_cache`), `time`, `timeit`, `tracemalloc`, `sys`.
- **Measuring:** `time.perf_counter()` for whole runs, `timeit` for small operations, `tracemalloc` for memory, and `cProfile` with `snakeviz` or `py-spy` when you need to find the slow line rather than confirm it.
- **When Python isn't the answer:** `pandas` and `NumPy` (Chapter 18) for vectorized work, the database (Chapter 28) for joins and aggregation over large tables, and `Polars` or `DuckDB` when a single machine has to chew through files.
- **Companion files** in `ch33/`: `generate_ch33_data.py` (176,110 order lines and 20,000 invoice lines from the logistics partner, seed 33), `match_slow.py` and `match_fast.py` (the project's before and after), and `ch33_timings.py`, which reproduces every timing in the chapter.

Tool note: profile before you optimize. `python3 -m cProfile -s cumtime script.py | head -20` names the functions where the time actually goes. Analysts routinely discover that 90% of a "slow algorithm" is CSV parsing, date conversion, or a database round trip in a loop. The algorithm is what's left after the measurement, not before it.

Option A: your own slow script. Anything that loops over rows and looks something up. Time it before you touch it.

Option B: Riverstone. Start from `match_slow.py` and rebuild it yourself before comparing with `match_fast.py`. `match_slow.py` has the nested loop but reads the order file once; to see where the story's hours went, add the re-read inside the loop yourself, run it on 250 invoices, and profile it.

Steps:

1. **Measure first.** Time the original on 250, 500, 1,000 and 2,000 invoices. Plot or tabulate the four numbers and say what shape they are.

2. **Predict.** From those numbers, how long would the full 20,000-line file take? Write the prediction down before running it.
3. **Find the real cost.** Profile it (`cProfile`). Is the time in the comparison, in parsing, or in reading the file?
4. **Index it.** Build a dictionary keyed by `(order_id, product_id)`, and rerun. Report the build time and the match time separately.
5. **Dedupe with a set.** Ignore invoice lines you have already processed, and prove it works by running the file twice.
6. **Stream it.** Convert the reader to a generator so memory stays flat, and check with `tracemalloc`.
7. **Reconcile.** Matched plus unmatched must equal the invoice count, and the totals must agree with the slow version's. Write those as tests (Chapter 29).
8. **Report both numbers:** the time, and the 500 unmatched invoice lines, which is what the business wanted all along.

Deliverables: the before-and-after scripts, a table of timings at each step, the tests, and three sentences on which change mattered most.

Stretch goals:

- Match on `(order_id, product_id)` **and** an amount tolerance of 2%, without making the lookup linear again.
- Handle one order line billed across two invoices: what has to change, and what does the grain become?
- Do the same job in SQL (`LEFT JOIN`, Chapter 28) and in pandas (`merge`, Chapter 18), and time all three.
- Make the script handle a 5 GB file on a laptop with 8 GB of memory.

REVIEW

Recap

- **Big-O** describes how work grows: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$. It's a shape, not a stopwatch, and quadratic is the shape that kills scripts.
- **Lists** keep order; **dictionaries** and **sets** find things in constant time; **tuples** are immutable and can be keys. The single most valuable habit in this chapter: **build an index before the loop**, which is a hand-written hash join.
- **Hashing** is why dictionary lookups don't slow down, why keys must be immutable, and how database hash joins work.
- **Stacks** are last in, first out; **queues** are first in, first out; use `collections.deque`, not `list.pop(0)`.
- **Recursion** needs a base case; Python's stack limit is 1,000 frames; use it for nested data, a loop for sequences.
- **Trees and graphs** are dictionaries of lists. **Depth-first** uses a stack, **breadth-first** uses a queue. On real data, guard them: a **seen** set for reachability, a **path** set for cycles and bills of materials.

- **Binary search** (`bisect`) is $O(\log n)$ on sorted data; **sorting** is $O(n \log n)$ and often makes everything after it easy. `heapq.nlargest` beats sorting for a top few.
- **Memoization** (`lru_cache`) turns repeated work into a lookup; **dynamic programming** fills a table of subproblems; **greedy** takes the best local step and is only sometimes right.
- **Memory** matters too: generators hold one row, lists hold all of them.
- **Measure first.** Profile, fix the real cost, and remember that the database or pandas may be the right answer instead of any algorithm.

Key terms

algorithm · complexity · Big-O · constant time · logarithmic time · linear time · linearithmic · quadratic · exponential · data structure · list · dictionary · set · tuple · immutability · index (in-memory) · generator expression · hash function · hash table · bucket · collision · hash join · stack · queue · deque · last in first out · first in first out · retry queue · recursion · base case · recursive case · call stack · recursion limit · tree · graph · node · edge · depth-first search · breadth-first search · seen set · path set · cycle detection · nested function · linear search · binary search · `bisect` · sorting · Timsort · stable sort · sort key · heap · `heapq` · memoization · decorator · `lru_cache` · dynamic programming · subproblem · greedy algorithm · generator · lazy evaluation · profiling · `cProfile` · `timeit` · `tracemalloc` · vectorization

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can name the complexity of a loop I wrote, and say what happens when the data doubles.
- ☐ I reach for a dictionary or a set the moment I see a lookup inside a loop.
- ☐ I can say why `x in a_set` is instant and `x in a_list` isn't.
- ☐ I can explain a hash table, and connect it to a database hash join.
- ☐ I use `deque` for queues and know why `pop(0)` is a trap.
- ☐ I can write a recursive function with a base case, and say when a loop would be clearer.
- ☐ I can walk a tree depth-first and breadth-first, and choose between a seen set and a path set.
- ☐ I know what `sorted(key=...)` costs and when sorting first makes everything easier.
- ☐ I recognize memoization, dynamic programming, and greedy, and know greedy needs checking.
- ☐ I think about memory as well as time, and use generators for big files.
- ☐ I measure before optimizing, and state complexity out loud in interviews.

Exercises

Work in `companion/ch33`, with the data built by `generate_ch33_data.py`. Predict each answer before running it.

Warm-up

1. What is the complexity of each: (a) summing a list; (b) checking `x in a_dict`; (c) `sorted(rows)`; (d) a loop over invoices with `if invoice_id in processed_list` inside?
2. Count the distinct `(order_id, product_id)` pairs in the order lines file, twice: once with a list of seen pairs, once with a set. Time both on the first 20,000 rows.
3. Build a dictionary of order lines by `order_id` (not by the pair). How many orders are there, and what is the largest number of lines on one order?
4. Use `Counter` to find which products account for most of the 500 **unmatched** invoice lines. Is one product's mapping broken, or is the problem somewhere else?

Core

5. Rewrite this quadratic snippet to run in linear time, and prove the results are identical: `[inv for inv in invoices if any(l["order_id"] == inv["order_id"] for l in order_lines)]`.
6. Write `band(amount)` with `bisect` to put order lines into size bands: tiny below ₹10,000, small from ₹10,000, medium from ₹25,000, large from ₹50,000, very large from ₹1,00,000. Test it on the boundary values exactly, and on a credit note of -500.
7. Walk the bill of materials in section 33.5 iteratively, with an explicit stack instead of recursion. Confirm you get the same quantities.
8. Find the 10 largest order lines by revenue with `sorted` and with `heapq.nlargest`, and time both on all 176,110 rows.
9. Detect whether a dictionary of dbt-style model dependencies contains a cycle, and report the cycle's members, not just that one exists.
10. Write a generator that yields only order lines above ₹50,000 from the file, and use it to compute the count and total without holding the file in memory.

Stretch

11. Match invoices to order lines where the amount must be within 2%, without a nested loop. What does the index have to hold?
12. Implement binary search yourself (no `bisect`) and test it against `bisect_left` on 1,000 random cases, including values that aren't present.
13. Memoize a function that parses dates from strings, and measure the difference on a file with many repeated dates.
14. Use `cProfile` on `match_slow.py` with 500 invoices. Which function has the highest cumulative time, and is it the one you expected?
15. Write the fewest-notes function from section 33.8 using recursion with `lru_cache` instead of a table. Which version would you rather maintain, and are they the same complexity?

Think about it (no code needed)

16. A colleague says "we'll just add more memory" to a script that is $O(n^2)$ in time. What do you say?
17. Your script does a database query inside a loop over 5,000 customers. Name three fixes, in order of how much they'd help.
18. When is an $O(n^2)$ algorithm perfectly acceptable?

Answers

1. (a) $O(n)$: every element is added. (b) $O(1)$: one hash, one bucket. (c) $O(n \log n)$: Timsort. (d) $O(n \times m)$: the `in` on a list is itself a scan, so the whole loop is quadratic. Changing `processed_list` to a set makes it $O(n)$, which is exercise 2's point.

2.

```
import csv, time

def load(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

rows = load("match_data/order_lines.csv")[:20_000]

def with_list(rows):
    seen = []
    for row in rows:
        key = (row["order_id"], row["product_id"])
        if key not in seen:
            seen.append(key)
    return seen

def with_set(rows):
    return {(row["order_id"], row["product_id"]) for row in rows}

list_time = time.perf_counter()
pairs_list = with_list(rows)
list_time = time.perf_counter() - list_time
set_time = time.perf_counter()
pairs_set = with_set(rows)
set_time = time.perf_counter() - set_time
```

```

print(f"list: {len(pairs_list):,} pairs")
print(f"set: {len(pairs_set):,} pairs")
print("same answer:", set(pairs_list) == pairs_set)
print("the set was faster by a factor of:", "hundreds" if set_time * 100 < list_time else "less than
100")

```

```

list: 20,000 pairs
set: 20,000 pairs
same answer: True
the set was faster by a factor of: hundreds

```

Same answer, and on the test machine the set version was about 940 times faster on these 20,000 rows (3.73 seconds against 0.004). The gap grows with the data, because the list version is $O(n^2)$: each not in scans everything found so far. On the full 176,110 rows it would take minutes; the set takes a fraction of a second.

3.

```

from collections import defaultdict

all_rows = load("match_data/order_lines.csv")
by_order = defaultdict(list)
for row in all_rows:
    by_order[row["order_id"]].append(row)

largest = max(by_order, key=lambda order_id: len(by_order[order_id]))
print(f"{len(by_order):,} orders")
print(f"largest order {largest} has {len(by_order[largest])} lines")
print("lines per order:", {n: sum(1 for v in by_order.values() if len(v) == n) for n in (1, 2, 3)})

```

```

66,667 orders
largest order 100001 has 3 lines
lines per order: {1: 1003, 2: 21885, 3: 43779}

```

Grouping into a dictionary of lists is the Python version of `GROUP BY`, and it's one pass. Note the grain (Chapter 14): at most one line per order and product, so no order can have more lines than there are products. In this data no order has more than three.

4.

```

from collections import Counter

invoices = load("match_data/carrier_invoice_lines.csv")
index = {(line["order_id"], line["product_id"]): line for line in all_rows}
unmatched = [invoice for invoice in invoices
              if (invoice["order_id"], invoice["product_id"]) not in index]
print(f"{len(unmatched)} unmatched invoice lines")
print("by product:", Counter(invoice["product_id"] for invoice in unmatched).most_common(3))
print("order ids from", min(i["order_id"] for i in unmatched), "to", max(i["order_id"] for i in
unmatched))

```

```

500 unmatched invoice lines
by product: [('106', 74), ('104', 68), ('101', 66)]
order ids from 900065 to 999880

```

No product stands out: 500 lines over eight products is about 62 each, and the top three are only a little above that. So no single product mapping is broken. The last line shows the real problem: every unmatched line bills an order id from 900065 upwards, and Riverstone's order ids in this file run from 100000 to 166666. The logistics partner is billing orders that don't exist, which is the question to send back to them. Counter answered "is it one product?" in one line, and the answer was no, which is worth knowing before anyone starts fixing product codes.

5.

```
order_ids = {line["order_id"] for line in all_rows}          # one pass, a set
linear = [inv for inv in invoices if inv["order_id"] in order_ids]

quadratic_sample = [inv for inv in invoices[:200]
                    if any(l["order_id"] == inv["order_id"] for l in all_rows)]
first_200_ids = {inv["invoice_line_id"] for inv in invoices[:200]}
print(f"linear version kept {len(linear):,} of {len(invoices):,} invoices")
print("same answer on the first 200:",
      [i["invoice_line_id"] for i in linear if i["invoice_line_id"] in first_200_ids]
      == [i["invoice_line_id"] for i in quadratic_sample])
```

```
linear version kept 19,500 of 20,000 invoices
same answer on the first 200: True
```

Note the set `first_200_ids`: even the check follows the chapter's rule, because `i in invoices[:200]` inside the list comprehension would scan 200 dictionaries for each of 19,500 invoices. The quadratic version is only run on 200 invoices here because the full version would take about a minute; that's the comparison worth internalizing.

6.

```
from bisect import bisect_right

thresholds = [0, 10_000, 25_000, 50_000, 100_000]
labels = ["tiny", "small", "medium", "large", "very large"]

def band(amount):
    if amount < thresholds[0]:
        raise ValueError(f"negative amount {amount}")
    return labels[bisect_right(thresholds, amount) - 1]

for amount in (0, 9_999, 10_000, 24_999, 25_000, 100_000, 100_001, -500):
    try:
        print(f"{amount:>8,}: {band(amount)}")
    except ValueError as exc:
        print(f"{amount:>8,}: ValueError: {exc}")
```

```
0: tiny
9,999: tiny
10,000: small
24,999: small
25,000: medium
100,000: very large
100,001: very large
-500: ValueError: negative amount -500
```

Boundaries are where banding bugs live: `bisect_right` puts a value *equal* to a threshold into the higher band, which matches "from ₹10,000". If the rule is "over ₹10,000", `bisect_left` is the one you want, but then 0 falls before the first threshold and index -1 silently wraps to the last label: negative indexes are the trap, so guard the low end. Write the test with the boundary values, not with 9,500 and 30,000.

7.

```
parts = {
    "Garden Chair": [("Seat shell", 1), ("Steel frame", 1),
                    ("Product label", 1), ("Shipping carton", 1)],
    "Seat shell": [("PP granules", 2.2), ("Masterbatch", 0.08)],
    "Steel frame": [("Leg assembly", 2), ("Steel tube", 1.0), ("Fastener pack", 1)],
    "Leg assembly": [("Steel tube", 1.2), ("Fastener pack", 1)],
}

def explode_iterative(part, quantity=1.0):
    totals, stack = {}, [(part, quantity)]
    while stack:
        current, current_quantity = stack.pop()
        if current not in parts:
            totals[current] = totals.get(current, 0) + current_quantity
            continue
        for child, child_quantity in parts[current]:
            stack.append((child, current_quantity * child_quantity))
    return totals

for material, amount in sorted(explode_iterative("Garden Chair").items()):
    print(f"{material:<16} {amount:>6.2f}")
```

Fastener pack	3.00
Masterbatch	0.08
PP granules	2.20
Product label	1.00
Shipping carton	1.00
Steel tube	3.40

Identical to the recursive version, because the stack is what recursion was using. The iterative form has no depth limit, which matters for deep structures; the recursive form is shorter to read. Choose per structure, and say which you chose in a code review.

8.

```
import heapq, time

values = [(float(row["net_revenue"]), row["order_id"]) for row in all_rows]

started = time.perf_counter()
top_sorted = sorted(values, reverse=True)[:10]
sort_time = time.perf_counter() - started

started = time.perf_counter()
top_heap = heapq.nlargest(10, values)
heap_time = time.perf_counter() - started

print("same answer:", top_sorted == top_heap)
print("largest line:", top_sorted[0])
print("nlargest was quicker:", heap_time < sort_time)
```



```
same answer: True
largest line: (140000.0, '166653')
nlargest was quicker: True
```

On the test machine that was 0.048 seconds against 0.006. `sorted` orders all 176,110 values to keep 10; `nlargest` keeps a heap of 10 and scans once. The gap widens as the data grows and narrows as k approaches n ; for k near n , just sort.

9.

```
models = {"stg_orders": [], "fct_sales": ["stg_orders", "dim_customer"],
          "dim_customer": ["snapshot"], "snapshot": ["fct_sales"]}

def find_cycle(graph):
    done = set()
    def visit(node, path):
        if node in path:
            return path[path.index(node):] + [node]
        if node in done:
            return None
        for neighbor in graph.get(node, []):
            found = visit(neighbor, path + [node])
            if found:
                return found
        done.add(node)
        return None
    for node in graph:
        found = visit(node, [])
        if found:
            return found
    return None

print("cycle:", " -> ".join(find_cycle(models)))
```

```
cycle: fct_sales -> dim_customer -> snapshot -> fct_sales
```

Carrying the path as a list, in order, rather than a bare set is what lets you report the members: `path.index(node)` finds where the loop starts, and the slice from there is the cycle. The `done` set keeps it $O(V + E)$, as in section 33.6. Naming the members is what makes the error useful, which is why dbt's own message for a circular `ref` lists the models in the loop, and why Chapter 28's `CYCLE` clause keeps the path in SQL.

10.

```
import csv

def big_lines(path, floor=50_000):
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            value = float(row["net_revenue"])
            if value > floor:
                yield value

count = total = 0
for value in big_lines("match_data/order_lines.csv"):
    count += 1
    total += value
print(f"{count:}, lines above 50,000, worth {total:,.0f}")
```

```
25,133 lines above 50,000, worth 2,175,373,500
```

One row is in memory at a time, so this works on a file of any size. The trade: the generator can be read only once, so computing a count and a total means either two passes or accumulating both in one loop, as here.

11. The index can't be keyed on the amount, because the tolerance makes it a range rather than a value. Key it on (`order_id`, `product_id`) as before, and check the invoice's `amount` against the order line's `net_revenue` *after* the lookup:

```
line = index.get((invoice["order_id"], invoice["product_id"]))
if line is not None and abs(float(invoice["amount"]) - float(line["net_revenue"])) <= 0.02 *
    float(line["net_revenue"]):
    ...
```

The lookup stays $O(1)$ and the tolerance is one comparison. If you had to match on amount *alone*, sort the lines by amount once and use `bisect` to find the window, which is $O(\log n)$ per invoice: the general lesson is that exact matches want a hash, and range matches want sorted data.

12.

```
from bisect import bisect_left
import random

def my_bisect_left(values, target):
    low, high = 0, len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < target:
            low = middle + 1
        else:
            high = middle
    return low

numbers = sorted(random.Random(33).sample(range(1_000_000), 5_000))
rng = random.Random(34)
tests = [rng.randrange(1_000_000) for _ in range(1_000)]
tests += [numbers[0], numbers[-1], -1, 1_000_000, numbers[100]] # first, last, below, above,
    present
print("distinct test values:", len(set(tests)))
print("matches bisect_left on every case:",
```

```
all(my_bisect_left(numbers, t) == bisect_left(numbers, t) for t in tests))
print("example:", my_bisect_left(numbers, numbers[100]), bisect_left(numbers, numbers[100]))
```

```
distinct test values: 1002
matches bisect_left on every case: True
example: 100 100
```

Create the random generator once: `random.Random(34)` inside the loop would restart the same sequence every time, so all 1,000 "random" cases would be the same number, a classic testing bug. The two traps when writing binary search by hand are the loop condition (`<` against `<=`) and the update (`middle + 1` against `middle`), which is why the test includes values that aren't in the list and both ends of it.

13. Wrap the parser in `@lru_cache` and call it on a column with many repeats:

```
from datetime import date
from functools import lru_cache

@lru_cache(maxsize=None)
def parse_date(text):
    year, month, day = (int(part) for part in text.split("-"))
    return date(year, month, day)

dates = [row["order_date"] for row in all_rows]
parsed = [parse_date(text) for text in dates]
info = parse_date.cache_info()
print(f"{len(dates):,} values, {len(set(dates)):,} distinct")
print(f"cache hits {info.hits:,}, misses {info.misses:,}")
print("earliest:", min(parsed), "latest:", max(parsed))
```

```
176,110 values, 700 distinct
cache hits 175,410, misses 700
earliest: 2024-01-01 latest: 2025-11-30
```

700 distinct dates in 176,110 rows, so the cache answers 99.6% of the calls. This is the shape of case where `lru_cache` is worth it: a pure function, expensive relative to a dictionary lookup, called repeatedly with few distinct inputs. It would be worth nothing if every value were unique.

14. Run `python3 -m cProfile -s cumtime match_slow.py 500 | head -15`. The highest cumulative time is `match`, as expected, but look at the next lines: on this data the comparisons themselves dominate, while `load` and `csv.DictReader` account for a fraction of the run. That's the useful outcome either way: profiling either confirms your suspicion (fix the algorithm) or overturns it (fix the parsing, the file reading, or the database calls), and guessing is often wrong.

15.

```
from functools import lru_cache

NOTES = (2000, 500, 200, 100, 50, 20, 10)

@lru_cache(maxsize=None)
def fewest_recursive(amount):
    if amount == 0:
        return 0
    options = [fewest_recursive(amount - note) for note in NOTES if note <= amount]
    options = [o + 1 for o in options if o is not None]
    return min(options) if options else None

for amount in (760, 2750, 45):
    print(f"₹{amount:>5}: {fewest_recursive(amount)}")
```

```
₹ 760: 4
₹ 2750: 4
₹ 45: None
```

Same answers, same complexity: with the cache, each amount is solved once, which is precisely what the table version did. The recursive version reads closer to the definition of the problem; the table version has no recursion limit to worry about and is easier to reason about for large amounts. Most teams take the recursive one for clarity and add a comment saying why the cache is there.

16. Memory isn't the constraint; time is. Doubling the memory does nothing for an algorithm whose work is quadratic, and the job will keep getting slower as the data grows. Say it in terms of growth: *"this takes four times as long each time our data doubles, so if it takes an hour today, next year it takes four hours, not two. Changing the lookup to a dictionary makes it linear, and it's twenty minutes of work."* If it really is memory-bound rather than time-bound, that's a different conversation, and a generator is usually cheaper than more hardware.

17. In order: (1) **Do it in one query.** Fetch all 5,000 customers' rows in a single statement with `WHERE customer_id IN (...)` or a join, then index the result in Python. That removes 4,999 round trips. (2) **Push the work into SQL entirely.** If the loop is computing something the database could compute, let it (Chapter 28). (3) **If the loop must stay, batch it:** 50 queries of 100 ids beat 5,000 of one. A fourth, only if the first three aren't available: cache repeated lookups. Network round trips inside loops are the single most common performance bug in analyst code, and they don't show up in a Big-O count at all.

18. When *n* is small and stays small: ranking ten regions, comparing eight products, reconciling two dozen accounts. When it runs once, by hand, and finishing in a minute is fine. When the quadratic version is plainly correct and the linear one would need an index that complicates the code for a file that will never grow. The judgment is about ***n* and its future**, not about the notation: write the simple version, measure it on real data, and change it when the measurement says so.

Where this leads

- **Chapter 35, The Math Under the Models**, does whole-matrix arithmetic in NumPy, the vectorized style this chapter's first Watch out describes, and shows why it beats a loop.
- **Chapter 46, Pipelines & Orchestration**, and **Chapter 49, Storage, Warehouses & Lakehouses**, are where these costs meet scheduled jobs and cloud bills.
- **Chapter 48, Big Data & Distributed Compute**, splits data into partitions across machines; its broadcast join is this chapter's dictionary index, copied to every machine.
- **Chapter 55, Building AI Applications**, builds a search index over documents, where the same trade of memory for lookup speed decides how retrieval scales.
- **Chapter 72A, Data Structures & Algorithms Question Bank**, has the coding questions (Chapter 72 has the Python and pandas ones); **Chapter 69** covers how to talk through them.

Looking back: Chapter 28 is the database version of this chapter (indexes are hash tables and B-trees, and a hash join is section 33.2's dictionary); Chapter 29 is how a fast script becomes a maintained one; and Chapter 18's pandas covers the vectorized alternatives that often beat any loop you could write.

Chapter 30. Inference & Experiments

Chapter at a glance

You will learn to: compute a standard error and a confidence interval, by hand and in Python · compare two groups with a t-test and know when it doesn't apply · report an effect size, not only a p-value · test categorical results with a two-proportion test and chi-square · compare several groups with ANOVA without inflating your error rate · calculate the sample size a test needs before you run it · design an A/B test end to end, and write the plan down first · analyze one properly, with a sample-ratio check and guardrail metrics · recognize peeking, multiple testing, novelty effects, and outliers by watching them happen in simulations · read regression coefficients as statements about the business.

Before you start: Chapter 18 (pandas, and the NumPy basics in section 18.1), Chapter 21 (distributions, spread, sampling, and the central limit theorem, section 21.8), Chapter 22 (confidence intervals, p-values, chi-square, Type I and II errors, A/B test design, peeking, multiple comparisons, and regression basics in section 22.10), and Chapter 29 (uv projects, and Python as tested code). The box "What's new since Chapter 22", after In plain English, says which parts of this chapter are a recap and which are new.

Time needed: 18–22 hours, spread over three weeks, in four sittings: (1) sections 30.1–30.4, standard errors, intervals, the t-test and effect size, about 5 hours; (2) sections 30.5–30.7, proportions, chi-square, ANOVA, power and sample size, about 5 hours, ending with a checkpoint; (3) sections 30.8–30.10, designing the test, analyzing it end to end, and the ways to fool yourself, about 5 hours; (4) sections 30.11–30.12, regression for inference and the write-up, about 4 hours. Allow 2 more for the project.

Tools: Python 3.14 as installed in Chapter 17 (any version from 3.11 on runs every example), in a small uv project for this chapter (section 30.1 sets it up), with `pandas`, `numpy`, `scipy`, `statsmodels`, `matplotlib`, `SQLAlchemy`, and `psycpg`. Everything runs on one laptop in seconds.

Practice data: Riverstone's website data, built by `generate_riverstone_web.py` in the companion folder: 214,528 sessions, 622,090 events, and 47,286 visitors in a two-week A/B test. Fixed seed, so your numbers match the book's exactly. The one-year sales database from Chapter 13, `riverstone_2025`, is used for the examples about order values.

In this chapter

START	Why this matters · In plain English	300
LEARN	30.1 The data, and the question	301
	30.2 Standard error and confidence intervals	304
	30.3 Comparing two groups: the t-test	308
	30.4 Effect size: how big, not just whether	311
	30.5 Categorical outcomes: proportions and chi-square	312
	30.6 More than two groups: ANOVA	315
	30.7 Power and sample size	319
	30.8 Designing an experiment	323
	30.9 Riverstone's website test, end to end	324
	30.10 Four ways to fool yourself, demonstrated	330
	30.11 Regression for inference	334
	30.12 Writing the result up	339
APPLY	Common mistakes · In the real world · Project	340
REVIEW	Recap · Key terms · Check yourself	343
PRACTISE	Exercises	344
	Answers	346
NEXT	Where this leads	352

START

Why this matters

Riverstone's marketing agency proposes a new enquiry form. Two weeks later they report: "Enquiries are up 14%. Roll it out everywhere."

Four questions decide whether that sentence is worth anything, and all four are this chapter:

- **Is it real, or is it noise?** 14% of what? Over how many visitors? A fortnight of ordinary variation can produce bigger swings than that.
- **How big is the effect, and how sure are we?** "Up 14%" is a point estimate. The honest version is a range, and the range is often wide enough to change the decision.
- **Was the test run properly?** Were visitors split evenly? Did someone check the numbers daily and stop when they looked good? Did the agency test five things and report the one that won?
- **What do we do now?** A result nobody can act on is a cost, not a finding.

Being the person who asks these questions, calmly and with the arithmetic to back them up, is most of what separates a senior analyst from a junior one. It's also the densest interview topic in the whole book: Chapter 73's bank is full of "how would you design this test" and "here's a result, what's wrong with it".

In plain English

Inference is what you say about a whole crowd after talking to a few people in it.

Suppose you want to know the average height of everyone at a wedding, and you measure ten guests. Your ten-person average won't be exactly the crowd's average, but it won't be wildly off either. The **standard error** measures how far off it's likely to be. Measure forty guests instead of ten, and the error halves: to be twice as precise you need four times as many people, which is why sample size questions have expensive answers.

A **confidence interval** turns that into a sentence you can say out loud: "the average is 168 cm, give or take 3 cm." A **hypothesis test** answers a narrower question: "if the two sides of the room were really the same height, how surprising is the difference I measured?" Surprising enough, and you stop believing they're the same. The **p-value** is the size of that surprise, and nothing else: not the chance you're wrong, and not the size of the difference.

An **experiment** is how you get a clean answer instead of an argument. Rather than comparing people who happened to see the new form with people who happened not to, you flip a coin for each visitor. Once the coin decides, the two groups differ only by chance and by the form, so a difference between them points at the form.

Three things ruin this, and all three are ordinary human behavior: stopping the moment the numbers look good (**peeking**), trying twenty ideas and reporting the one that worked (**multiple testing**), and the fact that anything new attracts attention for a week (**novelty**). The rest of the chapter is arithmetic; these three are discipline.

What's new since Chapter 22. Chapter 22 taught the ideas on small, tidy examples. This chapter uses them on a real-sized website test, and adds the tools a working analyst needs on top.

- **Recap, kept short, with pointers back:** standard errors and confidence intervals (section 22.1), p-values, chi-square and the two-proportion z-test (section 22.2), A/B test design and sample size (section 22.3), peeking and multiple comparisons (section 22.4), and fitting a line (section 22.10).
- **New here:** the unit of analysis; effect sizes (Cohen's d, Cramér's V, Cohen's h); ANOVA and Tukey's test; the sample-size formula checked against statsmodels; a whole test analyzed end to end; the sample-ratio check; guardrail metrics; novelty; regression with several variables and categories; and logistic regression with odds ratios.

LEARN

30.1 The data, and the question

Riverstone's website, `riverstone.example`, has three tables in the companion data:

Table	One row is	Rows
<code>web_sessions</code>	one visit: device, browser, channel, region, pages viewed, and whether the visitor sent an enquiry	214,528
<code>web_events</code>	one action inside a session: a page view, a form start, a form submit	622,090
<code>ab_test_assignments</code>	one visitor in the enquiry-form test, and which version they were shown	47,286

Setting up this chapter's project

Chapter 29 built a full package with uv. This chapter needs much less: a folder whose Python environment has the right libraries, with the versions written down in a lockfile. That's a **bare** uv project. In a terminal, in the companion folder:

```
# terminal, in companion/ch30
$ uv init --bare
Initialized project `ch30`
$ uv add pandas numpy scipy statsmodels matplotlib sqlalchemy "psycogp[binary]"
Using CPython 3.14.7
Creating virtual environment at: .venv
Resolved 25 packages in 515ms
Prepared 23 packages in 3.10s
Installed 23 packages in 64ms
+ contourpy==1.4.0
+ cycler==0.12.1
...
$ uv add --dev ipykernel pytest
Resolved 57 packages in 697ms
Prepared 28 packages in 1.05s
Installed 28 packages in 118ms
+ asttokens==3.0.2
+ comm==0.2.3
...
$ uv run python -c "import sys, scipy, statsmodels; print(sys.version.split()[0], scipy.__version__,
statsmodels.__version__)"
3.14.7 1.18.1 0.15.0
```

(The two package lists are shortened here: the first installs 23 packages, including pandas 3.0.6, numpy 2.5.3, scipy 1.18.1, and statsmodels 0.15.0; the second installs 28. Your version numbers will be the same or newer.)

- **uv init --bare** creates only `pyproject.toml`: no `src/` folder and no sample program, because this chapter's code lives in a notebook, not in a package.
- **uv add ...** lists the libraries the chapter uses, creates `.venv`, and writes `uv.lock`, exactly as in section 29.4. `psycogp[binary]` is quoted because some terminals treat square brackets specially.
- **uv add --dev ipykernel pytest** adds two tools you only need while working: `ipykernel` lets VS Code run notebook cells with this environment (the **kernel** from Chapter 17, section 17.0), and `pytest` runs the one test in section 30.9.
- **uv run python -c "..."** runs one line of Python inside the project's environment and prints the versions, as a check.

Then open the folder in VS Code, create a notebook, and choose the `.venv` environment with **Select Kernel**, as in section 17.0. Every Python cell in this chapter runs in that one notebook, in order.

Tool note: or keep your Chapter 17 environment. It already has every library this chapter uses: pandas, matplotlib, SQLAlchemy, and psycogp from Chapter 18, scipy from Chapter 21, and statsmodels from Chapter 22. The uv project is here because it pins this chapter's versions in `uv.lock`, which is what Chapter 29 recommended for anything you'll want to rerun later.

Building the data

The generator needs only the Python standard library. It takes about five seconds, and creates a folder `web_data/` with the three tables as CSV files, plus two scripts for loading them into a database:

```
# terminal, in companion/ch30
$ uv run python generate_riverstone_web.py
sessions 214,528  events 622,090  visitors in the test 47,286
$ ls web_data
ab_test_assignments.csv
load_mysql.sql
load_postgresql.sql
web_events.csv
web_sessions.csv
```

`uv run python` works the same on Windows, macOS, and Linux, so there's no need to remember whether your system calls Python `python3` or `py`.

The test itself, `enquiry_form_2026_02`, ran from 2 to 15 February 2026. Half the visitors saw the existing enquiry form (**control**), half saw a shorter one with fewer required fields (**variant_b**). The business question: *does the shorter form bring more enquiries?*

The first cells

The first cell loads every library the chapter uses, so later cells don't repeat imports:

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from sqlalchemy import create_engine
from statsmodels.stats.proportion import proportions_ztest, confint_proportions_2indep,
    proportion_effectsize
from statsmodels.stats.power import NormalIndPower, TTestIndPower
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from statsmodels.stats.multitest import multipletests
import statsmodels.formula.api as smf
print("ready")
```

```
ready
```

- `os`, `numpy`, `pandas`, `matplotlib.pyplot` and `scipy.stats` are old friends (Chapters 17, 18, and 21); `create_engine` from SQLAlchemy connects to a database, as in section 18.13.
- The `statsmodels` lines each bring in a few functions from one part of the library. You met `proportions_ztest` in section 22.2; the others are new, and each is explained where it's first used: `proportions` in section 30.5, `power` in section 30.7, Tukey's test in section 30.6, multiple-testing corrections in section 30.10, and `smf`, the formula interface for regression, in section 30.11.
- `print("ready")` is only there so the cell shows something when it has finished.

The second cell reads the data:

```
sessions = pd.read_csv("web_data/web_sessions.csv", parse_dates=["started_at"])
assignments = pd.read_csv("web_data/ab_test_assignments.csv", parse_dates=["assigned_at"])

print(sessions.shape, assignments.shape)
print(sessions[["device", "channel", "pages_viewed", "enquiry_submitted"]].head(3))
print(f"overall enquiry rate: {sessions['enquiry_submitted'].mean():.4f}")
```

```
(214528, 12) (47286, 4)
   device  channel  pages_viewed  enquiry_submitted
0  desktop  referral             1                  0
1  desktop  referral             3                  0
2  desktop  organic             2                  0
overall enquiry rate: 0.0359
```

- `parse_dates=["started_at"]` reads that column as dates and times rather than text (section 18.9), so later cells can compare and sort by it.
- `.shape` gives (rows, columns) for each table; `.head(3)` shows the first three rows of four chosen columns.
- `enquiry_submitted` is 1 when the session sent an enquiry and 0 when it didn't, so its mean is the share of sessions with an enquiry: 3.59%.

Simplification note. Riverstone's site is a stand-in for any website with a form on it, and the data is generated, not collected. Real clickstream data is messier: bots, blocked cookies, visitors who switch devices, sessions that never end. The statistics in this chapter are exactly what you'd run on real data; the cleaning that comes first is Chapters 14 and 47.

30.2 Standard error and confidence intervals

(A recap of Chapter 22, section 22.1, now on a large dataset.) Every sample gives a number that isn't quite the truth. The **standard error** (SE) says how much that number wobbles from sample to sample.

For an average, by hand

For a mean, the standard error is the standard deviation divided by the square root of the sample size. Start with five sessions, small enough to do with a pencil: they lasted 40, 75, 90, 120, and 210 seconds.

Session	Seconds, x	$x - \bar{x}$	$(x - \bar{x})^2$
1	40	-67	4,489
2	75	-32	1,024
3	90	-17	289
4	120	13	169
5	210	103	10,609
Total	535	0	16,580

1. Mean: $\bar{x} = 535 \div 5 = \mathbf{107 \text{ seconds}}$.
2. Standard deviation: $s = \sqrt{(16,580 \div (5 - 1))} = \sqrt{4,145} = \mathbf{64.38 \text{ seconds}}$. Dividing by $n - 1$ is Chapter 21's rule for a sample (section 21.2).
3. Standard error: $64.38 \div \sqrt{5} = 64.38 \div 2.236 = \mathbf{28.79 \text{ seconds}}$.
4. With five values, use the t-distribution's multiplier with $5 - 1 = 4$ degrees of freedom, $t^* = 2.776$ (section 22.1), not 1.96. Margin: $2.776 \times 28.79 = 79.94$.
5. Interval: $107 - 79.94$ to $107 + 79.94 = \mathbf{27.1 \text{ to } 186.9 \text{ seconds}}$.

In a spreadsheet, with the five values in A2:A6: `=STDEV.S(A2:A6)/SQRT(COUNT(A2:A6))` gives the standard error, 28.79, and `=CONFIDENCE.T(0.05,STDEV.S(A2:A6),5)` gives the margin, 79.94.

An interval 160 seconds wide says almost nothing. That's what five sessions buy.

The same recipe in pandas

Here are the same steps on all 214,528 sessions:

```
duration = sessions["duration_seconds"]
n = len(duration)
mean = duration.mean()
sd = duration.std(ddof=1)
se = sd / np.sqrt(n)

print(f"n          = {n:,}")
print(f"mean        = {mean:.2f} seconds")
print(f"sd          = {sd:.2f}")
print(f"se          = {se:.3f}")
print(f"95% CI      = {mean - 1.96 * se:.2f} to {mean + 1.96 * se:.2f} seconds")
```

```
n          = 214,528
mean       = 96.17 seconds
sd         = 114.91
se         = 0.248
95% CI     = 95.68 to 96.65 seconds
```

- `duration.std(ddof=1)` is the sample standard deviation, dividing by $n - 1$ as in step 2 (Chapter 21, section 21.2).
- `np.sqrt(n)` is $\sqrt{n}$, so `se` is step 3. The last line is the interval with 1.96 as the multiplier: with 214,528 values, t^* and 1.96 are the same to three decimals.

Reading it: individual sessions vary enormously (a standard deviation of 115 seconds against a mean of 96), but the *average* of 214,528 of them is known to within a quarter of a second: the 95% interval is 95.68 to 96.65. That's the whole idea: the sample mean is far more stable than the thing it's averaging.

The **1.96** is the number of standard errors that contains 95% of a normal distribution. For small samples, use the t-distribution instead, as in the hand calculation. `scipy` does it in one call, which section 22.1 introduced. Take a random 40 sessions:

```
sample = sessions["duration_seconds"].sample(40, random_state=30)
low, high = stats.t.interval(0.95, df=len(sample) - 1, loc=sample.mean(),
                             scale=stats.sem(sample))
print(f"40 sessions: mean {sample.mean():.1f} s, 95% CI {low:.1f} to {high:.1f} s")
```

40 sessions: mean 64.0 s, 95% CI 47.8 to 80.2 s

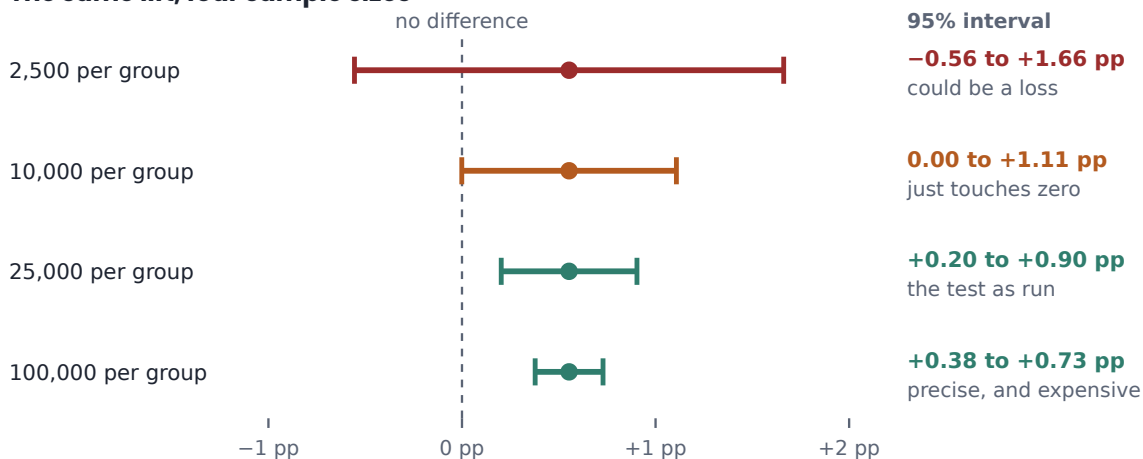
- `.sample(40, random_state=30)` picks 40 rows at random; `random_state=30` fixes the draw so you get the same 40 (Chapter 21, section 21.5).
- `stats.sem(sample)` is the **standard error of the mean**, $s \div \sqrt{n}$, in one call. It uses $n - 1$ by default.
- `stats.t.interval(0.95, df=..., loc=..., scale=...)` returns the two ends: the confidence level, the degrees of freedom ($40 - 1 = 39$), the center (`loc`, the sample mean), and the standard error (`scale`), exactly as in section 22.1. `low, high = ...` unpacks the two ends into two names.

Notice that this interval, 47.8 to 80.2 seconds, does not contain 96.17 seconds, the mean of all 214,528 sessions. That can happen: 95% intervals miss 1 time in 20. With skewed data like this it happens more often in small samples, because a sample of 40 rarely includes the few very long visits that pull the mean up (the longest session lasts 4,300 seconds). Section 30.3 comes back to skewed data; for measures like this, prefer larger samples, or the bootstrap from section 22.2.

Forty sessions give an interval tens of seconds wide; 214,528 give one a fraction of a second wide.

Precision costs data, and it costs it at the square root: four times the data for twice the precision.

The same lift, four sample sizes



Every row measured the same +0.55 points (dot); only the sample size changed.

Figure 30.1 – The same measured lift, four sample sizes. Only the sample size changed, and with it what you may say.

For a proportion

Conversion rates are proportions, and their standard error has its own formula: the square root of $p(1 - p) / n$ (section 22.1). A rate needs a unit to count, and the test randomizes visitors, so first make one row per visitor:

```
visitors = (sessions.groupby("visitor_id")["enquiry_submitted"].max()
            .rename("enquired").reset_index())
print(visitors.head(3))
print(len(visitors))
```

```

    visitor_id  enquired
0    v00000001         0
1    v00000002         0
2    v00000003         0
176000

```

Read the chain one step at a time:

- `groupby("visitor_id")` makes one group per visitor, holding all of that visitor's sessions.
- `["enquiry_submitted"].max()` takes the largest value in each group. The values are 0 and 1, so the maximum is 1 if *any* of the visitor's sessions sent an enquiry: it means "ever enquired".
- `.rename("enquired")` gives the result that name; `.reset_index()` turns `visitor_id` back from the row labels into an ordinary column.

176,000 visitors made the 214,528 sessions. Now the rate, its standard error, and the interval:

```

k = int(visitors["enquired"].sum())
n = len(visitors)
p = k / n
se = np.sqrt(p * (1 - p) / n)

print(f"{k:,} of {n:,} visitors sent an enquiry")
print(f"rate      = {p:.4f}  ({p * 100:.2f}%)" )
print(f"se       = {se:.5f}")
print(f"95% CI    = {100 * (p - 1.96 * se):.2f}% to {100 * (p + 1.96 * se):.2f}%")

```

```

7,643 of 176,000 visitors sent an enquiry
rate      = 0.0434  (4.34%)
se       = 0.00049
95% CI    = 4.25% to 4.44%

```

- `visitors["enquired"].sum()` counts the 1s, the visitors who enquired; `int(...)` makes it a plain whole number.
- `se` is $\sqrt{p(1-p)/n}$, line for line, and the interval is $p \pm 1.96 \times se$, printed as percentages.

Watch the unit of analysis. The rate above is per *visitor*, not per *session*: a visitor who came three times and enquired once counts as one converted visitor. The test randomizes visitors, so visitors are what you count. Mixing the two is one of the most common errors in A/B analysis, and it quietly inflates your sample size, which makes everything look more significant than it is.

Watch out: a confidence interval is about the method, not the number. "95% confident" means that if you repeated this sampling many times, 95% of the intervals you built this way would contain the true value. It does *not* mean there's a 95% chance the truth is inside this particular interval; the truth is fixed, and the interval is what varies. In a meeting, say it as a range: "*between 4.25% and 4.44%*". That's what people actually need.

30.3 Comparing two groups: the t-test

"Do Wholesale customers place bigger orders than Retail ones?" This is Chapter 13's sales data, and the tool is Welch's **two-sample t-test** from section 22.2: it asks how surprising the difference between two averages would be if the two groups really had the same average.

First, the orders. The connection details come from the environment variable `RIVERSTONE_DATABASE_URL`, set as in section 29.3, so the password never appears in the notebook. Set it in the terminal before you start VS Code, or keep it in a `.env` file that Git ignores:

```
engine = create_engine(os.environ["RIVERSTONE_DATABASE_URL"])
orders = pd.read_sql("""
    SELECT sl.order_id, c.segment, SUM(sl.net_revenue) AS order_value
    FROM sales_lines AS sl
    JOIN customers AS c ON c.customer_id = sl.customer_id
    GROUP BY sl.order_id, c.segment
""", engine)
print(orders.shape)
print(orders["segment"].value_counts().sort_index())
```

```
(173, 3)
segment
Hospitality    59
Retail         59
Wholesale     55
Name: count, dtype: int64
```

- `os.environ["RIVERSTONE_DATABASE_URL"]` reads the variable. If it isn't set, the cell stops with a `KeyError` naming it, which tells you exactly what to fix.
- The query adds up each order's lines from Chapter 13's `sales_lines` view, with the customer's segment. `sales_lines` already carries `order_id`, so no join to `orders` is needed, and it already leaves out cancelled orders (section 13.2): that's why there are 173 orders, not 175.
- `pd.read_sql(query, engine)` runs it and returns a `DataFrame` (section 18.13).
`.value_counts().sort_index()` counts the orders per segment, in alphabetical order.

Next, the two groups to compare:

```
retail = orders.loc[orders["segment"] == "Retail", "order_value"]
wholesale = orders.loc[orders["segment"] == "Wholesale", "order_value"]

print(f"Retail:    n = {len(retail):3d}, mean = {retail.mean():9,.0f}, sd = {retail.std():9,.0f}")
print(f"Wholesale: n = {len(wholesale):3d}, mean = {wholesale.mean():9,.0f}, sd = {wholesale.std():9,.0f}")
```

```
Retail:    n = 59, mean = 25,233, sd = 15,719
Wholesale: n = 55, mean = 30,957, sd = 21,094
```

`.loc[condition, "order_value"]` keeps the rows of one segment and the one column (section 18.4). In the f-strings, `:3d` prints a whole number in 3 characters and `:9,.0f` a number with thousands commas, no decimals, in 9 characters, so the columns line up.

Now the test:

```
result = stats.ttest_ind(wholesale, retail, equal_var=False)
print(f"difference = {wholesale.mean() - retail.mean():.0f}")
print(f"t = {result.statistic:.3f}, p = {result.pvalue:.4f}, df = {result.df:.1f}")
ci = result.confidence_interval()
print(f"95% CI for the difference: {ci.low:.0f} to {ci.high:.0f}")
```

```
difference = 5,724
t = 1.634, p = 0.1055, df = 99.5
95% CI for the difference: -1,228 to 12,676
```

How it works:

- **equal_var=False** runs **Welch's t-test**, which doesn't assume the two groups have the same spread. Use it by default: it costs almost nothing when the spreads do match, and it's right when they don't.
- **t** is the difference measured in standard errors. By hand, Welch's standard error is $\sqrt{s_1^2/n_1 + s_2^2/n_2} = \sqrt{(21,094^2/55 + 15,719^2/59)} = ₹3,504$, and $t = 5,724 \div 3,504 = 1.63$. **p** is the chance of seeing a difference at least this large if the true difference were zero.
- **df**, the degrees of freedom, come from a formula that allows for the two spreads being unequal: 99.5 here, a little under the $n_1 + n_2 - 2 = 112$ an equal-spread test would use. Fewer degrees of freedom mean a slightly larger t^* and a slightly wider interval.
- `result.confidence_interval()` is a method of the result, so it needs the brackets. It returns an object with two parts, `.low` and `.high`: the **confidence interval for the difference**. That's the part to quote. It answers "how much bigger?", which is the business question; the p-value only answers "could this be nothing?".

What this particular result says. Wholesale orders averaged ₹30,957 against Retail's ₹25,233, a difference of ₹5,724, but $p = 0.1055$ and the interval runs from -₹1,228 to +₹12,676. The interval contains zero, so 114 orders can't tell these two segments apart: the honest answer is "we don't know yet", not "Wholesale orders are 23% bigger". That's Chapter 22's warning about small samples, now measured. Section 30.7 works out how many orders would be needed to settle it.

What the p-value does not say

A recap of section 22.2's table, because these four misreadings turn up in every meeting:

People often say	What it actually means
"There's a 2% chance the result is wrong"	No. It's the chance of data this extreme <i>if there were no difference at all</i>
"p = 0.04 means the effect is real; p = 0.06 means there's nothing there"	0.05 is a convention, not a boundary in nature. Report the number and the interval
"p = 0.001 means the effect is big"	No. With enough data, a trivial difference gets a tiny p-value. See section 30.4
"p = 0.3 proves the two are the same"	No. It means this test couldn't tell them apart, which is also what too little data looks like

When a t-test isn't the right tool

The t-test compares means and assumes the *sample means* are roughly normally distributed. With samples of 30 or more that's usually fine even for skewed data, thanks to the central limit theorem (Chapter 21). Two situations need care:

- **Very skewed data with heavy tails**, such as Riverstone's enquiry values, where the largest enquiry (₹43.9 lakh) is worth nearly 200 ordinary ones (the median is ₹22,747). Then the mean itself may be the wrong summary: compare medians (the Mann-Whitney U test), or analyze log values, or cap the extremes, and say which you did.
- **Data that isn't independent**: several sessions from the same visitor, or several orders from the same customer. Aggregate to the randomization unit first, as section 30.2 did.

```
enquiries = sessions.loc[sessions["enquiry_submitted"] == 1, "enquiry_value"]
print(f"n = {len(enquiries):,}")
print(f"mean   = {enquiries.mean():,.0f}")
print(f"median = {enquiries.median():,.0f}")
print(f"largest three: {[f'{v:,.0f}' for v in enquiries.nlargest(3)]}")
print(f"the top 1% of enquiries hold {100 * enquiries.nlargest(len(enquiries) // 100).sum() /
      enquiries.sum():.1f}% of the value")
```

```
n = 7,711
mean   = 41,045
median = 22,747
largest three: ['4,388,000', '2,679,214', '2,002,875']
the top 1% of enquiries hold 16.6% of the value
```

- `enquiries.nlargest(3)` returns the three biggest values (Chapter 21). The list comprehension inside the f-string, `[f'{v:,.0f}' for v in ...]` (Chapter 17), formats each one; the `:,` format groups digits in threes, so 4,388,000 is ₹43,88,000 in the Indian style.
- `len(enquiries) // 100` is 1% of the count, as a whole number (`//` divides and drops the remainder): the 77 largest enquiries. Their total, divided by the total of all enquiries, is the top 1%'s share.

The mean (₹41,045) is almost twice the median (₹22,747), and 77 enquiries out of 7,711 hold a sixth of the value. That's the shape of data where a mean comparison can swing on a single row. Section 30.10 returns to it.

30.4 Effect size: how big, not just whether

A p-value answers "could this be nothing?". It says nothing about *how much*, and with a large enough sample almost any difference gets a small p-value. Watch that happen:

```
rng = np.random.default_rng(30)
group_a = rng.normal(100.0, 15, 200_000)
group_b = rng.normal(100.3, 15, 200_000)    # a difference of 0.3 in 15: trivial

t = stats.ttest_ind(group_b, group_a, equal_var=False)
d = (group_b.mean() - group_a.mean()) / np.sqrt((group_a.var(ddof=1) + group_b.var(ddof=1)) / 2)
print(f"difference = {group_b.mean() - group_a.mean():.3f}")
print(f"p-value     = {t.pvalue:.1e}")
print(f"Cohen's d    = {d:.3f}")
```

```
difference = 0.298
p-value     = 3.1e-10
Cohen's d    = 0.020
```

- `rng.normal(100.0, 15, 200_000)` draws 200,000 values from a normal distribution with mean 100 and standard deviation 15 (section 22.7 used the same call). Group B's mean is 100.3.
- The t-test is section 30.3's. `:.1e` prints the p-value in scientific notation (section 22.2): `3.1e-10` is 3.1×10^{-10} , nine zeros after the decimal point and then 31. A p-value is never exactly zero; printed with six decimals, this one would show as `0.000000`, which hides it.
- `d` is **Cohen's d**: the difference in means divided by the **pooled standard deviation**, the square root of the average of the two variances. It measures the gap in standard deviations, so it doesn't grow with the sample.

A difference of three tenths of a point, on a scale where people vary by 15, is not worth a meeting. The p-value calls it significant because the sample is enormous. **Significant means "measurable", not "important"**.

Effect size measures importance on a scale that doesn't grow with the sample:

Measure	For	Formula	Rough reading
Absolute difference	anything	$\text{mean}_2 - \text{mean}_1$, or $p_2 - p_1$	in the units the business uses
Relative lift	rates	$(p_2 - p_1) / p_1$	"14% more enquiries"
Cohen's d	two means	difference ÷ pooled standard deviation	0.2 small, 0.5 medium, 0.8 large
Odds ratio	proportions in regression	taught in section 30.11	1.0 is no effect
Cramér's V	chi-square tables	taught in section 30.5	0 to 1, higher is stronger

Riverstone's real difference in visit length, between sessions that sent an enquiry and sessions that didn't:

```
enquired = sessions.loc[sessions["enquiry_submitted"] == 1, "duration_seconds"]
browsed = sessions.loc[sessions["enquiry_submitted"] == 0, "duration_seconds"]
d = (enquired.mean() - browsed.mean()) / np.sqrt((enquired.var(ddof=1) + browsed.var(ddof=1)) / 2)

print(f"enquired: n = {len(enquired):,}, mean = {enquired.mean():.1f} s")
print(f"browsed: n = {len(browsed):,}, mean = {browsed.mean():.1f} s")
print(f"Cohen's d = {d:.2f}")
```

```
enquired: n = 7,711, mean = 173.9 s
browsed: n = 206,817, mean = 93.3 s
Cohen's d = 0.51
```

- The two `.loc` lines split the sessions' `duration_seconds` by `enquiry_submitted`: 1 for sessions that enquired, 0 for the rest.
- `d` is the Cohen's `d` formula from the first cell, applied to the two groups.

Sessions that end in an enquiry last nearly twice as long, and a `d` of 0.51 is a medium effect by the table's rough reading: a real difference, and one you could see on a chart.

Always report three numbers together: the effect (with its unit), the confidence interval, and the sample size. Section 30.9's result, as a preview: *"Enquiries rose 0.55 percentage points (95% CI 0.19 to 0.91), from 3.90% to 4.45%, over 47,286 visitors"* is a complete sentence. *" $p < 0.01$ "* is not.

Watch out: relative lift on a small base is a rhetorical device. A rise from 0.1% to 0.2% is "a 100% improvement" and also one extra enquiry per thousand visitors. Give both forms every time: the absolute change tells people what will land in their inbox, the relative one tells them how much the thing they changed mattered.

30.5 Categorical outcomes: proportions and chi-square

Most experiment metrics are proportions: converted or not, clicked or not, churned or not. Two tools cover them.

Two proportions

The difference between two rates, with its standard error, is section 30.2's arithmetic applied twice, and section 22.2's z-test. Which channels bring visitors who enquire? First give each visitor a channel. A visitor can come back through a different channel, so use one rule and say it: **a visitor's channel is the channel of their first visit.**

```
first_visit = sessions.sort_values("started_at").drop_duplicates("visitor_id")
by_visitor = visitors.merge(first_visit[["visitor_id", "channel", "device", "region"]],
    on="visitor_id")
counts = by_visitor.groupby("channel")["enquired"].agg(["sum", "count"])
print(counts)
```

	sum	count
channel		
direct	2236	42037
email	769	12485
organic	2685	66885
paid	1131	35179
referral	822	19414

- `sort_values("started_at")` puts every session in time order; `drop_duplicates("visitor_id")` then keeps the first row it meets for each visitor, which is their first visit (section 18.10).
- `visitors.merge(..., on="visitor_id")` adds that visit's channel, device, and region to each visitor's enquired flag (section 18.7). Device and region are for later sections. In this generated data every visitor keeps the same channel and device on every visit; real visitors don't, which is why the rule matters.
- `.agg(["sum", "count"])` gives two columns per channel: the visitors who enquired and all visitors.

Now compare email with paid search:

```
k_email, n_email = counts.loc["email", "sum"], counts.loc["email", "count"]
k_paid, n_paid = counts.loc["paid", "sum"], counts.loc["paid", "count"]
stat, pvalue = proportions_ztest([k_email, k_paid], [n_email, n_paid])
low, high = confint_proportions_2indep(k_email, n_email, k_paid, n_paid, method="wald")

print(f"email {k_email / n_email:.4f} vs paid {k_paid / n_paid:.4f}")
print(f"z = {stat:.2f}, p = {pvalue:.2e}")
print(f"95% CI for the difference: {low:.4f} to {high:.4f}")
```

```
email 0.0616 vs paid 0.0321
z = 14.45, p = 2.60e-47
95% CI for the difference: 0.0248 to 0.0340
```

How it works:

- `counts.loc["email", "sum"]` picks one cell: the row `email`, the column `sum`. Each line unpacks two cells into two names.
- `proportions_ztest([k_email, k_paid], [n_email, n_paid])` takes a list of successes, then a list of totals, as in section 22.2. The order sets the sign: email comes first, so a positive z means email's rate is higher. It returns z (`stat`) and the p -value.
- `confint_proportions_2indep(k_email, n_email, k_paid, n_paid, method="wald")` gives the 95% interval for the first rate minus the second. `method="wald"` is the textbook formula from section 22.1: the difference $\pm 1.96 \times \sqrt{(p_1(1 - p_1)/n_1 + p_2(1 - p_2)/n_2)}$.
- `:.2e` prints the p -value in scientific notation: 2.60e-47 is a decimal point, 46 zeros, then 26.

Check the interval by hand: $0.0616 - 0.0321 = 0.0294$; the standard error is $\sqrt{(0.0616 \times 0.9384 \div 12,485 + 0.0321 \times 0.9679 \div 35,179)} = 0.00235$; and $0.0294 \pm 1.96 \times 0.00235$ runs from **0.0248 to 0.0340** ✓.

Why two functions rather than one? The test asks "*what if there were no difference?*", so it computes its standard error as if the two groups shared one rate, the **pooled standard error**: here the pooled rate is $1,900 \div 47,664 = 0.0399$, and the pooled SE is 0.00204, which gives $z = 0.0294 \div 0.00204 = 14.45$.

The interval describes the difference you actually have, so it uses each group's own rate. The two standard errors differ a little, so compute each with its own function, and don't be surprised if a p-value just under 0.05 comes with an interval that just touches zero.

Chi-square: several categories at once

"Does enquiry rate depend on the channel at all?" A **chi-square test of independence** compares the counts you observed with the counts you'd expect if the two columns were unrelated. Section 22.2 worked one through by hand on a 2×2 table; here the table is 5×2:

```
table = pd.crosstab(by_visitor["channel"], by_visitor["enquired"])
chi2, p, dof, expected = stats.chi2_contingency(table)
cramers_v = np.sqrt(chi2 / (table.to_numpy().sum() * (min(table.shape) - 1)))

print(table)
print(f"chi-square = {chi2:.1f}, dof = {dof}, p = {p:.3e}")
print(f"Cramér's V = {cramers_v:.3f}")
print("expected counts if channel made no difference:")
print(pd.DataFrame(expected, index=table.index, columns=table.columns).round(0))
```

```
enquired    0    1
channel
direct    39801  2236
email     11716   769
organic    64200  2685
paid       34048  1131
referral   18592   822
chi-square = 321.3, dof = 4, p = 2.772e-68
Cramér's V = 0.043
expected counts if channel made no difference:
enquired    0    1
channel
direct    40211.0  1826.0
email     11943.0   542.0
organic    63980.0  2905.0
paid       33651.0  1528.0
referral   18571.0   843.0
```

How it works:

- `pd.crosstab` counts visitors for every (channel, enquired) pair (Chapter 21). `stats.chi2_contingency(table)` returns four things, unpacked into four names: the statistic, its p-value, the degrees of freedom, and the table of expected counts.
- The test adds up, over every cell, how far the observed count is from the expected one, relative to the expected one. Large total, small p-value.
- **Degrees of freedom** is $(\text{rows} - 1) \times (\text{columns} - 1)$.
- A significant chi-square says "these columns are related", not which cell drives it. Compare observed against expected to see that, then test the specific pair you care about, as above.
- **Cramér's V** is the effect size: $\sqrt{\chi^2 \div (n \times (\text{the smaller of rows and columns} - 1))}$, which runs from 0 (no relationship) to 1. In the code, `table.to_numpy().sum()` is n , all 176,000 visitors, and `min(table.shape) - 1` is $2 - 1 = 1$. The relationship here is real and small, which is typical of channel data.

- The test needs reasonable counts in each cell (a common rule: every expected count at least 5). For rare outcomes in small samples, use Fisher's exact test instead (`stats.fisher_exact`).

Watch out: significant does not mean causal. Email visitors enquire more often than paid ones, but people on the mailing list already know Riverstone. The channel didn't cause the enquiry; the relationship with the company did. That's section 22.5's correlation-and-causation point, and it's exactly what an experiment fixes, by deciding the groups with a coin instead of letting visitors sort themselves.

30.6 More than two groups: ANOVA

Comparing four regions with t-tests means six comparisons, and each one carries its own 5% chance of a false alarm (section 22.4's family-wise error rate). **Analysis of variance (ANOVA)** asks one question instead: "is there any difference among these groups?"

The idea, by hand

ANOVA compares two kinds of variation: how far the group means are from each other (**between** groups), and how far the values are from their own group's mean (**within** groups). If the groups really were alike, the gaps between their means would be about the size that the scatter within groups produces anyway. Take three channels and three visitors each, with the pages they viewed:

Channel	Pages viewed	Group mean
A	2, 3, 4	3
B	3, 4, 5	4
C	5, 6, 7	6

1. The grand mean of all nine values is $39 \div 9 = 4.33$.
2. **Between groups:** each group mean's distance from the grand mean, squared, times the 3 values in the group: $3 \times ((3 - 4.33)^2 + (4 - 4.33)^2 + (6 - 4.33)^2) = 3 \times 4.67 = \mathbf{14.0}$. There are 3 groups, so $3 - 1 = \mathbf{2}$ degrees of freedom, and $14.0 \div 2 = \mathbf{7.0}$.
3. **Within groups:** each value's distance from its own group mean, squared: $(1 + 0 + 1)$ in each group, so $2 + 2 + 2 = \mathbf{6.0}$. There are 9 values in 3 groups, so $9 - 3 = \mathbf{6}$ degrees of freedom, and $6.0 \div 6 = \mathbf{1.0}$.
4. **F** = $7.0 \div 1.0 = \mathbf{7.0}$. The gaps between the means are seven times what the scatter within groups would explain.

The formula: the F statistic. $F = (\text{between-group variation} \div (k - 1)) \div (\text{within-group variation} \div (N - k))$

- **k** is the number of groups and **N** the number of values. Each variation is a sum of squared distances, as in steps 2 and 3.
- When the groups are alike, F is close to 1. The bigger F is, the harder it is to explain the gaps between groups as noise.

scipy does the same arithmetic, and adds the p-value:

```
f_toy, p_toy = stats.f_oneway([2, 3, 4], [3, 4, 5], [5, 6, 7])
print(f"F = {f_toy:.2f}, p = {p_toy:.4f}")
```

```
F = 7.00, p = 0.0270
```

`stats.f_oneway` takes each group as a separate argument, here three lists, and returns F and its p-value. F = 7.00 matches the hand answer.

On Riverstone's visitors

The unit of analysis comes first (section 30.2): one row per visitor, not per session, or a visitor who came three times counts three times. Average each visitor's sessions, and add the channel and region from `by_visitor`:

```
averages = sessions.groupby("visitor_id")[["duration_seconds",
    "pages_viewed"]].mean().reset_index()
visitor_table = by_visitor.merge(averages, on="visitor_id")
print(visitor_table.shape)
```

```
(176000, 7)
```

`groupby("visitor_id")[...].mean()` gives each visitor's average visit length and average pages per visit; the merge puts them next to the visitor's channel, device, region, and enquired flag: 176,000 rows, 7 columns.

`stats.f_oneway` needs one list of values per group. A `for` loop over a `groupby` hands you the groups one at a time, as pairs of (name, group):

```
for name, group in visitor_table.groupby("region"):
    print(name, len(group))
```

```
East 21458
North 38637
South 45511
West 70394
```

Each time round the loop, `name` is the region and `group` is a DataFrame of that region's visitors. Now the ANOVA, first for visit length by region, then for pages viewed by channel:

```
by_region = [g["duration_seconds"].to_numpy() for _, g in visitor_table.groupby("region")]
f_stat, p_value = stats.f_oneway(*by_region)
print(visitor_table.groupby("region")["duration_seconds"].agg(["count", "mean"]).round(1))
print(f"F = {f_stat:.3f}, p = {p_value:.4f}")

by_channel = [g["pages_viewed"].to_numpy() for _, g in visitor_table.groupby("channel")]
f_stat, p_value = stats.f_oneway(*by_channel)
print(visitor_table.groupby("channel")["pages_viewed"].agg(["count", "mean"]).round(2))
print(f"F = {f_stat:.2f}, p = {p_value:.3f}")
```

```
      count  mean
region
East    21458  95.4
North   38637  96.5
South   45511  95.4
West    70394  96.8
F = 1.917, p = 0.1245
      count  mean
channel
direct    42037  2.75
email     12485  2.77
organic   66885  2.73
paid      35179  2.73
referral  19414  2.75
F = 2.57, p = 0.036
```

- The list comprehension (Chapter 17) builds one NumPy array per region, with `.to_numpy()` (section 18.1). `_` is Python's name for a value you don't need, here the region's name (section 22.2 used it the same way).
- `*by_region` unpacks the list: `stats.f_oneway(*by_region)` is the same as writing `stats.f_oneway(east, north, south, west)`, each array as a separate argument, without knowing in advance how many groups there are.
- The `groupby(...).agg(["count", "mean"])` lines print each group's size and mean, so you can see what the test is comparing.

Reading it: the four regions behave the same: F is small (1.9) and $p = 0.12$, so there's no evidence of any difference in how long visits last. The five channels do differ in pages viewed, at $p = 0.036$. Run the same test on sessions instead, and the channel p -value drops to 0.013: counting each visit as independent makes the evidence look stronger than it is, which is section 30.2's warning in action.

ANOVA says *something* differs. To find out what, run a **post-hoc test** that corrects for the number of comparisons. **Tukey's test** (Tukey HSD, for "honestly significant difference") compares every pair of groups while holding the family-wise error rate at 5%:

```
sample = visitor_table.sample(20_000, random_state=30)
tukey = pairwise_tukeyhsd(sample["pages_viewed"], sample["channel"], alpha=0.05)
print(tukey.summary())
```

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
direct	email	0.0922	0.4251	-0.0545	0.2388	False
direct	organic	0.028	0.911	-0.0607	0.1167	False
direct	paid	0.0178	0.99	-0.0856	0.1212	False
direct	referral	0.0184	0.9944	-0.106	0.1428	False
email	organic	-0.0642	0.7247	-0.2047	0.0764	False
email	paid	-0.0743	0.6599	-0.2246	0.0759	False
email	referral	-0.0737	0.7419	-0.2391	0.0917	False
organic	paid	-0.0102	0.9984	-0.1047	0.0843	False
organic	referral	-0.0096	0.9995	-0.1267	0.1075	False
paid	referral	0.0006	1.0	-0.128	0.1292	False

- `pairwise_tukeyhsd(values, groups, alpha=0.05)` takes the values, then the group label for each value, then the family-wise error rate to hold. `.summary()` prints the table. It runs on a random 20,000 visitors to keep it quick.
- **FWER=0.05** in the title is the **family-wise error rate** (section 22.4): the chance of at least one false alarm across all ten pairs, held at 5%.
- `meandiff` is group2's mean minus group1's. `p-adj` is each pair's p-value after allowing for ten comparisons, and `lower/upper` is the 95% interval for the difference. `reject = True` would mean "these two differ".

Not one pair is rejected here, and that's the lesson, not a failure: look again at the channel means, which run from 2.73 to 2.77 pages. On 176,000 visitors that gap is measurable ($p = 0.036$); on a 20,000-visitor sample no individual pair stands out; and in either case it is a few hundredths of a page. This is section 30.4 arriving from the other direction: before asking whether a difference is significant, ask whether the difference would change anything.

Why the correction matters, in one simulation: twenty comparisons between groups that are all identical.

```
rng = np.random.default_rng(30)
false_alarms = 0
for _ in range(20):
    a = rng.normal(100, 15, 500)
    b = rng.normal(100, 15, 500) # exactly the same population
    if stats.ttest_ind(a, b).pvalue < 0.05:
        false_alarms += 1
print(f"comparisons: 20, 'significant' results: {false_alarms}")
print(f"chance of at least one false alarm in 20 tests at 5%: {1 - 0.95 ** 20:.1%}")
```

```
comparisons: 20, 'significant' results: 2
chance of at least one false alarm in 20 tests at 5%: 64.2%
```

- Each time round the loop, `a` and `b` are 500 values drawn from the *same* normal distribution (mean 100, standard deviation 15), so any "difference" is chance.
- `stats.ttest_ind(a, b)` without `equal_var=False` is the ordinary t-test (section 22.10); `.pvalue` reads its p-value. `false_alarms += 1` counts the tests that come out below 0.05.

- The last line is section 22.4's arithmetic, $1 - 0.95^{20}$: the chance that at least one of twenty tests raises a false alarm.

Nothing differed, and yet results appeared. Run twenty tests at the 5% level and you'd expect one false alarm; the chance of at least one is 64%. That's the arithmetic behind section 30.10's multiple-testing rule.

30.7 Power and sample size

(A recap of section 22.3, which defined power, Type I and II errors, and the minimum detectable effect, and worked out a sample size by hand.) **Statistical power** is the chance a test detects an effect that's really there. The convention is 80%, and it depends on four things, any three of which fix the fourth:

Quantity	Meaning	Usual choice
α (alpha)	chance of a false positive you'll accept	0.05
Power ($1 - \beta$)	chance of catching a real effect	0.80
Effect size	the smallest difference worth detecting (the minimum detectable effect , MDE)	a business decision
n	sample size per group	what you're solving for

Section 22.3 used the full formula. Most calculators use a shorter version, which gives almost the same answer:

The formula: sample size per group for two proportions (short version). $n = 2 \times$

$$(z_{\alpha/2} + z_{\beta})^2 \times \bar{p}(1 - \bar{p}) \div d^2$$

- $z_{\alpha/2} = 1.960$, the multiplier for a two-sided test at 5%: `stats.norm.ppf(0.975)` (Chapter 21, section 21.5).
- $z_{\beta} = 0.842$, the multiplier for 80% power: `stats.norm.ppf(0.80)`.
- $\bar{p}$ is the average of the two rates, the baseline and the baseline plus the MDE.
- d is the MDE in proportion units: 0.005 for half a percentage point. It is not Cohen's d from section 30.4.

By hand, for Riverstone's enquiry rate of 3.9% and an MDE of half a point:

1. The two rates are 0.039 and 0.044, so $\bar{p} = 0.0415$ and $\bar{p}(1 - \bar{p}) = 0.0415 \times 0.9585 = 0.03978$.
2. $(1.960 + 0.842)^2 = 2.802^2 = 7.851$.
3. $n = 2 \times 7.851 \times 0.03978 \div 0.005^2 = 0.6246 \div 0.000025 = \mathbf{24,984 \text{ visitors per group}}$.

The same in Python, keeping every decimal:

```
baseline = 0.039          # Riverstone's enquiry rate before the test
mde = 0.005              # we care about half a percentage point
z_alpha, z_beta = stats.norm.ppf(1 - 0.05 / 2), stats.norm.ppf(0.80)
p_bar = baseline + mde / 2
n_by_hand = 2 * (z_alpha + z_beta) ** 2 * p_bar * (1 - p_bar) / mde ** 2

print(f"z_alpha = {z_alpha:.3f}, z_beta = {z_beta:.3f}")
print(f"by hand:      {n_by_hand:,.0f} visitors per group")
```

```
z_alpha = 1.960, z_beta = 0.842
by hand:      24,977 visitors per group
```

$p_bar = baseline + mde / 2$ is the average of 0.039 and 0.044, and n_by_hand is the formula, line for line. The hand answer was 24,984 because it rounded the multipliers to three decimals. (Section 22.3's full formula gives 24,976: the two versions agree to within a visitor.)

statsmodels solves the same problem on a slightly different scale:

```
effect = proportion_effectsize(baseline + mde, baseline)
n_statsmodels = NormalIndPower().solve_power(effect_size=effect, alpha=0.05, power=0.80, alternative="two-
sided")

print(f"Cohen's h:    {effect:.4f}")
print(f"statsmodels: {n_statsmodels:,.0f} visitors per group")
print(f"total needed: {2 * n_statsmodels:,.0f} visitors")
```

```
Cohen's h:    0.0251
statsmodels: 24,955 visitors per group
total needed: 49,909 visitors
```

- `proportion_effectsize(0.044, 0.039)` turns the two rates into **Cohen's h**, an effect size for proportions: $h = 2 \times \arcsin \sqrt{p_2} - 2 \times \arcsin \sqrt{p_1} = 0.0251$. It's a rescaled difference that makes a move from 3.9% to 4.4% comparable with a move from 50% to 50.5%, because a half-point move near 0% is harder to detect than one near 50%.
- `NormalIndPower()` is the power calculator for two independent groups compared with a normal (z) test. `.solve_power(...)` works whichever way round you need: **give it every quantity except the one you want, and it returns that one**. Here `nobs1`, the sample size of the first group, is left out, so it solves for it.
- `alternative="two-sided"` matches a test that counts a difference in either direction.

24,955 and 24,977 differ by 22 visitors because h works on the arcsine scale; either rounds to 25,000 per group.

What happens if you change it. Before running the next cell, predict: if the smallest lift worth detecting halves, from 0.5 points to 0.25, what happens to the sample size?

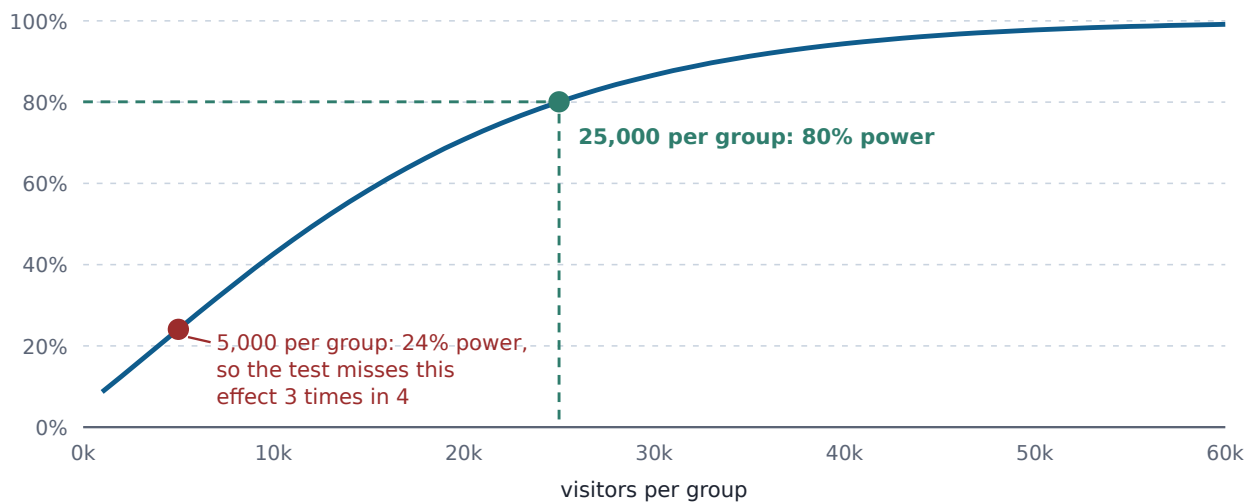
```
for small_lift in (0.0025, 0.005, 0.01):
    h = proportion_effectsize(baseline + small_lift, baseline)
    n_needed = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.80)
    print(f"MDE {100 * small_lift:.2f} points: {n_needed:>9,.0f} visitors per group")
```

MDE 0.25 points:	97,002 visitors per group
MDE 0.50 points:	24,955 visitors per group
MDE 1.00 points:	6,582 visitors per group

Halving the MDE roughly quadruples the sample, because the effect is squared in the formula's denominator; doubling it cuts the sample to about a quarter. That's why the MDE is the most expensive line in any test plan, and why it has to come from the business.

At about 3,400 new visitors a day, half in each group, reaching 25,000 per group takes a fortnight. Which is exactly how the fortnight in section 30.1 was chosen: **the duration came from the arithmetic, not from the calendar.**

Power: the chance of catching an effect that is really there



+0.5 percentage points on a 3.9% baseline, at the 5% level.

Figure 30.2 – Power against sample size. Riverstone's fortnight was chosen to land on the 80% mark.

The curve is one call to the same calculator, asked for power instead of sample size, at sixty sample sizes at once:

```
ns = np.arange(1_000, 60_001, 1_000)
power = NormalIndPower().power(effect_size=effect, nobs1=ns, alpha=0.05)
print(f"power at 5,000 per group: {power[ns == 5_000][0]:.2f}")
print(f"power at 25,000 per group: {power[ns == 25_000][0]:.2f}")

plt.plot(ns, power)
plt.axhline(0.8, linestyle="--")
plt.xlabel("visitors per group")
plt.ylabel("power")
plt.show()
```

```
power at 5,000 per group: 0.24
power at 25,000 per group: 0.80
```

- `np.arange(1_000, 60_001, 1_000)` makes the sample sizes 1,000, 2,000, ... 60,000. The stop value, 60,001, is left out, as with `range`, so 60,000 is the last.

- `.power(effect_size=..., nobs1=ns, alpha=0.05)` is the other direction: given the effect and a sample size, what's the power? Because `ns` is an array, it returns sixty powers, one per sample size.
- `power[ns == 5_000]` picks the power where the sample size is 5,000 (a Boolean mask, section 18.1), and `[0]` takes the single value out of the array.
- `plt.plot` draws the curve, and `plt.axhline(0.8, linestyle="--")` a dashed line at 80%, as in section 22.10. Figure 30.2 is the same chart, labeled.

Two more things the same machinery answers.

What effect could this test have detected? Run it backwards from the sample you actually have. This time leave out `effect_size` and give `nobs1`:

```
detectable = NormalIndPower().solve_power(nobs1=24_000, alpha=0.05, power=0.80, alternative="two-
sided")
print(f"effect size (Cohen's h) detectable with 24,000 per group: {detectable:.4f}")
print(f"that is roughly {detectable * np.sqrt(baseline * (1 - baseline)) * 100:.2f} percentage
points")
```

```
effect size (Cohen's h) detectable with 24,000 per group: 0.0256
that is roughly 0.50 percentage points
```

The second line turns `h` back into percentage points, roughly: near a rate p , one unit of h is about $\sqrt{p(1-p)}$ in proportion units. With 24,000 visitors per group, the smallest lift the test could reliably catch was half a point, which is what the plan asked for.

How many orders would settle section 30.3's question? Retail and Wholesale differed by ₹5,724 with a pooled standard deviation of about ₹18,500:

```
d = (wholesale.mean() - retail.mean()) / np.sqrt((retail.var(ddof=1) + wholesale.var(ddof=1)) / 2)
needed = TTestIndPower().solve_power(effect_size=d, alpha=0.05, power=0.80, alternative="two-
sided")
print(f"observed Cohen's d = {d:.3f}")
print(f"orders needed per segment for 80% power: {needed:,.0f}")
print(f"orders available: Retail {len(retail)}, Wholesale {len(wholesale)}")
```

```
observed Cohen's d = 0.308
orders needed per segment for 80% power: 167
orders available: Retail 59, Wholesale 55
```

`TTestIndPower()` is the calculator for comparing two means with a t-test, and it takes Cohen's d (section 30.4) as its effect size. Here we use the observed gap only as a starting guess for a *future* study. A better answer asks the sales head what gap in average order value would change a decision (say ₹5,000) and plans for that, as the Watch out below says.

So the answer to "is the difference real?" is "ask again when we have three years of orders", and that's a useful thing to be able to say with a number attached.

Watch out: power calculations done afterwards are not evidence. Computing "the power we had to detect the effect we observed" (sometimes called post-hoc power) tells you nothing new: it's just the p-value in different clothes. Power belongs *before* the test, with an effect size chosen from the business, not from the data.

Checkpoint: the end of the second sitting

Before section 30.8, check that sample sizes are yours. Email visitors enquire at about 6%. The marketing team wants to test a new email landing page and cares about a lift of 1 percentage point.

1. By hand, with the short formula: how many visitors per group, at 5% significance and 80% power?
2. Check it with `proportion_effectsize` and `NormalIndPower().solve_power`.
3. Email brings about 7% of Riverstone's 3,400 daily visitors. How many weeks would the test take? (Answers at the end of the chapter.)

30.8 Designing an experiment

Everything above is arithmetic. The design is where tests are won or lost, and it belongs in a written plan agreed before a single visitor is randomized. Riverstone's plan for the enquiry-form test fits on one page:

Section	Riverstone's answer
Question	Does a shorter enquiry form bring more enquiries?
Change	variant_b: five fields instead of nine, phone number optional
Primary metric	Enquiry rate per visitor (enquiries ÷ visitors)
Guardrail metrics	Pages viewed per visitor; sessions per visitor; median enquiry value; enquiry-to-order rate in the CRM
Randomization unit	Visitor (not session, not page view)
Split	50/50, assigned on first session during the window
Minimum detectable effect	0.5 percentage points on a 3.9% baseline
Sample size	24,955 visitors per group, rounded to 25,000 (section 30.7)
Duration	2–15 February 2026: two full weeks, whole weeks only
Stopping rule	Run to the planned end. No decisions from interim looks
Analysis plan	Two-proportion test, 95% CI, effect size; sample-ratio check first; segments reported as exploratory only
Decision rule	Ship if the interval's lower bound is above 0 and no guardrail worsens; otherwise keep the current form

An experiment, in the order the decisions are made

Before	Question and change	the shorter enquiry form
Before	Primary metric, guardrails	enquiry rate per visitor; pages, sessions, value
Before	MDE and sample size	0.5 pp → 25,000 per group
Before	Duration, stopping rule	two whole weeks, no interim decisions
▲ above the line: written down before any visitor is randomized		
During	Randomize by visitor	50/50, and watch the guardrails
After	Sample-ratio check	chi-square on group sizes: failed here
After	Primary metric with CI	+0.55 pp (0.19 to 0.91), p = 0.0026
After	Effect size, decision	+14% relative; ship, fix the Safari tag

Figure 30.3 – Everything above the line is decided before a visitor is randomized.

Why each choice matters:

- **One primary metric.** Choosing the winner afterwards from five metrics is multiple testing with extra steps. Everything else is a guardrail: watched to catch damage, not used to declare victory.
- **The randomization unit must match the analysis unit.** Randomize visitors, count visitors.
- **Whole weeks.** Weekday and weekend behavior differ (the data has a real weekend effect); a test that ends on a Wednesday weights midweek twice.
- **A stopping rule written in advance** is what makes the p-value mean what it says. Section 30.10 shows the damage when there isn't one.
- **Guardrails are as important as the metric.** A form that gets more enquiries by asking for less information may bring worse leads. That's why *enquiry-to-order rate* is on the list, even though it can't be measured for weeks.
- **A decision rule** turns a result into an action before anyone has a stake in the answer.

Try it. Write this table for a change you'd like to test where you work: a subject line, a page layout, a follow-up call within an hour. Most people find the *metric* and *decision rule* rows the hard ones, which is the point: they're where the disagreement lives, and it's cheaper to have it now.

30.9 Riverstone's website test, end to end

The plan is written, the fortnight has passed. Analysis follows the plan, in order, starting with the checks that can invalidate everything else.

Step 1: assemble the data at the right grain

First, the sessions that belong to the test: visitors who were assigned a version, visiting during the fortnight.

```
test_sessions = sessions.merge(assignments, on="visitor_id")
window = test_sessions["started_at"].between("2026-02-02", "2026-02-15 23:59:59")
test_sessions = test_sessions.loc[window]
first = test_sessions.sort_values("started_at").drop_duplicates("visitor_id")
print(len(test_sessions), len(first))
```

```
53692 47286
```

- `sessions.merge(assignments, on="visitor_id")` keeps only the sessions of visitors in the test, and adds each one's variant. A merge keeps only matching rows unless you ask otherwise (section 18.7).
- `.between("2026-02-02", "2026-02-15 23:59:59")` is True for sessions that started in the window, both ends included (section 18.4). pandas compares the text with the dates for you; the end is written as the last second of 15 February, so that the whole last day counts.
- `first` holds each visitor's first session in the test, the same `sort_values` and `drop_duplicates` pair as in section 30.5. Step 5, section 30.10, and section 30.11 use it for each visitor's device, browser, and channel.

53,692 sessions from 47,286 visitors. Now one row per visitor, and one summary row per version:

```
per_visitor = (test_sessions.groupby(["variant", "visitor_id"])
               .agg(enquired=("enquiry_submitted", "max"),
                    sessions=("session_id", "count"),
                    pages=("pages_viewed", "sum"),
                    value=("enquiry_value", "max"))
               .reset_index())
summary = per_visitor.groupby("variant").agg(visitors=("visitor_id", "count"),
                                             enquires=("enquired", "sum"))
summary["rate"] = summary["enquires"] / summary["visitors"]
print(summary)
```

	visitors	enquiries	rate
variant			
control	24036	937	0.038983
variant_b	23250	1035	0.044516

- `groupby(["variant", "visitor_id"])` makes one group per visitor, keeping their version. `.agg(...)` is Chapter 18's named aggregation: `enquired` is the largest `enquiry_submitted` (1 if any session enquired, as in section 30.2), `sessions` counts the visitor's sessions, `pages` adds up their pages, and `value` is the largest enquiry value (empty, NaN, for visitors who didn't enquire).
- `summary` counts the visitors and adds up the enquirers for each version, and `rate` divides one by the other.

Step 2: the sample-ratio check, before anything else

A 50/50 split should produce group sizes that differ only by chance. If they don't, something is wrong with the assignment or the logging, and the outcome numbers can't be trusted either. Test it with a different chi-square test from section 30.5's: a **goodness-of-fit test**, which compares observed counts with the counts a rule says to expect.

```
observed = summary["visitors"].to_numpy()
chi2, p_srm = stats.chisquare(observed)
print(f"visitors: control {observed[0]:,}, variant_b {observed[1]:,}")
print(f"split: {observed[0] / observed.sum():.3%} / {observed[1] / observed.sum():.3%}")
print(f"sample-ratio mismatch test: chi-square = {chi2:.1f}, p = {p_srm:.5f}")
```

```
visitors: control 24,036, variant_b 23,250
split: 50.831% / 49.169%
sample-ratio mismatch test: chi-square = 13.1, p = 0.00030
```

- `.to_numpy()` turns the `visitors` column into a plain array of two counts, `observed[0]` for control and `observed[1]` for the variant.
- `stats.chisquare(observed)` compares the observed counts with expected ones. With no `f_exp=` argument, it expects equal counts: $47,286 \div 2 = 23,643$ each. That default is exactly right for a 50/50 design. For a 90/10 design you would pass `f_exp=[0.9 * total, 0.1 * total]`, where `total` is the number of visitors in the test.
- `:.3%` prints a proportion as a percentage with three decimals.

By hand, as in section 22.2: $(24,036 - 23,643)^2 \div 23,643 + (23,250 - 23,643)^2 \div 23,643 = 6.53 + 6.53 = \mathbf{13.1}$ ✓.

This test fails. A 50.8/49.2 split of 47,286 visitors sounds harmless, but chance alone would produce a gap this large about once in 3,300 tests ($p = 0.0003$). That's a **sample-ratio mismatch (SRM)**, and the rule is firm: investigate before reading the result. Section 30.10 finds the cause.

Step 3: the primary metric, with an interval

```
k = summary["enquiries"].to_numpy()
n = summary["visitors"].to_numpy()
p_control, p_variant = k / n
diff = p_variant - p_control
se = np.sqrt(p_control * (1 - p_control) / n[0] + p_variant * (1 - p_variant) / n[1])
z, p_value = proportions_ztest([k[1], k[0]], [n[1], n[0]])

print(f"control    {p_control:.4f}  ({k[0]:,} of {n[0]:,})")
print(f"variant_b {p_variant:.4f}  ({k[1]:,} of {n[1]:,})")
print(f"absolute difference: {100 * diff:+.2f} percentage points")
print(f"relative lift:      {100 * diff / p_control:+.1f}%")
print(f"z = {z:.3f}, p = {p_value:.4f}")
print(f"95% CI (absolute):   {100 * (diff - 1.96 * se):+.2f} to {100 * (diff + 1.96 * se):+.2f} pp")
low_rel, high_rel = (diff - 1.96 * se) / p_control, (diff + 1.96 * se) / p_control
print(f"95% CI (relative):  {100 * low_rel:+.1f}% to {100 * high_rel:+.1f}%")
```

```
control    0.0390  (937 of 24,036)
variant_b  0.0445  (1,035 of 23,250)
absolute difference: +0.55 percentage points
relative lift:      +14.2%
z = 3.009, p = 0.0026
95% CI (absolute):  +0.19 to +0.91 pp
95% CI (relative):  +4.9% to +23.4%
```

- `k` and `n` are arrays of two numbers each, control first (the order of `summary`'s rows). `k / n` divides them element by element, and `p_control`, `p_variant = k / n` unpacks the two rates into two names (Chapter 17).
- `se` is the standard error of the difference, from section 22.1's formula, with each group's own rate.
- `proportions_ztest([k[1], k[0]], [n[1], n[0]])` lists the variant first, so that a positive `z` means the variant is higher. As in section 30.5, the test uses the pooled standard error and the interval uses the unpooled one; here they are 0.001839 and 0.001841, the same for any practical purpose.
- The relative interval divides both ends of the absolute interval by the control rate. That treats the control rate as known exactly, which it isn't quite. A more careful interval for a ratio, calculated on the log scale, gives +4.7% to +24.5%: the same story.

The agency's "up 14%" is the middle of a range that runs from +4.9% to +23.4%. At Riverstone's traffic of about 3,400 new visitors a day, 0.55 percentage points is roughly 560 extra enquiries a month, and the interval's ends are about 200 and 930. Both ends are good news; the gap between them is the difference between hiring another person to handle enquiries and not. The range belongs in the report.

Making it repeatable: a tested function

Section 29.7 tested the monthly report against numbers everyone had agreed. The same habit works for statistics. Save the arithmetic of Step 3 as a function in a file, `ab_summary.py`, in the chapter folder:

```
"""Chapter 30: the two-proportion summary used in section 30.9, as a function that can be tested."""
import numpy as np
from statsmodels.stats.proportion import proportions_ztest

def two_proportion_summary(k_a: int, n_a: int, k_b: int, n_b: int) -> dict:
    """Compare group B's rate with group A's: the difference, its 95% interval, and the p-value."""
    p_a, p_b = k_a / n_a, k_b / n_b
    diff = p_b - p_a
    se = np.sqrt(p_a * (1 - p_a) / n_a + p_b * (1 - p_b) / n_b)
    z, p_value = proportions_ztest([k_b, k_a], [n_b, n_a])
    return {"diff": diff, "ci_low": diff - 1.96 * se, "ci_high": diff + 1.96 * se, "p_value":
            p_value}
```

It's Step 3's lines, with the type hints of section 29.6: four whole numbers in (`k_a` and `n_a`, group A's enquirers and visitors, then the same for group B), a dictionary out, with the difference, the two ends of its interval, and the p-value. The notebook can now import it and use it for any test:

```
from ab_summary import two_proportion_summary

result = two_proportion_summary(k[0], n[0], k[1], n[1])
print({name: round(float(value), 4) for name, value in result.items()})
```

```
{'diff': 0.0055, 'ci_low': 0.0019, 'ci_high': 0.0091, 'p_value': 0.0026}
```

The dictionary comprehension (Chapter 17) rounds each value to four decimals for printing; `float(...)` turns NumPy's number types into plain Python ones, so they print as numbers. Then pin the book's numbers in a test, `test_ab_summary.py`, in the same folder:

```
import pytest
from ab_summary import two_proportion_summary

def test_enquiry_form_matches_the_book():
    result = two_proportion_summary(937, 24_036, 1_035, 23_250)
    assert result["diff"] == pytest.approx(0.0055, abs=0.0001)
    assert result["ci_low"] == pytest.approx(0.0019, abs=0.0001)
    assert result["ci_high"] == pytest.approx(0.0091, abs=0.0001)
    assert result["p_value"] == pytest.approx(0.0026, abs=0.0001)
```

As in section 29.7, pytest runs every function whose name starts with `test_`, and each `assert` line must be true for the test to pass. `pytest.approx(0.0055, abs=0.0001)` means "equal to 0.0055, give or take 0.0001" (section 29.7 used `approx` without `abs`, which allows a tiny relative difference instead). Run it from the terminal:

```
# terminal, in companion/ch30
$ uv run pytest -q
. [100%]
1 passed in 2.80s
```

`uv run pytest` runs pytest inside the project's environment, and `-q` (quiet) prints one character per test: the single dot is the one test, passing. If someone later "tidies" the function and breaks it, this test fails before the wrong number reaches a report. It's Chapter 29's reconciliation test, applied to statistics.

Step 4: guardrails

The plan named three guardrails that can be measured now: pages viewed per visitor, sessions per visitor, and the size of the enquiries.

```
guard = per_visitor.groupby("variant").agg(pages=("pages", "mean"),
                                           sessions=("sessions", "mean"),
                                           median_value=("value", "median"))

print(guard.round(3))

enq = per_visitor.dropna(subset=["value"])
u = stats.mannwhitneyu(enq.loc[enq["variant"] == "variant_b", "value"],
                      enq.loc[enq["variant"] == "control", "value"])
print(f"enquiry value, variant_b vs control: Mann-Whitney p = {u.pvalue:.3f}")
```

```
      pages  sessions  median_value
variant
control    3.092      1.135      23066.35
variant_b  3.130      1.136      22764.70
enquiry value, variant_b vs control: Mann-Whitney p = 0.952
```


- `guard` averages each visitor's pages and sessions within each version, and `median_value` takes the median enquiry value. The median skips the empty values of visitors who didn't enquire.
- `dropna(subset=["value"])` keeps only the visitors with an enquiry value (section 18.10). Enquiry values are heavily skewed (section 30.3), so they are compared with the Mann-Whitney test from section 22.2, which uses ranks, not the values themselves.

Pages viewed and sessions per visitor are unchanged, and the enquiry values are indistinguishable, so the extra enquiries show no sign of being worse. The guardrail that matters most, how many of these enquiries become orders, can't be answered for weeks, and the write-up should say so rather than implying the question is closed.

Step 5: segments, clearly labeled as exploratory

```
seg = (per_visitor.merge(first[["visitor_id", "device"]], on="visitor_id")
      .groupby(["device", "variant"])
      .agg(visitors=("visitor_id", "count"), rate=("enquired", "mean"))
      .reset_index())
print(seg.pivot(index="device", columns="variant", values=["visitors", "rate"]).round(4))
```

	visitors		rate	
variant	control	variant_b	control	variant_b
device				
desktop	10933.0	10852.0	0.0466	0.0519
mobile	11659.0	11031.0	0.0329	0.0378
tablet	1444.0	1367.0	0.0298	0.0402

- The merge adds each visitor's device, from their first session in the test (`first`, Step 1). The `groupby` and `agg` then count visitors and average `enquired`, the enquiry rate, for every (device, variant) pair.
- `.pivot(index="device", columns="variant", values=["visitors", "rate"])` spreads the long table out: devices down the side, and for each of the two values, one column per version. It's `pivot_table` (section 18.8) without the aggregation, for a table that already has one row per cell. The counts print as `10933.0` because the pivot stores both values in columns of decimals.

Every segment moves in the same direction, which is mildly reassuring. What you must not do is pick the biggest one and report "a 35% lift on tablet" (0.0298 to 0.0402), on only 2,811 tablet visitors, whose interval is far wider than the overall one: three devices means three more chances of a false positive, and the test was never powered for a single device. Segments are for generating the *next* hypothesis.

30.10 Four ways to fool yourself, demonstrated

Peeking

Section 22.4 showed what peeking does to an email test. Here is the same trap in the shape of Riverstone's website test. Checking daily and stopping when p first drops below 0.05 sounds prudent. Simulate it on two groups that are identical by construction. First, the settings:

```
rng = np.random.default_rng(30)
rate = 0.039
trials = 200
print(f"{trials} simulated tests, both groups at {rate:.1%}, one look a day for 14 days")
```

```
200 simulated tests, both groups at 3.9%, one look a day for 14 days
```

`rate` is Riverstone's enquiry rate before the test, and `trials` is how many fortnights to simulate. Then the fortnights themselves:

```
peeks_that_won = 0
for _ in range(trials):
    a = rng.random(14_000) < rate          # two identical groups
    b = rng.random(14_000) < rate
    for day in range(1, 15):               # look once a day for two weeks
        upto = day * 1000
        _, p = proportions_ztest([a[:upto].sum(), b[:upto].sum()], [upto, upto])
        if p < 0.05:
            peeks_that_won += 1
            break

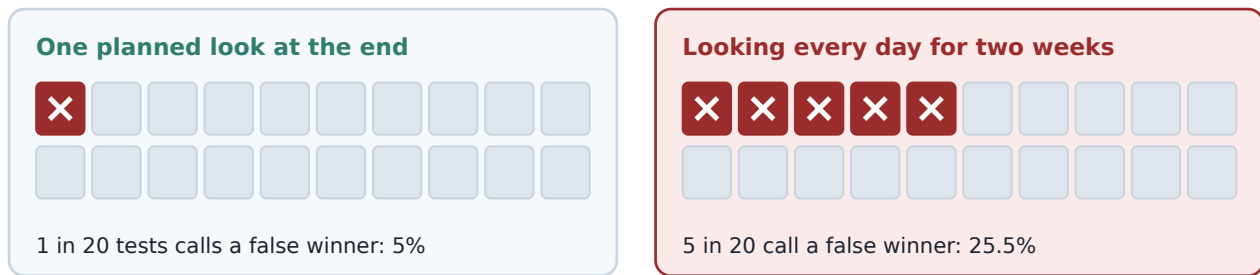
share = peeks_that_won / trials
print(f"{trials} tests where nothing was different")
print(f"'significant' at some point while peeking: {peeks_that_won} ({share:.1%})")
print("expected at a single planned look: 5%")
```

```
200 tests where nothing was different
'significant' at some point while peeking: 51 (25.5%)
expected at a single planned look: 5%
```

- `rng.random(14_000) < rate` makes 14,000 visitors, each True (enquired) with probability 0.039, as in section 22.4. `a` and `b` come from the same rate, so there is nothing to find.
- The inner loop plays the fortnight: on day 1 the dashboard shows the first 1,000 visitors per group, on day 2 the first 2,000, and so on to 14,000. `_, p = ...` keeps only the p -value.
- `break` stops that test at its first "win", as a person would.

Chapter 22, section 22.4, found 14.3% with five looks and 23.9% with twenty. Here, fourteen daily looks at a 3.9% rate give 25.5%. The settings differ, and 200 simulated tests carry about ± 3 points of noise of their own, but the story is the same: the more you look, the more "winners" chance hands you. With no real effect at all, peeking daily turns a 5% false-positive rate into something several times larger. Run the test to its planned end, or use a method designed for looking early (sequential testing, or a Bayesian approach) and agreed in advance.

Peeking: what daily checks do to a test where nothing is different



- a false winner: a test of two identical groups that looked significant
- a test that correctly found nothing

Simulated in section 30.10: 200 tests, 14 daily looks each, stopping at the first $p < 0.05$.

Figure 30.4 – Measured by simulation: peeking daily turns a 5% false-positive rate into 25%.

Multiple testing

Twenty metrics, and one or two of them will look good. The fix is to decide the primary metric first, and to correct when several must be tested. Section 22.4 applied Bonferroni and Benjamini-Hochberg by hand to seven p-values; statsmodels does both in one call:

```
rng = np.random.default_rng(31)
pvalues = []
for _ in range(20):
    a = rng.normal(100, 15, 3_000)
    b = rng.normal(100, 15, 3_000)
    pvalues.append(stats.ttest_ind(a, b).pvalue)

raw = sum(p < 0.05 for p in pvalues)
bonferroni = multipletests(pvalues, alpha=0.05, method="bonferroni")[0].sum()
fdr = multipletests(pvalues, alpha=0.05, method="fdr_bh")[0].sum()
print(f"smallest p-value of 20: {min(pvalues):.4f}")
print(f"'significant' uncorrected: {raw}")
print(f"after Bonferroni: {bonferroni}")
print(f"after Benjamini-Hochberg: {fdr}")
```

```
smallest p-value of 20: 0.0403
'significant' uncorrected: 2
after Bonferroni: 0
after Benjamini-Hochberg: 0
```

- The loop runs twenty t-tests on pairs of identical groups, 3,000 values each from `rng.normal(100, 15, 3_000)`, and `.append` keeps each p-value in the list `pvalues`.
- `sum(p < 0.05 for p in pvalues)` counts the p-values below 0.05: each comparison is True or False, and True counts as 1.
- `multipletests(pvalues, alpha=0.05, method="bonferroni")` returns several things; the first, `[0]`, is a True/False array saying which tests are still significant after the correction, and `.sum()` counts them. `method="fdr_bh"` is Benjamini-Hochberg.

Bonferroni divides the threshold by the number of tests: strict, simple, and fine for a handful.

Benjamini-Hochberg controls the share of your discoveries that are false rather than the chance of any error, which suits screening many metrics.

Novelty

Anything new draws attention. Split the test by week, keeping the unit of analysis: each visitor counts once, in the week of their first session in the test.

```
first_week = first.assign(week=np.where(first["started_at"] < "2026-02-09", "week 1", "week 2"))
by_week = per_visitor.merge(first_week[["visitor_id", "week"]], on="visitor_id")
print(by_week.groupby(["week", "variant"]).size().unstack())

weekly = by_week.groupby(["week", "variant"])["enquired"].mean().unstack()
weekly["lift"] = 100 * (weekly["variant_b"] - weekly["control"]) / weekly["control"]
print(weekly.round(4))
```

```
variant  control  variant_b
week
week 1    12766    12337
week 2    11270    10913
variant  control  variant_b    lift
week
week 1     0.0398     0.0486  22.0136
week 2     0.0381     0.0400   4.9564
```

- `np.where(condition, "week 1", "week 2")` labels each visitor's first session (section 18.1): before Monday 9 February is week 1. `.assign(week=...)` adds the labels as a new column, in a copy, so `first` itself is unchanged.
- The merge gives each row of `per_visitor` its week. `.size()` counts visitors per (week, variant), and `.unstack()` moves the variants into columns (section 18.6).
- `weekly` is the enquiry rate for each week and version, and `lift` the relative lift in percent.

Week one's lift is 22%; week two's is 5%. If the test had run for five days, the headline would have been far more exciting and far less true. Two weeks is the minimum for a website change, and a follow-up look a month later is how you find out which part of the effect survived.

Sample-ratio mismatch: finding the cause

Step 2 found an uneven split. Break the assignment down by anything that could affect tagging:

```
for column in ["device", "browser", "channel"]:
    table = pd.crosstab(first[column], first["variant"])
    table["share_control"] = table["control"] / table.sum(axis=1)
    table["srm_p"] = [stats.chisquare(row)[1] for row in table[["control",
        "variant_b"]].to_numpy()]
    print(table.round(4))
    print()
```

variant	control	variant_b	share_control	srp_p
device				
desktop	10933	10852	0.5019	0.5831
mobile	11659	11031	0.5138	0.0000
tablet	1444	1367	0.5137	0.1464

variant	control	variant_b	share_control	srp_p
browser				
Chrome	13598	13604	0.4999	0.9710
Edge	2126	2127	0.4999	0.9878
Firefox	628	634	0.4976	0.8659
Other	548	593	0.4803	0.1828
Safari	7136	6292	0.5314	0.0000

variant	control	variant_b	share_control	srp_p
channel				
direct	5739	5516	0.5099	0.0356
email	1717	1664	0.5078	0.3620
organic	9139	8859	0.5078	0.0369
paid	4856	4721	0.5070	0.1677
referral	2585	2490	0.5094	0.1824

- The loop makes one table per column. `pd.crosstab(first[column], first["variant"])` counts visitors for each value and version; `table.sum(axis=1)` adds across a row, so `share_control` is the control group's share of that row.
- `srp_p` runs Step 2's `stats.chisquare` on each row's two counts. The list comprehension goes through the rows of the two count columns, as a NumPy array, and keeps `[1]`, the p-value.

Two breakdowns flag a problem: mobile, and Safari. Safari visitors are split 53/47 instead of 50/50, while Chrome, Edge and Firefox are clean to within a rounding error. Which is the cause? Cross the two:

```
print(pd.crosstab(first["browser"], first["device"]))
```

device	desktop	mobile	tablet
browser			
Chrome	13630	12417	1155
Edge	4253	0	0
Firefox	1262	0	0
Other	0	1141	0
Safari	2640	9132	1656

Two thirds of Safari visitors are on phones (9,132 of 13,428), and about two in five mobile visitors use Safari (9,132 of 22,690). So Safari is the cause and mobile is a symptom: mobile's split is off because so much of mobile is Safari. That's a bug in the assignment script, not chance, and it's the kind of thing that only ever turns up because someone ran the check. Riverstone's options: fix the tag and rerun, or analyze only the browsers that split correctly and treat Safari separately. What you cannot do is ignore it, because the missing visitors may not be random.

So check whether the result survives without Safari, with the function from section 30.9:

```
with_browser = per_visitor.merge(first[["visitor_id", "browser"]], on="visitor_id")
no_safari = with_browser.loc[with_browser["browser"] != "Safari"]
g = no_safari.groupby("variant")["enquired"].agg(["sum", "count"])
print(g.assign(rate=(g["sum"] / g["count"]).round(4)))

result = two_proportion_summary(g.loc["control", "sum"], g.loc["control", "count"],
                                g.loc["variant_b", "sum"], g.loc["variant_b", "count"])
print(f"difference {100 * result['diff']:+.2f} pp, p = {result['p_value']:.4f}")
print(f"95% CI {100 * result['ci_low']:+.2f} to {100 * result['ci_high']:+.2f} pp")
share = g.loc["control", "count"] / g["count"].sum()
srm_p = stats.chisquare(g["count"].to_numpy())[1]
print(f"split: {share:.2%} control, SRM p = {srm_p:.2f}")
```

```
      sum  count  rate
variant
control   679  16900  0.0402
variant_b  780  16958  0.0460
difference +0.58 pp, p = 0.0084
95% CI +0.15 to +1.01 pp
split: 49.91% control, SRM p = 0.75
```

- `with_browser` adds each visitor's browser; `no_safari` keeps every other browser.
- `g` counts enquirers and visitors per version, and `.assign(rate=...)` adds the rate for printing.
- `two_proportion_summary(...)` is Step 3's arithmetic, now in one line; `share` and `srm_p` repeat Step 2's check on the new split.

Without Safari the split is even and the conclusion holds: same direction, similar size, still significant. That's the sentence the write-up needs, because "we found a bug" and "the result survives it" are two different facts, and the reader deserves both.

Watch out: outliers can decide a revenue test on their own. Riverstone's largest enquiry is worth nearly 200 ordinary ones (section 30.3). A revenue-per-visitor metric can be moved by which group that one enquiry lands in. Standard protections: use a rate rather than a total, cap extreme values at a percentile agreed in advance (**winsorizing**), analyze the median or a log transform, and always report how many rows the decision rests on.

30.11 Regression for inference

Section 22.10 fitted a straight line with one explanatory variable, by hand, in a spreadsheet and with `stats.linregress`. Here the same least-squares method explains an outcome with several variables at once, including categories, and the coefficients and their intervals are the answer. (Chapter 37 later uses regression to *predict*.)

The tool is statsmodels' **formula interface**, `smf`, imported in section 30.1. It takes the model written as text, the way you'd say it: *"duration, explained by pages viewed"*.

One x: the line from section 22.10

A random 40,000 sessions keep the examples quick; the full data tells the same story. Fit visit length on pages viewed, and put scipy's answer beside it:

```
model_data = sessions.sample(40_000, random_state=30)
m1 = smf.ols("duration_seconds ~ pages_viewed", data=model_data).fit()
line = stats.linregress(model_data["pages_viewed"], model_data["duration_seconds"])
print(m1.params)
print(f"linregress: intercept {line.intercept:.4f}, slope {line.slope:.4f}")
```

```
Intercept      91.860804
pages_viewed    1.709879
dtype: float64
linregress: intercept 91.8608, slope 1.7099
```

- `smf.ols(...)` sets up an **ordinary least squares** model, section 22.10's line. The formula `"duration_seconds ~ pages_viewed"` reads *"duration_seconds is modelled by pages_viewed"*: the outcome goes left of the `~`, the explanatory variables to its right, each a column name in `data=model_data`.
- The **intercept** is added automatically, so you don't write it. It appears as `Intercept`.
- `.fit()` does the least-squares arithmetic and returns the fitted model, here `m1`. `m1.params` holds the coefficients: the intercept and the slope. They match `linregress` exactly.

The full table of results has the columns section 22.10 promised:

```
print(m1.summary().tables[1])
print(f"linregress: stderr {line.stderr:.3f}, p-value {line.pvalue:.2g}")
```

```
=====
              coef    std err          t      P>|t|      [0.025      0.975]
-----
Intercept      91.8608      1.037     88.604      0.000      89.829      93.893
pages_viewed    1.7099      0.312      5.487      0.000        1.099        2.321
=====
linregress: stderr 0.312, p-value 4.1e-08
```

`.summary()` builds a long report; `.tables[1]` is its second part, the coefficient table (the first part describes the data and the fit). Reading across the `pages_viewed` row, in section 22.10's names:

- **coef** is the slope, and **std err** its standard error, `stderr`.
- **t** is `coef ÷ std err`, the slope measured in standard errors.
- **P>|t|** is the p-value for "the true slope is 0", `pvalue`. It prints as 0.000 when it is below 0.0005.
- **[0.025 and 0.975]** are the two ends of the 95% interval for the slope: `coef ± t* × std err`, with $n - 2$ degrees of freedom, as in section 22.10.

One category: device

x can also be a category. Section 22.10 put Kolkata in a regression as a 0/1 dummy variable. Device has three values, so it needs two dummies, and statsmodels makes them for you:

```
m2 = smf.ols("duration_seconds ~ C(device)", data=model_data).fit()
print(m2.params.round(2))
print(model_data.groupby("device")["duration_seconds"].mean().round(2))
```

```
Intercept          122.25
C(device)[T.mobile] -50.61
C(device)[T.tablet] -24.43
dtype: float64
device
desktop    122.25
mobile      71.64
tablet      97.82
Name: duration_seconds, dtype: float64
```

- `C(device)` means "treat device as categories". statsmodels makes one 0/1 column per device **except one**, the **reference category**, which is absorbed into the intercept. By default the reference is the first value in alphabetical order: desktop.
- The labels read `C(device)[T.mobile]`: the variable, then T. (for **treatment coding**, this way of comparing each category with the reference), then the category.
- So the **Intercept is desktop's mean**, and each other coefficient is that device's mean **minus desktop's**: the `groupby` below it shows the same numbers. It's section 22.10's Kolkata dummy, with three levels.

To compare with a different reference, name it: `C(device, Treatment("mobile"))` makes mobile the reference, and the coefficients become differences from mobile.

Several variables at once

Now device, channel, and pages viewed together:

```
linear = smf.ols("duration_seconds ~ C(device) + C(channel) + pages_viewed", data=model_data).fit()
print(linear.summary().tables[1])
print(f"R-squared: {linear.rsquared:.3f}")
```

```
=====
              coef    std err          t      P>|t|      [0.025      0.975]
-----
Intercept          118.3293         1.571      75.343      0.000      115.251      121.408
C(device)[T.mobile]    -50.5598         1.178     -42.938      0.000     -52.868     -48.252
C(device)[T.tablet]   -24.3993         2.506     -9.736      0.000     -29.311     -19.487
C(channel)[T.email]       2.0011         2.440       0.820      0.412      -2.780       6.783
C(channel)[T.organic]    0.5978         1.493       0.400      0.689      -2.329       3.525
C(channel)[T.paid]      -2.3396         1.736     -1.348      0.178      -5.743       1.063
C(channel)[T.referral]  -2.0592         2.054     -1.003      0.316      -6.085       1.966
pages_viewed           1.5367         0.305       5.043      0.000       0.939       2.134
=====
R-squared:  0.045
```

- In a formula, `+` means "and also this variable", not addition: the model gets a coefficient for each.

- **Reading it:** each coefficient is the difference associated with that variable, *holding the others fixed*. `C(device)[T.mobile]` is mobile compared with desktop, for sessions with the same channel and the same number of pages. Here it barely moves from the device-only model (−50.61 there, −50.56 here), because in this data device hardly goes with channel or pages. When explanatory variables are related, holding the others fixed can change a coefficient a lot, and that change is often the finding.
- `P>|t|` is the p-value for that coefficient, and the last two columns are its 95% interval. The channel intervals all include 0: once device and pages are known, channel tells you little about visit length.

R-squared is section 22.10's R^2 , the share of the variation the model explains. Compare the three models:

```
print(f"R-squared: pages only {m1.rsquared:.3f}, device only {m2.rsquared:.3f}")
print(f"          all three {linear.rsquared:.3f}")
```

```
R-squared: pages only 0.001, device only 0.044
          all three 0.045
```

`.rsquared` reads R^2 from a fitted model. Section 22.10's six customers gave 0.984; here the full model explains 4.5%. A low value is normal for behavioral data: people are not very predictable, and the question here is whether a difference exists, not whether we can forecast a visit.

Formula syntax at a glance

You write	It means
<code>y ~ x</code>	y explained by x, with an intercept
<code>y ~ x1 + x2</code>	y explained by x1 and x2 together, each holding the other fixed
<code>C(x)</code>	treat x as categories: one 0/1 column per category except the reference
<code>C(x, Treatment("level"))</code>	the same, with <code>level</code> as the reference category
<code>y ~ x1 * x2</code>	x1, x2, and their interaction (whether x1's effect depends on x2); Chapter 31 uses it

One caution on this model. It uses sessions, not visitors, and visit lengths are skewed, so treat its p-values as rough (section 30.2's unit-of-analysis warning applies here too). `.fit(cov_type="cluster", cov_kws={"groups": model_data["visitor_id"]})` gives standard errors that allow for repeat visitors; the coefficients stay the same.

Logistic regression: what moves the enquiry rate

For a yes/no outcome, a straight line is the wrong shape. Fit `enquired` (0 or 1) with a line, and nothing stops the line from predicting rates below 0 or above 1, which no rate can be. **Logistic regression** fixes that by fitting a straight line to the *log of the odds* instead.

The **odds** of something are the chance it happens divided by the chance it doesn't. Work them out by hand from section 30.9:

1. Control: 937 visitors enquired and $24,036 - 937 = 23,099$ didn't. The odds are $937 \div 23,099 = \mathbf{0.0406}$ (the rate is 3.90%).
2. Variant: 1,035 enquired and 22,215 didn't. The odds are $1,035 \div 22,215 = \mathbf{0.0466}$.
3. The **odds ratio** is $0.0466 \div 0.0406 = \mathbf{1.148}$: the variant's odds of an enquiry are about 15% higher.

The **log-odds** is the natural logarithm of the odds, which can be any number, positive or negative, so a straight line can model it safely. `np.exp` undoes the logarithm, turning a coefficient back into an odds ratio. First, the model with the variant alone:

```
logit_data = per_visitor.merge(first[["visitor_id", "device", "channel"]], on="visitor_id")
simple = smf.logit("enquired ~ C(variant, Treatment('control'))", data=logit_data).fit(dis=False)
print(simple.params.round(4))
print(np.exp(simple.params).round(4))
```

```
Intercept                -3.2049
C(variant, Treatment('control'))[T.variant_b]    0.1385
dtype: float64
Intercept                0.0406
C(variant, Treatment('control'))[T.variant_b]    1.1485
dtype: float64
```

- `logit_data` is one row per visitor in the test, with their first session's device and channel.
- `smf.logit` works like `smf.ols`, for a 0/1 outcome. `Treatment('control')` makes control the reference (it's first alphabetically anyway, but saying so makes the model readable). `.fit(dis=False)` means "don't print the fitting progress".
- `simple.params` are on the log-odds scale. `np.exp(...)` turns them into odds: the Intercept is the control group's odds, 0.0406, and the variant's coefficient is the odds ratio, 1.148, the hand answer.

Now with device and channel as well:

```
formula = "enquired ~ C(variant, Treatment('control')) + C(device) + C(channel)"
logit = smf.logit(formula, data=logit_data).fit(dis=False)
odds = pd.DataFrame({"odds_ratio": np.exp(logit.params),
                    "ci_low": np.exp(logit.conf_int()[0]),
                    "ci_high": np.exp(logit.conf_int()[1]),
                    "p": logit.pvalues})
print(odds.round(3).to_string())
```

	odds_ratio	ci_low	ci_high	p
Intercept	0.058	0.052	0.065	0.000
C(variant, Treatment('control'))[T.variant_b]	1.145	1.046	1.253	0.003
C(device)[T.mobile]	0.707	0.644	0.777	0.000
C(device)[T.tablet]	0.700	0.567	0.864	0.001
C(channel)[T.email]	1.170	0.990	1.384	0.066
C(channel)[T.organic]	0.778	0.694	0.871	0.000
C(channel)[T.paid]	0.657	0.571	0.756	0.000
C(channel)[T.referral]	0.734	0.621	0.869	0.000

- The formula is the one-variable model's, with `+ C(device) + C(channel)` added; it's kept in a variable, `formula`, only to keep the line short.
- `logit.conf_int()` returns a small DataFrame of 95% intervals on the log-odds scale, with its two columns named `0` (lower end) and `1` (upper end); hence `[0]` and `[1]`. `np.exp` turns each end into an odds ratio. `logit.pvalues` holds each coefficient's p-value.
- The **Intercept** is the odds for the reference visitor, control on desktop through a direct visit: 0.058. As a rate, that's $0.058 \div (1 + 0.058) = 5.5\%$.

Reading it: the variant's odds ratio says visitors shown the shorter form had about 15% higher odds of enquiring, with an interval that stays above 1, which agrees with section 30.9's simple two-proportion test. It moved only from 1.148 to 1.145 when device and channel came in, because randomization had already balanced them between the groups. That agreement is the point: regression didn't rescue a weak result, it re-stated a clear one while holding device and channel constant. An odds ratio is not a relative risk; at low rates like 4% they're close, but say "odds" when you mean odds.

Watch out: control variables can't fix a broken experiment. Adding `device` to the model adjusts for imbalance you can see in the data. It does nothing about imbalance you can't, which is exactly what a sample-ratio mismatch warns you about. Randomization is what makes the comparison fair; regression only sharpens it.

30.12 Writing the result up

The analysis is done in an hour. The write-up is what people act on, and it needs four things in this order: what you found, how sure you are, what to do, and what would change your mind.

Riverstone's enquiry-form test, as Meera sends it:

Shorter enquiry form: recommend shipping, with one caveat

Over two weeks (2–15 February), 47,286 visitors were split between the current form and a shorter one. The shorter form produced **4.45% enquiries against 3.90%**, a rise of **0.55 percentage points** (95% CI 0.19 to 0.91), or **+14%** relative (95% CI +5% to +23%). At February's traffic that's roughly **560 extra enquiries a month** (the interval's ends are about 200 and 930).

Guardrails: pages viewed, sessions per visitor, and the size of the enquiries are unchanged, so the extra enquiries don't look worse on the face of it. We can't yet say how many became orders; I'll report that in six weeks.

Caveat: Safari visitors were split 53/47 rather than 50/50, a tagging bug in the assignment script. Excluding Safari does not change the direction or significance of the result (+0.58 points, $p = 0.008$), but the bug should be fixed before the next test.

Recommendation: ship the shorter form, keep the phone-number field optional, and review order conversion in six weeks. If enquiry-to-order rate drops by more than a fifth, we should revisit.

Notice what it doesn't do: it doesn't say "statistically significant" without a number, doesn't hide the bug, doesn't quote a p-value as the headline, and doesn't promise revenue it hasn't measured.

Instead of	Write
"The test was significant ($p < 0.05$)"	"4.45% against 3.90%: +0.55 points, 95% CI 0.19 to 0.91"
"Enquiries rose 14%"	"+14% relative, which is about 560 extra enquiries a month at current traffic (interval roughly 200 to 930)"
"No difference between the forms"	"No difference we could detect; a gap smaller than 0.5 points would not have shown up in this test"
"Tablet users loved it (+35%)"	"Segments moved in the same direction; the test wasn't powered for individual devices"

APPLY

Common mistakes

Mistake	Symptom	Fix
Reporting a p-value with no effect size	"Significant!" and nobody knows whether to act	Effect, interval, and sample size in one sentence
Treating $p = 0.06$ as "no effect"	Real effects dropped for missing a convention	Report the number and the interval; 0.05 is not a law of nature
Analyzing sessions when you randomized visitors	Inflated sample, everything looks significant	Aggregate to the randomization unit first
Skipping the sample-ratio check	A broken test read as a real result	Chi-square the group sizes before anything else
Peeking and stopping at the first good day	False positives several times the stated rate (25% in section 30.10)	Fixed end date, or a sequential method agreed in advance
Choosing the winning metric afterwards	Something always wins	One primary metric, written down; guardrails are not winners
Testing many segments and reporting the best	"Tablet users loved it"	Segments are exploratory; power was for the whole test
Ignoring novelty	A one-week test reads half as good again as the fortnight (+22% against +14%), and more than four times week two (+5%)	Two whole weeks minimum, then re-check later
Letting one huge value decide a revenue test	Result flips when one row moves groups	Use rates, winsorize or take medians, and say so in advance
Mean comparisons on heavy-tailed data	A t-test on data where the mean is meaningless	Median tests, logs, or capped values
Post-hoc power calculations	"We had 12% power" used as an excuse	Power before the test, from a business-chosen effect

Mistake	Symptom	Fix
Reading a confidence interval as "95% chance the truth is here"	Overconfident statements	"Between X and Y" is the honest phrasing
Confusing an odds ratio with a relative rate	Overstated lifts in logistic results	Say "odds"; convert if the audience needs rates
Adding controls to fix imbalance	A broken randomization treated as fixed	Controls sharpen a fair comparison; they can't create one
Running a test with no decision rule	Endless argument after the result arrives	Agree "ship if..." before the data exists
Rounding away the uncertainty in a headline	" +14%" repeated until it becomes a fact	Carry the interval into the summary line

In the real world: the test that "won" on a Wednesday

The agency's email lands on Wednesday 4 February, two days into the fortnight: *"Early results are excellent: the new form is up 36%, significant at $p = 0.003$. We suggest rolling it out now and saving the remaining spend."*

Meera opens the plan the three of them signed on 30 January. It says: two full weeks, one look, decision rule agreed. She replies with four sentences and a request to wait.

Two weeks later the result is +14%, not +36%, with an interval of +5% to +23%. Three things had made Wednesday's number what it was:

- **Two days of data.** 7,592 visitors, about a sixth of the planned sample. At that size, about 3,800 per group, the test had a fair chance (80% power) of detecting only differences of about 1.3 percentage points or more, so anything it did detect was bound to look enormous.
- **Novelty.** The first days of any change draw disproportionate attention; week one's lift was 22% against week two's 5%.
- **Peeking.** The agency had checked daily since the test started. Section 30.10's simulation puts the false-positive rate for that habit at about 25%, not 5%.

The ending is the good kind: the form really is better, and Riverstone shipped it. But the decision rested on a number two and a half times too large, from a test that had a one-in-four chance of "winning" on noise alone, and it nearly cost the company the rest of the fortnight's data and the Safari bug that the final check uncovered.

What Meera tells Anita: *"The form works, and we're shipping it. The early number was two and a half times too big, which is what early numbers usually are. The agreement to wait two weeks is what let us report a range we can stand behind, and it's also why we found the tagging bug. I'd like the same one-page plan for every test we run with them."*

Notice that the discipline was not statistical skill. It was a page written before the test, and one person willing to say "wait" to a confident email.

Project: design and analyze an experiment

Goal: a complete, honest experiment write-up: a plan agreed in advance, an analysis that follows it, and a recommendation someone can act on.

Tools you'll need

Versions used for this chapter, checked in September 2026:

- **Python 3.14.7** in the chapter's uv project, with **pandas 3.0.6**, **numpy 2.5.3**, **scipy 1.18.1**, **statsmodels 0.15.0**, and **matplotlib 3.11.2**; every output was also checked on Python 3.11.15 with numpy 2.4.6, scipy 1.17.1, and matplotlib 3.10.8, with identical results. Section 30.1 installs them with `uv add`.
- **scipy.stats** for t-tests, chi-square, ANOVA, and distributions. **statsmodels** for proportion tests, power and sample size, multiple-testing corrections, Tukey, and regression with readable summaries.
- **PostgreSQL** to hold the website data if you'd rather query it in SQL: `web_data/load_postgresql.sql` builds `riverstone_web`.
- **An experiment platform** (Optimizely, VWO, GrowthBook, Statsig, or a warehouse-native setup) is what companies use to assign visitors and compute results. They automate the arithmetic here, including sample-ratio checks; they do not decide your metric, your MDE, or your stopping rule.
- **Companion files** in `ch30/`: `generate_riverstone_web.py` (the dataset, seed 30), `ab_summary.py` (section 30.9's tested function), and `test_ab_summary.py` (its test).

Tool note: Bayesian A/B testing. Some teams report "a 93% chance the variant is better, and a 7% chance it's worse by more than 0.2 points" instead of a p-value. That's the Bayesian framing: it starts from a prior belief, updates it with the data, and produces a probability distribution for the effect. It's popular because its sentences match how people think, and because stopping early is less damaging when the method expects it. It needs a prior, and a prior is a judgment that has to be defensible. The design work in section 30.8 is identical either way: metric, randomization, guardrails, duration, decision rule.

Option A: your own data. Any A/B test, pilot, or before-and-after change at work. If you have no experiment, design one on paper and analyze a past change as best you can, saying clearly what the design couldn't rule out.

Option B: Riverstone. The companion data has a second, unanalyzed change: the `/bulk-enquiry` landing page. Design a test for it, use the existing data to estimate the baseline and the sample size, and then analyze the enquiry-form test yourself from scratch before comparing with section 30.9.

Steps:

1. **Write the one-page plan** from section 30.8: question, change, primary metric, guardrails, randomization unit, MDE, sample size, duration, stopping rule, analysis plan, decision rule.
2. **Justify the MDE in money.** How many extra enquiries, orders, or rupees would make the change worth shipping? That number, not a convention, sets the sample size.
3. **Check the split** before looking at the outcome.

4. **Compute the primary metric** with its confidence interval and effect size, both absolute and relative.
5. **Check the guardrails**, and name the one you can't measure yet.
6. **Look for novelty** by comparing the first and second halves.
7. **Write the summary** in the shape of section 30.12: finding, uncertainty, recommendation, and what would change your mind.
8. **Have someone read it** who wasn't involved, and ask them what they would do next. If they can't tell, rewrite it.

Deliverables: the plan, the analysis notebook or script, and a summary of at most 300 words.

Stretch goals:

- Redo the analysis with a Bayesian two-proportion model and compare the sentences you'd say.
- Estimate how long the test would need to run to detect a 0.2 percentage point effect, and decide whether that test is worth running at all.
- Before reading section 30.10, analyze the test excluding Safari visitors yourself, and report whether the conclusion changes.
- Turn the analysis into a tested package (Chapter 29) with the reconciliation numbers as tests.

REVIEW

Recap

- **Standard error** measures how much a sample statistic wobbles; it shrinks with the square root of n . A **confidence interval** turns it into a range you can say out loud.
- A **p-value** is the chance of data this extreme if there were no effect. It is not the size of the effect, not the chance of being wrong, and not proof of no difference.
- **Effect size** (absolute difference, relative lift, Cohen's d , odds ratio, Cramér's V) is what makes a result actionable. Report effect, interval, and n together.
- **Welch's t-test** compares two means; **two-proportion tests** and **chi-square** compare rates and categories; **ANOVA** with a **post-hoc test** compares several groups without inflating the error rate.
- **Power** is the chance of catching a real effect. Fix alpha, power, and the **minimum detectable effect**, and the **sample size** follows. Do it before the test, never after.
- An **experiment** is designed on one page: question, change, primary metric, guardrails, randomization unit, split, MDE, sample size, duration, stopping rule, analysis plan, decision rule.
- Analyze in order: **sample-ratio check**, primary metric with interval, effect size, guardrails, then segments as exploratory.
- **Peeking** (25% false positives in the chapter's simulation), **multiple testing**, **novelty**, **SRM**, and **outliers** are the five ways a clean test goes wrong.
- **Regression** answers "how much, holding other things constant": coefficients for means, **odds ratios** for yes/no outcomes, always with intervals.
- A result is finished when someone can act on it and knows what would change your mind.

Key terms

inference · population · sample · standard error · confidence interval · t-distribution · proportion · unit of analysis · hypothesis test · null hypothesis · p-value · Type I error · Type II error · Welch's t-test · Mann-Whitney U · effect size · Cohen's d · relative lift · absolute difference · odds ratio · Cramér's V · two-proportion test · chi-square test of independence · expected counts · Fisher's exact test · ANOVA · F-statistic · post-hoc test · Tukey HSD · family-wise error rate · Bonferroni correction · Benjamini-Hochberg · false discovery rate · statistical power · alpha · beta · minimum detectable effect · sample size calculation · A/B test · control · variant · randomization unit · guardrail metric · primary metric · stopping rule · decision rule · sample-ratio mismatch · goodness-of-fit test · peeking · sequential testing · novelty effect · winsorizing · Cohen's h · pooled standard error · linear regression for inference · formula (statsmodels) · reference category · treatment coding · logistic regression · odds · log-odds · Bayesian A/B testing · prior

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can compute a standard error and a confidence interval for a mean and for a proportion, by hand.
- ☐ I report an effect size and an interval, never a p-value alone.
- ☐ I can say what a p-value does and doesn't mean, in one sentence, to a non-technical colleague.
- ☐ I choose the right test for the data: Welch's t-test, two proportions, chi-square, or ANOVA with a post-hoc correction.
- ☐ I compute the sample size before a test, from an effect size the business chose.
- ☐ I write the plan first: metric, guardrails, randomization unit, duration, stopping rule, decision rule.
- ☐ I check the sample ratio before I look at the outcome.
- ☐ I can explain peeking, multiple testing, and novelty, with numbers.
- ☐ I read regression coefficients and odds ratios as statements about the business, with their intervals.
- ☐ My write-ups say what would change my mind.

PRACTISE

Exercises

Work in `companion/ch30`, with the data built by `generate_riverstone_web.py`. Predict each answer before running it.

Warm-up

1. Compute the 95% confidence interval for the mean number of pages viewed per session, by hand and with `scipy`. Do they agree?
2. What is the enquiry rate for mobile visitors, with its interval? Is it different from desktop's?
3. A colleague says " $p = 0.2$, so the two groups are the same". Rewrite the sentence honestly.
4. Take 200 random sessions and compute the mean duration's interval. Take 2,000. Take 20,000. What happens to the width, and by how much?

Core

5. Test whether the enquiry rate differs between the `email` and `organic` channels. Report z , p , the difference, and its interval.
6. Run a chi-square test of device against enquiry, and compute Cramér's V . What does it tell you that the chi-square alone doesn't?
7. Riverstone wants to detect a 0.3 percentage point improvement on a 3.9% baseline. How many visitors per group, at 80% power? How long would that take at 3,400 new visitors a day?
8. Section 30.10 re-analyzed the test without any Safari visitors. Exclude only *mobile* Safari visitors instead, and compare. Does the split look clean, and does the conclusion change?
9. Compute the test's result per week, and say which week you'd quote to the board and why.
10. Compare enquiry *values* between the two variants with a t-test and with Mann-Whitney. Explain why they disagree, or why they agree.

Stretch

11. Simulate the peeking experiment with weekly looks instead of daily. How much of the damage does that remove?
12. Simulate 500 A/A tests (both groups identical) at the planned sample size, and plot the p -values. What shape should they have, and do they?
13. Fit a logistic regression of enquiry on variant, device, channel, and region. Which coefficients keep intervals away from 1, and what would you tell the marketing team?
14. The agency proposes testing four form variants at once. Write the design, including how you'd control the error rate, and estimate the sample size per group.

Think about it (no code needed)

15. A test shows +0.4 percentage points, 95% CI -0.1 to $+0.9$. What do you recommend, and what do you need to say about it?
16. Why is randomizing by visitor rather than session important for this particular test? Give one way randomizing by session could mislead.
17. A stakeholder asks for the test to keep running "until it's significant". Explain the problem in two sentences, without using the word p -value.
18. Riverstone's enquiry-to-order rate can only be measured six weeks after the test ends. How would you handle the decision in the meantime?

Answers

1.

```
pages = sessions["pages_viewed"]
se = pages.std(ddof=1) / np.sqrt(len(pages))
by_hand = (pages.mean() - 1.96 * se, pages.mean() + 1.96 * se)
by_scipy = stats.t.interval(0.95, df=len(pages) - 1, loc=pages.mean(), scale=stats.sem(pages))
print(f"mean {pages.mean():.4f}")
print(f"by hand: {by_hand[0]:.4f} to {by_hand[1]:.4f}")
print(f"by scipy: {by_scipy[0]:.4f} to {by_scipy[1]:.4f}")
```

```
mean 2.7413
by hand: 2.7334 to 2.7492
by scipy: 2.7334 to 2.7492
```

They agree to four decimals: with 214,528 rows the t-distribution is indistinguishable from the normal one. The difference only matters below about 30 observations.

2.

```
by_device = by_visitor.groupby("device")["enquired"].agg(["sum", "count"])
by_device["rate"] = by_device["sum"] / by_device["count"]
by_device["se"] = np.sqrt(by_device["rate"] * (1 - by_device["rate"]) / by_device["count"])
by_device["low"] = by_device["rate"] - 1.96 * by_device["se"]
by_device["high"] = by_device["rate"] + 1.96 * by_device["se"]
print(by_device.round(4).to_string())

stat, p = proportions_ztest([by_device.loc["mobile", "sum"], by_device.loc["desktop", "sum"]],
                           [by_device.loc["mobile", "count"], by_device.loc["desktop", "count"]])
print(f"mobile vs desktop: z = {stat:.2f}, p = {p:.3e}")
```

	sum	count	rate	se	low	high
device						
desktop	4183	81062	0.0516	0.0008	0.0501	0.0531
mobile	3049	84473	0.0361	0.0006	0.0348	0.0374
tablet	411	10465	0.0393	0.0019	0.0356	0.0430
mobile vs desktop: z = -15.43, p = 1.011e-53						

Mobile visitors enquire much less often than desktop ones, and the intervals don't overlap. That's a real difference and a large one in business terms, though it says nothing about *why*: people on phones may be browsing rather than buying.

3. *"This test couldn't detect a difference between the groups. With the sample we had, anything smaller than about X would have been invisible, so 'no difference' and 'a difference too small for us to see' look the same here."* Add the interval: if it runs from -2% to +3%, say so, because that rules out large effects even though it rules out nothing small.

4.

```
for n in (200, 2_000, 20_000):
    sample = sessions["duration_seconds"].sample(n, random_state=30)
    low, high = stats.t.interval(0.95, df=n - 1, loc=sample.mean(), scale=stats.sem(sample))
    print(f"n = {n:>6,}: mean {sample.mean():6.1f} s, CI {low:6.1f} to {high:6.1f}, width {high - low:5.1f} s")
```

```
n =      200: mean    82.5 s, CI    69.6 to    95.4, width   25.8 s
n =    2,000: mean    96.7 s, CI    91.0 to   102.4, width   11.4 s
n =   20,000: mean   97.2 s, CI    95.6 to    98.9, width    3.3 s
```

Each tenfold increase in data shrinks the width by roughly $\sqrt{10} \approx 3.2$ times: 25.8 seconds, then 11.4, then 3.3. The first step falls a little short of the rule because a 200-row sample is itself a lottery, which is the whole reason small samples are treated with suspicion. Notice too that the $n = 200$ interval, 69.6 to 95.4 seconds, misses the full-data mean of 96.17: the same skew as in section 30.2, where small samples rarely include the few very long visits.

5.

```
k_email, n_email = counts.loc["email", "sum"], counts.loc["email", "count"]
k_org, n_org = counts.loc["organic", "sum"], counts.loc["organic", "count"]
stat, p = proportions_ztest([k_email, k_org], [n_email, n_org])
low, high = confint_proportions_2indep(k_email, n_email, k_org, n_org, method="wald")
print(f"email {k_email / n_email:.4f} organic {k_org / n_org:.4f}")
print(f"difference {100 * (k_email / n_email - k_org / n_org):+.2f} pp, z = {stat:.2f}, p = {p:.2e}")
print(f"95% CI: {100 * low:+.2f} to {100 * high:+.2f} pp")
```

```
email 0.0616 organic 0.0401
difference +2.15 pp, z = 10.78, p = 4.08e-27
95% CI: +1.70 to +2.59 pp
```

Email converts about 2.2 percentage points better than organic search, and the interval is nowhere near zero. As in section 30.5, the channel didn't cause this: the mailing list is made of people who already know Riverstone.

6.

```
device_table = pd.crosstab(by_visitor["device"], by_visitor["enquired"])
chi2, p, dof, expected = stats.chi2_contingency(device_table)
v = np.sqrt(chi2 / (device_table.to_numpy().sum() * (min(device_table.shape) - 1)))
print(device_table)
print(f"chi-square = {chi2:.1f}, dof = {dof}, p = {p:.3e}, Cramér's V = {v:.3f}")
```

```
enquired      0      1
device
desktop    76879  4183
mobile     81424  3049
tablet     10054   411
chi-square = 244.1, dof = 2, p = 9.815e-54, Cramér's V = 0.037
```

The chi-square says device and enquiring are related, and with 176,000 visitors it would say that for almost any real difference. Cramér's V says the relationship is weak in absolute terms. Both are true: the effect is small per visitor and large in total, because the site gets a lot of visitors.

7.

```
needed = NormalIndPower().solve_power(effect_size=proportion_effectsize(0.039 + 0.003, 0.039),
                                       alpha=0.05, power=0.80, alternative="two-sided")
print(f"visitors per group: {needed:,.0f}")
print(f"total: {2 * needed:,.0f}")
print(f"days at 3,400 new visitors a day: {2 * needed / 3400:.0f}")
```

```
visitors per group: 67,756
total: 135,512
days at 3,400 new visitors a day: 40
```

Detecting 0.3 points instead of 0.5 takes nearly three times the visitors and about six weeks. That's the trade-off to put in front of the business: a longer test, or accepting that small improvements stay invisible.

8.

```
with_both = per_visitor.merge(first[["visitor_id", "device", "browser"]], on="visitor_id")
mobile_safari = (with_both["browser"] == "Safari") & (with_both["device"] == "mobile")
g2 = with_both.loc[~mobile_safari].groupby("variant")["enquired"].agg(["sum", "count"])
r2 = two_proportion_summary(g2.loc["control", "sum"], g2.loc["control", "count"],
                           g2.loc["variant_b", "sum"], g2.loc["variant_b", "count"])
print(f"without mobile Safari: {100 * r2['diff']:+.2f} pp, p = {r2['p_value']:.4f}")
print(f"95% CI {100 * r2['ci_low']:+.2f} to {100 * r2['ci_high']:+.2f} pp")
share = g2.loc["control", "count"] / g2["count"].sum()
print(f"split: {share:.2%} control, SRM p = {stats.chisquare(g2['count'].to_numpy())[1]:.2f}")
safari = with_both.loc[with_both["browser"] == "Safari"]
print(pd.crosstab(safari["device"], safari["variant"]))
```

```
without mobile Safari: +0.56 pp, p = 0.0068
95% CI +0.16 to +0.97 pp
split: 50.20% control, SRM p = 0.44
variant  control  variant_b
device
desktop    1390    1250
mobile     4883    4249
tablet      863     793
```

`~mobile_safari` means "not mobile Safari" for each row (section 18.4). The conclusion doesn't change: +0.56 points against section 30.10's +0.58, both clearly above zero. But the split only *looks* clean. The crosstab shows Safari's imbalance on desktop too (1,390 against 1,250), so the bug is in Safari, not in phones; removing mobile Safari just dilutes what's left until the overall check can't see it. An even overall split doesn't prove a clean test. Exclude every Safari visitor, as section 30.10 did, and report both results.

9.

```
for week, rows in by_week.groupby("week"):
    g = rows.groupby("variant")["enquired"].agg(["sum", "count"])
    r = two_proportion_summary(g.loc["control", "sum"], g.loc["control", "count"],
                              g.loc["variant_b", "sum"], g.loc["variant_b", "count"])
    print(f"{week}: {100 * r['diff']:+.2f} pp, p = {r['p_value']:.3f}")
    print(f"      95% CI {100 * r['ci_low']:+.2f} to {100 * r['ci_high']:+.2f} pp")
```

```
week 1: +0.88 pp, p = 0.001
        95% CI +0.37 to +1.38 pp
week 2: +0.19 pp, p = 0.468
        95% CI -0.32 to +0.70 pp
```

Each week's visitors come from section 30.10's `by_week`, one row per visitor, counted in the week of their first session. Neither week alone: quote the fortnight, +14%, and mention that week one ran hotter because the form was new. Week two's interval includes zero on its own, which is what a smaller sample and a fading novelty look like together. A board deck showing week one's 22% would be repeating the agency's mistake with better arithmetic.

10.

```
vals = per_visitor.dropna(subset=["value"])
control_v = vals.loc[vals["variant"] == "control", "value"]
variant_v = vals.loc[vals["variant"] == "variant_b", "value"]
t = stats.ttest_ind(variant_v, control_v, equal_var=False)
u = stats.mannwhitneyu(variant_v, control_v)
print(f"means:   control {control_v.mean():.0f} variant_b {variant_v.mean():.0f}")
print(f"medians: control {control_v.median():.0f} variant_b {variant_v.median():.0f}")
print(f"t-test p = {t.pvalue:.3f}; Mann-Whitney p = {u.pvalue:.3f}")
```

```
means:   control 42,204 variant_b 40,772
medians: control 23,066 variant_b 22,765
t-test p = 0.700; Mann-Whitney p = 0.952
```

Both tests agree that there's no detectable difference in enquiry value, which is the guardrail passing. They can disagree when a handful of enormous values pull one mean around; the median test ignores their size and only counts which side they fall on. When they disagree, look at the largest few rows before believing either.

11.

```
rng = np.random.default_rng(30)
wins = 0
for _ in range(200):
    a = rng.random(14_000) < 0.039
    b = rng.random(14_000) < 0.039
    for look in (7_000, 14_000):
        # one look mid-way, one at the end
        _, p = proportions_ztest([a[:look].sum(), b[:look].sum()], [look, look])
        if p < 0.05:
            wins += 1
            break
print(f"false 'winners' with two looks: {wins} of 200 ({wins / 200:.1%})")
```



```
false 'winners' with two looks: 15 of 200 (7.5%)
```

Two looks instead of fourteen brings the false-positive rate most of the way back towards 5%. The damage grows with the number of looks, which is why "just check on Friday" is so much safer than "check every morning", and why sequential methods exist for teams that must be able to stop early.

12.

```
rng = np.random.default_rng(30)
pvalues = []
for _ in range(500):
    a = rng.random(24_000) < 0.039
    b = rng.random(24_000) < 0.039
    pvalues.append(proportions_ztest([a.sum(), b.sum()], [24_000, 24_000])[1])
pvalues = np.array(pvalues)
print(f"below 0.05: {(pvalues < 0.05).mean():.1%}    below 0.10: {(pvalues < 0.10).mean():.1%}")
print(f"below 0.50: {(pvalues < 0.50).mean():.1%}    below 0.90: {(pvalues < 0.90).mean():.1%}")
counts_per_bar, _ = np.histogram(pvalues, bins=20, range=(0, 1))
print("p-values per bar of width 0.05:", counts_per_bar)

plt.hist(pvalues, bins=20, range=(0, 1))
plt.xlabel("p-value")
plt.ylabel("tests")
plt.show()
```

```
below 0.05: 6.0%    below 0.10: 11.2%
below 0.50: 49.0%    below 0.90: 90.6%
p-values per bar of width 0.05: [30 26 28 23 22 25 18 31 17 25 26 30 27 31 18 28 27 21 27 20]
```

When nothing is different, p-values are spread evenly between 0 and 1: about 5% below 0.05, half below 0.5. `np.histogram(pvalues, bins=20, range=(0, 1))` counts them in twenty bars of width 0.05 between 0 and 1, and returns the counts and the bar edges (kept in `_`); `plt.hist` with the same `bins` and `range` draws those bars: about 25 per bar, and the histogram is flat, apart from chance. That's what "a 5% false-positive rate" means, and running A/A tests like this is how teams check that their experiment platform is honest before trusting it with real ones.

13.

```
full = per_visitor.merge(first[["visitor_id", "device", "channel", "region"]], on="visitor_id")
model = smf.logit("enquired ~ C(variant, Treatment('control')) + C(device) + C(channel) +
                  C(region)",
                  data=full).fit(dispatch=False)
out = pd.DataFrame({"odds_ratio": np.exp(model.params), "low": np.exp(model.conf_int()[0]),
                    "high": np.exp(model.conf_int()[1]), "p": model.pvalues})
print(out.round(3).to_string())
```

	odds_ratio	low	high	p
Intercept	0.057	0.049	0.067	0.000
C(variant, Treatment('control'))[T.variant_b]	1.145	1.046	1.253	0.003
C(device)[T.mobile]	0.707	0.644	0.777	0.000
C(device)[T.tablet]	0.700	0.567	0.864	0.001
C(channel)[T.email]	1.170	0.990	1.384	0.066
C(channel)[T.organic]	0.778	0.694	0.871	0.000
C(channel)[T.paid]	0.657	0.571	0.756	0.000
C(channel)[T.referral]	0.734	0.621	0.869	0.000
C(region)[T.North]	1.050	0.894	1.234	0.552
C(region)[T.South]	1.036	0.885	1.212	0.659
C(region)[T.West]	1.006	0.867	1.167	0.936

The variant, device, and channel coefficients keep their intervals away from 1; region's don't: region doesn't predict enquiring once the others are in the model (and section 30.6 found no regional difference in visit length either). For marketing: mobile visitors convert at around 0.7 times the odds of desktop ones, and paid traffic at about two thirds the odds of direct traffic (the reference category), so the mobile form and the paid landing pages are where the next tests belong.

14. Four variants means three comparisons against control, so the error rate needs handling and the sample grows:

- Test all four against one control with a **Bonferroni-corrected alpha** of $0.05 / 3 = 0.0167$ for the comparisons, or use Dunnett's test (not covered here), which is designed for many-versus-one.
- The stricter alpha costs sample: at 0.0167 and 80% power for a 0.5-point effect, each group needs about 33,000 visitors rather than 25,000, and there are four groups instead of two.
- Four groups of 33,000 is 133,000 visitors; at 3,400 new visitors a day that's about **six weeks**, long enough that the site, the season, and the traffic mix will all have moved.
- The honest recommendation is usually: test the one or two variants with a real hypothesis behind them, not four because the tool allows it. If all four must run, agree in advance which comparison is primary.

15. The point estimate is positive, the interval crosses zero, so the test didn't settle it. Say: *"The best estimate is +0.4 points, but the range runs from -0.1 to +0.9, so this test can't tell us whether the change helps or does nothing. It does rule out anything larger than about a point."* Then choose deliberately: ship it anyway if it's cheap and the guardrails are fine (a coin-flip with a slight tilt in your favor), run longer if a decision must be right, or drop it if the upside at +0.9 still wouldn't pay for the work. What you must not do is call it a win.

16. Visitors come back: 12% of them have more than one session in the fortnight. If sessions were randomized, the same person could see both forms, which contaminates the comparison, and their sessions wouldn't be independent, which breaks the arithmetic behind the test. A specific way it misleads: people who enquire tend to browse more in that session, so the winning form would accumulate extra sessions, and session-level rates would drift apart for a reason that has nothing to do with the form.

17. *"If we keep looking until the numbers look good, we'll always find a moment when they do, even when the two forms are identical. I ran that experiment on data where nothing was different, and a quarter of the tests produced a 'winner' at some point."* Then offer the alternative: a fixed end date, or a method designed for stopping early, agreed now rather than mid-test.

18. Ship on the evidence you have, and put the six-week check in the plan as a commitment with a threshold attached: "We'll review enquiry-to-order rate on 31 March; if it has dropped by more than a fifth, we revert." Two supporting moves: track a faster proxy in the meantime (whether the sales team can reach the enquirer, which is known within days), and keep the old form available so reverting is a configuration change rather than a project.

Checkpoint (end of the second sitting, section 30.7). (1) The two rates are 0.06 and 0.07, so $\bar{p} = 0.065$ and $\bar{p}(1 - \bar{p}) = 0.065 \times 0.935 = 0.06078$; $n = 2 \times 2.802^2 \times 0.06078 \div 0.01^2 = 2 \times 7.851 \times 0.06078 \div 0.0001 =$ **about 9,540 visitors per group.** (2)

`NormalIndPower().solve_power(effect_size=proportion_effectsize(0.07, 0.06), alpha=0.05, power=0.80)` gives **9,527**; the small gap is the arcsine scale again. (3) Two groups need about 19,050 email visitors. Email brings 7% of 3,400 a day, about 238, so the test takes about 80 days: **eleven to twelve weeks**. That's a long test for a landing page, and a reason to ask whether a 1-point lift is really the smallest one worth knowing about.

NEXT

Where this leads

- **Chapter 31, Causal Inference Without Experiments**, answers the same questions when you can't randomize: difference-in-differences, matching, and regression discontinuity.
- **Chapter 22, Statistics Without Fooling Yourself**, is the intuition this chapter computes; reread its correlation-and-causation section after this one.
- **Chapter 37, Supervised Learning Algorithms**, uses regression to *predict*, where this chapter used it to explain.
- **Chapter 75, Product Sense, Metrics, Case Studies & Guesstimates**, is about choosing and diagnosing metrics like the ones in this chapter's plan.
- **Chapter 29, Python as Software, Not Scripts**, is how an analysis like this becomes a repeatable, tested pipeline instead of a notebook nobody can rerun.
- **Chapter 56, MLOps: Making Models Survive Production**, runs experiments on models, where the treatment is a model version and the guardrails are latency and fairness.
- **Chapter 73, Statistics, Probability & Experimentation Bank**, has about 70 interview questions, including three A/B test debugging cases built on exactly the faults in section 30.10.

Chapter 31. Causal Inference Without Experiments

Chapter at a glance

You will learn to: read a difference of logarithms as a percentage change · say what a causal claim would mean when nobody randomized anything · spot why the two obvious comparisons (before-and-after, and treated-versus-untreated) usually mislead · estimate an effect with difference-in-differences, by hand and as a regression with fixed effects · test the parallel-trends assumption it rests on · build a synthetic control from untreated groups · build a comparison group with matching and propensity scores, and check the balance you achieved · use a threshold in a business rule as a natural experiment (regression discontinuity) · recognize a valid instrumental variable, and why you'll rarely have one · judge how much to trust each method, and write the claim up with its assumptions attached.

Before you start: Chapter 22 (correlation is not causation, section 22.5; fitting a line, section 22.10), Chapter 30 (confidence intervals, experiments, Cohen's d in section 30.4, and section 30.11's regression: one x , categories with `C()`, several variables, R^2 , the formula syntax, and logistic regression), Chapter 18 (pandas `groupby`, `pivot_table`, `pd.cut`, `lambda`, and the NumPy basics in section 18.1), Chapter 21 (standard deviation), and Chapter 11 (`SUMPRODUCT`). Section 31.0 below teaches the logarithms this chapter uses.

Time needed: 15–18 hours, spread over two and a half weeks, in two parts. **Part A, comparisons over time** (sections 31.0–31.5: logs, difference-in-differences, parallel trends, synthetic control), about 8 hours, ending with a checkpoint. **Part B, comparisons across units** (sections 31.6–31.9: matching, regression discontinuity, instruments, and how much to trust each), about 8 hours. Allow 2 more for the project.

Tools: Python as installed in Chapter 17 (any version from 3.11 on runs every example), with pandas, numpy, scipy, statsmodels, and matplotlib. The `uv` project from Chapter 30 (section 30.1) or your Chapter 17 environment has them all. A spreadsheet (Chapter 10) for two by-hand steps.

Practice data: three datasets built by `generate_ch31_data.py` in the companion folder: 24 months of orders for Riverstone's four sales regions, 240 key accounts with a quarterly-review program, and 60,000 orders around the free-delivery threshold. Each one was generated with a **known true effect**, so every method in this chapter can be marked against the right answer. They are simulated at the scale of a much larger Riverstone, like Chapter 28's `riverstone_perf` and Chapter 30's website data, not the one-year database from Chapter 13; every number in them is invented.

In this chapter

START	Why this matters · In plain English	354
LEARN	31.0 Logs in ten minutes	355
	31.1 The data, and the three true answers	357
	31.2 Why the two obvious comparisons mislead	360
	31.3 Difference-in-differences	361
	31.4 The assumption: parallel trends	364
	31.5 Synthetic control	368
	31.6 Matching and propensity scores	372
	31.7 Regression discontinuity: let the rule do the randomizing	379
	31.8 Instrumental variables: the idea	383
	31.9 How much to trust each answer	384
APPLY	Common mistakes · In the real world · Project	387
REVIEW	Recap · Key terms · Check yourself	390
PRACTISE	Exercises	391
	Answers	393
NEXT	Where this leads	400

START

Why this matters

Chapter 30's tools need a coin flip. Most business questions never get one:

- "What did the 6% price rise in the North region actually cost us in volume?" The price rose for every North customer on the same day. There is no control group, and no going back.
- "Do quarterly business reviews grow accounts?" The accounts that got them were chosen because they were big and growing.
- "Is free delivery above ₹25,000 bringing customers back?" The threshold applies to everyone.
- "Did the new warehouse cut delivery times?" It opened once, in one place.

You can't randomize any of these, and you still have to answer them, because somebody is going to act on the answer. The methods in this chapter get closer to a causal answer than "revenue went up after we did it", by finding a comparison that stands in for the experiment you couldn't run. They are also what economists, policy analysts, and product data scientists spend most of their time on, and a favorite interview topic: "you can't A/B test this. How would you measure it?"

None of them is magic. Each replaces randomization with an **assumption**, and the discipline is to state the assumption, test what you can, and say what would break it.

In plain English

A causal claim is a comparison with a world that didn't happen.

"The price rise cost us 8% of our order volume" means: North's volume with the price rise, against North's volume in the same months *without* it. The second half doesn't exist. Every method here is a way of building a stand-in for it, called the **counterfactual**.

- **Before-and-after** uses North's own past as the stand-in. It fails whenever anything else changed at the same time: the season, the economy, a competitor.
- **Difference-in-differences** uses the other regions' change over the same months to measure "anything else", and subtracts it. The assumption is that North *would have moved like them* if nothing had happened.
- **Synthetic control** builds a fake North out of a weighted blend of the other regions, chosen so the blend tracks North's history closely, and then compares the real North with its double.
- **Matching** builds the stand-in customer by customer: for each account in the program, find an account that looked the same beforehand and didn't get it.
- **Regression discontinuity** uses a rule. An order of ₹24,900 and one of ₹25,100 are nearly identical, except one gets free delivery. Just either side of a threshold, the rule does the randomizing for you.
- **Instrumental variables** find something that nudges the treatment without touching the outcome any other way, and use only the variation that nudge explains.

Every one of them is an argument about the counterfactual, not a calculation that settles the matter. That's why this chapter spends as much time on assumptions and checks as on formulas.

LEARN

31.0 Logs in ten minutes

Every method in this chapter measures an effect as a *percentage*: "the price rise cost 8% of volume", "reviews add 7% to revenue". Percentages are awkward to subtract, and this chapter subtracts changes from changes all the time. The tool that makes percentages behave is the **logarithm**. Section 30.11 used it once, for log-odds, without stopping to explain it; this section teaches it properly, from the start.

The idea

The **natural logarithm** of a positive number x , written $\ln(x)$, is a second way of writing x in which **multiplying becomes adding**:

The one rule you need. $\ln(a \times b) = \ln(a) + \ln(b)$

- Growing by 5% means multiplying by 1.05. On the log scale it means **adding** $\ln(1.05) = 0.049$.
- The **log difference** $d = \ln(\text{after}) - \ln(\text{before})$ is therefore a measure of growth. For small changes it is close to the percentage change: 0.049 against 5%.
- **exp**, the exponential function, undoes $\ln$: $\exp(\ln(x)) = x$. So $\exp(d) - 1$ turns a log difference back into an exact percentage change.

(The "natural" log is the one based on the number $e \approx 2.718$. You won't need that fact here; spreadsheets and Python use it whenever they say **LN** or **log**.)

By hand, with this chapter's numbers

Section 31.2 finds that North averaged 2,032 orders a month before its price rise and 2,067 after. Three ways to measure the change:

Step	Arithmetic	Result
Ordinary percentage change	$2,067 \div 2,032 - 1$	0.0172, or +1.72%
Log difference	$\ln(2,067) - \ln(2,032) = 7.6339 - 7.6168$	0.0171
Back from logs	$\exp(0.0171) - 1$	0.0172, or +1.72%

The log difference, 0.0171, is almost the percentage itself, and converting it back gives exactly the ordinary answer.

In a spreadsheet

Type 2032 in A1 and 2067 in A2. Then `=LN(A2)-LN(A1)` in B1 returns 0.0171, and `=EXP(B1)-1` in B2 returns 0.0172. LN and EXP work the same in Excel and Google Sheets.

In Python

NumPy (section 18.1) has both functions. First the log difference:

```
import numpy as np

before, after = 2032, 2067
d = np.log(after) - np.log(before)
print(f"ln(before) = {np.log(before):.4f}, ln(after) = {np.log(after):.4f}")
print(f"log difference d = {d:.4f}")
```

```
ln(before) = 7.6168, ln(after) = 7.6339
log difference d = 0.0171
```

- `np.log(x)` is the natural log, the spreadsheet's LN. (NumPy's name for it is `log`, not `ln`.)
- `d` is the log difference, 0.0171, as by hand.

Then back to a percentage, checked against the ordinary calculation:

```
print(f"exp(d) - 1 = {np.exp(d) - 1:.4f}")
print(f"after/before - 1 = {after / before - 1:.4f}")
```

```
exp(d) - 1 = 0.0172
after/before - 1 = 0.0172
```

`np.exp(d)` is the spreadsheet's EXP. The two lines agree, so no information is lost by working in logs.

Where the shortcut breaks

"A log difference is roughly the percentage" holds only for small changes. Before you run the next cell, predict: what percentage change is a log difference of 0.56?


```
for d in [0.01, 0.05, 0.10, 0.56, -0.0729]:
    print(f"log difference {d:+.4f} -> {100 * (np.exp(d) - 1):+6.1f}%")
```

```
log difference +0.0100 -> +1.0%
log difference +0.0500 -> +5.1%
log difference +0.1000 -> +10.5%
log difference +0.5600 -> +75.1%
log difference -0.0729 -> -7.0%
```

Up to 0.05 the two are nearly equal. At 0.10 they differ by half a point, and at 0.56 the change is **+75%**, **not +56%**. So this chapter never reads a log difference as a percentage directly: it always converts with $\exp(d) - 1$. You'll meet both 0.56 and -0.0729 again.

Why this chapter works in logs

Riverstone's West region sells about 3,300 orders a month and East about 1,500. If both grow by 8%, West gains about 264 orders and East about 120: different numbers of orders, the same growth. In logs, both rise by $\ln(1.08) = 0.077$. So when a method asks whether North "moved like the other regions", comparing **changes in logs** asks whether they grew at the **same rate**, which is the fair question for regions of different sizes. That is why every comparison below is done in logs and converted back at the end.

31.1 The data, and the three true answers

Building the data

The generator needs only the Python standard library. In a terminal, in the companion folder (with your Chapter 30 uv project, type `uv run python` instead of `python`, as in section 30.1):

```
# terminal, in companion/ch31
$ python generate_ch31_data.py
region_month 96 rows · qbr_program 240 customers (68 in the program) · delivery_threshold 60000
orders
$ ls causal_data
delivery_threshold.csv
qbr_program.csv
region_month.csv
```

The script writes a folder `causal_data` with three CSV files. The seed is fixed (31), so your files are identical to the book's.

The first cell

Load all three and look at the first one:

```
import numpy as np
import pandas as pd

panel = pd.read_csv("causal_data/region_month.csv", parse_dates=["month"])
qbr = pd.read_csv("causal_data/qbr_program.csv")
orders = pd.read_csv("causal_data/delivery_threshold.csv")

print(panel.head(3).to_string(index=False))
print(f"\npanel: {panel.shape[0]} region-months, {panel['region'].nunique()} regions, "
      f"{panel['month'].min():%b %Y} to {panel['month'].max():%b %Y}")
print(f"qbr: {len(qbr)} accounts, {qbr['in_qbr_program'].sum()} in the program")
print(f"orders: {len(orders):,} orders around the ₹25,000 threshold")
```

region	month	orders	revenue	avg_order_value	price_rise_active	is_north	is_after
West	2024-07-01	3125	77605843.75	24833.87	0	0	0
West	2024-08-01	3061	78519945.53	25651.73	0	0	0
West	2024-09-01	3419	85941317.41	25136.39	0	0	0

```
panel: 96 region-months, 4 regions, Jul 2024 to Jun 2026
qbr: 240 accounts, 68 in the program
orders: 60,000 orders around the ₹25,000 threshold
```

- `parse_dates=["month"]` reads the `month` column as dates rather than text, so the code can compare and format them.
- `.nunique()` counts distinct values: four regions.
- The f-strings use some formats you haven't met; the box below lists every one this chapter uses.

Reading the f-string formats in this chapter. Inside `{...}`, what follows the colon says how to show the value:

- `:,.0f` adds thousands separators and no decimals: 2,032. `:.4f` gives four decimals: 0.0171.
- `:+.1f` also shows the **sign**, even for positive numbers: +1.7 and -7.0. `:+d` does the same for whole numbers: +2.
- `:%b %Y` formats a date as a short month and a year: Jul 2024.
- `:>5` right-aligns the value in 5 characters, and `:6.1f` makes a number 6 characters wide, so columns of output line up.
- Outside the braces, `'#' * 8` repeats a character: `#####`. Section 31.4 uses it to draw a bar chart out of text.

The other two datasets, three rows each:

```
print(qbr.head(3).to_string(index=False))
print()
print(orders.head(3).to_string(index=False))
```

customer_id	segment	region	years_as_customer	revenue_2024	growth_2024	in_qbr_program	
revenue_2025							
1	Retail	East	6	1015757.86	0.2296	0	
1276212.05							
2	Retail	North	3	169196.62	0.1483	0	
174958.74							
3	Hospitality	North	1	507678.95	-0.0138	1	
577513.93							
order_id	order_value	free_delivery	repeat_within_90_days	segment	region		
1	14813.32	0	0	Retail	South		
2	25728.20	1	1	Retail	East		
3	29660.00	1	1	Wholesale	South		

What each column means:

Dataset	Column	Meaning
region_month.csv	region, month	one row per region per month; month is the first day of the month
	orders, revenue, avg_order_value	that month's orders, revenue (₹) and revenue per order (₹)
	price_rise_active	1 for North from October 2025, when its prices were higher; otherwise 0
	is_north, is_after	1 for the North region; 1 for October 2025 onwards
qbr_program.csv	customer_id, segment, region	one row per key account
	years_as_customer	how long the account has bought from Riverstone
	revenue_2024, revenue_2025	the account's revenue (₹) in each year
	growth_2024	2024 revenue growth over 2023, as a fraction: 0.052 is 5.2%
	in_qbr_program	1 if the account got quarterly business reviews during 2025
delivery_threshold.csv	order_id, order_value	one row per order; its value in ₹
	free_delivery	1 if the order value is ₹25,000 or more
	repeat_within_90_days	1 if the customer ordered again within 90 days
	segment, region	the customer's segment and region

Because the data is generated, the chapter can do something no real dataset allows: mark each method's homework.

What happened	True effect	Method
North's list prices rose about 6% on 1 October 2025	order volume -8.0%	difference-in-differences, synthetic control
Some key accounts were given quarterly business reviews during 2025	revenue +7.0%	matching and propensity scores
Orders of ₹25,000 or more get free delivery	repeat-order rate +6.0 points	regression discontinuity

Keep those three numbers in mind. Every estimate below should be judged against them, and none of them lands exactly.

31.2 Why the two obvious comparisons mislead

Before and after

```
panel["log_orders"] = np.log(panel["orders"])
north = panel[panel["region"] == "North"]
before = north.loc[north["is_after"] == 0, "orders"].mean()
after = north.loc[north["is_after"] == 1, "orders"].mean()

print(f"North, 15 months before the price rise: {before:,.0f} orders a month")
print(f"North, 9 months after: {after:,.0f} orders a month")
print(f"change: {100 * (after / before - 1):+.1f}%")
```

```
North, 15 months before the price rise: 2,032 orders a month
North, 9 months after: 2,067 orders a month
change: +1.7%
```

- The first line adds a column of log orders (section 31.0) for every region-month. This comparison doesn't need it yet; every method after it does.
- `north` keeps North's 24 rows; `is_after` splits them into the 15 months before the change and the 9 after.

North's volume *went up* after a price rise that truly cost it 8% of its orders. Two things hide the damage: the business is growing by roughly three-quarters of a percent a month, and the price rose on 1 October, which is the start of the festive season. A before-and-after comparison credits the price rise with both.

Treated against untreated

```
post = panel[panel["is_after"] == 1]
north_post = post.loc[post["region"] == "North", "orders"].mean()
others_post = post.loc[post["region"] != "North", "orders"].mean()
print(f"North after: {north_post:,.0f} orders a month")
print(f"Other regions after: {others_post:,.0f} orders a month")
print(f"difference: {100 * (north_post / others_post - 1):+.1f}%")
```

```
North after: 2,067 orders a month
Other regions after: 2,683 orders a month
difference: -23.0%
```

Worse. North is a smaller region than West or South and always was, so most of that gap is history, not the price rise. The comparison measures the wrong thing entirely.

These two failures have names worth knowing:

- **Confounding:** something else moved at the same time as the treatment (the season, the growth trend) and gets counted as the treatment's effect.
- **Selection:** the treated group differs from the untreated one for reasons that already affected the outcome (North was always smaller; the biggest accounts were chosen for the program).

The methods in this chapter are all attempts to remove one or both.

31.3 Difference-in-differences

Difference-in-differences (DiD) takes the change in the treated group and subtracts the change in an untreated group over the same period. Whatever moved both groups (season, growth, the economy) cancels out.

Work it as a two-by-two table of average log orders:

```
panel["group"] = np.where(panel["region"] == "North", "North (treated)", "Other regions")
panel["period"] = np.where(panel["is_after"] == 1, "after", "before")
cells = panel.pivot_table(index="group", columns="period", values="log_orders", aggfunc="mean")
cells = cells[["before", "after"]]
cells["change"] = cells["after"] - cells["before"]

print(cells.round(4).to_string())
did = cells.loc["North (treated)", "change"] - cells.loc["Other regions", "change"]
print(f"\ndifference-in-differences (log): {did:+.4f}")
print(f"as a percentage change in orders: {100 * (np.exp(did) - 1):+.1f}%")
print("true effect: -8.0%")
```

```
period      before  after  change
group
North (treated)  7.6156  7.6328  0.0171
Other regions    7.7475  7.8375  0.0900

difference-in-differences (log): -0.0729
as a percentage change in orders: -7.0%
true effect: -8.0%
```

How it works:

- `np.where(test, a, b)` (section 18.1) labels each row: "North (treated)" or "Other regions", and "after" or "before".
- `pivot_table` (section 18.8) averages log orders for each group in each period, the four cells of the table; `aggfunc="mean"` asks for the average. `cells[["before", "after"]]` puts the columns in time order.
- `cells.loc[row, column]` reads one cell. North's own change is small and positive (0.0171); the other regions' change over the same months is larger (0.0900). The *gap between the two changes*, $0.0171 - 0.0900 = -0.0729$, is the estimate.
- Section 31.0's rule converts it: $\exp(-0.0729) - 1 = -7.0\%$. That's an honest result for 24 months of noisy data, against a true -8.0% , and a different universe from the $+1.7\%$ that before-and-after suggested.

The same thing as a regression

The two-by-two table only works for one treated group and two periods. Written as a regression, DiD extends to many groups, many periods, and controls. It uses the **dummy variables** of section 22.10, columns that are 1 when something is true and 0 when it isn't, and one new column made by multiplying two of them. First, look at the three columns for each of the four cells:

```
panel["treated"] = (panel["region"] == "North").astype(int)
dummies = panel.groupby(["group", "period"])["treated", "is_after"].first()
dummies["treated x is_after"] = dummies["treated"] * dummies["is_after"]
print(dummies.to_string())
```

		treated	is_after	treated x is_after
group	period			
North (treated)	after	1	1	1
	before	1	0	0
Other regions	after	0	1	0
	before	0	0	0

- `treated` is 1 for North; `is_after` is 1 from October 2025. `.first()` shows each cell's value once, since every row in a cell has the same values.
- `treated x is_after` is the two multiplied together. It is 1 in exactly one cell: **North, after the change**. That column is where the price rise lives.

In a statsmodels formula (section 30.11), `a:b` means "add a column that is a times b": an **interaction term**. (Section 30.11's formula box writes `treated * is_after`, which is shorthand for `treated + is_after + treated:is_after`.) So the regression is:

```
import statsmodels.formula.api as smf

simple = smf.ols("log_orders ~ treated + is_after + treated:is_after", data=panel).fit()
print(simple.summary().tables[1])
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	7.7475	0.045	171.944	0.000	7.658	7.837
treated	-0.1318	0.090	-1.463	0.147	-0.311	0.047
is_after	0.0900	0.074	1.224	0.224	-0.056	0.236
treated:is_after	-0.0729	0.147	-0.495	0.622	-0.365	0.219

`.summary()` prints several tables; `.tables[1]` is the second, the coefficient table, which is all this chapter needs. Each coefficient is one piece of the two-by-two table, exactly as in section 22.10, where a 0/1 x made the slope a difference in means:

Coefficient	Value	In the table
Intercept	7.7475	other regions, before: every dummy is 0
treated	-0.1318	North minus the others, before: $7.6156 - 7.7475$
is_after	0.0900	the other regions' change
treated:is_after	-0.0729	the extra change in North: the DiD

The interaction coefficient is difference-in-differences. Read its row aloud, as section 30.11 does: estimate -0.0729 , standard error 0.147 , $t = -0.0729 \div 0.147 = -0.495$, $p = 0.622$, and a 95% interval from -0.365 to $+0.219$. That interval is uselessly wide, because this model explains every difference between the 96 rows with three dummies, and everything else (the months, the regions' own levels) is left in the noise.

Fixed effects: one dummy per region and per month

Fixed effects give every region its own dummy and every month its own dummy, so the model can soak up "West is always bigger" and "October is always busy" before it looks at the price rise.

`C(region)` and `C(month)` do it, as `C()` did for categories in section 30.11:

```
fe = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit()
names = pd.Series(fe.params.index)
print(len(names), "coefficients")
print(names.str.split("[").str[0].value_counts().to_string())
```

```
28 coefficients
C(month)          23
C(region)          3
Intercept          1
treated:is_after    1
```

- The model has 28 coefficients: the intercept, 3 region dummies (one region is the reference category, as in section 30.11), 23 month dummies (one month is the reference), and the interaction.
- Each `C(month)` coefficient is **that month's shared shock**: how far every region's log orders sat from the reference month. Each `C(region)` coefficient is that region's usual level.
- `names.str.split("[").str[0]` keeps the part of each name before the first `[`, so the counts group `C(month)[T.Timestamp('2024-08-01 00:00:00')]` and its 22 siblings together.
- `treated` and `is_after` are left out on purpose. `C(region)` already knows which region is North, and `C(month)` already knows which months are after the change, so the two columns would repeat information the dummies hold. They are **absorbed** by the fixed effects.

The estimate and its interval:

```
effect = fe.params["treated:is_after"]
ci = fe.conf_int().loc["treated:is_after"]
print(f"effect (log): {effect:+.4f}")
print(ci.round(4).to_string())
```

```
effect (log): -0.0729
0    -0.1064
1    -0.0394
```

`fe.params[...]` reads one coefficient. `fe.conf_int()` is a table of every coefficient's 95% interval, and `.loc["treated:is_after"]` takes its row: two numbers, labelled 0 (the lower end) and 1 (the upper end), as in section 30.11. So `ci[0]` is the lower end and `ci[1]` the upper. Converted to percentages:

```
print(f"as a percentage: {100 * (np.exp(effect) - 1):+.1f}% "
      f"(CI {100 * (np.exp(ci[0]) - 1):+.1f}% to {100 * (np.exp(ci[1]) - 1):+.1f}%)")
print(f"R-squared: {fe.rsquared:.3f}")
```

```
as a percentage: -7.0% (CI -10.1% to -3.9%)
R-squared: 0.991
```

Same estimate, a far tighter interval, and the true -8% sits comfortably inside it. **Region fixed effects** absorb "North is smaller"; **month fixed effects** absorb the festive season and everything else that hit all regions at once. R^2 (section 22.10) is 0.991: regions and months explain almost all of the variation in log orders, which leaves a small residual, and a small residual gives a tight interval.

Watch out: four regions is not many. The interval above treats all 96 region-months as independent pieces of evidence. They aren't: a region's months are related to each other, since whatever made North unusual in March probably still applies in April. **Clustering** tells the formula to treat each region as one block of evidence: `smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit(cov_type="cluster", cov_kws={"groups": panel["region"]})`. Section 30.11 used the same setting for visitors with repeat sessions. Here `cov_type="cluster"` asks for clustered standard errors, and `cov_kws={"groups": ...}` names the column that says which block each row belongs to. But the clustering formula itself needs many clusters, and with only four it is unreliable in both directions (exercise 7 shows it). The honest options are resampling methods designed for few groups, of which the placebo test in exercise 6 is the simplest, or a clearly stated caveat. If your DiD has a handful of groups, say so in the write-up rather than quoting a tidy p-value.

31.4 The assumption: parallel trends

DiD rests on one assumption: **without the price rise, North's orders would have moved like the other regions' orders**. That world doesn't exist, so it can't be tested. What can be tested is whether the two moved together *before* the change.

First, a table with one row per month: North's log orders, the other regions' average, and the gap between them:

```
monthly = (panel.assign(side=np.where(panel["treated"] == 1, "North", "Others"))
              .pivot_table(index="month", columns="side", values="log_orders",
                           aggfunc="mean"))
monthly["gap"] = monthly["North"] - monthly["Others"]
print(monthly.round(3).head(3).to_string())
```

side	North	Others	gap
month			
2024-07-01	7.533	7.654	-0.121
2024-08-01	7.578	7.734	-0.157
2024-09-01	7.623	7.791	-0.168

- `.assign(side=...)` (section 18.6) adds a column labelling each row "North" or "Others", without changing `panel` itself.
- The `pivot_table` averages log orders by month and side: for "Others" that's the mean of three regions.
- `gap` is North minus the others. It is negative because North is smaller; what matters is whether it **stays level**.

Split the months at the change:

```
pre = monthly[monthly.index < "2025-10-01"]
post = monthly[monthly.index >= "2025-10-01"]
print(len(pre), "months before,", len(post), "after")
```

15 months before, 9 after

The index holds dates, and pandas compares a date with a date written as text, so `< "2025-10-01"` keeps the months before October 2025.

Now the summary numbers:

```
print("gap between North and the other regions (log orders)")
print(f" before the price rise: mean {pre['gap'].mean():+.4f}, sd {pre['gap'].std():.4f}")
print(f" after:                      mean {post['gap'].mean():+.4f}")
```

```
gap between North and the other regions (log orders)
before the price rise: mean -0.1318, sd 0.0446
after:                      mean -0.2047
```

The gap's average falls from -0.1318 to -0.2047 , a change of -0.0729 : the DiD again. Its standard deviation before the change (Chapter 21) says how much it wanders month to month: about 4.5%.

Is the gap drifting? Fit a straight line through the fifteen pre-change gaps, against month number 0, 1, 2 ... 14, with section 22.10's `linregress`, and read its slope:

```
from scipy import stats

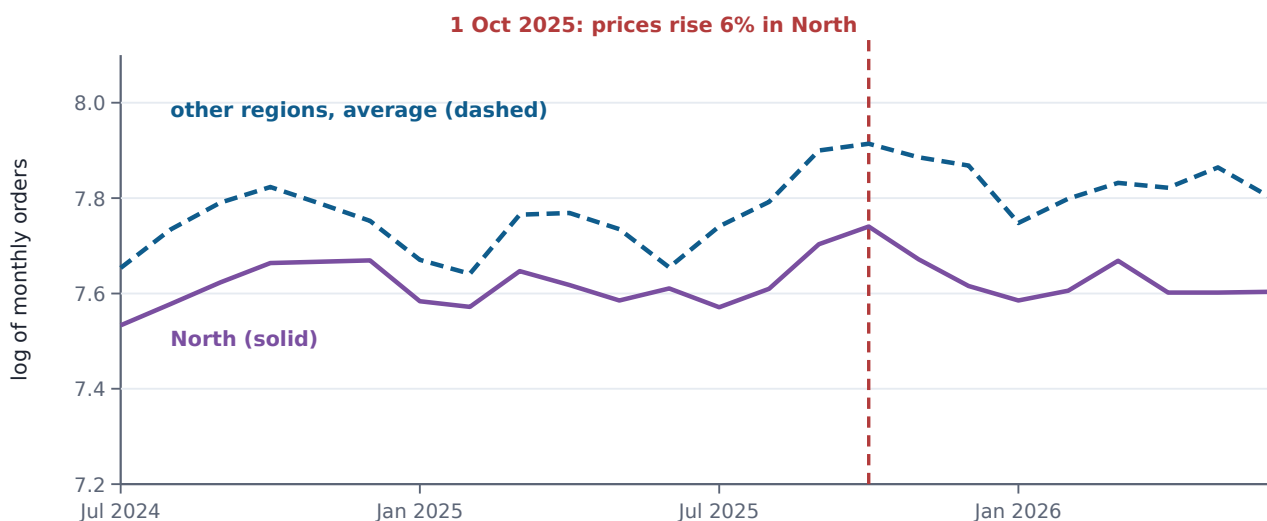
drift = stats.linregress(range(len(pre)), pre["gap"]).slope
print(f"drift per month before the change: {drift:+.5f}")
print(pre["gap"].round(3).tail(4).to_string())
```

```
drift per month before the change: -0.00175
month
2025-06-01    -0.044
2025-07-01    -0.170
2025-08-01    -0.182
2025-09-01    -0.196
```

- `range(len(pre))` is the month numbers 0 to 14, the x; `pre["gap"]` is the y. `.slope` is the change in the gap per month.
- `.tail(4)` shows the last four pre-change months.

Reading it: before the change, the gap wanders with a standard deviation of about 4.5% and drifts by -0.18% a month, which over fifteen months comes to about a third of the effect being measured: small, but not zero. After the change the gap drops by about 7 points and stays there. The last three pre-change months (July to September 2025) all sit below the pre-change average of -0.132 , which is why the next check tracks the gap over time instead of comparing two averages.

North against the other regions: parallel, then a step



Before the change the gap between the lines wanders without a clear direction; after it, North sits lower.
Difference-in-differences: -7.0% (95% CI -10.1% to -3.9%). True effect built into the data: -8% .

Figure 31.1 — Parallel before, a step at the change, parallel again after. This picture belongs in every DiD write-up.

A quarter-by-quarter view

A stricter look tracks the gap over time rather than averaging the whole period. Single months are too noisy for one region, so group them into quarters around the change, measured against the average gap in the fifteen months before it:

```
monthly["rel"] = ((monthly.index.year - 2025) * 12 + monthly.index.month - 10)
baseline = monthly.loc[monthly["rel"] < 0, "gap"].mean()      # all 15 pre-change months
monthly["quarter"] = np.floor(monthly["rel"] / 3).astype(int)
view = (monthly.groupby("quarter")["gap"].mean() - baseline).round(3)
for quarter, value in view.items():
    when = "before" if quarter < 0 else "after "
    print(f"quarter {quarter:+d} ({when}): {value:+.3f} {'#' * int(abs(value) * 100)}")
```

```
quarter -5 (before): -0.016 #
quarter -4 (before): +0.010 #
quarter -3 (before): +0.040 #####
quarter -2 (before): +0.017 #
quarter -1 (before): -0.051 #####
quarter +0 (after ): -0.082 #####
quarter +1 (after ): -0.041 #####
quarter +2 (after ): -0.096 #####
```

- `rel` counts months relative to the change: October 2025 is 0, September 2025 is -1, July 2024 is -15.
- `np.floor(rel / 3)` rounds down to whole quarters, so months -3 to -1 are quarter -1 and months 0 to 2 are quarter 0. `.astype(int)` makes them whole numbers.
- `view` is each quarter's average gap minus the pre-change baseline. The loop prints it, with one # per point (section 31.1's box).

Four of the five quarters before the change sit within four points of the baseline, with no direction to them. But the quarter just before the change is already 5 points low, about two-thirds of the effect. That could be noise (single quarters of one region move this much: see quarter -3 at +4) or the start of the fall. It is why exercise 13 re-estimates with a trend for North, and why the write-up should say "parallel for twelve of the fifteen months, with a dip in the last quarter". Every quarter after the change sits between four and ten points below.

Done properly as a regression, with a confidence interval for each period, this quarter-by-quarter view is called an **event study**. With one treated region and 24 months there isn't enough data for a formal event-study regression with trustworthy intervals, and pretending otherwise would be exactly the kind of tidy-looking mistake this chapter warns about.

Watch out: parallel trends can fail quietly. Common causes: the treated group was picked because it was already declining (Riverstone might have raised prices in North precisely because volume was soft); a different event hit only the treated group at the same time; or the groups have different seasonal patterns. Check what you can, and name what you can't: *"we assume the North region would have grown like the others, which held for most of the fifteen months we can see, with a dip in the last quarter before the change."*

31.5 Synthetic control

DiD compares North with the *average* of the other regions. But West is over one and a half times North's size and East is smaller; why should their plain average be the right stand-in?

Synthetic control builds a better one: a weighted blend of the untreated regions, with the weights chosen so the blend tracks the treated region as closely as possible *before* the change. Then it carries those weights forward and compares. The untreated regions it can draw on are the **donor pool**.

The idea, by hand

A blend is a weighted sum: with weights 0.2 for West, 0.3 for South and 0.5 for East, the synthetic North in a month is $0.2 \times \text{West} + 0.3 \times \text{South} + 0.5 \times \text{East}$, in log orders. The weights are never negative and add up to 1, so the blend is a genuine mixture of the donors. To score a set of weights, compare the blend with the real North in each of the fifteen months before the change, square each miss, and average the squares: that average is the **loss**. The best weights are the ones with the smallest loss.

Start with one row per month and one column per region, and save the pre-change months for the spreadsheet:

```
wide_panel = panel.pivot_table(index="month", columns="region", values="log_orders")
pre_period = wide_panel[wide_panel.index < "2025-10-01"]
print(len(pre_period), "months before the change")
print(pre_period.round(3).head(3).to_string())
pre_period.to_csv("pre_period_logs.csv")
```

15 months before the change				
region	East	North	South	West
month				
2024-07-01	7.224	7.533	7.690	8.047
2024-08-01	7.315	7.578	7.861	8.026
2024-09-01	7.347	7.623	7.888	8.137

`to_csv` writes the fifteen rows to `pre_period_logs.csv`, in the companion folder.

In a spreadsheet

Open `pre_period_logs.csv` in your spreadsheet. The months are in A2:A16, and the columns are in alphabetical order: East in B, North in C, South in D, West in E.

1. Put the three weights in H2 (East), I2 (South), and J2 (West), each `=1/3` to start. In K2, `=SUM(H2:J2)` checks they add up to 1.
2. The blend in F2: `=B2*$H$2+D2*$I$2+E2*$J$2`, filled down to F16. (`SUMPRODUCT`, section 11.2, does the same in one function when the columns sit side by side.)
3. The squared miss in G2: `=(C2-F2)^2`, filled down.
4. The loss in G18: `=AVERAGE(G2:G16)`. With equal weights it is 0.0192.
5. Now think: North's average log orders, 7.62, sits between South's 7.82 and East's 7.31, and West, at 8.11, is far above it. Try 0.5 for East, 0.5 for South and 0 for West. The loss drops to 0.0044.

In Excel, the **Solver** add-in (section 11.9) can search for the best weights: set the objective to G18, **To: Min**, by changing H2:J2, add the constraint K2 = 1, tick **Make Unconstrained Variables Non-Negative**, and choose the **GRG Nonlinear** method. Python does the same search below.

The same blend in Python

First the equal-weight blend, exactly as in the spreadsheet:

```
donors = ["West", "South", "East"]
w = np.array([1/3, 1/3, 1/3])
blend = (pre_period[donors] * w).sum(axis=1)
loss_equal = ((pre_period["North"] - blend) ** 2).mean()
print(f"loss with equal weights: {loss_equal:.4f}")
```

```
loss with equal weights: 0.0192
```

- `pre_period[donors] * w` multiplies each donor's column by its weight: West by the first number, South by the second, East by the third.
- `.sum(axis=1)` adds across each row (section 18.6's `axis=1`), giving one blended value per month: column F of the spreadsheet.
- The last calculation squares each month's miss and averages: the loss, 0.0192, as in G18.

Wrap the loss in a function, so any weights can be scored:

```
def loss(weights):
    blend = (pre_period[donors] * weights).sum(axis=1)
    return ((pre_period["North"] - blend) ** 2).mean()

print(f"equal weights: {loss(np.array([1/3, 1/3, 1/3])):.4f}")
print(f"South and East, half each: {loss(np.array([0, 0.5, 0.5])):.4f}")
```

```
equal weights: 0.0192
South and East, half each: 0.0044
```

The spreadsheet's two numbers again. Row by row, the blend is a `SUMPRODUCT` of the three donors and the three weights. NumPy has an operator for exactly that, applied to every row at once: `@`, the **matrix product**. Chapter 35 explains it fully; here it only needs to mean "multiply each row by the weights and add up". Check that it gives the same numbers:

```
by_columns = (pre_period[donors] * w).sum(axis=1)
by_matrix = pre_period[donors].to_numpy() @ w
print(by_columns.head(3).round(4).to_list())
print(by_matrix[:3].round(4))
```

```
[7.6537, 7.7342, 7.7907]
[7.6537 7.7342 7.7907]
```

`.to_numpy()` turns the three columns into a plain NumPy array, which `@` needs. The two lines agree, so from here on either form will do.

Letting the computer search: minimize

Trying weights by hand is slow. SciPy's `minimize` tries weights for you, moving them in whichever direction lowers the loss, until it can't go lower. It needs the rules, too: each weight between 0 and 1, and the three adding up to 1.

```
from scipy.optimize import minimize

start = np.array([1/3, 1/3, 1/3])
bounds = [(0, 1)] * 3
constraint = {"type": "eq", "fun": lambda w: w.sum() - 1} # zero when weights add up to 1
best = minimize(loss, start, bounds=bounds, constraints=[constraint], method="SLSQP")
print(best.success)
weights = {d: round(float(x), 3) for d, x in zip(donors, best.x)}
print("synthetic North =", weights)
print(f"loss at these weights: {best.fun:.5f}")
```

```
True
synthetic North = {'West': 0.187, 'South': 0.303, 'East': 0.509}
loss at these weights: 0.00184
```

- `minimize(loss, start, ...)` takes the function to make small and the weights to start from, the spreadsheet's =1/3 cells.
- `constraints=[constraint]` takes a list of rules; here there is one.
- `bounds` gives each weight a lowest and highest value. `[(0, 1)] * 3` repeats the pair three times, as `'#' * 8` repeats a character.
- `constraint` is a rule the weights must obey. `"type": "eq"` means "must equal zero", and the `lambda` (section 18.6) returns the weights' sum minus 1, which is zero exactly when they add up to 1: Solver's $K_2 = 1$.
- `method="SLSQP"` picks a search method that can handle bounds and an equality rule together.
- `best.success` is True when the search finished properly; `best.x` holds the weights it found, and `best.fun` the loss there, far below the 0.0044 found by hand. The dictionary comprehension (section 17.7) pairs each donor with its weight, and `round(float(x), 3)` turns each into a plain number with three decimals.

The weights add up to 0.999 only because each is rounded to three places.

The estimate

Carry the weights forward over all 24 months, and measure how far North sits from its double:

```
synthetic = (wide_panel[donors] * best.x).sum(axis=1)
gap = wide_panel["North"] - synthetic
fit_before = np.sqrt((gap[gap.index < "2025-10-01"] ** 2).mean())
print(f"pre-change fit (root mean squared error, log orders): {fit_before:.4f}")
```

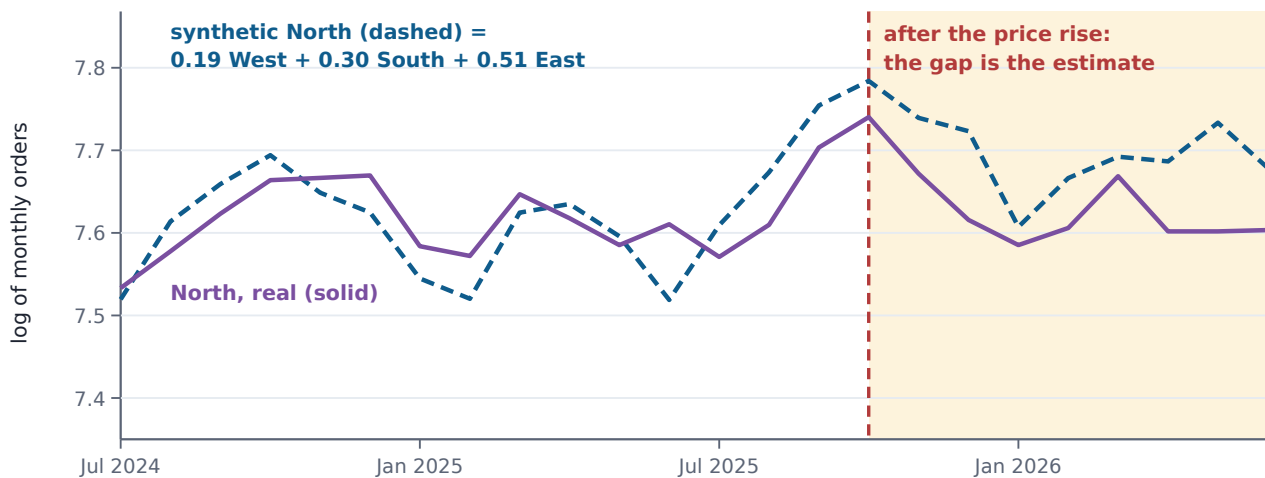
```
pre-change fit (root mean squared error, log orders): 0.0429
```


`fit_before` is the square root of the pre-change loss, the **root mean squared error**: the typical miss before the change, in log orders. About 0.043 is a close fit. Now the months after:

```
gap_after = gap[gap.index >= "2025-10-01"].mean()
print(f"average gap after the change: {gap_after:+.4f} log points "
      f"= {100 * (np.exp(gap_after) - 1):+.1f}%")
print("true effect: -8.0%")
```

average gap after the change: -0.0685 log points = -6.6%
true effect: -8.0%

Synthetic North: a blend of the other regions, fitted before the change



Weights chosen to track North for the fifteen months before October 2025 (root mean squared error 0.043).
Average gap in the shaded months: -6.6%, against a true effect of -8%.

Figure 31.2 — The synthetic region is built only from pre-change data. Everything after the line is the estimate.

How it works and what to watch:

- The weights are non-negative and sum to 1, so the synthetic region is a genuine blend, not an extrapolation. That restraint is what keeps the method honest.
- **Judge it by the pre-change fit first.** A synthetic control that doesn't track the treated unit before the change has no claim to track it after.
- With four regions there are only three donors, so the method is doing little more than a weighted DiD. Synthetic control earns its keep with dozens of candidate donors: shops, cities, states, or customers.
- **Inference** is done by pretending each untreated unit was treated (a **placebo test**) and asking how unusual the real gap looks against theirs. Exercise 11 does this.

Watch out: donors must be untouched. If the price rise spilled into a neighboring region (customers near the border ordering from West instead), West is contaminated and the estimate shrinks toward zero. Ask, every time: could the treatment have affected the comparison group? The same question ruins many DiD studies, and it has a name: **interference** between units.

Checkpoint: the end of Part A

Before Part B, do one difference-in-differences entirely by hand, with East as the only comparison region. The average log orders are:

	before	after
North	7.6156	7.6328
East	7.3118	7.3924

Compute each region's change, subtract East's from North's, and convert with $\exp(d) - 1$. Then check your answer against exercise 5, which fits the same comparison as a regression. (The answer: $0.0172 - 0.0806 = -0.0634$, which is -6.1% ; exercise 5 prints -6.2% , because the table is rounded to four decimals.) If that felt routine, go on. If not, reread sections 31.0 and 31.3 first: everything in Part B builds on them.

31.6 Matching and propensity scores

Quarterly business reviews went to 68 of Riverstone's 240 key accounts. The naive comparison is spectacular and wrong:

```
qbr["log_2025"] = np.log(qbr["revenue_2025"])
qbr["log_2024"] = np.log(qbr["revenue_2024"])
naive = (qbr.groupby("in_qbr_program")
         .agg(accounts=("customer_id", "count"),
              revenue_2024=("revenue_2024", "mean"),
              revenue_2025=("revenue_2025", "mean"),
              growth_2024=("growth_2024", "mean")))
print(naive.round(3).to_string())
```

	accounts	revenue_2024	revenue_2025	growth_2024
in_qbr_program				
0	172	755520.916	824816.990	0.052
1	68	1225516.825	1420723.119	0.062

The accounts in the program were already about 1.6 times the size of the others *before* it started (₹12.3 lakh against ₹7.6 lakh on average), and growing faster. Selection, in one table. Now the naive estimate, in logs as always:

```
means = qbr.groupby("in_qbr_program")["log_2025"].mean()
print(means.round(4).to_string())
gap = means[1] - means[0]
print(f"\nnaive difference in 2025 revenue: {100 * (np.exp(gap) - 1):+.1f}%")
print("true effect: +7.0%")
```

```
in_qbr_program
0    13.3138
1    13.8733

naive difference in 2025 revenue: +75.0%
true effect: +7.0%
```

- `means` holds the average log 2025 revenue of non-participants (row 0) and participants (row 1).
- `means[1] - means[0]` is the difference, converted to a percentage as in section 31.0.

The +75% compares average log revenue, which weights every account equally in percentage terms. The plain averages in the table give $\text{₹}14,20,723 \div \text{₹}8,24,817 - 1 = +72\%$. Either way, the gap is mostly selection.

Matching answers a narrower question: for each account that joined, what happened to a similar account that didn't? A **propensity score** compresses "similar" into one number: the modelled probability of joining, given what was known beforehand.

A propensity score, by hand first

Section 30.11 used logistic regression to read odds ratios. Here it is used for its other job: **turning a score into a probability**. The model adds up a score, $z = b_0 + b_1 \times \text{log_2024} + b_2 \times \text{growth_2024} + \dots$, exactly like a linear regression, and then squeezes z into the range 0 to 1 with the S-shaped **logistic function**:

$$p = 1 / (1 + e^{-z})$$

A z of 0 gives $p = 0.5$; large positive z gives p near 1; large negative z gives p near 0.

It is section 30.11's arithmetic in one line: z is the log-odds, so the odds are e^z , and the rate is $\text{odds} \div (1 + \text{odds})$, which is the same number.

The model uses only things known **before** the program started: 2024 revenue, 2024 growth, tenure, and segment (Common mistakes and exercise 16 say why that matters). Fit it and look at its coefficients:

```
score_model = smf.logit(
    "in_qbr_program ~ log_2024 + growth_2024 + years_as_customer + C(segment)",
    data=qbr).fit(dis=False)
print(score_model.params.round(4).to_string())
```

Intercept	-12.9603
C(segment)[T.Retail]	-0.3170
C(segment)[T.Wholesale]	0.4500
log_2024	0.8544
growth_2024	0.8568
years_as_customer	0.0964

`smf.logit` is section 30.11's logistic regression; `dis=False` stops it printing its fitting progress. The coefficients are on the log-odds scale, and Hospitality is the reference category, so a Retail account adds the `C(segment)[T.Retail]` coefficient and a Hospitality account adds nothing.

Now work the first account by hand: a Retail account with ₹10,15,758 of 2024 revenue, 2024 growth of 0.2296, and 6 years as a customer:

```
first = qbr.iloc[0]
b = score_model.params
z = (b["Intercept"] + b["C(segment)[T.Retail]"
    + b["log_2024"] * first["log_2024"]
    + b["growth_2024"] * first["growth_2024"]
    + b["years_as_customer"] * first["years_as_customer"])
p = 1 / (1 + np.exp(-z))
print(f"account {first['customer_id']}: log_2024 = {first['log_2024']:.4f}")
print(f"z = {z:.4f}, p = {p:.4f}")
print(f"the model's predict gives: {score_model.predict(qbr.iloc[[0]]).iloc[0]:.4f}")
```

```
account 1: log_2024 = 13.8311
z = -0.6851, p = 0.3351
the model's predict gives: 0.3351
```

- `z` adds up the coefficients times the account's values, one term per line of the formula.
- `p` is the logistic function; in a spreadsheet, with `z` in A1, it is $=1/(1+\text{EXP}(-A1))$, which turns `z = -0.6851` into 0.3351.
- `score_model.predict(...)` does the same arithmetic for any rows you give it. `qbr.iloc[[0]]` (double brackets) passes a one-row table, and the result matches the hand calculation.

So the propensity score is nothing more mysterious than a weighted sum passed through the S-curve. Chapter 37 teaches logistic regression as a prediction model in its own right. Now score every account:

```
qbr["propensity"] = score_model.predict(qbr)
summary = qbr.groupby("in_qbr_program")["propensity"].describe()
print(summary[["count", "mean", "min", "max"]].round(3).to_string())
```

	count	mean	min	max
in_qbr_program				
0	172.0	0.255	0.032	0.686
1	68.0	0.354	0.130	0.802

`predict(qbr)` returns one probability per account. `.describe()` summarizes each group, and the list in brackets keeps four of its columns.

The two groups overlap but sit in different parts of the range, which is exactly what a selected program looks like. **Overlap** (also called **common support**) is the part of the range where both groups have accounts. A participant scored above every non-participant (above 0.686 here) has no comparable control, which is why the matching below refuses matches that aren't close.

Matching, by hand

The rule: take the participants from the highest score down; give each the non-participant with the nearest score, provided it is within 0.05 (the **caliper**); and remove that control so nobody is used twice (**matching without replacement**). A toy example with four participants and six controls:

Participant	Score	Nearest available control	Distance	Result
A	0.60	P (0.57)	0.03	matched; P removed
B	0.48	Q (0.51)	0.03	matched; Q removed (R, at 0.44, is 0.04 away)
C	0.35	S (0.33)	0.02	matched; S removed
D	0.20	T (0.10)	0.10	unmatched : over the 0.05 caliper

The controls were P 0.57, Q 0.51, R 0.44, S 0.33, T 0.10, and U 0.70. Three pairs; D goes unmatched, and R and U are never used. Now the same steps in Python, one at a time.

First, the two groups, and the controls as a dictionary of `customer_id`: `score`, from which matched controls can be deleted:

```
participants = qbr[qbr["in_qbr_program"] == 1].copy()
controls = qbr[qbr["in_qbr_program"] == 0].copy()
available = controls.set_index("customer_id")["propensity"].to_dict()
print(len(participants), "participants,", len(available), "controls available")
print({cid: round(score, 3) for cid, score in list(available.items())[:3]})
```

```
68 participants, 172 controls available
{1: 0.335, 2: 0.071, 4: 0.155}
```

`set_index("customer_id")["propensity"]` makes a Series whose labels are customer ids, and `.to_dict()` turns it into a dictionary (Chapter 17). The last line shows its first three entries.

Second, match the highest-scoring participant only:

```
ordered = participants.sort_values("propensity", ascending=False)
top = ordered.iloc[0]
best = min(available, key=lambda cid: abs(available[cid] - top["propensity"]))
print(f"participant {top['customer_id']}: score {top['propensity']:.3f}")
print(f"nearest control {best}: score {available[best]:.3f}, "
      f"distance {abs(available[best] - top['propensity']):.3f}")
```

```
participant 74: score 0.802
nearest control 133: score 0.686, distance 0.116
```

- `ordered` puts participants from the highest score down: `ascending=False` reverses the usual smallest-first order.
- `min(available, key=...)` looks at every key of the dictionary and returns the one whose `key` value is smallest. The `lambda` gives each control id its distance from this participant's score, so `best` is **the control whose distance is smallest**.
- The distance is well over 0.05. This participant, with the highest score of all, sits outside the overlap: the caliper will refuse the match.

Third, the loop, which does that for every participant in turn:

```
pairs = []
for row in ordered.itertuples():
    if not available:
        break
    best = min(available, key=lambda cid: abs(available[cid] - row.propensity))
    if abs(available[best] - row.propensity) <= 0.05:      # the caliper
        pairs.append((row.customer_id, best))
        del available[best]                                # without replacement
print(f"matched pairs: {len(pairs)} of {len(participants)} participants")
print(f"unmatched: {len(participants) - len(pairs)} (no close enough comparison existed)")
for participant_id, control_id in pairs[:3]:
    print(f"  participant {participant_id} <-> control {control_id}")
```

```
matched pairs: 64 of 68 participants
unmatched: 4 (no close enough comparison existed)
  participant 67 <-> control 133
  participant 126 <-> control 112
  participant 239 <-> control 187
```

- `.itertuples()` (section 18.1) hands over one participant at a time, so `row.propensity` is that participant's score. A row loop is fine here: 68 rows, and each step depends on the ones before it, because matched controls are removed.
- `if not available: break` stops early if the controls run out (section 17.6's `break`).
- `<= 0.05` is the caliper: a pair is kept only if the scores are close.
- `del available[best]` removes the matched control from the dictionary, so it can't be matched again.

Fourth, collect every matched account and keep those rows:

```
matched_ids = []
for participant_id, control_id in pairs:
    matched_ids.append(participant_id)
    matched_ids.append(control_id)
matched = qbr[qbr["customer_id"].isin(matched_ids)]
print(len(matched), "accounts in the matched sample")
print(matched["in_qbr_program"].value_counts().to_string())
```

```
128 accounts in the matched sample
in_qbr_program
0    64
1    64
```

`isin` (section 18.4) keeps the rows whose id is in the list: 64 participants and their 64 controls.

Check balance before believing anything

Matching is only worth something if the matched groups now look alike on everything you matched on. The usual measure is the **standardized mean difference** (SMD), the same idea as section 30.4's Cohen's *d*.

$$\text{SMD} = (\text{mean}_1 - \text{mean}_0) \div \sqrt{((s_1^2 + s_0^2) \div 2)}$$

- mean_1 and mean_0 are the participants' and the non-participants' averages.
- s_1^2 and s_0^2 are their variances (Chapter 21: the standard deviation squared). The bottom line is the **pooled standard deviation**: the square root of the average of the two variances, so the gap is measured in standard deviations.
- The common rule of thumb: **|SMD| under 0.1** means the groups are balanced on that variable. The sign only says which group is higher.

By hand for 2024 revenue (in logs), before matching:

```
print(qbr.groupby("in_qbr_program")["log_2024"].agg(["mean", "std"]).round(4).to_string())
```

	mean	std
in_qbr_program		
0	13.2417	0.7658
1	13.7154	0.7256

$(13.7154 - 13.2417) \div \sqrt{((0.7256^2 + 0.7658^2) \div 2)} = 0.4737 \div \sqrt{0.5565} = 0.4737 \div 0.7460 = \mathbf{0.635}$. Now the same formula as a function, for every variable, before and after matching:

```
def smd(data, column):
    a = data.loc[data["in_qbr_program"] == 1, column]
    b = data.loc[data["in_qbr_program"] == 0, column]
    return (a.mean() - b.mean()) / np.sqrt((a.var(ddof=1) + b.var(ddof=1)) / 2)

rows = []
for column in ["log_2024", "growth_2024", "years_as_customer", "propensity"]:
    rows.append({"variable": column, "before matching": smd(qbr, column),
                "after matching": smd(matched, column)})
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

variable	before matching	after matching
log_2024	0.635	0.002
growth_2024	0.082	-0.098
years_as_customer	0.239	0.106
propensity	0.683	0.033

a and b are the two groups' values; `.var(ddof=1)` is the sample variance, as in section 30.4. The first row is the hand answer, 0.635, falling to 0.002 after matching.

Two variables are borderline: tenure is still 0.106, just over the rule, and 2024 growth is -0.098, just under it. Report that, and include them in the outcome regression below as a second line of defence. Segment is a category, so compare its shares instead:

```
for label, data in (("before matching", qbr), ("after matching", matched)):
    print(label)
    shares = pd.crosstab(data["in_qbr_program"], data["segment"], normalize="index")
    print(shares.round(3).to_string())
```


before matching			
segment	Hospitality	Retail	Wholesale
in_qbr_program			
0	0.297	0.506	0.198
1	0.309	0.412	0.279

after matching			
segment	Hospitality	Retail	Wholesale
in_qbr_program			
0	0.281	0.438	0.281
1	0.281	0.438	0.281

`pd.crosstab` (section 21.6) counts each combination; `normalize="index"` turns each row's counts into shares that add up to 1. Before matching, participants were less often Retail (41% against 51%) and more often Wholesale (28% against 20%); after matching the two groups' shares are identical. Segment goes into the outcome regression too.

Matching is only worth something if it balanced the groups

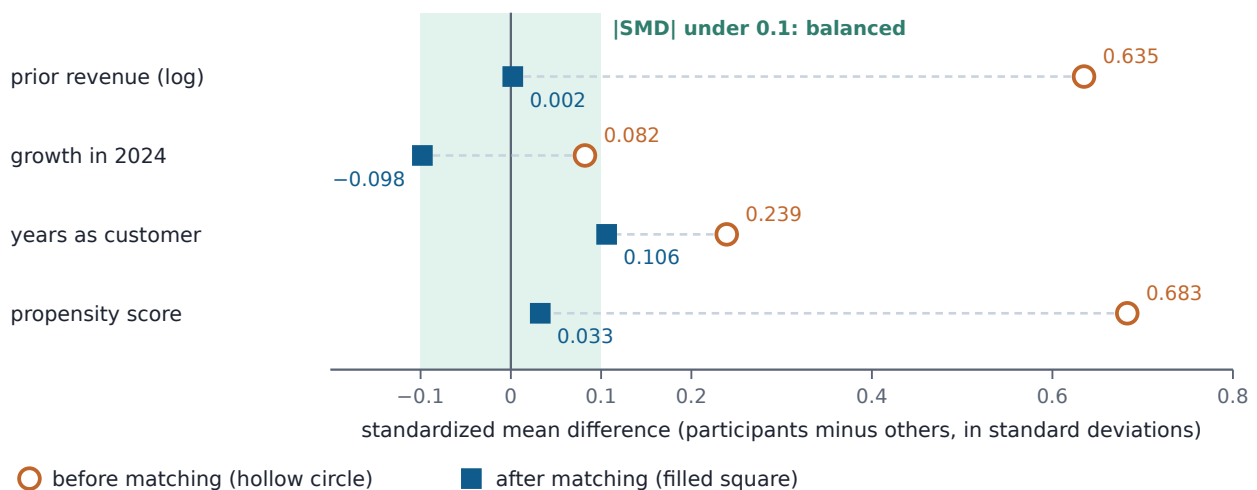


Figure 31.3 — Balance is the checkable part of matching. Report it, or the estimate means nothing.

The estimate

Two corrections: the difference in log 2025 revenue within the matched sample, and a **regression adjustment** on all 240 accounts that holds the pre-program variables fixed (section 22.10's "more than one x"):

```
matched_means = matched.groupby("in_qbr_program")["log_2025"].mean()
matched_effect = matched_means[1] - matched_means[0]
formula = ("log_2025 ~ in_qbr_program + log_2024 + growth_2024"
           " + years_as_customer + C(segment)")
model = smf.ols(formula, data=qbr).fit()
adjusted = model.params["in_qbr_program"]
ci = model.conf_int().loc["in_qbr_program"]

print(f"naive:                {100 * (np.exp(gap) - 1):+.1f}%")
print(f"after matching:       {100 * (np.exp(matched_effect) - 1):+.1f}%")
print(f"regression adjustment:  {100 * (np.exp(adjusted) - 1):+.1f}%  "
      f"(95% CI {100 * (np.exp(ci[0]) - 1):+.1f}% to {100 * (np.exp(ci[1]) - 1):+.1f}%)")
print(f"true effect:           +7.0%")
```

```
naive:                +75.0%
after matching:       +7.1%
regression adjustment: +7.5% (95% CI +4.4% to +10.7%)
true effect:         +7.0%
```

- `matched_means` and `matched_effect` repeat the two-step difference above, inside the matched sample only.
- `formula` is written as two pieces of text side by side, which Python joins into one, so the line stays short. The regression's x's are the program dummy and the four things known before the program; `C(segment)` adds segment as a category. Its `in_qbr_program` coefficient is the gap between participants and non-participants **holding the others fixed**, and `conf_int()` gives its interval, as in section 31.3.
- Everything is in logs, so each estimate is converted with $\exp(d) - 1$.

Both corrections land near the truth, and both are a world away from the naive +75%. Note what made this work: every variable that drove selection (size, growth, tenure) was **measured and available**. That is the assumption matching rests on, and it has a name.

Watch out: matching only fixes what you measured. The assumption is **no unmeasured confounding**: nothing you left out of the score influenced both joining and the outcome. Here it holds by construction, because the data was generated that way. In real life, the sales team's judgment about which accounts "had potential" is exactly the kind of unmeasured variable that ruins it. That's why matching is weaker evidence than a threshold rule or an experiment, and why sensitivity analysis (section 31.9) matters.

31.7 Regression discontinuity: let the rule do the randomizing

Riverstone's delivery policy: orders of ₹25,000 or more ship free; smaller orders pay a delivery charge. Does free delivery bring customers back?

Comparing all orders above the threshold with all orders below it is hopeless: big orders come from big customers, who reorder anyway. But an order of ₹24,900 and one of ₹25,100 are placed by much the same kind of customer, in much the same mood. One gets free delivery and one doesn't, for a reason that has nothing to do with either. **Just around the threshold, the rule is as good as a coin flip.** That's **regression discontinuity (RD)**.

The number that decides which side of the rule an order falls on, here the order value, is called the **running variable**. Measure it from the cut-off, in thousands of rupees, so that 0 means exactly ₹25,000:

```
orders["centred"] = (orders["order_value"] - 25_000) / 1000 # ₹000 from the cut-off
wide = orders[orders["centred"].abs() <= 5]
below = wide[wide["centred"] < 0]["repeat_within_90_days"]
above = wide[wide["centred"] >= 0]["repeat_within_90_days"]

print(f"orders within ₹5,000 of the threshold: {len(wide):,}")
print(f"just below: {100 * below.mean():.1f}% reorder within 90 days (n = {len(below):,})")
print(f"just above: {100 * above.mean():.1f}% (n = {len(above):,})")
print(f"raw gap: {100 * (above.mean() - below.mean()):+.1f} points")
```

```
orders within ₹5,000 of the threshold: 46,054
just below: 31.4% reorder within 90 days (n = 22,958)
just above: 42.4% (n = 23,096)
raw gap: +11.0 points
```

- `centred` is `-0.1` for a ₹24,900 order and `+0.1` for ₹25,100. `25_000` is 25,000; Python ignores the underscore, which is there for reading.
- `wide` keeps orders within ₹5,000 either side; `below` and `above` split them at the cut-off, with exactly ₹25,000 counted above, as the rule says.
- The mean of a 0/1 column is a share, so `below.mean()` is the reorder rate.

The raw gap overstates things, because the chance of reordering rises with order value anyway. RD fits a line on each side and measures the **jump at the cut-off**, not the level. One regression does both lines, using an interaction as in section 31.3:

```
rd = smf.ols("repeat_within_90_days ~ free_delivery + centred + free_delivery:centred",
             data=wide).fit()
print(rd.params.round(4).to_string())
```

Intercept	0.3312
free_delivery	0.0726
centred	0.0079
free_delivery:centred	0.0013

With b_0 to b_3 the four coefficients in that order, the two fitted lines are:

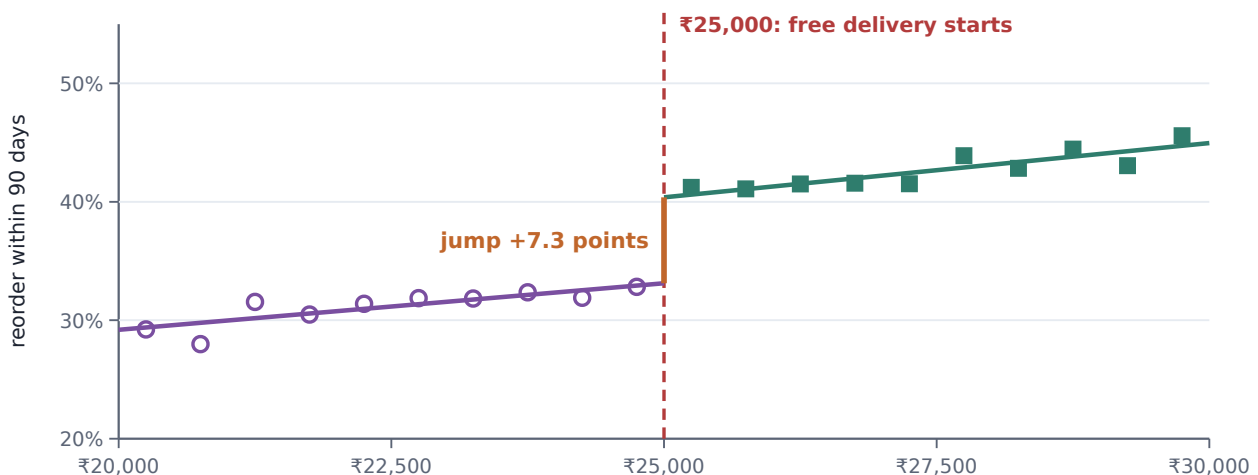
- **below the cut-off** ($\text{free_delivery} = 0$): $\hat{y} = b_0 + b_2 \times \text{centred}$
- **above it** ($\text{free_delivery} = 1$): $\hat{y} = (b_0 + b_1) + (b_2 + b_3) \times \text{centred}$

So `free_delivery:centred` lets the slope differ above the cut-off, and because `centred = 0` is ₹25,000, **b_1 , the `free_delivery` coefficient, is the gap between the two lines exactly at the threshold**: the jump. The outcome is 0 or 1, so a straight-line regression on it (a **linear probability model**) gives answers in probability points; that's fine near the middle of the range, where these rates sit. The jump, its interval, and both slopes:

```
jump = rd.params["free_delivery"]
ci = rd.conf_int().loc["free_delivery"]
slope_above = rd.params["centred"] + rd.params["free_delivery:centred"]
print(f"jump at the threshold: {100 * jump:+.1f} points "
      f"(95% CI {100 * ci[0]:+.1f} to {100 * ci[1]:+.1f})")
print(f"slope below the cut-off: {100 * rd.params['centred']:+.2f} points per ₹1,000")
print(f"slope above the cut-off: {100 * slope_above:+.2f} points per ₹1,000")
print("true effect: +6.0 points")
```

```
jump at the threshold: +7.3 points (95% CI +5.6 to +8.9)
slope below the cut-off: +0.79 points per ₹1,000
slope above the cut-off: +0.92 points per ₹1,000
true effect: +6.0 points
```

Free delivery at ₹25,000: the estimate is the size of the step



Each marker: the reorder rate in a ₹500 band (hollow circles below the cut-off, filled squares above).
 The slopes are the ordinary link between order size and reordering; only the jump is the rule's effect.
 True jump built into the data: +6.0 points.

Figure 31.4 – The estimate is the size of the step, not the difference between the two sides.

Bandwidth: how close is close enough?

The width of the window either side of the cut-off is the **bandwidth**. Narrow windows are more credible and noisier; wide ones are precise and lean more on the straight lines. Report several:

```
rows = []
for bandwidth in (0.5, 1, 2, 3, 5):
    window = orders[orders["centred"].abs() <= bandwidth]
    fit = smf.ols("repeat_within_90_days ~ free_delivery + centred + free_delivery:centred",
                 data=window).fit()
    low, high = fit.conf_int().loc["free_delivery"]
    rows.append({"bandwidth (₹000)": bandwidth, "orders": len(window),
                "estimate (pts)": 100 * fit.params["free_delivery"],
                "ci_low": 100 * low, "ci_high": 100 * high})
print(pd.DataFrame(rows).round(2).to_string(index=False))
print("true effect: +6.0 points")
```

bandwidth (₹000)	orders	estimate (pts)	ci_low	ci_high
0.5	5686	6.34	1.29	11.38
1.0	11331	8.30	4.75	11.84
2.0	22095	8.05	5.54	10.57
3.0	31502	7.65	5.57	9.72
5.0	46054	7.26	5.61	8.90

true effect: +6.0 points

Every window puts the jump between 6 and 9 points, and every interval contains the true 6. That stability across bandwidths is part of the evidence; an estimate that swings when the window changes is a warning, not a result. What happens if you change the bandwidth? The table is the answer: going from ₹500 either side to ₹5,000 uses eight times as many orders and narrows the interval from about 10 points wide (1.29 to 11.38) to about 3 (5.61 to 8.90).

The check that matters: can people move themselves across the line?

RD works only if nobody can precisely control which side of the threshold they land on. If customers nudge a ₹24,400 order up to ₹25,000 to get free delivery, the two sides stop being comparable, because the ones who nudged are the keen ones. The test is to look for **bunching** just above the cut-off: count orders in ₹500 bands and see whether the bands just above are fuller than those just below.

```
bins = pd.cut(orders["centred"], bins=np.arange(-5, 5.5, 0.5), right=False)
density = orders.groupby(bins, observed=True).size()
print(density.iloc[6:14].to_string())
```

centred	
[-2.0, -1.5)	2620
[-1.5, -1.0)	2738
[-1.0, -0.5)	2837
[-0.5, 0.0)	2831
[0.0, 0.5)	2855
[0.5, 1.0)	2808
[1.0, 1.5)	2770
[1.5, 2.0)	2636

- **bins=** gives `pd.cut` the band edges. **`np.arange(-5, 5.5, 0.5)`** makes them -5, -4.5, ... 5. It stops before 5.5, so 5 is the last edge.

- **right=False** (section 18.5) makes each band include its left edge, `[0.0, 0.5)`, so an order of exactly ₹25,000, which gets free delivery, is counted above the cut-off, as in the rule. Without it, it would land in the band just below.
- **observed=True** lists only bands that have orders in them. `.iloc[6:14]` shows the eight bands from -₹2,000 to +₹2,000.

Now the ₹1,000 either side, with ordinary filters:

```
just_below = (orders["centred"] >= -1) & (orders["centred"] < 0).sum()
just_above = (orders["centred"] >= 0) & (orders["centred"] < 1).sum()
print(f"orders in the ₹1,000 below: {just_below:,}")
print(f"orders in the ₹1,000 above: {just_above:,}")
print(f"ratio: {just_above / just_below:.3f} (bunching would push this well above 1)")
```

```
orders in the ₹1,000 below: 5,668
orders in the ₹1,000 above: 5,663
ratio: 0.999 (bunching would push this well above 1)
```

Adding up a True/False column counts the Trues. Smooth: the counts either side are nearly equal. In real data this check often fails: sales reps round orders up to hit thresholds, and expense claims cluster below approval limits. When it fails, RD is not available, and the bunching itself becomes the finding.

Watch out: RD only measures the effect at the cut-off. This estimate says what free delivery does for orders near ₹25,000. It says nothing about ₹5,000 orders, where the delivery cost is a much larger share of the order, or ₹2,00,000 ones, where nobody cares. That narrowness is the price of the credibility: a **local** answer you can trust more than a global one you can't.

31.8 Instrumental variables: the idea

Sometimes nothing above is available, because the thing you care about is chosen by the people you're studying. "Do site visits by a sales rep increase orders?" Reps visit the customers most likely to buy, and no threshold or clean before-and-after exists.

An **instrumental variable** is something that shifts the treatment but affects the outcome *only* through it. Suppose reps visit customers in their own city far more often, because travel time decides the day's route. Distance from the rep's office then nudges the number of visits without, in itself, making a customer buy more plastic crates. The method uses only the variation in visits that distance explains, and discards the rest, which is the part contaminated by the rep's judgment.

A valid instrument needs three things:

Requirement	What it means	For "distance to the rep's office"
Relevance	it really moves the treatment	nearby customers do get more visits: check it, don't assume it
Exclusion	it affects the outcome <i>only</i> through the treatment	distance must not affect orders any other way, which is doubtful: nearby customers may also get faster delivery
Independence	it isn't driven by something that also drives the outcome	offices were placed near big customers, so distance is already related to size

The last two are assumptions, and cannot be tested. The estimation itself is routine (two-stage least squares: predict the treatment from the instrument, then use the prediction), and it is never the hard part. **Finding a defensible instrument is.** Classic examples earned their fame because they're rare: distance to the nearest college for the effect of education on earnings, quarter of birth for years of schooling, rainfall for economic shocks.

For an analyst, the useful skill is recognizing the shape of the argument: "*we are using only the part of the variation caused by something unrelated to the outcome*". That lets you read a study, ask the exclusion question in a meeting, and sometimes spot an instrument hiding in your own systems: a policy that applied to some customers by an accident of timing, a system migration that happened alphabetically, an outage that hit one region's warehouse.

31.9 How much to trust each answer

Method	The comparison it builds	Key assumption	What you can check	Strength of evidence
Before and after	the group's own past	nothing else changed	other series over the same period	weak
Treated vs untreated	another group, right now	the groups were alike already	pre-period levels and trends	weak
Difference-in-differences	the untreated group's change	parallel trends	pre-trends, event study, placebo periods	moderate to strong
Synthetic control	a weighted blend of untreated units	the blend tracks the treated unit for the right reasons	pre-change fit, placebo units	moderate to strong
Matching / propensity	similar-looking untreated units	no unmeasured confounding	balance, overlap, sensitivity analysis	moderate
Regression discontinuity	units just either side of a rule	no precise manipulation of the running variable	bunching, covariate smoothness, bandwidth stability	strong, but local
Instrumental variables	variation caused by the instrument	relevance, exclusion, independence	relevance only	strong if the instrument is valid, which is rare
Randomized experiment (Chapter 30)	a coin flip	the randomization worked	sample-ratio and balance checks	strongest

Covariate smoothness, in the RD row, means that the other characteristics of the orders (segment, region) don't jump at the cut-off either; exercise 12 checks it.

Which method, and what it costs you

Question	Method, and its catch	Evidence
Can you randomize?	Run the experiment (Chapter 30) you decide who gets it	strongest
Is there a threshold rule?	Regression discontinuity the answer is local to the cut-off	strong, local
Before-and-after data for both groups?	Difference-in-differences needs parallel trends	moderate to strong
Many untreated units, one treated?	Synthetic control judge it by the pre-change fit	moderate to strong
Only one snapshot, good covariates?	Matching / propensity scores assumes nothing unmeasured	moderate
Shifts the treatment, not the outcome?	Instrumental variables exclusion can't be tested	strong if valid
None of the above?	Describe; don't claim a cause say so plainly	no causal claim

Figure 31.5 – Work down the list; the first row you can answer yes to is usually the strongest evidence available.

Sensitivity: how wrong could you be?

The question to ask of any observational estimate is not "is it right?" but "how strong would the thing I've missed have to be, to overturn this?" Start with how much of the naive gap the measured variables explained:

```
naive_pct = 100 * (np.exp(gap) - 1)
adjusted_pct = 100 * (np.exp(adjusted) - 1)
print(f"quarterly reviews, unadjusted: {naive_pct:+.1f}%")
print(f"after controlling for size, growth, tenure and segment: {adjusted_pct:+.1f}%")
share = 100 * (1 - adjusted_pct / naive_pct)
print(f"share of the naive gap that was selection: {share:.0f}%")
```

```
quarterly reviews, unadjusted: +75.0%
after controlling for size, growth, tenure and segment: +7.5%
share of the naive gap that was selection: 90%
```

Most of the apparent effect was selection. Now measure how strong the variables you *did* measure are, by leaving each out in turn and watching the estimate move. An unmeasured confounder as strong as one of them could move it about as far:

```
formulas = {
    "all four":
        "log_2025 ~ in_qbr_program + log_2024 + growth_2024 + years_as_customer + C(segment)",
    "without prior size":
        "log_2025 ~ in_qbr_program + growth_2024 + years_as_customer + C(segment)",
    "without 2024 growth":
        "log_2025 ~ in_qbr_program + log_2024 + years_as_customer + C(segment)",
}
for label, formula in formulas.items():
    estimate = smf.ols(formula, data=qbr).fit().params["in_qbr_program"]
    print(f"{label:20s}: {100 * (np.exp(estimate) - 1):+5.1f}%")
```

```
all four          : +7.5%
without prior size : +75.1%
without 2024 growth : +8.7%
```

`formulas` is a dictionary from a label to a regression formula: the full model, then the same without one variable. The loop fits each and prints its `in_qbr_program` estimate as a percentage; `:20s` pads each label to 20 characters so the numbers line up.

Leaving out prior size moves the estimate by about 68 points; leaving out 2024 growth moves it by about one. So a missing variable as strong as growth would barely matter, but one as strong as prior size would swamp the +7.5% that's left. The honest sentence is: *"about +7%, and that estimate depends on the sales team's choices being driven by things we can see."* Formal versions of this reasoning exist (Rosenbaum bounds, E-values); the informal version, said out loud with these numbers, is already most of the value.

Writing up a causal claim

Four sentences, in this order:

1. **The estimate, with its interval and its unit.** *"North's order volume fell about 7% (95% CI 4% to 10%) after the price rise."*
2. **The comparison that produced it.** *"Measured against the other three regions over the same months, with region and month effects removed."*
3. **The assumption it rests on, and the check you ran.** *"This assumes North would otherwise have grown like the others; their trends ran parallel for twelve of the fifteen months before the change, with a dip in the last quarter."*
4. **What would change your mind.** *"If something else hit North in October, such as a competitor or a lost distributor, this would overstate the price effect. I checked the sales log and found nothing, but it's the risk."*

Interview extra point. Asked "how would you measure something you can't A/B test?", name the method, then immediately name its assumption and the check: "difference-in-differences, assuming parallel trends, which I'd test on the pre-period and show as a chart." Chapter 69's method rewards exactly this: the claim, then the check. It separates people who have read about these methods from people who have used them.

APPLY

Common mistakes

Mistake	Symptom	Fix
Reading a log difference as a percentage	" $+0.56$ in logs, so $+56\%$ " when it is $+75\%$	Always convert with $\exp(d) - 1$
Before-and-after with no comparison group	The season, the trend, or a campaign gets credited to your change	Find an untreated group and use DiD
Comparing treated with untreated as they are	Differences that existed beforehand read as effects	Match, or use each group's <i>change</i>
Skipping the parallel-trends check	A DiD that measures a divergence already under way	Plot the pre-period; show the chart in the write-up
Reading a DiD from few clusters as precise	Tidy p-values from four regions	Cluster, then say plainly that the number of groups is small
Matching on variables measured after the treatment	The control variable is itself an effect; the estimate collapses	Only use what was known before treatment began
Not checking balance after matching	Matching that didn't work, reported as if it did	Standardized mean differences, before and after
Matching outside the region of overlap	Participants with no comparable control get matched anyway	A caliper, and report how many went unmatched
Treating matching as equal to an experiment	Confident causal language on shaky ground	Say "assuming nothing unmeasured drove selection"
RD with a manipulable threshold	Bunching just above the cut-off	Check the density; if it bunches, don't use RD
RD generalized beyond the cut-off	"Free delivery raises reorders by 7 points" for all orders	Say "for orders near ₹25,000"
One bandwidth, chosen after seeing results	An estimate the reader can't audit	Report a table of bandwidths
Synthetic control with a poor pre-change fit	A "control" that never tracked the treated unit	Judge the fit first; abandon the method if it's bad
Contaminated comparison groups	Effects that leak into the control and shrink the estimate	Ask whether the treatment could have touched the comparison
An instrument that affects the outcome directly	A causal claim resting on an untestable assumption nobody stated	State exclusion out loud; most instruments fail it
Causal language without the assumption	"The price rise cost us 7% " with nothing attached	Estimate, comparison, assumption, and what would change your mind

Mistake	Symptom	Fix
Chasing significance instead of magnitude	A statistically significant 0.4% effect driving a big decision	Chapter 30's rule: effect, interval, and sample size

In the real world: the price rise nobody wanted to measure

Riverstone's North region raised list prices 6% on 1 October 2025, to protect margin after a polymer price shock. In April 2026 the board asks the obvious question, and gets three answers in one meeting.

North's regional manager says volume is up 1.7% since the rise, so customers clearly accepted it. The finance manager says North sells 23% less than the other regions, so the rise has done real damage. Both are reading the same table.

Meera's answer takes an afternoon. She builds the region-month panel, plots North against the other three, and shows that the lines ran parallel for most of the fifteen months and then stepped apart in October. Her estimate is a 7% fall in order volume, with an interval from 4% to 10%. Combined with the 6% price rise, revenue is roughly flat (exercise 2). And since North now ships 7% fewer orders, each costing the same to make, gross margin is up: at Riverstone's 2025 gross margin of 26.2% (section 11.6), North's gross profit rises about 14%.

Then the useful part of the meeting. Anita Rao, the Sales Head, asks what else changed in North in October. Meera has checked: no distributor was lost, no competitor opened a depot, the festive season affected all four regions alike. She also says what she can't rule out: with one treated region and four groups in total, the interval is wider than it looks, a single unusual month moves it, and North was already a little low in the quarter before the rise.

The decision is not "the price rise failed". It is to hold the new prices, and to run a proper experiment on the next change: a randomized price test across a sample of customers in two regions, designed with Chapter 30's arithmetic, so that next year's version of this meeting has a coin flip behind it instead of an argument.

What she tells the board: *"Volume fell about 7%, not rose 1.7%. The 1.7% was the festive season and our overall growth, which every region got. Revenue is flat and margin is up, so the rise worked, at the cost of about 7% of volume. I'd like the next price change to go out as a proper test, because this method costs us an afternoon and a wide interval, and a test would cost a fortnight and give a straight answer."*

Project: measure the regional price change

Goal: a causal estimate you can defend in a meeting, with its assumption stated and checked.

Tools you'll need

Outputs in this chapter were checked with Python 3.11.15, pandas 3.0.6, numpy 2.4.6, scipy 1.17.1, and statsmodels 0.15.0, in September 2026; any Python from 3.11 on, with these libraries or newer, gives the same numbers.

- **pandas and numpy** for shaping the data. Nothing else is needed for the estimates: DiD, matching, and RD are all ordinary regressions once the data is shaped correctly.
- **statsmodels** for the regressions, including `cov_type="cluster"` for clustered standard errors and `smf.logit` for propensity scores.
- **scipy** for `linregress` (the pre-trend drift) and `scipy.optimize.minimize` (the synthetic control weights).
- **matplotlib** (Chapter 18) for the parallel-trends chart.
- **Worth knowing about:** `linearmodels` (panel models and two-stage least squares), `DoWhy` and `EconML` (causal graphs, sensitivity analysis, and heterogeneous effects), and `CausalImpact` (a Bayesian time-series version of the synthetic-control idea). They automate the arithmetic, not the argument.
- **Companion files** in `ch31/`: `generate_ch31_data.py` (all three datasets, seed 31, with the true effects in its header).

Option A: your own data. Any change that hit one group and not another, at a known date: a price change, a new opening hour, a policy in one branch, a product that shipped to one country first. Work on a copy, and keep customer data out of anything you share.

Option B: Riverstone. Estimate the effect of North's price rise on order volume, then extend it.

Steps:

1. **Write the counterfactual in one sentence** before touching the data: "North's order volume in October 2025 to June 2026 if prices had not risen."
2. **Compute both naive answers** (before-and-after, and treated-versus-untreated) and say in a line each why they're wrong here.
3. **Estimate the effect with DiD**, first as a two-by-two table, then as a regression with region and month fixed effects, with clustered standard errors.
4. **Check parallel trends:** the chart, and the quarter-by-quarter view.
5. **Run a placebo:** pretend the change happened on 1 January 2025 and re-estimate, using only the months before the real change (6 months before the fake date, 9 after). The placebo must sit entirely before the real change and leave enough months on each side. You should find nothing. If you find something, your design is picking up noise.
6. **Build a synthetic control** and compare it with the DiD estimate. Report the pre-change fit.
7. **Convert to money:** lost orders = the volume effect $\times$ North's average monthly orders before the change $\times$ 9 months; the revenue effect from a DiD on log revenue (exercise 2); and the margin effect, using Riverstone's 2025 gross margin of 26.2% (section 11.6) as the cost share of revenue before the rise: $\text{new gross profit} \div \text{old} = (1 + \text{volume effect}) \times (1.06 - 0.738) \div 0.262$.
8. **Write four sentences** in section 31.9's shape: estimate, comparison, assumption and check, what would change your mind.

Deliverables: the analysis, the parallel-trends chart, the placebo result, and the four-sentence summary.

Stretch goals:

- Estimate the effect on revenue rather than volume, and explain why the two differ.
- Run the DiD dropping one comparison region at a time. How much does the estimate move?
- Use the QBR data to estimate the program's effect with matching, and separately with a DiD on 2024 versus 2025 revenue. Do the two agree?
- Design the randomized price test Meera asked for, using Chapter 30: unit, metric, MDE, sample size, duration.

REVIEW

Recap

- Work in **logs**: a **log difference** is a growth rate, and $\exp(d) - 1$ turns it back into an exact percentage.
- A causal claim compares the world that happened with a **counterfactual** that didn't. Every method here is a way of building a stand-in for it.
- **Before-and-after** is ruined by **confounding**; **treated-versus-untreated** is ruined by **selection**. In Riverstone's data they gave +1.7% and -23% for an effect that was -8%.
- **Difference-in-differences** subtracts the untreated group's change. As a regression, it is the **interaction** of two dummies; add **region and month fixed effects**, and **cluster** standard errors, remembering that few groups means wide, fragile intervals.
- DiD rests on **parallel trends**: untestable for the future, checkable for the past. Show the chart and a period-by-period view, and report a dip when you see one.
- **Synthetic control** blends untreated units to track the treated one before the change, with weights found by minimizing the pre-change loss. Judge it by **pre-change fit**; infer with **placebo** units.
- **Matching** and **propensity scores** build comparison units one at a time. Use only pre-treatment variables, apply a **caliper**, report **balance** and how many units went unmatched, and state the **no unmeasured confounding** assumption.
- **Regression discontinuity** uses a threshold rule as a natural experiment. Fit a line either side of the **running variable's** cut-off, measure the **jump**, check for **bunching**, report several **bandwidths**, and remember the answer is **local** to the cut-off.
- **Instrumental variables** need **relevance**, **exclusion**, and **independence**; only the first is testable, which is why good instruments are rare.
- Evidence ranks roughly: experiment, then RD, then DiD and synthetic control, then matching, then naive comparisons. **Sensitivity analysis** asks how strong a missing confounder would have to be to overturn the result.
- Write causal claims in four sentences: estimate with interval, comparison used, assumption and check, and what would change your mind.

Key terms

logarithm · natural log · log difference · causal claim · counterfactual · confounding · selection · treated and untreated groups · before-and-after · difference-in-differences · two-by-two table · dummy variable · interaction term · fixed effects · absorbed · clustered standard errors · parallel trends · event study · placebo test · synthetic control · donor pool · loss · root mean squared error · pre-change fit · interference between units · matching · propensity score · logistic function · overlap (common support) · caliper · matching without replacement · balance · standardized mean difference · pooled standard deviation · regression adjustment · no unmeasured confounding · regression discontinuity · running variable · cut-off · linear probability model · bandwidth · local effect · bunching · manipulation check · covariate smoothness · instrumental variable · relevance · exclusion restriction · independence · two-stage least squares · sensitivity analysis

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ I can convert a log difference to a percentage change, and I know why the shortcut fails for large changes.
- ☐ I state the counterfactual in words before choosing a method.
- ☐ I can explain why before-and-after and treated-versus-untreated usually mislead, with an example.
- ☐ I can compute DiD as a table and as a regression, map each coefficient to a cell, and say what the fixed effects absorb.
- ☐ I check parallel trends and show the chart, and I know what to do when they aren't parallel.
- ☐ I can build a synthetic control and judge it by its pre-change fit.
- ☐ I can fit a propensity model, work one score by hand, match with a caliper, and report balance before and after.
- ☐ I know that matching assumes no unmeasured confounding, and I say so.
- ☐ I can spot a threshold rule in a business process and use it for regression discontinuity, including the bunching check.
- ☐ I understand that an RD estimate is local to the cut-off.
- ☐ I can state the three requirements for an instrument and explain why exclusion is usually the weak one.
- ☐ Every causal claim I write carries its assumption and what would change my mind.

PRACTISE

Exercises

Work in `companion/ch31`, with the data built by `generate_ch31_data.py`, in the same notebook as the chapter's code. Predict each answer before running it.

Warm-up

1. Compute average monthly orders for each region, before and after October 2025. Which regions grew, and by how much?
2. Run the DiD on **revenue** instead of order volume. Why is the answer so different from the volume effect, and which one answers "did the price rise work"?
3. For the QBR program, compare the average 2024 revenue of participants and non-participants. What does that tell you before you estimate anything?
4. How many orders in the threshold dataset are within ₹500 of the cut-off, and what share of them got free delivery?

Core

5. Re-estimate the price effect using only West and South as comparison regions. Then only East. How much does the estimate move, and what does that tell you?
6. Run a **placebo test**: pretend the price rise happened on 1 January 2025 and estimate a DiD using only the months before the real change (6 months before the fake date, 9 after). What should you find, and what do you find?
7. Add clustered standard errors to the fixed-effects DiD (`cov_type="cluster"` grouped by region). How does the interval change, and why shouldn't you trust it too far?
8. Estimate the QBR effect with a DiD on the same customers (2024 versus 2025 revenue, participants versus not) instead of matching. Compare with section 31.6's estimates.
9. Redo the matching with a tighter caliper of 0.01 and with no caliper at all. Report pairs matched, balance, and the estimate for each.
10. Fit the RD with a quadratic on each side of the threshold at a bandwidth of ₹5,000. Does the estimate change? Which specification would you report?

Stretch

11. Run the synthetic-control **placebo**: build a synthetic West, South, and East in turn, and compare their post-October gaps with North's. Is North's gap unusual?
12. The RD dataset contains segment and region. Check that these are **smooth** across the threshold, and explain why that check matters.
13. Estimate the price effect with a regression that also lets North follow its own straight-line trend. Does the effect survive?
14. Write the "what would change your mind" sentence for each of the three estimates in this chapter, and rank the three by how much you'd trust them.

Think about it (no code needed)

15. Riverstone's warehouse in Bhiwandi was expanded in March 2026, serving mostly West-region customers. How does that affect the price-rise DiD, and what would you do about it?
16. A colleague proposes matching customers on their 2025 revenue to estimate the 2025 effect of the QBR program. What's wrong with that?
17. Give one threshold rule in a business you know that could serve as a regression discontinuity, and one reason it might fail the bunching check.
18. When would you prefer a weaker method with data you have over a stronger one that would take six months to arrange?

Answers

1.

```
change = (panel.pivot_table(index="region", columns="is_after", values="orders", aggfunc="mean")
          .rename(columns={0: "before", 1: "after"}))
change["change_%"] = 100 * (change["after"] / change["before"] - 1)
print(change.round(1).to_string())
```

is_after	before	after	change_%
region			
East	1500.9	1625.4	8.3
North	2031.7	2067.1	1.7
South	2501.3	2697.9	7.9
West	3334.9	3725.3	11.7

Every region grew, North least of all. The gap between North's growth and the others' is the DiD estimate, seen in levels rather than logs.

2.

```
panel["log_revenue"] = np.log(panel["revenue"])
rev = smf.ols("log_revenue ~ treated:is_after + C(region) + C(month)", data=panel).fit()
vol = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit()
print(f"effect on volume: {100 * (np.exp(vol.params['treated:is_after']) - 1):+.1f}%")
print(f"effect on revenue: {100 * (np.exp(rev.params['treated:is_after']) - 1):+.1f}%")
print(f"price rise applied: +6.0%")
```

```
effect on volume: -7.0%
effect on revenue: +0.7%
price rise applied: +6.0%
```

Volume fell about 7%, and revenue is close to flat. If only the price had changed, revenue would have moved by $0.93 \times 1.06 - 1 = -1.4\%$. The estimate is +0.7%, so the average order value rose about 8%, more than the 6% price rise: the orders North lost were smaller ones. That mix shift is itself a finding; check it with a DiD on the log of avg_order_value. Both effects are true and they answer different

questions: the volume effect measures how customers responded, the revenue effect what it did to the top line. For "did the price rise work", the revenue (and ideally margin) effect is the one the board wants, with the volume effect as the explanation.

3.

```
columns = ["revenue_2024", "growth_2024", "years_as_customer"]
by_group = qbr.groupby("in_qbr_program")[columns].mean()
print(by_group.round(3).to_string())
ratio = by_group.loc[1, "revenue_2024"] / by_group.loc[0, "revenue_2024"]
print(f"ratio of average 2024 revenue: {ratio:.2f}x")
```

in_qbr_program	revenue_2024	growth_2024	years_as_customer
0	755520.916	0.052	5.023
1	1225516.825	0.062	5.632

ratio of average 2024 revenue: 1.62x

Participants were already 1.6 times the size of non-participants *before* the program began, and growing faster. Any raw comparison of 2025 revenue will mostly measure that. This table is the argument for matching, and it belongs at the top of the write-up.

4.

```
near = orders[orders["centred"].abs() <= 0.5]
print(f"orders within ₹500 of the cut-off: {len(near):,}")
print(f"share with free delivery: {near['free_delivery'].mean():.3f}")
rate_below = near.loc[near["free_delivery"] == 0, "repeat_within_90_days"].mean()
rate_above = near.loc[near["free_delivery"] == 1, "repeat_within_90_days"].mean()
print(f"reorder rate below: {rate_below:.3f}")
print(f"reorder rate above: {rate_above:.3f}")
```

orders within ₹500 of the cut-off: 5,686
share with free delivery: 0.502
reorder rate below: 0.328
reorder rate above: 0.412

Almost exactly half, which is what a smooth running variable around a threshold should give. It's also the cleanest comparison in the chapter: two groups of orders differing by a few hundred rupees and a delivery charge.

5.

```
for comparison in (["West", "South"], ["East"], ["West"], ["South"]):
    subset = panel[panel["region"].isin(["North"] + comparison)]
    fit = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=subset).fit()
    effect = fit.params["treated:is_after"]
    low, high = fit.conf_int().loc["treated:is_after"]
    print(f"comparison {str(comparison):22s}: {100 * (np.exp(effect) - 1):+5.1f}% "
          f"(CI {100 * (np.exp(low) - 1):+5.1f}% to {100 * (np.exp(high) - 1):+5.1f}%)")
```

```

comparison ['West', 'South']      : -7.5% (CI -10.9% to -3.8%)
comparison ['East']               : -6.2% (CI -9.8% to -2.4%)
comparison ['West']               : -9.1% (CI -13.1% to -4.9%)
comparison ['South']              : -5.8% (CI -10.0% to -1.4%)

```

The estimate moves by a couple of points depending on the comparison group, and every version is clearly negative. That spread is honest information: report the main specification, and say how much it moves when the comparison changes. An estimate that flips sign when you swap comparison regions is not a finding.

6.

```

placebo_data = panel[panel["month"] < "2025-10-01"].copy()
placebo_data["fake_after"] = (placebo_data["month"] >= "2025-01-01").astype(int)
placebo = smf.ols("log_orders ~ treated:fake_after + C(region) + C(month)",
                  data=placebo_data).fit()
effect = placebo.params["treated:fake_after"]
low, high = placebo.conf_int().loc["treated:fake_after"]
print(f"placebo effect: {100 * (np.exp(effect) - 1):+.1f}% "
      f"(95% CI {100 * (np.exp(low) - 1):+.1f}% to {100 * (np.exp(high) - 1):+.1f}%)")
print("a clean design finds nothing here")

```

```

placebo effect: +0.5% (95% CI -3.9% to +5.1%)
a clean design finds nothing here

```

The placebo effect is small and its interval comfortably includes zero, so the design isn't manufacturing effects out of noise. Run this before you trust any DiD: it costs three lines and catches designs that would "find" something whatever the data.

7.

```

clustered = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit(
    cov_type="cluster", cov_kws={"groups": panel["region"]})
plain = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit()
for name, fit in (("plain", plain), ("clustered", clustered)):
    low, high = fit.conf_int().loc["treated:is_after"]
    print(f"{name:10s}: effect {fit.params['treated:is_after']:+.4f}, "
          f"CI {low:+.4f} to {high:+.4f}, se {fit.bse['treated:is_after']:.4f}")

```

```

plain      : effect -0.0729, CI -0.1064 to -0.0394, se 0.0168
clustered  : effect -0.0729, CI -0.0978 to -0.0480, se 0.0127

```

`fit.bse` holds the standard errors. Notice that the clustered interval is *narrower*, the opposite of what clustering is supposed to do. With four clusters the formula is unreliable in both directions, and a narrower "corrected" interval is itself the warning sign. Quote the plain interval, and add the caveat: with four regions, treat any interval as indicative. The fixes are resampling methods built for few groups, such as the placebo test of exercise 6 run for every region in turn, and a write-up should say which one you used.

8.

```

long = pd.concat([
    qbr.assign(year=2024, log_revenue=qbr["log_2024"]),
    qbr.assign(year=2025, log_revenue=qbr["log_2025"])]))
long["after"] = (long["year"] == 2025).astype(int)
did_qbr = smf.ols("log_revenue ~ in_qbr_program:after + C(customer_id) + C(year)",
                  data=long).fit()
effect = did_qbr.params["in_qbr_program:after"]
print(f"DiD estimate: {100 * (np.exp(effect) - 1):+.1f}%")
print(f"matching:      {100 * (np.exp(matched_effect) - 1):+.1f}%")
print("true effect:  +7.0%")

```

```

DiD estimate: +9.0%
matching:      +7.1%
true effect:  +7.0%

```

`pd.concat` (section 18.7) stacks two copies of the accounts, one labelled 2024 and one 2025, so each account has a before row and an after row. The DiD uses each customer as their own control, which removes every fixed difference between accounts, measured or not. It lands two points high. Participants were chosen partly because they were already growing faster (2024 growth 6.2% against 5.2%), so they would have out-grown the others anyway: a parallel-trends failure. Prefer DiD when selection was on *levels* (size); when it was on *trends* (growth), combine the two: match on 2024 growth, then take the difference.

9.

```

def match(caliper):
    available = qbr[qbr["in_qbr_program"] == 0].set_index("customer_id")["propensity"].to_dict()
    ids = []
    ordered = qbr[qbr["in_qbr_program"] == 1].sort_values("propensity", ascending=False)
    for row in ordered.itertuples():
        if not available:
            break
        best = min(available, key=lambda cid: abs(available[cid] - row.propensity))
        if caliper is None or abs(available[best] - row.propensity) <= caliper:
            ids += [row.customer_id, best]
            del available[best]
    subset = qbr[qbr["customer_id"].isin(ids)]
    means = subset.groupby("in_qbr_program")["log_2025"].mean()
    return len(ids) // 2, smd(subset, "log_2024"), 100 * (np.exp(means[1] - means[0]) - 1)

for caliper in (0.01, 0.05, None):
    pairs_n, balance, effect = match(caliper)
    print(f"caliper {str(caliper):>5}: {pairs_n:2d} pairs, "
          f"balance on log_2024 {balance:+.3f}, effect {effect:+.1f}%")
print("true effect: +7.0%")

```

```

caliper 0.01: 56 pairs, balance on log_2024 -0.082, effect +0.4%
caliper 0.05: 64 pairs, balance on log_2024 +0.002, effect +7.1%
caliper None: 68 pairs, balance on log_2024 +0.071, effect +12.2%
true effect: +7.0%

```

Look at how much the answer moves: +0.4%, +7.1%, +12.2%, for the same data and the same method. With no caliper, every participant gets matched, including the largest accounts whose nearest "comparison" isn't close, and balance drifts. With a caliper of 0.01, only 56 pairs survive and the ones that do are a peculiar subset. The middle setting balances well and lands near the truth, but you couldn't have known that in advance without the answer key. This is the honest reason matching ranks below the other methods in section 31.9: several defensible choices give materially different answers, so report the choice, the balance, and how much the estimate moves when you change it.

10.

```
window = orders[orders["centred"].abs() <= 5]
linear = smf.ols("repeat_within_90_days ~ free_delivery + centred + free_delivery:centred",
               data=window).fit()
quad = smf.ols("repeat_within_90_days ~ free_delivery + centred + I(centred ** 2) + "
              "free_delivery:centred + free_delivery:I(centred ** 2)", data=window).fit()
for name, fit in (("linear", linear), ("quadratic", quad)):
    low, high = fit.conf_int().loc["free_delivery"]
    print(f"{name:10s}: {100 * fit.params['free_delivery']:+.2f} points "
          f"(CI {100 * low:+.2f} to {100 * high:+.2f})")
print("true effect: +6.0 points")
```

```
linear      : +7.26 points (CI +5.61 to +8.90)
quadratic   : +8.31 points (CI +5.87 to +10.74)
true effect: +6.0 points
```

In a formula, `I(...)` means "compute this inside the formula": `I(centred ** 2)` adds a column of centred squared, which lets each side curve. (Without `I()`, `**` has a different meaning in formulas.) The quadratic sits about a point higher with a wider interval, and the two overlap comfortably, so the finding doesn't depend on the shape you fit. Report the linear fit at a narrow bandwidth as the main result and the others as robustness: high-order polynomials are known to produce strange jumps at the boundary, and "we tried several and they agree" is a stronger sentence than any single specification.

11.

```
def synthetic_gap(target):
    donors = [r for r in ["West", "South", "East", "North"] if r != target]
    pre = wide_panel[wide_panel.index < "2025-10-01"]
    def loss(w):
        return float(np.mean((pre[target].to_numpy() - pre[donors].to_numpy() @ w) ** 2))
    fit = minimize(loss, np.repeat(1 / 3, 3), bounds=[(0, 1)] * 3,
                  constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}], method="SLSQP")
    blend = wide_panel[donors].to_numpy() @ fit.x
    after = wide_panel.index >= "2025-10-01"
    pre_fit = np.sqrt(np.mean((pre[target].to_numpy() - blend[:len(pre)]) ** 2))
    return pre_fit, (wide_panel[target].to_numpy() - blend[after]).mean()

for region in ["North", "West", "South", "East"]:
    pre_fit, gap_r = synthetic_gap(region)
    flag = " <-- the treated region" if region == "North" else ""
    print(f"{region:6s}: pre-change fit {pre_fit:.4f}, "
          f"post-change gap {100 * (np.exp(gap_r) - 1):+5.1f}%{flag}")
```

```

North : pre-change fit 0.0429, post-change gap -6.6% <-- the treated region
West  : pre-change fit 0.2918, post-change gap +38.2%
South : pre-change fit 0.0415, post-change gap -0.0%
East  : pre-change fit 0.3074, post-change gap -21.4%

```

The function is section 31.5 in one piece, with the @ form of the blend and `np.repeat(1 / 3, 3)` for the three starting weights. `blend[:len(pre)]` takes the pre-change months, and `after` marks the months from October 2025.

Read the fit column first. West and East can't be reproduced by a blend of the others at all (pre-change errors of 0.29 and 0.31 against North's 0.043): West is the biggest region and East the smallest, so no blend of the rest can reach them, and their gaps are noise that tell you nothing. South fits as well as North does, and its post-change gap is zero. So only South is a usable placebo. With North and South the only well-fitted units, the most extreme result North could have is "first of two", so the smallest p-value this test can give is $1/2 = 0.5$. Four regions cannot deliver significance by this route, which is another way of saying four regions is a very small dataset.

12.

```

window = orders[orders["centred"].abs() <= 2].copy()
for column in ["segment", "region"]:
    share = pd.crosstab(window["free_delivery"], window[column], normalize="index")
    print(share.round(3).to_string())
    print()

```

segment	Hospitality	Retail	Wholesale
free_delivery			
0	0.338	0.326	0.336
1	0.332	0.336	0.332

region	East	North	South	West
free_delivery				
0	0.254	0.248	0.248	0.250
1	0.254	0.256	0.245	0.245

The mix of segments and regions is nearly identical either side of the cut-off. If it weren't, the two sides would differ in ways beyond the rule, and the jump could be that difference rather than the free delivery. Covariate smoothness is to RD what balance is to matching, and what the sample-ratio check is to an A/B test: the test of whether the comparison is fair.

13.

```

panel["t"] = ((panel["month"].dt.year - 2024) * 12 + panel["month"].dt.month)
trend = smf.ols("log_orders ~ treated:is_after + C(region) + C(month) + treated:t",
               data=panel).fit()
effect = trend.params["treated:is_after"]
low, high = trend.conf_int().loc["treated:is_after"]
print(f"with a North-specific trend: {100 * (np.exp(effect) - 1):+.1f}% "
      f"(CI {100 * (np.exp(low) - 1):+.1f}% to {100 * (np.exp(high) - 1):+.1f}%)")
print("without it: -7.0%")

```

```

with a North-specific trend: -4.8% (CI -10.5% to +1.3%)
without it: -7.0%

```


t numbers the months; $t_{\text{treated}}:t$ is 0 for the other regions and the month number for North, so its coefficient is a straight-line trend in North *relative* to the others, which the month effects already handle. The effect shrinks to about -5% and its interval now crosses zero. Don't read that as "the result failed": with 24 months and a change two-thirds of the way through, a trend fitted on the whole period soaks up much of the post-change drop, which is exactly the effect being measured, and section 31.4's dip in the last pre-change quarter pulls the trend down. It is still a useful warning. The honest summary is: -7% in the main specification, -5% and not distinguishable from zero when North is allowed its own trend, and the two are not far apart in business terms. Report both, and let the reader see the range.

14. One sentence each, ranked by how much they'd survive an argument:

- **Regression discontinuity (free delivery, +7 points):** *"This would change if customers could steer their order values across ₹25,000, which the density check says they don't."* Strongest, because the comparison group is almost mechanical, and weakest in scope: it speaks only for orders near the threshold.
- **Difference-in-differences (price rise, -7%):** *"This would change if something else hit North around October 2025, or if North was already drifting away from the other regions."* Strong on the evidence available, with the caveats that four regions is few and the last pre-change quarter dipped.
- **Matching (quarterly reviews, +7%):** *"This would change if the sales team chose accounts using something we can't see, such as their sense of which accounts had potential."* Weakest, because that judgment is exactly the sort of thing that isn't in a table.

15. The expansion helps West-region customers from March 2026, which is *after* the price rise, so the comparison group's behavior changes partway through the post period. That inflates the other regions' growth and makes North look worse than it is. Three responses, in order of preference: end the analysis window in February 2026; drop West and use South and East as the comparison (exercise 5 shows the estimate holds); or add a control for the expansion period and report both. Whatever you choose, say it, because a reader who knows about the warehouse will ask.

16. Revenue in 2025 is measured *after* the program ran, so it's partly an effect of the thing being measured. Matching on it would pair participants with the non-participants who happened to reach the same revenue, throwing away the very difference the program caused, and biasing the estimate toward zero. The rule is absolute: match only on what was known before treatment began, which here means 2024 revenue, 2024 growth, tenure, and segment.

17. Any rule with a sharp edge: free shipping above a value, a discount tier at a volume, a credit check above an order size, a bonus for hitting a monthly target, an SLA that applies above a ticket priority. Bunching is likely wherever the person affected can see the threshold and control the number: sales reps write orders at exactly the discount tier, expense claims come in just under the approval limit, and salespeople hit their target by a rupee in the last week of the month. If the histogram spikes at the edge, the rule is being gamed, and that spike is itself worth reporting to the business.

18. Almost always, when the decision is soon and reversible. A DiD that takes an afternoon and gives ± 3 points is worth more than a perfect experiment that reports after the decision is made. The reverse holds when the decision is expensive, hard to undo, or repeated: a pricing policy for every

region, a system everyone will use for five years. The practical answer is usually both: measure what you can now with the weaker method, and design the experiment for the next change, which is exactly what Riverstone did after the price rise.

NEXT

Where this leads

- **Chapter 30, Inference & Experiments**, is the method to reach for whenever randomizing is possible; this chapter is what you do when it isn't.
- **Chapter 22, Statistics Without Fooling Yourself**, is the intuition behind the warnings here, and its section 22.10 is the regression every method in this chapter was built on.
- **Chapter 35, The Math Under the Models**, explains the @ of section 31.5: vectors, matrices, and the dot product.
- **Chapter 37, Supervised Learning Algorithms**, fits the same regressions, linear and logistic, to *predict* rather than to explain; **Chapter 40, Time Series & Forecasting**, fits lines through time. The machinery is shared: the difference is entirely in what you claim from it.
- **Chapter 56, MLOps**, keeps models honest in production, where telling "the model is good" from "the model was given the easy cases" needs this chapter's causal thinking.
- **Chapter 73, Statistics, Probability & Experimentation Bank**, has causal-inference questions in section 73.6: confounders, selection bias, and natural experiments.